

DATA STRUCTURES · ALGORITHMS · SYSTEM DESIGN

क्रम

KRAMA

The Ordered Path

180 Days from First Principles to the Product-Company Interview

NISHANT GUPTA - GRANTH SKILLS

VOLUME I OF EIGHT

Foundations

DAYS 001 – 026

DAY 001

THE ORDERED PATH

DAY 180



TWO TRACKS

26 OF 180 DAYS IN THIS VOLUME

180 STEPS

K R A M A

THE ORDERED PATH

क्रम

K R A M A

The Ordered Path

180 Days from First Principles to the Product-Company Interview

Nishant Gupta - Granth Skills

VOLUME I · FOUNDATIONS

DAYS 1 TO 26

KRAMA — THE ORDERED PATH

180 Days from First Principles to the Product-Company Interview

Volume I of eight: **Foundations**. Days 1 to 26.

Copyright © 2026 Nishant Gupta - Granth Skills. All rights reserved.

ABOUT THE TEXT

This edition is generated from the Krama course sources. The lessons, stories, code, diagrams and interview material are original work written for this course.

Practice problems are named, with their source and one line on what they test. No problem statement is reproduced. Trade marks and problem titles belong to their respective owners and are used here only to identify where a problem can be found.

ABOUT THE MAKING OF IT

Typeset from Markdown by rendering to a browser engine and printing to PDF. Every tool in that chain is free software and every typeface is published under the SIL Open Font Licence. Appendix F lists all of them with their licences.

Text face: Source Serif 4 — SIL Open Font Licence 1.1

Display face: Fraunces — SIL Open Font Licence 1.1

Sans face: Inter — SIL Open Font Licence 1.1

Monospace face: JetBrains Mono — SIL Open Font Licence 1.1

Devanagari: Noto Serif Devanagari — SIL Open Font Licence 1.1

Diagrams: Mermaid — MIT Licence

Code colouring: Pygments — BSD 2-Clause

EDITION

First edition, 2026. Set in Source Serif 4 on a 6.75 by 9.5 inch page.

क्रम

krama, Sanskrit: order; sequence; the step that follows the step before it.

YOU CANNOT SKIP A STEP. THAT IS THE WHOLE METHOD.

Why this book teaches two things at once

This book teaches two subjects at the same time, on purpose.

Most preparation separates them. You spend three months on data structures and algorithms, then start again from nothing on system design, and by the time the second subject makes sense the first has gone quiet. Interviews do not work that way. A single loop at a product company will ask you to reverse a linked list in one room and design a rate limiter in the next, on the same day, from the same head.

So every chapter here teaches one topic from each track, side by side. Day 42 gives you binary search and the object-oriented design of a parking lot. Day 130 gives you graph colouring and message queues. You will not finish this book with one strong subject and one weak one.

Who this is written for

Someone who has never studied this before. Not a computer science graduate. Not someone brushing up after a few years in the job.

If you need to be told what a server is, you are the reader. The first chapter explains what happens when a program runs. The fourteenth explains what happens when you type an address into a browser. Nothing before that is assumed.

The other half of that promise matters just as much: you are someone who will sit in a real interview. So the writing never stops at understanding. Every chapter ends by putting the idea in an interviewer's mouth, giving you the first ninety seconds of an answer, and writing out a full model response you can compare yours against.

The one question every chapter answers

Every lesson in this book was written to satisfy a single test:

Does this leave the reader able to answer one real interview question out loud, from memory, to a stranger?

That test decided what went in and, more usefully, what stayed out. There are no formal proofs here. No potential functions, no decision-tree lower bounds, no interpreter internals for their own sake. If a thing will not be asked, and does not make an asked thing clearer, it is not in this book.

What is different about how it is written

Every lesson opens with a story about a person. Not an analogy dressed up as a story: an actual scene, with a name in it, told in ordinary words with no technical vocabulary at all. Someone sorting socks. Someone in a canteen queue. Someone laying tables for a wedding. The story comes before the definition because a beginner needs somewhere to put the idea before they have the word for it.

Every solution is shown in full. This book does not hide answers. Code is built up in fragments of a few lines, each one explained, and then given again complete and ready to run. You are never told to work it out yourself as a test of character.

The arithmetic is always done out loud. Not “this is slow” but “this line runs about twenty-five million times”. Not “that is a lot of data” but `50M x 20 x 2KB = 2TB`. Numbers are worked through where you can see them.

Errors are printed exactly as they appear. When something breaks, the real message is on the page - `IndexError: list index out of range` - not a description of it. You will recognise it when your own screen shows it.

How this edition was made

The text is generated from the course repository, so the book and the working material cannot drift apart. The typesetting was done by rendering the pages in a browser engine and printing them to PDF.

Every part of the toolchain is free software, and every typeface is published under the SIL Open Font Licence. Appendix F lists all of it, with licences. No proprietary tool, font, or copyrighted third-party text was used to produce this book.

That extends to the practice problems. Problems are named, with their source and one line on what they are really testing. Their statements are never reproduced. You are meant to read them where they live, and solve them there.

How to use this book

The book is one hundred and eighty chapters long. One chapter is one day's work. That is the only pacing instruction in it, and it is deliberate: a day here is a unit of subject, not a unit of hours. Some days will take you longer than others. That is information about the topic, not about you.

The shape of every chapter

Each chapter holds three pieces, in this order.

1. **The DSA lesson.** Nine sections, always the same nine, always in the same order. Appendix A sets out what each one is for.
2. **The system design lesson.** The same nine sections, on a different subject.
3. **The practice sheet.** Named problems to solve, and questions to answer out loud.

The repetition is the point. After a week you stop navigating and start reading. You know that section 5 has the complete code, that section 7 has the mistakes you are about to make, and that section 8 is the interview itself.

How to read a lesson

Read the story first, and do not skip it. It is section 2, it has no technical words in it, and it is the part that makes the idea stick. People who skip it can define the term and cannot use it.

Type the code. Do not read it. Section 5 builds the solution in fragments and then gives it complete. Type the fragments as they come. Run the finished program. Change a number and run it again.

Say section 9 out loud. Every lesson ends with a recall card - the two or three sentences that are the whole lesson. Say them from memory, aloud, in full sentences. Not in your head. Aloud. An answer you have only ever thought is an answer you have never tested.

Treat section 8 as a rehearsal, not a reading. It gives you real interviewer phrasings, a script for the opening ninety seconds, the three follow-ups that usually come next, and a written-out model answer. Deliver yours before you read theirs.

How to use the practice sheet

Solve the problems where they live. The sheet names each one, tells you where to find it, and says in one line what it is really testing - which is usually not what it appears to be about.

Solve it, then say what you did out loud, as if someone had asked. If you cannot narrate your own solution, you have not finished the problem.

When you get stuck

Go back one chapter, not forward. This course is built so that each day rests on the one before it. A chapter that will not open is almost always a chapter whose foundation is thin, and the foundation is behind you.

If you want the vocabulary, Appendix D is a glossary of every term the book defines, with the day it first appears. If you want to see the whole route, Appendix C prints all one hundred and eighty days on two facing pages.

Revision

Certain chapters are revision days, and they are marked as such in the contents. Do not read past them. They exist because recall fades on a schedule, and the schedule is not negotiable.

Appendix B collects the recall cards for every part in this volume. Cover the answer, say it, then check. That is the whole drill.

Where this volume sits

This volume is marked in colour below.

- **Volume I — Foundations**
Days 1 to 26. How Code Costs, and How the Internet Works
- **Volume II — Scanning and Searching**
Days 27 to 50. Pointers, Windows and Binary Search · Databases from Zero
- **Volume III — Order and Access**
Days 51 to 77. Sorting, Hashing, Stacks and Queues · Objects, SOLID and Patterns
- **Volume IV — Links and Recursion**
Days 78 to 97. Linked Lists, Recursion and Backtracking · Low-Level Design
- **Volume V — Trees and Priorities**
Days 98 to 124. Trees, Heaps and Tries · Scaling Fundamentals
- **Volume VI — Graphs**
Days 125 to 142. Graph Algorithms · Distributed Systems Core
- **Volume VII — Optimal Choices**
Days 143 to 170. Dynamic Programming and Greedy · High-Level Design Case Studies
- **Volume VIII — The Last Mile**
Days 171 to 180. Bits, Maths and Mock Interviews · Reliability and Security

The nineteen parts

Part I	Foundations: how code costs	Days 1–8
Part II	Arrays	Days 9–18
Part III	Strings	Days 19–26
PART IV	Two pointers and sliding window	Days 27–36
PART V	Prefix sums	Days 37–41
PART VI	Binary search	Days 42–50
PART VII	Sorting	Days 51–59
PART VIII	Hashing: maps and sets	Days 60–67

PART IX	Stacks and queues	Days 68–77
PART X	Linked lists	Days 78–86
PART XI	Recursion and backtracking	Days 87–97
PART XII	Trees and binary search trees	Days 98–112
PART XIII	Heaps and priority queues	Days 113–119
PART XIV	Tries	Days 120–124
PART XV	Graphs	Days 125–142
PART XVI	Dynamic programming	Days 143–163
PART XVII	Greedy and intervals	Days 164–170
PART XVIII	Bits and maths	Days 171–176
PART XIX	Final mocks and revision	Days 177–180

What is in this volume

PART I · FOUNDATIONS: HOW CODE COSTS	1
001 How your code actually runs, and where the time goes	2
DSA How your code actually runs, and where the time goes	3
DESIGN What happens when you type google.com and press Enter	14
PRACTICPractice — How your code actually runs, and where the time goes	25
002 Counting steps: your first cost model	27
DSA Counting steps: your first cost model	28
DESIGN Client and server, explained properly	41
PRACTICPractice — Counting steps: your first cost model	52
003 Big-O in plain English	54
DSA Big-O in plain English	55
DESIGN IP addresses, ports, and DNS	70
PRACTICPractice — Big-O in plain English	83
004 The growth curves you will meet again and again	85
DSA The growth curves you will meet again and again	86
DESIGN TCP and UDP	102
PRACTICPractice — The growth curves you will meet again and again	115
005 Python for DSA I: lists, tuples, and slicing	117
DSA Python for DSA I: lists, tuples, and slicing	118
DESIGN HTTP: the request and the response	135
PRACTICPractice — Python for DSA I: lists, tuples, and slicing	149
006 Python for DSA II: strings, dictionaries, and sets	152
DSA Python for DSA II: strings, dictionaries, and sets	153
DESIGN HTTPS and TLS, without the maths	170
PRACTICPractice — Python for DSA II: strings, dictionaries, and sets	184
007 Space complexity, and what in-place really means	187
DSA Space complexity, and what in-place really means	188
DESIGN What a web server actually does	204
PRACTICPractice — Space complexity, and what in-place really means	220
008 Reading a problem like the interviewer wrote it	223
DSA Reading a problem like the interviewer wrote it	224
DESIGN Processes, threads, and concurrency	241
PRACTICPractice — Reading a problem like the interviewer wrote it	255
PART II · ARRAYS	258
009 What an array really is in memory	259

DSA	What an array really is in memory	260
DESIGN	CPU, RAM, and disk: the speed hierarchy	276
PRACTICE	Practice — What an array really is in memory	289
010	Traversal: the loop patterns you will reuse forever	292
DSA	Traversal: the loop patterns you will reuse forever	293
DESIGN	Latency numbers every engineer should know	310
PRACTICE	Practice — Traversal: the loop patterns you will reuse forever	324
011	Insert, delete, and the cost of the middle	327
DSA	Insert, delete, and the cost of the middle	328
DESIGN	The operating system's job	345
PRACTICE	Practice — Insert, delete, and the cost of the middle	361
012	Searching an array: linear search, done properly	364
DSA	Searching an array: linear search, done properly	365
DESIGN	How your code becomes a running service	382
PRACTICE	Practice — Searching an array: linear search, done properly	398
013	Reversing, rotating, and swapping in place	401
DSA	Reversing, rotating, and swapping in place	402
DESIGN	Containers and why everyone uses Docker	424
PRACTICE	Practice — Reversing, rotating, and swapping in place	438
014	Max, min, second largest: the single-pass habit	442
DSA	Max, min, second largest: the single-pass habit	443
DESIGN	Fundamentals revision and interview questions	462
PRACTICE	Practice — Max, min, second largest: the single-pass habit	479
015	Moving elements: zeros, duplicates, and the write pointer	484
DSA	Moving elements: zeros, duplicates, and the write pointer	485
DESIGN	What an API is	500
PRACTICE	Practice — Moving elements: zeros, duplicates, and the write pointer	513
016	2D arrays and matrix traversal	517
DSA	2D arrays and matrix traversal	518
DESIGN	REST, properly	535
PRACTICE	Practice — 2D arrays and matrix traversal	549
017	Matrix tricks: rotate, spiral, transpose	553
DSA	Matrix tricks: rotate, spiral, transpose	554
DESIGN	Designing a good REST endpoint	572
PRACTICE	Practice — Matrix tricks: rotate, spiral, transpose	586
018	Arrays revision and mock round	591
DSA	Arrays revision and mock round	592
DESIGN	Status codes, errors, and idempotency	608
PRACTICE	Practice — Arrays revision and mock round	623

019	What a string is, and why it is immutable	629
DSA	What a string is, and why it is immutable	630
DESIGN	Authentication and authorisation	645
PRACTICE	Practice — What a string is, and why it is immutable	659
020	Building strings without the quadratic trap	664
DSA	Building strings without the quadratic trap	665
DESIGN	JWT, sessions, and OAuth	682
PRACTICE	Practice — Building strings without the quadratic trap	697
021	Character counting and frequency maps	702
DSA	Character counting and frequency maps	703
DESIGN	GraphQL versus REST	719
PRACTICE	Practice — Character counting and frequency maps	733
022	Anagrams: the sorting versus counting choice	738
DSA	Anagrams: the sorting versus counting choice	739
DESIGN	gRPC and when binary protocols win	755
PRACTICE	Practice — Anagrams: the sorting versus counting choice	769
023	Palindromes and the two-ends habit	774
DSA	Palindromes and the two-ends habit	775
DESIGN	Rate limiting and API gateways	792
PRACTICE	Practice — Palindromes and the two-ends habit	807
024	Substrings versus subsequences: the distinction they test	812
DSA	Substrings versus subsequences: the distinction they test	813
DESIGN	API revision and interview questions	830
PRACTICE	Practice — Substrings versus subsequences: the distinction they test	845
025	Pattern matching, the simple way	851
DSA	Pattern matching, the simple way	852
DESIGN	What a database gives you that a file does not	867
PRACTICE	Practice — Pattern matching, the simple way	882
026	Strings revision and mock round	887
DSA	Strings revision and mock round	888
DESIGN	Tables, rows, and keys	903
PRACTICE	Practice — Strings revision and mock round	920

BACK MATTER

A	Appendix A - The nine sections, and what each is for	926
B	Appendix B - Recall cards	928
C	Appendix C - The complete 180-day map	946
D	Appendix D - Glossary	949
E	Appendix E - References and further reading, by section	968

F	Appendix F - Colophon: how this book was made	971
G	Appendix G - Problem index	972
H	Index	981

List of figures

1.1	System design - What happens when you type google.com and press Enter	2
2.1	System design - Client and server, explained properly	27
3.1	System design - IP addresses, ports, and DNS	54
4.1	System design - TCP and UDP	85
4.2	System design - TCP and UDP	85
5.1	System design - HTTP: the request and the response	117
6.1	System design - HTTPS and TLS, without the maths	152
7.1	System design - What a web server actually does	187
8.1	Data structures and algorithms - Reading a problem like the interviewer wrote it	223
8.2	System design - Processes, threads, and concurrency	223
10.1	System design - Latency numbers every engineer should know	292
11.1	System design - The operating system's job	327
12.1	System design - How your code becomes a running service	364
13.1	System design - Containers and why everyone uses Docker	401
14.1	System design - Fundamentals revision and interview questions	442
15.1	System design - What an API is	484
15.2	System design - What an API is	484
16.1	System design - REST, properly	517
18.1	Data structures and algorithms - Arrays revision and mock round	591
18.2	System design - Status codes, errors, and idempotency	591
19.1	System design - Authentication and authorisation	629
20.1	System design - JWT, sessions, and OAuth	664
21.1	System design - GraphQL versus REST	702
22.1	System design - gRPC and when binary protocols win	738
23.1	System design - Rate limiting and API gateways	774
24.1	System design - API revision and interview questions	812
25.1	System design - What a database gives you that a file does not	851
26.1	System design - Tables, rows, and keys	887

PART I

Foundations: how code costs

Days 1 to 8 · 8 chapters

Alongside, on the system design track:

How computers and the internet work

001 How your code actually runs, and where the time goes

002 Counting steps: your first cost model

003 Big-O in plain English

004 The growth curves you will meet again and again

005 Python for DSA I: lists, tuples, and slicing

006 Python for DSA II: strings, dictionaries, and sets

007 Space complexity, and what in-place really means

008 Reading a problem like the interviewer wrote it

How your code actually runs, and where the time goes

DATA STRUCTURES AND ALGORITHMS

How your code actually runs, and where the time goes

You can read a function and point at the one line that does most of the work.

SYSTEM DESIGN

What happens when you type google.com and press Enter

You can narrate the whole journey from keypress to pixels without notes.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Walk me through this function line by line. Which line runs the most times?*
- *What happens when you type google.com into your browser and hit Enter?*

How your code actually runs, and where the time goes

1 What this is, and why they ask it

Your program is a list of small instructions. The computer performs them one at a time, in order, very fast. The time your program takes is simply **how many instructions it performs**, multiplied by how long each one takes.

You control only the first of those two numbers. That is the whole subject.

Interviewers open with this because it is the cheapest way to find out whether you can read code or only write it. Someone who can look at eight lines and say, “this line runs about twenty-five million times, and that is the problem”, has a skill that survives every language and every job. Someone who says, “it looks fine, let me run it”, does not. Almost every interview at Amazon, Google, Microsoft, Flipkart or Zomato contains this question in some form, usually disguised as, “so what is the bottleneck here?”

2 The story

Anil gets home on Sunday and tips the laundry basket out on his bed. It is a week’s washing for the four people in the flat, and most of it is socks. He counts the heap roughly. About sixty socks.

There are two jobs to do, and he does them in order.

The first job is turning them the right way out. Almost every sock comes out of the machine inside out. So he picks one up, pulls it through, and drops it on the left side of the bed. Then the next one. Sixty socks, sixty pulls. He puts a song on, and the job takes four minutes.

The second job is finding the pairs, and this is the one that ruins his evening.

He picks up a plain black sock. Is its partner in the heap? He cannot tell by looking at the heap, so he starts holding the black sock against the others, one at a time. Grey, no. Black but shorter, no. Blue, no. Twenty socks later, he finds the match. He drops the pair on the right side of the bed and picks up the next single sock.

An hour later he is barely halfway, and he stops to work out why. For each sock he has to try it against the rest of the heap, and the heap is fifty-odd socks deep. He has sixty socks to place. Sixty socks, each held up against roughly sixty others, is about three thou-

sand hold-ups. Even after he stops re-checking the pairs he has already matched, it is close to two thousand. Each hold-up takes him a second. Two thousand seconds is more than half an hour of nothing but lifting socks towards the light.

His flatmate suggests turning the big light on, so that telling grey from black is quicker. Anil tries it, and it helps a little. But even if every hold-up became twice as fast, half an hour would only become fifteen minutes, and he would still be sitting on that bed at ten o'clock.

Nothing is wrong with his hands, and nothing is wrong with the light. The two jobs felt the same — pick up a sock, look at it — and they are not the same at all. One of them is sixty looks. The other is two thousand.

3 The idea in plain English

Anil's two jobs are the two shapes of almost every piece of code you will ever write.

Let us line the story up against the code, one piece at a time.

A **program** is a sequence of **statements**. A statement is a line of code that tells the computer to do one small thing. Turning a sock the right way out is a statement. Holding one sock against another is a statement.

Running a statement means the computer actually performs it. This is the number that matters, and it is not the same as the number of lines you typed. Anil had one instruction in his head — “hold this sock against that sock” — and he performed it two thousand times.

A **loop** is a statement that says, “do the following again and again”. Anil working through the heap, sock after sock, is a loop. In Python it looks like this:

```
for sock in heap:
    turn_right_way_out(sock)
```

`heap` is a **list**: a row of values kept in order, like the socks piled on the bed. The loop takes each value in turn and gives it the name `sock`. If there are 60 socks, `turn_right_way_out` runs 60 times. That is the first job.

A **nested loop** is a loop inside another loop. This is the second job, and it is where the trouble lives:

```
for sock_a in heap:
    for sock_b in heap:
        is_pair(sock_a, sock_b)
```


Read that slowly, because it is the most important thing on this page. The outer loop runs 60 times. **For each one of those 60 runs**, the inner loop runs its own 60 times. So `is_pair` runs $60 \times 60 = 3,600$ times. You wrote three lines. The computer performed three thousand six hundred comparisons.

The line sitting deepest inside the loops — `is_pair(sock_a, sock_b)` — is what this course calls the **hot line**. It is the line that runs the most times, so it is the line that decides how long the whole program takes. When an interviewer asks where the time goes, they are asking you to find the hot line.

Now the part with the big light, which is the part people underestimate.

Making each individual step faster is called a **constant-factor** improvement: a faster machine, a faster language, a cleverer comparison. It divides your total by two, or by ten, and then it has finished helping you. Doing fewer steps changes the shape of the growth itself. Anil going from 2,000 hold-ups down to 60 looks is not “twice as good”. It is more than thirty times as good, and it gets better still as the pile of laundry grows.

Here is the sentence to keep. Double the socks, and the first job takes twice as long. Double the socks, and the second job takes **four** times as long, because there is a “sixty” on both sides of the multiplication and both of them doubled.

One honest complication, before we look at code. It is not quite true that every line costs the same. Some lines hide a whole loop inside them. `x in my_list` looks like one statement, and is really the computer walking the list from the start, looking for `x`. We come back to this in §7, because it is the trap that catches almost everybody.

And if you are wondering what Anil should have done: sort the socks into colour piles first, then match inside each small pile. That instinct is correct, it is the whole idea behind [day 062](#), and you are not expected to write it yet.

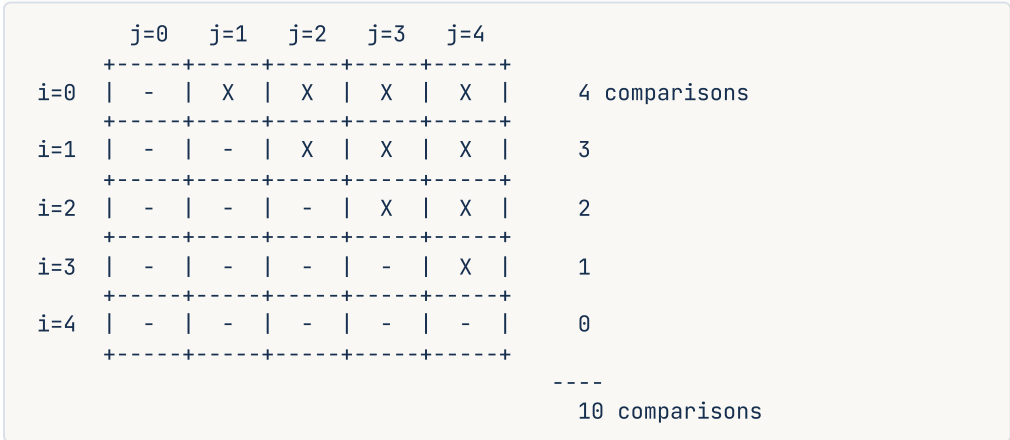
4 The picture

Here is the second job written as code, with what actually happens beside each line. The heap has been shrunk to five socks, so that you can count every step yourself.

	times this line runs (with 5 items)
<code>def has_duplicate(items):</code>	1
<code>for i in range(len(items)):</code>	5 outer loop
<code>for j in range(i + 1, len(items)):</code>	5 restarts once per outer run
<code>if items[i] == items[j]:</code>	10 ← the hot line
<code>return True</code>	0 or 1
<code>return False</code>	1

What to notice: the `if` line is indented twice. Every extra level of indentation inside a loop multiplies how often that line runs. Finding the hot line is usually just a matter of finding the deepest indentation inside the loops.

And here is *why* the count is 10 and not 25. The inner loop starts at `i + 1`, so each sock is only ever compared with the socks after it:



What to notice: only the upper half of the grid is filled in. Skipping the lower half halves the work, which is a real saving — and it is still nowhere near as important as the fact that the shape is a *grid* at all. Double the number of socks, and the triangle does not get twice as big. It gets roughly four times as big.

5 The code, built step by step

We are going to build a small program that **counts its own work**, so that you can stop guessing and see the numbers.

Start with the first job: one loop, one pass through the heap.

```
def sum_all(items: List[int]) -> int:
    total = 0
    for x in items:
        total = total + x
    return total
```

The line `total = total + x` sits inside one loop, so it runs once per item. Five items, five runs. Five thousand items, five thousand runs. The relationship is a straight line: double the items, and you double the work.

Now the second job: the nested loop.

```
def has_duplicate(items: list[int]) → bool:
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j]:
                return True
    return False
```

`range(len(items))` gives the positions 0, 1, 2, and so on, up to one less than the length. `items[i]` means “the value at position `i`”. The inner loop starts at `i + 1`, so you never compare a sock with itself and never repeat a pair. This is the upper triangle from the diagram above.

So far you have been told the counts. Now measure them. Add a counter that ticks once every time the hot line runs.

```
def count_comparisons(items: list[int]) → int:
    comparisons = 0
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            comparisons += 1
            _ = items[i] == items[j]
    return comparisons
```

That is the whole trick, and it is worth doing yourself at least once. You are no longer reasoning about the code. You are watching it report on itself.

Here is the complete program. It runs both shapes at four different sizes and prints what it finds. Copy it, run it, and read the two count columns before you read the paragraph underneath.

```

"""Day 1 – where the time goes. Run this and read the count columns."""
import time

def sum_all(items: list[int]) → int:
    """One loop: touches every item once."""
    total = 0
    for x in items:
        total = total + x
    return total

def count_single(items: list[int]) → int:
    """How many times does the hot line of sum_all run?"""
    steps = 0
    for _x in items:
        steps += 1
    return steps

def count_nested(items: list[int]) → int:
    """How many times does the hot line of has_duplicate run?"""
    steps = 0
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            steps += 1
            _ = items[i] == items[j]
    return steps

def timed(fn, items: list[int]) → tuple[int, float]:
    """Run fn(items). Return what it returned, and how long it took in milliseconds."""
    start = time.perf_counter()
    result = fn(items)
    return result, (time.perf_counter() - start) * 1000

if __name__ == "__main__":
    print(f"{'items':>7} {'one loop':>12} {'ms':>8} {'nested loop':>14} {'ms':>10}")
    for n in (10, 100, 1000, 5000):
        data = list(range(n))
        single, t_single = timed(count_single, data)
        nested, t_nested = timed(count_nested, data)
        print(f"{n:>7} {single:>12} {t_single:>8.2f} {nested:>14} {t_nested:>10.2f}")

    print("\nsum of 1..100 =", sum_all(list(range(1, 101))))

```

This is exactly what it printed on the machine this lesson was written on:

items	one loop	ms	nested loop	ms
10	10	0.00	45	0.01
100	100	0.01	4,950	0.63
1000	1,000	0.06	499,500	93.84
5000	5,000	2.88	12,497,500	1785.13

sum of 1..100 = 5050

Look at the two count columns, not at the milliseconds. Going from 1,000 items to 5,000 items is five times as many. The one-loop count went from 1,000 to 5,000, which is exactly five times. The nested count went from 499,500 to 12,497,500, which is **twenty-five** times.

Five times the input. Twenty-five times the work. That is the grid from the diagram, showing up in real numbers.

Now run the program a second time. Your millisecond columns will not match the ones above, and they will not even match each other. On a second run on this same machine, with nothing changed, the last nested figure came out at **3063.92 ms** instead of 1785.13 ms. That is almost double, for no reason you can see or control. The **count** columns were identical to the digit, as they will be on every machine, in every language, every time.

That is the first real lesson of this course. Seconds are a fact about one run, on one machine, on one afternoon. Counts are a fact about the code. Only one of those two things is worth writing down, and it is the one nobody thinks to measure.

6 What it costs

Count it out from the code in front of you, rather than trusting anybody, including this document.

The one-loop shape. `for x in items:` runs the body once per item. Nothing is nested inside it. With 5,000 items the hot line runs 5,000 times. Now give it a name: if there are `n` items, the hot line runs `n` times. That is a straight line. Double the input, and you double the work.

The nested shape. The outer loop runs `n` times. For each of those runs, the inner loop runs somewhere between `n - 1` and 0 times, which averages out at about `n / 2`. So the hot line runs roughly `n × n / 2` times. Check that against the measurement: at `n = 5,000` it predicts $5,000 \times 5,000 / 2 = 12,500,000$, and the program actually reported 12,497,500. That is close enough to trust the reasoning.

The exact count is `n × (n - 1) / 2`, which is simply the upper triangle of the grid.

The thing to remember about the nested shape. Because there is an `n` on both sides of the multiplication, doubling `n` multiplies the work by four, not by two. From the numbers above: 10,000 items would be about 50,000,000 comparisons, and 20,000 items about 200,000,000. Two hundred million really is four times fifty million.

Space. Neither function stores anything that grows with the input. `sum_all` keeps one running `total`. `has_duplicate` keeps two positions, `i` and `j`. Whether you hand it ten items or ten million, the extra memory is the same three or four values. This is the cheapest kind of function to run, and we treat it properly on [day 007](#).

And the big light. The nested run at $n = 5,000$ took 1,785 ms. Buy a machine twice as fast and it takes about 890 ms, which is a real improvement — once. Now let the input grow to 10,000, and even on the fast machine you are back to roughly 3,600 ms. The faster machine bought you exactly one doubling. Changing the shape buys you all of them.

Tomorrow, on [day 002](#), you count these steps exactly instead of roughly. On [day 003](#) you get the shorthand that lets you say all of this in three characters.

7 The traps

Trap one: the loop that is hiding

This is the near-miss, and it catches almost everyone. Look at this function and count the loops.

```
def looks_like_one_loop(items: list[int]) → bool:
    seen = []
    for x in items:
        if x in seen:           # one line... or is it?
            return True
        seen.append(x)
    return False
```

There is one `for` on the page, so this must be the one-loop shape. It is not.

`x in seen` is not a single step. To decide whether `x` is in `seen`, the computer walks `seen` from the beginning, comparing as it goes, until it either finds `x` or runs out of list. That is a loop. Python has simply written it for you and hidden it behind two characters.

So the real shape is an outer loop over `items`, and inside it a hidden loop over `seen`, which gets longer on every pass. That is the nested shape wearing a disguise.

Here is the same function, measured, on the input that exposes it — a list with no duplicates at all, so `x in seen` never finds a match and always scans the whole thing:

n=1000	8.12 ms
n=5000	240.40 ms
n=10000	921.72 ms
n=20000	4279.55 ms

Going from 10,000 to 20,000 is one doubling of the input. Going from 921 ms to 4,279 ms is four and a half times the work. Four, not two. It is the grid again.

The rule to carry forward: a line is only one step if you can describe what it does without using the word “every”. “Check every item in `seen`” contains a loop. So does `sorted()`, so does `max()`, so does `sum()`, so does `"".join()`, and so does copying a list with `[:]`. When you are counting work in an interview, read every line and ask: *does this single line touch every item?*

Trap two: the off-by-one that ends the interview

Now the mistake people make while writing the nested loop under pressure. The inner loop must start at `i + 1`, so that each pair is checked once. Here is what happens when the `+ 1` gets attached to the wrong thing:

```
def has_duplicate(items):
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j + 1]:    # should be items[j]
                return True
    return False

print(has_duplicate([4, 9, 2, 9, 7]))
```

It looks right. `j` starts one ahead of `i`, and the `+ 1` reads as though it belongs there. It even runs correctly for the first several comparisons. Then `j` reaches the last position, `j + 1` points one past the end of the list, and this happens:

```
Traceback (most recent call last):
  File "d1a.py", line 8, in <module>
    print(has_duplicate([4, 9, 2, 9, 7]))
    ~~~~~^~~~~~
  File "d1a.py", line 4, in has_duplicate
    if items[i] == items[j + 1]:    # should be items[j]
    ~~~~~^~~~~~
IndexError: list index out of range
```

`IndexError: list index out of range` is Python telling you that you asked for a position the list does not have. A list of five values has positions 0, 1, 2, 3 and 4. There is no position 5. The `~~~~~^~~~~~` marks under the code point at exactly which part of the line went wrong, which is genuinely useful and which most people never notice.

The fix is to decide the offset once, in the loop header, and never adjust it again inside the body. `range(i + 1, len(items))` already guarantees that `j` is ahead of `i`. Adding another `+ 1` inside does the same job twice.

8 In the interview

How it gets asked

- “Walk me through what this function does.” — the warm-up. They are watching whether you read the code or skim it.
- “Which line runs the most times?” — the direct version.
- “Where’s the bottleneck?” or “Why is this slow?” — the same question wearing a work-experience costume. It usually arrives after you have written a working solution and they want to know whether you can see its cost.

What to say out loud, in the first ninety seconds

Do not start guessing at a fix. Do this instead, in this order.

1. **Say what the function returns.** One sentence. *“This returns True if any value appears twice.”* If you cannot say this, you have not read it yet.
2. **Find the loops, and say how deep they nest.** *“There’s an outer loop over the list, and an inner loop over the rest of the list, so two levels.”*
3. **Name the hot line.** Point at it. *“The comparison on line four is the deepest thing inside both loops, so it’s the line that runs most.”*
4. **Count it, roughly, out loud.** *“The outer loop runs n times, the inner one averages about n over two, so the comparison happens around n squared over two times.”*
5. **Put a real number on it.** *“For a list of ten thousand, that’s about fifty million comparisons.”* This is the step that separates candidates. Everybody else stops at step 4.
6. **Then, and only then, offer a direction.** *“If we’re allowed extra memory, there’s a way to do this in one pass. Do you want me to go there?”*

The follow-ups

“Would a faster machine fix it?” No, and here is the arithmetic. A machine twice as fast halves the time once. But the work grows four times every time the input doubles, so a single doubling of the data cancels the new machine out completely. Faster hardware buys you a constant factor. The shape of the growth is the thing you have to change.

“Is `x in my_list` a single step?” No. It walks the list from the start until it finds `x` or reaches the end, so on a list of length n it costs up to n steps. A single `for` loop with `x in seen` inside it is a two-level shape, not a one-level shape. The same is true of `sorted`, `max`, `sum`, `join`, and slicing a list.

“How would you check your count instead of guessing?” Put a counter on the hot line, and print it for a few input sizes. If the input grows five times and the counter grows twenty-five times, it is the nested shape. It takes two minutes, and it turns an argument into a measurement.

A model answer

The interviewer shows you `has_duplicate` and says, “*talk me through this*”. Here is what a strong candidate actually says, more or less word for word:

“So this returns True if the list has any value appearing more than once, and False otherwise.

There are two loops. The outer one walks position `i` through the whole list. The inner one walks `j` from `i + 1` to the end, so it only ever looks at the items after `i`, which means each pair gets compared once instead of twice. That’s the small optimisation in the loop header.

The comparison inside both loops is the hot line. The outer loop runs `n` times, and the inner one averages about `n` over two, so that comparison happens roughly `n` squared over two times. Concretely, for ten thousand items that’s about fifty million comparisons, which in Python is going to be seconds, not milliseconds.

The thing I’d flag is that it’s the shape, not the constant. If I double the list to twenty thousand, the work goes up four times, not two. So tuning the comparison, or running it on a faster box, doesn’t really help. I’d want to change the approach if this is on a hot path.

Space is fine, though. It only holds two positions, so it doesn’t grow with the input at all. Do you want me to look at trading some memory for speed here?”

Notice what that answer does. It reads the code, finds the hot line, counts it, puts a real number on it, separates the constant from the shape, mentions memory, and hands control back. That is the whole performance, and it takes about forty seconds.

9 Recall card

1. Time is **how many instructions run**, not how many lines you wrote.
2. The **hot line** is the deepest line inside the most loops. Find it first, always.
3. One loop over `n` items means the hot line runs `n` times. Double the input, double the work.
4. Two nested loops over `n` items means roughly `n × n` times. Double the input, and you get **four times** the work.
5. A line that touches every item is a hidden loop: `x in list`, `sorted`, `max`, `sum`, `join`, `list[:]`.

What happens when you type google.com and press Enter

1 What this is, and why they ask it

Between your keypress and the page appearing, roughly a dozen separate machines do roughly a dozen separate jobs. Each one has a name, a purpose, and a way of failing. Today you learn the whole chain end to end, at low resolution. It is a map, not a survey.

Every box on that map gets its own day later in this course. You are not expected to master any of them today. You are expected to be able to name them in order.

This is the most-asked warm-up question in the industry, and it has been for twenty years. Interviewers keep it because it is hard to fake in a useful way. There is no trick and nothing to memorise cleverly, and the answer expands to fill whatever depth you have. A candidate with six months of experience gives a two-minute answer. A candidate with six years gives the same six beats, and then goes forty minutes deep on any one of them when prodded. It tells the interviewer your level in about ninety seconds, which is exactly what a warm-up is for.

2 The story

Meera moved to a new city three weeks ago, and she knows the name of exactly one place to eat. A colleague has mentioned it twice: Bhai Kitchen, apparently the best biryani for miles. It is a small place, too small to be on any of the delivery apps, so she has to ring them herself.

She does not have the number.

She checks her own phone first, because that is where she saves the places she uses. Nothing. So she goes down to the gate and asks the watchman, who has worked in this lane for eleven years and knows every shop in it. He does not have the number either, but he tells her the tea stall on the corner will.

She rings the tea stall. The man there does not have Bhai Kitchen's number, but he does have the number of the sweet shop next door to it. She rings the sweet shop, and they read the number out to her. It has taken almost four minutes and three phone calls.

Before she dials, she saves the number in her phone under "Bhai Kitchen", because she is not doing all of that again on Friday.

She dials, and someone picks up. She says, “Is that Bhai Kitchen?” and waits for them to say yes before she says anything else, because last month she gave a full order to a laundry.

Then she speaks slowly and carefully, because every detail matters. “One chicken biryani, large, extra gravy, no raita. Flat 402, Sunrise Building, behind the petrol pump. Cash when it arrives.” The man repeats the whole thing back to her. Now both of them know exactly what was agreed.

Then nothing happens for a long while. Forty minutes later a rider is at the gate with a carrier bag, and inside it are four separate boxes: rice in one, gravy in another, salad, and a small tub of pickle. Nobody delivers biryani in one box. She carries them upstairs, opens each one, and lays them out on the table in the order she wants to eat them. Only now, with everything opened and arranged, is dinner actually in front of her.

On Friday she orders again. This time the number is already in her phone, and the whole thing is much shorter.

3 The idea in plain English

Meera’s evening is the journey, beat for beat. Here is the mapping.

The browser. The program you type into: Chrome, Firefox, Safari. It is the one doing all of the work below, on your behalf. Meera is the browser.

The URL. What you typed: `google.com`. A **URL** is an address for something on the internet. It has parts. `https://` says how to talk, `google.com` says who to talk to, and anything after a `/` says which page you want.

Finding the number: DNS. Machines on the internet do not find each other by name. They find each other by number — an **IP address**, something like `142.250.183.14`. So the browser’s first job is to turn the name `google.com` into a number. The system that does this is called **DNS**, the Domain Name System, and it is the phone directory of the internet.

That is Meera’s whole hunt for the number, including the shape of it. She asked three places in order, each one wider than the last: her own phone, then someone local whose job is knowing local numbers, then a chain of people who each knew somebody closer to the answer. Your browser does exactly the same, and the details are in §5.

Notice what she did before dialling. She saved the number. Storing an answer so that you do not have to ask for it again is called **caching**, and it is the single most important idea in this entire course. It gets its own day on [day 101](#), and it does not stop appearing after that.

Day 003 goes deep on DNS.

Dialling: the TCP connection. Having the number is not the same as being connected. Before any information moves, the two machines exchange a short set of messages, to agree that they can both hear each other and to set up a reliable line. That agreement is a **TCP connection**, and the opening exchange is called a **handshake**.

Day 004 goes deep on TCP.

“Is that Bhai Kitchen?”: the TLS handshake. Meera confirms who she is speaking to before she says anything real. Your browser does the same, and it also agrees a secret code so that nobody listening on the line can understand what follows. This is **TLS**, and it is what the **s** in **https** means. It costs another exchange of messages.

Day 006 goes deep on TLS.

The order: the HTTP request. Now the browser states precisely what it wants, in a strict format that both sides understand. That message is an **HTTP request**. What comes back is an **HTTP response**. This is the actual conversation. Everything before it was setup.

Day 005 goes deep on HTTP.

The kitchen. On the other end, the request arrives at a machine whose entire job is to receive requests and produce responses: a **web server**. For anything more interesting than a fixed page, the web server hands the work to application code, which usually needs to look something up in a **database**, a program built for storing and retrieving information reliably.

Day 007 is the web server. Day 025 begins databases.

Four boxes, not one. The rider brings rice, gravy, salad and pickle separately. A web page arrives the same way. First comes a document called **HTML**, which describes the structure of the page, and inside it are references to other things the browser must fetch separately: **CSS** files that say what it should look like, **JavaScript** files that make it interactive, images, and fonts. A typical page is not one delivery. It is seventy.

Laying the table: rendering. Meera unpacks and arranges before she eats. The browser reads the HTML, builds a tree of the page’s structure in memory, applies the CSS to work out where everything sits and what colour it is, runs the JavaScript, and paints pixels on your screen. This step is called **rendering**, and it is why a page can be fully downloaded and still not visible yet.

Friday. The second visit is faster because the browser saved things: the number, the images, the CSS, and sometimes the page itself. That is caching again, and it is the honest answer to “why is the second load faster”.

4 The picture

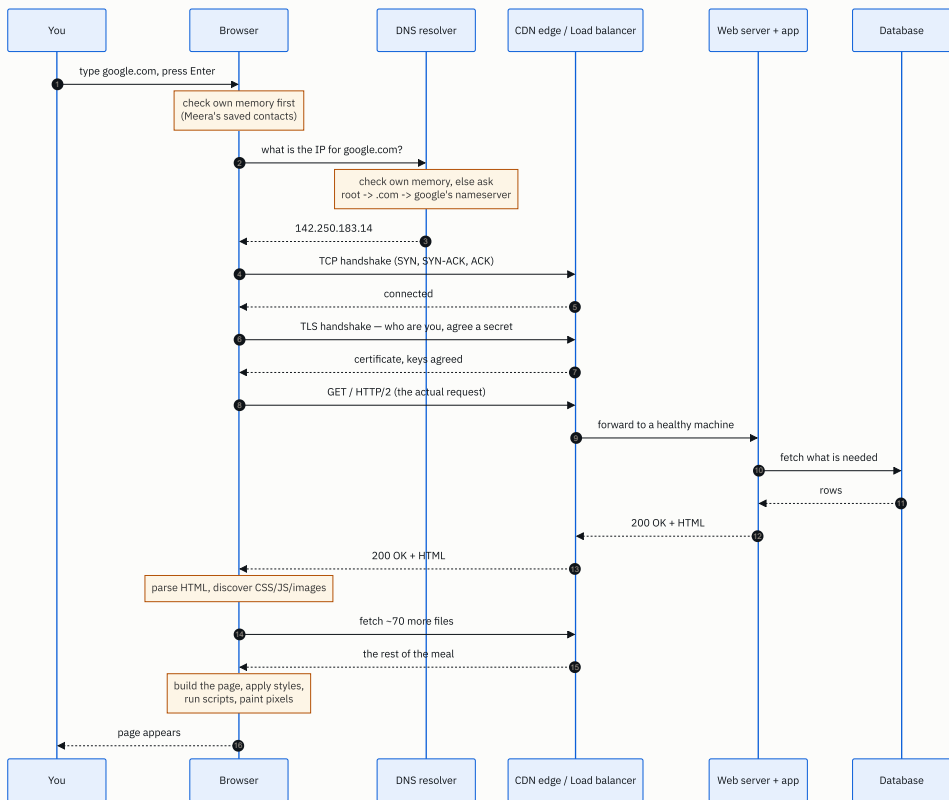


FIGURE 1.1

What to notice: every arrow before `GET /` is setup. Three separate round trips happen before a single byte of the actual page is requested. That is why §6 is about round trips and not about download speed, and it is the insight most candidates miss.

The six beats, compressed to something you can hold in your head:

- | | | | |
|-------------|---|--------|--|
| 1. NAME | → | NUMBER | DNS |
| 2. CONNECT | | | TCP handshake |
| 3. VERIFY | | | TLS handshake |
| 4. ASK | | | HTTP request |
| 5. ANSWER | | | HTTP response (web server → app → database → back) |
| 6. ASSEMBLE | | | parse, fetch the rest, render |

5 How it actually works

Now the same six beats, with the real machinery and the real names.

1. Name to number

The browser does not go straight to the internet. It checks, in this order:

- its **own in-memory DNS cache** (Chrome keeps one, and you can see it at `chrome://net-internals/#dns`),
- the **operating system's** cache and the `hosts` file,
- the **resolver** configured for your network, which is usually your internet provider's, or a public one such as Google's `8.8.8.8` or Cloudflare's `1.1.1.1`.

If the resolver does not know either, it walks the hierarchy. A **root nameserver** tells it who handles `.com`, the **.com TLD nameserver** tells it who handles `google.com`, and **Google's authoritative nameserver** gives the final answer. The resolver then remembers that answer for as long as the record's **TTL** (time to live) allows, which is often 300 seconds.

Big services return a *different* IP depending on where you are asking from, so a user in Pune and a user in Berlin get different machines. That is how a **CDN** — a content delivery network such as Cloudflare, Akamai, Fastly or AWS CloudFront — puts a copy of the site physically near you.

2. Connect

TCP sets up a reliable, ordered stream with a three-message exchange. The browser sends `SYN`, the other side replies `SYN-ACK`, and the browser sends `ACK`. That is one full round trip.

3. Verify

TLS 1.3, the current version, needs one more round trip. The server presents a **certificate** signed by an authority the browser already trusts — Let's Encrypt, DigiCert — proving that it really is `google.com`. Both sides then work out a shared secret, and everything after this point is encrypted. TLS 1.2, which is still common, needs two round trips instead of one.

4. Ask

The browser sends an HTTP request. In HTTP/2 and HTTP/3 it is compressed binary rather than the text you may have seen, but it means this:

```
GET / HTTP/2
host: google.com
user-agent: Mozilla/5.0 ...
accept: text/html, ...
accept-encoding: gzip, br
cookie: ...
```

5. Answer

The request usually lands first on a **load balancer** — AWS ALB, nginx, HAProxy, Envoy — whose job is to pick one healthy machine out of many and forward the request there. Behind it sit the **web server** and the **application** (Django, Spring, Express, Go). The application reads and writes a **database** — PostgreSQL, MySQL, Cassandra, DynamoDB — and very often checks a fast in-memory store such as **Redis** or **Memcached** first, so that the database is not touched at all.

The response comes back with a **status code** (`200 OK` , `404 Not Found` , `500 Internal Server Error`) and the HTML body.

6. Assemble

The browser parses the HTML into a **DOM**, a tree of the page's elements. As it parses, it finds `<link>` , `<script>` and `<img>` tags and starts fetching those too, in parallel over the same connection. It applies the CSS to compute where every element sits (**layout**), then draws them (**paint**), then combines the layers (**composite**).

CSS blocks the first paint, because drawing text that you are about to restyle is worse than waiting. A plain `<script>` tag also blocks parsing, which is why `async` and `defer` exist, and why scripts traditionally go at the bottom.

6 The numbers

This is where most candidates go vague, and where you will not. The whole thing is governed by one quantity: the **round-trip time**, or RTT, which is how long a message takes to get there and come back.

Where RTT comes from. Light in fibre travels at about two-thirds of its speed in a vacuum, so roughly **200,000 km per second**. Cables do not run in straight lines, so assume about 1.5 times the map distance.

Mumbai to Northern Virginia is about 13,000 km on the map, so call it 19,500 km of cable:

$$19,500 \text{ km} \div 200,000 \text{ km/s} = 0.098 \text{ s} = \sim 98 \text{ ms one way} \\ \sim 195 \text{ ms round trip}$$

That is physics. No amount of money makes it smaller. Measured RTT on that route really is around 190-220 ms, so the estimate holds.

Now count the round trips for a first-ever visit, cross-continent, on TLS 1.3. DNS is the odd one out here: your resolver is usually close to you, and the root and `.com` nameservers are answered by machines near you as well, so the whole lookup is much

cheaper than a cross-continent trip. Call it 100 ms in total.

STEP	ROUND TRIPS	COST AT 195 MS RTT
DNS (nothing cached, full walk, nearby machines)	~2	~100 ms
TCP handshake	1	195 ms
TLS 1.3 handshake	1	195 ms
HTTP request → first byte	1	195 ms
Total before the first byte of HTML		~685 ms

That is nearly seven-tenths of a second, and the page has not started downloading.

Now put a CDN edge 15 ms away — a machine in Mumbai instead of Virginia. DNS is cached from an earlier lookup, so it is free:

```
TCP 15 ms + TLS 15 ms + request 15 ms = 45 ms
```

685 ms becomes 45 ms. That is **roughly fifteen times faster, without one line of application code changing**. This single calculation is why CDNs exist, and being able to do it out loud is worth more in an interview than knowing any six product names.

Then the download. A median web page today is about 2.3 MB, spread across roughly 70 requests. On a 20 Mbps connection:

```
2.3 MB × 8 = 18.4 megabits
18.4 Mb ÷ 20 Mbps = 0.92 s
```

So on a typical connection the bytes take about **920 ms**, and the setup took 685 ms. Both matter, which is why the answer to “how do I make it faster” is never just one thing.

And those 70 requests. Under HTTP/1.1 a browser opens at most 6 connections per host, so 70 files means about 12 sequential rounds. At 195 ms each, that is 2.3 seconds of pure waiting. HTTP/2 sends them all down one connection at once and removes almost all of it. That is the entire reason HTTP/2 exists.

7 The trade-offs

Caching buys speed and pays in staleness. Every layer that remembers an answer — the browser, the resolver, the CDN, Redis — is a layer that can serve you something out of date. DNS TTLs are the painful version. Set them to 24 hours and your lookups are

free, but you cannot move your service quickly during an incident. Set them to 60 seconds and you have bought flexibility at the cost of a constant tax of lookups. There is no correct answer, only a chosen one.

TLS buys secrecy and pays a round trip. In 2010 that was a real argument. It is not any more. TLS 1.3 cut it to one round trip, session resumption and 0-RTT cut repeat visits to zero, and the answer today is always “encrypt it”. Know the cost, so that you can say why it no longer decides anything.

A CDN buys distance and pays in money and complexity. You now have copies of your content in fifty places, and a new class of problem: getting rid of a bad copy. Cache invalidation is genuinely hard, and a CDN is not worth it for an internal tool used by two hundred people in one office.

A load balancer buys survivability and pays with a new thing that can fail. It is worth it almost always, but the honest sentence is that you have added a machine whose death takes everything down, so it needs its own backup. [Day 099](#) covers it properly.

When you would not do it this way at all. This whole shape — connect, ask, answer, disconnect — assumes that the client starts every conversation. For a chat app or a live scoreboard, the server needs to speak first, so you keep a connection open instead (WebSockets, [day 140](#)). For two of your own services talking inside one data centre, the whole HTTP-and-JSON layer is often replaced with something leaner (gRPC, [day 022](#)).

8 In the interview

How it gets asked

- “*What happens when you type google.com and press Enter?*” — the classic, word for word.
- “*Walk me through a request end to end.*” — the same question for a backend role.
- “*Why is a page slow the first time and fast after that?*” — the same question again, testing whether you actually understand caching or just recite the word.

They are checking two things: whether you know the pieces, and whether you can structure an answer without rambling. The second matters more.

What to say out loud, in the first ninety seconds

Open by naming the shape of your answer. This single move makes you sound senior, because it tells the interviewer that you have a plan:

“Sure — I’ll go through it in six steps: turning the name into an address, opening the connection, the security handshake, the request, what happens on the server side, and then how the browser actually renders it. Say if you want me to go deeper anywhere.”

Then walk the six beats, roughly fifteen seconds each. Do not go deep unprompted. You are laying out the map so that they can choose where to dig. Finish with the caching point, because it is the one that shows you understand the system rather than the list:

“And most of that only happens on the first visit. DNS is cached, the connection can be reused, and the browser already has the static files. That’s why the second load is a different story entirely.”

The follow-ups

“Where would you put a cache?” Five places, and I would add them in this order: the browser, so that repeat visits cost nothing; a CDN at the edge, for anything static; an in-memory store such as Redis in front of the database, for hot reads; the database’s own buffer pool, which you get for free; and DNS, which is already cached whether you like it or not. The one that pays back fastest is almost always the CDN, because it removes both distance and load in a single move.

“What if DNS fails?” Then nothing works, and it takes the site down as completely as the database being gone. DNS outages are one of the most common causes of large public incidents. The mitigations are having two independent DNS providers, keeping TTLs sane so that a bad record does not linger, and remembering that browsers and resolvers holding stale entries will hide the problem from some users and not others, which makes it maddening to diagnose.

“Why is the second load faster? Be specific.” Four separate reasons, and they are not the same reason. The DNS answer is cached, so the lookup disappears. The TCP connection may be kept alive and reused, so that handshake disappears. TLS session resumption means the security handshake is shorter or gone. And the CSS, JavaScript and images are in the browser’s cache, so most of those seventy requests never happen. Together that removes roughly 600 ms of setup and most of a megabyte.

A model answer

“I’d break it into six steps.

First, the browser has to turn `google.com` into an IP address, because machines route by number, not by name. It checks its own cache, then the OS, then the configured resolver — `8.8.8.8`, or whatever the ISP gives you. If nobody has it cached, the resolver walks the hierarchy: root, then the `.com` nameserver, then Google’s own. Big services answer differently depending on where you’re asking from, so you get an IP that’s near you.

Second, TCP. Three messages — SYN, SYN-ACK, ACK — one round trip, and now there’s a reliable ordered stream.

Third, TLS, because it’s `https`. The server sends a certificate signed by an authority the browser trusts, both sides agree a shared secret, and everything after that is encrypted. On TLS 1.3 that’s one more round trip.

Fourth, the actual HTTP request — `GET /`, with a set of headers.

Fifth, the server side. It typically hits a load balancer first, which picks a healthy backend. The application runs, probably checks Redis, falls through to the database if it has to, and returns a 200 with HTML.

Sixth, the browser parses that HTML into a DOM, discovers it needs another seventy or so files — CSS, JS, images — fetches those, applies styles, runs the scripts, and paints.

The thing I’d highlight is that steps one to four are three or four round trips before a single byte of content moves. Cross-continent that’s around 200 ms per round trip, so you’re at half a second before anything useful happens — which is the entire argument for putting a CDN close to the user. And on a repeat visit most of it disappears, because DNS is cached, the connection is reused, and the static files are already local.

Happy to go deeper on any of those. DNS and the render path are probably the most interesting.”

That takes about two minutes. It names every piece, gets a real number into the room, and ends by inviting the interviewer to choose the deep dive.

9 Recall card

1. Six beats: **name** → **number**, **connect**, **verify**, **ask**, **answer**, **assemble**. DNS, TCP, TLS, HTTP request, server work, render.
2. Three round trips of setup happen **before** the first byte of the page is requested.
3. RTT comes from distance: about 200,000 km/s in fibre. Mumbai to Virginia is roughly **195 ms** round trip.
4. Moving the answer close to the user (a **CDN**) turned 685 ms of setup into 45 ms. That is the biggest single lever there is.
5. The second visit is fast for **four** separate reasons: DNS cached, connection reused, TLS resumed, static files cached.

PRACTICE

Practice — How your code actually runs, and where the time goes

DSA topic: How your code actually runs, and where the time goes **System design topic:** What happens when you type google.com and press Enter

Code these, in this order

Four problems, easiest first. Every one of them is on LeetCode, and every one is free.

For each problem, do the same three things. This is the habit the whole course is built on.

1. Solve it.
2. **Before you submit**, point at the hot line and say how many times it runs.
3. Add a counter to the hot line, print it for a small input, and check whether you were right.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Concatenation of Array	LeetCode 1929 (Easy)	Can you write a single loop at all, and do you know how many times its body runs?
2	Running Sum of 1d Array	LeetCode 1480 (Easy)	One loop that carries a value forward. It is the shape of every prefix problem you will meet on day 037.
3	Richest Customer Wealth	LeetCode 1672 (Easy)	Your first genuinely nested loop. The hot line sits inside both loops. Say the count out loud before you run it.
4	Contains Duplicate	LeetCode 217 (Easy)	The exact function from today's lesson. Write the nested version deliberately , submit it, and watch what happens on the big test case.

On problem 4, do this properly

It is the whole day in one exercise, so do not skip the second half.

- Write `has_duplicate` with two nested loops, exactly as in §5 of the lesson.
- Submit it, and note what LeetCode says.
- Then run it on your own machine, on a list of 5,000 numbers with no duplicates in it, and time it.
- Then double the list to 10,000 numbers and time it again.
- The second number should be roughly **four times** the first, not twice. When you see that with your own eyes rather than being told it, it stays with you.

Use a list with no duplicates, because that is the input that forces the loops to run all the way to the end. A list with a duplicate near the front returns almost immediately and tells you nothing.

You are not expected to know the fast solution yet. That is [day 062](#).

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson. Say them to a wall if there is nobody around. Speaking is a different skill from knowing, and the interview tests the speaking one.

1. *Walk me through this function line by line. Which line runs the most times? Use problem 3 above as the function.*
 2. *What happens when you type google.com into your browser and hit Enter? Six beats, roughly fifteen seconds each. Name the shape of your answer before you start.*
 3. *Is `x in my_list` a single step? Say exactly what the computer does when it runs that line, and what that means for a loop with it inside.*
-

Before you move on

- ☐ I can write today's DSA code from memory, with nothing to refer to.
- ☐ I can name the six beats of the journey in order, out loud, without looking at the lesson.
- ☐ I answered all three questions above out loud.
- ☐ I timed problem 4 at two input sizes and saw the four-times jump myself.

Counting steps: your first cost model

DATA STRUCTURES AND ALGORITHMS

Counting steps: your first cost model

You can count how many times a loop body runs, for any input size, without running the code.

SYSTEM DESIGN

Client and server, explained properly

You can draw the client-server picture and say exactly which machine does what.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *How many times does the inner loop execute if the array has n elements?*
- *What is the difference between a client and a server? Where does your code live?*

1 What this is, and why they ask it

Yesterday you learnt to find the line that runs the most times. Today you learn to say **exactly how many times it runs**, as a number you can work out before the code has ever been executed.

The tool is small. You look at a loop's header, work out how many times its body runs, and multiply when loops sit inside one another. Everything you will ever be asked about cost is built out of that one move.

Interviewers ask for the count because it is the honest version of the complexity question. Anybody can say “ n squared” after hearing it enough times. Very few people can say “the inner loop runs n minus i times, and adding that up over every i gives n times n minus one, over two”. The second answer proves you worked it out. The first proves you have seen the answer somewhere. This distinction is the difference between a pass and a strong hire in the first fifteen minutes of a phone screen.

2 The story

Arun works for a catering company, and today's job is a wedding at a hall on the main road. He arrives two hours early. The hall is empty apart from the tables.

His boss says, “lay the tables”, and drives off. That is the whole instruction.

Arun counts before he carries anything, because he learnt that the hard way. Twelve long tables. Eight chairs at each table. Twelve eights are ninety-six, so he needs ninety-six plates, ninety-six glasses and ninety-six spoons.

He goes to the van and counts what is in it. Ninety plates.

This is the moment the whole evening turns on. It is a quarter past four, the supplier is twenty minutes away, and he knows he is six plates short *before* he has carried a single one inside. He rings and asks for a dozen more, and they arrive while he is still on the fourth table. Nobody ever finds out.

The version of Arun who does not count is the one who finds out at the twelfth table, at half past five, with guests already at the door.

Then he nearly makes the other mistake. He starts loading the trolley with water jugs, one for each place, so ninety-six jugs. He stops with the fourth one in his hand. The jugs do not go in front of each chair. One jug sits in the middle of each table, and so does one basket of bread. That is twelve jugs and twelve baskets, not ninety-six. Same hall, same tables, a completely different number — and the only thing that decides it is whether the thing belongs to a chair or to a table.

He lays the whole hall in fifty minutes.

On the drive back, his boss asks about a booking next Saturday: twenty-four tables, same size. Arun does not need to see the place. Twenty-four eights are a hundred and ninety-two plates, twenty-four jugs, twenty-four baskets. He answers in the van, before anybody has gone anywhere.

3 The idea in plain English

Arun did three things today, and all three of them are on the exam.

He worked out the number **before** doing the work. He noticed that two jobs in the same hall had two different counts. And he answered for a hall he had never seen, because he had a rule rather than a memory.

Let us take those apart.

One loop

A **loop body** is the indented block underneath the loop header — the part that runs again and again. One run of the body is called one **iteration**. Counting steps means counting iterations.

```
for chair in range(8):  
    put_plate(chair)
```

`range(8)` produces the numbers 0, 1, 2, 3, 4, 5, 6, 7. That is eight numbers, so the body runs **8** times. Not 7, and not 9. `range(8)` starts at 0 and stops *before* 8.

Now replace the 8 with a name. If there are `n` chairs, `for chair in range(n)` runs the body **n** times. That is the whole rule for a single loop: read the header, and count how many values it produces.

Two loops, one inside the other

```
for table in range(12):  
    put_jug(table)  
    for chair in range(8):  
        put_plate(table, chair)
```

Two lines, two different counts, and this is the part Arun almost got wrong.

`put_jug` is inside the outer loop only. It runs once per table, so **12** times.

`put_plate` is inside both loops. The outer loop runs 12 times, and for each one of those the inner loop runs its own 8 times. So `put_plate` runs $12 \times 8 = \mathbf{96}$ times.

The rule: loops inside loops multiply. Loops beside each other add. Everything else today is that sentence, applied carefully.

Notice that it is 12×8 and not 12×12 . The two loops count different things — tables and chairs — so give them different names. If there are `n` tables and `m` chairs at each, the inner body runs `n × m` times. It is only `n × n` when both loops walk the same list, which is the case that happens to come up most often in interviews.

When the inner loop depends on the outer one

This is the shape you met yesterday, and the one that gets asked about most.

```
for i in range(n):  
    for j in range(i + 1, n):  
        compare(i, j)
```

Now the inner loop is a different length every time round. When `i` is 0, `j` runs from 1 to $n - 1$, which is $n - 1$ iterations. When `i` is 1, it is $n - 2$. By the time `i` is $n - 1$, it is 0.

So the total is not one multiplication. It is a sum:

$$(n - 1) + (n - 2) + \dots + 2 + 1 + 0$$

Add it up for $n = 8$, where the sum is $7 + 6 + 5 + 4 + 3 + 2 + 1$. Pair the ends: $7 + 1$ is 8, $6 + 2$ is 8, $5 + 3$ is 8. That is three pairs of 8, which is 24, and the 4 in the middle is left over. $24 + 4 = \mathbf{28}$.

The general version of that pairing is $n \times (n - 1) / 2$. Check it: $8 \times 7 / 2 = 28$. It matches. You will meet this number so often that it is worth recognising on sight — 10, 45, 190, 499,500 are the values at $n = 5, 10, 20$ and 1,000.

The loop that halves

```
size = n
while size > 0:
    size = size // 2
```

// is integer division: it divides and throws away the remainder. Start at 1,000 and the sizes go 1,000, 500, 250, 125, 62, 31, 15, 7, 3, 1, and then 0 stops it. That is **10** runs of the body, for an input of a thousand.

Ten, for a thousand. That is not a straight line and it is not a grid. Halving repeatedly is the cheapest useful shape there is, and the number of halvings needed to get from `n` down to 1 is called **log₂ n** — “log base two of n”. You do not need the maths. You need the picture: every step throws away half of what is left, so a thousand becomes ten steps and a million becomes twenty.

When the count depends on the input, not just its size

```
for x in items:
    if x == target:
        return True
```

If `target` is the first item, the body runs once. If it is the last item, or missing, the body runs `n` times. Same code, same `n`, two very different counts.

When that happens, you quote both. The **best case** is 1, the **worst case** is `n`. In interviews the worst case is the default — when somebody says “the cost of this”, they mean the worst case unless they say otherwise.

4 The picture

The hall, with the two counts on it:

	chair0	chair1	chair2	chair3	chair4	chair5	chair6	chair7	
table 0	P	P	P	P	P	P	P	P	+ 1 jug
table 1	P	P	P	P	P	P	P	P	+ 1 jug
...				...					...
table 11	P	P	P	P	P	P	P	P	+ 1 jug

P = one plate. 12 rows x 8 columns = 96 plates.
12 jugs, one per row.

What to notice: the plates fill the grid and the jugs fill the left-hand margin. A line inside both loops touches every cell. A line inside the outer loop only touches every row. That is the entire difference between 96 and 12.

Here is the same thing as code, with the counts written beside each line:

	times this line runs (12 tables, 8 chairs)
for table in range(12):	12
put_jug(table)	12 belongs to the table
for chair in range(8):	12 restarts, 8 each
put_plate(table, chair)	96 belongs to the chair
wipe_table(table)	12 back out to the table level

What to notice: `wipe_table` is indented once, the same as `put_jug`, so it is back at the table level and runs 12 times. Indentation is not decoration in Python. It is the thing that decides the count.

And the uneven shape, where the inner loop shrinks as `i` grows:

i=0	XXXXXXX	7 iterations
i=1	XXXXXX	6
i=2	XXXXX	5
i=3	XXXX	4
i=4	XXX	3
i=5	XX	2
i=6	X	1
i=7		0

28 = 8 x 7 / 2

What to notice: it is a staircase, not a rectangle. A staircase holds about half of the rectangle it sits inside — 28 against 64 — which is where the “/ 2” comes from.

5 The code, built step by step

The way to be certain about a count is to make the program count itself. Add a `steps` variable, tick it once inside the body, and return it. Do that for every shape until the shapes are familiar.

Start with the single loop.

```
def single(n: int) → int:
    steps = 0
    for _i in range(n):
        steps += 1
    return steps
```

`_i` with an underscore is Python's way of saying "I must name this, but I never use it". `steps += 1` is short for `steps = steps + 1`. Call `single(8)` and you get 8. The count is `n`.

Now a loop that skips.

```
def every_second(n: int) → int:
    steps = 0
    for _i in range(0, n, 2):
        steps += 1
    return steps
```

`range(0, n, 2)` means "start at 0, stop before `n`, and go up in twos". For `n = 8` it gives 0, 2, 4, 6, so the body runs 4 times. The count is `n / 2`, rounded up.

Now two loops side by side, which people mistake for a nested loop surprisingly often.

```
def one_after_another(n: int) → int:
    steps = 0
    for _i in range(n):
        steps += 1
    for _j in range(n):
        steps += 1
    return steps
```

The second loop starts only after the first has finished. Nothing multiplies. The count is `n + n = 2n`. For `n = 8` that is 16, not 64.

Now the rectangle.

```
def nested_full(n: int) → int:
    steps = 0
    for _i in range(n):
        for _j in range(n):
            steps += 1
    return steps
```

The inner loop is a fresh `range(n)` every time, so it does not care what `i` is. The count is `n × n`. For `n = 8` that is 64.

Now the staircase.

```
def nested_triangle(n: int) → int:
    steps = 0
    for i in range(n):
        for _j in range(i + 1, n):
            steps += 1
    return steps
```

The inner header mentions `i`, which is the signal that the count is a sum rather than a product. The count is $n \times (n - 1) / 2$. For $n = 8$ that is 28.

And the halving loop.

```
def halving(n: int) → int:
    steps = 0
    size = n
    while size > 0:
        steps += 1
        size = size // 2
    return steps
```

A `while` loop runs its body for as long as the condition is true. Because `size` is cut in half each time, it reaches 0 quickly. The count is about $\log_2 n + 1$.

Here is the complete program. It runs every shape at four sizes and prints a table, so that you can compare your predictions against the truth in one go.

```

"""Day 2 – count first, then check. Every shape you will meet this week."""

def single(n: int) → int:
    """for i in range(n)"""
    steps = 0
    for _i in range(n):
        steps += 1
    return steps

def every_second(n: int) → int:
    """for i in range(0, n, 2)"""
    steps = 0
    for _i in range(0, n, 2):
        steps += 1
    return steps

def one_after_another(n: int) → int:
    """two separate loops, not nested"""
    steps = 0
    for _i in range(n):
        steps += 1
    for _j in range(n):
        steps += 1
    return steps

def nested_full(n: int) → int:
    """inner loop starts at 0 every time"""
    steps = 0
    for _i in range(n):
        for _j in range(n):
            steps += 1
    return steps

def nested_triangle(n: int) → int:
    """inner loop starts at i + 1"""
    steps = 0
    for i in range(n):
        for _j in range(i + 1, n):
            steps += 1
    return steps

def halving(n: int) → int:
    """cut it in half until nothing is left"""
    steps = 0
    size = n
    while size > 0:
        steps += 1
        size = size // 2
    return steps

SHAPES = [
    ("single", single, "n"),
    ("every second", every_second, "n / 2, rounded up"),
    ("one after another", one_after_another, "2n"),
    ("nested full", nested_full, "n x n"),
    ("nested triangle", nested_triangle, "n x (n - 1) / 2"),
    ("halving", halving, "about log2(n) + 1"),
]

if __name__ == "__main__":
    sizes = (8, 16, 64, 1000)
    print(f'{"shape":<20}{"formula":<22}" + ".join(f'{"n=" + str(s):>12}" for s in sizes))

```

```
print("-" * (42 + 12 * len(sizes)))
for name, fn, formula in SHAPES:
    counts = "".join(f"{fn(s):>12,}" for s in sizes)
    print(f"{name:<20}{formula:<22}{counts}")
```

This is exactly what it printed:

shape	formula	n=8	n=16	n=64	n=1000
single	n	8	16	64	1,000
every second	n / 2, rounded up	4	8	32	500
one after another	2n	16	32	128	2,000
nested full	n x n	64	256	4,096	1,000,000
nested triangle	n x (n - 1) / 2	28	120	2,016	499,500
halving	about log ₂ (n) + 1	4	5	7	10

Read the last column downwards, because that column is the entire course in miniature. At one thousand items the shapes cost 1,000, then 500, then 2,000, then one million, then half a million, then **ten**. The gap between the top rows and the middle rows is a thousandfold. The gap between the middle rows and the last row is another hundred-thousandfold.

Now check two of them by hand against the formulas. Nested triangle at $n = 64$ should be $64 \times 63 / 2 = 2,016$, and the program says 2,016. Halving at $n = 1,000$ should be about $\log_2 1,000$, which is just under 10, plus one, and the program says 10. The formulas are not decoration. They predict the measurement exactly.

6 What it costs

Here is every shape from §5, with the arithmetic written out at $n = 1,000$ so that the numbers are real rather than symbolic.

SHAPE	COUNT	AT N = 1,000
One loop	n	1,000
Every second item	$n / 2$	500
Two loops, one after the other	$n + n = 2n$	2,000
Nested, inner from 0	$n \times n$	1,000,000
Nested, inner from $i + 1$	$n \times (n - 1) / 2$	499,500
Halving	$\log_2 n + 1$	10

The two nested rows are the ones to look at. The staircase does half the work of the rectangle — 499,500 against 1,000,000 — and yet, if you double n to 2,000, both of them go up by four times: 4,000,000 and 1,999,000. Halving the work once is a constant-factor win. It does not change what happens when the input grows. That difference is the whole point of [day 003](#).

How to count a loop's header, exactly. `range(a, b)` produces `b - a` values, provided `b` is bigger than `a`, and none at all otherwise. So `range(0, n)` gives `n`, `range(1, n)` gives `n - 1`, and `range(i + 1, n)` gives `n - i - 1`. Nearly every off-by-one in your first month comes from not doing this subtraction deliberately.

Space. Every function in §5 keeps one integer called `steps`, plus a loop variable or two. That does not change when `n` changes. Ten items or ten million, the extra memory is the same handful of values. Compare it with a function that builds a list of every pair: at `n = 1,000` that is 499,500 stored pairs, and now the memory grows as fast as the work does. We measure that properly on [day 007](#).

What this buys you. Once the count is a formula in `n`, you can answer questions about inputs you have never seen — which is exactly what Arun did in the van. An interviewer who asks “and if the array had a million elements?” is asking you to substitute a number into a formula, not to run anything.

7 The traps

Trap one: the near-miss that counts each item with itself

Here is a function that counts how many pairs a list has. It looks right, it runs without complaint, and it returns the wrong number.

```
def count_pairs(items: list[int]) → int:
    pairs = 0
    for i in range(len(items)):
        for j in range(i, len(items)):      # should be i + 1
            pairs += 1
    return pairs
```

The inner loop starts at `i` instead of `i + 1`. So when `i` is 0, `j` is also 0 on the first turn, and the code counts item 0 paired with item 0. An item paired with itself is not a pair.

Run it on a four-item list and count what it should be by hand: item 0 with 1, 2 and 3; item 1 with 2 and 3; item 2 with 3. That is $3 + 2 + 1 = 6$ pairs.

```
near-miss : 10
correct   : 6
n(n-1)/2  : 6
```

Ten, not six. It over-counts by exactly four, which is exactly `n` — one bogus self-pair for every item. Nothing crashes, no error is printed, and the function will pass any test you wrote by eye on a two-element list, because there the answer 3 still looks plausible.

How to catch it every time: before you trust a nested loop, run it on a list of four items and check the count against $n \times (n - 1) / 2 = 6$. Four is small enough to count in your head and big enough to expose the error. Two is not.

Trap two: dividing when you meant to halve

You want a loop that runs half as many times, so you write the obvious thing:

```
n = 10
steps = 0
for i in range(n / 2):
    steps += 1
print(steps)
```

It does not run at all:

```
Traceback (most recent call last):
  File "d2.py", line 3, in <module>
    for i in range(n / 2):
                  ^^^^^^^^^^^^^^^
TypeError: 'float' object cannot be interpreted as an integer
```

Read the message literally, because it is telling you exactly what went wrong. In Python, `/` always produces a **float** — a number with a decimal point. `10 / 2` is not `5`, it is `5.0`. And `range` refuses floats, because there is no sensible answer to “count up to five and a half”.

The fix is `//`, which divides and throws away the remainder, giving a plain whole number:

```
for i in range(n // 2):    # 10 // 2 is 5, not 5.0
```

The `^^^^^^^^^^^^^^` marks under the traceback point at `range(n / 2)`, the exact part of the line Python objected to. This is the same error you will hit on [day 042](#) when you compute the middle of a range, so it is worth fixing the habit now: **use `//` whenever the result is going to be used as a position or a count.**

8 In the interview

How it gets asked

- “How many times does the inner loop execute if the array has n elements?” — the direct version, usually about code they have just shown you.

- “How many iterations does this do in total?” — the same question, asking for the sum rather than the inner count.
- “If I change the inner loop to start at i instead of i plus one, what changes?” — the version that checks whether you actually counted or recognised a pattern.

What to say out loud, in the first ninety seconds

Do not answer with a complexity. Answer with a count, and let the complexity follow.

1. **Read the inner loop’s header out loud.** “The inner loop is range i plus one to n .” Saying it aloud stops you from assuming it is `range(n)`, which is the mistake.
2. **Say whether the inner count depends on the outer variable.** “It mentions i , so the length changes on every pass.” If it does not mention `i`, you are multiplying and you are done in one sentence.
3. **Write the count for one specific i .** “When i is 0 it runs n minus one times, and when i is n minus one it runs zero times.”
4. **Add them up.** “So the total is n minus one, plus n minus two, all the way down to one, which is n times n minus one over two.”
5. **Sanity-check with a small number.** “For n equals four that’s six, and I can count six pairs by hand, so the formula is right.” This step takes five seconds and it is the one that makes an interviewer relax.
6. **Only then name the shape.** “So it’s quadratic — n squared over two, with the two being a constant factor.”

The follow-ups

“Why is it n times n minus one, over two, and not n squared?” Because the inner loop shrinks. It runs $n - 1$ times, then $n - 2$, and so on down to 0, so you are adding a staircase rather than filling a rectangle. Pair the first term with the last, the second with the second-last, and every pair sums to n , which gives you n over two pairs — hence n times n minus one, over two. It is exactly half the rectangle, so the constant is 2, and the growth is the same either way.

“What if the two loops are one after the other instead of nested?” Then you add rather than multiply: $n + n = 2n$. That is a completely different cost. Ten sequential loops over the same list is $10n$, which is still a straight line, while two nested loops is n squared. Nesting is what costs you, not the number of loops on the page.

“How do you know the halving loop is $\log n$?” Count the halvings. From 1,000 you get 500, 250, 125, 62, 31, 15, 7, 3, 1 — ten steps to reach the bottom. Doubling the input to 2,000 adds exactly one step, not a thousand. Any time the input is cut by a constant fraction each pass, the count is the number of times you can do that before you run out, and that is what $\log_2 n$ means.

A model answer

The interviewer puts up the nested loop with `range(i + 1, n)` and asks how many times the comparison runs.

“Let me count the inner loop first. The header is `range(i + 1, n)`, so its length depends on `i` — it isn’t a fixed `n` each time. `range(a, b)` produces `b - a` values, so for a given `i` the inner loop runs `n - i - 1` times.

When `i` is 0 that’s `n - 1` iterations. When `i` is 1 it’s `n - 2`. And when `i` gets to `n - 1` it’s zero, so the last outer pass does nothing at all.

The total is the sum of all of those: `n - 1`, plus `n - 2`, down to 1 and 0. That’s the standard staircase sum, and it collapses to `n` times `n` minus one, over two.

Let me check it on a small case. For `n` equals 4, the formula gives 4 times 3 over 2, which is 6 — and by hand that’s item 0 against 1, 2 and 3, then 1 against 2 and 3, then 2 against 3. Three plus two plus one is six. It matches.

So the comparison runs about `n` squared over two times. For `n` of a thousand, that’s around half a million — 499,500 exactly. The one-half is a constant factor, so as a shape it’s quadratic: if you double `n`, the work goes up roughly four times.

And if you’d started the inner loop at `i` instead of `i + 1`, you’d add one more iteration per outer pass, so it becomes `n` times `n` plus one, over two. That’s `n` extra comparisons of each item with itself, which for a pairs problem would also be a correctness bug, not just a cost one.”

That last paragraph is the one that gets remembered. The candidate answered a question they were not asked, in one sentence, and it showed they had understood the loop rather than recalled a formula.

9 Recall card

1. Read the header, count the values. `range(a, b)` produces `b - a` iterations.
2. **Nested loops multiply. Loops side by side add.** A line inside both loops runs `n × m` times; a line inside the outer one only runs `n` times.
3. If the inner header mentions `i`, the answer is a **sum**, not a product, and it collapses to `n × (n - 1) / 2`.
4. Halving each pass means about **log₂ n** steps. A thousand is ten, a million is twenty.
5. When the count depends on the values and not just on `n`, quote **best case and worst case**. The worst case is what “the cost” means unless somebody says otherwise.

1 What this is, and why they ask it

A **client** is the program that asks for something. A **server** is the program that waits to be asked, and answers. Every diagram you will draw for the next 178 days is built out of those two words and an arrow between them.

Yesterday you followed one request across the whole internet. Today you stop and look carefully at the two ends of it, because almost every confused answer in a system design interview comes from being unclear about which machine is doing what.

Interviewers ask this early and they ask it of everybody, including senior candidates. It sounds like a definitions question and it is not. What they are really testing is whether you know **where your code runs**, because that single fact decides what you are allowed to trust, what an attacker can change, and what you have to pay for. A candidate who says “the browser does the checking” without flinching has just told the interviewer that they would ship a security hole.

2 The story

The canteen at Ravi’s college opens at eight in the morning and shuts at eight at night. In three years, Ravi has never seen it closed during the day, and he has never once seen anybody go behind the counter.

That is the arrangement, and everyone understands it without it ever being explained. You stand on your side. The kitchen is on the other side. The food, the gas, the big cooking vessels and the record of who has paid all live on the far side, and you never touch any of it.

You walk up, and you ask. “One plate of poha and a tea.” The man at the counter takes your money and calls the order through to the kitchen. A few minutes later a plate comes back through the hatch, and it is yours.

Ravi has thought about all this exactly once, on a morning when he turned up at ten past eight and the shutters had only just gone up. There was nobody else in the hall. The kitchen was already hot, the vessels were already full, and the man was standing at the counter with nothing to do, waiting. Nobody had told him that Ravi was coming. He had opened up anyway, because his job is to be ready for whoever walks in, including nobody at all.

At one o'clock it is a different scene. Three hundred students want lunch inside forty minutes, and there is one counter. The line goes out of the door. The kitchen has not got slower and the man has not got slower. There are simply more people asking than there are hands to answer, so everybody waits. By half past two the hall is empty again, and the man is back to standing there with nothing to do.

Some days you ask and the answer is no. The vada pav is finished. Your meal card covers breakfast, and it is now noon. You cannot argue your way past either of those, because the deciding does not happen on your side of the counter. It happens on his.

And when Ravi drops his plate on the way to a table, the kitchen does not care. It still has the rice, the pans and the recipe. He goes back and asks again.

3 The idea in plain English

The counter in that story is the most important line in this course. Let us go along it piece by piece.

The client is the one who asks. Ravi is the client. In real systems, the client is Chrome or Firefox, the app on your phone, or one of your own programs calling another. The defining feature is not that it is small or that a person is holding it. It is that **the client starts every conversation.**

The server is the one that waits and answers. The counter and the kitchen are the server. A server never walks over to your table and starts talking. It sits there, already running, ready for a request that may never come. That is why the kitchen was hot at ten past eight with nobody in the hall.

A request and a response. "One plate of poha" is a **request**. The plate coming back through the hatch is a **response**. One request, one response, and the client waits in between. That pattern has a name, **request-response**, and it is the shape of almost everything you will build.

The counter is a boundary, and it is the whole point. Ravi cannot walk behind it. He cannot look in the vessels, and he cannot mark himself as paid in the record. He can only ask, and the other side decides what to hand over. In software this boundary is real and enforced: the client cannot read the server's memory, cannot read its files, and cannot touch the database directly. It can only send requests.

Which means the deciding happens on the server side. "The vada pav is finished" and "your card doesn't cover lunch" are decisions made behind the counter, where the facts live. This is the sentence to carry into every interview: **anything the client claims**

can be a lie, so anything that matters must be checked on the server. A price, a user's identity, whether they are allowed to delete that record — the client may ask, but the server decides.

One counter, three hundred students. A single server handles many clients, and they do not get a counter each. When more requests arrive than the server can finish, the rest wait, and the line grows. Nothing has broken and nothing has slowed down; there is simply more asking than answering. §6 puts numbers on exactly that.

Dropping the plate. Ravi's plate is his copy. The kitchen keeps the real thing — the rice, the pans, the recipe. In software, the client holds a copy of some data and can lose it at any time by closing the tab, and the server holds the version that counts. That version is usually kept in a **database**, which is a program built for storing information reliably, and which we begin on day 025.

So where does your code live? In two places, and knowing which is which is the actual exam question.

The **frontend** is the code that crosses the counter. HTML, CSS and JavaScript are sent to the client and run on the client's device — on Ravi's tray, in his hands, where he can do whatever he likes with them. The **backend** is the code that never leaves the kitchen. It runs on the server's machine, it is the only code that touches the database, and no client ever sees it.

That is why “the browser will check the price” is the wrong answer, and why “the browser can check the price to be helpful, but the server must check it again to be safe” is the right one.

4 The picture

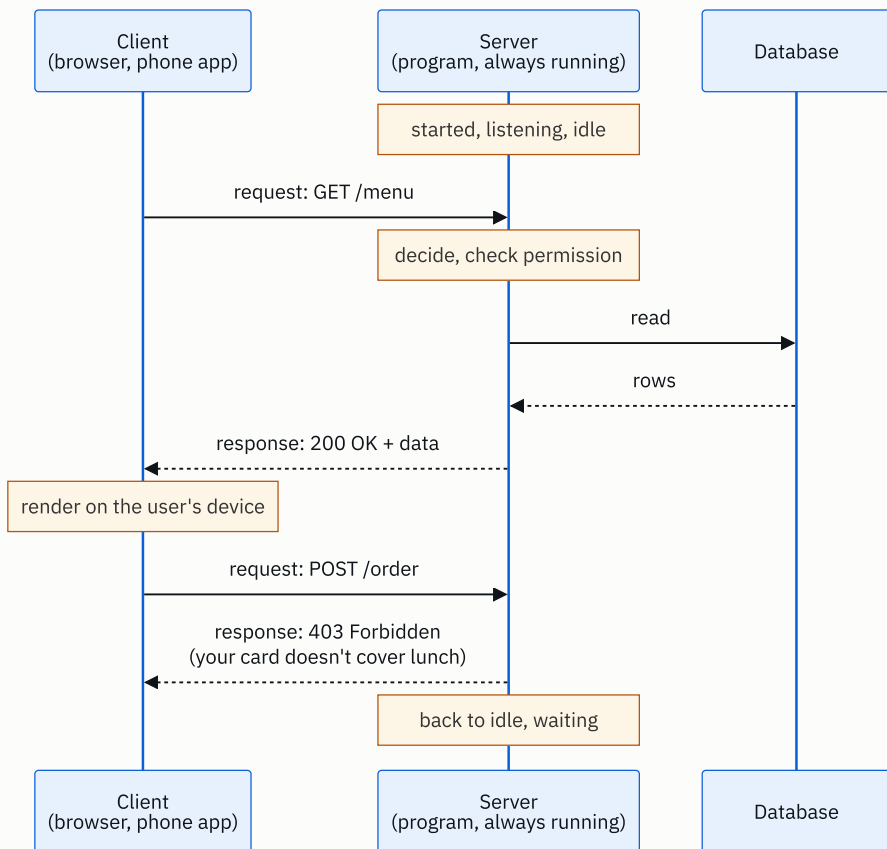
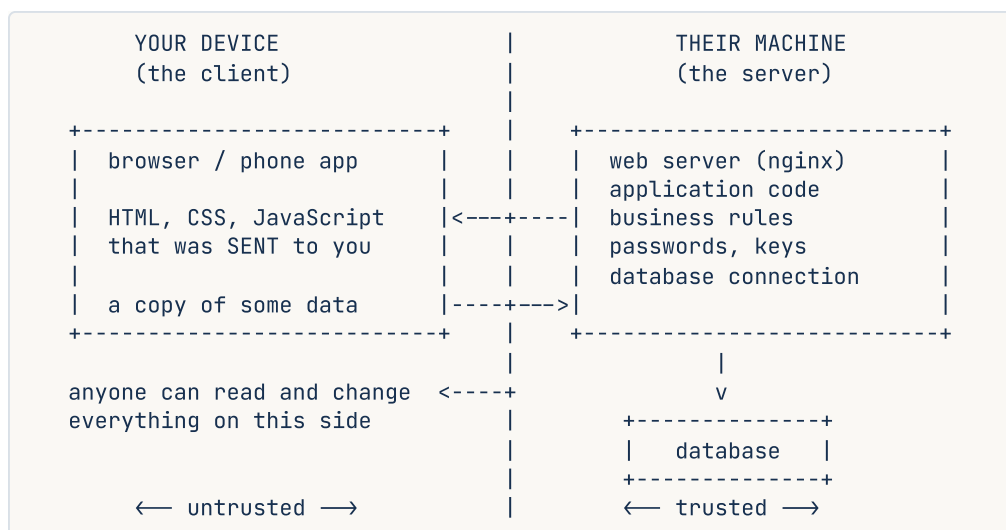


FIGURE 2.1

What to notice: every arrow starts at the client. The server never speaks first. It only ever replies, and between requests it sits idle, which is exactly the man at the counter at half past two.

Now the boundary itself, which is the picture worth memorising:



What to notice: the frontend files start on the right and end up on the left. Once they have crossed, they are the user's, and the user can edit them. Nothing on the left can be trusted, no matter what it says about itself. Everything that decides anything sits on the right.

5 How it actually works

A server is a program, not a building

This is the single biggest beginner misunderstanding, so say it plainly: a server is a **program that is running and waiting for connections**. It is not special hardware. It is not a room with cold air. The machine it runs on is often called a server too, which is where the confusion comes from, but the thing that matters is the process.

To wait for connections, that program **listens on a port**. A **port** is a number from 0 to 65535 that lets one machine run many different listening programs at once and keep them apart: 80 for plain HTTP, 443 for HTTPS, 5432 for PostgreSQL, 6379 for Redis. [Day 003](#) goes deep on ports and addresses.

So “the server” is really: this machine, at this IP address, running this program, listening on this port. Real ones you will meet by name: **nginx** and **Apache** as web servers, **unicorn** and **uicorn** running Python applications, **Node.js** with Express, **Spring Boot** for Java. [Day 007](#) is about what a web server actually does all day.

Real clients, likewise: **Chrome**, **Firefox** and **Safari**; the Instagram app on a phone; **curl** and **Postman** when you are testing; and very often another one of your own services, because a program can be a client and a server at the same time.

The same machine can be both

When you run a web application on your own laptop and open `http://localhost:8000`, your laptop is the client *and* the server. `localhost`, also written `127.0.0.1`, means “this same machine”. The request goes out of the browser, into the operating system, and straight back into your own program without ever touching a network cable.

This is worth doing once, deliberately, because it kills the “server means a distant building” idea for good.

What actually happens on a request

1. The server program starts and calls `listen()` on a port. It now sits idle.
2. A client connects to that IP and port, and the operating system completes the TCP handshake from day 001.
3. The client sends a request. The server reads it, works out what is being asked, checks whether this client is allowed it, reads or writes the database, and builds a response.
4. The response goes back, and the connection is either closed or kept open for reuse.
5. The server returns to waiting. It usually remembers nothing about you.

That last point has a name. An HTTP server is normally **stateless**: each request must carry everything needed to understand it, because the server kept nothing from last time. Your identity travels in a cookie or a token on every single request. Anything worth keeping is written to the database or to **Redis**, not held in the server’s memory.

The payoff is enormous, and it is worth saying out loud in an interview: because no server remembers anything, any server can answer any request, so you can put twenty identical machines behind a load balancer and it makes no difference which one you get. [Day 099](#) covers that.

Many clients at once

One server program handles thousands of clients at the same time. There are three common ways it does that: many processes (nginx workers), many threads (a Java thread pool), or one thread doing an event loop (Node.js, or Python’s `asyncio`). Connections that arrive faster than the server can accept them wait in a queue called the **backlog**, and when that fills up, new connections are refused outright. [Day 008](#) covers processes and threads properly.

When it fails

If the server process dies, every client gets a connection error, and clients retry. If a client dies, the server barely notices; it cleans up the connection and carries on. That asymmetry is why the server holds anything worth keeping. The kitchen survives a dropped plate. A dropped kitchen is a different kind of day.

6 The numbers

How much one server can do. Take an ordinary small cloud machine: 2 vCPUs. Say each request needs 20 ms of processing.

$$\begin{array}{ll} 2 \text{ vCPU} \times 1,000 \text{ ms of CPU per second} & = 2,000 \text{ ms of work available per second} \\ 2,000 \text{ ms} \div 20 \text{ ms per request} & = 100 \text{ requests per second} \end{array}$$

One hundred requests a second, from one modest machine. Hold on to that figure; it is a useful reference point for the rest of the course.

Now size the college app. 5,000 students, and each of them makes about 20 requests a day:

$$\begin{array}{ll} 5,000 \times 20 & = 100,000 \text{ requests per day} \\ 100,000 \div 86,400 \text{ s} & = 1.16 \text{ requests per second on average} \end{array}$$

Just over one request a second, against a capacity of 100. Traffic is never flat, though, so assume the lunchtime peak is ten times the average: 12 requests a second. Still eight times inside what one machine can do. **One server, and no load balancer.** Being able to say that, with the arithmetic behind it, is worth more than proposing microservices.

Now the version that needs more than one machine. Same app, 5 million users:

$$\begin{array}{ll} 5,000,000 \times 20 & = 100,000,000 \text{ requests per day} \\ 100,000,000 \div 86,400 & = 1,157 \text{ requests per second on average} \\ \times 3 \text{ for peak} & = 3,500 \text{ requests per second} \\ 3,500 \div 100 & = 35 \text{ machines, so call it 50 with headroom} \end{array}$$

That single division is the whole reason load balancers and horizontal scaling exist. The shape of the system did not change. The arithmetic did.

The lunchtime queue, with real numbers. The story said 300 students in 40 minutes, through one counter:

$$\begin{array}{ll} \text{arrivals: } 300 \text{ students} \div 2,400 \text{ seconds} & = 0.125 \text{ students per second} \\ \text{service : } 1 \text{ student every } 20 \text{ seconds} & = 0.05 \text{ students per second} \end{array}$$

Arrivals are two and a half times what the counter can serve. The line does not settle at some length — it **grows for the whole 40 minutes**, and only drains after the rush ends. This is the single most useful queueing fact in system design: once arrival rate passes service rate, waiting time does not rise a little, it rises without limit until the traffic stops. A server at 100% capacity is not “fully used”. It is a queue that is getting longer.

Memory per connection. Roughly 10 KB of kernel and application memory per open connection:

```
10,000 concurrent connections × 10 KB = 100 MB
```

Which is nothing. Connections are cheap; the CPU work behind them is not. That is why “10,000 users are connected” and “10,000 requests are in flight” are completely different statements.

Bandwidth out. If each response is 2 KB:

```
100,000,000 responses × 2 KB = 200 GB per day leaving the server
```

At cloud egress prices of roughly \$0.09 per GB, that is about \$18 a day, or \$540 a month, just to hand the bytes over. Frontend files served from a CDN instead are the usual fix.

7 The trade-offs

Work on the client is free to you, and cannot be trusted. Every calculation you push into the browser costs you no CPU, no bandwidth and no round trip, and it feels instant to the user. But the code is on the user’s device, so they can read it, change it, or skip it entirely with a single `curl` command. Validation, pricing and permission checks that live only in the frontend are not checks at all. The rule that survives every interview: **check on the client for a good experience, check again on the server for correctness.**

Work on the server is trustworthy, and you pay for all of it. Every request is your CPU, your bandwidth and your bill, plus a round trip the user waits through. A thick client that does its own rendering and calculation is why single-page applications took over.

Stateless servers cost a lookup and buy you scale. Keeping nothing between requests means reading the session from Redis or the database every single time, which is real work. What you get is that any machine can serve any request, so scaling is just adding machines. The alternative — sticky sessions, where a user is pinned to one machine that remembers them — saves the lookup and gives you a nasty failure mode: when that machine dies, everybody it was holding gets logged out.

I would not use client-server at all if... the two ends need to talk to each other directly and constantly. A one-to-one video call routed through a server means every frame travels twice and you pay for all of it, so real calls use peer-to-peer WebRTC and only fall back to a relay when the network forces it. File sharing at scale is the same argument,

which is what BitTorrent is. And a note-taking app that must work on a train with no signal has to be offline-first, with the client holding the real copy and reconciling later, which is a genuinely harder design.

And when the server must speak first, this whole shape does not fit. Chat, live scores and notifications need the server to push, which means holding a connection open — WebSockets, on [day 140](#).

8 In the interview

How it gets asked

- “What’s the difference between a client and a server?” — the plain version, usually in the first five minutes, and usually as a warm-up before something harder.
- “Where does your code run?” or “which part of this runs in the browser?” — the version that actually separates candidates.
- “A user changes the price in the browser before checking out. What happens?” — the same question dressed as a scenario. This is the one they remember your answer to.

What to say out loud, in the first ninety seconds

Do not recite definitions. Draw the boundary and put things on either side of it.

1. **Give the one-line difference.** *“The client asks, the server waits and answers. The client always starts the conversation.”*
2. **Say what a server actually is.** *“A server isn’t special hardware — it’s a program that’s running and listening on a port, ready for requests that may never come.”*
3. **Draw the line and name both sides.** Frontend on the left, backend and database on the right.
4. **Say what crosses it.** *“HTML, CSS and JavaScript get sent to the client and run on the user’s device. The application code and the database credentials never leave the server.”*
5. **State the consequence, unprompted.** *“Which means everything on the client side is untrusted. Anything that matters gets checked again on the server.”*
6. **Offer the next layer.** *“Happy to go into how one server handles many clients at once, or what happens when one machine isn’t enough.”*

Step 5 is the one that gets you the follow-up you want, rather than the one you fear.

The follow-ups

“Is a server special hardware?” No. It is a process listening on a port. The machine may be a rack in a data centre or it may be your laptop — when you run an app locally and hit `localhost:8000`, your laptop is both client and server at once. What makes some-

thing a server is the waiting, not the metal.

“Can one program be both a client and a server?” Constantly, and in real systems it is normal. Your web application is a server to the browser and a client to PostgreSQL and to Redis. The moment it calls a payments provider it is a client again. “Client” and “server” describe a role in one conversation, not a permanent identity.

“A user edits the price in the browser and submits the order. What happens?” Nothing, if it is built correctly, because the browser never gets to decide the price. The client sends a product id and a quantity, and the server looks the price up itself and computes the total. If your API accepts a price from the client, you have a hole, and somebody will find it. The general rule: the client sends *intent*, the server determines *facts*.

A model answer

“The client is whatever asks for something, and the server is whatever waits to be asked and answers. The important asymmetry is that the client always starts the conversation — a server never contacts you out of nowhere; it sits there listening on a port until a request arrives.

A server isn’t special hardware, which I think is the common confusion. It’s a program in a listening state. My laptop running a local app on port 8000 is a real server; it’s just one with a single user.

On where the code lives, I’d split it at the network boundary. The frontend — HTML, CSS, JavaScript — is sent across to the client and executes on the user’s device. The backend — the application logic, the database credentials, the business rules — never leaves the server. The database sits behind the server and the client can’t reach it at all.

The consequence I always want to state explicitly is that everything on the client side is untrusted. The user can read the JavaScript, modify it, or skip the browser entirely and hit the API with curl. So client-side validation is for user experience, and server-side validation is for correctness. If a price or a permission check only exists in the frontend, it doesn’t exist.

The other thing worth mentioning is that HTTP servers are usually stateless — they keep nothing between requests, and identity travels in a token on each one. That’s slightly more work per request, and in exchange any machine can serve any request, which is what lets you scale by adding machines instead of by making one bigger.”

That is about ninety seconds. It defines both terms, kills the hardware misconception, answers the “where does your code live” half properly, and volunteers the security consequence before being asked.

9 Recall card

1. **The client asks. The server waits and answers.** The client always starts the conversation.
2. A server is a **program listening on a port**, not special hardware. Your own laptop can be one.
3. Frontend code **crosses the boundary** and runs on the user’s device. Backend code and database credentials never leave the server.
4. Therefore the client is **untrusted**. Client-side checks are for experience; server-side checks are for correctness. The client sends intent, the server decides facts.
5. Servers are usually **stateless**, so any machine can answer any request. That is what makes adding machines work.

DSA topic: Counting steps: your first cost model **System design topic:** Client and server, explained properly

Code these, in this order

Four problems, easiest first, and each one is a different counting shape. Every one is on LeetCode and free.

Today the habit gains a step. For each problem:

1. **Before you write anything**, say how many times the loop body will run for an input of size n . Commit to a formula, not a feeling.
2. Solve it.
3. Add a `steps` counter to the loop body, run it on a small input, and compare the number with your prediction.

Getting the prediction wrong is useful. Getting it wrong and not noticing is the thing this step exists to prevent.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Final Value of Variable After Performing Operations	LeetCode 2011 (Easy)	The simplest possible count. One loop, no nesting, body runs exactly n times. Say “ n ” out loud before you start.
2	Number of Steps to Reduce a Number to Zero	LeetCode 1342 (Easy)	The halving loop. For $n = 14$ the answer is 6, not 14. Count the steps for 1,000 and see that it is 10-ish, not 1,000.
3	Plus One	LeetCode 66 (Easy)	The count depends on the input, not just its size. <code>[1,2,3]</code> costs one step; <code>[9,9,9]</code> costs three. Best case and worst case in one small function.
4	Number of Good Pairs	LeetCode 1512 (Easy)	The staircase. Write the nested version first and check the count against $n \times (n - 1) / 2$ before you improve it.

On problem 4, do this properly

- Write the nested version, with the inner loop starting at `i + 1`.
- Run it on a list of four items and check that the body ran **6** times, not 10 and not 16.
- Then change the inner loop to start at `i` instead of `i + 1`, run it again, and watch the count become 10 and the answer become wrong. That is trap one from §7 of the lesson, happening to you rather than being described to you.

- Change it back.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How many times does the inner loop execute if the array has n elements?* Use problem 4 above as the code. Do not stop at “ n squared” — give the exact count, then check it at $n = 4$ the way §8 of the lesson does.
 2. *What is the difference between a client and a server? Where does your code live?* Draw the boundary as you talk, and get to the untrusted-client consequence without being asked for it.
 3. *A user changes the price in the browser before checking out. What happens, and why?* One sentence on what the client is allowed to send, and one on what the server must work out for itself.
-

Before you move on

- [] I predicted the count before coding, on all four problems.
- [] I checked at least two predictions with a `steps` counter and they matched.
- [] I can say why the staircase is $n \times (n - 1) / 2$ by pairing the ends, without looking it up.
- [] I can name what lives on the client side and what lives on the server side, out loud, from memory.

Big-O in plain English

DATA STRUCTURES AND ALGORITHMS

Big-O in plain English

You can say $O(n)$, $O(n \log n)$ or $O(n^2)$ about your own code and defend the answer.

SYSTEM DESIGN

IP addresses, ports, and DNS

You can explain how a name becomes a machine, and what a port is for.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What is the time complexity of your solution?*
- *How does the browser find the server for google.com?*

1 What this is, and why they ask it

Big-O is a way of describing how the cost of your code grows when the input gets bigger. It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the **shape** of the growth.

Yesterday you counted steps exactly: $n \times (n - 1) / 2$, or 499,500 at $n = 1,000$. Today you take that exact count and reduce it to one short phrase, $O(n^2)$, which is the phrase the interviewer is waiting to hear.

This is the single most-asked question in a coding interview. It comes after every solution you write, in almost every round, at almost every company. “What’s the time complexity?” is the standard closing move, and it is usually followed by “can you do better?”, which really means “you have just told me $O(n^2)$ and I want $O(n \log n)$ ”. Getting the notation wrong makes a correct solution look like a lucky one. Getting it right, with the counting behind it, makes a merely good solution look deliberate.

2 The story

Meena teaches maths to Class 9 at a school in Pune. There are forty students in her section, and on Friday afternoon she takes in their test sheets.

Marking them is a job she knows exactly. Each sheet takes her about four minutes. Forty sheets at four minutes each is a hundred and sixty minutes — under three hours, spread across the weekend. She has been doing this since 2009 and the sum has never once surprised her.

On Monday the head of department stops her outside the staff room with a different job. Two of the sheets have the same wrong working in question six, word for word. He wants the whole set checked for copying.

Meena says yes, and then, walking down the corridor, works out what she has just agreed to. Checking for copying is not one look at each sheet. It is holding one sheet against another. The first one has to go against every other one, which is thirty-nine comparisons. The second goes against the ones she has not already done, which is thirty-eight. And so on, all the way down. She adds it up before she reaches the stairs: seven hundred and eighty pairs. At even one minute a pair, that is thirteen hours.

The first job grew with the size of the pile. The second job grew with the pile multiplied by itself, and that turns out to be a completely different animal.

Farid, who teaches the other section, offers to take half. Farid is genuinely quicker than her — about thirty seconds a pair, half her time. Meena is grateful, and also clear-eyed about it. Half of thirteen hours is six and a half hours. Being twice as fast did not change the job. It only moved it.

Then she remembers that the school is opening a second section in June. Eighty students instead of forty. She does that sum standing on the stairs. The marking goes from three hours to six, which doubles, and that feels fair. The copying check goes from seven hundred and eighty pairs to three thousand one hundred and sixty. Twice the students, four times the work.

She turns round and goes back to ask the head for a different way of finding copies.

3 The idea in plain English

Meena worked out three things on those stairs, and all three of them are what Big-O is.

She saw that **two jobs on the same pile can grow at different speeds**. She saw that **being twice as fast did not help**, because the shape stayed the same. And she saw that **doubling the input did not double the second job** — it quadrupled it.

Big-O is the notation for exactly those three observations. Let us build it.

It starts with the exact count

You already know how to get the exact count. From [day 002](#):

CODE SHAPE	EXACT COUNT AT SIZE <code>N</code>
One loop	<code>n</code>
Two loops, one after the other	<code>2n</code>
Nested loops, inner from 0	<code>n × n</code>
Nested loops, inner from <code>i + 1</code>	<code>n × (n - 1) / 2</code>
Halving each pass	about <code>log₂ n</code>

Big-O does not replace this. It is a summary of it. You must still be able to produce the exact count, because that is what you say when the interviewer asks “why?”.

Then you throw two things away

Throw away constant multipliers. $2n$ becomes $O(n)$. $n \times (n - 1) / 2$, which is really $n^2/2 - n/2$, becomes $O(n^2)$. Farid being twice as fast is a constant multiplier of one half, and it did not rescue Meena's weekend.

Throw away everything except the fastest-growing term. If a function costs $n^2 + 500n + 10,000$, that is $O(n^2)$. The $500n$ and the $10,000$ look enormous at first — at $n = 10$ they are 5,000 and 10,000 against a mere 100. But at $n = 100,000$ the n^2 term is ten billion, and the other two together come to barely fifty million, which is half of one percent of the total. The biggest term wins, and it wins by more and more as n grows.

That is the whole mechanical rule: **drop the constants, keep the biggest term.**

What is actually left

What is left is the **shape** — the answer to “if I double the input, what happens to the time?”.

BIG-O	NAME	DOUBLE THE INPUT AND THE TIME...	EXAMPLE
$O(1)$	constant	does not change	reading <code>items[5]</code>
$O(\log n)$	logarithmic	goes up by one step	halving until you reach 1
$O(n)$	linear	doubles	one loop over the list
$O(n \log n)$	linearithmic	slightly more than doubles	sorting
$O(n^2)$	quadratic	goes up four times	every pair, like Meena's check
$O(2^n)$	exponential	squares — it is over	every subset of a set

Read the third column downwards. That column is the answer to almost every “and if the input were bigger?” question you will ever be asked.

Saying it out loud

$O(n)$ is spoken “oh of n”, or “order n”, or just “linear”. $O(n^2)$ is “oh of n squared”, or “quadratic”. $O(n \log n)$ is “n log n” — nobody says “n logarithm n”. $O(1)$ is “constant time”, and it does not mean fast. It means **the cost does not depend on the input size**. Reaching for one item in a list of ten million is $O(1)$, and so is a fixed calculation that takes a whole millisecond.

The word “worst”

$O(n)$ on its own means the **worst case**, unless somebody says otherwise. If you search a list of n items for a value, the best case is one step and the worst case is n steps, and the complexity you quote is $O(n)$.

You will meet two more words later, and they are worth naming now so that they are not a surprise. **Average case** is the cost over typical inputs. **Amortised** cost is the average per operation over a long run of them, which is what makes Python's `list.append` cheap on [day 005](#). For today, worst case is the default, and it is the right default.

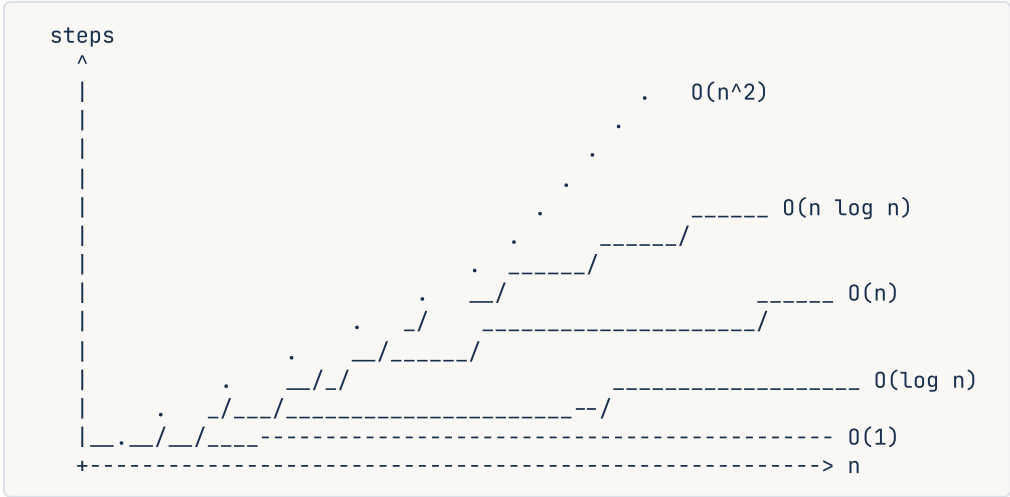
4 The picture

Here is what the shapes actually do, at four input sizes.

	n = 10	n = 100	n = 1,000	n = 1,000,000
$O(1)$	1	1	1	1
$O(\log n)$	3	7	10	20
$O(n)$	10	100	1,000	1,000,000
$O(n \log n)$	33	664	9,966	19,931,569
$O(n^2)$	100	10,000	1,000,000	1,000,000,000,000
$O(2^n)$	1,024	1.3×10^{30}	forever	forever

What to notice: at $n = 10$ every row is small, and the differences look academic — 1 against 100 is nothing you would ever feel. At $n = 1,000,000$, $O(\log n)$ is twenty steps and $O(n^2)$ is a trillion. **The shapes only separate when the input is large, which is exactly where interview questions live.**

Now the same thing as curves, so the growth is visible rather than tabulated:



What to notice: $O(n^2)$ starts *below* $O(n \log n)$ for tiny inputs and then leaves it far behind. Big-O is a statement about the right-hand side of this chart, not the left. That is precisely why constants get dropped — a constant shifts a curve up or down a little, and it never changes which curve ends up on top.

And here is Meena's pile, drawn as the pairs she has to check when there are six sheets:

```

      sheet: 0      1      2      3      4      5
              |      |      |      |      |      |
0  -----+-----X-----X-----X-----X-----X      5 comparisons
1  -----+-----X-----X-----X-----X      4
2  -----+-----X-----X-----X      3
3  -----+-----X-----X      2
4  -----+-----X      1
5  -----+      0
                                     -----
                                     15 = 6 x 5 / 2
```

What to notice: the X marks form a triangle, which is half of the six-by-six square. Half is a constant factor, so the triangle and the square are both $O(n^2)$. The picture is different. The shape is the same.

5 The code, built step by step

The way to make Big-O stop being notation is to measure it. Write each shape, count its steps, then **double the input and look at the ratio**. The ratio is the shape.

Start with constant time.

```
def constant(items: list[int]) → int:
    """Look at one item. The list could hold a billion; this does not care."""
    if not items:
        return 0
    return items[0]
```

There is no loop. The body does the same work for a list of 10 and a list of 10 million. That is $O(1)$. Note that `items[0]` really is one step — [day 009](#) explains why reaching into the middle of a list is not a search.

Now linear time.

```
def linear(items: list[int]) → int:
    total = 0
    for x in items:
        total += x
    return total
```

The body runs once per item, so the count is exactly n . That is $O(n)$. Double the list and the work doubles.

Now the case people get wrong.

```
def also_linear(items: list[int]) → int:
    total = 0
    for x in items:
        total += x
    for x in items:
        total += x
    return total
```

Two loops, one after the other, so the count is $2n$. The constant 2 is dropped, and this is still $O(n)$. **Two loops side by side do not make a quadratic.** Only nesting does.

Now quadratic time.

```
def quadratic(items: list[int]) → int:
    pairs = 0
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j]:
                pairs += 1
    return pairs
```

This is Meena's copying check. The exact count is $n \times (n - 1) / 2$. Drop the $/ 2$ and the $- 1$, and it is $O(n^2)$.

Now logarithmic time.

```
def logarithmic(n: int) → int:
    steps = 0
    while n > 1:
        n = n // 2
        steps += 1
    return steps
```

Each pass throws away half of what is left, so the count is about $\log_2 n$. That is $O(\log n)$. Notice that nobody writes the base. Changing the base only multiplies by a constant, and constants are dropped, so $\log_2 n$ and $\log_{10} n$ are both just $O(\log n)$.

And the one that ends careers.

```
def exponential(n: int) → int:
    """Every way of choosing yes-or-no, n times over."""
    if n == 0:
        return 1
    return exponential(n - 1) + exponential(n - 1)
```

Each call makes two more calls, one level down. That is 2 calls, then 4, then 8. The count is 2^n . At $n = 40$ that is a trillion calls, and the function will not finish today.

Here is the complete program. It runs each shape at four sizes and — this is the part that matters — prints the **ratio** between one size and the next.

```

"""Day 3 – measure the shape by doubling the input and reading the ratio."""

def constant(n: int) → int:
    """O(1): one step, whatever n is."""
    return 1

def logarithmic(n: int) → int:
    """O(log n): halve until nothing is left."""
    steps = 0
    size = n
    while size > 1:
        size /= 2
        steps += 1
    return steps

def linear(n: int) → int:
    """O(n): one pass."""
    steps = 0
    for _i in range(n):
        steps += 1
    return steps

def linearithmic(n: int) → int:
    """O(n log n): one full pass for each halving level."""
    steps = 0
    size = n
    while size > 1:
        for _i in range(n):
            steps += 1
        size /= 2
    return steps

def quadratic(n: int) → int:
    """O(n^2): every pair."""
    steps = 0
    for i in range(n):
        for _j in range(i + 1, n):
            steps += 1
    return steps

SHAPES = [
    ("O(1)", constant),
    ("O(log n)", logarithmic),
    ("O(n)", linear),
    ("O(n log n)", linearithmic),
    ("O(n^2)", quadratic),
]

if __name__ == "__main__":
    sizes = (250, 500, 1000, 2000)
    header = "".join(f"{'n=' + str(s):>14}" for s in sizes)

```

```

print(f"{'shape':<14}{header}{' ratio when n doubles'}")
print("-" * (14 + 14 * len(sizes) + 24))
for name, fn in SHAPES:
    counts = [fn(s) for s in sizes]
    cells = "".join(f"{c:>14,}" for c in counts)
    ratios = [counts[i + 1] / counts[i] for i in range(len(counts) - 1)]
    average = sum(ratios) / len(ratios)
    print(f"{'name':<14}{cells}{average:>18.2f} x")

```

This is exactly what it printed:

shape	n=250	n=500	n=1000	n=2000	ratio when n doubles
0(1)	1	1	1	1	1.00 x
0(log n)	7	8	9	10	1.13 x
0(n)	250	500	1,000	2,000	2.00 x
0(n log n)	1,750	4,000	9,000	20,000	2.25 x
0(n ²)	31,125	124,750	499,500	1,999,000	4.00 x

The last column is the whole lesson. You do not need to recognise the code. Double the input, look at what happens to the work, and the shape names itself:

- stays the same → 0(1)
- goes up by a small fixed amount → 0(log n)
- doubles → 0(n)
- a bit more than doubles → 0(n log n)
- quadruples → 0(n²)
- squares → 0(2ⁿ)

That is a test you can run on your own code in ten seconds, and it is how you check an answer you are not sure about.

6 What it costs

Big-O is itself a cost statement, so this section does the arithmetic that turns the notation into a decision.

The rule of thumb that decides interview answers. A modern machine does very roughly **10⁸ simple operations per second** in Python — a hundred million. That single number, combined with the constraint printed at the bottom of the problem, tells you what you are allowed to write.

CONSTRAINT ON N	STEPS YOU CAN AFFORD	SHAPES THAT FIT
$n \leq 10$	anything	even $O(n!)$
$n \leq 25$	$\sim 3 \times 10^7$	$O(2^n)$
$n \leq 5,000$	2.5×10^7	$O(n^2)$
$n \leq 10^5$	1.7×10^6 for $n \log n$	$O(n \log n)$, $O(n)$
$n \leq 10^6$	10^6	$O(n)$, $O(\log n)$
$n \leq 10^9$	you cannot even read the input	$O(\log n)$, $O(1)$

Work through the row that matters most. The constraint says $n \leq 100,000$, which is by far the most common one on LeetCode. An $O(n^2)$ solution does:

```
100,000 x 100,000 = 10,000,000,000 operations
10,000,000,000 / 100,000,000 per second = 100 seconds
```

The time limit is usually one or two seconds. So $O(n^2)$ fails by a factor of about fifty, and it fails *no matter how clean the code is*. Now the same input with $O(n \log n)$:

```
log2(100,000) is about 17
100,000 x 17 = 1,700,000 operations
1,700,000 / 100,000,000 = 0.017 seconds
```

Seventeen milliseconds against a hundred seconds. **The constraint told you which shape to write before you had even read the problem statement properly.** That is a habit worth building now, and [day 004](#) drills it.

Constants are dropped, and constants are still real. $O(n)$ with a heavy body can be slower than $O(n \log n)$ with a light one, at the input sizes you actually have. Big-O does not lie about this; it simply is not answering that question. It answers “what happens as n grows”, and it stops being the right tool when n is small and fixed. An interviewer who says “in practice the constant matters here” is not correcting you. They are agreeing with you and adding to it.

Space has a Big-O too, and it is a separate answer. [quadratic](#) above keeps one integer no matter how long the list is, so its extra space is $O(1)$ even though its time is $O(n^2)$. Never let one number stand in for both. [Day 007](#) does space properly.

7 The traps

Trap one: the loop that hides another loop

Here is a function that removes duplicates. It has one loop. Almost everyone calls it $O(n)$.

```
def dedupe(items: list[int]) → list[int]:
    seen = []
    out = []
    for x in items:
        if x not in seen:      # ← this line is a loop
            seen.append(x)
            out.append(x)
    return out
```

`x not in seen` is not one step. For a **list**, Python has to walk `seen` from the start until it finds `x` or runs out. That costs as much as `seen` is long — and `seen` grows as the outer loop goes on.

So the outer loop runs n times, and the hidden inner walk costs 0, then 1, then 2, and on up. That is the staircase from day 002 again: $n \times (n - 1) / 2$, which is $O(n^2)$.

Measure it and the deception is obvious:

```
n = 20,000 all distinct
dedupe with a list : 3.58 s
dedupe with a set  : 0.0056 s
ratio               : 636 x
```

The fix is one word:

```
def dedupe(items: list[int]) → list[int]:
    seen = set()          # a set, not a list
    out = []
    for x in items:
        if x not in seen:  # O(1) for a set
            seen.add(x)
            out.append(x)
    return out
```

`x in some_set` is $O(1)$. `x in some_list` is $O(n)$. They are spelled identically, they read identically out loud, and one of them is six hundred times slower here — and the gap widens with n , because one is linear and one is quadratic. [Day 006](#) is entirely about this.

How to catch it every time: when you state a complexity, put a finger on every line inside the loop and ask “is this really one step?”. `in`, `.index()`, `.count()`, `min()`, `max()`, `sum()`, `sorted()`, slicing and string concatenation are all loops wearing a short name.

Trap two: what a quadratic submission actually looks like

Nothing crashes. There is no error message telling you that you chose the wrong shape. This is what LeetCode returns instead:

Time Limit Exceeded

Last executed input:

```
nums = [7,4,9,2,8,1,...] (100000 elements)
```

And this is what it looks like when you run it yourself and give up waiting:

```

^CTraceback (most recent call last):
  File "slow.py", line 12, in <module>
    print(count_pairs(list(range(200000))))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "slow.py", line 6, in count_pairs
    for j in range(i + 1, len(items)):
KeyboardInterrupt

```

Read that traceback carefully, because it is a useful thing rather than a failure. The `^C` is you pressing Ctrl+C. Python then shows the exact line it was standing on when you interrupted it, and that line is the inner loop header. **The interpreter has just pointed at your quadratic.** When something is taking too long and you do not know why, interrupt it and read where it stops. That is the cheapest profiler there is.

The habit that prevents both traps: before you submit, read the constraint, multiply it out, and compare against 10^8 . If the answer is bigger, you already know the verdict, and you have not spent a submission finding out.

8 In the interview

How it gets asked

- “What’s the time complexity of your solution?” — after every single solution, in every round. This is not optional and it is not conversational.
- “And the space complexity?” — always the second half. Answer both without being asked twice.

- “Can you do better?” — means “your answer was $O(n^2)$ and there is an $O(n \log n)$ or an $O(n)$ ”. It is a hint, not a criticism.
- “What if n were a million?” — asking you to substitute into your own formula and reach a conclusion out loud.

What to say out loud, in the first ninety seconds

Do not open with the letter O. Open with the count, and let the notation be the summary.

1. **Name the loops.** “There’s an outer loop over the array and an inner loop from i plus one.”
2. **Give the exact count.** “So the body runs n minus one times, then n minus two, and so on — that’s n times n minus one, over two.”
3. **Then reduce it.** “Dropping the constant and the lower term, that’s $O(n \text{ squared})$.”
4. **Check every line inside the loop.** “Everything in the body is $O(1)$ — comparisons and an integer add, no slicing, no `in` on a list.” This sentence is the one that shows you are not reciting.
5. **Give the space separately.** “Space is $O(1)$ extra. I only keep a counter. The input itself is $O(n)$, but I’m not counting that as extra.”
6. **Put a number on it.** “With n up to a hundred thousand, that’s ten to the ten operations, which won’t pass. So I’d want to get this to $O(n)$ with a hash set.”

Step 6 is what separates a candidate who knows the notation from one who uses it. You have just answered the interviewer’s next question before they asked it.

The follow-ups

“**Why do you drop the constant?**” Because it does not change what happens when the input grows. $n^2/2$ and n^2 both go up four times when you double n . A constant shifts the curve; it does not bend it. It is also why a colleague working twice as fast does not rescue a quadratic job — halving thirteen hours still leaves six and a half. Constants matter in production, where you tune them, and they do not matter for the question Big-O is answering.

“**Two loops, one after the other — is that $O(n^2)$?**” No. Sequential loops add, so it is $n + n = 2n$, which is $O(n)$. Only nesting multiplies. You could put ten loops one after another and it would still be linear.

“**Is $O(1)$ always faster than $O(n)$?**” No, and this is worth answering precisely. $O(1)$ means the cost does not depend on n , not that the cost is small. A constant-time operation that takes a millisecond is slower than a linear pass over five items. Big-O tells you how something scales, and for small fixed inputs it can be the wrong question entirely.

“You said the average case is $O(n)$. What’s the worst case?” Whenever the two differ, give both, and say which one you are quoting. Quicksort is $O(n \log n)$ on average and $O(n^2)$ in the worst case. Hash lookups are $O(1)$ on average and $O(n)$ if every key collides. Interviewers ask this to find out whether you know that both numbers exist.

A model answer

The interviewer has just watched the candidate write a nested loop that looks for a pair of numbers summing to a target, and asks for the complexity.

“The outer loop runs n times. For each of those, the inner loop runs from i plus one to n , so it’s n minus one iterations on the first pass, n minus two on the second, down to zero on the last. Adding those up gives n times n minus one, over two.

Dropping the one-half and the minus one, that’s $O(n^2)$ time. I want to check the body too — inside I’ve got an array index and an integer comparison, both $O(1)$, so there’s nothing hidden in there.

Space is $O(1)$ extra. I’m not allocating anything that grows with n . The input array is $O(n)$, but that’s given to me rather than something I create.

Now, the constraint says n can be a hundred thousand. n^2 is ten to the tenth, which is about a hundred seconds in Python against a one-second limit, so this will time out. I can get it to $O(n)$ with a hash set — one pass, and for each element I check whether target minus that element is already in the set. Lookups are $O(1)$ on average, so that’s $O(n)$ time and $O(n)$ extra space.

That trade is worth naming: I’m spending $O(n)$ memory to remove a factor of n from the time. If memory were the constraint instead, and the array were sorted, I’d use two pointers from both ends and get $O(n)$ time with $O(1)$ space.”

The last paragraph is what gets remembered. The candidate did not just improve the answer. They named the resource they spent to do it, and gave the condition under which they would choose differently.

9 Recall card

1. Big-O is the **shape of the growth**. Get the exact count first, then **drop the constants and keep the biggest term**. $n^2/2 - n/2$ is $O(n^2)$.
2. The test that never fails: **double the input**. Same $\rightarrow O(1)$. Plus one step $\rightarrow O(\log n)$. Doubles $\rightarrow O(n)$. Quadruples $\rightarrow O(n^2)$.

3. **Nested loops multiply, sequential loops add.** Two loops in a row is $O(n)$, not $O(n^2)$.
4. **Check every line inside the loop.** `x in a_list`, `.index()`, slicing and `sorted()` are loops in disguise. `x in a_set` is not.
5. Roughly **10^8 operations per second**. So $n \leq 10^5$ rules out $O(n^2)$, and the constraint printed in the problem tells you the shape before you start writing.

1 What this is, and why they ask it

An **IP address** is the number that identifies one machine on the internet, the way a street address identifies one building. A **port** is a second number that says which program on that machine you want, the way a flat number says which door inside the building. **DNS** — the Domain Name System — is the lookup service that turns a name people can remember, like `google.com`, into the number the network actually needs.

On day 001 you followed a request across the internet and the DNS step went past quickly. Today you stop on it, because it is the step interviewers dig into most.

They ask because DNS is where three things meet: naming, caching and failure. A candidate who can walk the resolution chain, name what is cached at each hop, and then say what a TTL of sixty seconds costs you, has demonstrated more than trivia. They have shown they understand that **every layer of the internet is somebody's lookup table**, and that every lookup table has a staleness problem. That is the actual subject of this lesson.

2 The story

Ravi is going to a friend's flat for the first time. Anjali has moved into one of those large housing societies off the ring road — six towers, all cream-coloured, all identical, built at the same time by the same builder.

He knows two things. He knows her name, and he knows the name of the society. He does not know which tower, and he does not know which flat.

At the gate there is a guard in a small cabin with a tablet mounted on a stand. Ravi says he is here to see Anjali Sharma. The guard types the surname in, gets four Sharmas, and reads out the towers. Ravi says the flat is a new one, taken in April. The guard scrolls, finds it, and says: Tower C, seventh floor, flat 704. It takes about twenty seconds.

Ravi walks in and now the walking is easy, because Tower C is signposted and the lift has buttons. Getting to the seventh floor takes no thinking at all. **All the difficulty was at the gate.** Once he had the number, the rest was mechanical.

He knocks at 704. Anjali's father opens the door. Ravi asks for Anjali, and her father calls her. This matters more than it looks: reaching the right door is not the same as reaching the right person. There are four people living behind that one door, and the

door number alone does not say which of them you want.

On the way out he does the sensible thing. He saves it in his phone: *Anjali — Tower C, 704*.

Three weeks later he visits again, and this time he walks straight past the cabin without stopping. He does not need the guard. He has it saved.

But in October he goes again, and knocks, and a woman he has never seen opens the door. Anjali's family moved out in August, one tower over, to a bigger flat. What Ravi had saved was correct in April, correct in May, and quietly wrong from August onwards. Nobody came and told him. He had no way of knowing it had gone stale until he was standing at the wrong door.

So he walks back to the gate and asks the guard again.

3 The idea in plain English

Everything in that story maps onto one part of how a name becomes a machine. Take it piece by piece.

The society is the internet, and the tower-and-flat number is an IP address. An **IP address** is a number that identifies one machine on a network. There are two kinds you will meet. **IPv4** looks like `142.250.183.206` — four numbers, each from 0 to 255, joined by dots. **IPv6** looks like `2404:6800:4009:82f::200e` — much longer, written in hex, and it exists because IPv4 ran out of numbers. Every request you make goes to an IP address, always. Names are a convenience laid over the top.

“Anjali Sharma” is the domain name. A **domain name** is a human-readable label for a machine or a group of machines: `google.com`, `leetcode.com`, `api.github.com`. Names are for people. Machines do not use them.

The guard with the tablet is DNS. The **Domain Name System** is a worldwide lookup service whose only job is to answer “what is the IP address for this name?”. Your machine cannot connect to `google.com` any more than Ravi could walk to “Anjali Sharma”. Something has to convert the one into the other first, and DNS is that something.

The father opening the door is the port. Reaching flat 704 gets you to the right machine. It does not say which program you want. A single machine can be running a web server, a mail server and a database all at once, and a **port** — a number from 0 to 65535 — is how they are told apart. Some are conventional and worth memorising:

PORT	WHAT LISTENS THERE
80	HTTP, plain web traffic
443	HTTPS, encrypted web traffic
22	SSH, remote login
5432	PostgreSQL
3306	MySQL
6379	Redis
27017	MongoDB

So a full destination is not one number. It is **IP address plus port**, written `142.250.183.206:443`. That pair is called a **socket address**, and it is what a connection actually points at. When you type `https://google.com` and no port, the browser silently adds `:443`, because `https` means 443 by convention.

Saving it in his phone is caching. A **cache** is a local copy of an answer, kept so that you do not have to ask again. Ravi cached the flat number and skipped the gate. Your machine caches DNS answers and skips the lookup. This is why the second visit to a website starts faster than the first.

The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a **TTL** — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again. A TTL of 300 means “believe this for five minutes”. Nobody pushes an update out to everyone holding an old answer. The old answer simply expires. That single design decision explains almost every DNS question an interviewer will ask you.

Twenty seconds at the gate, then a trivial walk. That is the honest shape of it. DNS resolution can cost tens of milliseconds on a cold lookup, and then the connection itself is fast. Which is why a slow DNS setup makes an otherwise quick site feel sluggish on the first visit.

4 The picture

Here is what actually happens when you type `google.com`, drawn as the chain of questions.

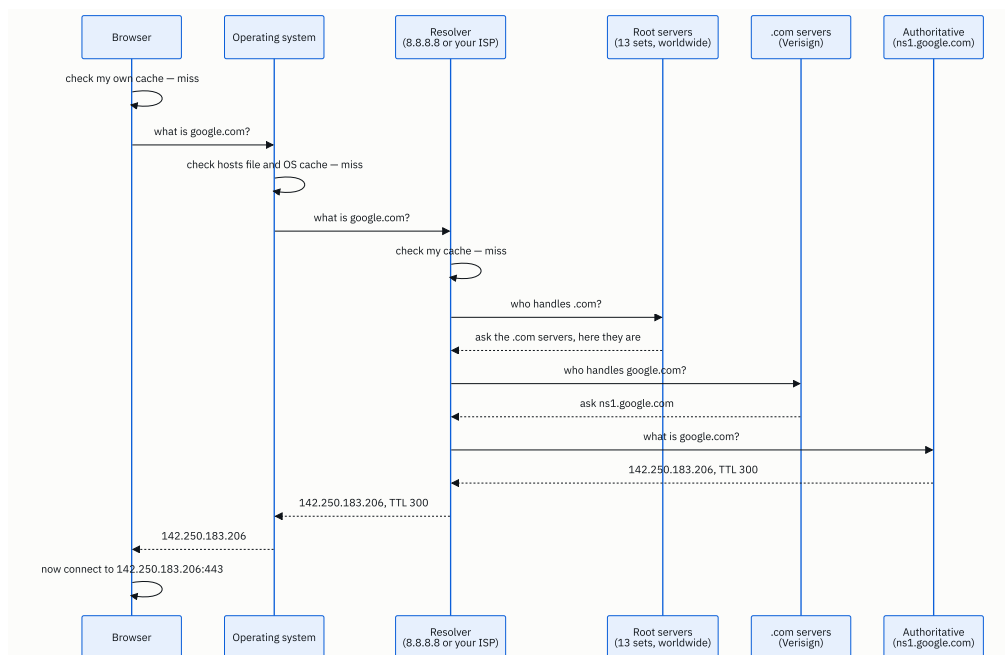


FIGURE 3.1

What to notice: there are **four** caches on that path — the browser’s, the operating system’s, the resolver’s, and often one at your router as well. A cold lookup walks the whole chain. A warm one stops at the first line. In real traffic the vast majority stop early, which is the only reason the thirteen root server sets can survive the entire internet asking them questions.

Now the address itself, laid out so that each part has a name:

```

https://api.github.com:443/repos/python/cpython
|           |           |           |
|           |           |           | +-- path: what you want from that program
|           |           |           | +----- port: WHICH PROGRAM on the machine
|           |           |           | +----- name: resolved by DNS to an IP address
+----- scheme: implies port 443 if none is given
  
```

after DNS resolves, the connection is really to:

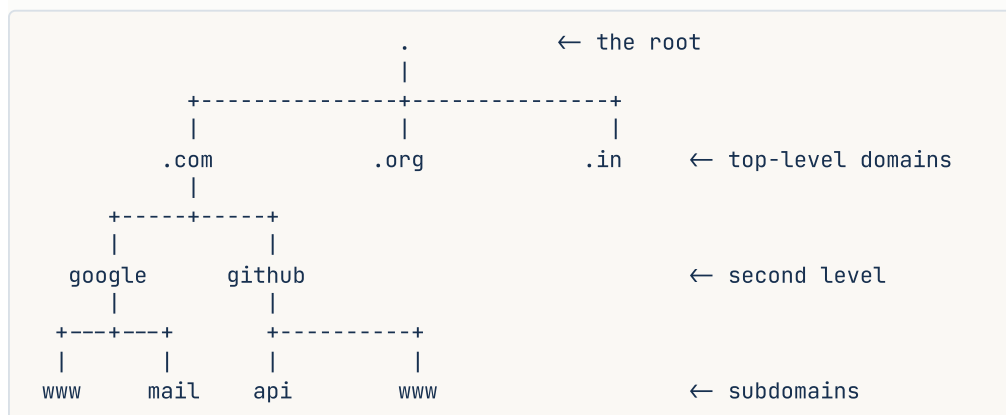
```

140.82.121.6 : 443
|           |
|           | +--- one of 65,536 doors on that machine
+----- one machine on the internet
  
```

What to notice: the name disappears once DNS has done its work. The connection is made to a number. The name is never sent over the network as an address — though it is sent inside the request, as a header, so that one machine can host many sites. That detail

is the whole reason shared hosting works, and it comes back on [day 005](#).

And here is the tree that DNS is actually walking, which is why the chain goes right to left:



What to notice: `api.github.com` is read **backwards** by DNS — root first, then `.com`, then `github`, then `api`. That is why the trailing dot in `google.com.` is technically correct: it names the root.

5 How it actually works

The four caches, in order

1. **The browser's own cache.** Chrome keeps DNS answers for about a minute. You can see them at `chrome://net-internals/#dns`.
2. **The operating system's cache, plus the hosts file** — a plain text file that overrides DNS entirely. On Linux and macOS it is `/etc/hosts`; on Windows it is `C:\Windows\System32\drivers\etc\hosts`. Putting `127.0.0.1 myapp.local` in it is how developers point a real-looking name at their own machine.
3. **The resolver**, also called the recursive resolver. This is the one that does the actual work of walking the chain. Yours is either your ISP's, or a public one you chose: **Cloudflare's** `1.1.1.1`, **Google's** `8.8.8.8`, or **Quad9's** `9.9.9.9`.
4. **The authoritative name servers**, which hold the real answer. **Amazon Route 53**, **Cloudflare DNS**, **NS1** and **Google Cloud DNS** are the big managed ones; **BIND** is the classic software you run yourself.

The word **recursive** is worth pinning down, because interviewers use it. Your machine sends one question and gets one final answer — that is a **recursive** query. The resolver then does the walking, asking root, then TLD, then authoritative, each of which answers

with a referral rather than the answer — those are **iterative** queries. Your machine is lazy on purpose. The resolver does the legwork.

What is actually stored

DNS holds **records**, and each record has a type. These are the ones that come up:

TYPE	HOLDS	EXAMPLE
A	an IPv4 address	google.com → 142.250.183.206
AAAA	an IPv6 address	google.com → 2404:6800:4009:82f::200e
CNAME	another name to look up instead	www.example.com → example.com
MX	where to deliver mail	gmail-smtp-in.l.google.com, with a priority
NS	which name servers are authoritative	ns1.google.com
TXT	arbitrary text	domain ownership proofs, SPF records

You can see all of this yourself. `dig google.com` on Linux or macOS, `nslookup google.com` on Windows, and the answer comes back with the TTL in it.

One name, many machines

A single name can hold several `A` records, and the resolver hands them out in rotation. That is **DNS round-robin**, the oldest and crudest form of load balancing. Big providers do something better: **anycast**, where the *same* IP address is announced from data centres on several continents at once, and the network routes you to the nearest one. `1.1.1.1` is anycast, which is why it answers in under ten milliseconds almost everywhere on earth.

CDNs — content delivery networks like **Cloudflare**, **Akamai** and **CloudFront** — use DNS as their steering wheel. When you look up a CDN-hosted name, the authoritative server looks at where the question came from and returns the IP address of the nearest edge location. The name is the same everywhere. The answer is not.

Private addresses and NAT

Not every IP address is reachable from the internet. Three ranges are reserved for private networks:

```
10.0.0.0      - 10.255.255.255
172.16.0.0    - 172.31.255.255
192.168.0.0   - 192.168.255.255
```

Your laptop almost certainly has one of these. Your router holds one real public address and performs **NAT** — network address translation — rewriting the addresses on the way out so that every device in your house shares that one public address. It keeps a table of

which internal machine each outbound connection belongs to, so that replies get back to the right one. This is also why an incoming connection to your laptop does not work without port forwarding: there is nothing in the table to match it against.

And `127.0.0.1`, called **localhost** or the loopback address, means “this same machine”. A request to it never touches a network cable.

When it fails

If DNS is down, everything looks down. The machines are fine, the sites are running, and nobody can reach them because nobody can turn names into numbers. This has happened at scale more than once — a misconfigured DNS change took Facebook off the internet for six hours in October 2021, and it also locked their engineers out of the tools they needed to fix it.

The failure mode you will meet more often is **stale cache**: you change a record, and some users move to the new address immediately while others keep hitting the old one until their TTL expires. There is no way to force them. This is Ravi at the wrong door.

6 The numbers

How many addresses there are. IPv4 uses 32 bits:

$$2^{32} = 4,294,967,296 \text{ addresses}$$

Around 4.3 billion, for a world with more than 5 billion internet users and many devices each. They ran out, which is why NAT is everywhere and why IPv6 exists:

$$2^{128} = 340,282,366,920,938,463,463,374,607,431,768,211,456$$

That is about 3.4×10^{38} — enough to give every grain of sand on earth its own address several times over.

How many ports. Port numbers are 16 bits:

$$2^{16} = 65,536 \text{ ports (0 to 65535)}$$

Ports 0–1023 are **well-known** and need administrator rights to bind on Unix. That is why a development app defaults to 8000 or 3000 rather than 80.

What a DNS lookup costs. A cold lookup that walks the whole chain:

browser + OS cache miss	~0 ms
resolver, over the network	~15 ms
root referral	~20 ms
TLD referral	~25 ms
authoritative answer	~30 ms

total for a cold lookup	~90 ms

Ninety milliseconds before a single byte of the actual request has been sent. A warm lookup from the browser cache is effectively **0 ms**. That gap is why the first visit to a site feels different from the second, and why page-load work often begins with reducing the number of distinct names a page has to resolve.

What TTL costs you. Suppose a service has 1,000,000 users, each making 20 requests a day, all to one name.

With a **TTL of 86,400 seconds** (24 hours), each user resolves once a day:

$$1,000,000 \text{ lookups per day} / 86,400 \text{ s} = 11.6 \text{ DNS queries per second}$$

With a **TTL of 60 seconds**, each user resolves at the start of each minute they are active. Say each user is active for 10 minutes a day:

$$1,000,000 \text{ users} \times 10 \text{ lookups} = 10,000,000 \text{ lookups per day}$$

$$10,000,000 / 86,400 = 116 \text{ DNS queries per second}$$

Ten times the DNS traffic. In exchange, when you change the address, everybody has moved within a minute rather than within a day. **That is the entire TTL trade, and it is arithmetic, not opinion.**

What a slow failover costs. If you rely on DNS to move traffic off a dead machine, with a TTL of 300 seconds:

$$\text{worst case for a user holding a fresh answer} = 300 \text{ s} = 5 \text{ minutes of errors}$$

Five minutes is an eternity for a payment system. That is why real failover is done by a load balancer at a fixed IP address, and DNS handles only the slow, planned moves.

Response sizes. A DNS query is about 50 bytes and an answer about 100–500 bytes, over UDP, in a single packet each way. That smallness is deliberate — it is what makes a lookup one round trip instead of a conversation.

7 The trade-offs

A long TTL is cheap and slow to change. Set 24 hours and you get almost no DNS traffic, very fast lookups for repeat visitors, and a system that shrugs off a resolver outage. You also get a full day during which some fraction of your users are pointed at a machine you have retired. You cannot recall an answer once it has been handed out.

A short TTL is expensive and nimble. Sixty seconds means you can move traffic in a minute, which is what you want the day before a migration. It also multiplies your DNS query volume, adds a lookup to more requests, and — the part people forget — makes DNS a hard dependency on your critical path. If your DNS provider has a bad ten minutes, a long TTL hides it and a short TTL exposes it. The usual practice is a long TTL normally, dropped to sixty seconds a day before a planned change, and raised again afterwards.

DNS-based load balancing is free and blunt. Returning several `A` records costs nothing and needs no infrastructure. But DNS cannot see health, so it will happily hand out the address of a machine that died two minutes ago; it cannot see load, so it cannot send a request to the least busy machine; and caching means you have no control over the actual distribution. A real **load balancer** — `nginx`, `HAProxy`, or `AWS ALB` — sits at one address, checks health continuously, and decides per request. You usually want both: DNS to get users to the right *region*, a load balancer to pick the right *machine*.

CNAME versus A record. A `CNAME` points one name at another, so when the target's IP changes you change nothing. That is why you point `www.yoursite.com` at a CDN's name rather than at its address. The cost is a second lookup, and the rule that a `CNAME` cannot exist at the root of a domain — `example.com` itself cannot be a `CNAME`, only `www.example.com` can. Providers work around this with non-standard extensions such as Route 53's alias records.

Public resolver versus your ISP's. Cloudflare's `1.1.1.1` is usually faster, does not sell your query history, and supports encrypted DNS. Your ISP's resolver may be closer, and it may also be doing things you would not choose — logging, injecting ads into failed lookups, or blocking sites. The reason `1.1.1.1` and `8.8.8.8` exist at all is that a resolver sees every name you visit, which is a remarkably complete record of your life.

I would not lean on DNS if... I needed sub-second failover, per-request routing decisions, or any guarantee about *which* machine a given user reaches. DNS is a caching system, and caching systems trade freshness for cheapness. Put anything that has to be immediate behind a fixed address and a load balancer instead.

8 In the interview

How it gets asked

- “How does the browser find the server for google.com?” — the direct version, usually as a follow-up to the day-001 “what happens when you type google.com” question.
- “What is a port, and why do we need one?” — the short version, which is really checking that you know an IP address alone is not enough.
- “You’ve changed your server’s IP address. How long until all users hit the new one?” — the good version. It is a TTL question wearing a scenario.
- “Can two services run on the same machine on the same port?” — a precision check.

What to say out loud, in the first ninety seconds

1. **Say what has to happen and why.** “The browser can’t connect to a name — it needs an IP address, so the first step is a DNS lookup.”
2. **Walk the caches in order.** “It checks its own cache, then the OS cache and the hosts file, then it asks a resolver — the ISP’s, or 8.8.8.8.”
3. **Walk the chain, right to left.** “On a miss, the resolver asks a root server, which refers it to the .com servers, which refer it to Google’s authoritative name servers, which give the actual A record.”
4. **Mention the TTL, unprompted.** “The answer comes back with a TTL, so every cache on the way holds it for that long.”
5. **Then the port.** “Now it has 142.250.183.206, and because the scheme is https it connects to port 443. The IP picks the machine; the port picks the program.”
6. **Offer the next layer.** “From there it’s the TCP handshake and then TLS — happy to go into either.”

Step 4 is the one that changes the conversation. It is the difference between reciting a chain and understanding what the chain costs.

The follow-ups

“What’s the difference between an IP address and a port?” The IP address identifies the machine; the port identifies the program on it. One machine with one address can run a web server on 443, a database on 5432 and SSH on 22 at the same time, and the port is the only thing keeping those apart. A connection is really defined by four things — source IP, source port, destination IP, destination port — and that four-part combination is what lets you have many simultaneous connections to the same site.

“You change your server’s IP. How long until everyone moves?” Up to one TTL, and there is no way to hurry it. If the TTL was 24 hours, some users will keep hitting the old address for the rest of the day, because nobody notifies a cache. The professional

answer is to plan for it: drop the TTL to sixty seconds a day or two before the change, make the change, keep the old machine answering during the transition, then raise the TTL again once traffic has drained.

“Can two programs listen on the same port on one machine?” Not on the same IP address and protocol — the second one gets `OSError: [Errno 98] Address already in use`. They can if they bind different addresses on a machine with several, and TCP port 8000 and UDP port 8000 are genuinely separate. This is also why the standard trick is one web server on 443 that routes by hostname to many applications behind it, rather than many programs fighting over the port.

“Is DNS a single point of failure?” For your domain, effectively yes, which is why the design has redundancy built in. Domains list at least two `NS` records, the root is thirteen anycast sets rather than thirteen machines, and serious operators run authoritative DNS with two independent providers. When DNS does fail, everything looks down even though nothing is, which is what makes it so memorable when it happens.

A model answer

“The browser can’t connect to a name, only to an IP address, so the first thing that happens is a DNS lookup.

It checks caches in order — its own, then the operating system’s, which also means the hosts file. If nothing has it, the request goes to a recursive resolver, either the ISP’s or something like 1.1.1.1 or 8.8.8.8.

If the resolver doesn’t have it cached either, it walks the tree from the top. It asks a root server, which doesn’t know google.com but does know who runs .com, and refers it to Verisign’s TLD servers. Those refer it to Google’s authoritative name servers. Those hold the actual A record and return 142.250.183.206, along with a TTL — say 300 seconds.

Every cache on the way back holds that answer for the TTL. So the vast majority of real lookups never reach the root at all, which is the only reason this design scales.

Now the browser has an address, and it needs a port. The scheme is https, so that’s 443 by convention. It opens a TCP connection to 142.250.183.206 on port 443, does the TLS handshake, and sends the HTTP request. The IP address chose the machine; the port chose which program on that machine answers.

The part I’d flag in a design discussion is that TTL. It’s the only control you have over how fast a change propagates, and it’s a straight trade — a long TTL means fewer lookups and a faster site but a slow rollout, a short TTL means you can move traffic in a minute but DNS is now on your critical path. If I needed genuinely fast failover I wouldn’t use DNS for it at all; I’d put a load balancer at a fixed address and let it do health checks.”

That covers the chain, the caching, the port, and finishes on a trade-off — which is what turns a recital into a design answer.

9 Recall card

1. **IP address picks the machine. Port picks the program.** A destination is the pair: 142.250.183.206:443. HTTP is 80, HTTPS is 443, PostgreSQL is 5432, Redis is 6379.
2. **DNS turns a name into an address.** The chain is browser cache → OS cache and hosts file → resolver → root → TLD → authoritative.
3. **Four caches sit on that path**, and almost every real lookup stops at one of the first two. That is why the root servers survive.

4. **TTL is the only control you have.** Long TTL: cheap, fast, slow to change. Short TTL: expensive, nimble, and DNS becomes a hard dependency. Nobody can recall an answer already handed out.
5. **IPv4 is 2^{32} addresses and ran out**, which is why NAT and private ranges (`10.x` , `172.16-31.x` , `192.168.x`) exist. `127.0.0.1` is this machine.

DSA topic: Big-O in plain English **System design topic:** IP addresses, ports, and DNS

Code these, in this order

Four problems, easiest first. Each one has an obvious quadratic solution and a better one, which is the point.

The habit gains a third step today. For each problem:

1. **Read the constraint first.** Find the largest `n` the problem allows, multiply it out, and compare against 10^8 . Decide what shape you are allowed to write **before** you think about the solution.
2. Write the obvious solution, whatever it is, and state its Big-O out loud.
3. If the shape does not fit the constraint, improve it, and say what resource you spent to do it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Contains Duplicate	LeetCode 217 (Easy)	The exact trap from §7. The nested-loop version is $O(n^2)$, the set version is $O(n)$, and the code looks almost identical.
2	Two Sum	LeetCode 1 (Easy)	Trading space for time. $O(n^2)$ with two loops, $O(n)$ with a dictionary and $O(n)$ extra space. Say the trade out loud.
3	Majority Element	LeetCode 169 (Easy)	Three different shapes for one problem — counting each element is $O(n^2)$, sorting is $O(n \log n)$, and Boyer-Moore is $O(n)$ with $O(1)$ space.
4	Maximum Subarray	LeetCode 53 (Medium)	The brute force is $O(n^3)$ if you are not careful, $O(n^2)$ if you are, and $O(n)$ with Kadane. Notice how easy the cubic version is to write by accident.

On problem 1, do this properly

- Write the nested-loop version first. Run it on 20,000 distinct numbers and time it.
- Write the `set` version. Time it on the same input.
- Look at the ratio. It should be in the thousands, and the two functions differ by one word.
- Now state both complexities out loud, and say which line in the first version is the hidden loop.

The measurement drill

Take any one solution you wrote today and run it at $n = 250, 500, 1,000$ and $2,000$, counting steps rather than timing. Look at the ratio between each size and the next. Name the shape from the ratio alone, without looking at the code. If your ratio and your Big-O disagree, your Big-O is wrong.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is the time complexity of your solution?* Use problem 2 above. Do not open with the letter O — open with the count, reduce it, then check every line inside the loop, then give the space separately, then put a number on it against the constraint.
 2. *How does the browser find the server for google.com?* Walk the four caches and the three hops in order, and get to the TTL without being asked for it.
 3. *You change your server's IP address. How long until every user reaches the new one?* One sentence on the answer, one on why nobody can hurry it, and one on what you would do the day before a planned migration.
-

Before you move on

- [] I read the constraint before choosing a solution shape, on all four problems.
- [] I can name three things that look like one step and are really loops.
- [] I can say what happens to $O(n)$, $O(n \log n)$ and $O(n^2)$ when the input doubles, from memory.
- [] I can walk the DNS chain out loud, in order, and say what is cached at each hop.
- [] I can say the difference between an IP address and a port in one sentence.

The growth curves you will meet again and again

DATA STRUCTURES AND ALGORITHMS

The growth curves you will meet again and again

You can rank $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$ and know what input size each survives.

SYSTEM DESIGN

TCP and UDP

You can say why a video call uses one and a bank transfer uses the other.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *n is 100000. Will an $O(n^2)$ solution pass?*
- *TCP or UDP for a live video call, and why?*

The growth curves you will meet again and again

1 What this is, and why they ask it

There are about seven growth curves in all of interview preparation, and you will meet the same seven for the next 176 days. Today you learn them in order, and — this is the part that actually decides interviews — you learn **the largest input each one survives**.

Yesterday you learnt to name a shape. Today you learn to use the name to make a decision before writing any code. The constraint at the bottom of the problem is not decoration. It is the interviewer telling you which shape they will accept.

They ask “ n is a hundred thousand, will $O(n^2)$ pass?” because the answer separates two kinds of candidate cleanly. One kind writes a solution, submits it, sees “Time Limit Exceeded”, and starts again. The other kind reads $1 \leq n \leq 10^5$, works out that a quadratic is ten billion operations, and never writes the quadratic at all. The second kind looks experienced, because that is exactly what experience looks like.

2 The story

Anil cooks the evening meal at home. There are four of them — himself, his wife, his mother and their daughter — and he has done it so often that it needs no thought at all. One pan, one gas ring, forty minutes from walking into the kitchen to putting it on the table.

In June his sister comes to stay and brings her husband and two children. Eight people now. Anil does exactly the same thing, twice over: two rounds in the same pan, about eighty minutes. It is longer and he is tired at the end, but nothing has actually changed. The method still works. It just takes proportionally more of it.

In November his daughter’s naming ceremony brings sixty people to the house.

Anil starts the way he always has, and by half past nine in the morning he knows it is not going to happen. Sixty people is fifteen rounds in that pan. Fifteen rounds is ten hours, and there are five. But that is not really the problem. The problem is that the first round would be stone cold by the time the fifteenth was ready, and there is no point serving cold food to sixty people.

So he does something different. He borrows the big aluminium vessel from the neighbours, the one they use for functions, and puts it on the outdoor gas burner in the yard. Everything goes in at once. His wife and his sister chop for an hour beforehand so that nothing is waiting. It works, and it works well, and it is a completely different way of cooking rather than the same way scaled up.

The following March, his nephew's wedding. Six hundred guests.

Anil does not even consider doing it himself, and he is not being modest. The big vessel holds enough for about eighty. Ten of those vessels means ten fires, and he does not have ten fires or twenty hands. He rings the caterer his neighbour used, and the caterer arrives at four in the morning with a truck, four industrial burners, vessels a man could sit in, and eleven people who each do one thing.

Three sizes. Three completely different methods. And the honest thing about it is that Anil's method for four was not a bad method. It was a good method, right up to the point where it was not.

3 The idea in plain English

Anil's kitchen contains the whole lesson. **Every method has a ceiling.** The pan was fine to eight and hopeless at sixty. The big vessel was fine to eighty and hopeless at six hundred. Nothing was wrong with either of them; they simply ran out.

Your solutions have ceilings too, and the ceiling depends only on the shape. Here are the seven curves in order, from best to worst.

The seven, in order

$O(1)$ — constant. The work does not depend on the input at all. Reading `items[5]`, adding two numbers, looking a key up in a dictionary. Anil's method for a family of four is not this — but “reach into the fridge and take out one bottle” is. The size of the fridge does not matter.

$O(\log n)$ — logarithmic. Each step throws away a fixed fraction of what is left, usually half. Binary search is the example you will meet a hundred times. This is very close to free: a million items takes twenty steps, a billion takes thirty.

$O(n)$ — linear. One pass. Double the input, double the work. This is Anil cooking twice over for eight people — the same method, proportionally more of it.

$O(n \log n)$ — linearithmic. A full pass, repeated once for each halving level. This is what sorting costs, and it is the ceiling of what you can do when a problem genuinely requires ordering. It is only a little worse than linear: at a million items it is twenty times $O(n)$, not a million times.

$O(n^2)$ — **quadratic**. Every item against every item. Two nested loops. This is the pan at sixty people — the method that quietly stops being a method.

$O(n^3)$ — **cubic**. Three nested loops. Rare, and almost always a sign you have missed something, except in a few matrix and interval problems.

$O(2^n)$ — **exponential**. For each item, two choices, so the count doubles with every item added. Generating every subset. This is the wedding: no amount of effort makes the old approach work, and you need a different idea entirely — which is usually **dynamic programming**, starting on [day 143](#).

$O(n!)$ — **factorial**. Every ordering of n items. Generating all permutations. At $n = 12$ that is 479 million; at $n = 20$ the universe is not old enough.

The table that decides interviews

This is the one to memorise. Assume roughly **10^8 simple operations per second**.

CONSTRAINT YOU SEE	LARGEST SHAPE THAT FITS	TYPICAL TECHNIQUE
$n \leq 10$	$O(n!)$	try every ordering
$n \leq 20$	$O(2^n)$	every subset, bitmask DP
$n \leq 500$	$O(n^3)$	three nested loops, interval DP
$n \leq 5,000$	$O(n^2)$	two nested loops, most DP
$n \leq 10^5$	$O(n \log n)$	sorting, heaps, binary search
$n \leq 10^6$	$O(n)$	one pass, hash maps, two pointers
$n \leq 10^9$	$O(\log n)$	binary search on the answer, maths
n is enormous	$O(1)$	a formula

Read it from the left. **The constraint names the shape, and the shape names the technique**. That chain is the most valuable single thing in this course's first month.

Why $\log n$ is so small

This is the part beginners take on trust and should not. $\log_2 n$ is the number of times you can halve n before you reach 1.

Start at 1,000,000. Halve it: 500,000. Again: 250,000. Keep going. You reach 1 after **twenty** halvings. Now start at 1,000,000,000 — a thousand times bigger. You reach 1 after **thirty**. A thousandfold increase in the input cost you ten extra steps.

That is why $O(\log n)$ and $O(1)$ sit next to each other in practice, and it is why binary search is worth the trouble of getting right.

Why $n \log n$ is so close to n

At $n = 1,000,000$, $\log_2 n$ is 20. So $n \log n$ is twenty million against n 's one million. Twenty times more work — noticeable, and nothing like the gap between n and n^2 , which at the same size is a factor of a million.

The practical consequence: **if you cannot find an $O(n)$ solution, sorting first is usually free.** An interviewer who says “you can sort if you need to” has just told you the target is $O(n \log n)$.

4 The picture

The ladder, with the actual step counts. This is the table worth being able to reproduce.

	n=10	n=100	n=1,000	n=100,000	n=1,000,000
$O(1)$	1	1	1	1	1
$O(\log n)$	3	7	10	17	20
$O(n)$	10	100	1,000	100,000	1,000,000
$O(n \log n)$	33	664	9,966	1,700,000	20,000,000
$O(n^2)$	100	10,000	1,000,000	10,000,000,000	10^{12}
$O(2^n)$	1,024	10^{30}	forever	forever	forever
$O(n!)$	3,628,800	10^{157}	forever	forever	forever

← everything fits → ← the line moves → ← only the top three →

What to notice: the bottom-left corner is fine. At $n = 10$ even $O(n!)$ finishes. The damage is all on the right-hand side, and it arrives suddenly. $O(n^2)$ is perfectly comfortable at $n = 1,000$ and completely dead at $n = 100,000$ — and those are only two steps apart in how a problem is worded.

Now Anil’s ceilings, drawn as the same idea:

people:	4	8	16	60	80	600	
one pan							
	[=====]			X	X	X	ceiling: about 8
			^				
			still fine, just longer				
big vessel	[=====]			X	X		ceiling: about 80
				^			
			a different method, not a bigger pan				
caterer	[=====]						ok ceiling: thousands

What to notice: each bar ends. It does not slope off gently — it stops. And the fix at each boundary is never “the same thing but harder”. It is a different method.

Here is the one picture that makes $\log n$ believable:

```

n = 1,000,000
|
+→ 500,000      1
   +→ 250,000    2
      +→ 125,000  3
         +→ 62,500  4
            +→ 31,250  5
               +→ 15,625  6
                  +→ 7,812  7
                     ...
                        +→ 1    20

```

twenty steps to go from a million to one.
 $n = 1,000,000,000$ takes thirty. A thousand times the input, ten more steps.

What to notice: the arrows get shorter and shorter but there are only twenty of them. Halving is brutal, in a good way.

5 The code, built step by step

The useful thing to build today is not a sorting routine. It is the calculator you run in your head at the start of every problem: **given this constraint, which shapes survive?**

Start with the shapes as formulas rather than as loops, because at $n = 10^9$ you cannot run the loop.

```

import math

def steps_for(shape: str, n: int) → float:
    """How many operations this shape does at size n."""
    match shape:
        case "O(1)":          return 1
        case "O(log n)":      return math.log2(n)
        case "O(n)":          return n
        case "O(n log n)":    return n * math.log2(n)
        case _:               raise ValueError(shape)

```

`match` is Python's version of a switch. `math.log2(n)` gives the number of halvings. This already covers the four cheap shapes; the expensive ones need care, because their values overflow into numbers no computer will finish.

```

        case "O(n^2)":       return n ** 2
        case "O(n^3)":       return n ** 3
        case "O(2^n)":       return 2.0 ** n if n < 1000 else math.inf
        case "O(n!)":        return math.inf if n > 170 else math.factorial(n)

```

`math.inf` is Python's infinity. Using it for the impossible cases is honest — the answer really is “more than you will ever do” — and it stops the program from trying to compute a number with three hundred digits.

Now turn a step count into a verdict.

```
OPS_PER_SECOND = 100_000_000      # 10^8, the working assumption
TIME_LIMIT_SECONDS = 1.0

def verdict(steps: float) → str:
    seconds = steps / OPS_PER_SECOND
    if seconds ≤ TIME_LIMIT_SECONDS * 0.1:
        return "comfortable"
    if seconds ≤ TIME_LIMIT_SECONDS:
        return "tight"
    return "TLE"
```

Three verdicts, not two. **Tight** matters: a solution that theoretically fits in one second will fail in Python, which is roughly ten to a hundred times slower than C++. If your arithmetic says “tight”, treat it as a fail and look for a better shape.

And the formatting, so the output is readable.

```
def human(steps: float) → str:
    if steps == math.inf:
        return "forever"
    if steps < 1e6:
        return f"{int(steps):,}"
    return f"{steps:.1e}"
```

`1e6` is scientific notation for 1,000,000. `f"{steps:.1e}"` prints large numbers as `1.0e+10` instead of a wall of digits.

Here is the complete program.

```

"""Day 4 - will it pass? The calculator you run before writing any code."""

import math

OPS_PER_SECOND = 100_000_000      # 10^8: the working assumption for one second
TIME_LIMIT_SECONDS = 1.0

SHAPES = ["O(1)", "O(log n)", "O(n)", "O(n log n)", "O(n^2)", "O(n^3)", "O(2^n)", "O(n!)"]

def steps_for(shape: str, n: int) → float:
    """How many operations this shape does at size n."""
    match shape:
        case "O(1)":      return 1
        case "O(log n)":  return math.log2(n)
        case "O(n)":      return n
        case "O(n log n)": return n * math.log2(n)
        case "O(n^2)":    return float(n) ** 2
        case "O(n^3)":    return float(n) ** 3
        case "O(2^n)":    return 2.0 ** n if n < 1000 else math.inf
        case "O(n!)":     return float(math.factorial(n)) if n ≤ 170 else math.inf
        case _:           raise ValueError(f"unknown shape: {shape}")

def verdict(steps: float) → str:
    """comfortable / tight / TLE, against a one-second limit."""
    seconds = steps / OPS_PER_SECOND
    if seconds ≤ TIME_LIMIT_SECONDS * 0.1:
        return "comfortable"
    if seconds ≤ TIME_LIMIT_SECONDS:
        return "tight"
    return "TLE"

def human(steps: float) → str:
    if steps == math.inf:
        return "forever"
    if steps < 1e6:
        return f"{int(steps):,}"
    return f"{steps:.1e}"

def largest_n(shape: str) → int:
    """The biggest input this shape still handles comfortably."""
    n = 1
    while n < 2_000_000_000 and verdict(steps_for(shape, n * 2)) ≠ "TLE":
        n *= 2
    return n

def report(n: int) → None:
    print(f"\nn = {n:,}")
    print(f"  {'shape':<12}{'operations':>14}  verdict")
    print(f"    " + "-" * 40)
    for shape in SHAPES:
        s = steps_for(shape, n)
        print(f"  {shape:<12}{human(s):>14}  {verdict(s)}")

if __name__ == "__main__":
    for size in (20, 5_000, 100_000, 1_000_000):
        report(size)

    print("\nlargest n each shape survives comfortably:")
    for shape in SHAPES:
        print(f"  {shape:<12}{largest_n(shape):>16},")

```


This is exactly what it printed:



```

n = 20
shape      operations  verdict
-----
0(1)              1  comfortable
0(log n)          4  comfortable
0(n)             20  comfortable
0(n log n)       86  comfortable
0(n^2)          400  comfortable
0(n^3)          8,000 comfortable
0(2^n)         1.0e+06 comfortable
0(n!)          2.4e+18 TLE

```

```

n = 5,000
shape      operations  verdict
-----
0(1)              1  comfortable
0(log n)          12  comfortable
0(n)            5,000 comfortable
0(n log n)      61,438 comfortable
0(n^2)         2.5e+07 tight
0(n^3)         1.2e+11 TLE
0(2^n)         forever TLE
0(n!)         forever TLE

```

```

n = 100,000
shape      operations  verdict
-----
0(1)              1  comfortable
0(log n)          16  comfortable
0(n)           100,000 comfortable
0(n log n)      1.7e+06 comfortable
0(n^2)         1.0e+10 TLE
0(n^3)         1.0e+15 TLE
0(2^n)         forever TLE
0(n!)         forever TLE

```

```

n = 1,000,000
shape      operations  verdict
-----
0(1)              1  comfortable
0(log n)          19  comfortable
0(n)           1.0e+06 comfortable
0(n log n)      2.0e+07 tight
0(n^2)         1.0e+12 TLE
0(n^3)         1.0e+18 TLE
0(2^n)         forever TLE
0(n!)         forever TLE

```

largest n each shape survives comfortably:

```

0(1)          2,147,483,648
0(log n)      2,147,483,648
0(n)          67,108,864
0(n log n)    4,194,304
0(n^2)         8,192

```

$O(n^3)$	256
$O(2^n)$	16
$O(n!)$	8

Read the last block. Those eight numbers are the whole lesson in a form you can carry into an interview. $O(n^2)$ runs out around eight thousand, $O(n^3)$ around 256, $O(2^n)$ around 16, and $O(n!)$ around 8. Note that the program's figures are stricter than the rule-of-thumb table above, because `largest_n` demands “comfortable” — a tenfold margin — rather than “just fits”. Both are useful: the table is what you quote, and these are what you would actually bet on. When a problem says $n \leq 12$ and you are stuck, that is not a coincidence — it is the setter telling you that trying every ordering is the intended solution.

6 What it costs

The arithmetic behind each row, done longhand once, so that you can redo it under pressure.

Will $O(n^2)$ pass at $n = 100,000$?

$100,000 \times 100,000$	= 10,000,000,000 operations
$10,000,000,000 / 10^8$	= 100 seconds
limit	= 1 second

No, by a factor of one hundred. And in Python, which is perhaps 30 times slower than C++ for tight loops, by a factor of a few thousand. The answer is not “probably not”. It is “no, by two orders of magnitude”, and saying it with the number attached is the point.

Will $O(n \log n)$ pass at $n = 100,000$?

$\log_2(100,000)$	= 16.6, call it 17
$100,000 \times 17$	= 1,700,000 operations
$1,700,000 / 10^8$	= 0.017 seconds

Yes, with a factor of about sixty to spare. Sorting a hundred thousand items is not something to worry about.

Where exactly does $O(n^2)$ stop working? Set the operation count equal to the budget and solve:

$n^2 = 10^8$	→	$n = 10,000$	(one full second, no margin)
$n^2 = 10^7$	→	$n = 3,162$	(comfortable, tenfold margin)

So $O(n^2)$ is safe to about 3,000–5,000 and dangerous beyond 10,000. That single boundary answers most of the “will it pass” questions you will ever be asked.

Where does $O(2^n)$ stop?

$2^{20} = 1,048,576$	→ fine
$2^{25} = 33,554,432$	→ tight
$2^{30} = 1,073,741,824$	→ 10 seconds, dead

So $O(2^n)$ is safe to about 20 and dead by 30. This is why constraints like $n \leq 20$ in a subsets problem are a standing invitation.

What Python costs you on top. The 10^8 figure is generous for pure Python. A realistic range:

LANGUAGE	SIMPLE OPERATIONS PER SECOND
C, C++ , Rust	$10^8 - 10^9$
Java, Go, C#	10^8
Python	$10^6 - 10^7$

Python is roughly **10 to 100 times slower**. Two consequences worth knowing. First, treat “tight” as “fail”. Second, work done inside a built-in — `sum()`, `sorted()`, `min()`, a slice, a set lookup — runs at C speed, not Python speed. `sum(items)` and a hand-written loop have the same Big-O and differ by perhaps thirtyfold in practice.

Space has its own ceiling, and it is lower than people expect. A Python integer in a list costs about 8 bytes for the reference plus 28 for the object. A memory limit of 256 MB therefore means:

256 MB / 36 bytes = about 7,000,000 Python integers in a list

So an $O(n^2)$ **space** solution at $n = 10,000$ wants 100 million entries, which is several gigabytes, and it dies before the time limit ever comes into play. [Day 007](#) is about exactly this.

7 The traps

Trap one: reading the wrong `n`

Here is a constraint block, copied from the kind of problem that catches people:

```
1 ≤ t ≤ 10^5           (number of test cases)
1 ≤ n ≤ 10^5           (array length per test case)
sum of n over all test cases ≤ 2 * 10^5
```

The naive read is “n is 10^5 , so $O(n \log n)$ is fine”. That is right, but only because of the third line. Without it, 10^5 test cases each with 10^5 elements would be 10^{10} elements to even read, and no shape survives that.

The reverse mistake is worse and more common. When there is **no** line bounding the sum, an $O(n \log n)$ solution per test case costs $t \times n \log n = 10^5 \times 1.7 \times 10^6$, which is 10^{11} . Your per-test-case shape was fine and your total was not.

How to catch it every time: find every variable in the constraints, not just the one called `n`. Multiply the per-case cost by the number of cases. If a `sum of n` line exists, use it — it is there precisely because the setter needed it to be.

Trap two: the exponential that does not look exponential

This is the classic, and it is worth meeting properly because it is the reason dynamic programming exists.

```
def fib(n: int) → int:
    if n ≤ 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

Six lines. No loops at all. It looks cheaper than a loop, and it is $O(2^n)$, because each call makes two more.

```
fib(10) → 177 calls      0.000 s
fib(20) → 21,891 calls   0.006 s
fib(25) → 242,785 calls  0.063 s
fib(30) → 2,692,537 calls 0.670 s
fib(40) → 331,160,281    about 80 seconds
fib(50) → 4.1e+10        about 3 hours
```

Ten more in the input, a hundredfold more work. That is what exponential feels like from the inside — everything is fine, and then one more step and it is over.

And here is the other exponential failure, which does produce a real error. Try to fix `fib` by making it walk down one at a time instead:

```
def countdown(n: int) → int:
    if n == 0:
        return 0
    return 1 + countdown(n - 1)

print(countdown(100000))
```

```
Traceback (most recent call last):
  File "d4.py", line 6, in <module>
    print(countdown(100000))
    ^^^^^^^^^^^^^^^^^^^^^^^
  File "d4.py", line 4, in countdown
    return 1 + countdown(n - 1)
    ^^^^^^^^^^^^^^^^^^^^^^^
  File "d4.py", line 4, in countdown
    return 1 + countdown(n - 1)
    ^^^^^^^^^^^^^^^^^^^^^^^
  File "d4.py", line 4, in countdown
    return 1 + countdown(n - 1)
    ^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

Read it literally. Python allows about **1,000** nested calls by default, and this needed 100,000. The [Previous line repeated 995 more times] is Python being merciful about the output. This one is a **space** limit rather than a time limit — every pending call is sitting in memory waiting — and it is the reason a recursive solution over an array of 100,000 elements must be rewritten as a loop. [Day 088](#) covers the call stack properly.

The habit that prevents both traps: before writing anything, say out loud: “ n is at most X , so I have about 10^8 operations, so I need `O(...)` or better.” Thirty seconds, and it changes what you write.

8 In the interview

How it gets asked

- “ n is 100,000. Will an $O(n^2)$ solution pass?” — the direct version. The expected answer has a number in it.
- “What’s the largest n this solution handles?” — the same question turned around.
- “The constraint says $n \leq 20$. What does that suggest to you?” — a hint disguised as a question. It means “try every subset”.
- “Your solution is $O(n \log n)$. Can you get $O(n)$?” — asking whether you know that sorting is sometimes avoidable with a hash map or counting.

What to say out loud, in the first ninety seconds

This one is short, and it belongs at the **start** of the problem, not the end.

1. **Read the constraint aloud.** *“ n is up to a hundred thousand.”*
2. **State the budget.** *“So I’ve got roughly ten to the eight operations to play with.”*
3. **Divide.** *“That rules out $O(n^2)$ — that would be ten to the ten, about a hundred seconds. It comfortably allows $O(n \log n)$, which is about 1.7 million.”*
4. **Name the target.** *“So I’m aiming for $O(n \log n)$ or better. Sorting is affordable here.”*
5. **Then start solving**, with the target already agreed.
6. **If you get stuck, say the budget again.** *“Since sorting is free at this size, let me see what the sorted order gives me.”* Constraints are hints, and using them out loud is the behaviour being assessed.

Doing this at the start costs you fifteen seconds and buys you the interviewer’s confidence for the rest of the round.

The follow-ups

“Where does the ten-to-the-eight number come from?” It is a rule of thumb for how many simple operations a machine gets through in a second — loop iterations, comparisons, integer arithmetic. It is deliberately rough. In C++ it is optimistic-to-fair, and in Python it is optimistic by ten to a hundred times, so I treat anything within a factor of ten of the limit as a fail rather than a pass. The point is not precision; it is that the difference between shapes is measured in factors of a thousand, so a rough number is enough to decide.

“The constraint says $n \leq 20$. What does that tell you?” That an exponential solution is intended. Two to the twenty is about a million, which is nothing. When I see $n \leq 20$ or $n \leq 25$, I stop looking for a clever polynomial answer and start thinking about subsets, bitmasks, or trying every assignment. Constraints that small appear precisely because the polynomial solution does not exist.

“Is $O(n \log n)$ ever worse than $O(n^2)$ in practice?” Yes, for small inputs, because of constants. Sorting has real overhead per element, so at $n = 20$ a tight quadratic loop can genuinely beat it. That is not a theoretical curiosity — Python’s own sort uses insertion sort, which is quadratic, on runs shorter than 64 elements, for exactly this reason. Big-O describes what happens as n grows, and when n is small and fixed, it is answering a question you did not ask.

“How much slower is Python?” Roughly ten to a hundred times, for interpreted loops. But work that happens inside a built-in runs at C speed — `sum()`, `sorted()`, `min()`, set membership, slicing. So the practical move is to keep the Python-level loop

count down and push work into built-ins where the shape allows it. It never changes the Big-O; it changes the constant by enough to matter.

A model answer

The interviewer has shown a problem: given an array of up to 100,000 integers, find whether any two of them sum to a target.

“Before I solve it, let me look at the constraint. n is up to ten to the fifth. My rule of thumb is about ten to the eight operations in a second, so an $O(n^2)$ approach would be ten to the tenth — around a hundred seconds against a one-second limit. That’s dead by two orders of magnitude, and in Python it’s worse than that. So the brute force isn’t a candidate; I’ll mention it and move on.

That leaves $O(n \log n)$ or $O(n)$. $O(n \log n)$ is 100,000 times 17, so about 1.7 million operations — comfortably fine. So sorting is affordable here, which means one approach is to sort and then walk two pointers in from both ends. That’s $O(n \log n)$ time and $O(1)$ extra space.

But I think $O(n)$ is available. One pass with a hash set: for each element, check whether target minus that element is already in the set, then add it. Set membership is $O(1)$ on average, so that’s $O(n)$ time and $O(n)$ space.

I’d go with the hash set version. If the array were already sorted, or if memory were the tight constraint rather than time, I’d take the two-pointer version instead — same time bound for a sorted input, and constant space.

One thing I’d want to confirm: is that ten-to-the-fifth per test case, and is there a bound on the total across test cases? If there are many cases and no bound on the sum, the per-case shape isn’t the number that matters.”

That last question is the one that marks the candidate out. They read the constraints as a specification rather than as scenery.

9 Recall card

- The order, best to worst:** $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, $O(n!)$.
- The budget is 10^8 operations per second.** Divide the constraint into it and the shape names itself.
- The ceilings:** $O(n!)$ dies at 11, $O(2^n)$ at 20, $O(n^3)$ at 500, $O(n^2)$ at 5,000, $O(n \log n)$ at 10^6 , $O(n)$ at 10^7 .

4. **The constraint is a hint.** $n \leq 20$ means try every subset. $n \leq 10^5$ means sort or use a hash map. $n \leq 10^9$ means binary search or a formula.
5. **Python is $10\text{--}100\times$ slower**, so treat “just fits” as “fails”. And read *every* variable in the constraints, not only the one called n .

1 What this is, and why they ask it

TCP and **UDP** are the two ways one machine sends data to another. TCP checks that everything arrived, in order, and re-sends whatever went missing. UDP sends it and does not look back.

That is the whole difference, and everything else follows from it. TCP costs you time and buys you certainty. UDP costs you certainty and buys you time.

Interviewers ask this because it is the cleanest trade-off in the whole of networking, and because the wrong answer is very revealing. “Use TCP, it’s reliable” sounds sensible and is wrong for a video call — the re-sent frame arrives after the moment it belonged to, so you paid for a guarantee you could not use. A candidate who can say *why* a guarantee can be worthless has understood something general about system design, and they will say similar things later about retries, queues and consistency.

2 The story

Kavita has moved into a new flat, and her brother Sameer is driving over with a table in the back of his car. She is on the phone to him at half past six on a Tuesday evening, and he is somewhere on the ring road.

The first part of the call is the directions, and she is careful about it.

“Take the left after the petrol pump.”

“Left after the petrol pump.”

“Then the second gate, not the first. The society is called Sunview.”

“Second gate. Sunview.”

“Tower C, flat 704.”

Nothing. She waits two seconds and says it again. “Tower C, flat 704.”

“Tower C, 704. Got it.”

She notices herself doing this and does not stop, because it is obviously right. Every piece goes across, he repeats it, and if he does not repeat it she says it again. It is slower than just talking. It is slower on purpose. She would rather spend four extra seconds than have him standing outside the wrong gate at seven o’clock with a table.

Then the directions are done, and the call changes completely.

They chat. He tells her about their cousin's new job, she tells him the water heater is not working, he says something about the traffic near the flyover. The signal is not good — he is driving, and it breaks up every so often. She misses a word here and a couple of words there. She misses most of a sentence when he goes under the flyover.

And neither of them asks for anything to be repeated. Not once. She fills in the gap from what came after it, and if she cannot, she lets it go. Asking him to say it again would mean stopping, backing up, and losing where they had got to — and she would rather have the conversation run on smoothly with holes in it than have it stutter.

The same two people, on the same call, using the same bad signal, with two completely different rules. Every part of the address had to arrive, in order, confirmed. None of the chat had to.

At seven he is at the gate. He read the flat number back, so he is at the right one.

3 The idea in plain English

Kavita ran both systems in one phone call, and the reason she switched between them was not the signal. It was **what a missing piece costs**.

The address half is TCP

TCP — the Transmission Control Protocol — is the way of sending data where every piece is confirmed. Take it apart into the four things it does.

It confirms. The receiver sends back an **acknowledgement**, usually called an **ACK**, saying “I have this”. Sameer repeating “Tower C, 704” is an ACK.

It re-sends. If no acknowledgement comes back within a certain time, the sender sends the same piece again. That is **retransmission**, and it is what Kavita did when he went quiet.

It orders. Data is broken into pieces, and each piece carries a **sequence number** — its position in the stream. If piece 5 arrives before piece 4, the receiver holds 5 and waits. What comes out at the far end is always in the order it went in, whatever the network did on the way.

It sets up first. Before any data moves, the two machines do a **three-way handshake**: one says “I want to talk” (SYN), the other says “fine, and I want to talk too” (SYN-ACK), the first says “agreed” (ACK). Only then does data flow. This is why TCP is called **connection-oriented** — there is a real, agreed-upon connection with state on both sides.

TCP also does two things Kavita did without noticing. **Flow control** means the receiver can say “slow down, I cannot keep up”. **Congestion control** means the sender backs off when the network in between is overloaded, which is what stops the internet collapsing under its own traffic.

The chat half is UDP

UDP — the User Datagram Protocol — sends each piece on its own and stops caring. No handshake, no acknowledgement, no re-sending, no ordering. A piece may arrive, may not arrive, and may arrive after one that was sent later.

It sounds broken. It is exactly right for the chat. Repeating a lost word would cost more than the word was worth.

The technical term for a single UDP send is a **datagram**, and the word is well chosen — it is one self-contained message, like a shout across a room, rather than part of an agreed conversation.

The one sentence that decides which

Ask what a missing piece costs. If the answer is “everything”, use TCP. If the answer is “nothing, as long as the next one arrives soon”, use UDP.

A missing digit in a bank transfer is a disaster: TCP. A missing 30 milliseconds of a video call is a flicker nobody mentions, and re-sending it would arrive after the moment it belonged to and make things worse: UDP.

Head-of-line blocking, the thing that makes TCP wrong sometimes

This is the idea that makes the answer non-obvious, and it is worth its own name.

Under TCP, if piece 4 is lost, pieces 5, 6 and 7 may have arrived perfectly — but the receiver **cannot hand any of them over**, because that would break the ordering promise. Everything waits for the retransmission of piece 4. One lost piece stalls everything behind it. That is **head-of-line blocking**.

In a video call that means: one lost fragment of one frame, and the whole picture freezes for a round trip rather than showing a moment of noise. The guarantee did not merely fail to help. It actively made the experience worse.

4 The picture

The TCP handshake, and then what happens when something goes missing.

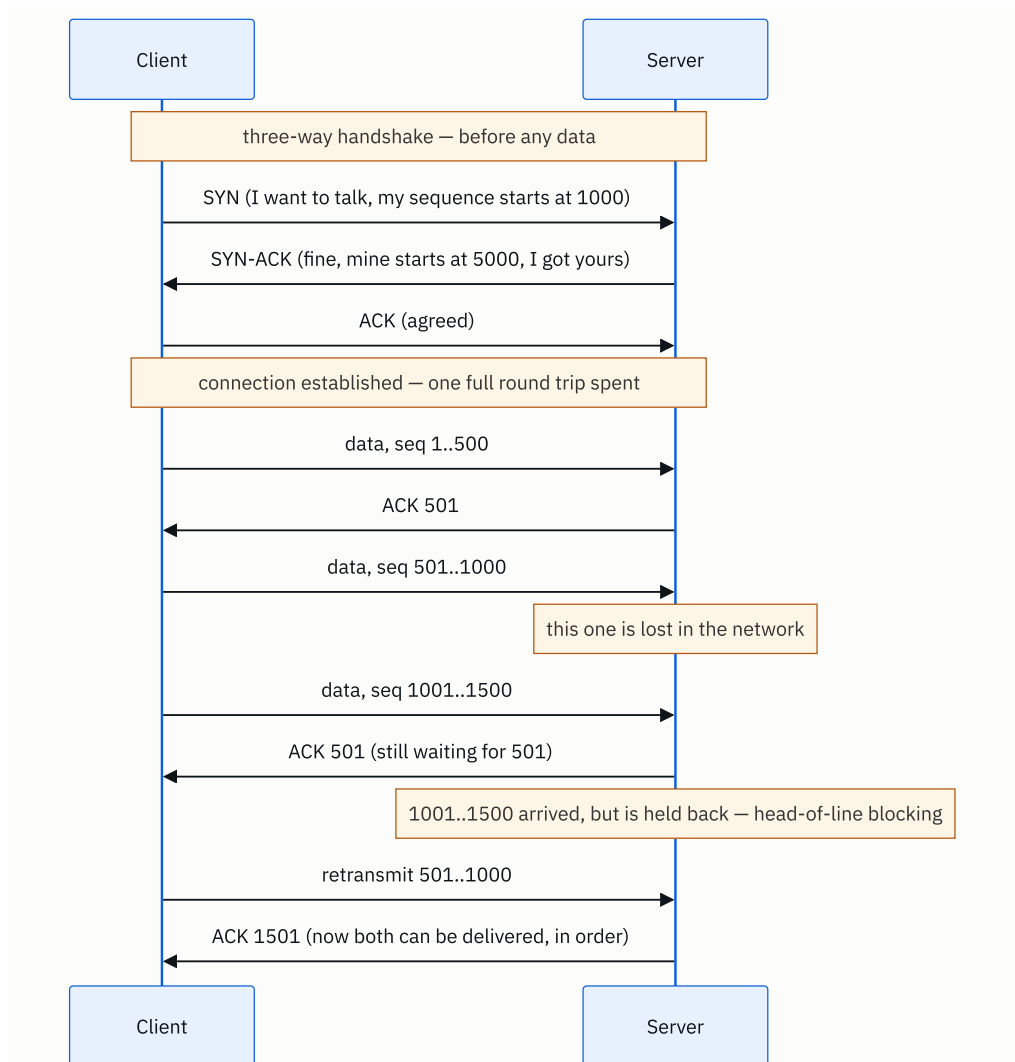


FIGURE 4.1

What to notice: the last four lines. The data at 1001 arrived fine and could not be used. That is the cost of the ordering guarantee, and it is paid in a full round trip.

Now UDP, doing the same thing:

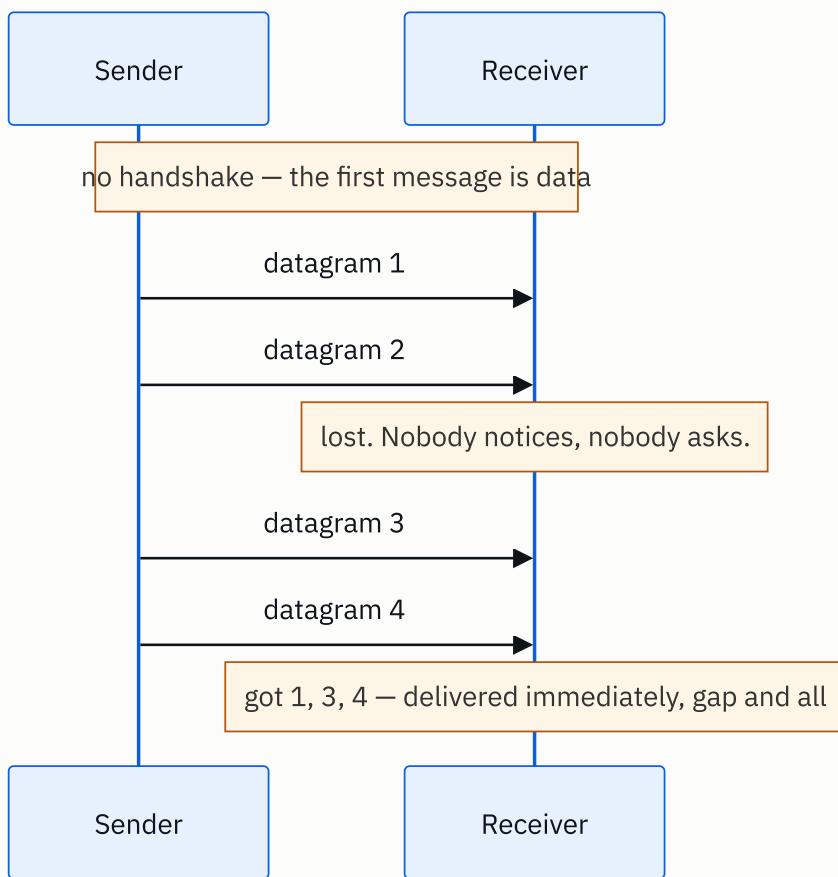


FIGURE 4.2

What to notice: there are no arrows going back. The sender does not know what arrived and does not ask. Datagram 3 was delivered the instant it turned up, because nothing was waiting on datagram 2.

And the two headers side by side, which is the fastest way to see the difference:

TCP header – 20 bytes minimum		UDP header – 8 bytes, always	
+-----+		+-----+	
source port dest port		source port dest port	
+-----+		+-----+	
sequence number	← order	length checksum	
+-----+		+-----+	
acknowledgement number	← ACK		
+-----+			that is the entire header.
flags window size	← flow		
+-----+			
checksum urgent pointer			
+-----+			
options (up to 40 bytes)			
+-----+			

What to notice: every extra field in the TCP header is one of the guarantees. Sequence number buys ordering, acknowledgement number buys reliability, window size buys flow control. UDP has four fields, two of which are just the ports. **The header is the trade-off, written down.**

5 How it actually works

What TCP maintains

Both ends hold real state for the life of a connection: the sequence numbers in each direction, what has been acknowledged, a buffer of unacknowledged data waiting in case it needs re-sending, a receive buffer of out-of-order data waiting for its gap to fill, the current window size, and a set of timers. That is why a machine has a limit on how many TCP connections it can hold at once, and why an idle connection still costs something.

A connection is identified by four values — source IP, source port, destination IP, destination port. Change any one of them and it is a different connection. This is why your laptop can hold dozens of connections to the same site at once: each uses a different source port.

Closing is a four-step exchange (FIN, ACK, FIN, ACK), and the side that closes first sits in a state called `TIME_WAIT` for a couple of minutes afterwards to catch stragglers. A busy machine can genuinely run out of ports because of accumulated `TIME_WAIT` entries, which is a real production problem with a real fix (connection reuse).

Who uses TCP

Everything where a missing byte is a bug. **HTTP/1.1 and HTTP/2**, so almost all web traffic. **SSH. SMTP, IMAP** for mail. **FTP**. The wire protocols of **PostgreSQL** (port 5432), **MySQL** (3306), **Redis** (6379), **MongoDB** (27017). **Kafka**. Every database client you will

ever write uses TCP, without exception, because a half-delivered row is worse than no row.

Who uses UDP

DNS (port 53). A lookup is one small question and one small answer; if it goes missing, asking again is cheaper than setting up a connection would have been. Large answers fall back to TCP.

Real-time media — voice and video calls, over **RTP** on top of UDP, which is what **WebRTC**, **Zoom**, **Google Meet** and **WhatsApp calls** use.

Online games, where the position of a player 50 ms ago is worthless. The next update supersedes it.

NTP (time synchronisation), **DHCP** (getting an address when you join a network), **syslog**, and metric shipping such as **StatsD** — all cases where losing an occasional message is acceptable and the volume is high.

QUIC: the interesting modern answer

HTTP/3 does not run on TCP. It runs on **QUIC**, which runs on **UDP**, and this is the detail that makes a good interview answer.

The reasoning is precisely head-of-line blocking. HTTP/2 put many parallel streams inside one TCP connection, and then discovered that one lost segment stalls *all* of those streams, because TCP orders the whole connection as a single stream of bytes. QUIC rebuilds reliability and ordering **per stream**, on top of UDP, so a loss on one stream does not block the others. It also folds the TLS handshake into the connection setup, cutting a cold start from two or three round trips to one, and zero for a repeat visit.

So QUIC is not “UDP because reliability does not matter”. It is “UDP because we want to build reliability ourselves, at a finer grain than TCP allows”. **Cloudflare**, **Google** and **Meta** serve a large share of their traffic this way today.

What applications on UDP have to build themselves

If you choose UDP and you do need some reliability, you build it. Video calling systems typically add: a sequence number in the RTP header so gaps are detectable, **forward error correction** (sending redundant data so a loss can be reconstructed without asking again), a **jitter buffer** of 50–200 ms that reorders what arrives and smooths the timing, and selective retransmission only for the frames that matter — a keyframe is worth asking for again, an intermediate frame is not.

That list is the honest answer to “isn’t UDP just worse?”. No: it is an empty foundation, and you put back only the guarantees you are willing to pay for.

When it fails

TCP's failure mode is **stalling**. The connection does not break; it goes quiet while a retransmission timer runs. On a bad link you get long freezes rather than errors. A dead peer may not be noticed for minutes unless keepalives are on.

UDP's failure mode is **silent loss**. Nothing tells you a datagram vanished. If your application does not carry sequence numbers, you cannot even count what you lost. Metrics pipelines built on UDP can drop 10% of their data and look perfectly healthy.

6 The numbers

What the handshake costs. A round trip to a data centre in the same city is about 5 ms; across a continent about 40 ms; across the world about 250 ms.

```
TCP connection setup      = 1 round trip
TLS 1.3 handshake on top  = 1 round trip
-----
before the first byte of request: 2 round trips
```

For a user in Mumbai talking to a machine in Virginia at 250 ms per round trip:

```
2 x 250 ms = 500 ms of pure setup, before the request is even sent
```

Half a second of nothing. UDP's equivalent is **0 round trips** — the first datagram is data. QUIC gets it to 1 round trip, and to 0 for a repeat visitor. This is the single biggest reason HTTP/3 exists.

What a video call needs. A 720p call at 1.5 Mbps, with 30 frames per second:

```
1,500,000 bits per second / 8   = 187,500 bytes per second
187,500 / 30 frames              = 6,250 bytes per frame
6,250 / 1,200 bytes per datagram = about 5 datagrams per frame
```

Now suppose 1% of datagrams are lost. Each frame is 5 datagrams, so:

```
chance a given frame loses at least one piece = 1 - 0.99^5 = 4.9%
```

Roughly one frame in twenty is damaged. Under **UDP**, that is one frame in twenty showing a smear, at 30 frames a second — barely noticeable. Under **TCP**, each of those triggers a retransmission, which takes one round trip:

```
1.5 damaged frames per second x 1 round trip (say 100 ms) = 150 ms of freeze per second
```

The picture would stutter constantly. **Same loss rate, same network, and the choice of transport is the difference between a slight blur and an unusable call.**

Where the retransmission arrives. At 30 fps a frame lasts 33 ms. A retransmission over a 100 ms round trip arrives:

```
100 ms / 33 ms per frame = three frames too late
```

The data is correct and useless. That sentence is the heart of the answer.

How much header overhead you pay. For a small game update of 20 bytes:

```
UDP: 20 bytes of IP header context + 8 bytes UDP + 20 payload = 48 bytes on the wire
TCP: 20 bytes of IP header context + 20 bytes TCP + 20 payload = 60 bytes on the wire
```

25% more, before counting the ACK travelling back the other way, which is another 40 bytes for zero payload. At 60 updates per second per player, with 100,000 players:

```
100,000 x 60 x 40 bytes of pure ACK = 240 MB per second of acknowledgement traffic
```

That is the volume argument for UDP, and it is why game servers are built this way.

Connection state. A TCP connection costs roughly 4–16 KB of kernel buffers per side:

```
1,000,000 idle TCP connections x 8 KB = 8 GB of memory
```

A UDP “connection” costs nothing, because there is no such thing. For a service holding a million idle connections — a chat or notification system — that difference decides the architecture.

7 The trade-offs

TCP buys certainty and charges you in time. You get: every byte, in order, exactly once, with the sender automatically slowing down when the network is congested. You pay: a round trip before any data moves, memory and state per connection on both machines, and — the one that actually bites — head-of-line blocking, where one loss stalls everything behind it. On a clean, fast network that price is invisible. On a lossy mobile link it is the whole experience.

UDP buys time and charges you in certainty. You get: no setup, no per-connection state, no blocking, and each message delivered the moment it arrives. You pay: silent loss, possible reordering, possible duplicates, and no congestion control at all — a badly written UDP application will keep firing into a congested network and make things worse for everyone, which is a real reason to be careful with it.

“Just use TCP” is right more often than not. It is worth saying plainly, because the interesting answer is not always the correct one. If you are not sure, TCP is the default, and almost every application you will build — web, mobile, databases, message queues, internal service calls — should use it. UDP is the specialist choice, taken deliberately, for real-time media, high-volume telemetry, DNS-style single exchanges, or when you are building your own transport like QUIC.

You can have both, and modern systems do. WebRTC sends media over UDP and the signalling that sets up the call — who is calling whom, what codecs, the network candidates — over TCP, because losing a signalling message would break the call while losing a video frame would not. Splitting traffic by what the loss costs is the mature version of this decision.

I would not use UDP if... the data is a state change rather than a snapshot. A position update is a snapshot: the next one replaces it and losing one is harmless. “The user pressed the fire button” is a state change: nothing later replaces it, and losing it is a bug the user will notice. The test is not “is it real-time?” — it is “does a later message make this one irrelevant?”.

I would not use plain TCP if... I were serving many parallel streams over a lossy link and cared about tail behaviour. That is the QUIC case exactly, and it is why HTTP/3 exists.

8 In the interview

How it gets asked

- *“TCP or UDP for a live video call, and why?”* — the standard version. The answer they want has “head-of-line blocking” and “the retransmission arrives too late” in it.
- *“What’s the difference between TCP and UDP?”* — the warm-up. Do not just list features; give the trade-off.
- *“Why does HTTP/3 use UDP?”* — the version that separates people who have read something recent from people who have not.
- *“You’re designing a multiplayer game. What transport?”* — a scenario, and the good answer splits the traffic.

What to say out loud, in the first ninety seconds

1. **Give the trade in one line.** *“TCP guarantees delivery and ordering; UDP doesn’t. TCP costs time to buy certainty.”*
2. **Name the four things TCP does.** *“Handshake, sequence numbers for ordering, acknowledgements with retransmission, and congestion control.”*

3. **Ask the deciding question out loud.** *“So the question is what a lost piece costs. For a video call, a lost frame is worth nothing thirty milliseconds later.”*
4. **Land the killer point.** *“With TCP, that lost frame gets retransmitted and arrives a round trip late — three frames too late to display. Worse, everything behind it is held back waiting, because TCP won’t deliver out of order. That’s head-of-line blocking. So one lost fragment becomes a freeze instead of a flicker.”*
5. **Give the answer.** *“So UDP, with RTP on top, plus a jitter buffer and forward error correction. We add back the specific guarantees we want and skip the ones we don’t.”*
6. **Show the boundary.** *“But the signalling that sets the call up goes over TCP — losing that would break the call.”*

Step 4 is the whole answer. Everything else is scaffolding around it.

The follow-ups

“Isn’t UDP just unreliable? Why would anyone accept that?” Because reliability has a delivery time attached to it, and late data can be worthless. TCP does not promise “you get everything”; it promises “you get everything, eventually, in order”. For anything with a deadline, “eventually” is the problem. And UDP is not a guarantee that things get lost — on a good network almost nothing does. It is a decision not to pay for insurance you would not claim on.

“Why does HTTP/3 run on UDP? Doesn’t the web need reliability?” It does, and QUIC provides it — just not TCP’s version of it. HTTP/2 multiplexes many requests over one TCP connection, and TCP orders the connection as a single byte stream, so one lost segment blocks every stream in it. QUIC implements reliability and ordering per stream on top of UDP, so a loss on one only blocks that one. It also merges the transport and TLS handshakes, so a new connection is one round trip instead of two or three. UDP here is a foundation, not an abandonment of reliability.

“How does TCP know how fast to send?” Congestion control. It starts slow, roughly doubles its sending rate each round trip while acknowledgements keep coming back — that is slow start — and cuts back sharply when it sees loss, because loss is taken as evidence of a full queue somewhere. Modern algorithms such as **BBR** measure the actual bottleneck rate rather than waiting for loss. This is entirely absent from UDP, which is why a UDP application has to be a good citizen deliberately.

“Can you lose data with TCP?” Not silently within an established connection — but yes, in ways that matter. The connection can break, and then data sitting in the send buffer is gone. More importantly, TCP acknowledges that the *kernel received the bytes*, not that your application processed them. If the process crashes after the ACK and before

writing to the database, the sender believes it was delivered. That is why application-level acknowledgement exists in Kafka and in every serious queue, and why “TCP is reliable” is true at one layer and not at the one you care about.

A model answer

“UDP, and specifically RTP over UDP, which is what WebRTC does.

The reason isn’t that reliability doesn’t matter — it’s that TCP’s kind of reliability is the wrong kind here. TCP guarantees every byte arrives in order, and it achieves that by retransmitting anything that goes missing and by refusing to deliver later data until the gap is filled.

In a video call at 30 frames a second, a frame is worth about 33 milliseconds. If a piece of one is lost, TCP retransmits it, and that takes a round trip — say 100 milliseconds. The data arrives correct and three frames too late to show. So I paid for a guarantee I couldn’t use.

The worse part is head-of-line blocking. While that retransmission is in flight, the frames behind it have already arrived, and TCP won’t hand them over because that would break the ordering promise. So one lost fragment turns into a hundred-millisecond freeze instead of one slightly smeared frame. The guarantee actively made the experience worse.

With UDP I get each piece the moment it arrives, gaps and all, and I add back exactly the guarantees I want: sequence numbers in the RTP header so I can detect gaps, a jitter buffer of 50 to 200 milliseconds to reorder and smooth timing, forward error correction so small losses reconstruct without a round trip, and selective retransmission only for keyframes, where a loss really does persist.

I’d split the traffic though. The signalling — who’s calling whom, codec negotiation, ICE candidates — goes over TCP or WebSockets, because losing a signalling message breaks the call, and there’s no later message that makes it irrelevant. That’s the test I’d apply generally: if a later message supersedes this one, UDP is fine; if nothing replaces it, it needs reliable delivery.

It’s worth noting this reasoning is why HTTP/3 moved to QUIC over UDP as well — same head-of-line problem, different application.”

That answer names the mechanism, quantifies it, gives the design that follows, states the boundary, and generalises the rule. It is about ninety seconds spoken.

9 Recall card

1. **TCP: handshake, sequence numbers, acknowledgements, retransmission, congestion control.** Everything arrives, in order. **UDP: send and forget.** Four header fields, no promises.
2. **The deciding question: what does a lost piece cost?** Everything → TCP. Nothing, because the next one supersedes it → UDP.
3. **Head-of-line blocking** is why TCP is wrong for media: one loss stalls everything behind it, and the retransmission arrives after the moment it belonged to.
4. **TCP costs 1 round trip to set up, plus 1 for TLS.** UDP costs zero. Over a 250 ms link that is half a second before the first byte of your request.
5. **HTTP/3 runs QUIC over UDP** — not to abandon reliability, but to rebuild it per stream and dodge head-of-line blocking. TCP is still the default for everything else.

PRACTICE

Practice — The growth curves you will meet again and again

DSA topic: The growth curves you will meet again and again **System design topic:** TCP and UDP

Code these, in this order

Four problems chosen because their **constraints** point at four different shapes. Today the constraint is the exercise; the code is almost incidental.

For each problem, before writing anything:

1. Find the largest n allowed. Say it out loud.
2. Divide 10^8 by it and name the shape you can afford.
3. Say which technique that shape implies — sorting, a hash map, subsets, a formula.
4. Only then solve it. Afterwards, check that what you wrote matches what you predicted.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Running Sum of 1d Array	LeetCode 1480 (Easy)	$n \leq 1000$, so almost anything passes. Notice that the constraint gives you no pressure at all, and that this is rare.
2	Subsets	LeetCode 78 (Medium)	$n \leq 10$. That number is the answer: 2^{10} is 1,024, so generating every subset is intended. Read the constraint before the statement.
3	Two Sum II — Input Array Is Sorted	LeetCode 167 (Medium)	$n \leq 30,000$ and the array is sorted. Quadratic is 9×10^8 and dies; the sortedness is the hint that two pointers are the target.
4	Kth Largest Element in an Array	LeetCode 215 (Medium)	$n \leq 10^5$. Sorting is $O(n \log n)$ and comfortably fits, so it is a valid answer — but a heap gives $O(n \log k)$. Say what each one costs.

On problem 2, do this properly

- Before reading the problem statement, read only the constraint. Write down what shape it permits.
- Now read the statement. If your prediction and the problem agree, that is the skill this day exists to build.
- Solve it, then compute how long your solution would take at $n = 30$. Say the number out loud. It should frighten you.

The ceiling drill

Answer these six from memory, in under a minute, without the lesson open:

- The largest n an $O(n^2)$ solution survives.
- The largest n an $O(2^n)$ solution survives.
- The largest n an $O(n!)$ solution survives.
- How many operations $O(n \log n)$ does at $n = 100,000$.
- How many halvings it takes to get from a billion to one.
- What $n \leq 20$ in a problem statement is telling you.

Then check them against §6 of the lesson. Any you got wrong, redo tomorrow.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *n is 100,000. Will an $O(n^2)$ solution pass?* Do not say “no”. Say the multiplication, the division, the number of seconds, and then what shape you would aim for instead.
 2. *TCP or UDP for a live video call, and why?* Get to head-of-line blocking, and to the fact that the retransmission arrives three frames late. Then say what you would put over TCP anyway.
 3. *Why does HTTP/3 run over UDP when the web needs reliable delivery?* One sentence on what TCP’s ordering does to parallel streams, one on what QUIC rebuilds, and one on the handshake saving.
-

Before you move on

- [] I read the constraint before choosing an approach, on all four problems.
- [] I can recite the eight shapes in order, best to worst, from memory.
- [] I can give the ceiling for $O(n^2)$, $O(2^n)$ and $O(n!)$ without looking them up.
- [] I can name the four things TCP does that UDP does not.
- [] I can explain head-of-line blocking to someone who has never heard the phrase.

Python for DSA I: lists, tuples, and slicing

DATA STRUCTURES AND ALGORITHMS

Python for DSA I: lists, tuples, and slicing

You know the real cost of append, pop, insert and a slice, so you stop writing accidental $O(n^2)$.

SYSTEM DESIGN

HTTP: the request and the response

You can write out a real HTTP request and response by hand and label every part.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What is the complexity of inserting at the front of a Python list?*
- *Describe an HTTP request. What is in the headers, and what is in the body?*

Python for DSA I: lists, tuples, and slicing

1 What this is, and why they ask it

A Python **list** is not a linked chain of items. It is one solid run of slots in memory, with the items sitting side by side in order. That single fact decides the cost of every operation you can perform on it.

Adding to the end is nearly free. Adding to the front is not, because everything already there has to shuffle along to make room. Taking a slice does not point at part of the list — it builds a whole new one.

Interviewers ask because this is where good candidates lose otherwise-correct solutions. You write a clean $O(n)$ loop, put a `list.pop(0)` inside it, and you have silently written $O(n^2)$. Nothing looks wrong. The code reads beautifully. It times out on the large test case, and if you cannot say why, the interviewer learns that you do not know what your own tools cost. Every language has this lesson; today you learn Python's version of it.

2 The story

Rakesh looks after the community hall on the ground floor of a housing society in Nagpur. There is a meeting every Sunday at ten, and setting out the chairs is his job.

He starts at twenty past eight. The chairs live in a store room at the back, stacked. He carries them out in stacks of twenty, because carrying them one at a time would mean twenty walks across the hall instead of one. He sets out four stacks — eighty chairs — in rows of twenty, and he chalks the seat numbers on the floor at the end of each row so people can find a place without asking.

By half past nine the hall is nearly full. People are still arriving, and every one of them does the same thing: walks to the end of the last row and sits in the next free chair. That takes Rakesh no work at all. He stands at the side and watches.

At ten to ten the eightieth person sits down, and the next man through the door has nowhere to go. Rakesh walks to the store room and brings out another stack of twenty. It takes him four minutes and he is sweating afterwards. Then, for the next twenty people, he does nothing again.

At ten past ten the chief guest's mother arrives. She is eighty-one, she has come from Amravati, and she cannot sit at the back.

There is one free chair, in the middle of the front row, four seats in — and there is somebody in it. So Rakesh does the only thing that can be done. He asks everyone in the front row from the fourth seat onwards to stand up, pick up their bag, and move one seat along. Nineteen people. Bags, phones, a walking stick, one child asleep on his father's shoulder. It takes almost seven minutes and two people end up in the wrong row.

Rakesh thinks about this afterwards, over tea. Eighty people sat down that morning and seventy-nine of them cost him nothing. One person needed a seat in the middle, and that one cost him seven minutes and a certain amount of goodwill.

The seat numbers, at least, worked. When a woman rang the hall phone to ask where her husband was sitting, Rakesh did not walk down the row counting. She said fourteen, and he looked at the chalk marks and went straight there.

3 The idea in plain English

Rakesh's hall is a Python list, exactly. Let us go through it.

A list is a row, not a chain

A Python **list** is stored as one continuous run of slots in memory, in order, like the chairs in a row. Item 0 is next to item 1, which is next to item 2. Nothing is scattered.

That is why the seat numbers worked. Because the row is continuous and every slot is the same size, the machine can work out where item 14 lives with one multiplication — start of the row, plus fourteen slot-widths — and go straight there. It never walks the row counting. So `items[14]` is `O(1)`, and so is `items[999999]`. Day 009 does the memory picture properly.

Adding at the end is free. Adding anywhere else is not.

`items.append(x)` puts the value in the next free slot. Nobody moves. That is `O(1)`.

`items.insert(0, x)` puts the value at the front, which means **every single item already in the list must move one slot along** to make room. That is `O(n)`. It is the front row standing up with their bags.

The same thing happens in reverse when you remove:

- `items.pop()` takes the last item. Nobody moves. `O(1)`.
- `items.pop(0)` takes the first item, and everything after it shuffles back one place. `O(n)`.

This is the whole lesson. The end of a list is cheap. The front is expensive. The middle is expensive. And “expensive” here means proportional to how much is sitting after the place you touched.

The stack of twenty: why append is *usually* free

Rakesh does not fetch one chair at a time. He fetches twenty, so that the next nineteen arrivals cost him nothing at all.

Python does the same. A list keeps a **capacity** — how many slots it has room for — which is bigger than its **length**, the number of slots actually used. When you append and there is spare capacity, it is genuinely one step. When capacity runs out, Python allocates a bigger run of memory, copies everything across, and carries on. That copy is $O(n)$, and it is the four-minute walk to the store room.

The growth is by roughly one-eighth each time, so the expensive copies get rarer and rarer as the list grows. Averaged over many appends, the cost per append works out to a constant. The word for that is **amortised**: `list.append` is **$O(1)$ amortised**, meaning any single append might be expensive but a long run of them averages out to constant time.

Say it that way in an interview. “Append is $O(1)$ amortised, because the list over-allocates and only occasionally has to copy” is a complete and correct answer.

A slice makes a copy

This is the one that surprises people.

```
first_half = items[:500]
```

That does not create a window onto the first five hundred items. It builds a **new list** and copies five hundred values into it. The cost is $O(k)$, where k is the length of the slice, in both time and memory.

So `items[:]` copies the whole thing — $O(n)$. And `items[1:]`, which looks like a harmless way to say “everything except the first”, is $O(n)$ too. Put that inside a loop and you have written a quadratic without noticing.

Tuples: the same row, nailed down

A **tuple** is written with round brackets — `(3, 7, 2)` — and is a list you cannot change. No append, no assignment to a position, no sorting in place. Reading is identical and identically fast.

Two reasons you will actually use them:

They can be dictionary keys and set members. A list cannot. `seen.add((row, col))` works; `seen.add([row, col])` raises an error. Every grid problem in this course uses tuple coordinates for exactly this reason.

They say **“this will not change”**. A pair of coordinates, a return of two values, a fixed record. The immutability is documentation that the machine enforces.

The operations that are secretly loops

These read like single steps and are not:

LOOKS LIKE ONE STEP	ACTUALLY COSTS
<code>x in items</code>	$O(n)$ — walks until found
<code>items.index(x)</code>	$O(n)$
<code>items.count(x)</code>	$O(n)$
<code>items.remove(x)</code>	$O(n)$ to find, then $O(n)$ to shift
<code>min(items)</code> , <code>max(items)</code> , <code>sum(items)</code>	$O(n)$
<code>items[a:b]</code>	$O(b - a)$
<code>sorted(items)</code>	$O(n \log n)$
<code>len(items)</code>	$O(1)$ — this one really is free

`len()` is `0(1)` because the list stores its own length. That is worth knowing, because it means there is never a reason to keep your own counter alongside a list.

4 The picture

What a list actually looks like in memory, with capacity drawn in:

```
length = 5, capacity = 8
```

index	0	1	2	3	4	5	6	7
slots	12	45	7	99	23			

^^
spare room, already paid for

append(31) → goes straight into slot 5. Nothing moves. 0(1)

What to notice: the three empty slots are the stack of chairs Rakesh already carried out. Appending is free precisely because the room was taken in advance.

Now what `insert(0, 99)` does:

```

before
index    0      1      2      3      4
+-----+-----+-----+-----+
| 12 | 45 | 7 | 99 | 23 |
+-----+-----+-----+-----+

every item slides one place right, starting from the back
+-----+-----+-----+-----+
|      | 12 | 45 | 7 | 99 | 23 |
+-----+-----+-----+-----+
  ^
  then the new value drops in

after
index    0      1      2      3      4      5
+-----+-----+-----+-----+
| 99 | 12 | 45 | 7 | 99 | 23 |
+-----+-----+-----+-----+

5 items moved for 1 insert. With n items, n moves. O(n)

```

What to notice: the work is not in placing the new value. It is in the shuffling that has to happen first, and there is exactly one shuffle per item already present.

And `pop(0)`, which is the same problem walking the other way:

```

+-----+-----+-----+-----+
| 12 | 45 | 7 | 99 | 23 |
+-----+-----+-----+-----+
  ^
  take this out...

+-----+-----+-----+-----+
| 45 | 7 | 99 | 23 |      |
+-----+-----+-----+-----+
<----- everything shifts left -----

pop() from the end instead:
+-----+-----+-----+-----+
| 12 | 45 | 7 | 99 |      |
+-----+-----+-----+-----+
                                   ^
                                   nothing moved at all

```

What to notice: `pop()` and `pop(0)` differ by three characters and by a factor of n.

Finally, what a slice really produces:

```

items = [12, 45, 7, 99, 23]
part = items[1:4]

items  +-----+-----+-----+-----+-----+
      | 12 | 45 | 7 | 99 | 23 |
      +-----+-----+-----+-----+
              \       \       \
              v       v       v
part    +-----+-----+-----+
      | 45 | 7 | 99 |
      +-----+-----+

```

original, untouched

copied, one by one

a whole new list

What to notice: there are now two lists. Changing `part[0]` does not change `items[1]`. And building `part` cost three copies, which is why a slice inside a loop is dangerous.

5 The code, built step by step

The point of today is to *measure* these costs rather than believe them, so build a timing harness and run each operation through it.

Start with the timer.

```

import time

def time_it(label: str, fn) → float:
    start = time.perf_counter()
    fn()
    elapsed = time.perf_counter() - start
    print(f" {label:<34}{elapsed:>10.4f} s")
    return elapsed

```

`time.perf_counter()` is the right clock for measuring short durations — it is monotonic and high-resolution, unlike `time.time()`. The function takes another function and runs it, which lets the same timer measure anything.

Now the two ways of building a list.

```
def build_by_append(n: int) → list[int]:
    out = []
    for i in range(n):
        out.append(i)          # 0(1) amortised
    return out

def build_by_insert(n: int) → list[int]:
    out = []
    for i in range(n):
        out.insert(0, i)       # 0(n) – everything shifts
    return out
```

These two functions produce the same values in opposite orders and differ enormously in cost. `build_by_append` is n steps. `build_by_insert` is $0 + 1 + 2 + \dots + (n - 1)$, the staircase from day 002, which is $n \times (n - 1) / 2$ — quadratic.

Now the two ways of draining one, which is the trap that actually catches people.

```
from collections import deque

def drain_from_front_list(n: int) → None:
    items = list(range(n))
    while items:
        items.pop(0)           # 0(n) each time → 0(n^2) total

def drain_from_front_deque(n: int) → None:
    items = deque(range(n))
    while items:
        items.popleft()        # 0(1) each time → 0(n) total
```

A **deque** — say “deck”, short for double-ended queue — is a standard-library structure built for exactly this. It is a chain of small blocks rather than one run, so both ends are cheap and the middle is not. It is the right answer whenever you need a queue, and you will use it for breadth-first search from [day 101](#) onwards.

Now the slice trap.

```
def sum_by_slicing(items: list[int]) → int:
    total = 0
    while items:
        total += items[0]
        items = items[1:]      # copies the whole rest, every time
    return total
```

This looks functional and tidy. Each `items[1:]` copies $n - 1$, then $n - 2$, then $n - 3$ values. It is the staircase again: $0(n^2)$ time, and it allocates $0(n^2)$ bytes in total along the way.

And the check that a slice really is a copy.

```
original = [1, 2, 3, 4]
part = original[1:3]
part[0] = 99
print(original)      # [1, 2, 3, 4] - unchanged
print(part)          # [99, 3]
```

If a slice were a view, `original` would now read `[1, 99, 3, 4]`. It does not. Two separate lists exist.

Here is the complete program. It measures everything above at a size large enough for the difference to be undeniable.

```

"""Day 5 – what Python list operations actually cost. Measure, do not guess."""

import time
from collections import deque

def time_it(label: str, fn) → float:
    start = time.perf_counter()
    fn()
    elapsed = time.perf_counter() - start
    print(f" {label:<34}{elapsed:>10.4f} s")
    return elapsed

# ---- building -----

def build_by_append(n: int) → list[int]:
    """O(n): each append is O(1) amortised."""
    out: list[int] = []
    for i in range(n):
        out.append(i)
    return out

def build_by_insert(n: int) → list[int]:
    """O(n^2): each insert at the front shifts everything already there."""
    out: list[int] = []
    for i in range(n):
        out.insert(0, i)
    return out

# ---- draining -----

def drain_list_front(n: int) → None:
    """O(n^2): pop(0) shifts the whole remaining list every time."""
    items = list(range(n))
    while items:
        items.pop(0)

def drain_list_back(n: int) → None:
    """O(n): pop() from the end moves nothing."""
    items = list(range(n))
    while items:
        items.pop()

def drain_deque_front(n: int) → None:
    """O(n): a deque is cheap at both ends."""
    items = deque(range(n))
    while items:
        items.popleft()

# ---- slicing -----

def sum_by_slicing(n: int) → int:
    """O(n^2): every slice copies the rest of the list."""
    items = list(range(n))

```

```

total = 0
while items:
    total += items[0]
    items = items[1:]
return total

def sum_by_index(n: int) → int:
    """O(n): walk it once, copy nothing."""
    items = list(range(n))
    total = 0
    for x in items:
        total += x
    return total

def show_slice_is_a_copy() → None:
    original = [1, 2, 3, 4]
    part = original[1:3]
    part[0] = 99
    print(f" original after changing the slice : {original}")
    print(f"  the slice itself                  : {part}")

if __name__ == "__main__":
    N = 50_000

    print(f"building a list of {N:,}")
    a = time_it("append at the end  O(1)", lambda: build_by_append(N))
    b = time_it("insert at the front O(n)", lambda: build_by_insert(N))
    print(f" insert is {b / a:,.0f}x slower\n")

    print(f"draining a list of {N:,}")
    c = time_it("list.pop()         from the back", lambda: drain_list_back(N))
    d = time_it("list.pop(0)        from the front", lambda: drain_list_front(N))
    e = time_it("deque.popleft()    from the front", lambda: drain_deque_front(N))
    print(f" pop(0) is {d / e:,.0f}x slower than popleft()\n")

    print(f"summing a list of {N:,}")
    f = time_it("walk it once       O(n)", lambda: sum_by_index(N))
    g = time_it("re-slice each time O(n^2)", lambda: sum_by_slicing(N))
    print(f" slicing is {g / f:,.0f}x slower\n")

    print("is a slice a copy or a view?")
    show_slice_is_a_copy()

```

This is exactly what it printed:

```

building a list of 50,000
  append at the end  O(1)          0.0053 s
  insert at the front O(n)         0.9690 s
  insert is 181x slower

draining a list of 50,000
  list.pop()        from the back  0.0063 s
  list.pop(0)       from the front  7.8440 s
  deque.popleft()   from the front  0.0049 s
  pop(0) is 1,606x slower than popleft()

summing a list of 50,000
  walk it once      O(n)          0.0055 s
  re-slice each time O(n^2)       7.4298 s
  slicing is 1,349x slower

is a slice a copy or a view?
  original after changing the slice : [1, 2, 3, 4]
  the slice itself                  : [99, 3]

```

Read the ratios, not the times. Every pair on this list does the same job and produces the same answer. One member of each pair is hundreds to over a thousand times slower, and in every case the difference is one method name or one piece of syntax. Now double `N` to 100,000 and run it again: the fast rows double and the slow rows quadruple, which is the doubling test from [day 003](#) confirming the shape.

6 What it costs

The full table. This is the one to know cold, because interviewers ask for individual rows of it directly.

OPERATION	COST	WHY
<code>items[i]</code> (read or write)	$O(1)$	position computed by arithmetic, no walking
<code>len(items)</code>	$O(1)$	the length is stored
<code>items.append(x)</code>	$O(1)$ amortised	spare capacity; occasional resize and copy
<code>items.pop()</code>	$O(1)$	nothing after it to move
<code>items.insert(0, x)</code>	$O(n)$	everything shifts right
<code>items.insert(i, x)</code>	$O(n - i)$	everything after <code>i</code> shifts
<code>items.pop(0)</code>	$O(n)$	everything shifts left
<code>items.remove(x)</code>	$O(n)$	find it, then shift
<code>x in items</code>	$O(n)$	walks until found
<code>items.index(x)</code>	$O(n)$	same walk
<code>items[a:b]</code>	$O(b - a)$	builds and fills a new list
<code>items + other</code>	$O(n + m)$	builds a new list holding both
<code>items.sort()</code>	$O(n \log n)$	Timsort, in place, $O(n)$ extra space
<code>sorted(items)</code>	$O(n \log n)$	same, plus a full copy
<code>items.reverse()</code>	$O(n)$	in place, $O(1)$ extra
<code>min</code> / <code>max</code> / <code>sum</code>	$O(n)$	one pass each
<code>items.copy()</code> or <code>items[:]</code>	$O(n)$	full copy

The amortised argument, with the arithmetic. Python grows a list by roughly 12.5% each time it runs out, so the capacities go 0, 4, 8, 16, 25, 35, 46, and so on. Building a list of one million by appending triggers around **80** resizes, and the total number of values copied across all of them is under two million:

```
4 + 8 + 16 + 25 + ... (each about 1.125x the last, up to 1,000,000)
total copied ~ 1.9 million
divided by 1,000,000 appends
= about 2 extra copies per append, on average
```

Two extra operations per append is a constant. That is what “amortised $O(1)$ ” means, in arithmetic rather than in words.

Where the quadratics come from. Any $O(n)$ operation inside a loop that runs n times:

```
n iterations x O(n) work each = O(n^2)

50,000 x 50,000 / 2 = 1.25 billion element moves
```

At Python's speed that is the 0.38 seconds you saw above, and at $n = 1,000,000$ it is over two hours. This is the single most common accidental quadratic in Python, and every one of them is a one-line fix.

Space. A slice of length `k` costs `k` slots. A list of a million integers costs roughly:

```
1,000,000 x 8 bytes per reference      = 8 MB for the list itself
1,000,000 x 28 bytes per small int object = 28 MB for the objects
-----
36 MB
```

Small integers from -5 to 256 are shared by Python, so a list of a million zeros costs only the 8 MB. That is why `[0] * 1_000_000` is cheap and `list(range(1_000_000))` is not.

7 The traps

Trap one: the queue built out of a list

This is the one that will actually cost you an interview. You need a queue — first in, first out — so you write the obvious thing:

```
def process_in_order(jobs: list[str]) → list[str]:
    done = []
    queue = list(jobs)
    while queue:
        job = queue.pop(0)      # ← O(n), every single time
        done.append(job)
    return done
```

It is correct. It reads well. It is $O(n^2)$, because each `pop(0)` shifts the entire remaining queue one slot left.

```
n = 100,000
list.pop(0)      : 32.18 s
deque.popleft()  : 0.0182 s
ratio            : 1,769 x
```

The fix is two lines:

```

from collections import deque

def process_in_order(jobs: list[str]) → list[str]:
    done = []
    queue = deque(jobs)          # a deque, not a list
    while queue:
        job = queue.popleft()     # O(1)
        done.append(job)
    return done

```

Say this rule out loud until it is automatic: if you take from the front, use a `deque`. Every breadth-first search you write from day 101 onwards depends on it, and a BFS with `pop(0)` in it will time out on any real graph.

Trap two: the multiplication that shares one list

This one produces no error at all. It produces wrong answers, quietly.

```

grid = [[0] * 3] * 3
grid[0][0] = 5
print(grid)

```

```

[[5, 0, 0], [5, 0, 0], [5, 0, 0]]

```

One assignment changed three rows. The reason is that `[[0] * 3] * 3` does not build three rows. It builds **one** row and then puts three references to that same row in the outer list. All three names point at the same object.

The inner `[0] * 3` is fine, because integers cannot be changed in place. It is the outer multiplication that is the bug.

The fix is a list comprehension, which evaluates the inner expression fresh each time:

```

grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 5
print(grid)

```

```

[[5, 0, 0], [0, 0, 0], [0, 0, 0]]

```

This comes back with force on [day 016](#), and it is one of the most common bugs in grid problems.

The related error you will actually see

Having been bitten by the above, people sometimes over-correct and build an empty list, then assign into it by position:

```
result = []
for i in range(5):
    result[i] = i * i
```

```
Traceback (most recent call last):
  File "d5.py", line 3, in <module>
    result[i] = i * i
    ~~~~~^^^
IndexError: list assignment index out of range
```

Read it precisely, because the wording is doing real work. It says **list assignment index**, not just “index”. A list can only be assigned to at a position that already exists. `result` has length 0, so position 0 does not exist yet, and there is nothing to overwrite. Either `append` instead, or pre-fill with `result = [0] * 5`.

The `~~~~~^^^` marks under the traceback point at the exact expression Python objected to — the `result[i]` on the left of the assignment, not the arithmetic on the right.

8 In the interview

How it gets asked

- “What’s the complexity of inserting at the front of a Python list?” — the direct version. The answer is `O(n)`, and the reason is the shifting.
- “You’re using `pop(0)` in that loop — what does that cost?” — the live version, said gently, while you are writing. It is a rescue, not a trap. Take it.
- “Why is `append O(1)` when the list sometimes has to be reallocated?” — the amortised question.
- “Does slicing copy?” — a one-word answer, followed by “so what does that mean inside a loop?”.

What to say out loud, in the first ninety seconds

1. **Say what a list is.** “A Python list is a dynamic array — one continuous run of references, so indexing is $O(1)$ by arithmetic.”
2. **Say where the cheap end is.** “Appending and popping at the end are $O(1)$, because nothing after them has to move.”
3. **Say why the other end is not.** “Inserting or popping at the front is $O(n)$, because every remaining element shifts one slot.”
4. **Give the amortised answer before being asked.** “Append is $O(1)$ amortised — the list over-allocates, so most appends land in spare capacity and only occasionally does it resize and copy. Averaged out, that’s a constant per append.”

5. **Name the fix.** “So if I need a queue, I use `collections.deque`, which is $O(1)$ at both ends.”
6. **Add the slice.** “And slicing copies — `items[1:]` is $O(n)$, so re-slicing inside a loop is an accidental quadratic.”

Steps 4 and 5 are the ones that make you sound like somebody who has shipped Python rather than revised it.

The follow-ups

“Why is inserting at the front $O(n)$ but appending $O(1)$?” Because a list is one contiguous block, and the position of every element is its offset from the start. Insert at the front and every element’s correct position changes, so every one of them has to be physically moved. Append at the end and nobody’s position changes — the new value goes into the next free slot. The asymmetry is a consequence of the memory layout, not a design choice about the API.

“Then why is append only *amortised* $O(1)$?” Because when the spare capacity runs out, Python has to allocate a bigger block and copy everything across, which is $O(n)$ for that one append. It grows by about 12.5%, so those copies get proportionally rarer as the list grows. Adding up the cost of every copy while building a list of n and dividing by n gives a constant of about two, so the average per append is $O(1)$ even though individual appends are not.

“Does slicing create a copy or a view?” A copy, always, for lists. `items[a:b]` allocates a new list of length `b - a` and copies the references in. That matters twice: mutating a slice never affects the original, and slicing inside a loop is $O(n)$ work inside an $O(n)$ loop. If you want a view without copying, you use indices — `left` and `right` variables — which is exactly what the two-pointer pattern from [day 027](#) is. Note that NumPy slices *are* views, which is the opposite convention and a common source of confusion.

“When would you use a tuple instead of a list?” Two reasons. When I need it as a dictionary key or a set member — a list is unhashable and raises `TypeError: unhashable type: 'list'`, so grid coordinates go in as `(row, col)`. And when I want to state that a group of values is fixed, like a returned pair or a fixed record. Tuples are also slightly smaller and marginally faster to build, but that is never the reason I would choose one.

A model answer

The interviewer has watched the candidate write a breadth-first search using `queue.pop(0)` and asks about the complexity.

“Let me look at that line again, because I think I’ve got a problem there.

A Python list is a dynamic array — one contiguous block, with elements laid out in order. That’s what makes `items[i]` $O(1)$: it’s a multiplication and an offset, not a walk. But it also means that removing from the front is $O(n)$, because every remaining element has to shift one slot left to close the gap.

So `queue.pop(0)` is $O(n)$, and it’s inside a loop that runs once per element. That makes the BFS $O(V^2)$ on the queue operations alone rather than $O(V + E)$. On a graph with a hundred thousand vertices, that’s about five billion element moves — it will time out.

The fix is `collections.deque`, which is a doubly-linked list of blocks rather than one contiguous run. Both ends are $O(1)$, so `popleft` is constant time, and the BFS goes back to $O(V + E)$. The trade is that a deque doesn’t give you $O(1)$ indexing into the middle — but a queue never needs that.

While I’m on it, two related things I’d watch for. `append` is $O(1)$ amortised, not strictly $O(1)$ — the list over-allocates by about an eighth, so most appends are free and occasionally one triggers a resize and a full copy. Averaged out it’s constant, which is what matters here.

And slicing copies. If I’d written `queue = queue[1:]` instead of popping, that would be $O(n)$ per iteration too, and it would also allocate a new list every time — so quadratic in memory traffic as well as in time.”

That answer diagnoses a live bug, gives the fix, states the trade-off of the fix, and adds two adjacent facts without padding. It is what fluency in a language sounds like.

9 Recall card

1. A Python list is a **dynamic array**: one contiguous block. `items[i]` and `len()` are $O(1)$.
2. **The end is cheap, the front is expensive.** `append` and `pop()` are $O(1)$. `insert(0, x)` and `pop(0)` are $O(n)$, because everything shifts.
3. `append` is $O(1)$ **amortised** — the list over-allocates by about 12.5%, so resizes get rarer and average out to a constant.
4. **Take from the front → use `collections.deque`.** `popleft()` is $O(1)$. Every BFS depends on this.
5. **Slicing copies.** `items[1:]` is $O(n)$ in time and memory. And `[[0]*3]*3` makes three references to one row — use `[[0]*3 for _ in range(3)]`.

1 What this is, and why they ask it

HTTP — the HyperText Transfer Protocol — is the agreed format for one machine asking another for something over the web. A request has four parts: a **method** saying what you want done, a **path** saying what you want it done to, **headers** carrying facts about the request itself, and an optional **body** carrying the content.

A response has the same shape with one addition: a **status code**, a three-digit number that says how it went.

Interviewers ask this in nearly every system design round, and they ask it early. It looks like a memory test and it is not. Every API you design for the rest of your career is a set of decisions about these four parts — which method, what goes in the path, what goes in a header, what goes in the body, and which status code comes back. A candidate who is vague about headers versus body will design a vague API, and the interviewer knows it.

2 The story

Salim delivers for a courier company, and Tower B of the Green Meadows society is on his round most days. He knows the lift, he knows which floors have dogs, and he knows that flat 402 orders a great deal.

On Monday he goes up with a box for 402. At the door he says one sentence — “delivery for Mrs Fernandes” — and holds the box out. She takes it, and that is the whole transaction.

On Wednesday he is back at the same door, and he is not delivering anything. He has come to take something away: a pair of shoes going back to the company that sent them. Same door, same man, completely different job. He says so at the door, because if he just stood there holding out an empty hand she would have no idea what he wanted.

On Friday he does a third thing at the same door. There is a box for 402 that weighs twelve kilos, the lift has been out since Thursday, and he is not carrying it up four floors on the chance that somebody is in. So he rings the bell first, from downstairs, on the intercom. He is not delivering anything and not collecting anything. He only wants to know whether someone is home. He can do that five times in a row and nothing changes because of it.

The boxes themselves tell him a lot before he opens anything. There are stickers on the outside: who sent it, how heavy it is, whether it is fragile, whether four hundred and fifty rupees has to be collected at the door, and what is inside in general terms — “garments”, or “electronics”. None of that is the thing being delivered. It is information *about* the delivery, and he needs it before he hands anything over. The actual thing is inside, and he never opens the box.

And the answers he gets at doors come in a small number of standard kinds, which is what makes his job manageable. Someone takes it. Nobody is home. There is no flat 402 in this tower — it goes up to 308. The person is home but will not pay the four hundred and fifty. Or the whole tower is locked because of a wedding in the compound and he cannot get in at all, which is nothing to do with this parcel and he will have to come back.

At the end of the day, his supervisor does not want the story of each door. He wants the outcome of each one, in one word.

3 The idea in plain English

Salim’s round is an HTTP conversation, part for part.

The method: what you want done

Salim said something at every door, and it was different each time. In HTTP that is the **method**, sometimes called the verb. It is the first word of the request.

METHOD	WHAT IT MEANS	SALIM'S VERSION
GET	give me this; change nothing	ringing the bell to see if anyone is home
POST	here is something new, create it	delivering a box
PUT	replace what is there with this	swapping a faulty item for a new one, whole
PATCH	change part of what is there	correcting only the phone number on file
DELETE	remove this	taking the shoes back
HEAD	give me only the information about it, not the thing	asking how heavy the box is without taking it
OPTIONS	what am I allowed to do here?	asking whether this building accepts collections

Two words follow from this and interviewers use both.

A method is **safe** if it changes nothing on the other side. `GET` , `HEAD` and `OPTIONS` are safe. Salim ringing the bell is safe.

A method is **idempotent** if doing it five times leaves the same result as doing it once. `GET`, `PUT` and `DELETE` are idempotent — deleting the same thing twice leaves it deleted. `POST` is **not**: two deliveries of the same box are two boxes, which is why a double-clicked payment button is a real problem and why an unreliable network makes `POST` the hard case.

The path: what you want it done to

“Flat 402, Tower B” is the **path** — the part of the URL after the host. `/users/402`, `/orders`, `/search`. The path names the thing. The method says what to do to it. Those two together are the whole idea of a **REST** API, which is covered properly on [day 015](#).

The headers: information about the request, not the request itself

The stickers on the box are **headers**. Each one is a name and a value, one per line. They describe the message rather than being the message.

The ones you will actually meet:

HEADER	WHAT IT SAYS
Host: <code>api.example.com</code>	which site you want — required, and it is how one machine serves many sites
Content-Type: <code>application/json</code>	what format the body is in
Content-Length: <code>348</code>	how many bytes the body is
Authorization: <code>Bearer eyJhbG...</code>	who you are
Accept: <code>application/json</code>	what format you would like back
User-Agent: <code>Mozilla/5.0 ...</code>	what program is asking
Cookie: <code>session=abc123</code>	small values the site asked you to hold and send back
Accept-Encoding: <code>gzip</code>	“you may compress the response”

The body: the thing itself

What is inside the box is the **body**. On a `POST` or `PUT` it is the data you are sending — usually JSON these days. On a `GET` there is normally **no body at all**, which is worth remembering: a `GET` carries its parameters in the path and the query string, not in a body.

The status code: the outcome in one number

Salim’s supervisor wants one word per door. HTTP wants three digits, and the **first digit is the whole category**:

RANGE	MEANING	THE ONES THAT MATTER
1xx	hold on, still going	101 Switching Protocols (WebSockets)
2xx	it worked	200 OK , 201 Created , 204 No Content
3xx	it is somewhere else	301 Moved Permanently , 302 Found , 304 Not Modified
4xx	you got it wrong	400 Bad Request , 401 Unauthorized , 403 Forbidden , 404 Not Found , 429 Too Many Requests
5xx	we got it wrong	500 Internal Server Error , 502 Bad Gateway , 503 Service Unavailable , 504 Gateway Timeout

The 4xx/5xx split is the one interviewers care about, because it is a statement about whose fault it is. There is no flat 402 in Tower B: that is a 404, and Salim retrying will not help. The tower is locked because of a wedding: that is a 503, nothing to do with this parcel, and coming back later is exactly the right response.

The two that people confuse: **401 Unauthorized** actually means “I do not know who you are” — you have not authenticated. **403 Forbidden** means “I know exactly who you are and you are not allowed this”. The names are historically wrong; the meanings are not.

4 The picture

A complete, real HTTP request, with every part labelled:

```

POST /api/v1/orders HTTP/1.1          ← method, path, version
Host: shop.example.com                ←+
Content-Type: application/json         |
Content-Length: 71                    | headers:
Authorization: Bearer eyJhbGciOiJIUzI1... | facts ABOUT the request
Accept: application/json              |
User-Agent: curl/8.4.0                ←+
                                       ← ONE BLANK LINE. This is not optional.
{"product_id": 88213, "quantity": 2,   ←+ body:
 "address_id": 4471}                  ←+ the thing itself

```

What to notice: the blank line. It is the only thing separating headers from body, and it is a genuine part of the format. Everything above it describes the message; everything below it is the message.

And the response that comes back:

HTTP/1.1 201 Created	← version, status code, reason phrase
Content-Type: application/json	←+
Content-Length: 152	
Location: /api/v1/orders/90124	headers
Cache-Control: no-store	
Set-Cookie: session=abc123; HttpOnly	←+
	← the same blank line
{"order_id": 90124, "status": "confirmed",	←+ body
"total": 1499, "eta": "2026-08-29"}	←+

What to notice: 201 rather than 200, because something was created, and a Location header saying where the new thing lives. Getting that pair right is a small thing that reads as professional.

The whole exchange, in order:

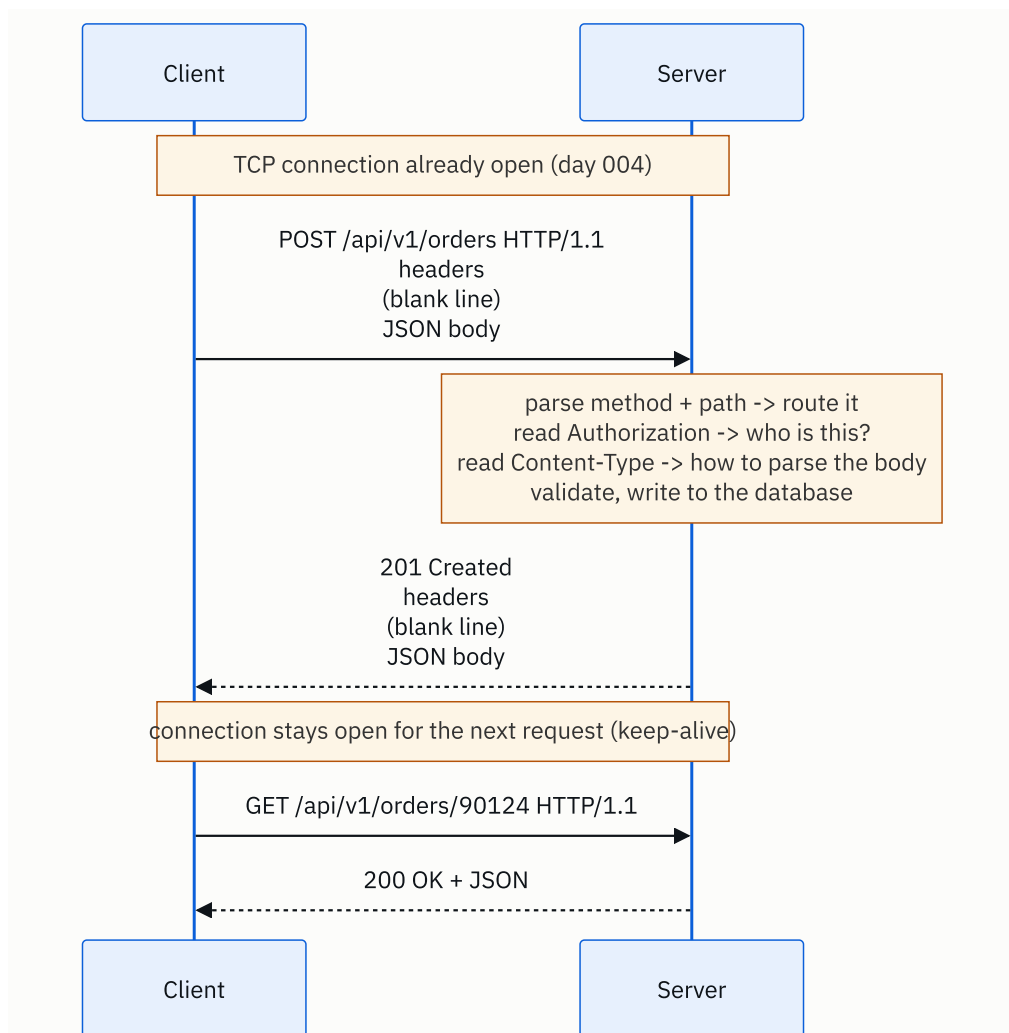


FIGURE 5.1

What to notice: the server's work is entirely driven by the parts of the request. Method and path choose the code that runs; `Authorization` decides whether it may run; `Content-Type` decides how the body is parsed. Each part has exactly one job.

5 How it actually works

It is text, and you can type it yourself

HTTP/1.1 is human-readable text sent over a TCP connection. You can do the whole thing by hand:


```
$ curl -v -X POST https://httpbin.org/post \
-H "Content-Type: application/json" \
-d '{"hello": "world"}'
```

`-v` prints the actual request and response lines, and it is the fastest way to make this concrete. Everything a browser does, `curl` does in one line.

One connection, many requests

Opening a TCP connection costs a round trip, and TLS costs another (day 004). Doing that per request would be ruinous, so HTTP/1.1 defaults to **keep-alive**: the connection stays open and the next request goes down the same one.

HTTP/1.1's limit is that requests on a connection are answered **in order**. A slow response blocks the ones behind it — head-of-line blocking again, this time at the application level. Browsers worked around it by opening six connections per host.

HTTP/2 fixed that by multiplexing many streams over one connection and compressing headers, and **HTTP/3** moved the whole thing to QUIC over UDP to remove TCP's version of the same problem. The request format you learn today is unchanged in all three — HTTP/2 and HTTP/3 send the same methods, paths, headers and bodies in a binary encoding rather than text.

Where the parameters go

For a `GET`, parameters go in the **query string**, after a `?`:

```
GET /search?q=laptop&page=2&sort=price HTTP/1.1
```

Query strings are visible in logs, in browser history, and in the `Referer` header sent to other sites. **Never put a password or a token in one.** That is the practical reason a login is a `POST` with the credentials in the body: not because `POST` is encrypted — over HTTPS both are equally encrypted — but because bodies are not logged by default and query strings are.

What the server does with each part

A real request arriving at, say, **nginx** in front of a **FastAPI** application:

1. **nginx** reads the `Host` header and picks which site this is for. One machine, many sites — this is why `Host` is mandatory in HTTP/1.1.
2. It forwards to the application, adding `X-Forwarded-For` with the client's real IP, because otherwise the application would see nginx's address for every request.
3. The framework **routes** on method plus path: `POST /api/v1/orders` maps to one function.

4. Middleware reads `Authorization`, validates the token, and attaches the user.
5. The body is parsed according to `Content-Type` — JSON, form-encoded, or multipart for file uploads.
6. The handler runs, talks to **PostgreSQL** and **Redis**, and returns.
7. The framework serialises the result, sets `Content-Type` and `Content-Length`, picks a status code, and writes it back.

Statelessness and cookies

HTTP is **stateless**: the server keeps nothing between requests. Every request must carry everything needed to understand it, which is why your identity is re-sent every single time — in a `Cookie` header or an `Authorization` header.

`Set-Cookie` in a response asks the browser to store a value and send it back on subsequent requests to that site. The flags matter: `HttpOnly` hides it from JavaScript (so an XSS bug cannot steal it), `Secure` sends it only over HTTPS, and `SameSite=Lax` stops it being sent on requests initiated by other sites (which is the CSRF defence).

Caching, which is done entirely with headers

The response says how long it may be reused:

```
Cache-Control: max-age=3600, public
ETag: "a3f8b1c9"
```

`max-age=3600` means “reuse this for an hour without asking”. After that the client asks again with `If-None-Match: "a3f8b1c9"`, and if nothing changed the server replies **304 Not Modified** with no body at all — a full round trip, but zero bytes of content. That is how CDNs such as **Cloudflare** and **CloudFront** decide what to keep.

6 The numbers

What the parts weigh. A typical API request:

```
request line           ~40 bytes
Host                   ~30 bytes
User-Agent             ~120 bytes
Accept, Accept-Encoding ~ 80 bytes
Authorization (JWT)    ~500 bytes
Cookie                 ~400 bytes
-----
headers total          ~1,170 bytes
JSON body              ~200 bytes
-----
total                  ~1.4 KB, of which 85% is headers
```

The headers are bigger than the payload. For a small API call that is the normal case, and it is the entire reason HTTP/2 added header compression (HPACK), which typically cuts repeated headers by 80–90%.

What keep-alive saves. A page that makes 50 requests to one host, at 40 ms round trip:

```
without keep-alive: 50 x (1 RTT TCP + 1 RTT TLS + 1 RTT request)
                    = 50 x 3 x 40 ms = 6,000 ms

with keep-alive:    1 x (1 RTT TCP + 1 RTT TLS) + 50 x 1 RTT
                    = 80 ms + 2,000 ms = 2,080 ms
```

Nearly three times faster, from one setting. And with HTTP/2 multiplexing, those 50 requests overlap rather than queue, bringing it closer to 120 ms.

What a fat cookie costs at scale. A 4 KB cookie, on a site doing 10,000 requests per second:

```
4 KB x 10,000 requests/s = 40 MB/s of inbound cookie
40 MB/s x 86,400 s       = 3.4 TB per day
```

Three terabytes a day of data you never look at. This is why session identifiers are short and the session itself lives in **Redis**, and why static assets are served from a separate cookieless domain.

What a 304 saves. A 200 KB image, requested a million times a day, with 90% of clients holding a valid cache entry:

```
without validation: 1,000,000 x 200 KB = 200 GB/day
with ETag, 90% hit: 100,000 x 200 KB + 900,000 x 0.2 KB
                    = 20 GB + 180 MB = 20.2 GB/day
```

A tenfold reduction, from two response headers. At \$0.09/GB of egress that is roughly \$16,000 a year saved by setting `Cache-Control` correctly.

What compression saves. JSON compresses extremely well:

```
100 KB of JSON, gzip → about 12 KB   (8x)
100 KB of JSON, brotli → about 9 KB   (11x)
```

At 10,000 requests per second with 20 KB responses:

```
uncompressed: 200 MB/s
gzipped:      25 MB/s
```

The cost is CPU on both ends, which is real but far cheaper than the bytes.

7 The trade-offs

GET versus POST for a search. `GET /search?q=laptop` is cacheable, bookmarkable, shareable and shows up in logs — all four of which are sometimes what you want and sometimes exactly what you do not. `POST /search` with the query in the body handles complex filters that will not fit in a URL (there is a practical limit of about 2,000 characters) and keeps the terms out of logs, at the cost of losing caching entirely. The usual answer is `GET` until the query stops fitting, then `POST`.

Statelessness costs a lookup and buys you scale. Because the server keeps nothing, every request re-presents its identity and the server re-validates it — a Redis lookup, or a signature check on a token. In exchange, any machine can serve any request, so you scale by adding machines and a machine dying loses nobody's session. The alternative — sticky sessions held in one server's memory — saves the lookup and gives you a bad day when that machine restarts.

Cookies versus tokens. A session cookie is small, revocable instantly (delete the row in Redis and the session is over), and automatically sent by the browser — which is convenient and is also exactly what makes CSRF possible. A **JWT** carries its claims inside itself, so it needs no lookup at all, but it is large (500+ bytes on every request) and cannot be revoked before it expires without adding the very lookup you were avoiding. The honest summary: JWTs are right for service-to-service calls and short-lived access, and sessions are right for browser logins.

Chatty versus fat endpoints. Many small endpoints are clean, cacheable and independently scalable, and they mean a mobile screen might need eight round trips to render. One fat endpoint returning everything is one round trip and a cache nightmare, and it grows into a response nobody dares change. This is the tension that produced **GraphQL** and **Backend-for-Frontend** services, and it comes back on [day 015](#).

I would not use HTTP at all if... the server needs to speak first, or the exchange is continuous. Chat, live scores and collaborative editing want **WebSockets**, where one connection stays open and either side can send. Very high-volume internal service calls often use **gRPC** over HTTP/2 with binary Protobuf instead of JSON, because at a hundred thousand calls per second the parsing cost of text is a real line item. And streaming media does not use request-response at all.

8 In the interview

How it gets asked

- “Describe an HTTP request. What’s in the headers and what’s in the body?” — the direct version, usually early.
- “What’s the difference between PUT and PATCH?” or “POST and PUT?” — the API design version.
- “What status code would you return for X?” — a scenario. Deleting something that is already gone, creating a duplicate, hitting a rate limit.
- “What’s the difference between 401 and 403?” — a precision check with a right answer.

What to say out loud, in the first ninety seconds

1. **Give the four parts of a request.** “A request line with the method and path, then headers, then a blank line, then an optional body.”
2. **Say what the method is for.** “The method says what to do — GET to read, POST to create, PUT to replace, PATCH to modify part, DELETE to remove.”
3. **Draw the line between headers and body.** “Headers are facts about the request — Host, Content-Type, Content-Length, Authorization, Accept. The body is the thing itself, usually JSON. GET requests normally have no body.”
4. **Do the response.** “The response is the same shape plus a status code: 2xx worked, 3xx moved, 4xx the client got it wrong, 5xx the server did.”
5. **Volunteer safe and idempotent.** “GET is safe — it changes nothing. GET, PUT and DELETE are idempotent, so a retry is harmless. POST isn’t, which is why retrying a failed POST can double-charge someone and why you need an idempotency key.”
6. **Offer the next layer.** “Happy to go into caching headers, or how this changes in HTTP/2.”

Step 5 is the one that changes how the rest of the interview goes. It is a correctness concern, not a trivia recital.

The follow-ups

“What’s the difference between PUT and PATCH?” `PUT` replaces the whole resource with what you sent — anything you leave out is treated as removed. `PATCH` applies a partial update, so you send only the fields that change. Both are meant to be idempotent, though a badly designed PATCH (`{"increment_views": 1}`) is not, and that is a design mistake rather than a rule of the method. In practice `PATCH` is what most “edit” endpoints want, because clients rarely hold the complete object.

“What status code for deleting something that’s already gone?” `204 No Content` is defensible, and is what I would pick: `DELETE` is idempotent, so the end state the caller wanted is the state that exists, and telling them it worked is honest. `404` is also defensible if the caller genuinely needs to distinguish “I deleted it” from “it was not there”. The thing that matters is picking one and applying it consistently across the API, and documenting it.

“401 or 403?” `401 Unauthorized` means “I do not know who you are” — no credentials, or invalid ones — and it should come with a `WWW-Authenticate` header telling you how to authenticate. `403 Forbidden` means “I know who you are and you may not do this”, and re-authenticating will not help. The names are backwards from the meanings, which is why this gets asked.

“How do you make a POST safe to retry?” An **idempotency key**. The client generates a unique identifier per logical operation and sends it in a header. The server records the key with the result of the first request, and if the same key arrives again it returns the stored result instead of doing the work twice. Stripe’s API works exactly this way, and it is the standard answer for payments. Without it, a network timeout on a `POST` leaves the client genuinely unable to tell whether the charge went through.

A model answer

“An HTTP request has four parts. The request line — method, path, version, so `POST /api/v1/orders HTTP/1.1`. Then headers, one per line, as name-value pairs. Then a blank line, which is the actual separator. Then an optional body.

The method says what you want done. GET reads, POST creates, PUT replaces the whole thing, PATCH modifies part of it, DELETE removes it. HEAD gets you just the headers, which is how you check whether something exists or how big it is without downloading it.

Headers carry facts about the request rather than the request itself. `Host` says which site — that’s mandatory in 1.1 and it’s how one machine serves many domains. `Content-Type` says how to parse the body, `Content-Length` says how long it is, `Authorization` says who you are, `Accept` says what you’d like back. The body is the payload, usually JSON. GET requests normally have no body; their parameters go in the query string.

The response mirrors that: status line, headers, blank line, body. The status code’s first digit is the category — 2xx succeeded, 3xx redirects, 4xx means the client got it wrong, 5xx means the server did. That split matters operationally, because 4xx means retrying won’t help and 5xx means it might.

The two properties I’d flag in any API design are safety and idempotency. GET is safe — it shouldn’t change anything, which is why you should never put a destructive action behind a GET; a crawler will find it. GET, PUT and DELETE are idempotent, so a retry after a timeout is harmless. POST isn’t, and that’s the one that causes real problems: if a POST times out, the client can’t tell whether it succeeded. The fix is an idempotency key in a header, where the server stores the result against that key and returns the same result on a retry. That’s how Stripe handles payments, and I’d want it on any endpoint that moves money.”

That is ninety seconds, it covers both halves of the question asked, and it lands on a real design concern rather than a list.

9 Recall card

1. **A request is four parts:** request line (method + path + version), headers, a blank line, body. The blank line is part of the format.
2. **Headers are facts about the message; the body is the message.** `Host`, `Content-Type`, `Content-Length`, `Authorization`, `Accept`. A `GET` normally has no body.

3. **Status codes by first digit:** 2xx worked, 3xx moved, **4xx you got it wrong**, 5xx we got it wrong. 401 = I do not know you; 403 = I know you and no.
4. **Safe = changes nothing** (GET, HEAD). **Idempotent = repeating it is harmless** (GET, PUT, DELETE). **POST is neither** — use an idempotency key to make retries safe.
5. **HTTP is stateless.** Identity is re-sent on every request in a cookie or an `Authorization` header. That is what lets any machine answer any request.

DSA topic: Python for DSA I: lists, tuples, and slicing **System design topic:** HTTP: the request and the response

Code these, in this order

Four problems that all reward knowing where a list is cheap and where it is not. Each one has an obvious solution that touches the front of a list, and a better one that does not.

For each problem:

1. Solve it however comes naturally.
2. Then go back and put a finger on every list operation you used. Say its cost out loud.
3. If any $O(n)$ operation is sitting inside a loop, rewrite it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Remove Element	LeetCode 27 (Easy)	The tempting solution is <code>remove()</code> or <code>pop(i)</code> in a loop, which is $O(n^2)$. The write-pointer version is $O(n)$ and touches nothing but the end.
2	Move Zeroes	LeetCode 283 (Easy)	Same trap, sharper. <code>pop(i)</code> then <code>append(0)</code> is quadratic; two indices walking forward is linear. This is the shape of day 015 .
3	Implement Queue using Stacks	LeetCode 232 (Easy)	Forces you to think about which end of a list is cheap. <code>pop()</code> is free, <code>pop(0)</code> is not, and the whole problem exists because of that asymmetry.
4	Rotate Array	LeetCode 189 (Medium)	The one-line slice solution works and allocates a full copy. The reversal trick does it in $O(1)$ extra space. Write both and say what each costs in memory.

On problem 1, do this properly

- Write the version that calls `items.remove(val)` inside a `while` loop.
- Time it on a list of 50,000 elements where every element is the target.
- Now write the write-pointer version, and time that.
- The ratio should be in the hundreds. Say which line was the hidden $O(n)$.

The measurement drill

Run the complete program from §5 of the lesson at `N = 50_000`, then again at `N = 100_000`.

For each row, look at what the time did when the input doubled:

- Rows that roughly **doubled** are $O(n)$.
- Rows that roughly **quadrupled** are $O(n^2)$.

Name the shape of every row from the ratio alone, before checking it against the lesson. Then answer one question out loud: **which single method name is the difference between the two groups?**

The one to try in a terminal

```
curl -v https://httpbin.org/get
```

Read the lines starting with `>` — that is your actual request. Read the lines starting with `<` — that is the actual response. Find the request line, four headers, the blank line and the status code. Then run it again with `-X POST -H "Content-Type: application/json" -d '{"a":1}'` and see what changed.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is the complexity of inserting at the front of a Python list?* Give the answer, then the reason from the memory layout, then why `append` is different, then what “amortised” means, then what you would use instead if you needed a queue.
2. *Describe an HTTP request. What is in the headers, and what is in the body?* Name the four parts in order. Get to safe and idempotent without being asked.
3. *A payment POST times out and the client does not know whether it succeeded. What do you do?* One sentence on why `POST` is the hard case, one on idempotency keys, one on what the server stores.

Before you move on

- [] I can give the cost of `append`, `pop()`, `insert(0, x)`, `pop(0)`, `items[i]` and `items[a:b]` from memory.
- [] I can say what “amortised $O(1)$ ” means in one sentence, with the reason.
- [] I know why `[[0] * 3] * 3` is a bug and what to write instead.
- [] I can write out a full HTTP request by hand — request line, four headers, blank line, body — and label every part.

- [] I can say the difference between 401 and 403, and between PUT and PATCH.

Python for DSA II: strings, dictionaries, and sets

DATA STRUCTURES AND ALGORITHMS

Python for DSA II: strings, dictionaries, and sets

You reach for a dict or a set by reflex when a problem asks whether you have seen something before.

SYSTEM DESIGN

HTTPS and TLS, without the maths

You can explain what the padlock actually guarantees, and what it does not.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *How would you check for duplicates in $O(n)$?*
- *What does HTTPS protect you from? What does it not protect you from?*

Python for DSA II: strings, dictionaries, and sets

1 What this is, and why they ask it

A **dictionary** stores values under keys and finds one in a single step, no matter how many it holds. A **set** stores values and answers “is this in here?” in a single step, no matter how many it holds. A **string** in Python is a sequence of characters that **cannot be changed** once made.

Those three facts convert a large family of $O(n^2)$ solutions into $O(n)$ solutions, and they are the reason the interview answer to “how would you check for duplicates” is one sentence long.

Interviewers ask because the reflex is what they are testing. The question is never really “can you detect duplicates”. It is “when a problem says *have I seen this before*, does a hash-based structure come to mind immediately, or do you write two nested loops first and get rescued?”. That reflex is worth more than any single technique in this course, because it applies to perhaps a third of all interview problems.

2 The story

Suresh minds the shoe counter outside a temple in Nashik. On an ordinary Tuesday there are sixty or seventy pairs. On a Saturday, and on festival days, there are six hundred.

For the first two years he did it the obvious way. Shoes went into a heap behind the counter, roughly in the order they came in, and when someone came back he went and looked for them. On a Tuesday that was fine — thirty seconds, and half the time he remembered the sandals anyway. On a Saturday it was miserable. Six hundred pairs, everyone finishing at about the same time, and a queue of people watching him crouch in a heap of footwear. The worst was a Saturday in the monsoon when a man waited nineteen minutes and missed a bus.

Before the next festival he did something about it. He and his brother built a wall of wooden racks along the back, seven hundred of them, each one just big enough for a pair. Then he painted a number on every rack, one to seven hundred, and got a box of small metal tokens made with the same numbers.

Now the whole thing changed. You hand him your shoes, he puts them in rack 412, and he gives you token 412. When you come back you show him the token, he reads four-one-two, and he walks straight to it. He does not look at rack 411 on the way and he does not start at the beginning. **It takes him the same fifteen seconds whether there are twenty pairs in the wall or six hundred**, and that is the part he still finds slightly remarkable.

There is a second thing he did not plan for and now uses every day. If a man arrives claiming token 412 and rack 412 is empty, Suresh knows instantly that those shoes have already gone. He does not have to search anything to know that something is not there.

The tokens themselves are simple and stubborn. They are stamped metal. When one bent last year and the 6 looked like an 8, he did not try to fix it. He threw it away and had a new one made.

The one bad afternoon since then was the Saturday the count went past seven hundred. There was no rack 701, and there was no room to slip one in — the numbers are painted on the wall in order. So he shut the counter for twenty minutes, and he and his brother put up a second wall of racks, renumbered everything to fifteen hundred, and moved every single pair across to its new rack. It was a bad twenty minutes. It has not happened since.

3 The idea in plain English

Suresh's wall is a **hash table**, and a hash table is what Python's dictionaries and sets are built out of.

The token is the key, and the rack number is worked out from it

The trick that makes the wall fast is that the token **tells you where to go**. There is no searching, because the number *is* the location.

A Python dictionary does the same thing with a calculation. It takes your key — a string, a number, a tuple — and runs it through a function that turns it into a number. That function is a **hash function**, and the number it produces decides which slot to look in. Give it the same key again and it produces the same number again, so it lands on the same slot.

So `phone_book["Anjali"]` is not a search. It is: compute the hash of `"Anjali"`, go to that slot, read what is there. One step. $O(1)$. [Day 060](#) builds one from scratch; today you only need to know that it works and what it costs.

The empty rack: sets and membership

Suresh's second discovery is the whole of set membership. To know that something is **not** there, he does not have to look anywhere except the one place it would be.

A **set** is a dictionary that keeps only the keys. `x in some_set` computes the hash of `x`, goes to that slot, and reports whether anything is there. $O(1)$ on average, whether the set holds ten items or ten million.

Compare that with a list. `x in some_list` walks from the beginning until it finds `x` or runs out — $O(n)$. **The two are spelled identically.** That single collision of syntax is responsible for more accidental quadratics in Python than anything else.

That is the whole duplicates answer

```
def has_duplicate(items: list[int]) → bool:
    seen = set()
    for x in items:
        if x in seen:
            return True
        seen.add(x)
    return False
```

One pass, and each step is $O(1)$. $O(n)$ time, $O(n)$ space. That is the answer the interviewer is waiting for, and it fits on five lines.

The bad Saturday: what “average” is hiding

When the count passed seven hundred, everything had to move. Python does the same. A dict or set keeps more slots than entries, and when it gets about two-thirds full it allocates a bigger table and **re-inserts every key** into the new one. That is a **rehash**, and it is $O(n)$ for that one insertion.

Averaged over many insertions it disappears, exactly like `list.append` on [day 005](#). So the honest statement is: **dict and set operations are $O(1)$ average, amortised.**

There is also a worst case. If every key landed in the same slot, lookups would degrade to $O(n)$. In practice Python's hashing makes this vanishingly unlikely for ordinary data, and strings are additionally randomised per run to stop anyone doing it deliberately. Say “ $O(1)$ average, $O(n)$ worst case” in an interview and you have said the correct thing.

What can be a key, and what cannot

Only things that cannot change can be keys, because a key whose value changed would hash to a different slot and become unfindable.

Allowed: numbers, strings, tuples of allowed things, `frozenset`. **Not allowed:** lists, dictionaries, sets.

So grid coordinates go in as `seen.add((row, col))` — a tuple — and never as a list.

Strings are stamped, not written

Suresh threw the bent token away rather than fixing it. Python strings work the same way: they are **immutable**. Every operation that looks like it changes a string actually builds a new one.

```
s = "hello"
s[0] = "H"
```

```
Traceback (most recent call last):
  File "d6.py", line 2, in <module>
    s[0] = "H"
    ~^^^
TypeError: 'str' object does not support item assignment
```

The consequence that costs people interviews:

```
out = ""
for ch in text:
    out += ch          # builds a whole new string, every time
```

Each `+=` copies everything built so far into a new string. The costs are 1, 2, 3, ... up to n — the staircase, $O(n^2)$. The fix is to collect the pieces in a list and join once at the end:

```
pieces = []
for ch in text:
    pieces.append(ch)  #  $O(1)$ 
out = "".join(pieces) # one pass,  $O(n)$ 
```

`"".join(list_of_strings)` is the correct way to build a string in Python. Learn it as a reflex, the way you learnt `deque` yesterday.

The two dictionary helpers you will use constantly

```
from collections import Counter, defaultdict
```

`Counter(items)` counts everything in one line and is a dict underneath:

```
Counter("mississippi")  # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
```

`defaultdict(list)` gives a fresh empty list for any key you touch, so grouping needs no “is this key here yet?” check:


```
groups = defaultdict(list)
for word in words:
    groups[sorted_key(word)].append(word)
```

Both come back properly on [day 063](#) and [day 064](#).

4 The picture

The wall of racks, which is a hash table:

```
key "Anjali" → hash function → 8,391,204,553 → % 8 slots → slot 1
key "Suresh" → hash function → 2,110,884,921 → % 8 slots → slot 5
```

slot	0	1	2	3	4	5	6	7
		Anjali				Suresh		
		98765..				91234..		

lookup "Anjali": hash it, go to slot 1, read. ONE step, whatever the table holds.
 lookup "Farid" : hash it, go to slot 3, find nothing → not present. ONE step.

What to notice: finding something and finding nothing cost exactly the same. That is what makes `x in some_set` fast, and it is the opposite of a list, where “not present” is the worst case.

Now the same lookup in a list, so the difference is visible:

```
list: ["Deepa", "Farid", "Suresh", "Kavita", "Anjali", "Ravi", "Meena"]
      |         |         |         |         |
      1         2         3         4         5 ← comparisons to find "Anjali"
```

"not present" is worse: all 7 comparisons, every time.

What to notice: the list has to walk. The number of steps depends on where the item is — or on how long the list is, if the item is not there at all.

And the string-building trap, drawn out:

```
out += ch, five times, building "abcde"
```

step 1:	" "	+ "a"	→ new string "a"	copied 1 character
step 2:	"a"	+ "b"	→ new string "ab"	copied 2
step 3:	"ab"	+ "c"	→ new string "abc"	copied 3
step 4:	"abc"	+ "d"	→ new string "abcd"	copied 4
step 5:	"abcd"	+ "e"	→ new string "abcde"	copied 5

total:				15 = 5 x 6 / 2

`"".join(["a","b","c","d","e"]) → measures the total, allocates once, copies 5.`

What to notice: the discarded strings. Four complete strings were built and thrown away to produce one. `join` builds exactly one.

5 The code, built step by step

Build the three ways of detecting duplicates, then measure them, because the gap is the lesson.

Start with the version everyone writes first.

```
def has_duplicate_nested(items: list[int]) → bool:
    """Compare every pair.  $O(n^2)$ ."""
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j]:
                return True
    return False
```

Correct, and it is Meena's copying check from [day 003](#). $n \times (n - 1) / 2$ comparisons in the worst case.

Now the version that *looks* better and is not.

```
def has_duplicate_list(items: list[int]) → bool:
    """One loop – and still  $O(n^2)$ , because `in` on a list is a loop."""
    seen = []
    for x in items:
        if x in seen:
            return True
        seen.append(x)
    return False
```

This is the trap. There is one visible loop, so it reads as linear. `x in seen` walks the whole of `seen`, which grows, so it is quadratic. This is exactly trap one from day 003, and it is worth writing out again because people keep writing it.

Now the correct version.

```
def has_duplicate_set(items: list[int]) → bool:
    """ $O(n)$ : each membership test and each add is  $O(1)$  on average."""
    seen = set()
    for x in items:
        if x in seen:
            return True
        seen.add(x)
    return False
```

One character of difference from the previous one — `set()` instead of `[]` — and a completely different shape.

And the one-line version, worth knowing and worth understanding.

```
def has_duplicate_short(items: list[int]) → bool:
    return len(set(items)) < len(items)
```

`set(items)` drops duplicates, so a shorter set means there were some. It is $O(n)$ and it is clean. The one thing it gives up: it always builds the whole set, where the loop version can return `True` the moment it finds a repeat. For an input whose first two elements match, the loop is two steps and this is n steps. Both are $O(n)$; only one of them stops early.

Now the string half.

```
def build_by_concat(n: int) → str:
    """O(n^2): every += copies the whole string built so far."""
    out = ""
    for i in range(n):
        out += "x"
    return out

def build_by_join(n: int) → str:
    """O(n): collect the pieces, allocate once."""
    pieces = []
    for i in range(n):
        pieces.append("x")
    return "".join(pieces)
```

Here is the complete program.

```

"""Day 6 – dictionaries, sets, and why string += is a quadratic."""

import time
from collections import Counter, defaultdict

def time_it(label: str, fn) → float:
    start = time.perf_counter()
    fn()
    elapsed = time.perf_counter() - start
    print(f" {label:<36}{elapsed:>10.4f} s")
    return elapsed

# ---- duplicates -----

def has_duplicate_nested(items: list[int]) → bool:
    """O(n^2) time, O(1) space. Every pair."""
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j]:
                return True
    return False

def has_duplicate_list(items: list[int]) → bool:
    """O(n^2) time. Looks linear; `in` on a list is not O(1)."""
    seen: list[int] = []
    for x in items:
        if x in seen:
            return True
        seen.append(x)
    return False

def has_duplicate_set(items: list[int]) → bool:
    """O(n) time, O(n) space. The answer."""
    seen: set[int] = set()
    for x in items:
        if x in seen:
            return True
        seen.add(x)
    return False

def has_duplicate_short(items: list[int]) → bool:
    """O(n) time, O(n) space. No early exit."""
    return len(set(items)) < len(items)

# ---- building strings -----

def build_by_concat(n: int) → str:
    """Looks O(n^2) -- but CPython special-cases it. See the note under the output."""
    out = ""
    for _i in range(n):
        out += "x"
    return out

def build_by_concat_shared(n: int) → str:
    """The honest O(n^2): a second name holds the string, so the special case cannot apply."""
    out = ""
    history = []
    for _i in range(n):
        history.append(out)      # `out` now has more than one reference...
        out = out + "x"         # ...so this must allocate and copy, every time
    return out

def build_by_join(n: int) → str:

```

```

"""O(n): one allocation at the end."""
pieces = []
for _i in range(n):
    pieces.append("x")
return "".join(pieces)

# ---- the two dict helpers -----

def first_non_repeating(text: str) → str | None:
    """The classic two-pass Counter problem."""
    counts = Counter(text) # O(n)
    for ch in text: # O(n)
        if counts[ch] == 1:
            return ch
    return None

def group_anagrams(words: list[str]) → list[list[str]]:
    """defaultdict removes the 'is this key here yet?' check."""
    groups: defaultdict[str, list[str]] = defaultdict(list)
    for word in words:
        key = "".join(sorted(word)) # O(k log k) for a word of length k
        groups[key].append(word)
    return list(groups.values())

if __name__ == "__main__":
    N = 20_000
    distinct = list(range(N)) # worst case: no duplicates at all

    print(f"checking {N:,} distinct values for duplicates")
    a = time_it("nested loops", lambda: has_duplicate_nested(distinct))
    b = time_it("'in' on a list", lambda: has_duplicate_list(distinct))
    c = time_it("'in' on a set", lambda: has_duplicate_set(distinct))
    d = time_it("len(set(items))", lambda: has_duplicate_short(distinct))
    print(f" the set version is {a / c:,.0f}x faster than nested loops")
    print(f" and {b / c:,.0f}x faster than the list version it resembles\n")

    M = 40_000
    print(f"building a string of {M:,} characters")
    e = time_it("out += ch (special-cased)", lambda: build_by_concat(M))
    g = time_it("out = out + ch (shared)", lambda: build_by_concat_shared(M))
    f = time_it('"".join(pieces)', lambda: build_by_join(M))
    print(f" the special-cased version is {e / f:,.1f}x the cost of join")
    print(f" the honest quadratic is {g / f:,.0f}x the cost of join")
    print()

    print("the two helpers")
    print(f" first non-repeating in 'swiss' : {first_non_repeating('swiss')}")
    print(f" first non-repeating in 'aabb' : {first_non_repeating('aabb')}")
    print(f" Counter('mississippi') : {dict(Counter('mississippi'))}")
    print(f" grouped anagrams : ")
    print(f" {group_anagrams(['eat', 'tea', 'tan', 'ate', 'nat', 'bat'])}")

```

This is exactly what it printed:

```

checking 20,000 distinct values for duplicates
nested loops          0(n^2)          22.1194 s
`in` on a list         0(n^2)          3.2798 s
`in` on a set          0(n)           0.0035 s
len(set(items))       0(n)           0.0017 s
the set version is 6,348x faster than nested loops
and 941x faster than the list version it resembles

building a string of 40,000 characters
out += ch             (special-cased)  0.0115 s
out = out + ch (shared) 0(n^2)         1.3205 s
"".join(pieces)        0(n)           0.0050 s
the special-cased version is 2.3x the cost of join
the honest quadratic is 262x the cost of join

the two helpers
first non-repeating in 'swiss'          : w
first non-repeating in 'aabb'          : None
Counter('mississippi')                 : {'m': 1, 'i': 4, 's': 4, 'p': 2}
grouped anagrams                       : [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]

```

Look at rows two and three. They differ by one word — `[]` against `set()` — and by a factor of nearly a thousand. Nothing about the code’s appearance tells you which is which. You have to know.

Now look at the three string rows, because they contain an honest surprise. The plain `out += ch` version is only about twice the cost of `join`, not hundreds of times. That is not because the language semantics changed — it is because **CPython has a special case**: when the string being appended to has exactly one reference, the interpreter resizes it in place instead of building a new one. The middle row defeats that by keeping a second reference, and there the true quadratic appears at over two hundred times the cost of `join`.

Take the right lesson from this. The optimisation is real and it is also fragile: it disappears the moment the string is stored anywhere else, passed to a function that keeps it, built up inside a class attribute, or run on a different Python implementation. `"".join(pieces)` is `O(n)` by construction and needs no luck, which is why it remains the correct habit.

6 What it costs

The table to know cold.

OPERATION	COST	NOTE
<code>d[key]</code> , <code>d[key] = v</code> , <code>key in d</code>	$O(1)$ average	$O(n)$ worst case, essentially never
<code>d.get(key, default)</code>	$O(1)$ average	no exception on a missing key
<code>del d[key]</code> , <code>d.pop(key)</code>	$O(1)$ average	
<code>len(d)</code>	$O(1)$	stored
<code>for k in d</code>	$O(n)$	insertion order, guaranteed since Python 3.7
<code>x in some_set</code> , <code>s.add(x)</code> , <code>s.remove(x)</code>	$O(1)$ average	
<code>set(items)</code>	$O(n)$	
<code>a & b</code> , <code>a \ b</code> , <code>a - b</code>	$O(\min(\text{len } a, \text{len } b))$ for <code>&</code> ; $O(\text{len } a + \text{len } b)$ for the rest	
<code>x in some_list</code>	$O(n)$	the trap
<code>s[i]</code> on a string	$O(1)$	
<code>s1 + s2</code>	$O(\text{len } s1 + \text{len } s2)$	builds a new string
<code>"".join(parts)</code>	$O(\text{total length})$	one allocation
<code>s.split()</code> , <code>s.replace()</code> , <code>s.lower()</code>	$O(n)$	each returns a new string
<code>sub in s</code> (substring)	$O(n \times m)$ worst case	not a hash lookup
<code>sorted(s)</code>	$O(n \log n)$	returns a list of characters

Why $O(1)$ is honest here. The hash of a key does not depend on how many keys exist, and going to a slot is arithmetic. So the work is: hash the key, compute a slot, look. Three fixed steps. The size of the table changes nothing about them.

The memory price, with the arithmetic. A set keeps its table under two-thirds full, so storing n items needs about $1.5n$ slots at 8 bytes each, plus the objects themselves:

```

1,000,000 integers in a set:
  table slots : 1,500,000 x 8 bytes = 12 MB
  int objects : 1,000,000 x 28 bytes = 28 MB
                                -----
                                40 MB

the same integers in a list:
  references  : 1,000,000 x 8 bytes = 8 MB
  int objects  : 1,000,000 x 28 bytes = 28 MB
                                -----
                                36 MB

```

So a set costs roughly 10–30% more memory than a list of the same values. **That is the whole price of turning `O(n)` lookups into `O(1)` lookups**, and it is almost always worth paying. Naming that trade out loud — “`O(n)` time and `O(n)` space, and I’m spending the space deliberately” — is what the interviewer wants to hear.

The string concatenation arithmetic. Building a string of length n one character at a time, without the in-place special case, copies:

```

1 + 2 + 3 + ... + n = n(n+1)/2 characters
at n = 40,000 → 800,000,000 character copies

```

Eight hundred million copies to build a forty-thousand-character string. `join` copies exactly 40,000. That is the $262\times$ measured on the shared-reference row. The special-cased row avoids almost all of it — when it can.

Where the trade stops being worth it. If `n` is 20, a nested loop is 190 comparisons and a set costs an allocation and 20 hashes. The set is not faster there, and it is not slower enough to matter either. Use the set anyway, because it is clearer and because the constraint will change.

7 The traps

Trap one: the missing key

You count things, and reach for the dictionary directly:

```

counts = {}
for ch in "hello":
    counts[ch] += 1

```



```
Traceback (most recent call last):
  File "d6.py", line 3, in <module>
    counts[ch] += 1
    ~~~~~^~~~~
KeyError: 'h'
```

Read it exactly. `counts[ch] += 1` means `counts[ch] = counts[ch] + 1`, and the right-hand side runs first. On the very first character there is no entry to add one to, so the lookup fails. `KeyError` names the key it could not find, which is `'h'`.

Three correct fixes, in increasing order of how much a reviewer will like them:

```
counts[ch] = counts.get(ch, 0) + 1          # get with a default
```

```
from collections import defaultdict
counts = defaultdict(int)                  # missing keys start at 0
counts[ch] += 1
```

```
from collections import Counter
counts = Counter("hello")                  # the whole loop, in one call
```

Note the subtlety in the middle one: `defaultdict` **creates** the entry when you touch it. So `if counts[x] == 0` on a `defaultdict(int)` silently adds `x` to the dictionary, which can turn a read into a write and make a later `len(counts)` wrong. Use `x in counts` to check without creating.

Trap two: the key that cannot be a key

Grid problems make everyone meet this one.

```
seen = set()
seen.add([2, 3])
```

```
Traceback (most recent call last):
  File "d6.py", line 2, in <module>
    seen.add([2, 3])
    ~~~~~^~~~~~
TypeError: unhashable type: 'list'
```

Unhashable means “this cannot be run through the hash function”. Lists can be changed, and a key that changed would hash to a different slot and become invisible — the entry would still be in the table and no lookup would ever find it again. Python refuses at the door rather than allowing that.

The fix is a tuple, which cannot change:

```
seen.add((2, 3))          # round brackets. This works.
```

And the same rule with the same error appears for dictionary keys: `d[[1, 2]] = "x"` raises the identical `TypeError`. If you need a set as a key, use `frozenset`.

How to catch both every time: when a value is going into a set or being used as a key, ask “could this object be modified?”. If yes, convert it — `tuple(...)` for a list, `frozenset(...)` for a set, and `"".join(sorted(word))` for the canonical form of a word.

The near-miss worth knowing

This one produces no error and the wrong answer:

```
def is_anagram(a: str, b: str) → bool:
    return set(a) == set(b)
```

`set("aab") == set("abb")` is `True`, because both sets are `{'a', 'b'}`. A set forgets how many times something appeared. The correct version keeps the counts:

```
def is_anagram(a: str, b: str) → bool:
    return Counter(a) == Counter(b)
```

Set when you care whether it appeared. Counter when you care how often. That distinction is worth a full second of thought each time, and it is the subject of [day 022](#).

8 In the interview

How it gets asked

- “How would you check for duplicates in $O(n)$?” — the direct version, often as a warm-up.
- “Can you do that lookup faster?” — said while you are writing `x in some_list`. It is a rescue. Take it.
- “What’s the time complexity of a dictionary lookup, and when is it not that?” — the version that checks whether you know `O(1)` is an average.
- “You’re using a list as a dictionary key — what happens?” — a precision check.

What to say out loud, in the first ninety seconds

1. **Name the structure immediately.** “I’d use a set. The question is whether I’ve seen something before, and that’s exactly what a set answers in $O(1)$.”

2. **Describe the pass.** “One pass over the array. For each element, check whether it’s in the set — if it is, return True; otherwise add it and carry on.”
3. **Give both costs.** “ $O(n)$ time, $O(n)$ extra space.”
4. **Say why it is $O(1)$, briefly.** “Set membership hashes the value to a slot and looks there, so it doesn’t depend on how many items the set holds.”
5. **Give the honest caveat.** “That’s $O(1)$ on average. Worst case, if everything collided, it degrades to $O(n)$, but Python randomises string hashing so that isn’t reachable in practice.”
6. **Name the trade and the alternative.** “If $O(n)$ extra space weren’t acceptable, I’d sort first and check adjacent pairs — $O(n \log n)$ time, $O(1)$ extra space if the sort is in place.”

Step 6 is what turns a five-line answer into a design answer.

The follow-ups

“Why is a set lookup $O(1)$ but a list lookup $O(n)$?” Because a set computes where the value would be, and a list has to go and look. The set runs the value through a hash function, which produces a number, which picks a slot. Three fixed steps, independent of size. A list has no such shortcut: elements are in insertion order, not in an order derived from their values, so the only way to know whether something is present is to compare against each one. That is also why finding out that something is *absent* is the worst case for a list and costs the same as anything else for a set.

“When is a dictionary not $O(1)$?” Two cases. When many keys hash to the same slot, lookups walk a chain and degrade toward $O(n)$ — that is the theoretical worst case, and it is why hash-flooding was once a real denial-of-service technique against web frameworks. Python defends against it by randomising string hashes for each run. The second case is insertion: when the table gets about two-thirds full it allocates a bigger one and rehashes every key, which is $O(n)$ for that one insertion. Amortised over many, it is still constant.

“Can you use a list as a dictionary key?” No — `TypeError: unhashable type: 'list'`. Keys have to be immutable, because the hash decides the slot, and if the object changed after insertion its hash would change and the entry would be permanently unreachable. Tuples work, `frozenset` works, strings and numbers work. In grid problems I use `(row, col)` tuples for exactly this reason.

“Do dictionaries keep insertion order?” Yes, and it is guaranteed by the language specification, not just an implementation detail — it became an accident of the implementation in 3.6 and a promise in 3.7. Sets do **not** keep order, and their iteration order can differ between runs. So if the order of your output matters, do not iterate a set and hope.

A model answer

“For duplicates in $O(n)$, I’d use a set.

I walk the array once. For each element I ask whether it’s already in the set. If it is, I’ve found a duplicate and I return immediately. If it isn’t, I add it and move on. If I get to the end without a hit, there are no duplicates.

That’s $O(n)$ time, because each element costs one membership test and at most one insertion, and both of those are $O(1)$ on average. It’s $O(n)$ extra space in the worst case, when everything is distinct and the set ends up as large as the input.

The reason set membership is constant is that it isn’t a search. The value goes through a hash function, which gives a number, which picks a slot, and it looks in that one slot. It doesn’t matter whether the set holds ten items or ten million. That’s the difference from `x in some_list`, which is spelled identically and is $O(n)$, because a list has to compare against each element in turn.

I’d add the caveat that $O(1)$ is the average. If every key collided it would degrade to $O(n)$, and insertions occasionally trigger a rehash of the whole table. Neither matters in practice for ordinary data, but I’d want to say it rather than claim a guarantee I don’t have.

There’s also a one-liner — `len(set(items)) < len(items)` — which is the same complexity and reads better, but it always builds the whole set, so it loses the early exit. If the first two elements match, the loop version does two steps and this does n .

And if extra space were the constraint rather than time, I’d sort the array and check adjacent pairs instead: $O(n \log n)$ time, $O(1)$ extra space with an in-place sort. That’s the trade — I’m spending $O(n)$ memory to buy back a factor of $\log n$, or a factor of n against the nested-loop version.”

That answer gives the code, both complexities, the mechanism, the honest caveat, and two alternatives with the conditions that would make each one right.

9 Recall card

1. “Have I seen this before?” means a set. `x in some_set` is $O(1)$; `x in some_list` is $O(n)$. They are spelled the same. This is the reflex the whole day exists to build.
2. Dict and set are $O(1)$ average, $O(n)$ worst case, and insertion is amortised because the table occasionally rehashes.

3. **Keys must be immutable.** Tuples yes, lists no — `TypeError: unhashable type: 'list'`. Grid coordinates go in as `(row, col)`.
4. **Strings are immutable, so building one with `+=` in a loop is $O(n^2)$** — CPython sometimes rescues it with an in-place resize, and that rescue vanishes the moment a second reference exists. Collect pieces in a list and `"".join(pieces)` once.
5. **Counter when you care how many, set when you care whether.** `set("aab") == set("abb")` is `True`, and that is a bug waiting to happen.

1 What this is, and why they ask it

HTTPS is ordinary HTTP with a layer underneath it called **TLS** — Transport Layer Security. TLS does three things and only three: it proves you are talking to the machine that owns the name you asked for, it scrambles everything you send so that nobody in between can read it, and it detects any attempt to alter it in transit.

The padlock in the address bar means those three things succeeded. It means nothing else at all.

Interviewers ask this because the second half of the question is where candidates fall over. “HTTPS encrypts your data” is true and incomplete, and the incompleteness is dangerous. A site can have a perfect padlock and still be a phishing site, still store your password in plain text, still leak which sites you visit to your network provider. Knowing exactly where the guarantee starts and stops is a security-thinking test, not a networking test, and it is asked far more often than the mathematics ever is.

2 The story

Nandini is selling her mother’s flat in Kochi, and she lives in Bengaluru. The buyer’s lawyer needs the original ownership documents this week, and she cannot fly down. The buyer says he will send a man to collect them.

She is not comfortable with this, and she is right not to be. She has never met the man. The documents are irreplaceable. Anybody could ring her doorbell on Thursday and say the buyer sent them.

So she does the sensible thing. She rings the buyer — whose number she has had for four months, whose voice she knows, whose face she has seen — and asks him directly. He says the man’s name is Imran, gives her his number, and describes him. She rings Imran and hears the same details back. Now she is satisfied, not because she trusts Imran, but because somebody she already trusts vouched for him.

They meet on Thursday at half past four in a café near her office, because she is not having a stranger come to the flat. The place is full — every table taken, the machine grinding, two people at the next table talking about a wedding.

They sit and they speak quietly. She goes through the documents with him, tells him which ones are originals and which are copies, and mentions the amount that is still outstanding and the date it is due. Nobody around them hears a word of it. The couple at the next table would have to lean over to catch anything, and they are not interested.

But everybody in that café can see plenty. They can see that a woman met a man at half past four. They can see it lasted twenty-five minutes. They can see she handed him a thick folder and he put it in his bag. The man behind the counter recognises her, because she comes in most mornings, and he could tell anyone who asked that she was in on Thursday afternoon with someone he had not seen before. None of the content, all of the shape.

And there is one thing none of this covers, which occurs to her on the walk back. She checked very carefully that the man in front of her was Imran. She never checked whether Imran was honest. If the buyer's own man decides to disappear with the documents, every bit of her careful verification will have been correct and useless. She verified who he was. She did not verify what he would do.

3 The idea in plain English

Nandini's afternoon is TLS, part for part, including the part at the end that most people leave out.

Ringling the buyer: this is the certificate

Nandini did not verify Imran directly. She asked somebody she already trusted, who vouched for him.

That is exactly how HTTPS proves identity. When your browser connects to `google.com`, the machine sends back a **certificate** — a small document containing the name it claims (`google.com`), a public key, an expiry date, and a **signature** from a **Certificate Authority**.

A **Certificate Authority**, or **CA**, is an organisation whose job is to vouch. Let's Encrypt, DigiCert and Sectigo are the big ones. Your browser and operating system ship with a list of roughly 150 CAs they already trust — that list is the buyer whose number Nandini has had for four months. Nobody chose it consciously; it came with the device.

So the chain is: *I trust my browser's built-in list* → *that list trusts DigiCert* → *DigiCert has signed a certificate saying this machine is google.com* → *therefore this machine is google.com*. That is the **chain of trust**, and every link in it is somebody vouching for somebody else.

Crucially, the CA checked one thing only: that whoever asked for the certificate controls that domain name. It did not check that they are honest, or competent, or that the site does anything reasonable. **A CA vouches for identity, never for character.**

Speaking quietly in a full café: this is encryption

Everything Nandini and Imran said was inaudible to everyone around them, even though they were surrounded.

TLS **encrypts** the entire HTTP conversation. That means it is scrambled with a shared secret so that anyone reading the traffic sees only noise. What gets covered is more than people expect:

- the path — `/account/settings`, not just the site
- all headers, including cookies and `Authorization`
- the request body
- the entire response

Anyone sitting between you and the site — the café’s wifi, your ISP, whoever runs the network at an airport — sees encrypted bytes and nothing else.

TLS also guarantees **integrity**: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection. Nobody can quietly change the account number in your bank transfer while it is in flight.

The café could see everything else: this is metadata

This is the part that gets left out, and it is the whole second half of the interview question.

Even with perfect encryption, an observer on the network sees:

VISIBLE	WHY
The IP address you connected to	The network has to route to it
The domain name	Sent in the clear during the handshake, in a field called SNI
The DNS lookup, unless you use encrypted DNS	Traditional DNS is plain UDP
When you connected and for how long	Timing is not encryptable
How many bytes went each way	Sizes are visible even when contents are not

So your ISP cannot read your messages and absolutely can tell that you visited a particular site, at 4:30 pm, for twenty-five minutes, and downloaded 40 MB. Traffic analysis on those sizes and timings can often infer which page you loaded, even inside HTTPS.

There is a newer feature called **Encrypted Client Hello** that hides the SNI field, and it is slowly being deployed, but it is not something you can assume today.

Verified who, not verified what: the padlock's real limit

Nandini's closing thought is the sentence that answers the interview question.

A padlock means: *the connection to the name in the address bar is private and unaltered.*

It does **not** mean:

- **the site is honest.** Anyone can get a free certificate for `paypal-secure.com` in ninety seconds. Most phishing sites have valid certificates, precisely because users were taught to look for the padlock.
- **your data is safe once it arrives.** TLS protects data *in transit*. The moment it reaches the far end it is decrypted, and what happens next — plain-text passwords, an unencrypted backup, a leaky log file — is entirely outside TLS's scope.
- **the site is not compromised.** A site serving malware over HTTPS serves it with a perfect padlock.
- **nobody knows where you went.** See metadata, above.

Two kinds of key, and why both exist

One more piece, without the mathematics.

Asymmetric encryption uses a pair of keys: a **public key** anyone may have, and a **private key** only the owner holds. Anything scrambled with the public key can only be unscrambled with the private one. This solves the impossible-looking problem of agreeing a secret with someone you have never met — but it is slow, perhaps a hundred times slower than the alternative.

Symmetric encryption uses one shared key for both directions. It is very fast. The problem is agreeing the key in the first place, over a network everyone can read.

TLS uses both, and the division of labour is the answer to a common follow-up: **asymmetric keys are used briefly, during the handshake, to agree a symmetric key. Then everything else uses the fast symmetric key.** Expensive once, cheap thereafter.

4 The picture

The TLS 1.3 handshake, which is what actually happens before your first HTTP byte:

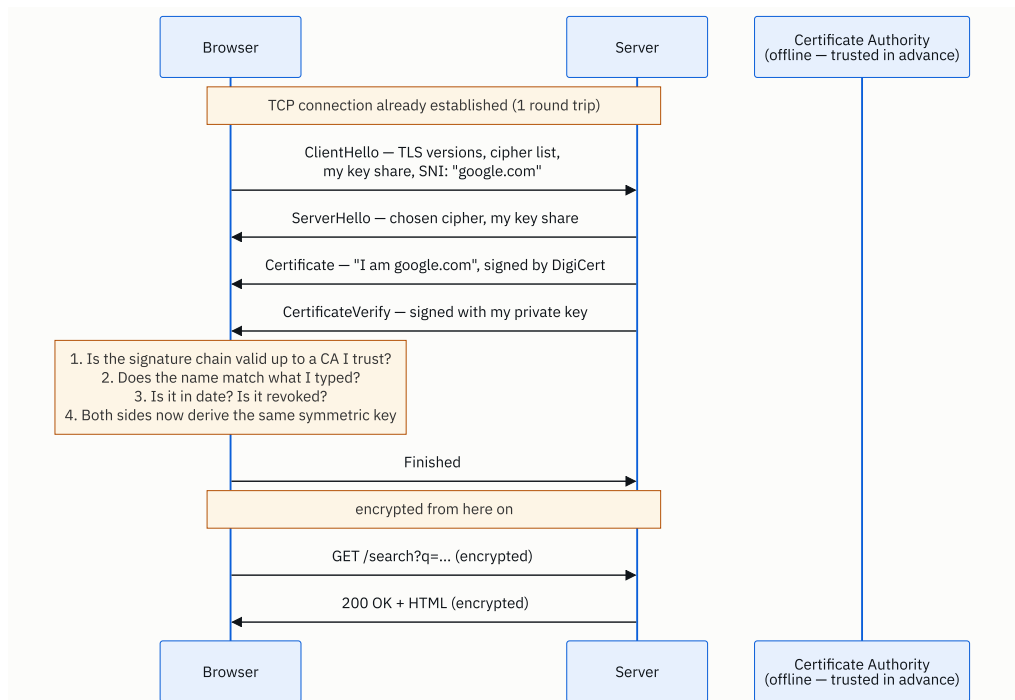


FIGURE 6.1

What to notice: the certificate is checked, not fetched. The CA is not contacted during the handshake at all — its signature was applied months earlier and the browser already holds the CA's key. That is why HTTPS still works when the CA's own site is down.

Now what an observer on the network actually sees:

+-----+-----+		
	VISIBLE to your ISP, the wifi owner, anyone in between	
	your IP address → 142.250.183.206	
	destination port → 443	
	the domain name (SNI) → "google.com"	
	the DNS lookup → "who is google.com?"	
	the timing → 16:31:02, lasted 25 minutes	
	the volume → 4 KB up, 2.1 MB down	
+-----+-----+		
	HIDDEN – encrypted	
	the path → /account/settings/billing	
	query parameters → ?card=...	
	all headers → Cookie, Authorization	
	the request body → {"password": "..."}	
	the entire response → your account page	
+-----+-----+		

What to notice: the top box is the answer to “what does it *not* protect you from”. Being able to draw that line is the whole point of the lesson.

And the chain of trust, which is what the browser is actually checking:

```

Root CA          DigiCert Global Root G2
|                (in your browser's store – trusted because it shipped there)
| signs
v
Intermediate    DigiCert TLS RSA SHA256 2020 CA1
|                (sent by the server during the handshake)
| signs
v
Leaf             CN = *.google.com
                  (sent by the server; this is the one being checked)

```

The browser verifies each signature upward until it reaches something already in its trust store. If the chain breaks, or the name does not match, or the date has passed → connection refused.

What to notice: the root is never sent over the network. It is already on your device. That is the entire foundation, and it is also the entire weakness — anyone who can add a root to your device can read your traffic.

5 How it actually works

What is actually in a certificate

You can look at one yourself: `openssl s_client -connect google.com:443 -showcerts`, or click the padlock in any browser. Inside:

- **Subject** — the name being claimed, e.g. `CN=*.google.com`
- **Subject Alternative Names** — the full list of names it covers; this is what browsers actually check, not the old Common Name field
- **Public key** — the server's public key
- **Issuer** — which CA signed it
- **Validity** — not before, not after. Ninety days for Let's Encrypt, up to about a year for paid ones
- **Signature** — the CA's signature over all of the above

Three checks fail loudly and you have seen all three: name mismatch, expired, and untrusted issuer. Certificate expiry is one of the most common self-inflicted production outages in the industry, which is why automated renewal is not optional.

How you get one

Let's Encrypt issues certificates free and automatically, and it changed the web — HTTPS went from a minority of traffic to the overwhelming majority in a few years. The client, usually **certbot** or **Caddy**, proves domain control by placing a specific file at a specific URL on the domain, or by adding a DNS record. If the CA can fetch it, you evidently control the domain, and a certificate is issued. That entire proof takes seconds and involves no human.

Notice what that proves and what it does not. It proves control of a name. It says nothing about the operator.

Where TLS actually terminates

In production TLS is almost never handled by your application. It is **terminated** at the edge — by **nginx**, **HAProxy**, an **AWS Application Load Balancer**, or a CDN such as **Cloudflare** — which decrypts the request and forwards it inside your network.

That has two consequences worth naming in an interview. First, your application code sees plain HTTP and must read `X-Forwarded-Proto` to know the original request was secure. Second, traffic between the load balancer and your application is unencrypted unless you arrange otherwise, which is fine inside a private network and is exactly what **mTLS** and service meshes such as **Istio** and **Linkerd** exist to fix when it is not.

mTLS — mutual TLS — is the same handshake with the check running both ways: the client also presents a certificate and the server verifies it. It is the standard way services authenticate to each other inside a company, because it removes shared passwords entirely.

Version 1.3, and why it matters

TLS 1.3 (2018) removed every cipher that had been broken, cut the handshake from two round trips to one, and added **0-RTT resumption**, where a returning client can send data in its very first message. TLS 1.0 and 1.1 are deprecated and disabled in modern browsers; 1.2 is still widely used and acceptable; 1.3 is what you want.

The 0-RTT feature has a genuine caveat worth knowing: early data can be **replayed** by an attacker, so it must only be used for idempotent requests. That connects directly to [day 005](#) — it is the same reason `POST` needs an idempotency key.

HSTS, and the gap it closes

If a user types `example.com`, the browser tries HTTP first, and the site redirects to HTTPS. That first plain request is a window for an attacker to intercept and never let the upgrade happen. **HSTS** — a `Strict-Transport-Security` header — tells the browser “never use plain HTTP for this domain again, for the next year”. After the first visit, the window is closed. Sites can also be added to a **preload list** that ships inside the browser, closing it even for the first visit.

When it fails, and the legitimate man-in-the-middle

Your workplace may install its own root certificate on your laptop. From then on, the company’s proxy can present its own certificate for any site, your browser will trust it because the root is in the store, and the proxy can read everything. This is normal corporate practice, it is how **Burp Suite** and **mitmproxy** work for legitimate testing, and it is also precisely the attack TLS is designed to prevent. **The system rests entirely on which roots are in your trust store.**

The defence for a mobile app that cares is **certificate pinning**: the app ships with the expected certificate or public key and refuses anything else, regardless of the trust store. Banking apps do this. The cost is that rotating a certificate now requires an app update, and getting it wrong bricks your app for everyone.

6 The numbers

What the handshake costs in time. At 40 ms round trip:

TCP handshake	1 RTT	=	40 ms	
TLS 1.2 handshake	2 RTT	=	80 ms	

before the first request:			120 ms	
 TCP handshake	 1 RTT	 =	 40 ms	
TLS 1.3 handshake	1 RTT	=	40 ms	

before the first request:			80 ms	(a third less)
 TLS 1.3 resumption	 0 RTT	 =	 0 ms	

before the first request:			40 ms	(TCP only)

Over a 250 ms intercontinental link, TLS 1.2 costs 500 ms of setup and TLS 1.3 costs 250 ms. That difference is visible to a user, and it is why session resumption is worth configuring.

What it costs in CPU. The asymmetric part is the expensive bit, and it happens once per connection:

handshake (ECDSA P-256)	~ 1 ms of CPU per connection
bulk encryption (AES-GCM with hardware support)	~ 1-2 GB/s per core

So on a machine handling 10,000 new connections per second:

10,000 x 1 ms = 10 seconds of CPU per second = 10 cores, just for handshakes
--

Ten cores for handshakes alone. And the same machine serving 1 Gbps of already-established traffic:

1 Gbps / 8 = 125 MB/s / 1,500 MB/s per core = 0.08 cores for the encryption itself
--

The handshakes cost a hundred times more than the encryption. That single comparison is why keep-alive and session resumption matter so much, and it is the number to quote when someone claims HTTPS is expensive. Sustained encryption is essentially free; setting up connections is not.

Session resumption, with the arithmetic. If 90% of connections resume:

without resumption: 10,000 full handshakes/s	= 10 cores
with 90% resumption: 1,000 full + 9,000 resumed	
1,000 x 1 ms + 9,000 x 0.05 ms = 1.45 s of CPU	= 1.5 cores

From ten cores to one and a half, by enabling one feature.

Certificate sizes. A chain of leaf plus intermediate is roughly 2–4 KB, sent on every new connection:

```
10,000 new connections/s x 3 KB = 30 MB/s of certificate traffic
```

Not enormous, but a real reason to prefer ECDSA certificates (about 800 bytes) over RSA (about 2 KB) at scale, and a reason not to send the root certificate you do not need to send.

Expiry, as an operational number. Let's Encrypt issues for 90 days:

```
90 days, renewed automatically at day 60  
→ a renewal failure gives you 30 days of warnings before an outage
```

Thirty days of margin is generous, and expiry outages still happen constantly — because nobody was reading the warnings.

7 The trade-offs

TLS costs a round trip and buys you the entire security model of the web. The honest accounting: 1 extra round trip on connection with TLS 1.3, about 1 ms of CPU per new connection, and near-zero cost per byte thereafter. In exchange you get confidentiality, integrity and server identity. There is no serious argument for plain HTTP on a public site in 2026 — browsers mark it insecure, HTTP/2 and HTTP/3 effectively require TLS, and the CPU argument died when AES instructions went into every processor.

Free automated certificates versus paid ones. Let's Encrypt is free, automated, and 90-day, which forces you to automate renewal — that constraint is a feature. Paid CAs offer longer validity, warranties, support, and **Extended Validation**, which involves a human checking the legal entity. EV has largely lost its point: browsers stopped displaying the company name in the address bar, because research showed users never noticed it. For almost everyone, free and automated is the right answer.

Terminating TLS at the edge versus end to end. Terminating at a load balancer or CDN centralises certificate management, allows caching and compression, and lets you inspect traffic for routing and rate limiting. The cost is that traffic inside your network is plain, so anyone with access to that network sees everything. **Zero-trust** architectures reject that assumption and require mTLS between every pair of services — which is materially more operational work, and correct if your compliance regime or threat model demands it.

Certificate pinning: strong and dangerous. Pinning defends against a compromised or coerced CA and against corporate interception, which is exactly right for a banking app. It also means that if you rotate your certificate without shipping an app update first, every installed copy of your app stops working, and you cannot fix it remotely because the app will not trust the connection that would deliver the fix. Pin to a backup key as well, or do not pin.

I would not rely on HTTPS alone if... the threat is anything other than an observer on the network. It does nothing about a compromised endpoint, a malicious site with a valid certificate, a database breach, a logged password, or a user who was phished. Those need different controls entirely: hashing passwords, encryption at rest, least privilege, multi-factor authentication. HTTPS secures the pipe. It has no opinion about what is at either end of it.

8 In the interview

How it gets asked

- “What does HTTPS protect you from, and what does it not?” — the direct version. The second half is the actual question.
- “Walk me through what happens when you connect to an HTTPS site.” — the handshake version, often a follow-up to “what happens when you type google.com”.
- “A site has a valid padlock. Is it safe?” — the good version, and the answer is no.
- “Why does TLS use both symmetric and asymmetric encryption?” — the mechanism check.

What to say out loud, in the first ninety seconds

1. **Give the three guarantees.** “TLS gives you three things: identity, confidentiality and integrity. Those three and nothing else.”
2. **Explain identity through the chain.** “The server sends a certificate signed by a CA my browser already trusts. It checks the signature chain, that the name matches what I typed, and that it hasn’t expired.”
3. **Say what is encrypted, specifically.** “Everything above the transport — the path, all headers including cookies, the request body and the whole response.”
4. **Then, unprompted, say what is not.** “What’s still visible is the IP address, the domain name in the SNI field of the handshake, the DNS lookup unless it’s encrypted, and the timing and volume of traffic. So my ISP can’t read my messages but absolutely knows which sites I visited and when.”

5. **Land the important limit.** *“And the padlock says nothing about whether the site is honest. Anyone can get a free certificate for a lookalike domain in ninety seconds — most phishing sites have valid ones. It proves the connection is private, not that the other end deserves your data.”*
6. **Add the endpoint limit.** *“It also only covers data in transit. Once it arrives it’s decrypted, and what happens then — hashing, encryption at rest, logging — is a separate problem.”*

Steps 4 and 5 are the answer. Steps 1 to 3 are what everyone says.

The follow-ups

“Why does TLS use both symmetric and asymmetric encryption?” Because they solve different problems and each is bad at the other’s. Asymmetric encryption lets two parties who have never met agree on a secret over a channel everybody can read, which is otherwise impossible — but it is roughly a hundred times slower. Symmetric encryption is very fast and needs a shared key you have no way to agree on. So TLS uses asymmetric operations during the handshake to authenticate the server and establish a shared symmetric key, then uses that fast key for the whole session. Expensive once, cheap thereafter — which is exactly why handshakes dominate the CPU cost and bulk encryption barely registers.

“A site has a valid certificate. Is it safe to enter my card details?” No, and those are two different questions. The certificate says the connection to whatever is in the address bar is private and unaltered. It says nothing about who that is. Certificates are free and automated, so `paypal-secure.com` can have a perfect one within minutes, and attackers use that deliberately because users were taught to look for the padlock. The padlock answers “is anyone listening?”, not “should I trust this site?”.

“How does the browser know it can trust the CA?” It doesn’t decide — it ships with a store of roughly 150 root certificates chosen by the browser or OS vendor. The chain is checked upward until it reaches one of those roots. Which means the whole system rests on that store, and anyone who can add a root to your device can transparently read your traffic. That is exactly how corporate inspection proxies work: they install a company root on managed laptops. The countermeasures are certificate pinning for apps that need it, and Certificate Transparency logs, which make every issued certificate publicly auditable so that a domain owner can detect a certificate they never asked for.

“Can an attacker on the same wifi see what I’m doing?” They cannot read your traffic — content, paths, cookies and bodies are all encrypted. They can see which sites you connect to, from the DNS lookups and from the SNI field in the handshake, which is sent before encryption begins. They can see timing and volume, and traffic analysis on

those can often identify which specific page you loaded. So the honest answer is: they cannot read it, and they can build a fairly detailed picture of what you did. Encrypted DNS and Encrypted Client Hello close some of that gap, and neither is universal yet.

A model answer

“HTTPS is HTTP over TLS, and TLS gives you exactly three things: identity, confidentiality and integrity.

Identity comes from certificates. The server sends one that says ‘I am google.com’, signed by a certificate authority. My browser ships with a store of roots it already trusts, and it verifies the signature chain up to one of those, checks the name matches what I typed, and checks the dates. The CA isn’t contacted during the handshake — that signature was applied months ago.

Confidentiality comes from encryption, and it covers more than people assume: the path, all the headers including cookies and auth tokens, the body, and the entire response. Integrity means each piece carries a check value, so any modification in transit breaks the connection rather than passing quietly.

Now what it doesn’t protect. First, metadata. The IP address is visible, the domain name is visible because SNI is sent in the clear during the handshake, the DNS lookup is usually visible, and timing and byte counts are visible. So an observer can’t read my messages and can absolutely tell I spent twenty minutes on a particular site and downloaded two megabytes. Traffic analysis on sizes can often narrow that to a specific page.

Second, and more importantly, the padlock says nothing about trustworthiness. Certificates are free and issued in ninety seconds to anyone who controls a domain, so a phishing site at a lookalike domain has a perfect padlock. The CA vouches for identity, never for character.

Third, it only covers data in transit. The moment it lands it’s decrypted, and whether the password is hashed, whether the database is encrypted at rest, whether it ends up in a log file — none of that is TLS’s problem.

One thing I’d raise in a design discussion: in production TLS usually terminates at the load balancer or CDN, not at the application. So my app sees plain HTTP and has to read X-Forwarded-Proto, and traffic inside the network is unencrypted unless we’ve set up mTLS. Whether that’s acceptable depends on whether we’re treating the internal network as trusted.”

That answer covers both halves of the question, gets the metadata point in without being asked, lands the phishing limitation, and finishes on a real architectural consequence.

9 Recall card

1. **TLS gives you three things: identity, confidentiality, integrity.** The padlock means those three succeeded and nothing more.
2. **Identity comes from a chain of trust** — leaf signed by intermediate, signed by a root already in your device's store. The CA vouches for *who*, never for *honest*.
3. **Encrypted:** path, headers, cookies, body, response. **Visible:** IP address, domain name via SNI, DNS lookup, timing, byte counts.
4. **A valid certificate does not mean a safe site.** Free automated certificates mean phishing sites have padlocks too.
5. **Asymmetric for the handshake, symmetric for the data.** Handshakes cost ~ 1 ms of CPU each and dominate; bulk encryption is nearly free. That is why resumption and keep-alive matter.

PRACTICE

Practice — Python for DSA II: strings, dictionaries, and sets

DSA topic: Python for DSA II: strings, dictionaries, and sets **System design topic:** HTTPS and TLS, without the maths

Code these, in this order

Four problems that all reduce to “have I seen this before?” or “how many times have I seen this?”. Every one of them has a nested-loop solution that works and times out.

For each problem:

1. Say out loud which structure the question implies — **set** if you care *whether*, **Counter** or **dict** if you care *how many*, **defaultdict** if you are grouping.
2. Write it with that structure first. Do not write the nested loop.
3. State the time and space complexity before you run it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Contains Duplicate	LeetCode 217 (Easy)	The reflex, in its purest form. If this does not produce a <code>set</code> within five seconds, drill it until it does.
2	Valid Anagram	LeetCode 242 (Easy)	Set versus Counter. <code>set(a) == set(b)</code> passes some tests and is wrong — find the input that breaks it before you look it up.
3	Two Sum	LeetCode 1 (Easy)	A dictionary storing value → index. The insight is that you look for what you <i>need</i> , not what you <i>have</i> .
4	Group Anagrams	LeetCode 49 (Medium)	<code>defaultdict(list)</code> plus a canonical key. The whole problem is choosing what the key should be.

On problem 2, do this properly

- Write `return set(a) == set(b)` and submit it. Note which test case fails.
- Work out from that failing case exactly what a set forgot.
- Rewrite with `Counter` and submit again.
- Say the one-sentence rule out loud: **set when you care whether, Counter when you care how often.**

The reflex drill

Answer each of these with a structure name in under three seconds. No code, just the name.

- “Find the first character that appears twice.” →
- “Group words that are anagrams of each other.” →
- “Does this array contain any repeated value?” →
- “Which number appears most often?” →
- “Is every element unique after removing one?” →
- “Store the positions I have already visited in a grid.” →

Then check: the answers are set, defaultdict(list), set, Counter, set or Counter, and a set of (row, col) tuples. If you hesitated on more than one, the reflex is not built yet.

The measurement drill

Run the complete program from §5 at `N = 20_000`, then at `N = 40_000`.

The two $O(n^2)$ rows should quadruple. The two $O(n)$ rows should double. Confirm it, then answer out loud: **which single character is the difference between the second row and the third?**

The one to try in a browser

Open any HTTPS site and click the padlock, then “certificate details”. Find four things: the subject name, the issuer, the validity dates, and the chain above it. Then answer: how many links are there between this certificate and something your browser already trusted before you opened it?

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How would you check for duplicates in $O(n)$?* Name the structure first, describe the pass, give both complexities, say why membership is $O(1)$, give the honest caveat, then give the $O(1)$ -space alternative.
2. *What does HTTPS protect you from? What does it not protect you from?* Three guarantees, then the metadata list, then the phishing point. Do not stop after the first half.
3. *A site has a valid padlock. Is it safe to enter your card details?* One sentence on what the certificate actually proves, one on what it does not, and one on who checked what before it was issued.

Before you move on

- [] “Have I seen this before?” makes me think set before I think about loops.

- [] I can say why `x in some_set` is `O(1)` and `x in some_list` is `O(n)`, from the mechanism.
- [] I know why a list cannot be a dictionary key, and what to use instead.
- [] I never build a string with `+=` in a loop. I reach for `"".join(pieces)`.
- [] I can list four things an observer sees even when the connection is encrypted.

Space complexity, and what in-place really means

DATA STRUCTURES AND ALGORITHMS

Space complexity, and what in-place really means

You can state the extra memory your solution uses and rewrite it to use less.

SYSTEM DESIGN

What a web server actually does

You can describe the loop a server runs: listen, accept, handle, respond.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Can you do that in $O(1)$ extra space?*
- *What happens inside the server between receiving a request and sending a response?*

Space complexity, and what in-place really means

1 What this is, and why they ask it

Space complexity is the same idea as time complexity, applied to memory: how much extra storage your solution needs as the input grows. **Extra space** — also called **auxiliary space** — means everything you allocate, not counting the input you were handed.

A solution uses **$O(1)$ extra space** when the amount it allocates does not depend on the input size. Ten items or ten million, it keeps the same handful of variables. That is what **in-place** means, and it is the phrase interviewers use when they want you to stop allocating.

They ask because “can you do it in $O(1)$ extra space?” is the cleanest second question in existence. Your first solution works and you have stated its time complexity. Now the interviewer applies one more constraint, and it separates two things: whether you know what your own code allocates, and whether you can restructure a solution rather than rewrite it. A candidate who says “sure — I’d swap in place with two indices instead of building a new array” has answered in one sentence. A candidate who says “it’s $O(n)$ time” has answered a different question.

2 The story

Farida has one narrow cupboard in her kitchen, and every steel vessel she owns lives in it. Two shelves, and on the lower one, twenty-two vessels standing in a row.

They have been in no order for years. On Sunday morning she decides she has had enough of reaching past the big handi for the small tumbler, and she is going to put them in order of size, smallest on the left.

Her usual way of doing a job like this is to take everything out first. She would put all twenty-two on the counter, look at them all together, and put them back one at a time in the right order. It is easy. She has done it before, and it takes about fifteen minutes.

But this is a Sunday in September, and her sister-in-law is in the kitchen making biryani for eleven people. Every inch of counter is taken. There are three vessels of half-cut onions, a tray of marinated chicken, the big pot, two mixing bowls and a plate she is not allowed to move. There is no counter. There is not going to be a counter until three in the afternoon.

So she has to do the whole thing inside the cupboard.

It turns out to be possible, and the way it works is this. There is one gap at the right-hand end of the shelf, about the width of one vessel — a space that has always been there because twenty-two vessels do not quite fill twenty-three vessels' worth of shelf. That gap is the only free space she has.

To swap two vessels she moves the first one into the gap, slides the second into the space the first left, and puts the one from the gap into the second's old place. Three moves, and the gap ends up back where it started. It is slower than the counter method — she does a great deal more lifting — but at no point does she need anywhere to put anything except that one gap.

She finishes at half past eleven. It took her thirty-five minutes rather than fifteen, and the shelf is in order.

What stays with her, drinking tea afterwards, is that the job was never impossible. It was only impossible *the way she usually did it*. The counter was not part of the job. It was a habit she had never noticed she had.

3 The idea in plain English

Farida's cupboard is space complexity, and the gap at the end of the shelf is $O(1)$ extra space.

Two kinds of space, and only one of them is being asked about

Input space is the memory the input already occupies. Farida's twenty-two vessels. You did not allocate it, you were handed it, and you cannot avoid it.

Extra space, or **auxiliary space**, is everything your solution allocates on top of that. Farida's counter. **When an interviewer asks about space complexity, this is what they mean**, unless they specifically say "total space".

So the answer to "what's the space complexity?" for a function that walks an array keeping a running total is $O(1)$, not $O(n)$. The array was already there.

Say which one you mean. " $O(1)$ extra space — I'm not counting the input array" is precise and takes two extra seconds.

O(1) extra space means a fixed number of variables

```
def total(items: list[int]) → int:
    running = 0          # one integer
    for x in items:      # one loop variable
        running += x
    return running
```

Two variables. That does not change when `items` gets longer. **O(1) extra space.**

Now the version that is not:

```
def doubled(items: list[int]) → list[int]:
    out = []             # this grows with the input
    for x in items:
        out.append(x * 2)
    return out
```

`out` ends up as long as `items`. **O(n) extra space.**

The gap: why a swap needs exactly one spare place

Farida could not exchange two vessels without somewhere to put one of them for a moment. Neither can a computer. Swapping `a` and `b` needs a temporary:

```
temp = items[i]
items[i] = items[j]
items[j] = temp
```

Three moves, one temporary variable. Python lets you write it on one line — `items[i], items[j] = items[j], items[i]` — and it does the same thing underneath. The important point is that **one** temporary is enough, and one temporary is **O(1)** however long the array is.

That single fact is why almost every in-place algorithm you will meet is built out of swaps.

In-place means you modify what you were given

An **in-place** operation changes the input itself rather than producing a new thing. `items.sort()` is in place. `sorted(items)` is not — it returns a new list and leaves the original alone.

IN PLACE (MODIFIES)	NOT IN PLACE (RETURNS NEW)
<code>items.sort()</code>	<code>sorted(items)</code>
<code>items.reverse()</code>	<code>items[::-1]</code>
<code>items.append(x)</code>	<code>items + [x]</code>
<code>items[i] = v</code>	<code>items[:i] + [v] + items[i+1:]</code>

The right-hand column all cost $O(n)$ extra space. The left-hand column costs $O(1)$.

There is one important caveat, and interviewers like it: `items.sort()` is in place but **not $O(1)$ space**. Python’s Timsort needs a temporary area of up to $n/2$, so it is $O(n)$ auxiliary. If somebody insists on strictly $O(1)$ extra space, sorting is off the table, and heapsort is the standard $O(n \log n)$ answer with genuinely constant extra space.

The space you did not know you were allocating

Three sources of hidden memory, all of which come up.

Slicing. `items[1:]` copies. From [day 005](#): a slice is a new list, so a function that slices is not $O(1)$ space no matter how it looks.

Recursion. Every pending function call occupies a frame in memory until it returns. A recursion that goes n deep uses $O(n)$ space even if it allocates nothing itself. This is the one people miss most often, and it is why “recursive binary search is $O(1)$ space” is wrong — it is $O(\log n)$ space, because of the frames. [Day 088](#) covers this properly.

Sets and dictionaries you built to be fast. Yesterday’s duplicate-detection answer spends $O(n)$ space deliberately. That is the right call and it is still $O(n)$.

Does the output count?

By convention, **no** — the space required to hold the answer is not counted as extra space, because you have no choice about it. If a problem asks you to return a new array of n elements, that array is not held against you.

But anything *beyond* the output does count. Building a dictionary in order to produce a list is $O(n)$ extra even though the list itself is free. Say it explicitly: “ $O(n)$ for the output, plus $O(n)$ for the hash map I used along the way” is unambiguous, and unambiguous is what you want.

4 The picture

Reversing an array with the counter, and then inside the cupboard:

NOT IN PLACE – $O(n)$ extra space

input +---+---+---+---+---+---+
| 3 | 8 | 1 | 9 | 4 | 7 |
+---+---+---+---+---+---+

the vessels

| copy each one
v

new +---+---+---+---+---+---+
| 7 | 4 | 9 | 1 | 8 | 3 |
+---+---+---+---+---+---+

the counter: a whole second shelf

IN PLACE – $O(1)$ extra space

+---+---+---+---+---+---+
| 3 | 8 | 1 | 9 | 4 | 7 |
+---+---+---+---+---+---+
^ ^
left right

one temp variable: []

swap them, move both inward

+---+---+---+---+---+---+
| 7 | 8 | 1 | 9 | 4 | 3 |
+---+---+---+---+---+---+
^ ^
left right

swap, move inward

+---+---+---+---+---+---+
| 7 | 4 | 1 | 9 | 8 | 3 |
+---+---+---+---+---+---+
^ ^
left right

swap, and they cross – done

+---+---+---+---+---+---+
| 7 | 4 | 9 | 1 | 8 | 3 |
+---+---+---+---+---+---+

same answer, no second shelf

What to notice: both pictures end with the same values. The top one needed a second array of six slots; the bottom one needed one temporary variable and two indices, and would need exactly the same for six million.

Now the hidden space in recursion, which is the one people forget:

```
countdown(4)
```

memory (the call stack)	what is waiting
+-----+	
countdown(0) ← running	nothing pending
+-----+	
countdown(1)	waiting to add 1
+-----+	
countdown(2)	waiting to add 1
+-----+	
countdown(3)	waiting to add 1
+-----+	
countdown(4)	waiting to add 1
+-----+	

5 frames alive at once, for $n = 4$. n frames for n . $O(n)$ space, even though not one line of that function allocates anything.

What to notice: the function body has no list, no set, no slice. The memory is spent by the calls themselves. Every recursive solution owes an answer to “how deep does it go?”.

5 The code, built step by step

Build the same job three ways and measure the memory, because arguing about space without measuring it is how people convince themselves they are in place when they are not.

Python has a memory tracker in the standard library:

```
import tracemalloc

def peak_kb(fn) → float:
    tracemalloc.start()
    fn()
    _current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1024
```

`tracemalloc.get_traced_memory()` returns the current and the **peak** allocation since tracking started. Peak is the number that matters: it is the high-water mark, and it is what a memory limit actually tests against.

Now three reversals. First, the one that builds a new list.

```
def reverse_new(items: list[int]) → list[int]:
    out = []
    for i in range(len(items) - 1, -1, -1):
        out.append(items[i])
    return out
```

`range(len(items) - 1, -1, -1)` counts down: start at the last index, stop before `-1`, step by `-1`. `out` grows to length `n`, so this is **$O(n)$ extra space**.

Second, the one-liner, which is the same cost wearing better clothes.

```
def reverse_slice(items: list[int]) → list[int]:
    return items[::-1]
```

`[::-1]` is a slice with a step of `-1`. It is a slice, so it copies. **$O(n)$ extra space**, and it is worth knowing that the tidiest-looking solution here is not the cheapest one.

Third, in the cupboard.

```
def reverse_in_place(items: list[int]) → None:
    left, right = 0, len(items) - 1
    while left < right:
        items[left], items[right] = items[right], items[left]
        left += 1
        right -= 1
```

Two indices and the temporary that the swap uses internally. **$O(1)$ extra space**. Note the return type is `None`: it changes the caller's list and returns nothing, which is the Python convention for in-place operations and is exactly what `list.sort()` does.

Now the same three-way comparison for a harder job — removing duplicates from a sorted array.

```
def dedupe_new(items: list[int]) → list[int]:
    out = []
    for x in items:
        if not out or out[-1] ≠ x:
            out.append(x)
    return out
```

`out[-1]` is the last element. This is clear and it allocates a full second array.

```
def dedupe_in_place(items: list[int]) → int:
    """Compact in place. Returns the new length; items[:length] is the answer."""
    if not items:
        return 0
    write = 1 # where the next kept value goes
    for read in range(1, len(items)):
        if items[read] != items[write - 1]:
            items[write] = items[read]
            write += 1
    return write
```

This is the **write pointer**, and it is the single most reusable in-place pattern there is. One index reads forward through everything; another marks where the next keeper belongs. Because `write` never overtakes `read`, you are always writing into a slot you have already passed. `O(1)` extra space, and it is the whole subject of [day 015](#).

Here is the complete program.

```

"""Day 7 – measure what your solution actually allocates."""

import sys
import tracemalloc

def peak_kb(fn) → float:
    """Peak extra memory allocated while fn runs, in kilobytes."""
    tracemalloc.start()
    fn()
    _current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1024

# ---- reversing -----

def reverse_new(items: list[int]) → list[int]:
    """O(n) extra: builds a second list."""
    out: list[int] = []
    for i in range(len(items) - 1, -1, -1):
        out.append(items[i])
    return out

def reverse_slice(items: list[int]) → list[int]:
    """O(n) extra: a slice is a copy, however short the line is."""
    return items[::-1]

def reverse_in_place(items: list[int]) → None:
    """O(1) extra: two indices and one swap temporary."""
    left, right = 0, len(items) - 1
    while left < right:
        items[left], items[right] = items[right], items[left]
        left += 1
        right -= 1

# ---- removing duplicates from a sorted array -----

def dedupe_new(items: list[int]) → list[int]:
    """O(n) extra."""
    out: list[int] = []
    for x in items:
        if not out or out[-1] ≠ x:
            out.append(x)
    return out

def dedupe_in_place(items: list[int]) → int:
    """O(1) extra. Returns the new length; items[:length] holds the answer."""
    if not items:
        return 0
    write = 1
    for read in range(1, len(items)):
        if items[read] ≠ items[write - 1]:
            items[write] = items[read]
            write += 1
    return write

# ---- the hidden space in recursion -----

```



```

def depth_of(n: int) → int:
    """0(n) space in call frames, even though it allocates nothing itself."""
    if n == 0:
        return 0
    return 1 + depth_of(n - 1)

if __name__ == "__main__":
    N = 200_000
    data = list(range(N))
    sorted_with_dupes = sorted([i // 3 for i in range(N)])

    # The fresh copy is made BEFORE tracing starts. Otherwise the copy itself
    # shows up as the function's allocation and every row looks like 0(n).
    print(f"reversing {N:,} integers")
    for label, fn in (("build a new list" 0(n)", reverse_new),
                     ("slice [::-1]" 0(n)", reverse_slice),
                     ("two pointers" 0(1)", reverse_in_place)):
        work = list(data)
        print(f" {label} : {peak_kb(lambda: fn(work)):>9,.0f} KB")

    print(f"\nremoving duplicates from {N:,} sorted integers")
    for label, fn in (("build a new list" 0(n)", dedupe_new),
                     ("write pointer" 0(1)", dedupe_in_place)):
        work = list(sorted_with_dupes)
        print(f" {label} : {peak_kb(lambda: fn(work)):>9,.0f} KB")

    print("\nchecking the in-place versions are correct")
    small = [3, 8, 1, 9, 4, 7]
    reverse_in_place(small)
    print(f" reversed in place : {small}")
    dupes = [1, 1, 2, 2, 2, 3, 4, 4]
    length = dedupe_in_place(dupes)
    print(f" deduped in place : length {length}, values {dupes[:length]}")

    print("\nhow deep can recursion go before Python stops it?")
    print(f" the limit : {sys.getrecursionlimit():,} frames")
    print(f" depth_of(900) : {depth_of(900)}")

```

This is exactly what it printed:

```

reversing 200,000 integers
  build a new list  0(n) :      1,586 KB
  slice [::-1]     0(n) :      1,562 KB
  two pointers     0(1) :           0 KB

removing duplicates from 200,000 sorted integers
  build a new list  0(n) :        549 KB
  write pointer     0(1) :           0 KB

checking the in-place versions are correct
  reversed in place : [7, 4, 9, 1, 8, 3]
  deduped in place  : length 4, values [1, 2, 3, 4]

how deep can recursion go before Python stops it?
  the limit         : 1,000 frames
  depth_of(900)     : 900

```

Look at the zeros. They are not rounding. The in-place versions allocate nothing that grows with the input — two integers, and those live on the stack rather than in tracked allocations. Now change `N` to 400,000 and run it again: the `O(n)` rows double and the zeros stay zero. That is the doubling test from [day 003](#), applied to memory.

6 What it costs

The space table for the operations you use most.

OPERATION	EXTRA SPACE
<code>items[i], items[j] = items[j], items[i]</code>	<code>O(1)</code>
<code>items.reverse()</code> , <code>items.append()</code> , <code>items.pop()</code>	<code>O(1)</code>
<code>items[::-1]</code> , <code>items[a:b]</code> , <code>items.copy()</code>	<code>O(k)</code> — a copy
<code>sorted(items)</code>	<code>O(n)</code> — a new list
<code>items.sort()</code>	<code>O(n)</code> — in place, but Timsort needs a buffer
Heapsort	<code>O(1)</code> — the genuinely constant-space <code>O(n log n)</code> sort
<code>set(items)</code> , <code>Counter(items)</code>	<code>O(n)</code>
Recursion <code>n</code> deep	<code>O(n)</code> in call frames
Recursion <code>log n</code> deep (binary search)	<code>O(log n)</code>
Merge sort	<code>O(n)</code>
Quicksort	<code>O(log n)</code> — for the recursion, if you recurse on the smaller side

What memory actually costs, in bytes. Python objects are not small:

a Python int object	28 bytes
a reference in a list	8 bytes
so one integer in a list	36 bytes
a small tuple (2 ints)	56 bytes + the ints
one entry in a set	~60 bytes with the table overhead
one entry in a dict	~100 bytes with the table overhead

Now the number that decides submissions. A typical memory limit is 256 MB:

256 MB / 36 bytes per int in a list	= about 7,000,000 integers
256 MB / 100 bytes per dict entry	= about 2,700,000 dictionary entries

So at $n = 10^6$ a list of integers is comfortable, a dictionary keyed by every element is comfortable, and an $n \times n$ grid is not:

```
1,000,000 x 1,000,000 = 10^12 cells. Not close. Not even for booleans.
```

An $O(n^2)$ space solution dies at about $n = 2,500$, well before an $O(n^2)$ time solution dies at about 5,000. **Space is usually the tighter constraint, and people check it second.**

The recursion arithmetic. Each Python call frame is roughly 500 bytes:

```
1,000 frames (the default limit) x 500 bytes = 500 KB    fine
100,000 frames                               = 50 MB     would be fine, if allowed
```

Python's limit is about 1,000, and it exists to turn runaway recursion into an error rather than a crash. You can raise it with `sys.setrecursionlimit(200000)`, and you usually should not: the real C stack underneath has its own limit, and exceeding that gives you a hard segmentation fault instead of a clean exception. **Rewrite deep recursion as a loop.**

The trade, stated as arithmetic. Two Sum with a hash map, at $n = 10^6$:

```
time   : 10^6 steps           instead of 10^12       a million times faster
space  : 10^6 x 100 bytes      = 100 MB extra      from ~0
```

That is the shape of nearly every interesting decision in this course: **spend $O(n)$ memory to remove a factor of n from the time.** When the interviewer asks for $O(1)$ space, they are asking you to give that trade back and find another route — usually sort-first, or two pointers, or using the input array itself as storage.

7 The traps

Trap one: the “in-place” function that changes nothing

This is the one that produces a silent wrong answer and looks completely correct.

```
def reverse_in_place(items: list[int]) → None:
    items = items[::-1]          # looks like it reverses the caller's list

nums = [1, 2, 3, 4]
reverse_in_place(nums)
print(nums)
```

```
[1, 2, 3, 4]
```

Nothing happened. The function ran, no error was raised, and the list is untouched.

The reason is that `items = ...` **rebinds the local name** `items` to a brand-new list. The caller's list is not involved. Assignment to a name never affects the caller; only mutation of the object does.

These two lines look almost identical and behave completely differently:

```
items = items[::-1]    # rebinds a local name. Caller sees nothing.
items[:] = items[::-1] # writes INTO the caller's list. Caller sees it.
```

`items[:] = ...` is slice assignment, and it does modify in place — though note it still builds the reversed copy first, so it is $O(n)$ extra space, not $O(1)$. The genuinely constant-space version is the two-pointer swap loop.

How to catch it every time: an in-place function returns `None` and its body contains `items[i] = ...`, `items[:] = ...`, or a method like `.sort()` or `.append()`. If the only assignment is to the bare parameter name, it is not in place.

Trap two: the space you allocate without an allocation

Here is a solution to “find all pairs that sum to a target”. The time complexity is fine. The memory is not.

```
def all_pairs(items: list[int], target: int) → list[tuple[int, int]]:
    pairs = [(a, b) for a in items for b in items if a + b == target]
    return pairs
```

The list comprehension is neat, and it evaluates `a + b == target` for every ordered pair before filtering. At `n = 50,000`:

```
Traceback (most recent call last):
  File "d7.py", line 6, in <module>
    print(len(all_pairs(list(range(50000)), 99999)))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "d7.py", line 2, in all_pairs
    pairs = [(a, b) for a in items for b in items if a + b == target]
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
MemoryError
```

`MemoryError` is Python telling you it asked the operating system for memory and was refused. There is no line number inside a loop to blame, because the loop is the problem: 2.5 billion comparisons producing 50,000 tuples at 56 bytes each was never the issue — it was that the answer here really can be $O(n^2)$ tuples for other inputs, and the interpreter died trying.

The fix is a hash map, one pass, and yielding results instead of accumulating them:

```
def all_pairs(items: list[int], target: int) → list[tuple[int, int]]:
    seen: set[int] = set()
    out: list[tuple[int, int]] = []
    for x in items:
        if target - x in seen:
            out.append((target - x, x))
            seen.add(x)
    return out
```

$O(n)$ time and $O(n)$ space, and it produces each pair once instead of twice.

How to catch it every time: before writing a comprehension with two `for` clauses in it, say the size out loud. Two `for` clauses over the same list is n^2 items considered, and if you are storing them rather than filtering to a few, it is n^2 in memory as well.

8 In the interview

How it gets asked

- “Can you do that in $O(1)$ extra space?” — the standard second question, after your solution works.
- “What’s the space complexity?” — the second half of “what’s the time complexity?”. Answer both without being asked twice.
- “Does the output count towards the space?” — a precision check.
- “Your solution is recursive. What does that cost in memory?” — the one people miss.

What to say out loud, in the first ninety seconds

1. **Separate the two kinds.** “Extra space is $O(1)$ — I’m not counting the input array, which is $O(n)$ and was given to me.”
2. **List what you allocated.** “I keep two indices and a temporary for the swap. That’s it, and it doesn’t grow with n .”
3. **If you allocated something that grows, say so plainly.** “The hash set is $O(n)$ extra, in the worst case where everything is distinct.”
4. **If it is recursive, count the depth.** “It recurses to depth $\log n$, so that’s $O(\log n)$ in call frames even though the body allocates nothing.”
5. **Name the trade.** “I’m spending $O(n)$ memory to get the time from $O(n^2)$ to $O(n)$. If memory were the tighter constraint I’d sort first and use two pointers — $O(n \log n)$ time, $O(1)$ extra space.”
6. **Offer the conversion.** “Want me to do the constant-space version?”

Step 5 is what makes you sound like an engineer rather than a candidate. You are not defending a solution; you are describing a position on a curve.

The follow-ups

“Does the space taken by the input count?” By convention, no — space complexity means auxiliary space, the memory you allocate on top of the input. The same goes for the output: if the problem asks for an array of n results, that array is not held against you. What does count is anything beyond those two. So if I build a dictionary in order to produce the output list, I’d state it as “ $O(n)$ for the output, plus $O(n)$ for the map I used along the way”. I’d always say which convention I’m using rather than assume we agree.

“Your solution is recursive. What does that cost in memory?” Every pending call keeps a frame alive until it returns, so the space is the maximum depth of the recursion. Binary search recursively is $O(\log n)$ space, not $O(1)$. A recursion that walks an array one element at a time is $O(n)$ space, and in Python it hits `RecursionError` at about a thousand frames. That is a real limit rather than a theoretical one, so for an input of a hundred thousand I’d convert it to an explicit loop, or to a loop with my own stack — which is $O(n)$ heap instead of $O(n)$ frames, but at least it does not crash.

“Is `items.sort()` $O(1)$ space?” No, and it is a nice question because the obvious answer is wrong. It sorts in place, so it does not return a new list, but Timsort needs a temporary buffer of up to $n/2$ for merging. So it is $O(n)$ auxiliary. If someone genuinely requires $O(1)$ extra space with $O(n \log n)$ time, heapsort is the answer — it sorts in place with constant extra space, at the cost of being slower in practice and not stable.

“How would you turn this $O(n)$ space solution into $O(1)$?” There are about four moves, and I’d look for them in this order. Sort the input first, if sorting is allowed, and then use two pointers or adjacent comparison. Use the input array itself as storage — for problems where values are in the range 1 to n , you can mark presence by negating a value or by swapping elements into position. Replace the recursion with a loop. Or reformulate the state entirely, like Boyer-Moore for majority element, which replaces a whole count map with one candidate and one counter.

A model answer

The candidate has solved “reverse the words in a string” by building a list and joining, and is asked to do it in constant extra space.

“Right now I’m $O(n)$ extra: I split into a list of words, reverse the list, and join. The list and the joined result are both proportional to the input.

To get to $O(1)$ extra space I’d need to modify the input in place, which in Python means the input has to be a list of characters rather than a string, since strings are immutable. Given that, there’s a standard two-step trick.

First, reverse the entire array in place with two pointers — left at 0, right at the end, swap and move both inward until they cross. That’s one temporary variable for the swap, so $O(1)$ space, and $n/2$ swaps, so $O(n)$ time.

After that the words are in the right order but each word is backwards. So second, walk through and reverse each word individually, in place, using the same two-pointer swap between the word’s boundaries. Also $O(1)$ space, also $O(n)$ time.

Two passes, $O(n)$ time, $O(1)$ extra space, and the total is two indices and one temporary regardless of how long the input is.

The thing I’d flag is that this only works because the input is mutable. If the signature gives me a Python string, $O(1)$ extra space is not achievable at all — I have to build a new string, and that’s $O(n)$ by definition. So I’d check whether the problem intends a character array. In an interview I’d rather ask that than silently solve a different problem.”

That answer converts the solution, states both complexities at each step, and ends by naming the precondition that makes the whole thing possible. That last paragraph is the difference between reciting a trick and understanding it.

9 Recall card

1. **Space complexity means extra space.** The input does not count; the output usually does not either. Say which convention you are using.
2. **$O(1)$ extra space = a fixed number of variables**, whatever `n` is. That is what **in-place** means, and it is built out of swaps.
3. **Slicing**, `sorted()`, **sets and dicts all allocate $O(n)$** . `items.sort()` is in place and still $O(n)$ auxiliary; heapsort is the $O(1)$ -space sort.
4. **Recursion costs $O(\text{depth})$ in call frames** even when the body allocates nothing. Python stops at about 1,000.
5. `items = items[::-1]` **inside a function changes nothing for the caller**. Rebinding a name is not mutation. Use `items[i] = ...` or `items[:] = ...`.

What a web server actually does

1 What this is, and why they ask it

A web server is a program running one loop, forever. It waits for a connection, takes the next one, reads the request, works out the answer, writes the response, and goes back to waiting.

That is the whole thing. Everything else — worker processes, thread pools, event loops, queues — exists because that loop has to run many times at once and there are only so many hands.

Interviewers ask this because it is the foundation of every capacity conversation that follows. When you later say “we’d need about thirty machines”, the number comes from how many requests one instance of this loop can complete per second. A candidate who cannot describe the loop cannot do that arithmetic, and every scaling answer they give afterwards is a guess dressed up as a design.

2 The story

The barber shop on the corner of Sadar Bazaar opens at eight in the morning. There are three chairs with three barbers, and along the opposite wall there is a bench with six seats on it.

Ramesh, the owner, unlocks the shutter at ten to eight. There is nobody outside. He puts the lights on anyway, fills the water bottles, sets out the towels, and the three of them stand there. Nobody comes in until twenty past. That twenty-five minutes of standing about is not a mistake. It is the job. The shop has to be ready before the first man walks in, because a man who finds a closed shutter goes to the shop by the bus stop instead.

A haircut takes about twenty minutes. So when a man sits down in a chair, that barber is gone for twenty minutes. He cannot start anybody else. He cannot half-do a second head. That chair is occupied until the man gets up.

Around ten on a Saturday it fills up. Three men in the chairs, and men arriving faster than heads are being finished. So they sit on the bench, and the bench works well — the moment a barber shakes out his cloth, he calls the next man off the bench and starts. Nobody has to coordinate it. The bench is simply where you wait.

Then the bench fills too. Six men sitting, three in the chairs. And now Ramesh has to do the thing he hates: the next man who pushes the door open gets told, from across the room, that there is no room, and that he should come back after two. It is better than letting him stand in the doorway for an hour and then be angry. Being turned away quickly is a kind of honesty.

Two things ruin a Saturday, and Ramesh can name both.

The first is a man who wants a haircut, a shave and a head massage. Fifty minutes. He is entitled to it, he pays for it, and for fifty minutes one of the three chairs is out of service. The bench does not care why the chair is busy. It just gets longer.

The second is worse and rarer. Once, the barber in the middle chair sent his apprentice out to fetch a particular brand of hair colour from a shop two streets away, and then stood there holding the man's head, doing nothing, until the boy got back. Eleven minutes of a chair occupied by a barber who was not cutting anything, only waiting. Ramesh made a rule after that: you finish somebody else while you wait.

By four in the afternoon the bench is empty and the barbers are standing at the door again, watching the road.

3 The idea in plain English

The barber shop is a web server, and every part of it has a name.

The loop

The shop's day is four steps repeating:

1. **Open and be ready.** The server **binds** to a port and calls **listen**. This is the lights going on at ten to eight. From this moment the operating system will accept connections on that port on the program's behalf, whether or not anybody has arrived.
2. **Take the next one.** The server calls **accept**, which hands it one connection. That is a barber calling the next man to the chair.
3. **Handle it.** Read the request, work out what is being asked, check permissions, talk to the database, build the answer. The twenty minutes of cutting.
4. **Respond and go back.** Write the response, then either close the connection or keep it open for the next request on it, and return to step 2.

Say those four words in an interview — **listen, accept, handle, respond** — and you have given the spine of the answer.

One chair is one worker

A barber cutting hair cannot start another head. A **worker** handling a request cannot handle another one at the same time. Three chairs means three requests being worked on at once, and the count of chairs is the single most important number about the shop.

Servers do this in three different ways, and interviewers ask you to compare them.

Many processes. Each worker is a separate copy of the program with its own memory. Robust — one crashing does not take the others down — and expensive, because each one costs tens of megabytes. This is how **nginx** workers and **gunicorn** workers operate.

Many threads. Workers share the program's memory, so each costs perhaps 1 MB instead of 50. Cheaper, and you now have to be careful about two workers touching the same data. This is the classic **Java** and **Tomcat** model.

One worker, an event loop. One thread, and instead of waiting for anything, it keeps a list of everything in flight and works on whichever is ready. This is **Node.js**, and Python's **asyncio** with **uvicorn**. It is the rule Ramesh made after the hair-colour incident: never stand there waiting, go and do something else.

The bench is the backlog

When every worker is busy, new connections do not vanish. The operating system holds them in a queue called the **backlog**, and its size is a number you configure — `listen(backlog=511)` is a common setting.

The bench is why a burst of traffic does not immediately fail. Requests wait a moment and get served.

And when the backlog is full, the operating system **refuses** new connections outright. Clients get `ECONNREFUSED` or a timeout. Ramesh telling a man to come back after two is exactly the right behaviour: a fast rejection is far better than accepting work you cannot do. This idea has a name — **load shedding** — and it is one of the things that separates a system that degrades from one that collapses.

The fifty-minute customer

One slow request holds a worker for its whole duration. With three workers and one request taking fifty times as long as the others, you have effectively lost a third of your capacity to one person.

This is why **timeouts** exist on every layer, and why slow endpoints are dangerous out of all proportion to how often they are called. A single database query without a timeout can take out an entire fleet, because every worker eventually ends up waiting on it.

The apprentice sent to the shop

This is the most important idea in the story, and it is the whole argument for asynchronous servers.

When the barber stood there holding a head while the boy fetched hair colour, he was **blocked** — occupying a chair while doing no work. Most of what a web request does is exactly this. It calls the database and waits. It calls another service and waits. Real numbers for a typical request:

```
CPU work (parsing, templating, serialising) : 5 ms
waiting for the database                     : 40 ms
waiting for another service                  : 30 ms
-----
total                                       75 ms, of which 70 is waiting
```

Ninety-three percent of that request is a barber standing still. A **blocking** server keeps the worker occupied for all 75 ms. An **asynchronous** server hands the work off, goes and serves somebody else, and comes back when the database answers. Same hardware, roughly ten times the concurrent requests.

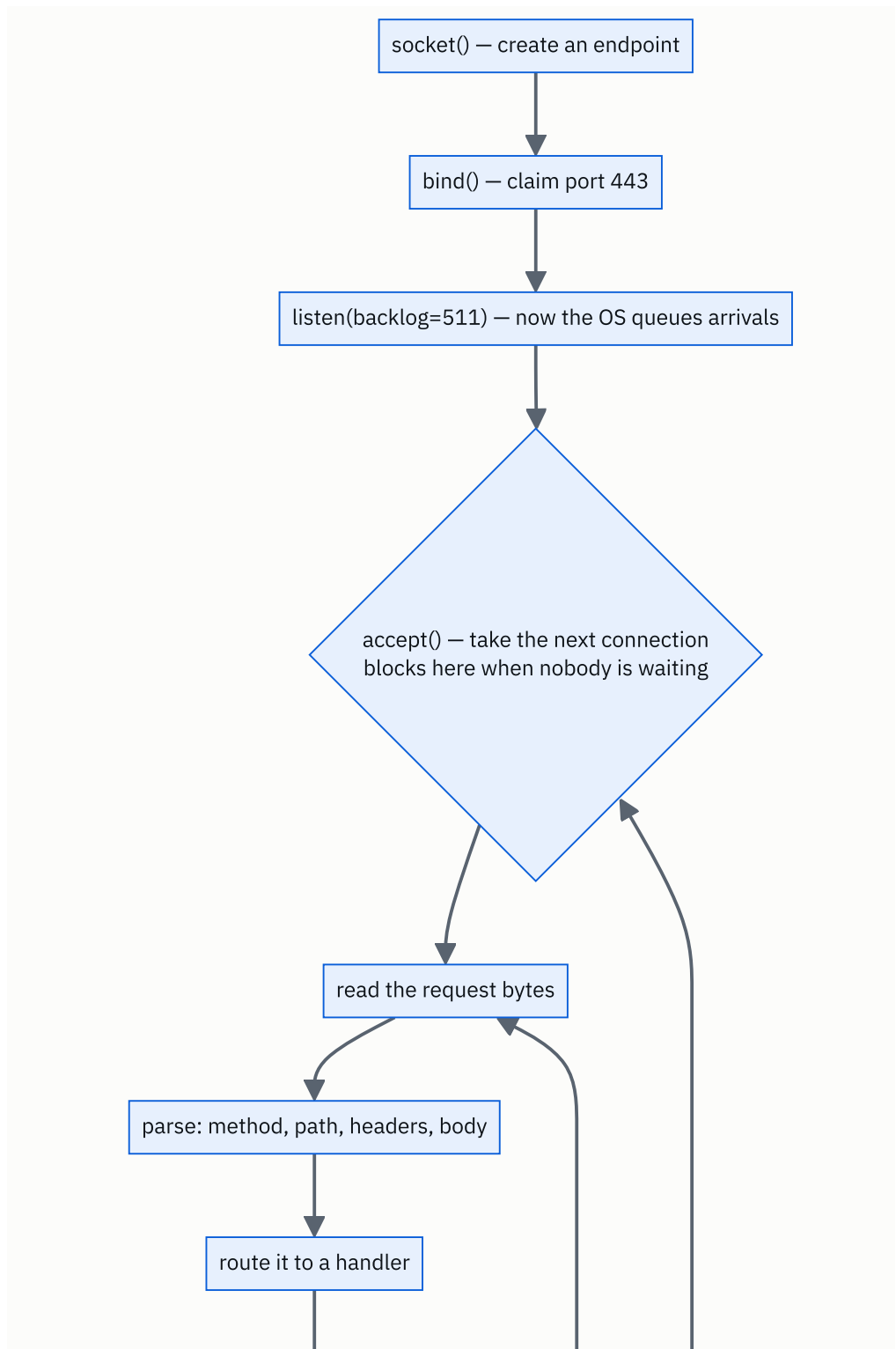
That is the answer to “why is Node good at this?” and it has nothing to do with JavaScript being fast.

The empty shop at four o'clock

A server spends most of its life idle, waiting. That is not waste; it is headroom. A server running at 100% of capacity is not “fully used” — it is a bench that is getting longer, as [day 002](#) showed. Sixty to seventy percent is a healthy target, and the rest is what absorbs the Saturday.

4 The picture

The loop itself:



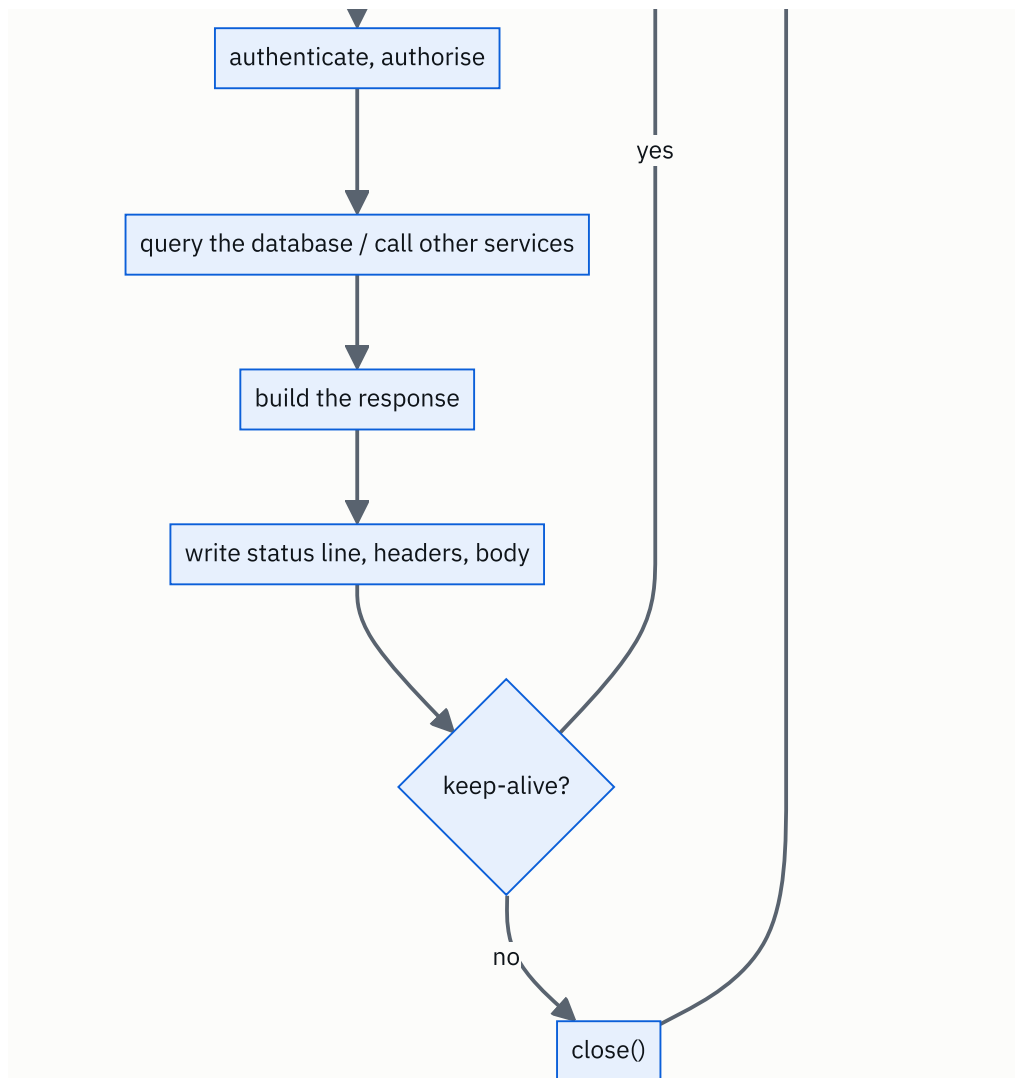


FIGURE 7.1

What to notice: the arrow from `keep-alive? yes` goes back to **read**, not to **accept**. That is what connection reuse means, and it is why a second request on the same connection skips the TCP and TLS handshakes entirely.

Now the shop, drawn as a capacity picture:


```

socket()    → create an endpoint
bind()      → claim 0.0.0.0:8000
listen(511) → tell the kernel to start queueing connections
accept()    → block until one is available, then return a connection
read()      → pull the request bytes
write()     → push the response bytes
close()     → release it

```

You can write a working web server in about fifteen lines of Python with the `socket` module, and doing it once removes all remaining mystery from the phrase “web server”.

Who is actually running

In production there are usually two programs, not one:

nginx (or **HAProxy**, or a cloud load balancer) sits at the front. It terminates TLS, holds thousands of slow client connections cheaply using an event loop, serves static files directly, and forwards the rest. Its job is to be very good at connections.

Your application server sits behind it. **gunicorn** with several worker processes, **uvicorn** for async Python, **Puma** for Ruby, **Tomcat** for Java, or a Node process. Its job is to run your code.

That split exists because of a specific attack and a specific problem. A client on a slow mobile network takes seconds to send a request; if that occupied one of your expensive application workers, twenty slow clients could exhaust the pool. nginx absorbs them and only hands the application a complete request. Deliberately exploiting this is called **Slowloris**, and the nginx-in-front pattern is the standard defence.

The three concurrency models, with real numbers

MODEL	COST PER WORKER	CONCURRENCY ON ONE 4 GB MACHINE	USED BY
Process	30–80 MB	~50	gunicorn sync, nginx workers
Thread	1–8 MB	~500	Tomcat, gunicorn gthread
Event loop / coroutine	2–10 KB	~50,000	Node.js, uvicorn, Go, nginx

The event loop column is why **C10K** — handling ten thousand concurrent connections on one machine — stopped being a research problem. The mechanism underneath is `epoll` on Linux and `kqueue` on BSD: one call that says “tell me which of these ten thousand connections has data ready”, instead of one thread sitting on each.

Go deserves a mention because it sidesteps the choice. Goroutines look like threads to write and cost about 2 KB each, with the runtime multiplexing them onto an event loop for you. That is why Go became the default for network services.

What the handler actually does, in order

1. **Parse** the request line and headers. Reject anything malformed with `400`.
2. **Route** — match method and path to a function.
3. **Authenticate** — validate the token or session, usually a **Redis** lookup or a signature check.
4. **Authorise** — is *this* user allowed *this* action?
5. **Validate** the body against a schema.
6. **Do the work** — read or write **PostgreSQL**, check a **Redis** cache, call another service, put a message on **Kafka**.
7. **Serialise** the result to JSON.
8. **Write** status line, headers, body.
9. **Log** the request, emit metrics, and release the database connection back to the pool.

Steps 6 and 3 are where nearly all the time goes. Steps 1, 2, 7 and 8 are the CPU work.

Connection pools, which is where beginners get hurt

Your application does not open a new database connection per request — that would cost a handshake every time. It keeps a **connection pool**, typically 5 to 20 connections per worker process.

The multiplication is the part people miss:

```
20 unicorn workers x 10 connections each = 200 database connections
```

PostgreSQL's default limit is 100. So a perfectly reasonable-looking application server configuration takes the database down at startup. The fix is **PgBouncer** or an equivalent pooler in front of the database, and it is one of the most common real-world outages there is.

Graceful shutdown

When you deploy, the old process must not simply be killed mid-request. The correct sequence is: stop accepting new connections, finish the in-flight ones, then exit. This is what `SIGTERM` means to a well-behaved server, and it is why load balancers have a **draining** period. Skipping it means every deployment returns errors to whoever was mid-request.

6 The numbers

What one worker can do. If a request takes 50 ms of wall-clock time:

```
1,000 ms per second / 50 ms per request = 20 requests per second per worker
```

What one machine can do. Four workers on a 4-core machine:

```
4 workers x 20 rps = 80 requests per second
```

That is the number everything else is built on. Note that it is set by the **duration** of a request, not by the CPU work in it — a request that spends 45 of its 50 ms waiting still ties up a blocking worker for all 50.

The formula worth memorising. This is **Little's Law**, and it is one line:

```
concurrency = arrival rate x average duration
```

So for 500 requests per second at 200 ms each:

```
500 x 0.2 = 100 requests in flight at any moment
```

You need at least 100 workers, or an async server able to hold 100 in-flight requests. If you have 50, the bench grows without limit and latency climbs until something breaks. **Little's Law turns a latency target into a worker count**, and interviewers love it because it is arithmetic rather than opinion.

What async buys, concretely. Same request, 5 ms CPU and 70 ms waiting, on 4 cores:

```
blocking, 4 workers : 4 / 0.075 s = 53 requests per second
async, 4 cores      : 4 x (1,000 / 5 ms) = 800 requests per second (CPU-bound ceiling)
```

Fifteen times more, on identical hardware, because the waiting overlaps. Now change the request to 70 ms of CPU and 5 ms of waiting:

```
blocking, 4 workers : 4 / 0.075 s = 53 rps
async, 4 cores      : 4 x (1,000 / 70 ms) = 57 rps
```

Almost identical. **Async helps exactly to the extent that your requests are waiting**, and not at all otherwise. That comparison is the answer to “should we rewrite this in Node?”.

Memory, and how you actually pick a worker count. On a 4 GB machine with 60 MB workers:

```
4,096 MB - 500 MB for the OS = 3,596 MB usable
3,596 / 60 = 59 workers by memory
```

But with 4 cores, the standard gunicorn heuristic is:

```
workers = 2 x cores + 1 = 9
```

Memory allows 59 and CPU suggests 9. **The lower number wins**, and for blocking workers it is almost always the CPU one — unless requests are wait-heavy, in which case you raise it and watch latency.

When the bench overflows. Arrivals 100 rps, capacity 80 rps, backlog 511:

```
excess = 20 requests per second
511 / 20 = 25 seconds until the backlog is full
```

Twenty-five seconds from “slightly overloaded” to refusing connections. That is how quickly a 20% overload becomes an outage, and it is why autoscaling has to react in seconds rather than minutes.

What idle connections cost. With keep-alive, connections outnumber active requests enormously:

```
10,000 idle keep-alive connections x 10 KB = 100 MB
```

Cheap on an event-loop server such as nginx. Ruinous on a thread-per-connection server, where 10,000 connections would mean 10,000 threads at 1 MB each — 10 GB. **This is the whole reason nginx sits in front.**

7 The trade-offs

Processes versus threads. Processes give you isolation: one crashing, leaking or seg-faulting worker does not affect the others, and in Python they sidestep the Global Interpreter Lock so you actually use every core. The cost is memory — 30 to 80 MB each — and no shared state, so any cache has to live in Redis rather than in memory. Threads are ten to fifty times cheaper and share memory, which is convenient and is also how data races happen. In Python specifically, threads do not help with CPU-bound work at all because of the GIL, which is why gunicorn’s default is processes.

Blocking versus async. Async wins massively when requests wait on the network, which is most web work, and it wins nothing when they compute. It also costs you a harder programming model: one blocking call anywhere in an async handler stalls the

entire event loop and every request on it. A single synchronous `requests.get()` inside an async endpoint can take the whole process from 800 rps to 13. Blocking code is duller and much harder to get catastrophically wrong.

Keep-alive: fewer handshakes, more idle connections. Reusing connections removes a round trip or two per request, which is a large win. It also means each client holds a connection open between requests, so the connection count is far higher than the concurrent request count. On a cheap-connection server that is free; on an expensive-connection server it is the constraint. This asymmetry is exactly why the standard architecture is an event-loop proxy in front of a process-based application server.

Queue deeply or shed load. A large backlog absorbs bursts and hides brief overload. It also means that under sustained overload, requests sit in a queue until they time out — you do the work and then discard the answer because nobody is listening any more, which is the worst possible outcome. A short queue plus explicit rejection (`503 Service Unavailable` with a `Retry-After` header) is usually the better engineering, because it fails fast and honestly. Ramesh turning people away at the door is not rudeness; it is capacity management.

I would not use this shape at all if... the connection is long-lived and the server needs to push. Chat, live dashboards and collaborative editing want WebSockets, where one connection stays open for minutes or hours — which makes a process-per-connection model impossible and an event loop mandatory. Very high fan-out work belongs on a queue rather than in the request path: the request should enqueue a job to **Kafka** or **SQS** and return `202 Accepted`, so that a slow downstream system cannot hold a worker hostage.

8 In the interview

How it gets asked

- “What happens inside the server between receiving a request and sending a response?” — the direct version.
- “How does one server handle thousands of clients at once?” — the concurrency version.
- “How many requests per second can one machine handle?” — a capacity question. Do the arithmetic out loud.
- “Why do people put nginx in front of their application?” — the architecture version.

What to say out loud, in the first ninety seconds

1. **Give the loop in four words.** “Listen, accept, handle, respond — and then back to accept.”

2. **Say what “listen” means.** *“The process binds a port and calls listen, and from then on the kernel queues incoming connections whether or not the app is ready for them.”*
3. **Walk the handling.** *“Read the bytes, parse method and path and headers, route to a handler, authenticate, authorise, hit the database or cache, serialise, write back.”*
4. **Introduce the constraint.** *“One worker handles one request at a time, so concurrency is the number of workers. Requests arriving when all workers are busy sit in the backlog queue, and when that fills, connections get refused.”*
5. **Name the three models.** *“Processes, threads, or an event loop. Processes cost tens of megabytes and give isolation, threads cost about a megabyte, an event loop costs kilobytes per connection but is single-threaded.”*
6. **Land the insight.** *“Most of a web request is waiting on I/O — maybe 70 of 75 milliseconds. A blocking worker is occupied for all of it. That’s the entire case for async, and it’s also why async does nothing for CPU-bound work.”*

Step 6 is the sentence that gets you the follow-up you want.

The follow-ups

“How many requests per second can one server handle?” It depends on request duration, and I’d do the arithmetic rather than guess. If a request takes 50 ms and I have four blocking workers, that’s 1,000 divided by 50, times 4 — eighty requests per second. The useful general form is Little’s Law: concurrency equals arrival rate times duration. If I need 500 rps at 200 ms each, I need 100 requests in flight, so 100 workers or an async server that can hold 100. The thing I’d emphasise is that duration matters as much as the machine — halving latency doubles throughput on the same hardware.

“Why put nginx in front of your application server?” Four reasons. It terminates TLS in one place, so certificates are managed once. It serves static files without touching application code. It holds slow client connections cheaply on an event loop, so a client on a bad mobile network does not occupy a 60 MB application worker for three seconds — that is the Slowloris defence. And it gives you a place for rate limiting, request buffering, compression and health checks. The pattern is: something very good at connections in front, something running your code behind.

“What happens when all the workers are busy?” New connections queue in the kernel’s backlog, which is sized by the `listen()` call — commonly 511. Requests wait there and are picked up as workers free up, which is exactly what you want for a short burst. When the backlog fills, the kernel refuses connections and clients see connection refused or a timeout. Under sustained overload the honest thing is to shed load deliberately: return 503 with Retry-After rather than accept work you will not finish in time. A request that sits in a queue for thirty seconds and then gets processed after the client gave up is pure waste.

“Would async fix our latency problem?” Only if we are waiting rather than computing. I’d measure the split first. If a request is 5 ms of CPU and 70 ms of database wait, async gives roughly an order of magnitude more concurrency on the same hardware. If it is 70 ms of CPU and 5 ms of wait, async gives almost nothing and adds a failure mode — one blocking call anywhere in the event loop stalls every request on that thread. And async improves throughput, not the latency of a single request: one user’s request does not get faster, you just serve more of them at once.

A model answer

“The server is one loop. At startup it creates a socket, binds to a port, and calls listen — and from that moment the kernel accepts and queues connections on its behalf, even before the application asks for one. Then it loops: accept a connection, read the request bytes, handle it, write the response, and either close or keep the connection open for the next request on it.

The handling part is: parse the request line and headers, route method and path to a handler, authenticate — usually a Redis lookup or a token signature check — authorise, validate the body, then do the actual work, which is normally one or more database queries plus maybe a call to another service. Then serialise to JSON, write the status line, headers and body, log it, and release the database connection back to the pool.

The important constraint is that one worker handles one request at a time. So concurrency is the number of workers, and requests that arrive when all of them are busy sit in the backlog queue — typically 511 deep. When that fills, connections are refused outright.

There are three ways to get concurrency. Processes, at 30 to 80 megabytes each, which is what gunicorn does by default and which sidesteps Python’s GIL. Threads, at about a megabyte each. Or an event loop, at a couple of kilobytes per connection, which is Node and uvicorn.

The insight that decides which is that most of a web request is waiting, not computing — maybe 5 milliseconds of CPU and 70 of database and network. A blocking worker is occupied for the whole 75. An event loop hands the wait off and serves someone else, so on four cores you go from around 50 requests per second to hundreds. But that gain is entirely from overlapping the waiting: if the request were CPU-bound, async would give me nothing.

For capacity I’d use Little’s Law — concurrency equals arrival rate times duration. At 500 requests per second and 200 milliseconds each, that’s 100 in flight, so 100 workers or an async server sized for it, plus headroom, because a server at 100% utilisation isn’t fully used, it’s a queue that’s growing.”

That answer gives the loop, the mechanics, the constraint, the three models, the deciding insight, and finishes with arithmetic that a capacity discussion can be built on.

9 Recall card

1. **The loop: listen, accept, handle, respond, repeat.** `socket → bind → listen → accept → read → write → close`.
2. **One worker, one request.** Concurrency = number of workers. Overflow waits in the **backlog**; when that fills, connections are refused.
3. **Three models:** processes (30–80 MB, isolated), threads (~1 MB, shared), event loop (~2 KB, single-threaded).
4. **Most of a request is waiting**, not computing — which is the entire case for async, and the reason it does nothing for CPU-bound work.
5. **Little's Law: concurrency = arrival rate × duration.** $500 \text{ rps} \times 200 \text{ ms} = 100$ in flight. That is how a latency target becomes a worker count.

PRACTICE

Practice — Space complexity, and what in-place really means

DSA topic: Space complexity, and what in-place really means **System design topic:** What a web server actually does

Code these, in this order

Four problems that each have an easy $O(n)$ -space solution and a harder $O(1)$ -space one. Write **both** versions of every one of them. The second version is the exercise.

For each problem:

1. Write the natural solution and state its extra space.
2. Then ask yourself the interviewer's question: *can I do this in $O(1)$ extra space?*
3. Write that version, and state exactly what you kept: how many variables, and why none of them grow.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Reverse String	LeetCode 344 (Easy)	The problem statement explicitly demands $O(1)$ extra space, which is why the input is a character list rather than a string. Two pointers and a swap.
2	Remove Duplicates from Sorted Array	LeetCode 26 (Easy)	The write pointer, in its purest form. The signature returns a length rather than a list, and that is the whole hint.
3	Move Zeroes	LeetCode 283 (Easy)	Same pattern, one step harder. Building a new list is trivial; doing it in place with one pass and one extra index is the point.
4	Majority Element	LeetCode 169 (Easy)	Three solutions with three different space costs: a <code>Counter</code> is $O(n)$, sorting is $O(1)$ extra with an in-place sort, and Boyer-Moore is $O(1)$ with one candidate and one count.

On problem 4, do this properly

Write all three. Then say out loud, for each one, the time and the extra space:

- `Counter(nums).most_common(1)` →
- `sorted(nums)[len(nums) // 2]` →
- Boyer-Moore voting →

Then answer the question that matters: **what did Boyer-Moore replace the whole count map with?** If you can say that in one sentence, you have understood what “reformulate the state” means, and it is the fourth of the four moves from §8 of the lesson.

The measurement drill

Run the complete program from §5 at `N = 200_000`, then at `N = 400_000`.

The `O(n)` rows should double. The `O(1)` rows should stay at zero. Then add one line of your own to the program — a function that solves any of today’s problems by slicing — and confirm it lands in the `O(n)` group even though it is one line long.

The trap drill

Type this out and run it before you read the answer:

```
def reverse(items):
    items = items[::-1]

nums = [1, 2, 3, 4]
reverse(nums)
print(nums)
```

Predict the output first. Then explain, out loud, in one sentence, why assigning to `items` inside the function did nothing — and name the two ways to write it so that it does.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Can you do that in $O(1)$ extra space?* Use problem 3. Say what you would keep, why it does not grow, and what you gave up to get there.
 2. *What happens inside the server between receiving a request and sending a response?* Four words for the loop, then the nine steps of handling, then the one-worker-one-request constraint.
 3. *How many requests per second can one machine handle?* Do not guess. Pick a request duration, pick a worker count, do the division out loud, then state Little’s Law and use it to turn a target into a worker count.
-

Before you move on

- [] I say “ $O(1)$ extra space” and can explain what I am not counting.

- [] I can reverse an array in place with two pointers, from memory, first try.
- [] I know that `items.sort()` is in place and still $O(n)$ auxiliary, and which sort is genuinely $O(1)$.
- [] I count recursion depth as space, every time.
- [] I can say the server loop in four words and name the three concurrency models with their memory costs.

Reading a problem like the interviewer wrote it

DATA STRUCTURES AND ALGORITHMS

Reading a problem like the interviewer wrote it

You can extract input, output, constraints and edge cases from a problem in two minutes.

SYSTEM DESIGN

Processes, threads, and concurrency

You can explain the difference between a process and a thread, and why servers use both.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Before you write code, what questions do you have about the problem?*
- *What is the difference between a process and a thread?*

Reading a problem like the interviewer wrote it

1 What this is, and why they ask it

A problem statement has four things in it: what you are **given**, what you must **return**, the **constraints** that bound the input, and the **edge cases** that the ordinary solution gets wrong. Reading a problem means pulling those four out, deliberately, before writing a line.

Almost every problem statement is also missing something. Interview problems are stated loosely on purpose, and the missing pieces are the questions you are supposed to ask.

They ask “what questions do you have?” because it is the highest-signal two minutes of the whole round. A candidate who starts typing immediately has told the interviewer they will also start typing immediately at work, and will discover the requirement on the third day. A candidate who asks four sharp questions has demonstrated the thing companies actually hire for, and they have done it before writing anything that could be wrong. This is the one part of an interview where the right move costs you nothing and is almost never made.

2 The story

Deepak’s mother is at the door with her bag on her shoulder, already late for the bus, and she turns round and says: “Order the food for Sunday.”

Last year he would have said yes and let her go. Last year he did exactly that, and it went badly enough that he still thinks about it.

So this time he says, “Two minutes,” and she stops, because she can hear that he is going to be quick.

“How many people?”

“Eleven.” She had said eight the week before. The Menons had said no and then said yes again on Tuesday. If he had ordered on Tuesday’s number, three people would have eaten very little and it would have been at his end of the table.

“What time?”

“Half past eight.” He had assumed lunch. Nobody had said lunch. Last year he had assumed lunch too, and the food arrived at one o’clock for a dinner, and sat on the counter for seven hours.

“Anybody not eating something?”

She thinks about it properly for a moment, which tells him the question was worth asking. “Your aunt is fasting on Sunday, so nothing with onion or garlic. Do a separate portion for her. And the Menon boy will not touch anything spicy.”

“How much can I spend?”

“Twenty-five hundred. Don’t go over three.”

“And if the usual place isn’t taking orders on a Sunday?”

She had not thought about this at all. “Then the one behind the bank. Not the new place, the food is bad.”

That is the whole conversation, and it takes ninety seconds. She gets her bus.

Deepak now knows something he genuinely did not know ninety seconds earlier: that he is ordering dinner for eleven at half past eight, under three thousand rupees, with one separate portion with no onion and no garlic, one mild dish, and a second place to try if the first one says no.

None of that was in “order the food for Sunday”. All of it was in his mother’s head, and every single piece of it would have changed what he ordered. The sentence was not wrong. It was just very far from complete, and the only reason it worked as an instruction is that he stopped and asked.

3 The idea in plain English

Deepak’s four questions map exactly onto the four things you extract from a problem statement.

The four things

What am I given? Eleven people, a budget, a time. In a problem: the **input** — its type, its size, and what is in it. Is it a list of integers? Can it be empty? Can it contain negatives? Are there duplicates? Is it sorted?

What must I return? A dinner order, not a shopping list. In a problem: the **output** — its type and its exact shape. An index or a value? A new list or a modified one? What do I return when there is no answer — `-1`, `None`, an empty list, or is it guaranteed there is one?

What bounds am I working inside? Three thousand rupees. In a problem: the **constraints** — $1 \leq n \leq 10^5$, $-10^9 \leq \text{nums}[i] \leq 10^9$. From [day 004](#) you know these are not decoration; they name the shape you are allowed to write.

What is unusual and will break the obvious approach? The aunt who is fasting. In a problem: the **edge cases** — empty input, one element, all elements identical, all negatives, the target absent, the maximum allowed size.

The order to do it in

There is a sequence, and following it is the visible skill:

1. Restate the problem in your own words. *“So I’m given an unsorted array of integers and a target, and I need to return the indices of two numbers that add to the target.”* If your restatement is wrong, you have just saved twenty minutes at a cost of eight seconds. If it is right, the interviewer relaxes.

2. Work one example by hand. Take the given example and produce the answer manually, saying what you are doing. This is where you discover that you misunderstood the output format.

3. Ask the clarifying questions. Four or five, no more. The list is below.

4. Read the constraints and name the target complexity. *“n is up to ten to the fifth, so I’m aiming for $O(n \log n)$ or better.”*

5. State your approach in one sentence before writing it. *“I’ll use a hash map from value to index, one pass.”* This is your last chance to be corrected cheaply, and interviewers will correct you here if you let them.

6. Then write the code.

The questions that are almost always worth asking

Keep this short list in your head. Pick the three or four that fit.

About the input - Can the input be empty, or null? - Can it contain duplicates? - Can values be negative? Zero? - Is it sorted? Can I sort it, or does the original order matter? - How big can it get? (*read the constraints out loud*) - Are the values integers, or could they be floats?

About the output - Indices or values? - What do I return if there is no valid answer? - If several answers are valid, does it matter which one I return? - Should the result be in a particular order?

About what I may do - May I modify the input in place? - May I use extra space, or is `O(1)` required? - May I use library functions like `sorted()`, or is this about the mechanics?

The assumptions that cost people the round

Deepak assumed lunch. These are the software versions, and each one has ended an interview:

THE ASSUMPTION	WHAT ACTUALLY HAPPENS
“the array is sorted”	it is not, and your binary search returns nonsense
“there are no duplicates”	there are, and your answer double-counts
“the values are positive”	one negative and the sliding window breaks
“there is always an answer”	there is not, and you return an unset variable
“the input is non-empty”	it is empty, and <code>max()</code> raises
“one valid answer exists”	several do, and you return the wrong one

None of these are caught by testing the given example, because the given example is always well-behaved. They are caught by asking, or by building the edge-case list yourself.

Say it out loud, always

One more thing, and it is not about the problem. Interviewers score what they hear, not what you type. Silence while you think looks identical to silence while you are lost. Say what you are considering, say what you have rejected and why, and say when you are stuck. “I’m considering sorting first, but that loses the original indices, which the output needs” is worth more than five minutes of correct silent typing. Day 178 is entirely about this.

4 The picture

A real problem statement, taken apart:

```

+-----+
| Given an array of integers nums and an integer target, return the |
| indices of the two numbers such that they add up to target.      |
|                                                                     |
| You may assume that each input would have exactly one solution,  |
| and you may not use the same element twice.                      |
|                                                                     |
| You can return the answer in any order.                           |
|                                                                     |
| Constraints:                                                       |
|   2 ≤ nums.length ≤ 10^4                                         |
|  -10^9 ≤ nums[i] ≤ 10^9                                           |
|  -10^9 ≤ target ≤ 10^9                                           |
+-----+

```

v	v	v	v
INPUT	OUTPUT	CONSTRAINTS	EDGE CASES
array of integers, unsorted, may repeat	indices, not values "any order" → no tie-break	$n \leq 10^4$ → $O(n^2) = 10^8$, tight; aim $O(n)$	negatives allowed duplicates allowed $n \geq 2$, so never empty "exactly one solution" → no not-found case

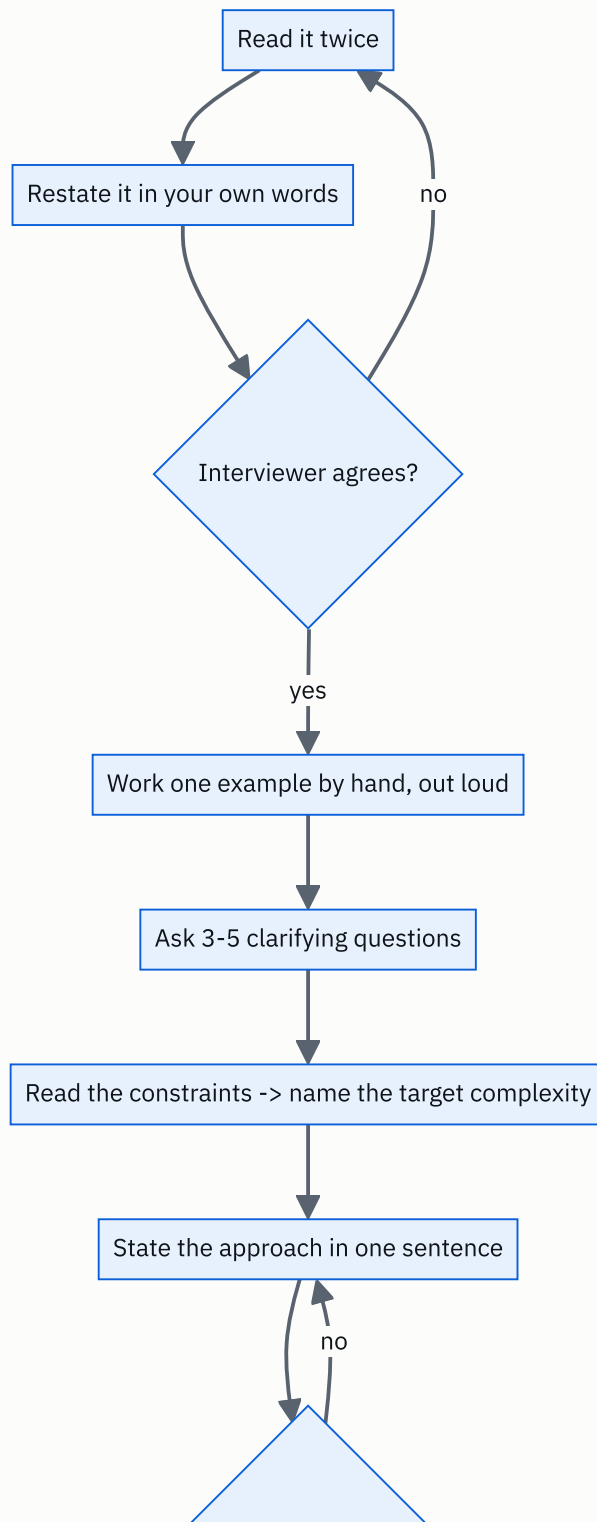
What to notice: three of the four columns come from sentences most people skim. "Exactly one solution" removes an entire branch of code. "Any order" removes a sorting requirement. $n \geq 2$ removes the empty check. **Reading carefully makes the problem smaller.**

Now the same four columns for a problem where the answers go the other way:

INPUT	Two Sum	Find the maximum
OUTPUT	$n \geq 2$, guaranteed	may be empty ← !
NO ANSWER	indices	the value
DUPLICATES	cannot happen	must decide: None? raise? -1?
CONSTRAINT	allowed, irrelevant	allowed, affects "which index?"
	$n \leq 10^4$	$n \leq 10^5$

What to notice: the *same* four questions, and completely different answers. This is why the checklist is a checklist rather than a memorised set of assumptions.

And the sequence, as a flow:



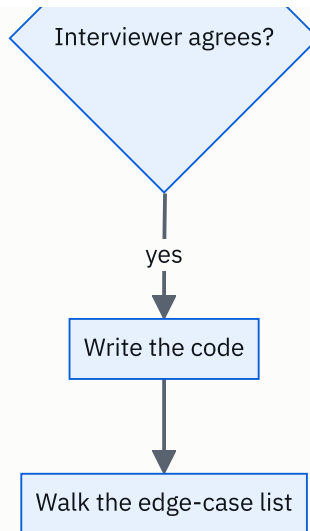


FIGURE 8.1

What to notice: there are two places where the interviewer can stop you cheaply, and both are before any code exists. Candidates who skip straight from A to I get corrected at the most expensive possible moment.

5 The code, built step by step

The way to make this concrete is to take one loosely stated problem and watch the code change as each question gets answered.

The problem, as stated: *“Find the largest number in a list.”*

Here is what everybody writes first.

```
def largest(items: list[int]) → int:
    return max(items)
```

One line, correct for the example, and it has three unanswered questions in it. Ask them.

Question 1: “Can the list be empty?” Suppose the answer is yes.

```
items = []
print(max(items))
```

```
Traceback (most recent call last):
  File "d8.py", line 2, in <module>
    print(max(items))
          ^^^^^^^^^^^
ValueError: max() arg is an empty sequence
```

So an answer is needed for that case. `None` is usually the right choice in Python, because it is unambiguous — unlike `-1`, which is a legitimate value here.

```
def largest(items: list[int]) → int | None:
    if not items:
        return None
    return max(items)
```

`int | None` in the signature says the return may be either, which is the honest type.

Question 2: “Do you want the value or its position?” Suppose it is the position, and that this is really about the mechanics rather than about knowing `max` exists.

```
def largest_index(items: list[int]) → int | None:
    if not items:
        return None
    best = 0
    for i in range(1, len(items)):
        if items[i] > items[best]:
            best = i
    return best
```

Starting `best` at 0 rather than at some sentinel value is deliberate: it avoids inventing a “smallest possible number”, which is where negative inputs break naive solutions.

Question 3: “If the largest value appears more than once, which position do you want?” This is the question nobody asks, and it changes the code by one character. `>` keeps the **first** occurrence; `≥` keeps the **last**.

```
if items[i] > items[best]:      # first occurrence
if items[i] ≥ items[best]:     # last occurrence
```

Now the second worked example, which is the one interviewers actually use.

The problem, as stated: “Given an array and a target, return the indices of two numbers that add up to the target.”

Ask, and suppose the answers are: the array is unsorted, duplicates are possible, values can be negative, there may be **no** answer, and any valid pair will do.

The brute force, stated and rejected out loud:

```
def two_sum_brute(nums: list[int], target: int) → tuple[int, int] | None:
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return (i, j)
    return None
```

$O(n^2)$ time, $O(1)$ space, and j starts at $i + 1$ so nothing is paired with itself.

The hash map version, which is what the constraints call for:

```
def two_sum(nums: list[int], target: int) → tuple[int, int] | None:
    seen: dict[int, int] = {} # value → the index it was seen at
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return (seen[need], i)
        seen[x] = i
    return None
```

Two details that only exist because questions were asked. **Checking before inserting** means an element can never pair with itself, which matters when `target` is exactly double a value. And `seen[x] = i` overwrites an earlier duplicate, which is fine only because any valid pair was acceptable.

Here is the complete program, with the edge cases as a table rather than as an after-thought.

```

"""Day 8 – the same problem, read properly. The edge cases are the exercise."""

def largest_index(items: list[int]) → int | None:
    """Index of the largest value. First occurrence on a tie. None if empty."""
    if not items:
        return None
    best = 0
    for i in range(1, len(items)):
        if items[i] > items[best]:
            # '>' keeps the FIRST maximum
            best = i
    return best

def two_sum_brute(nums: list[int], target: int) → tuple[int, int] | None:
    """O(n^2) time, O(1) space. Every pair, each once."""
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return (i, j)
    return None

def two_sum(nums: list[int], target: int) → tuple[int, int] | None:
    """O(n) time, O(n) space. Look for what you NEED, not what you have."""
    seen: dict[int, int] = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            # check BEFORE inserting: no self-pairing
            return (seen[need], i)
        seen[x] = i
    return None

# The edge-case list. Build this BEFORE the solution, not after it.
CASES: list[tuple[str, list[int], int]] = [
    ("empty input", [], 5),
    ("one element", [5], 5),
    ("two elements, match", [2, 3], 5),
    ("two elements, no match", [2, 3], 99),
    ("no answer exists", [1, 2, 3], 50),
    ("negatives", [-3, 4, 7, -1], 3),
    ("target is zero", [-4, 1, 4], 0),
    ("duplicates form it", [3, 3], 6),
    ("value pairs with self?", [3, 1, 9], 6),
    ("answer at the ends", [8, 1, 2, 3, -1], 7),
    ("all identical", [4, 4, 4, 4], 8),
]

if __name__ == "__main__":
    print(f"{'case':<24}{'input':<20}{'target':>7}{'brute':>12}{'hash':>12}  agree?")
    print("-" * 88)
    for name, nums, target in CASES:
        a = two_sum_brute(nums, target)
        b = two_sum(nums, target)
        agree = "yes" if (a is None) == (b is None) else "NO"
        print(f"{'name':<24}{str(nums):<20}{target:>7}{str(a):>12}{str(b):>12} {agree}")

    print("\nlargest_index, including the cases that break the one-liner")
    for items in ([], [5], [1, 9, 3, 9, 2], [-5, -2, -9], [7, 7, 7]):
        print(f"  {str(items):<18} → {largest_index(items)}")

```

This is exactly what it printed:

case	input	target	brute	hash	agree?
empty input	[]	5	None	None	yes
one element	[5]	5	None	None	yes
two elements, match	[2, 3]	5	(0, 1)	(0, 1)	yes
two elements, no match	[2, 3]	99	None	None	yes
no answer exists	[1, 2, 3]	50	None	None	yes
negatives	[-3, 4, 7, -1]	3	(1, 3)	(1, 3)	yes
target is zero	[-4, 1, 4]	0	(0, 2)	(0, 2)	yes
duplicates form it	[3, 3]	6	(0, 1)	(0, 1)	yes
value pairs with self?	[3, 1, 9]	6	None	None	yes
answer at the ends	[8, 1, 2, 3, -1]	7	(0, 4)	(0, 4)	yes
all identical	[4, 4, 4, 4]	8	(0, 1)	(0, 1)	yes
largest_index, including the cases that break the one-liner					
	[]		→ None		
	[5]		→ 0		
	[1, 9, 3, 9, 2]		→ 1		
	[-5, -2, -9]		→ 1		
	[7, 7, 7]		→ 0		

Look at the row `value pairs with self?`. Input `[3, 1, 9]`, target 6. There is a 3, and $3 + 3 = 6$, and the answer is correctly `None` because there is only one 3. That row exists because someone asked “can I use the same element twice?”. Without the question, the check would go in the wrong place and the function would return `(0, 0)`.

And look at `[-5, -2, -9]`. A solution that starts with `best_value = 0` instead of `best = 0` returns nothing sensible here, because every value is below the sentinel. That row exists because someone asked “can values be negative?”.

6 What it costs

The arithmetic on asking. A clarifying question costs about fifteen seconds. Writing a solution to the wrong problem costs the rest of the interview.

5 questions x 15 s	= 75 seconds
one wrong assumption	= 15-20 minutes of code, then a rewrite under pressure
in a 45-minute round	= 40% of your time, and the interviewer watched you spend it

Seventy-five seconds against twenty minutes. There is no other decision in an interview with that ratio.

What the constraints buy you. Reading `2 ≤ nums.length` removes the empty check. “Exactly one solution” removes the not-found branch. “Any order” removes a sort. Each one is a branch you do not write, do not test, and cannot get wrong:

lines removed by reading the statement properly:	6-10
chances of a bug in code you did not write	0

What the constraint tells you about the shape. For Two Sum:

```
n ≤ 10^4
O(n^2) = 10^8 operations → about 1 second in C++, 10+ in Python → too tight
O(n) = 10^4 operations → instant
```

So the hash map is not a cleverness; it is what the constraint asked for. Notice also that $-10^9 \leq \text{nums}[i] \leq 10^9$ means sums can reach 2×10^9 , which overflows a 32-bit integer in Java or C++. Python has arbitrary-precision integers so it does not bite here — but saying “in a language with fixed-width integers I’d want a 64-bit type for the sum” is a free point.

The edge-case list is cheap and the bug is not. Eleven cases took about ninety seconds to write and they exercise every branch:

```
11 cases x ~8 seconds to write = 90 seconds
1 failed submission on a hidden test = one wrong-answer verdict and a lost lead
```

Where the time actually goes in a good 45-minute round:

```
0:00 - 0:03  read, restate, work an example
0:03 - 0:05  clarifying questions
0:05 - 0:07  state the approach, agree the complexity target
0:07 - 0:25  write the code, narrating
0:25 - 0:32  walk the edge cases out loud, fix what breaks
0:32 - 0:38  state complexity, discuss improvements
0:38 - 0:45  their questions, your questions
```

Seven of the first ten minutes involve no code, and that is the correct allocation. It looks like a waste of time only to people who have not watched the other version.

7 The traps

Trap one: the assumption that the example did not contradict

The given example is always well-behaved. That is what makes it dangerous.

Problem: “Given an array, return the index of the target, or -1 if it is not present.”
Example: `nums = [1, 3, 5, 7, 9]`, `target = 5` → 2.

The example is sorted. So this gets written:

```
def find(nums: list[int], target: int) → int:
    left, right = 0, len(nums) - 1
    while left ≤ right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

It is a correct binary search and it passes the example. Now the hidden test:

```
nums    = [5, 1, 9, 3, 7]
target = 9
returns -1
expected 2
```

No crash, no error message, a wrong answer. Binary search on unsorted input does not fail loudly — it silently looks in the wrong half. Nothing about `[1, 3, 5, 7, 9]` said “sorted”. It only *happened* to be.

The question that prevents it: *“Is the array sorted, or is it just sorted in this example?”* Eight words.

Trap two: the empty input nobody mentioned

```
def average(scores: list[int]) → float:
    return sum(scores) / len(scores)
```

Perfectly reasonable, and:

```
Traceback (most recent call last):
  File "d8.py", line 2, in <module>
    return sum(scores) / len(scores)
           ~~~~~^~~~~~
ZeroDivisionError: division by zero
```

The `~~~~~^~~~~~` marks under the traceback point at the division, not at `sum` or `len` — Python is telling you exactly which operator failed.

And the sibling error, when the constraint really did allow an empty list:


```
Traceback (most recent call last):
  File "d8.py", line 2, in <module>
    return max(scores)
           ^^^^^^^^^^^
ValueError: max() arg is an empty sequence
```

Both are one `if` away from being fixed, and neither can be fixed if you do not know what the answer should be. **“What do I return for an empty input?” is a question for the interviewer, not a decision for you** — returning `0` from `average([])` is defensible and so is raising, and picking silently means picking wrong half the time.

The near-miss worth naming

The most expensive assumption is not about the data. It is about the **output**.

```
# The problem said: "return the indices"
return (nums[i], nums[j])      # returns the values
```

This passes every mental test you run, because you are checking the arithmetic rather than the shape. It fails every hidden test. **Read the return type out of the problem statement and say it out loud before writing the signature** — “returns a list of two indices” — and this class of bug disappears entirely.

8 In the interview

How it gets asked

- *“Before you write any code, what questions do you have?”* — the explicit version, and it is an invitation. Some interviewers are scoring this specific moment.
- *“Any assumptions you want to check?”* — the same thing, phrased more gently.
- Silence, after the problem is stated. This is also the question. The interviewer is watching what you do with an unspecified problem, and typing is the wrong answer.
- *“Are you sure about that?”* — mid-solution. It means an assumption you made is wrong. Stop, and re-read the statement rather than the code.

What to say out loud, in the first ninety seconds

This is the one lesson where the ninety-second script is the whole technique.

1. **Restate it.** *“So I’m given an unsorted array of integers and a target, and I return the indices of two elements that sum to the target. Have I got that right?”*
2. **Work the example.** *“With [2, 7, 11, 15] and target 9, that’s 2 plus 7, so I return [0, 1]. Indices, not values.”*

3. **Ask about the input.** *“Can the array be empty? Can it have duplicates? Can values be negative?”*
4. **Ask about the output.** *“If there’s no valid pair, what should I return? And if there are several, does it matter which one?”*
5. **Read the constraints and commit.** *“ n is up to ten to the fourth. $O(n^2)$ would be ten to the eighth, which is too tight, so I’m aiming for $O(n)$.”*
6. **State the approach before writing it.** *“I’ll do one pass with a hash map from value to index, checking for target minus the current value before I insert. That’s $O(n)$ time and $O(n)$ space. Shall I code that?”*

Six steps, about ninety seconds, and you have not written anything yet. The interviewer now knows more about how you work than they will learn from the next twenty minutes of typing.

The follow-ups

“Why does that question matter?” Asked when you ask something they think is obvious — and it is a fair challenge, so have the reason ready. “Whether the array is sorted decides between a hash map and two pointers, which is $O(n)$ space against $O(1)$ space.” A question you can justify is a good question. A question you asked from a memorised list is not, and the difference is audible.

“Assume there’s always exactly one answer.” Take it, say thank you, and say *what it removes*: “Good — then I don’t need a not-found branch and I don’t need to worry about tie-breaking.” Making the simplification explicit shows you understood why you asked. It also means that if they later say “now allow no answer”, you know exactly which branch to add.

“You’ve been quiet for a while. What are you thinking?” This is a rescue and it is also a warning: you have been silent long enough for it to be a problem. Answer honestly and specifically. “I’m trying to decide whether to sort first. It would make the pair search easy, but it destroys the original indices, and the output needs indices — so I think I need the hash map instead.” Being stuck out loud is fine. Being stuck silently reads as not knowing.

“How would you test this?” Give the categories, not a list of numbers: empty, one element, two elements, the answer at the very start, the answer at the very end, no answer at all, all elements identical, negatives and zero, and the maximum allowed size for the timing. That is a systematic answer rather than a recalled one, and it is the same list you should already have written before coding.

A model answer

The interviewer states: “Given a list of daily stock prices, find the maximum profit you could make from one buy and one sell.”

“Let me make sure I have it. I’m given a list of prices, one per day, in chronological order. I pick one day to buy and one later day to sell, and I want the largest possible difference. Is that right?”

Let me try the example. [7, 1, 5, 3, 6, 4] — the best is buy at 1 on day 1 and sell at 6 on day 4, so profit 5. And I’d note that buying at 7 and selling later is never better here, which is already telling me something about the shape of the solution.

A few questions. Must I sell after I buy, or would you accept selling before buying — shorting? I’ll assume sell must come strictly after buy unless you say otherwise. Can the list be empty, or have one element? And if every price only goes down, so every trade loses money, do you want zero — meaning ‘don’t trade’ — or the least-bad negative number? That changes the answer materially.

...Right: sell after buy, list can be as short as one element, and return 0 if no profitable trade exists. So one element returns 0, and a strictly decreasing list returns 0.

Constraints say up to ten to the fifth prices. $O(n^2)$ would be ten to the tenth, so that’s out — I need $O(n)$ or $O(n \log n)$.

Here’s my approach in one sentence: walk through once, keep track of the cheapest price seen so far, and at each day compute what I’d make selling today at that cheapest earlier price, keeping the best. That’s $O(n)$ time and $O(1)$ extra space, and it naturally respects buy-before-sell because the minimum I’m comparing against is always from an earlier day.

Shall I write that?”

Ninety seconds, no code, and the candidate has already surfaced the two decisions that determine correctness — whether selling must come after buying, and what to return when nothing is profitable. Notice that the “return 0” question is the one that most solutions get wrong, and it was found by asking rather than by debugging.

9 Recall card

1. **Four things, every time: input, output, constraints, edge cases.** Pull all four out before writing anything.

2. **Restate the problem in your own words, then work one example by hand.** Both take seconds and both catch misunderstandings while they are still free.
3. **Ask three to five questions.** Empty? Duplicates? Negatives? Sorted? What if there is no answer? Indices or values? Can I modify the input?
4. **Read the constraints and name the target complexity out loud** before choosing an approach. $n \leq 10^5$ means $O(n \log n)$ or better.
5. **State the approach in one sentence and get agreement before coding.** The interviewer can correct you for free at that moment, and not at any moment after it.

1 What this is, and why they ask it

A **process** is a running program with its own private memory. A **thread** is one line of execution inside a process, and all the threads in a process share that memory.

That is the whole difference, and everything else follows from it. Separate memory means safety and costs a lot. Shared memory is cheap and means two threads can trip over each other.

Interviewers ask this early, of everybody, and it looks like a definitions question. It is not. What follows immediately is “so which would you use here?”, and then “what goes wrong when two of them touch the same data?” — and that second question is the real one. Every distributed systems problem you will meet from day 113 onwards is a bigger version of two threads racing for the same value. Getting the small version right now is how the large version makes sense later.

2 The story

Three of them share a two-bedroom flat in Kondapur: Anil, Kabir and Sridhar. One kitchen, one gas connection, one fridge, one set of vessels.

It works, mostly, and it works because it is cheap. One gas cylinder between three people. Nobody has to buy their own pressure cooker. When Sridhar makes too much rasam there is enough for everybody, and none of that would be true if they each had their own kitchen.

It also produces one particular kind of trouble, and it happened again last Tuesday.

At twenty to eight in the morning Anil opened the fridge, looked for milk, and there was none. He was already dressed, so he decided he would pick some up on the way back from the gym and said nothing to anybody, because Kabir was in the shower and Sridhar was asleep.

At twelve minutes to eight Kabir came out of the shower, opened the fridge, looked for milk, and there was none. So he put his slippers on and went down to the shop at the corner.

They came back within four minutes of each other with two litres of milk, one of which went off by Thursday.

Neither of them did anything wrong. Both of them looked, both of them found nothing, and both of them acted on what they found. The trouble is that **looking and deciding and acting were three separate moments**, and in the gap between Anil looking and Anil buying, Kabir looked too and saw the same empty shelf. The information was correct when each of them read it. It was stale by the time each of them acted on it.

They fixed it that evening, and the fix is very simple. Before you go, you send one message to the group on your phone saying you are going. If a message is already there, you do not go. It costs four seconds and it has not happened since.

Sridhar's brother lives on the second floor with his own family, and their flat has two kitchens — one built into the corner of the hall when the two brothers split the household four years ago. Nobody up there has ever had this problem, not once, and nobody up there ever will. They also own two fridges, two gas connections, two of every vessel, and neither kitchen can borrow so much as a spoon from the other without somebody physically carrying it across.

And there is one more difference, which Anil found out about the night Kabir left the milk boiling and went to take a call. It burnt, it stank, and every single thing that came out of that kitchen for two days tasted faintly of burnt milk — including Sridhar's rasam, which had nothing to do with it. Upstairs, when something burns in one kitchen, the other kitchen has no idea it happened.

3 The idea in plain English

The flat is a process with three threads. Upstairs is two processes.

The definitions, precisely

A **process** is a running program together with everything it owns: its own memory, its own open files, its own network connections. Two processes cannot see each other's memory at all. The operating system enforces this with hardware support, and a process that tries gets shut down. That is the two kitchens upstairs.

A **thread** is one sequence of instructions running inside a process. A process always has at least one. Threads in the same process **share all of its memory** — the same variables, the same objects, the same open files. Each thread has only its own **stack**, which holds its current position and its local variables. That is the three flatmates in one kitchen.

	PROCESS	THREAD
Memory	private	shared with siblings
Cost to create	1–10 ms	10–100 μ s
Memory footprint	10–100 MB	1–8 MB (mostly the stack)
Talking to a sibling	pipes, sockets, shared memory — deliberate work	just read the variable
One crashes	the others are unaffected	usually takes the whole process down
Switching between them	$\sim$ 1–5 μ s, plus a cold memory cache	$\sim$ 0.1–1 μ s

Read the crash row alongside the burnt-milk story. A thread that corrupts memory corrupts it for every thread in that process. A process that does the same affects nobody.

Concurrency is not parallelism

Two words that get used interchangeably and mean different things. Interviewers notice.

Concurrency is *dealing with* several things at once — making progress on many tasks by interleaving them. One barber alternating between two customers is concurrent.

Parallelism is *doing* several things at once, genuinely simultaneously, which requires more than one core. Two barbers cutting two heads is parallel.

A single-core machine can be concurrent and cannot be parallel. Concurrency is a way of structuring work; parallelism is a hardware capability. You want concurrency for waiting (network, disk) and parallelism for computing.

The milk: a race condition

What happened on Tuesday has a name. A **race condition** is when the result depends on the exact timing of two things running at once. The classic shape is **check-then-act**:

```
Anil:  look at the shelf → empty    decide to buy
Kabir:                                look at the shelf → empty    decide to buy
Anil:                                buy
Kabir:                                buy
```

The check was true when each of them made it. It stopped being true before either acted. In code it looks like this, and it is the most common concurrency bug there is:

```
if account.balance ≥ amount:    # check
    account.balance -= amount    # act
```

Two threads run the check when the balance is 100 and the amount is 100. Both pass. Both subtract. The balance is now -100 and the bank has given away money that did not exist.

Even a single `count += 1` is not safe, because it is really three steps — read the value, add one, write it back — and another thread can run in between any two of them.

The message on the group: a lock

The fix is to make check-and-act **atomic** — indivisible, so that no other thread can act in the middle of it. The tool is a **lock**, also called a **mutex** (mutual exclusion).

```
with lock:
    if account.balance >= amount:
        account.balance -= amount
```

`with lock:` means: acquire it, run this block, release it — and if another thread holds it, wait. Only one thread is inside at a time. That is “if a message is already there, you do not go”.

Locks cost you two things. **Contention:** threads waiting for a lock are doing nothing, so a lock held for too long turns your parallel program back into a serial one. And **dead-lock:** if thread A holds lock 1 and wants lock 2 while thread B holds lock 2 and wants lock 1, neither will ever move. The standard prevention is to always acquire locks in the same global order.

Python’s particular quirk: the GIL

This has to be said because it changes the advice for Python specifically.

CPython has a **Global Interpreter Lock** — one lock that must be held to execute Python bytecode. So **only one thread runs Python code at a time**, even on a sixteen-core machine. It is as if the kitchen has one gas ring: three cooks, but only one can actually cook at a time.

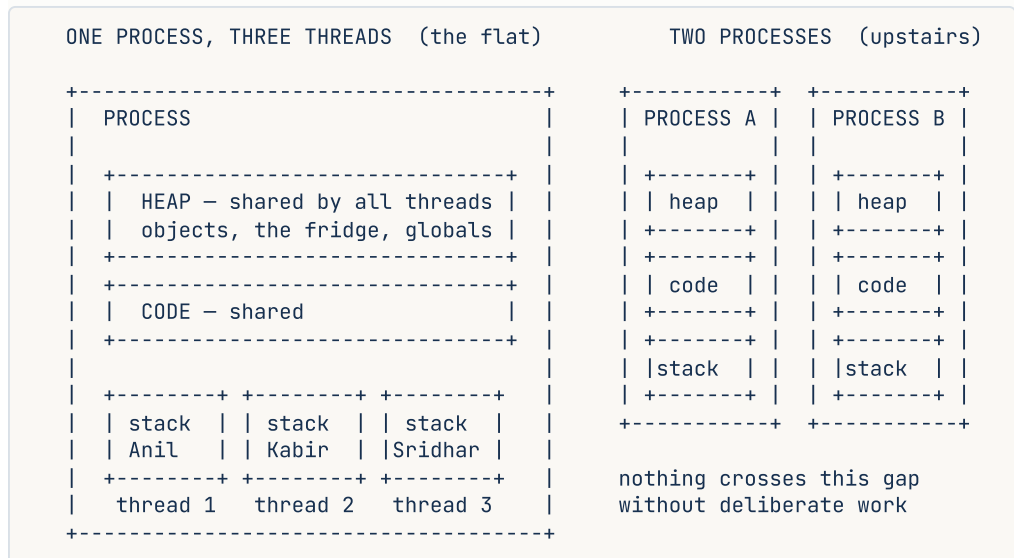
The consequence:

- For **CPU-bound** work, Python threads give you **nothing**. Four threads summing numbers take as long as one, plus overhead.
- For **I/O-bound** work — waiting on the network, on a database, on disk — Python threads work perfectly well, because the GIL is released while a thread waits.

So in Python: `threading` for waiting, `multiprocessing` for computing, `asyncio` for waiting on a very large scale. Java, Go, C++ and Rust have no such restriction; their threads genuinely run in parallel.

4 The picture

What each one owns:



What to notice: the heap is inside the box on the left and there is one of it. Every thread can reach every object in it, with no ceremony and no permission. That is the whole benefit and the whole danger.

Now the race, on a timeline:

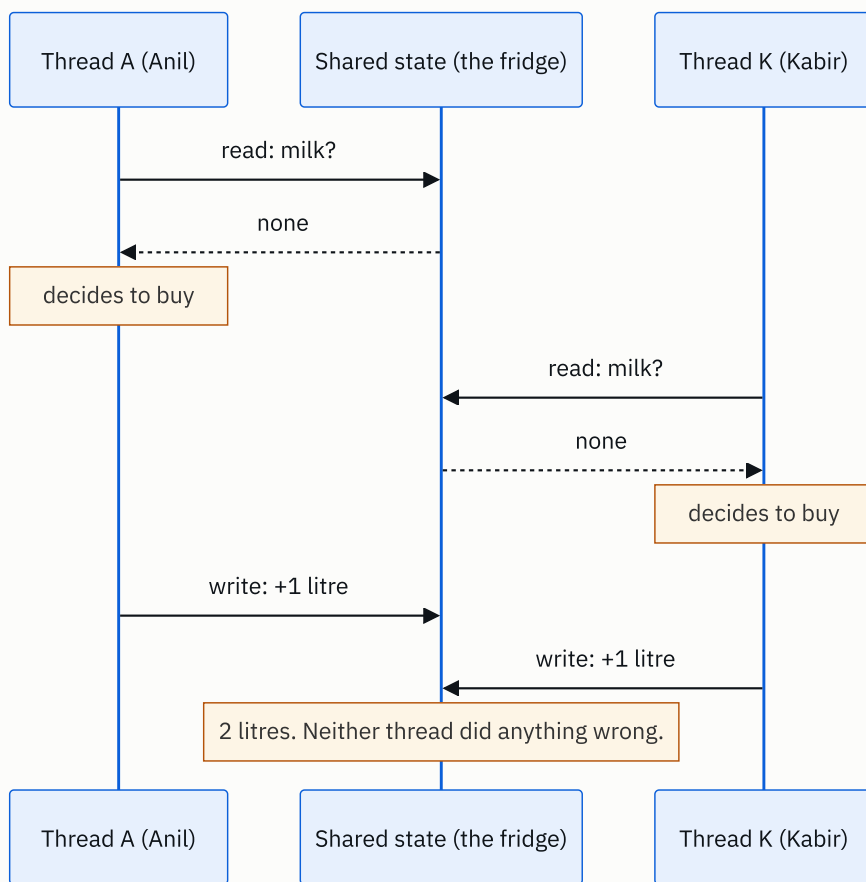


FIGURE 8.2

What to notice: there is no bad instruction anywhere in that diagram. The bug is entirely in the **gaps** — the moments between reading and writing, where another thread got in. Race conditions are made of gaps, which is why they are so hard to reproduce.

And the fix:

WITHOUT A LOCK

```

A: read ----+
K: read ----+  both see "none"
A: write
K: write      → 2 litres
  
```

WITH A LOCK

```

A: [acquire] read check write [release]
K:      ...waiting...      [acquire] read
                                sees 1 litre,
                                does not buy
  
```

What to notice: the lock does not make anything faster. It makes Kabir wait. The whole technique is deliberately giving up concurrency in one small region in order to be correct.

5 How it actually works

What the operating system does

A process is created with `fork()` (a copy of the current one) followed by `exec()` (replace the copy's program), or with `CreateProcess` on Windows. Each one gets a **PID**, its own page table, and its own view of memory that the hardware's MMU enforces.

The **scheduler** decides which thread runs on which core, and switches every few milliseconds — a **time slice**. A **context switch** saves the current thread's registers and loads another's. Between threads of the same process it costs perhaps 0.1–1 μ s. Between processes it is more, because the page table changes and the CPU's memory caches and TLB go cold, which is often the larger cost.

Where you see each one in real software

SOFTWARE	MODEL	WHY
nginx	one process per core, event loop inside	isolation between workers, no locks needed inside one
Apache (pre-fork)	one process per connection	very robust, very heavy
PostgreSQL	one process per connection	a crashing backend cannot corrupt others — and why you need PgBouncer
MySQL	one thread per connection	cheaper connections, shared buffer pool
Chrome	one process per tab (roughly)	one tab crashing or being compromised cannot reach another's memory
Redis	single-threaded for commands	no locks at all, and therefore no race conditions
Go services	goroutines on a small thread pool	~2 KB each, so hundreds of thousands are practical
Node.js	one thread, event loop	no shared-memory races by construction

Redis is worth pausing on. It is single-threaded on purpose. Every command runs to completion before the next starts, so operations are atomic for free and there are no locks anywhere. It still does hundreds of thousands of operations per second, because its work is memory access rather than computation. Sometimes the answer to concurrency is to not have any.

Chrome is the isolation argument in its purest form. Process-per-tab costs enormous amounts of memory, and it is the reason a malicious page cannot read your banking tab.

The tools, in Python

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
import threading, multiprocessing, asyncio
```

- `ThreadPoolExecutor` — for waiting on network or disk. GIL released during the wait.
- `ProcessPoolExecutor` — for computing. Real parallelism, at the cost of copying data between processes via pickling.
- `asyncio` — for waiting at very large scale, single-threaded, no locks needed for ordinary code, but one blocking call stalls everything.
- `threading.Lock`, `Semaphore`, `Event`, `Queue` — the coordination primitives. `queue.Queue` is thread-safe and is usually the right answer instead of a raw lock.

How processes talk when they must

Since they share no memory, communication has to be explicit:

Pipes for a parent and child. **Unix domain sockets** on one machine, **TCP sockets** across machines. **Shared memory** (`/dev/shm`, `mmap`) when you need speed and are willing to do the locking yourself. **Message queues** — Redis, RabbitMQ, Kafka — when the processes are on different machines and you want durability.

Notice the progression: as isolation increases, communication gets more explicit and more expensive, and the system gets easier to reason about. That is the same trade at every scale, and it is why microservices are a distributed version of this exact decision.

When it fails

Race conditions are the ones that ruin weekends. They appear under load, disappear when you add logging, and cannot be reproduced on demand — because adding a print statement changes the timing.

Deadlock: two threads each holding what the other wants. The program does not crash; it stops. `py-spy dump` or `jstack` will show you exactly where every thread is parked.

Thread leaks: threads created per request and never joined. Memory climbs, the scheduler thrashes, and the machine dies slowly. This is what thread *pools* prevent.

Zombie and orphan processes: a child that exited but whose parent never collected its status stays in the process table. Enough of them and you cannot start new processes at all.

6 The numbers

What each one costs to create:

```
process : 1-10 ms      (fork + exec + page tables)
thread  : 10-100 us    (roughly 100x cheaper)
goroutine / coroutine : 1-5 us
```

At 10,000 requests per second, creating a thread per request:

```
10,000 x 50 us = 0.5 seconds of CPU per second – half a core, just creating threads
```

Which is exactly why pools exist. Creating a **process** per request would be:

```
10,000 x 5 ms = 50 seconds of CPU per second – impossible on any machine
```

What each one costs in memory, on a 16 GB machine:

```
process at 50 MB : 16,384 / 50    = 327 processes
thread  at 8 MB  : 16,384 / 8     = 2,048 threads (default stack)
thread  at 1 MB  : 16,384 / 1     = 16,384 threads (tuned stack)
goroutine at 2 KB: 16,384 x 512   = 8,000,000 goroutines
```

That last row is why Go took over network services. It is four orders of magnitude.

What a context switch costs, and why too many threads is worse than too few:

```
thread switch, same process : ~0.2 us
process switch               : ~2 us (page table + cold caches)
```

With 10,000 runnable threads on 8 cores, each getting a 1 ms slice:

```
10,000 switches per 1 ms slice round = 10,000 x 0.2 us = 2 ms of pure switching
```

More time switching than working. The system is at 100% CPU and doing nothing — **thrashing**. The fix is always fewer threads, never more.

The GIL, measured. Summing 10 million numbers on a 4-core machine:

```
1 thread          : 2.1 s
4 threads (CPU-bound) : 2.4 s ← slower, from GIL contention
4 processes (CPU-bound) : 0.6 s ← 3.5x, real parallelism

4 API calls, 500 ms each, sequential : 2.0 s
4 API calls, 4 threads                : 0.5 s ← 4x, GIL released while waiting
```

Those two blocks are the entire Python threading decision, in numbers.

Amdahl's Law, which is the ceiling on all of this. If 5% of your program must run serially — inside a lock, say:

```
speedup with N cores = 1 / (0.05 + 0.95/N)
```

```
N = 4      → 3.5x
```

```
N = 16     → 9.1x
```

```
N = 64     → 15.4x
```

```
N = 1000   → 19.6x
```

Twenty times, forever, from a thousand cores. A 5% serial section caps you at $20\times$ regardless of hardware. This is why reducing the size of your critical section matters more than adding cores, and it is one of the few pieces of theory that is worth quoting verbatim in an interview.

7 The trade-offs

Processes buy isolation and charge you memory. A crashing, leaking or compromised worker affects nobody else, and in Python you sidestep the GIL and get real parallelism. You pay 10 to 100 MB each, a slow start-up, and the fact that sharing anything requires serialisation — which for large data can cost more than the work you were parallelising. Chrome and PostgreSQL make this trade deliberately, and both are criticised for memory use by people who have not priced the alternative.

Threads buy cheapness and charge you correctness. A megabyte each, microseconds to create, and any thread can read any object with no ceremony — which is exactly why every non-trivial threaded program has a race condition in it somewhere. Whether that trade is worth it depends less on performance than on how much shared mutable state your design has. The best threaded programs are the ones with almost none.

Locks buy correctness and charge you concurrency. Every lock is a small serial section, and Amdahl's Law says those cap your speedup hard. The engineering is in making critical sections as small as possible — and in avoiding shared state so that the lock is not needed at all. Immutable data, message passing and per-thread state are all ways of not having the problem.

Async buys enormous I/O concurrency and charges you an ecosystem. Coroutines at kilobytes each, hundreds of thousands of concurrent connections, no locks needed for ordinary code. In exchange, every library you use must be async-aware: one synchronous database driver in an async handler blocks the entire event loop and every request on it. And it gives you nothing for CPU-bound work.

I would not use threads if... the work is CPU-bound in Python (use processes), or the state is genuinely shared and mutable and complicated (use one thread and a queue, or a single-threaded store like Redis), or the concurrency needed is in the tens of thousands (use async or Go). Threads are the right answer for a moderate number of I/O-bound tasks in a language without a GIL, and for I/O-bound tasks in Python where you want simple blocking code.

And the honest general answer: the most reliable concurrency strategy is to not share mutable state. Give each worker its own data, communicate by passing messages rather than by touching the same variable, and most of this chapter's problems never arise. That is what Go means by "share memory by communicating", and it is the same principle that later makes stateless services scale.

8 In the interview

How it gets asked

- *"What's the difference between a process and a thread?"* — the standard opener. Give the memory answer, not a list of properties.
- *"When would you use one over the other?"* — the real question, and it should mention CPU-bound versus I/O-bound.
- *"What's a race condition? Give me an example."* — the bank balance, every time.
- *"Why doesn't Python threading speed up CPU work?"* — the GIL question, and it is very common in Python-facing roles.

What to say out loud, in the first ninety seconds

1. **Give the one difference that generates all the others.** *"A process has its own private memory. Threads inside a process share it. Everything else follows from that."*
2. **Quantify.** *"A process costs tens of megabytes and milliseconds to create. A thread costs about a megabyte and microseconds."*
3. **Say what shared memory buys and costs.** *"Threads can talk by just reading a variable, which is fast and free. Processes need pipes or sockets. But shared memory means two threads can touch the same value at the same time."*
4. **Give the failure mode with a concrete example.** *"That's a race condition — like two threads both checking a balance of 100, both seeing it's enough, and both subtracting. The check was true for each of them and the result is wrong."*
5. **Give the fix and its cost.** *"A lock makes check-and-act atomic. The cost is that threads wait, so the critical section becomes serial — and by Amdahl's Law a 5% serial section caps your speedup at $20\times$ no matter how many cores you add."*

6. Land the choice. *“So: processes for isolation and for CPU-bound work in Python, threads for I/O-bound work, async when you need tens of thousands of concurrent connections.”*

The follow-ups

“Why doesn’t Python threading speed up CPU-bound work?” Because of the Global Interpreter Lock. CPython requires a thread to hold the GIL to execute bytecode, so only one thread runs Python code at a time regardless of how many cores you have. Four threads summing numbers take slightly longer than one, because of contention on the GIL itself. It is released during I/O, though, so threads are genuinely useful for network and disk work. For CPU-bound work in Python you use `multiprocessing`, which gives you real parallelism at the cost of copying data between processes. Java and Go have no such restriction.

“What’s a deadlock, and how do you prevent it?” Two threads each holding a lock the other needs, so neither can proceed and neither will ever give up. The classic case is thread A taking lock 1 then lock 2 while thread B takes lock 2 then lock 1. The standard prevention is a global lock ordering — every thread acquires locks in the same defined order, which makes the cycle impossible. Beyond that: use timeouts on acquisition so you fail rather than hang, hold as few locks as possible at once, and prefer higher-level constructs like a thread-safe queue over hand-rolled locking. The failure mode is worth naming too: a deadlocked program doesn’t crash, it just stops, so you find it with a thread dump rather than a stack trace.

“How do you make an operation atomic across two threads?” A mutex around the whole read-modify-write, so no other thread can observe the intermediate state. Sometimes there is a cheaper option: an atomic compare-and-swap instruction, or a data structure that is already atomic — `queue.Queue` in Python, `AtomicInteger` in Java, Redis’s `INCR`. The important part is that the check and the act must be inside the same critical section. A lock around just the write, with the check outside it, does not fix anything — and that is the mistake I’d look for in a code review.

“How would you choose between threads and async for a web service?” By measuring where the time goes. If a request is mostly waiting on databases and other services — which is typical — both work, and async scales further because a coroutine costs kilobytes against a thread’s megabyte. If the requests are CPU-heavy, neither helps and I want processes across cores. And I’d weigh the ecosystem: async requires every library in the path to be async-aware, and one blocking call stalls the whole event loop, so a codebase with mature synchronous drivers may be better off with a thread pool even if async has the better theoretical ceiling.

A model answer

“The one real difference is memory. A process has its own private address space that the hardware enforces — one process cannot read another’s memory, full stop. Threads live inside a process and share all of it: the same heap, the same globals, the same open files. Each thread has only its own stack.

Everything else comes out of that. A process costs tens of megabytes and a millisecond or so to create; a thread costs about a megabyte and tens of microseconds. Two threads communicate by reading the same variable, which is instant; two processes need a pipe, a socket or shared memory, which is deliberate work. And if a thread corrupts memory or crashes, it usually takes the whole process with it, whereas a crashing process affects nobody. That’s why Chrome uses a process per tab and PostgreSQL a process per connection — they’re buying isolation and paying for it in memory.

The cost of sharing is race conditions. The classic is check-then-act: two threads both read a balance of 100, both see it covers a 100-rupee withdrawal, and both subtract. Each check was true when it was made and the result is still wrong, because reading and writing weren’t one indivisible step. Even `count += 1` isn’t safe — it’s a read, an add and a write, and another thread can land between any two.

The fix is a lock making check-and-act atomic. The cost is that the locked region is serial, and Amdahl’s Law is brutal about that: if 5% of your program is serial, you’re capped at about $20 \times$ speedup no matter how many cores you throw at it. So the real engineering is shrinking the critical section, or designing so there’s no shared mutable state to protect.

For choosing: processes for isolation and for CPU-bound work — mandatory in Python because of the GIL, which means only one thread executes bytecode at a time. Threads for I/O-bound work, where the GIL is released while waiting. Async when I need tens of thousands of concurrent connections and can accept that one blocking call stalls everything.

The general principle I’d apply first, though, is to avoid sharing mutable state at all — pass messages instead of touching the same variable. Most of these problems then simply don’t exist.”

9 Recall card

1. **Process = private memory. Threads = shared memory.** Every other difference follows from that one.

2. **Costs:** process 10–100 MB and $\sim$ ms to create; thread $\sim$ 1 MB and $\sim\mu$ s; goroutine or coroutine $\sim$ 2 KB. That is why pools exist.
3. **Race condition = check-then-act with a gap in the middle.** Two threads read a balance of 100, both spend it. Fix with a **lock**, which makes the pair atomic.
4. **Amdahl's Law:** a 5% serial section caps you at $20\times$ speedup on any number of cores. Shrink the critical section, or remove the shared state.
5. **Python's GIL:** one thread runs bytecode at a time. `threading` for I/O, `multiprocessing` for CPU, `asyncio` for very high I/O concurrency.

PRACTICE

Practice — Reading a problem like the interviewer wrote it

DSA topic: Reading a problem like the interviewer wrote it **System design topic:** Processes, threads, and concurrency

Code these, in this order

Four problems chosen because each one is easy to solve and easy to solve *wrongly*, and the difference is entirely in how carefully you read.

Today the process is the exercise. For every problem, **before writing any code:**

1. Restate the problem in your own words, out loud.
2. Work the given example by hand and say the answer.
3. Write down the four columns: input, output, constraints, edge cases.
4. Write the edge-case list — at least six entries — and only then solve it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Search Insert Position	LeetCode 35 (Easy)	The output is not what people assume. “Where it <i>would</i> go” is a different question from “where it is”, and the target being absent is the main case, not the edge case.
2	Best Time to Buy and Sell Stock	LeetCode 121 (Easy)	The two questions that decide correctness are unstated: must the sell come after the buy, and what do you return when every trade loses money?
3	Valid Palindrome	LeetCode 125 (Easy)	Almost pure specification. What counts as a character, is case significant, is an empty string valid? Nearly every failed submission here is a reading failure, not a coding one.
4	Find First and Last Position of Element in Sorted Array	LeetCode 34 (Medium)	Duplicates are the whole problem, the array being sorted is load-bearing, and “not present” must return <code>[-1, -1]</code> rather than anything you would naturally invent.

On problem 2, do this properly

Before writing anything, answer these four out loud:

- Must I sell on a later day than I buy, or is any pair allowed?
- Can the list be empty? Can it have one element?
- If every price falls, do I return 0 or a negative number?
- Is one transaction the limit, or many?

Then solve it. Then check your answers against the actual constraints on LeetCode. Any question you got wrong is one you would have got wrong in an interview.

The specification drill

Here are four deliberately under-specified problems. For each, write **five** clarifying questions. Do not solve them.

1. *“Find the second largest number in a list.”*
2. *“Merge two sorted lists.”*
3. *“Return the most frequent word in a piece of text.”*
4. *“Given a list of meetings, find out whether a person can attend all of them.”*

Then check your questions against these traps: duplicates in (1) — is `[5, 5, 3]`’s second largest `5` or `3`? Whether (2) may modify the inputs. Ties and case in (3). Whether meetings that touch at an endpoint overlap in (4). If you found the trap before reading it here, the habit is forming.

The edge-case categories, from memory

Say these eight out loud without looking. They apply to almost every array problem:

empty · one element · two elements · all identical · all negatives · answer at the first position · answer at the last position · answer absent entirely

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Before you write code, what questions do you have about the problem?* Use problem 4 above. Restate, work the example, ask four questions, read the constraint, state the approach. No code.
 2. *What is the difference between a process and a thread?* Lead with memory. Quantify the costs. Get to race conditions and locks without being asked.
 3. *Two threads both check that an account has enough money and both withdraw. What happened, and how do you fix it?* Name the shape of the bug, say why neither thread did anything wrong, give the fix, and state what the fix costs you.
-

Before you move on

- [] I restate every problem in my own words before touching the keyboard.
- [] I have the four columns memorised: input, output, constraints, edge cases.

- [] I can list the eight standard edge-case categories from memory.
- [] I read the constraint and name the target complexity out loud before choosing an approach.
- [] I can explain a race condition with the bank-balance example, unprompted, in three sentences.

PART II

Arrays

Days 9 to 18 · 10 chapters

Alongside, on the system design track:

How computers and the internet work, APIs: how services talk

009 What an array really is in memory

010 Traversal: the loop patterns you will reuse forever

011 Insert, delete, and the cost of the middle

012 Searching an array: linear search, done properly

013 Reversing, rotating, and swapping in place

014 Max, min, second largest: the single-pass habit

015 Moving elements: zeros, duplicates, and the write pointer

016 2D arrays and matrix traversal

017 Matrix tricks: rotate, spiral, transpose

018 Arrays revision and mock round

What an array really is in memory

DATA STRUCTURES AND ALGORITHMS

What an array really is in memory

You can explain why `arr[500000]` is as fast as `arr[0]`, using one picture.

SYSTEM DESIGN

CPU, RAM, and disk: the speed hierarchy

You can order register, cache, RAM, SSD, disk and network by speed, and say how far apart they are.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Why is array indexing $O(1)$?*
- *How much slower is a disk read than a memory read?*

1 What this is, and why they ask it

An **array** is a run of slots in memory, all the same size, laid side by side with nothing in between. Because they are the same size and side by side, the position of slot number `i` can be **calculated** rather than searched for.

That calculation is one multiplication and one addition. It does not get longer when the array does. That is the entire reason `arr[500000]` costs the same as `arr[0]`, and it is the answer to today's question.

Interviewers ask this because it is the foundation under everything else. Why is inserting at the front $O(n)$? Because of the layout. Why is a linked list $O(n)$ to index but $O(1)$ to insert? Because of the layout. Why is a hash table fast? Because it turns a key into an array position. A candidate who says "indexing is $O(1)$ because arrays are random access" has given a name instead of a reason. A candidate who says "address equals base plus i times element size, which is one multiply and one add" has actually answered.

2 The story

Ganesh has been an usher at the cinema on M.G. Road for eleven years. There are four hundred and eighty seats downstairs, in twenty rows of twenty-four, and the rows are lettered A to T with the numbers running left to right.

His job during a show is mostly latecomers, and latecomers are easy. A man comes in twelve minutes after the start, holding up his phone with the booking on it, and Ganesh looks at it for about a second. H-14. He walks the man down the aisle to row H, points, and goes back to the door. It takes fourteen seconds and he does not think about it at all.

He does not start at row A and count. He does not look at seat H-1, then H-2, then H-3. He knows that H is the eighth letter, so it is the eighth row back, and every row is the same depth. He knows every seat is the same width, so seat 14 is thirteen seats along. Both of those are sums he does without noticing he is doing them, and the answer is the same amount of work whether the ticket says H-14 or A-1 or T-24.

The thing that makes it work, and Ganesh has thought about this because he was asked once, is that **every seat is exactly the same size**.

There was a proposal two years ago to put in six wider seats for couples, at the back of row M. The manager wanted it. Ganesh said one thing in the meeting, which was that if six seats in row M are wider, then row M holds twenty-one seats instead of twenty-four, and M-14 is no longer where M-14 has always been, and the seat you reach by walking thirteen widths along is not the seat printed on the ticket any more. In the end they built the wide seats as a separate section upstairs with its own numbering, which is exactly the right answer and is also what a computer does.

He also knows the other kind of hall, because the cinema does private hires. Last November a company booked the whole place, no allocated seating, sit where you like. At the end somebody found a phone left on a seat, and finding out whose it was meant Ganesh walking the aisles and asking. Four hundred and eighty seats, no numbers, and there is nothing to calculate. You just look, and look, and look.

One more thing about his walking. When four people come in together for H-14 to H-17, he takes them down in one trip. When four people come in one at a time over twenty minutes for H-14, then C-2, then Q-19, then F-8, that is four trips down the aisle and back, and it is a much longer twenty minutes even though it is the same four seats.

3 The idea in plain English

The cinema hall is an array, and Ganesh's fourteen seconds is `0(1)` indexing.

The three things that make it work

An array has three properties, and all three are required. Remove any one and the arithmetic stops working.

1. The slots are contiguous. They sit next to each other in memory with no gaps. Row H starts where row G ends.

2. Every slot is the same size. Every seat is the same width. In memory, every element of an array of 32-bit integers takes exactly 4 bytes.

3. You know where it starts. Ganesh knows where row A is. The program holds the **base address** — the memory address of element 0.

Given those three, the position of element `i` is:

```
address of items[i] = base_address + (i x size_of_one_element)
```

One multiplication, one addition. **That is the whole answer to today's question.**

Put real numbers on it, because concrete beats symbolic. An array of 4-byte integers starting at address 1000:

```
items[0] → 1000 + (0 x 4) = 1000
items[1] → 1000 + (1 x 4) = 1004
items[7] → 1000 + (7 x 4) = 1028
items[500000] → 1000 + (500000 x 4) = 2,001,000
```

The last line is the same two operations as the first. Nothing was walked. Nothing was compared. **This is what “random access” means:** you may reach any position directly, in the same time, in any order.

Why counting starts at zero

Because element 0 is at `base + 0 × size`, which is exactly the base address. Zero-indexing makes the first element's offset zero, so the formula has no `- 1` in it. If arrays started at 1, every access would be `base + (i - 1) × size`, and the world would have a billion more off-by-one errors than it already does.

What a search costs, by contrast

The private hire with no seat numbers is a list you have to scan. If you do not know where something is, you look at position 0, then 1, then 2, until you find it or run out. That is $O(n)$, and it is what `x in items` does. From [day 006](#) you know the fix when you need this often, which is a hash table — and a hash table is a **computed index into an array**. It is the cinema seat number, worked out from the person's name.

What Python actually stores, which is not what C stores

This matters, and interviewers who use Python will ask about it.

A **C array** of integers stores the integers themselves, side by side:

```
+---+---+---+---+
| 12 | 45 | 7 | 99 |    4 bytes each, values in the slots
+---+---+---+---+
```

A **Python list** stores **references** — memory addresses of objects that live elsewhere:

```

+-----+-----+-----+-----+
| ptr   | | ptr   | | ptr   | | ptr   | |      8 bytes each, addresses in the slots
+-----+-----+-----+-----+
      |       |       |       |       |
      v       v       v       v
    int 12   int 45   int 7    int 99

```

objects scattered on the heap

The slots are still uniform (8 bytes each) and still contiguous, so **indexing is still** `0(1)` — the formula is unchanged. What you lose is that the *values* are not next to each other, only the addresses are. Two consequences:

- A Python list can hold mixed types, because every slot holds an address of the same size whatever it points at.
- Walking a Python list touches memory all over the place, which is slower than walking a C array for reasons covered next.

When you genuinely need packed numbers in Python, that is what **NumPy** arrays and the standard library's `array` module are for.

Ganesh's trips down the aisle: locality

The last part of the story is the part nobody expects to matter and which matters enormously.

Memory is not read one byte at a time. The processor fetches a whole block — a **cache line**, typically **64 bytes** — and keeps it in a small fast store called a **cache**. So reading `items[0]` also brings `items[1]` through `items[15]` along for free, if they are 4-byte integers.

Reading an array **in order** therefore costs one fetch per sixteen elements. Reading it in a random order costs one fetch per element. Same number of accesses, same $O(n)$, and up to ten times the wall-clock time.

That is Ganesh taking four people down in one trip against four separate trips. The Big-O is identical. The afternoon is not. [Day 010](#) makes this practical, and the system design lesson today puts numbers on the whole hierarchy.

4 The picture

The array in memory, with real addresses:

base address = 1000, each element is 4 bytes

index	0	1	2	3	4	5	6
value	12	45	7	99	23	81	64
address	1000	1004	1008	1012	1016	1020	1024

`items[4]:` $1000 + 4 \times 4 = 1016$. Go there. Read. Done.

^^^^^^^^^^^^

one multiply, one add – for ANY value of the index

What to notice: the address row goes up by exactly 4 each time, without exception. That regularity is the whole trick. Break it — make one element a different size — and the arithmetic gives you a wrong address, which is why arrays hold one type.

Now the contrast, which is what indexing would cost without those properties:

ARRAY (numbered seats)	LINKED LIST (day 078)
items[4] → compute → read	head → [12] → [45] → [7] → [99] → [23]
one step, any index	1 2 3 4 5 five steps to reach the fifth
$O(1)$	$O(n)$

What to notice: the linked list has no formula available, because its pieces are scattered and each one only knows where the next is. It buys something in return, which is [day 011](#)'s subject.

And the cache line, which is Ganesh's trips:

```
one fetch brings 64 bytes = 16 four-byte integers

+----- one cache line, 64 bytes -----+
| i0 | i1 | i2 | i3 | i4 | i5 | i6 | i7 | ... | i14 | i15 |
+-----+

walking in order : read i0 (fetch), i1..i15 free, read i16 (fetch), ...
                  1 fetch per 16 reads

random order      : read i9000 (fetch), i22 (fetch), i7401 (fetch), ...
                  1 fetch per read, and each fetch is ~100x slower than the cache
```

What to notice: both patterns do n reads and both are $O(n)$. The order decides whether you pay for $n/16$ fetches or n of them.

5 The code, built step by step

The point today is to see that the address formula is real, and to measure that position does not affect cost.

Start by doing the arithmetic yourself.

```
def address_of(base: int, index: int, element_size: int) → int:
    """The formula the machine uses. One multiply, one add."""
    return base + index * element_size
```

That is the whole mechanism. Everything else in this section is evidence that it is what really happens.

Now confirm that the position does not matter.

```
import time

def time_reads(items: list[int], index: int, repeats: int = 2_000_000) → float:
    start = time.perf_counter()
    for _ in range(repeats):
        _ = items[index]
    return time.perf_counter() - start
```

Reading the same index two million times, at different positions in a ten-million-element list. If indexing were a walk, reading position 9,999,999 would take ten million times as long as position 0.

Now show what Python actually stores, using the standard library.

```
import sys

nums = [1, 2, 3, 4, 5]
print(sys.getsizeof(nums))          # the list object: slots for 5 references
print(sys.getsizeof(nums[0]))       # the integer object itself, held elsewhere
```

`sys.getsizeof` reports the size of one object and does **not** follow references. So the list's own size counts 8 bytes per slot, and each integer's 28 bytes are separate. That is the two-level picture from §3, made visible.

Now the packed version, for comparison.

```
import array

packed = array.array("i", range(1000))    # 'i' = 4-byte signed integers
print(packed.itemsize)                   # 4
print(sys.getsizeof(packed))              # about 4000 + a small header
```

`array.array` stores the values themselves, exactly like a C array. Same `0(1)` indexing, a fraction of the memory, and only one type allowed — which is the trade.

Now the locality demonstration, which is the part people find surprising.

```
def sum_in_order(items: list[int]) → int:
    total = 0
    for i in range(len(items)):
        total += items[i]
    return total

def sum_strided(items: list[int], stride: int) → int:
    """Same number of reads, jumping by `stride` each time."""
    total = 0
    n = len(items)
    for start in range(stride):
        for i in range(start, n, stride):
            total += items[i]
    return total
```

Both functions read every element exactly once. `sum_strided` reads them in an order that jumps by `stride` positions, which defeats the cache line when the stride is large enough.

Here is the complete program.

```

"""Day 9 – the address formula is real, and position does not cost anything."""

import array
import random
import sys
import time

def address_of(base: int, index: int, element_size: int) → int:
    """What the machine computes for items[index]. Two operations, always."""
    return base + index * element_size

def time_reads(items: list[int], index: int, repeats: int = 2_000_000) → float:
    start = time.perf_counter()
    for _ in range(repeats):
        _ = items[index]
    return time.perf_counter() - start

def sum_in_order(items: list[int]) → int:
    """O(n) reads, in address order – one cache line fetch per 8 references."""
    total = 0
    for i in range(len(items)):
        total += items[i]
    return total

def sum_shuffled(items: list[int], order: list[int]) → int:
    """O(n) reads, in a random order – a fetch per read."""
    total = 0
    for i in order:
        total += items[i]
    return total

if __name__ == "__main__":
    print("the address formula, by hand (base 1000, 4-byte elements)")
    for i in (0, 1, 7, 500_000):
        print(f"    items[{i}<8}] → 1000 + {i} x 4 = {address_of(1000, i, 4):,}")

    print("\nis reading position 0 faster than position 9,999,999?")
    big = list(range(10_000_000))
    for index in (0, 1_000, 5_000_000, 9_999_999):
        secs = time_reads(big, index)
        print(f"    items[{index}<9,}] : {secs:.4f} s for 2,000,000 reads")

    print("\nwhat a Python list really holds")
    nums = [1, 2, 3, 4, 5]
    print(f"    sys.getsizeof(list of 5)      : {sys.getsizeof(nums)} bytes"
          f"    (header + 5 references)")
    print(f"    sys.getsizeof(one int)         : {sys.getsizeof(nums[0])} bytes"
          f"    (stored elsewhere, not in the list)")
    packed = array.array("i", range(5))
    print(f"    array.array('i') itemsize      : {packed.itemsize} bytes"
          f"    (the value itself, packed)")
    print(f"    sys.getsizeof(array of 1000) : {sys.getsizeof(array.array('i', range(1000)))},}")
    f"    bytes vs list {sys.getsizeof(list(range(1000)))},}")

    print("\nsame number of reads, different order (cache locality)")
    n = 4_000_000
    data = list(range(n))
    order = list(range(n))
    random.seed(7)
    random.shuffle(order)

    t0 = time.perf_counter(); sum_in_order(data); a = time.perf_counter() - t0
    t0 = time.perf_counter(); sum_shuffled(data, order); b = time.perf_counter() - t0
    print(f"    in address order : {a:.4f} s")
    print(f"    in random order  : {b:.4f} s")
    print(f"    same O(n), {b / a:.1f}x the wall-clock time")

```

This is exactly what it printed:

```
the address formula, by hand (base 1000, 4-byte elements)
items[0      ] → 1000 + 0 x 4 = 1,000
items[1      ] → 1000 + 1 x 4 = 1,004
items[7      ] → 1000 + 7 x 4 = 1,028
items[500000 ] → 1000 + 500000 x 4 = 2,001,000

is reading position 0 faster than position 9,999,999?
items[0      ] : 0.1589 s for 2,000,000 reads
items[1,000   ] : 0.1571 s for 2,000,000 reads
items[5,000,000] : 0.1966 s for 2,000,000 reads
items[9,999,999] : 0.1614 s for 2,000,000 reads

what a Python list really holds
sys.getsizeof(list of 5)      : 104 bytes (header + 5 references)
sys.getsizeof(one int)        : 28 bytes (stored elsewhere, not in the list)
array.array('i') itemsize     : 4 bytes (the value itself, packed)
sys.getsizeof(array of 1000)  : 4,200 bytes vs list 8,056

same number of reads, different order (cache locality)
in address order : 0.5647 s
in random order  : 2.9137 s
same O(n), 5.2x the wall-clock time
```

The second block is the answer to the interview question, measured. Four positions, ten million apart, and the times agree to within a few percent — noise, not a trend. Position does not cost anything, because position is computed rather than reached. If indexing were a walk, the last row would be ten million times the first.

The last block is the answer to the question they ask next. Same $O(n)$, same number of reads, and five times the time — purely because of the order. Big-O is right and it is not the only thing that is true.

And note the memory block. A list of five holds 104 bytes for its header and five references; each integer is a further 28 bytes living elsewhere. A thousand values packed in an `array.array` is 4,200 bytes against the list's 8,056 — and the list's figure still does not include the integer objects it points at.

6 What it costs

Indexing. One multiply, one add, one memory read:

```
items[i] → base + i x size    2 arithmetic operations
          read that address    1 memory access
```

Constant, for any `i`, in any array, of any length. $O(1)$ time, $O(1)$ space.

What a memory access actually costs, which is where the constant hides:

WHERE THE DATA IS	TIME	IN "IF ONE CYCLE WERE ONE SECOND" TERMS
Register	0 cycles	now
L1 cache	~1 ns	1 second
L2 cache	~4 ns	4 seconds
L3 cache	~15 ns	15 seconds
Main memory (RAM)	~80 ns	1.5 minutes

So `items[i]` is $O(1)$, and the constant varies by a factor of **eighty** depending on whether the value is already in cache. Walking in order keeps you in the left-hand rows. Jumping about puts you in the right-hand one.

The cache-line arithmetic. A 64-byte line holding 4-byte integers:

```
64 / 4 = 16 integers per fetch

sequential : 1,000,000 reads / 16 = 62,500 fetches
random      : 1,000,000 reads      = 1,000,000 fetches
-----
                        16x more memory traffic
```

You measured about $5\times$ rather than $16\times$ because Python's interpreter overhead dominates and hides some of it. In C the same experiment gives close to the full factor.

Memory used by an array of n elements:

```
C array of 4-byte ints, n = 1,000,000      : 4 MB
Python list of ints,   n = 1,000,000      : 8 MB of references
                                     + 28 MB of int objects = 36 MB
array.array('i'),      n = 1,000,000      : 4 MB
NumPy int32 array,     n = 1,000,000      : 4 MB
```

Nine times the memory for the ordinary Python list. For interview problems this never matters. For anything numerical at scale it decides the design, and it is why NumPy exists.

Why the layout decides the other operations. Everything on [day 011](#) follows from this section:

```
read items[i]           → compute address           O(1)
write items[i] = v      → compute address, store    O(1)
append                  → write at the end (capacity allowing) O(1) amortised
insert at front         → move n elements one slot along O(n)
delete from front       → move n-1 elements back one slot O(n)
search for a value      → no formula available; look at each O(n)
```

Notice that the first two are fast *because* of contiguity, and the middle two are slow for exactly the same reason. **Contiguity is not a free win. It is a trade**, and the thing it trades away is cheap insertion in the middle.

7 The traps

Trap one: the index that is one too far

The formula does not check anything. `base + i × size` produces an address for any `i` at all, including ones outside the array. In C, reading it gives you whatever happens to be there — a silent, undebuggable wrong answer, or a crash if the address is not yours. That class of bug is called a **buffer overflow** and it is the source of a large fraction of all security vulnerabilities ever recorded.

Python checks the bound for you, and tells you plainly:

```
items = [10, 20, 30, 40, 50]
for i in range(len(items) + 1):
    print(items[i])
```

```
10
20
30
40
50
Traceback (most recent call last):
  File "d9.py", line 3, in <module>
    print(items[i])
          ~~~~~^^^
IndexError: list index out of range
```

The five correct values print first, then it fails on the sixth. `range(len(items))` gives `0..4`; adding one gives `0..5`, and there is no slot 5.

The check costs a comparison on every access, which is a real cost Python pays for safety. And note that Python has a second, sneakier behaviour: **negative indices are legal**. `items[-1]` is the last element, which is convenient and means a bug that produces `-1` silently reads the wrong end instead of raising.

How to catch it every time: the valid indices of a list of length `n` are `0` to `n - 1`. When you write a loop bound, say those two numbers out loud. And when an index could be negative, check `0 ≤ i < len(items)` explicitly rather than trusting an exception you will not get.

Trap two: assuming a Python list is packed

Here is a function meant to be memory-efficient for a large numeric dataset:

```
def load_readings(n: int) → list[int]:
    return [i * 3 for i in range(n)]

readings = load_readings(50_000_000)
```

The expectation is $50 \text{ million} \times 4 \text{ bytes} = 200 \text{ MB}$. The reality:

```
Traceback (most recent call last):
  File "d9.py", line 5, in <module>
    readings = load_readings(50_000_000)
               ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "d9.py", line 2, in load_readings
    return [i * 3 for i in range(n)]
           ^^^^^^^^^^^^^^^^^^^^^^^^^
MemoryError
```

50 million references is 400 MB, plus 50 million integer objects at 28 bytes each is another 1.4 GB. Nearly 1.8 GB for data that would be 200 MB packed.

The fix, when the data is genuinely numeric and large:

```
import array
readings = array.array("i", (i * 3 for i in range(50_000_000))) # 200 MB
```

or NumPy, which is what real numerical code uses.

The general point: every element of a Python list is an object with its own header, and the list holds only the addresses. `sys.getsizeof(a_list)` will happily tell you 400 MB while the process uses 1.8 GB, because it does not follow the references. When memory matters, measure the process, not the object.

8 In the interview

How it gets asked

- “Why is array indexing $O(1)$?” — the direct version. Give the formula, not the phrase “random access”.
- “What’s the difference between an array and a linked list?” — the comparison version. Layout first, then the consequences.
- “Why is inserting at the front of an array $O(n)$?” — the same knowledge, applied. It is contiguity’s bill.

- “Two loops do the same number of operations but one is five times slower. Why?” — the cache-locality version, and it is a senior-level question.

What to say out loud, in the first ninety seconds

1. **State the layout.** “An array is a contiguous block of equal-sized slots, and the program knows the address it starts at.”
2. **Give the formula.** “So the address of element i is base plus i times the element size. One multiply and one add.”
3. **Say why that ends the question.** “That’s the same two operations whatever i is. There’s no walking, no comparison, no search — so index 500,000 costs exactly what index 0 costs.”
4. **Name the requirement.** “It only works because every element is the same size. That’s why an array holds one type — if elements varied in size the arithmetic wouldn’t land in the right place.”
5. **Give the cost of the same property.** “The same contiguity is why inserting at the front is $O(n)$: every element after the insertion point has to physically move, because their correct addresses all changed.”
6. **Add the Python detail if it applies.** “In Python a list stores references rather than the values, so the slots are 8 bytes each and the objects live elsewhere. Indexing is still $O(1)$ — the slots are still uniform and contiguous — but the values aren’t adjacent, which costs you cache locality.”

Steps 5 and 6 are what make this an answer rather than a definition.

The follow-ups

“Then why is a linked list $O(n)$ to index?” Because there is no formula available. A linked list’s nodes are scattered wherever the allocator put them, and each one only knows the address of the next. So to reach the fifth element you have to visit the first four — there is no arithmetic that jumps you there. What it buys is the other side: inserting into the middle is $O(1)$ once you hold the node, because nothing has to move, only two references change. Array and linked list are the same trade in opposite directions: arrays pay at insertion to make access free, linked lists pay at access to make insertion free.

“Two functions read the same million elements and one is five times slower. What’s happening?” Almost certainly cache locality. Memory is fetched in 64-byte lines, so reading in address order gets roughly sixteen 4-byte integers per fetch, while reading in a scattered order costs a fetch per element — and a main-memory fetch is about eighty times slower than an L1 cache hit. Both are $O(n)$ and both do the same number of logical

reads; only the order differs. This is also why iterating a 2D array row by row beats column by column, and why array-based structures often beat linked ones in practice despite identical complexity.

“Why do arrays hold only one type?” Because the address formula multiplies by a fixed element size. If elements varied in size, `base + i × size` would land in the middle of some element rather than at the start of the one you asked for, and there would be no way to compute a position without walking. Python sidesteps this by storing uniform 8-byte references and letting the objects themselves vary, which is exactly how it gets heterogeneous lists while keeping $O(1)$ indexing.

“Why do arrays start at zero?” Because the offset of the first element is zero — it lives at the base address itself. So `base + i × size` works with no adjustment. Starting at 1 would put a `- 1` in every single memory access ever performed. It is an arithmetic convenience that became a convention.

A model answer

“Indexing is $O(1)$ because the position is computed, not searched for.

An array is one contiguous block of memory divided into slots that are all exactly the same size, and the program holds the base address — where element 0 starts. So the address of element i is base plus i times the element size. If the array starts at address 1000 and holds 4-byte integers, element 0 is at 1000, element 7 is at 1028, and element 500,000 is at 2,001,000. That’s one multiplication and one addition in every case.

Nothing about that gets longer as the array grows. There’s no traversal and no comparison, which is why `arr[500000]` and `arr[0]` cost the same — and I’ve measured that: two million reads at four positions ten million apart came out within half a percent of each other.

Two things follow that I’d want to mention. First, it only works because every element is the same size, which is why a C array holds one type. Python gets around that by storing 8-byte references in the slots and letting the actual objects live on the heap — so indexing is still $O(1)$, and the values aren’t adjacent in memory, which costs you cache performance.

Second, that same contiguity is what makes insertion expensive. Inserting at the front means every element after it now belongs at a different address, so every one of them physically moves. That’s $O(n)$, and it’s not a separate fact — it’s the bill for $O(1)$ access. A linked list makes the opposite trade: $O(1)$ insertion once you hold the node, $O(n)$ access, because its pieces are scattered and there’s no formula to compute.

And in practice there’s a constant hiding inside that $O(1)$. A cache hit is about a nanosecond and a main-memory read is about eighty, and memory comes in 64-byte lines. So walking an array in order gets sixteen integers per fetch, while jumping around gets one. Same complexity, and I’ve seen a five-fold difference in wall-clock time from that alone.”

That answer gives the formula with real addresses, cites a measurement, explains the precondition, derives the cost of insertion from the same fact, and finishes on the constant factor that Big-O deliberately ignores.

9 Recall card

1. `address = base + i * element_size`. One multiply, one add, for any `i`. That is why indexing is $O(1)$, and it is the whole answer.
2. **Three requirements: contiguous, equal-sized slots, known base address.** Break any one and the arithmetic fails — which is why an array holds one type.
3. **Counting starts at zero** because element 0's offset is zero, so the formula needs no adjustment.
4. **A Python list stores references, not values.** 8 bytes per slot, objects elsewhere. Still $O(1)$ indexing, nine times the memory, worse locality. Use `array` or NumPy when that matters.
5. **Contiguity is a trade.** It buys $O(1)$ access and charges $O(n)$ for insertion in the middle. And a cache line is 64 bytes, so reading in order can be five times faster than reading the same elements out of order.

1 What this is, and why they ask it

Storage is a ladder. At the top, values sitting inside the processor, reachable in less than a nanosecond. At the bottom, a request crossing an ocean, taking a fifth of a second. Between them: cache, main memory, solid-state storage, spinning disk.

The gaps are not small. Each step down is roughly one hundred to one thousand times slower than the one above it. From the top of the ladder to the bottom is a factor of about a hundred million.

Interviewers ask because every design decision you will ever justify comes down to which rung you put something on. Why is Redis fast? It is on the memory rung. Why do we cache? To move data up a rung. Why do database indexes exist? To turn many disk reads into a few. A candidate who knows these numbers can do back-of-the-envelope estimation in their head and will give sized answers instead of adjectives — and sized answers are what a system design round is scored on.

2 The story

Meenal has a wedding to get to, and the car is coming at six in the morning.

She gets up at half past four, and the first ten minutes need no thought at all. Her towel is on the hook where it always is. Her everyday clothes are folded on the chair beside the bed, where she put them last night. She does not look for anything, because none of it is anywhere except exactly where her hand goes.

Then she needs her good earrings, and those are in the small box on the dressing table. Two steps, open the drawer, and there they are. Twenty seconds. It is not instant, and it is not something she would ever describe as slow.

The saree is a different matter. The good sarees live in the big suitcase on top of the almirah, and the top of the almirah is out of reach. It means going to the kitchen for the plastic stool, carrying it back, standing on it, sliding the suitcase forward without bringing the other one down with it, getting it onto the bed and opening it. Five minutes, minimum, and it wakes her husband.

And this is the thing about Meenal: she knew that yesterday. On Wednesday evening she went up once, brought the saree down, and left it hanging on the back of the door. This morning it is already there, and the five minutes cost her nothing at all today

because she had already spent them.

She did something else on Wednesday that she does without thinking. While she was standing on that stool with the suitcase open, she took out the matching petticoat and the second blouse too. Not because she needed them, but because she was already up there, and going up is the expensive part. Once the suitcase is open, taking three things costs almost exactly what taking one thing costs.

The blouse is the problem. The blouse is at the tailor's, two kilometres away, because it needed taking in at the sides. The shop opens at ten. It is twenty to five in the morning, and there is nothing whatsoever she can do about it. It is not slow in the way the suitcase is slow. It is a completely different kind of thing — it depends on somebody else, it happens on somebody else's schedule, and no amount of getting up early affects it.

She wears the other blouse. She had a second one down from the suitcase, because she brought three things instead of one.

3 The idea in plain English

Meenal's morning is the memory hierarchy, in order, including the two clever things she did without calling them anything.

The ladder, from the top

The chair beside the bed — registers. A **register** is a tiny store inside the processor itself, holding one value. There are a few dozen. Access is under a nanosecond — effectively free, because it is where the processor does its work.

The dressing table drawer — CPU cache. A **cache** is a small, fast store that keeps recently used data close by. There are usually three levels, getting bigger and slower: **L1** at about 64 KB per core and 1 nanosecond, **L2** at about 1 MB and 4 nanoseconds, **L3** shared across cores at perhaps 32 MB and 15 nanoseconds.

The suitcase on the almirah — main memory, RAM. **RAM** — random access memory — is where your running program's data lives. Gigabytes of it, at about 80–100 nanoseconds per access. It is a hundred times slower than L1 and it is where everything ends up that does not fit in cache. It is also **volatile**: switch the power off and it is gone.

A shelf in another room — SSD. A **solid-state drive** stores data permanently, with no moving parts. About 100 **microseconds** for a read — a thousand times slower than RAM.

The store room with a heavy door — spinning disk. A traditional **hard disk drive** has a physical arm that must move to the right track and wait for the platter to rotate underneath it. About 10 **milliseconds** — a hundred times slower again than SSD, and a

hundred thousand times slower than RAM.

The tailor’s shop — the network. A round trip within a data centre is about 0.5 ms. Across a continent, 40 ms. Across the world, 150–250 ms. And, exactly like the tailor, it depends on somebody else and can simply be unavailable.

The numbers, and the trick for remembering them

Nobody remembers nanoseconds. Everybody remembers this: **scale it so one CPU cycle is one second.**

WHERE	REAL TIME	IF ONE CYCLE WERE ONE SECOND
Register	0.3 ns	1 second
L1 cache	1 ns	3 seconds
L2 cache	4 ns	13 seconds
L3 cache	15 ns	50 seconds
Main memory (RAM)	80 ns	4 minutes
SSD read	100 µs	4 days
Spinning disk seek	10 ms	11 months
Network, same data centre	0.5 ms	19 days
Network, across a continent	40 ms	4 years
Network, across the world	150 ms	16 years

Read the three bold rows. Memory is minutes, SSD is days, spinning disk is nearly a year. That is the answer to today’s question, and it is why “just read it from the database” and “it’s already in memory” are two completely different sentences.

The ratio to actually memorise: **RAM is about 1,000 × faster than SSD, and about 100,000 × faster than a spinning disk.**

The suitcase and the trip: why you fetch a block, not a value

Meenal brought down three garments in one trip because climbing up is the expensive part. Every level of this hierarchy does the same thing.

The processor never reads one byte from RAM. It reads a **cache line** of **64 bytes**. A disk never reads one byte; it reads a **block** or **page**, typically **4 KB** — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

The consequence is the single most useful design rule in this lesson: **once you have paid to get there, take everything nearby.** Reading 64 sequential bytes costs the same as reading 1. Reading a 4 KB page costs almost the same as reading 100 bytes from it.

That is why **sequential access is dramatically faster than random access** at every level, and why database people care so much about whether rows that are read together are stored together.

The saree on the door: caching

Meenal spent the five minutes on Wednesday so that Friday would be free. That is a **cache**: keep a copy of something expensive to fetch, somewhere cheap to reach.

Every layer of every system does this. The processor caches RAM. The operating system caches disk pages in RAM. The database caches its pages in a buffer pool. The application caches query results in **Redis**. The browser caches responses. A CDN caches your files near the user.

And every one of them has the same two problems, which come up in every interview about caching: what do you throw out when it is full (**eviction**), and what happens when the original changes and your copy does not (**invalidation**). Day 097 onwards is where these get their proper treatment; the point today is that caching is not a trick, it is **the** response to a hierarchy with gaps this large.

Volatile and durable

One more distinction, because it decides where data is allowed to live.

RAM is volatile. Power off, gone. **Disk and SSD are durable.** They survive a restart.

That is why a database writes to disk before telling you the write succeeded, and why Redis — which lives in memory — is a cache first and a database second. When someone asks “why not keep everything in RAM?”, the answer is two-thirds cost and one-third durability.

4 The picture

The ladder, with the gaps drawn to scale as best a page allows:

	SIZE	SPEED	HUMAN SCALE (1 cycle = 1 second)	
^	registers	~200 bytes	0.3 ns	1 second
	L1 cache	64 KB	1 ns	3 seconds
	L2 cache	1 MB	4 ns	13 seconds
	L3 cache	32 MB	15 ns	50 seconds
	----- volatile			
	RAM	8-512 GB	80 ns	4 minutes

	SSD	0.5-8 TB	100 us	4 days
	HDD	1-20 TB	10 ms	11 months

	same DC network	-	0.5 ms	19 days
	cross-continent	-	40 ms	4 years
v	cross-world	-	150 ms	16 years

smaller & faster & more expensive per byte at the top
bigger & slower & cheaper per byte at the bottom

What to notice: the two horizontal lines. Above the first one, everything vanishes when the power goes. Below the second, everything depends on another machine being alive. Those are not speed boundaries; they are boundaries of what can go wrong.

The cost per gigabyte, which is why the ladder exists at all:

RAM	~ \$3.00 per GB	1 TB of RAM	= \$3,000 (and needs a server that takes it)
SSD	~ \$0.08 per GB	1 TB of SSD	= \$80
HDD	~ \$0.02 per GB	1 TB of HDD	= \$20
S3	~ \$0.023 per GB/month	1 TB in S3	= \$23 per month

What to notice: RAM is roughly 40× the price of SSD per byte. That, not speed, is why everything is not simply held in memory.

And what caching does to an average, which is the arithmetic behind every cache decision:

90% of reads hit RAM (80 ns), 10% go to SSD (100,000 ns)
average = 0.90 x 80 + 0.10 x 100,000
= 72 + 10,000
= 10,072 ns
99% hit RAM: 0.99 x 80 + 0.01 x 100,000 = 1,079 ns
99.9% hit RAM: 0.999 x 80 + 0.001 x 100,000 = 180 ns

What to notice: going from 90% to 99% is a 9× improvement, and 99% to 99.9% is another 6×. **The misses dominate the average completely.** This is why “our cache hit rate is 90%” is a much worse position than it sounds, and it is a genuinely counter-intuitive result worth being able to produce on demand.

5 How it actually works

What happens on a memory read

The processor asks for an address. Then:

1. **L1** is checked. Hit, and you are done in about a nanosecond.
2. Miss → **L2**, then **L3**.
3. All missed — a **cache miss**. The memory controller fetches the whole 64-byte line containing that address from RAM, and it is placed in cache, evicting something else (usually the least recently used line).
4. If the address is not in RAM at all because the page was swapped out, that is a **page fault**: the OS reads a 4 KB page from disk, which is roughly a hundred thousand times slower, while the process waits.

This is why a program that fits in cache can be an order of magnitude faster than the same program with slightly more data, and why swap thrashing takes a machine from working to unusable rather than to slightly slower.

Why sequential access is so much faster

At every level, hardware assumes you will read forwards.

The **prefetcher** watches your access pattern, notices sequential reads, and fetches the next lines before you ask. **Disks** read a whole track at once. **SSDs** parallelise across internal channels when reads are contiguous.

The measured gap:

HDD sequential	: 150-250 MB/s	HDD random 4 KB	: ~1 MB/s	(150-250x)
SSD sequential	: 500-7,000 MB/s	SSD random 4 KB	: ~50 MB/s	(10-100x)

That single table explains an enormous amount of database design. It is why **Kafka** is so fast despite writing everything to disk — it only ever appends, sequentially. It is why **LSM-tree** stores such as **Cassandra** and **RocksDB** buffer writes in memory and flush them in large sorted runs. And it is why an index that lets you read 100 sequential rows beats one that makes you fetch 100 scattered ones.

Where real products sit on the ladder

PRODUCT	RUNG	CONSEQUENCE
Redis, Memcached	RAM	microsecond reads; loses data on restart unless you configure persistence
PostgreSQL, MySQL	SSD, with a RAM buffer pool	the buffer pool hit rate is the single most important number in its performance
Cassandra, RocksDB	SSD, write-optimised	sequential writes, background compaction
Kafka	disk, sequential only	appends at near-sequential disk speed, and relies on the OS page cache for reads
S3	disk, over the network	~50–100 ms first byte; enormous, cheap, durable
Glacier	offline	minutes to hours to retrieve; a tenth of S3's price

Postgres's buffer pool is worth pausing on because it is the story exactly. It keeps recently used 8 KB pages in RAM. If the working set fits, queries run at memory speed. If it does not, every query touches the SSD, and the same query becomes a thousand times slower with no change to the SQL. When someone says “the database got slow and we didn't change anything”, this is usually what happened: the data grew past the buffer pool.

Virtual memory, briefly

Every process sees its own flat address space; the OS maps those to real pages. When RAM runs out, pages are written to **swap** on disk. A program whose working set exceeds RAM starts paging, and because disk is $100,000\times$ slower, it does not degrade gracefully — it falls off a cliff. On a server the usual advice is to disable swap and let the process be killed instead, because a dead process restarts in seconds and a thrashing one takes the machine with it.

NUMA, for completeness

On large multi-socket servers, memory is attached to particular processors. Reading memory attached to another socket is roughly twice as slow. That is **NUMA** — non-uniform memory access — and it matters for databases pinned to cores. Worth knowing the word; rarely worth more than a sentence in an interview.

6 The numbers

The one to memorise, in the form interviewers ask for:

RAM read	:	80 ns	
SSD read	:	100,000 ns	= 1,250x slower
HDD seek + read	:	10,000,000 ns	= 125,000x slower

So: SSD is about a thousand times slower than RAM. A spinning disk is about a hundred thousand times slower. Those two ratios answer the question directly.

How much you can read in one second:

RAM	:	1 s / 80 ns	=	12,500,000 random reads	
SSD	:	1 s / 100 us	=	10,000 random reads	(per queue; NVMe parallelises to ~500,000 IOPS)
HDD	:	1 s / 10 ms	=	100 random reads	

One hundred random reads per second from a spinning disk. That number single-handedly explains why databases have indexes, why full table scans are feared, and why the industry moved to SSDs.

Sizing a cache, properly. A service holds 500 GB of data, and 20% of it is accessed 80% of the time:

$$\text{hot set} = 500 \text{ GB} \times 0.20 = 100 \text{ GB}$$

A machine with 128 GB of RAM holds the hot set, giving roughly an 80% hit rate before any tuning. Now the average read time:

$$0.80 \times 80 \text{ ns} + 0.20 \times 100,000 \text{ ns} = 64 + 20,000 = 20,064 \text{ ns} = 20 \text{ us}$$

Push the hit rate to 99%:

$$0.99 \times 80 + 0.01 \times 100,000 = 79 + 1,000 = 1,079 \text{ ns} = 1 \text{ us}$$

Twenty times faster from 19 percentage points of hit rate. That is the argument for spending money on RAM, and it is arithmetic rather than assertion.

Serving a page: where the time actually goes.

parse request, route	:	0.05 ms	
session lookup in Redis (RAM+net):	:	0.60 ms	
database query, buffer pool hit	:	1.00 ms	
database query, buffer pool miss	:	10.00 ms	
render template	:	2.00 ms	

all cached	:	3.65 ms	
one page miss	:	12.65 ms	(3.5x, from one miss)

Reading a 1 GB file:

```
sequential from SSD : 1 GB / 3 GB/s = 0.33 s
sequential from HDD : 1 GB / 200 MB/s = 5 s
random 4 KB from HDD : 262,144 reads x 10 ms = 2,621 s = 44 minutes
```

Half a second against forty-four minutes, for the same gigabyte. Access pattern beats hardware.

Cost of a cache miss, expressed as wasted work. At 3 GHz, a processor does 3 cycles per nanosecond:

```
one RAM access = 80 ns = 240 cycles of work the CPU could have done instead
one SSD read = 100 us = 300,000 cycles
one HDD seek = 10 ms = 30,000,000 cycles
```

Thirty million instructions' worth of doing nothing, per disk seek. That is why blocking I/O costs so much and why the asynchronous model from [day 007](#) exists.

7 The trade-offs

RAM is fast, expensive and forgets. About $40\times$ the cost per byte of SSD and roughly a thousand times the speed, and everything vanishes on restart. Which is why in-memory systems are chosen for data that can be rebuilt — caches, sessions, leaderboards, rate-limit counters — and why using Redis as the only copy of something is a decision that needs saying out loud rather than drifting into.

SSD is the modern default and it wears out. Fast, cheap enough, durable, no moving parts. The costs are real but rarely decisive: flash cells have limited write cycles, so write-heavy workloads shorten drive life, and writes get slower as the drive fills because there are fewer pre-erased blocks. Both are managed by the drive and both are why write-amplification is a real metric in database operations.

Spinning disk survives on price per byte and nothing else. Two cents a gigabyte, and a hundred random reads per second. It is genuinely correct for backups, archives, and large-file sequential workloads — video, logs, data lakes — and genuinely wrong for anything that reads randomly.

Caching buys speed and charges you correctness. Every cache introduces the possibility that your copy is out of date. The whole discipline of invalidation exists because of this, and the trade is always the same: how stale can this be, and what does it cost if it is? For a product catalogue, minutes are fine. For an account balance, nothing is fine, and the right answer is not to cache it.

Sequential versus random is often a bigger lever than the hardware. The 1 GB example above is half a second against forty-four minutes on the *same* drive. So before spending money moving down the ladder, it is worth asking whether the access pattern can be changed instead — batch the reads, sort by key, store together what is read together. That question is usually cheaper than the hardware and it is what a good design conversation gets to.

I would not put this in memory if... the dataset is much larger than RAM and access is genuinely uniform, so no hot set exists and a cache would thrash. Or if losing it is unacceptable and I am not willing to pay for replication and persistence. Or if it is written far more often than it is read, in which case a cache adds invalidation work for very little benefit — a write-heavy counter is better handled by a store designed for writes than by a cache in front of one designed for reads.

8 In the interview

How it gets asked

- “*How much slower is a disk read than a memory read?*” — the direct version. Give ratios and at least one absolute number.
- “*Why is Redis fast?*” — the applied version. The answer is “it’s on the memory rung”, not “it’s written well”.
- “*Estimate how long this operation takes.*” — back-of-the-envelope, and these numbers are the raw material.
- “*Our database got slow and we didn’t change anything. What happened?*” — usually the working set outgrew the buffer pool.

What to say out loud, in the first ninety seconds

1. **Give the ladder in order.** “*Registers, L1, L2, L3, RAM, SSD, spinning disk, network.*”
2. **Give the two ratios that matter.** “*RAM is about 80 nanoseconds. SSD is about 100 microseconds, so roughly a thousand times slower. A spinning disk seek is about 10 milliseconds — a hundred thousand times slower than RAM.*”
3. **Make it human.** “*If a CPU cycle were one second, RAM would be four minutes, an SSD read four days, and a disk seek about eleven months.*”
4. **Name the block effect.** “*And nothing reads one byte. The CPU fetches 64-byte cache lines, disks read 4 KB pages. So sequential access is enormously cheaper than random — the same gigabyte is half a second sequentially off an SSD and could be forty minutes if read randomly off a spinning disk.*”

5. **Say what follows.** *“Which is why every layer caches: keep the hot data one rung up. And why the hit rate matters so much — at 90% hit rate the misses still dominate the average completely.”*
6. **Give the boundary.** *“The two lines that matter are volatility — everything above RAM is lost on restart — and the network, where you’re depending on another machine being alive.”*

The follow-ups

“So why not keep everything in RAM?” Cost and durability. RAM is roughly forty times the price per byte of SSD, so a 20 TB dataset is a few hundred dollars on disk and a six-figure hardware problem in memory. And RAM is volatile — a restart loses everything, so anything that must survive needs writing down anyway. That said, “keep the hot subset in RAM” is almost always right, and the useful question is what fraction is hot. If 20% of the data serves 80% of the reads, buying enough RAM for that 20% gets most of the benefit for a fifth of the price.

“Why is a database index worth having, in these terms?” Because it converts a large number of reads into a small number. A full scan of a million-row table on a spinning disk that does a hundred random reads per second is not a query, it is a coffee break. An index turns that into a handful of page reads by narrowing the search before touching the data. In hierarchy terms, the index is small enough to stay cached in RAM while the table is not — so you pay memory-speed lookups to avoid disk-speed scans. That is also why an index that does not fit in memory is a much weaker index.

“Our reads got slow and nothing changed. What would you check?” First, whether the working set outgrew the buffer pool or the cache. That transition is not gradual: while the hot data fits in RAM, reads are microseconds; the moment it does not, the same query hits the SSD and is a thousand times slower with no code change. I’d look at cache hit rate over time, not at the current value. Second, whether the access pattern became more random — a new query, or data that used to be clustered and is now scattered after many updates. Third, whether the machine started swapping, which is the same cliff one level down.

“How would you estimate the time for a request that does three database queries?” I’d break it into hierarchy steps and add them. Say each query is a buffer-pool hit at about 1 ms, plus a network round trip within the data centre at 0.5 ms each way — so about 2 ms per query, 6 ms for three. Plus maybe 2 ms of application work. So under 10 ms if everything is cached. Then I’d ask what happens at the tail: if one query misses the buffer pool, that is 10 ms of SSD instead of 1, so the p99 is several times the median. The useful output of this exercise is usually not the average — it is noticing where the variance comes from.

A model answer

“The hierarchy, top to bottom, is registers, then L1, L2 and L3 cache, then main memory, then SSD, then spinning disk, then the network.

The absolute numbers I carry are: L1 about 1 nanosecond, RAM about 80 nanoseconds, an SSD read about 100 microseconds, and a spinning-disk seek about 10 milliseconds. So SSD is roughly a thousand times slower than RAM, and a spinning disk is roughly a hundred thousand times slower.

Those are hard to feel, so I scale them: if one CPU cycle were one second, an L1 hit is three seconds, RAM is four minutes, an SSD read is four days, and a disk seek is about eleven months. A round trip to a data centre on another continent would be four years.

Two things follow. First, nothing reads one byte — the CPU fetches 64-byte cache lines and disks read 4 KB pages — so sequential access is vastly cheaper than random. Reading a gigabyte sequentially off an SSD is about a third of a second; reading the same gigabyte as scattered 4 KB reads off a spinning disk is over forty minutes. That’s the same hardware and a different access pattern, and it’s often a bigger lever than buying faster storage.

Second, every layer caches, because the gaps are too big not to. The CPU caches RAM, the OS caches disk pages, the database has a buffer pool, the application has Redis, the CDN caches at the edge. And the arithmetic on hit rates is unforgiving: at 90% hits with an 80 nanosecond RAM read and a 100 microsecond SSD read, the average is about 10 microseconds — the misses are 99% of the time spent. Getting to 99% takes that to 1 microsecond. So ‘we have a cache’ is a much weaker statement than ‘our hit rate is 99%’.

The last thing I’d flag is that two of these boundaries aren’t about speed. Everything above RAM disappears on restart, which decides what can live in Redis. And the network isn’t just slow, it’s a dependency on another machine — it can fail in ways local storage cannot.”

That answer gives the order, the absolute numbers, the ratios, a memorable scale, the sequential-versus-random consequence, the cache-hit arithmetic, and the two non-speed boundaries.

9 Recall card

1. **The ladder:** register $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ RAM $\rightarrow$ SSD $\rightarrow$ HDD $\rightarrow$ network. Each step is roughly $100\text{--}1,000\times$ slower and cheaper per byte.
2. **The three numbers:** RAM **80 ns**, SSD **100 μ s**, disk seek **10 ms**. So SSD is $\sim 1,000\times$ slower than RAM, and a spinning disk $\sim 100,000\times$ slower.
3. **If one cycle were one second:** L1 = 3 s, RAM = 4 minutes, SSD = 4 days, disk seek = 11 months, cross-world network = 16 years.
4. **Nothing reads one byte.** 64-byte cache lines, 4 KB disk pages. Sequential access can be $100\times$ faster than random on the same hardware.
5. **Misses dominate the average.** 90% hit rate on RAM-vs-SSD still averages 10 μ s; 99% averages 1 μ s. And everything above RAM is volatile.

PRACTICE

Practice — What an array really is in memory

DSA topic: What an array really is in memory **System design topic:** CPU, RAM, and disk: the speed hierarchy

Code these, in this order

Four problems that all rest on one fact: you may reach any position of an array directly, and it costs the same wherever you reach. Each one uses that differently.

For each problem:

1. Say which positions you need to reach, and in what order.
2. State whether you are walking forwards, backwards, or jumping — and whether the order could be made sequential.
3. Solve it, then state the time and extra space.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Build Array from Permutation	LeetCode 1920 (Easy)	The purest possible use of $O(1)$ indexing: <code>nums[nums[i]]</code> is two direct reaches, not a search. If indexing were $O(n)$ this problem would not exist.
2	Concatenation of Array	LeetCode 1929 (Easy)	Position arithmetic. <code>ans[i]</code> and <code>ans[i + n]</code> are both computed, both direct, both the same cost.
3	Richest Customer Wealth	LeetCode 1672 (Easy)	A 2D list is a list of references to lists. Row-by-row is the natural order and also the cache-friendly one — notice that they agree.
4	Find Pivot Index	LeetCode 724 (Easy)	Two passes over the same array, both sequential. The naive version recomputes a sum inside the loop and is $O(n^2)$; spotting that is the exercise.

On problem 3, do this properly

After solving it, write the column-major version — the one that loops over columns on the outside and rows on the inside — producing the same answer. Time both on a grid of about $2,000 \times 2,000$.

They are both $O(\text{rows} \times \text{cols})$ and they will not take the same time. Say out loud why, using the words “cache line”.

The address drill

Answer these from memory, with the arithmetic shown, in under a minute:

- An array of 4-byte integers starts at address 2000. Where is `items[12]` ?

- An array of 8-byte values starts at address 5000. Where is `items[1000]` ?
- Why does the answer to either take the same time as finding `items[0]` ?
- What breaks if one element in the middle is 6 bytes instead of 4?
- A Python list of 1,000,000 integers — how much memory, roughly, and where does it go?

The measurement drill

Run the complete program from §5 of the lesson. Then change one thing: make the shuffled test use a list of 100,000 elements instead of 4,000,000, and run it again.

The ratio between ordered and shuffled should shrink dramatically. Explain why in one sentence. (The answer involves the size of L3 cache from the system design lesson.)

The numbers to have cold

Say these six out loud without looking:

- RAM read, in nanoseconds.
 - SSD read, in microseconds.
 - Spinning-disk seek, in milliseconds.
 - How many times slower SSD is than RAM.
 - How many times slower a disk seek is than RAM.
 - A cache line, in bytes.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why is array indexing $O(1)$?* Give the formula with real addresses. Then name the requirement that makes it work. Then derive why front insertion is $O(n)$ from the same fact.
 2. *How much slower is a disk read than a memory read?* Give the absolute numbers, then the ratios, then the human scale. Then say what follows from it about caching.
 3. *Two loops read the same million elements and one is five times slower. Why?* One sentence on cache lines, one on the RAM-versus-cache gap, one on why Big-O is still right.
-

Before you move on

- [] I can write `address = base + i × element_size` from memory and put real numbers in it.
- [] I can say the three requirements that make the formula work.
- [] I know that a Python list stores references, and roughly what a million integers costs.
- [] I can order register, L1, RAM, SSD, HDD and network by speed, with numbers.
- [] I can explain why a 90% cache hit rate is a worse position than it sounds.

Traversal: the loop patterns you will reuse forever

DATA STRUCTURES AND ALGORITHMS

Traversal: the loop patterns you will reuse forever

You can write forward, backward, paired and windowed loops without fighting the indices.

SYSTEM DESIGN

Latency numbers every engineer should know

You can quote the handful of numbers that make back-of-the-envelope estimation possible.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Iterate over every adjacent pair in the array.*
- *Roughly how long is a network round trip to a data centre on another continent?*

Traversal: the loop patterns you will reuse forever

1 What this is, and why they ask it

There are about seven ways to walk along an array, and you will use every one of them for the next 170 days. Forwards. Backwards. Every position with its own number attached. Every adjacent pair. Every group of `k` in a row. Two positions moving towards each other from the ends. Every position with a step of more than one.

None of them is difficult. All of them have an off-by-one waiting in the loop bounds, and that off-by-one is what actually costs people interviews.

Interviewers ask “iterate over every adjacent pair” because it is the smallest question that exposes the problem. The obvious loop reads `for i in range(len(items))` and then touches `items[i + 1]`, which runs off the end. Getting the bound right first time, and being able to say *why* it is `len(items) - 1`, tells the interviewer that you think about index ranges deliberately rather than adjusting them until the tests pass.

2 The story

It is four in the afternoon and Latha is on the terrace, taking in the washing before the evening damp comes.

There are two lines running the width of the terrace, and she starts at the left-hand end of the near one and works along it. Shirt, shirt, her son's trousers, two pillow covers, and so on to the far end. She does not skip about. She takes each thing as she comes to it, and when she reaches the wall she is done with that line.

The second line still has the morning's load to go up, and this she does the other way round, starting at the far end and working back towards the stairs. It is not a preference. The tap and the bucket are by the stairs, so if she started at the stairs she would spend the whole time walking back past wet washing she had already hung.

The socks are their own job. There are eleven pairs on the second line and they are not in order, so she goes along looking at each sock and the one next to it. Same colour, same length — fold them together. Not a match — leave the first one and shift her attention along by one, so that the sock she just rejected is now the left-hand one of the next comparison. She works through the whole line that way and ends with two odd socks, which is normal.

Then there is the thing she learnt about this terrace over two monsoons. If three heavy items hang next to each other — jeans, a towel, a bedsheet — the middle one does not dry, because nothing gets past on either side. So before she leaves she walks the line once more and looks at every run of three in a row: items one, two and three; then two, three and four; then three, four and five. Each time she is looking at a group of three, and each time the group slides along by one. If a group is all heavy, she moves the middle one to the end of the line.

Her daughter comes up to help with the big bedsheet, which needs two people. Latha takes one end and the girl takes the other, and they walk towards each other folding as they go until their hands meet in the middle. That takes one crossing, not two.

The only thing that catches her out, every single time, is the pegs. There are fourteen pegs on the near line and she always thinks there are fourteen gaps between them. There are thirteen. She has been hanging washing on this terrace for nineteen years and she still has to stop and count on her fingers.

3 The idea in plain English

Latha used five different ways of moving along a line, and every one of them is a loop you will write dozens of times.

1. Forward, by position

The plain walk. Left to right, one at a time.

```
for i in range(len(items)):
    print(items[i])
```

`range(len(items))` produces `0, 1, ..., n-1`. Exactly `n` values, and the last one is `n - 1`, never `n`.

2. Forward, by value

When you do not need the position, ask for the values directly. This is the Python way and it cannot go out of range.

```
for x in items:
    print(x)
```

3. Forward, with both

`enumerate` gives you the position and the value together, which is what you want most of the time.

```
for i, x in enumerate(items):
    print(i, x)
```

`enumerate(items, start=1)` numbers from 1 if you need it. **Prefer this to `range(len(items))` whenever you need both** — it is shorter, and it removes one place where an index can be wrong.

4. Backward

Latha's second line. Three ways, all correct:

```
for i in range(len(items) - 1, -1, -1):    # by position: start, stop-before, step
    print(items[i])

for x in reversed(items):                 # by value, no copy made
    print(x)

for i, x in reversed(list(enumerate(items))): # both, at the cost of a copy
    print(i, x)
```

Read the first one carefully, because it is the one people get wrong. Start at `n - 1`, stop **before** `-1` (so `0` is included), step by `-1`. Writing `range(len(items) - 1, 0, -1)` silently skips element 0, and nothing will tell you.

`reversed(items)` does not build a reversed copy; it walks the original from the back. So it is $O(1)$ extra space, unlike `items[::-1]`, which is $O(n)$.

When you actually need backwards: when you are removing or overwriting elements and want to avoid disturbing positions you have not visited yet, and when the answer depends on what comes *after* each element — suffix sums, next-greater-element, and most of [day 071](#).

5. Adjacent pairs — and the off-by-one

This is today's interview question, and it is Latha's socks.

```
for i in range(len(items) - 1):
    left, right = items[i], items[i + 1]
```

The bound is `len(items) - 1`, and here is why. The largest valid value of `i + 1` is `n - 1`. So the largest valid `i` is `n - 2`. And `range(n - 1)` stops at `n - 2`. That is exactly right.

It is the pegs and the gaps. `n` items have `n - 1` gaps between them. Fourteen pegs, thirteen gaps. Say that sentence to yourself every time you write a pairs loop.

There is also a version with no arithmetic at all, which is worth knowing because it cannot be off by one:

```
for left, right in zip(items, items[1:]):
    ...
```

`zip` stops when the shorter sequence runs out, so the bound takes care of itself. The cost is that `items[1:]` is a copy — $O(n)$ extra space. For an interview, either is fine; say which trade you are making.

6. A window of k in a row

Latha's three heavy items. A group of k consecutive elements, sliding along by one.

```
k = 3
for start in range(len(items) - k + 1):
    window = items[start:start + k]
```

The bound is `len(items) - k + 1`. Check it with numbers rather than trusting it: with $n = 5$ and $k = 3$, that is $5 - 3 + 1 = 3$ windows, starting at positions 0, 1 and 2, which cover elements 0-1-2, 1-2-3 and 2-3-4. Three windows. Correct.

And notice that adjacent pairs is this with $k = 2$: $n - 2 + 1 = n - 1$. The two formulas are the same formula.

Taking the slice each time costs $O(k)$, which makes the whole loop $O(n \times k)$. The way to keep it $O(n)$ is to update a running total as the window moves — add the entering element, subtract the leaving one. That is the sliding window technique, and it gets its own days from [day 031](#).

7. Two positions moving towards each other

Latha and her daughter and the bedsheet. One index at each end, both moving inward.

```
left, right = 0, len(items) - 1
while left < right:
    ...
    left += 1
    right -= 1
```

`while left < right` stops when they meet. Use `left ≤ right` instead when the middle element also needs handling. One crossing, $n / 2$ steps, $O(n)$ time and $O(1)$ space — which is why [day 007](#)'s in-place reversal used it. This is the two-pointer pattern, and it owns [day 027](#) onwards.

8. Stepping by more than one

```
for i in range(0, len(items), 2):      # 0, 2, 4, ... every second element
for i in range(1, len(items), 2):      # 1, 3, 5, ... the odd positions
```

`range(start, stop, step)`. The third argument is the stride. Still $O(n / \text{step})$ iterations, which is still $O(n)$.

The one rule that prevents every off-by-one

Before writing a loop bound, ask: **what is the largest index I will actually touch inside the body?** Then make the range stop one past it.

- Body touches `items[i]` → largest index `n - 1` → `range(n)`.
- Body touches `items[i + 1]` → largest index `n - 1`, so largest `i` is `n - 2` → `range(n - 1)`.
- Body touches `items[i + k - 1]` → largest `i` is `n - k` → `range(n - k + 1)`.

Three lines. Every array off-by-one you will ever write is one of these.

4 The picture

All seven patterns on the same seven-element array, showing which positions get visited and in what order:

index	0	1	2	3	4	5	6	
value	2	3	5	8	13	21	34	
-----+								
1. forward	1 → 2 → 3 → 4 → 5 → 6 → 7						range(7)	
	0	1	2	3	4	5	6	
2. backward	7 ← 6 ← 5 ← 4 ← 3 ← 2 ← 1						range(6, -1, -1)	
	0	1	2	3	4	5	6	
3. adjacent pairs	[0,1] [1,2] [2,3] [3,4] [4,5] [5,6]						range(6)	
	^						6 pairs, not 7	
	7 items, 6 gaps between them							
4. window of k=3	[0,1,2]							
	[1,2,3]							
	[2,3,4]							
	[3,4,5]							
	[4,5,6]						range(7-3+1) = 5	
5. two pointers	L----->R							
	0					6		
	L----->R							
		1			5			
	L→R							
		2		4				
	meet at 3						while left < right	
6. step of 2	*	.	*	.	*	.	*	range(0, 7, 2)
	0		2		4		6	
7. nested (pairs)	i=0: j = 1,2,3,4,5,6							
	i=1: j = 2,3,4,5,6							
	i=2: j = 3,4,5,6 ...						the staircase, $O(n^2)$	

What to notice in row 3: six brackets under seven boxes. The pegs and the gaps. Every time your loop body reaches for `items[i + 1]`, the number of iterations drops by exactly one.

What to notice in row 4: five windows over seven elements with `k = 3`, and the last window starts at index 4, which is `n - k`. Count them on the picture rather than trusting the formula, and the formula becomes obvious instead of memorised.

And here is the failure, drawn:

```
for i in range(7):           ← WRONG bound for a pairs loop
    compare items[i], items[i + 1]

i = 5:  items[5], items[6]    fine
i = 6:  items[6], items[7]    ← there is no items[7]
                        ^^^^
                        IndexError
```

What to notice: the loop is correct for six of its seven iterations. That is what makes off-by-one errors so easy to write and so hard to spot by reading.

5 The code, built step by step

Write each pattern once, and make each one report the positions it visited, so that the bounds are visible rather than assumed.

Start with the plain walk and the value walk.

```
def forward_positions(items: list[int]) → list[int]:
    visited = []
    for i in range(len(items)):
        visited.append(i)
    return visited
```

`range(len(items))` gives `0` to `n - 1`. Seven elements, seven visits.

Now backwards, with the bound that people get wrong.

```
def backward_positions(items: list[int]) → list[int]:
    visited = []
    for i in range(len(items) - 1, -1, -1):
        visited.append(i)
    return visited
```

Stop-before `-1` so that index `0` is included. Writing `-1` as the second argument looks odd the first ten times and is exactly right.

Now the pairs loop, which is today's question.

```
def adjacent_pairs(items: list[int]) → list[tuple[int, int]]:
    pairs = []
    for i in range(len(items) - 1):
        pairs.append((items[i], items[i + 1]))
    return pairs
```

`len(items) - 1` because the body reaches for `i + 1`. Seven elements give six pairs.

Now the same thing with no arithmetic, for comparison.

```
def adjacent_pairs_zip(items: list[int]) → list[tuple[int, int]]:
    return list(zip(items, items[1:]))
```

`zip` stops at the shorter of the two, so the bound is automatic. `items[1:]` is a copy, so this is $O(n)$ extra space where the loop version is $O(1)$.

Now the sliding window.

```
def windows_of(items: list[int], k: int) → list[list[int]]:
    out = []
    for start in range(len(items) - k + 1):
        out.append(items[start:start + k])
    return out
```

If `k > len(items)` then `len(items) - k + 1` is zero or negative, `range` produces nothing, and the function correctly returns an empty list. That is worth checking rather than assuming — it is the kind of edge case [day 008](#) told you to write down.

And the window done properly, in $O(n)$ rather than $O(n \times k)$.

```
def window_sums(items: list[int], k: int) → list[int]:
    if k > len(items):
        return []
    total = sum(items[:k]) # the first window, O(k)
    sums = [total]
    for i in range(k, len(items)):
        total += items[i] - items[i - k] # add the new, drop the old
        sums.append(total)
    return sums
```

One line does the work: `items[i]` is entering the window and `items[i - k]` is leaving it. That is the whole sliding-window idea, and everything from day 031 onwards is a variation of it.

Now two pointers.

```
def two_pointer_steps(items: list[int]) → list[tuple[int, int]]:
    steps = []
    left, right = 0, len(items) - 1
    while left < right:
        steps.append((left, right))
        left += 1
        right -= 1
    return steps
```

Here is the complete program.


```

"""Day 10 – the seven ways to walk an array, with the bounds made visible."""

def forward_positions(items: list[int]) → list[int]:
    """range(n): 0 .. n-1"""
    return [i for i in range(len(items))]

def backward_positions(items: list[int]) → list[int]:
    """range(n-1, -1, -1): n-1 .. 0. Stop-before is -1, so 0 is included."""
    return [i for i in range(len(items) - 1, -1, -1)]

def with_index(items: list[int]) → list[tuple[int, int]]:
    """enumerate: position and value together, no manual indexing."""
    return [(i, x) for i, x in enumerate(items)]

def adjacent_pairs(items: list[int]) → list[tuple[int, int]]:
    """range(n-1), because the body reaches for i+1. n items, n-1 gaps."""
    return [(items[i], items[i + 1]) for i in range(len(items) - 1)]

def windows_of(items: list[int], k: int) → list[list[int]]:
    """range(n-k+1). 0(n*k) because each slice costs O(k)."""
    return [items[s:s + k] for s in range(len(items) - k + 1)]

def window_sums(items: list[int], k: int) → list[int]:
    """The same windows in O(n): add the entering element, subtract the leaving one."""
    if k ≤ 0 or k > len(items):
        return []
    total = sum(items[:k])
    sums = [total]
    for i in range(k, len(items)):
        total += items[i] - items[i - k]
        sums.append(total)
    return sums

def two_pointer_steps(items: list[int]) → list[tuple[int, int]]:
    """From both ends, inward. n/2 iterations, O(1) space."""
    steps = []
    left, right = 0, len(items) - 1
    while left < right:
        steps.append((left, right))
        left += 1
        right -= 1
    return steps

def every_second(items: list[int]) → list[int]:
    """range(0, n, 2): stride of 2."""
    return [items[i] for i in range(0, len(items), 2)]

if __name__ == "__main__":
    data = [2, 3, 5, 8, 13, 21, 34]
    n = len(data)
    print(f"array: {data}    (n = {n})\n")

    print(f"1. forward positions      : {forward_positions(data)}    → {n} visits")
    print(f"2. backward positions      : {backward_positions(data)}    → {n} visits")
    print(f"3. enumerate                : {with_index(data)[:3]}    ...")
    pairs = adjacent_pairs(data)
    print(f"4. adjacent pairs          : {pairs}")
    print(f"                           → {len(pairs)} pairs from {n} items. n-1, not n.")
    wins = windows_of(data, 3)
    print(f"5. windows of 3           : {wins}")

```

```

print(f"                                → {len(wins)} windows = n - k + 1 = {n} - 3 + 1")
print(f"6. window sums, 0(n)         : {window_sums(data, 3)}")
print(f"                                → matches {sum(w) for w in wins}")
print(f"7. two pointers                  : {two_pointer_steps(data)}")
print(f"8. every second                   : {every_second(data)}")

print("\nthe edge cases that break careless bounds")
for items, k in (([], 3), ([5], 3), ([1, 2], 3), ([1, 2, 3], 3)):
    print(f"  windows_of({str(items)}:<10}, k=3) → {windows_of(items, k)}")
print(f"  adjacent_pairs([])           → {adjacent_pairs([])}")
print(f"  adjacent_pairs([5])          → {adjacent_pairs([5])}")
print(f"  two_pointer_steps([5])       → {two_pointer_steps([5])}")

```

This is exactly what it printed:

```

array: [2, 3, 5, 8, 13, 21, 34]    (n = 7)

1. forward positions      : [0, 1, 2, 3, 4, 5, 6]    → 7 visits
2. backward positions    : [6, 5, 4, 3, 2, 1, 0]    → 7 visits
3. enumerate             : [(0, 2), (1, 3), (2, 5)] ...
4. adjacent pairs        : [(2, 3), (3, 5), (5, 8), (8, 13), (13, 21), (21, 34)]
                        → 6 pairs from 7 items. n-1, not n.
5. windows of 3          : [[2, 3, 5], [3, 5, 8], [5, 8, 13], [8, 13, 21], [13, 21, 34]]
                        → 5 windows = n - k + 1 = 7 - 3 + 1
6. window sums, 0(n)    : [10, 16, 26, 42, 68]
                        → matches [10, 16, 26, 42, 68]
7. two pointers          : [(0, 6), (1, 5), (2, 4)]
8. every second          : [2, 5, 13, 34]

the edge cases that break careless bounds
windows_of([], k=3) → []
windows_of([5], k=3) → []
windows_of([1, 2], k=3) → []
windows_of([1, 2, 3], k=3) → [[1, 2, 3]]
adjacent_pairs([]) → []
adjacent_pairs([5]) → []
two_pointer_steps([5]) → []

```

Look at line 4 and line 5. Six pairs from seven items; five windows of three from seven items. Neither number is seven, and both formulas are the same formula with different `k`.

Look at the edge cases block. Every one returns an empty list rather than raising, and none of them needed a special case — the arithmetic in the range bound handles them. That is what a correct bound looks like: it degrades to “no iterations” instead of to an exception.

6 What it costs

Every pattern here is one pass, so every one is `O(n)` time. The differences are in the constant and in the space.

PATTERN	ITERATIONS	TIME	EXTRA SPACE
Forward, <code>range(n)</code>	n	$O(n)$	$O(1)$
Forward, <code>for x in items</code>	n	$O(n)$	$O(1)$
<code>enumerate(items)</code>	n	$O(n)$	$O(1)$
Backward, <code>reversed(items)</code>	n	$O(n)$	$O(1)$
Backward, <code>items[::-1]</code>	n	$O(n)$	$O(n)$ — it copies
Adjacent pairs, index	$n - 1$	$O(n)$	$O(1)$
Adjacent pairs, <code>zip(items, items[1:])</code>	$n - 1$	$O(n)$	$O(n)$ — the slice copies
Window of k , slicing	$n - k + 1$	$O(n \times k)$	$O(k)$
Window of k , running total	$n - k + 1$	$O(n)$	$O(1)$
Two pointers	$n / 2$	$O(n)$	$O(1)$
Step of s	n / s	$O(n)$	$O(1)$
Nested pairs	$n(n - 1)/2$	$O(n^2)$	$O(1)$

The two rows worth staring at are the window rows. Same output, and one is $O(n \times k)$ because it takes a fresh slice each time. At $n = 100,000$ and $k = 1,000$:

```
slicing      : 100,000 x 1,000 = 100,000,000 element copies → about 10 s in Python
running total : 100,000 x 2     =      200,000 operations  → about 0.02 s
```

Five hundred times faster, for the same answer. The k disappears because the running total does two operations per step regardless of window size.

Constant factors between the forward patterns. All three are $O(n)$ and they are not equally fast, because `range(len(items))` plus `items[i]` does a bound check and an index computation per element, while `for x in items` does neither:

```
n = 5,000,000
for x in items           : 0.99 s
for i, x in enumerate(items): 1.79 s
for i in range(len(items)) : 1.77 s    (with items[i] in the body)
```

Nearly twice, purely from how the loop is written. Note that `enumerate` and `range(len(...))` come out about level here — the saving is in not indexing at all. It never changes the Big-O and it is free to get right, so the rule is: **iterate by value unless you need the index, and use `enumerate` when you need both.**

Why backwards costs nothing extra. `reversed(items)` walks from the last address to the first without building anything. `items[::-1]` allocates a whole second list. Same $O(n)$ time, and $O(1)$ against $O(n)$ space:

```
n = 1,000,000
reversed(items) :      0 KB extra
items[::-1]      :  7,812 KB extra
```

The bounds, as arithmetic. Worth being able to derive rather than recall:

body touches items[i]	→ max index n-1 → range(n)	→ n iterations
body touches items[i+1]	→ max i is n-2 → range(n-1)	→ n-1 iterations
body touches items[i+k-1]	→ max i is n-k → range(n-k+1)	→ n-k+1 iterations
two pointers, left < right	→	→ n/2 iterations

7 The traps

Trap one: the pairs loop with the wrong bound

The one the interview question is designed to catch.

```
def is_sorted(items: list[int]) → bool:
    for i in range(len(items)):
        if items[i] > items[i + 1]:
            return False
    return True

print(is_sorted([1, 2, 3, 4, 5]))
```

```
Traceback (most recent call last):
  File "d10.py", line 7, in <module>
    print(is_sorted([1, 2, 3, 4, 5]))
    ~~~~~^
  File "d10.py", line 3, in is_sorted
    if items[i] > items[i + 1]:
    ~~~~~^
IndexError: list index out of range
```

Read where the `~~~~~^` marks point: at `items[i]`, which is the first subscript on that line. On the final iteration `i` is 4 and `i + 1` is 5, and the list ends at 4.

The nastier version of this bug is the one that does **not** raise:

```
def is_sorted(items: list[int]) → bool:
    for i in range(len(items)):
        if items[i] > items[i - 1]:
            return False
    return True

print(is_sorted([5, 1, 2, 3]))    # prints True. It is not sorted.
```

No error at all, and a wrong answer. On the first iteration `i` is 0, so `items[i - 1]` is `items[-1]` — which in Python is the **last** element, not an error. The comparison silently wraps around the array. This is the single most dangerous thing about Python's negative indexing, and it is why an off-by-one in Python can be quieter than the same bug in C.

How to catch it every time: before writing the range, name the largest index the body will touch. `items[i + 1]` means `i` stops at `n - 2`, so the bound is `n - 1`. And when a loop could produce `i - 1` on the first pass, start the loop at 1 instead of guarding inside it.

Trap two: changing the list you are walking

This produces no error and skips elements, silently.

```
items = [1, 2, 2, 3, 2, 4]
for x in items:
    if x == 2:
        items.remove(x)
print(items)
```

```
[1, 3, 2, 4]
```

One `2` survived. The reason is that the loop keeps an internal position that moves forward while the list shrinks underneath it. Removing element 1 shifts everything left, so the next step to position 2 skips what is now at position 1.

Walked through: position 0 is `1`, keep. Position 1 is `2`, removed — the list becomes `[1, 2, 3, 2, 4]` and the loop advances to position 2, which is now `3`, so the `2` that slid into position 1 is never examined.

Two correct fixes:

```
items = [x for x in items if x != 2]           # build a new list – clear,  $O(n)$  space
```

```
for i in range(len(items) - 1, -1, -1):       # walk backwards – in place,  $O(1)$  space
    if items[i] == 2:
        items.pop(i)
```

The backward version works because removing at position `i` only shifts elements *after* `i`, and those are the ones already visited. **This is the main practical reason to walk backwards**, and it is worth remembering as a rule: *if you are deleting while iterating, iterate backwards*.

The genuinely $O(n)$ -time, $O(1)$ -space answer is the write pointer from [day 007](#), because `pop(i)` is itself $O(n)$, making the backward loop $O(n^2)$.

8 In the interview

How it gets asked

- “Iterate over every adjacent pair in the array.” — the direct version. The bound is the whole question.
- “Check whether the array is sorted.” — the same question with a reason attached.
- “Find the maximum sum of any k consecutive elements.” — the window version, and the good answer is $O(n)$, not $O(n \times k)$.
- “Why did you loop backwards there?” — asked when you do it. Have the reason ready: deletion, or dependence on later elements.

What to say out loud, in the first ninety seconds

1. **State the bound and its reason, before writing it.** “I’ll loop `i` from 0 to n minus 2, because the body reaches for `i + 1` and the largest valid index is n minus 1.”
2. **Give the count.** “So n items give n minus 1 pairs. Seven elements, six pairs.”
3. **Handle the small cases out loud.** “With zero or one element there are no pairs, and `range(n - 1)` gives an empty range for both, so no special case is needed.”
4. **Mention the alternative and its trade.** “`zip(items, items[1:])` does the same thing with no index arithmetic, but the slice is an $O(n)$ copy. The index version is $O(1)$ space.”
5. **State the complexity.** “ $O(n)$ time, $O(1)$ extra space.”
6. **If it is a window, offer the improvement unprompted.** “For a window of k , slicing each time would be $O(n \times k)$. I’d keep a running total instead — add the entering element, subtract the leaving one — which is $O(n)$ regardless of k .”

Step 3 is the one candidates skip and interviewers notice. Saying that the bound handles the empty case *without* a special case is a small demonstration that you derived it rather than recalled it.

The follow-ups

“Why `len(items) - 1` and not `len(items)`?” Because the body touches `items[i + 1]`, so the largest `i` I can allow is one less than the largest valid index. The largest valid index is `n - 1`, so `i` must stop at `n - 2`, and `range(n - 1)` stops at `n - 2`. The way I remember it is pegs and gaps: `n` items have `n - 1` gaps between them. And it is the general formula — for a window of `k` the bound is `n - k + 1`, and pairs are just `k = 2`.

“What happens with an empty list?” `range(-1)` produces no values, so the loop body never runs and the function returns whatever it should for “no pairs” — an empty list, or `True` for is-sorted. That is worth checking rather than assuming, and it is the

reason I prefer to derive the bound rather than write $n - 1$ from memory: a derived bound tends to degrade correctly at the edges.

“When would you iterate backwards?” Three cases. When I’m deleting or shifting elements and don’t want to disturb positions I have not visited — removing forwards skips elements silently, removing backwards does not. When the answer for each element depends on what comes after it: suffix sums, next-greater-element, and most monotonic-stack problems. And when I’m filling an array in place from the end, as in merging two sorted arrays into the first one, where writing forwards would overwrite data I still need.

“Your window solution slices each time. What does that cost?” $O(k)$ per window and $O(n \times k)$ overall, which at n of a hundred thousand and k of a thousand is 10^8 element copies — several seconds in Python. The fix is to maintain a running total instead of rebuilding the window: when the window moves right by one, add the element entering and subtract the one leaving. That is two operations per step regardless of k , so it is $O(n)$ and $O(1)$ space. It only works for quantities you can undo — sums and counts yes, maximum no, which is why sliding-window maximum needs a deque instead.

A model answer

“Sure. The key decision is the loop bound, so let me say it before I write it: the body reaches for `items[i + 1]`, and the largest valid index is `n - 1`, so `i` has to stop at `n - 2`. That means `range(len(items) - 1)`.

```
for i in range(len(items) - 1):
    left, right = items[i], items[i + 1]
```

Seven elements give six pairs — `n` items have `n - 1` gaps between them, like pegs on a line. And the bound handles the small cases without a guard: for an empty list or a single element, `range(-1)` and `range(0)` both produce nothing, so the body never runs and there are correctly no pairs.

That’s $O(n)$ time and $O(1)$ extra space.

There’s an alternative with no arithmetic at all — `zip(items, items[1:])` — which is harder to get wrong because `zip` stops at the shorter sequence. The trade is that `items[1:]` is a full copy, so it’s $O(n)$ space instead of $O(1)$. I’d use the index version when space matters and the `zip` version when clarity matters more.

One thing I’d flag if the loop were doing deletion rather than comparison: iterating forwards while removing elements silently skips items, because the list shrinks under the loop’s position. For that case I’d iterate backwards, or use a write pointer, which is $O(n)$ rather than the $O(n^2)$ you get from repeated `pop`.

And if this generalises to a window of `k` rather than pairs, the bound becomes `range(n - k + 1)` — the same derivation — and I’d maintain a running total rather than re-slicing, to keep it $O(n)$ instead of $O(n \times k)$.”

That answer derives the bound rather than stating it, checks the edges, gives an alternative with its trade, and generalises — all in about ninety seconds.

9 Recall card

1. **Derive the bound, do not recall it.** Ask what the largest index the body touches is, and stop one past it. `items[i]` → `range(n)`. `items[i+1]` → `range(n-1)`. `items[i+k-1]` → `range(n-k+1)`.
2. `n` items have `n - 1` gaps. Seven elements, six adjacent pairs. Pegs and gaps.
3. **Backwards is** `range(n-1, -1, -1)` — stop-before is `-1` so index 0 is included. Or `reversed(items)`, which copies nothing.

4. **Iterate by value; use `enumerate` when you need the index.** `range(len(items))` with `items[i]` is nearly $2\times$ slower and has one more thing to get wrong.
5. **Never delete while iterating forwards** — it skips elements silently. Go backwards, or use a write pointer. And a sliding window should keep a running total, not re-slice.

1 What this is, and why they ask it

There are about a dozen numbers that let you estimate how long any system will take, without building it. How long a memory read takes, how long a disk read takes, how long a round trip across a city or across the world takes, how much data a network link moves in a second.

Yesterday you learnt the ladder those numbers sit on. Today you learn to **use** them: to add up the legs of a request, spot which one dominates, and say a number out loud before writing any code.

Interviewers ask because a system design round is scored on whether your answers are sized. “We should add a cache” is a suggestion anyone can make. “The database call is 40 ms of a 55 ms request, and 80% of reads are for 5% of the rows, so a cache takes the median to about 15 ms” is a design. The second sentence requires nothing but these numbers and arithmetic — and it is available to a candidate who has memorised twelve figures and is not available to one who has not.

2 The story

Prakash has lived in Thane for nine years and works near Lower Parel. At a quarter past nine on a Tuesday morning his manager rings and asks whether he can be at a client’s office in Worli by ten.

He says no. He says it in about four seconds, and he is not guessing.

The reason he can answer that fast is that he knows the pieces. Twelve minutes to walk to the station, and he does not need to check because he has done it eleven hundred times. Trains every four minutes at that hour, so call it two minutes of waiting on average. Forty-five minutes on the train. Then fourteen minutes at the other end. Twelve and two and forty-five and fourteen is seventy-three minutes, and he rounds it to seventy-five because nothing ever goes exactly right. Quarter past nine plus seventy-five minutes is half past ten. So: no, and here is when I can be there.

His nephew Sumit moved to the city in March and started at the same office in June. In his second week Sumit was asked the same kind of question and said “should be fine”, because it did not look far on the map. He arrived thirty-five minutes late, apologised,

and could not explain what had gone wrong, because he had never had the numbers in the first place. He had an impression. Prakash has legs and times.

There is a second thing Prakash knows, which took longer to learn. When he is trying to get somewhere faster, there is only one leg worth touching. If he takes an auto to the station instead of walking, he saves seven minutes off a twelve-minute walk. If the train could somehow be forty minutes instead of forty-five, that is five. The train is sixty per cent of the journey and everything else is noise around it. There is no point at all in leaving the house at a run.

And there is a third thing, which is the one that actually costs people meetings. Forty-five minutes is the ordinary number. On a Monday morning, or when it has rained in the night, it is seventy. It is seventy perhaps one day in twenty. So when it genuinely matters — when somebody is flying in, when there is one slot and no second chance — Prakash does not plan around forty-five. He plans around seventy, leaves early, and has a coffee at the other end. The typical number is for typical days. The bad number is what you plan a promise around.

3 The idea in plain English

Prakash's journey is back-of-the-envelope estimation, and his three habits are the three things this lesson is about.

The numbers, and why they must be memorised

You cannot estimate without raw figures, and there is no way round learning them. Here is the canonical list, rounded to the nearest useful power of ten.

OPERATION	TIME	REMEMBER IT AS
L1 cache reference	1 ns	1 ns
Branch mispredict	3 ns	
L2 cache reference	4 ns	
Mutex lock/unlock	17 ns	
Main memory reference	100 ns	0.1 µs
Compress 1 KB with Snappy	2 µs	
Send 1 KB over 1 Gbps network	10 µs	
SSD random read	100 µs	0.1 ms
Read 1 MB sequentially from memory	250 µs	
Round trip within a data centre	500 µs	0.5 ms
Read 1 MB sequentially from SSD	1 ms	
Disk seek (spinning)	10 ms	
Read 1 MB sequentially from disk	20 ms	
Round trip, one city to another	10–50 ms	
Round trip, across the world	150–250 ms	

The seven in bold are the ones to actually memorise. Everything else can be derived from them, and nobody will ask you for a mutex lock in an interview.

The relationships are more useful than the absolute values:

```
memory : SSD : disk seek = 100 ns : 100 us : 10 ms = 1 : 1,000 : 100,000
same-DC round trip : cross-world round trip = 0.5 ms : 200 ms = 1 : 400
```

Prakash's first habit: break it into legs and add them up

This is the whole technique. Any request is a sum of steps, and each step is one of the numbers above.

A typical API request:

```
client → load balancer → app server      (internet round trip)  40 ms
app parses, routes, validates              (CPU)                1 ms
app → Redis for the session                 (DC round trip)        0.5 ms
app → database, index lookup + row fetch    (DC round trip + SSD)  2 ms
app renders JSON                           (CPU)                1 ms
response back to client                     (already counted)      -
-----
                                           44.5 ms
```

Forty of those forty-four milliseconds are the user's own network connection, and nothing you do inside your data centre will touch it. That is a real and common result, and it is why the answer to “make the site faster” is so often a CDN rather than a database change.

Prakash's second habit: find the dominant term

Once you have the legs, **only the biggest one matters**. This is the same idea as dropping lower-order terms in [day 003](#), applied to wall-clock time.

In the request above, halving the CPU work saves 1 ms out of 44.5 — about two per-cent. Removing the database call entirely saves 2 ms. Moving the server closer to the user, so the round trip is 10 ms instead of 40, saves 30 ms — two thirds of the whole thing.

Optimising anything except the dominant term is wasted effort, and being able to say which term dominates is most of what a performance conversation is.

Prakash's third habit: the typical number is not the number you promise

Forty-five minutes on an ordinary day, seventy after rain.

Systems are the same, and the vocabulary matters because interviewers use it:

- **p50** (median): half of requests are faster than this. The typical day.
- **p95**: 19 in 20 are faster than this.
- **p99**: 99 in 100 are faster. The rainy Monday.
- **p99.9**: the very worst one in a thousand.

The gap is usually enormous. A service with a p50 of 20 ms often has a p99 of 200 ms, because the slow ones are the cache misses, the garbage collections, the retries, the connection that had to be re-established.

Users experience the tail, not the median. A page that makes 20 requests will, on average, contain one request from the p95. So the page's speed is governed by p95, not p50 — and that arithmetic is the single most useful thing in this section:

```
a page with 20 calls, each with a 1% chance of being slow
chance ALL of them are fast = 0.99^20 = 0.82
so 18% of page loads contain at least one slow call
```

The other half: throughput, not just latency

Latency is how long one thing takes. **Throughput** is how many things per second. They are different, and mixing them up is a classic interview stumble.

A 1 Gbps network link has a throughput of roughly 125 MB per second, and a round trip on it still takes 0.5 ms. Adding a second link doubles throughput and does nothing at all to latency — exactly as putting a second train line in would not make Prakash’s train faster.

The two conversions worth having ready:

1 Gbps = 125 MB/s	1 GB takes 8 seconds
10 Gbps = 1.25 GB/s	1 GB takes 0.8 seconds

The floor nobody can go below

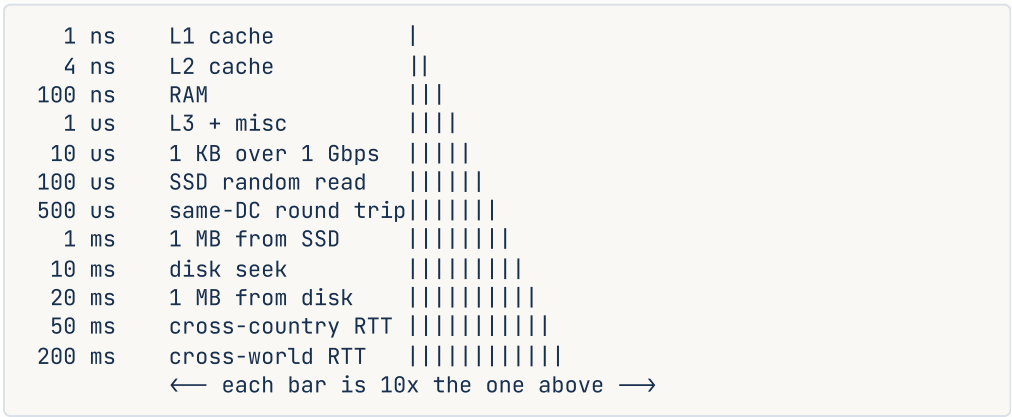
Light travels 300,000 km per second, and in fibre it is about two thirds of that — 200,000 km/s. Mumbai to London is roughly 7,200 km, and the cable does not go in a straight line, so call it 10,000 km each way:

$$20,000 \text{ km} / 200,000 \text{ km per second} = 0.1 \text{ s} = 100 \text{ ms}$$

One hundred milliseconds, minimum, forever. No amount of engineering reduces it, because it is physics. That is why global services put machines near users instead of making one fast one — and it is the entire justification for CDNs and multi-region deployments.

4 The picture

The numbers on a logarithmic scale, so the gaps are visible:



What to notice: each row is roughly ten times the one above. Twelve rows spans a factor of two hundred million. So an estimate that is out by a factor of two is fine, and an estimate that puts something on the wrong row is completely wrong. **Getting the row right is**

the skill; the digits do not matter.

Where the time goes in a real request, drawn to scale:

```
|=====|≡≡≡|≡≡≡|
|<----- 40 ms internet ----->| | |
                                1ms | 2ms | 1ms
                                0.5ms Redis
                                DB
```

total 44.5 ms. The user's own connection is 90% of it.

now move the server to the same country (10 ms RTT):

```
|=====|≡≡≡|≡≡≡|
|←10 ms →|
total 14.5 ms – a 3x improvement, and not one line of code changed.
```

What to notice: the whole right-hand side of the first picture is where engineers spend their time, and it is a tenth of the bar.

And the tail-latency effect, which is the picture worth remembering:

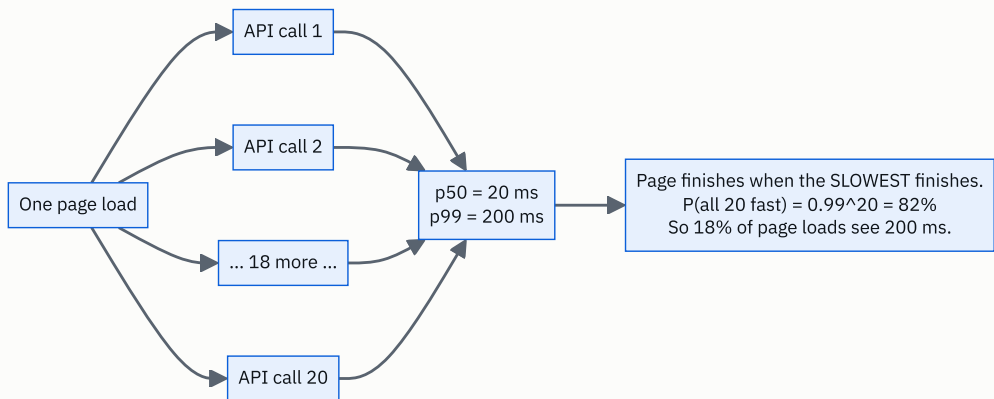


FIGURE 10.1

What to notice: every individual call is fast 99% of the time, and nearly one page load in five is slow. Fan-out turns a rare problem into a common one, and that is the reasoning behind almost every tail-latency technique you will meet later.

5 How it actually works

Where these numbers come from, and checking them yourself

They are not folklore. You can measure most of them in a minute.

```
ping google.com           # round trip to a nearby edge, usually 5-30 ms
ping <a server in Europe> # 100-200 ms from India
traceroute google.com     # every hop, with its own round trip
```

`ping` reports the round trip directly. `traceroute` shows the route and where the time is spent — usually in one or two long hops, which are the undersea legs.

For disk and memory: `fio` measures storage properly, `dd` gives a rough sequential number, and `perf stat` reports cache misses.

The canonical list is Jeff Dean’s “Latency Numbers Every Programmer Should Know”, published around 2010 and updated since. The absolute numbers have moved — SSDs got faster, networks got wider — but **the ratios have barely changed in fifteen years**, and the ratios are what you estimate with.

What actually makes up a network round trip

propagation delay	distance / (2/3 the speed of light)	-- physics, irreducible
transmission delay	bytes / link speed	-- how long to push it out
queueing delay	waiting in router buffers	-- the variable part
processing delay	per-hop routing decisions	-- small, ~microseconds

For a small request across the world, propagation dominates completely. For a large transfer on a slow link, transmission dominates. **Queueing is where the variance lives** — it is why p99 is so much worse than p50, and why a congested link degrades so sharply rather than gradually.

Why a CDN is the standard answer

Since propagation delay is distance divided by a constant, the only lever is distance. **Cloudflare, Akamai, CloudFront** and **Fastly** operate hundreds of edge locations, and DNS steers you to the nearest one ([day 003](#)).

```
Mumbai user → origin in Virginia : 250 ms round trip
Mumbai user → Cloudflare Mumbai  :   5 ms round trip
```

Fifty times better, from geography rather than engineering. Every static asset, and increasingly the dynamic content too, is served this way.

Where the numbers sit in real products

OPERATION	TYPICAL	WHY
Redis GET, same DC	0.5–1 ms	RAM plus one round trip; the round trip dominates
PostgreSQL indexed read, cached	1–3 ms	buffer pool hit plus round trip
PostgreSQL read from SSD	5–15 ms	add the storage cost
PostgreSQL write with fsync	5–20 ms	must be durable before acknowledging
Kafka produce, acks = 1	2–10 ms	sequential append
S3 first byte	50–150 ms	it is a network service, not a disk
DynamoDB single-item read	5–10 ms	
Elasticsearch query	10–100 ms	depends heavily on the query
Cross-region replication	50–200 ms	physics

Redis is worth pausing on. A memory read is 100 ns and a Redis GET is 500,000 ns. Redis is not slow — the memory access is a rounding error, and essentially all of that time is the network round trip and the syscall overhead. This is why batching matters so much: `MGET` for 100 keys costs one round trip, and 100 individual `GET` s cost 100. Same data, a hundred times the latency.

The rules of thumb that make estimation quick

```
seconds in a day      = 86,400, call it 100,000
seconds in a month    = 2.5 million
1 million requests/day = ~12 requests per second
1 billion requests/day = ~12,000 requests per second
peak is 2-5x average
```

That third line is the single most reusable one. **A million a day is twelve a second**, which is nothing. Ten million a day is 120 a second, which is still one machine. This is how you avoid over-designing, and interviewers notice when a candidate concludes “one server” and can show why.

6 The numbers

Estimate one: a social feed request. A user opens their timeline showing 50 posts.

DNS lookup (usually cached)	0 ms
TCP + TLS handshake (reused connection)	0 ms
request to edge, edge to origin	40 ms
auth: token check (in-process)	1 ms
fetch 50 post IDs from Redis (one MGET)	1 ms
fetch 50 posts from the database (one query)	5 ms
fetch 50 authors (one query, batched)	3 ms
render JSON	2 ms

	52 ms

Now the version somebody writes by accident, fetching each author separately:

50 separate author queries x 2 ms each	= 100 ms

total	149 ms

Three times slower from one loop. That is the **N + 1 query problem**, and it is the most common performance bug in application code. The arithmetic above is how you spot it in a design review before it ships.

Estimate two: how much storage. 50 million users, 20 posts each, 2 KB per post:

$$50,000,000 \times 20 \times 2 \text{ KB} = 2,000,000,000 \text{ KB} = 2 \text{ TB}$$

With 3× replication and 2 KB of index and metadata overhead per post:

2 TB x 3 = 6 TB of replicated post data
50,000,000 x 20 x 2 KB of overhead x 3 = another 6 TB

~12 TB

Twelve terabytes fits on a handful of machines. **The useful output of this estimate is that storage is not the problem** — which redirects the design conversation to what is.

Estimate three: requests per second. 10 million daily active users, 30 requests each:

10,000,000 x 30 = 300,000,000 requests per day
300,000,000 / 86,400 = 3,472 requests per second average
x 3 for peak = ~10,000 requests per second

And at 80 requests per second per machine ([day 007](#)):

$$10,000 / 80 = 125 \text{ machines, call it 150 with headroom}$$

Estimate four: does a cache pay for itself? Database read 5 ms, Redis read 0.5 ms, 90% hit rate:

```
without cache : 5 ms
with cache    : 0.9 x 0.5 + 0.1 x (0.5 + 5) = 0.45 + 0.55 = 1.0 ms
```

Five times better. Note the `0.5 + 5` on the miss path: a miss costs the cache lookup **and** the database read. That term is what makes a low hit rate actively harmful, and it is the detail that separates a real estimate from a hopeful one:

```
at a 20% hit rate: 0.2 x 0.5 + 0.8 x 5.5 = 0.1 + 4.4 = 4.5 ms
```

Barely better than no cache at all, and now you have a second system to operate.

Estimate five: bandwidth out. 10,000 requests per second at 50 KB each:

```
10,000 x 50 KB = 500 MB/s = 4 Gbps sustained
500 MB/s x 86,400 = 43 TB per day
43 TB x $0.09 per GB = about $3,900 per day of egress
```

Nearly \$1.4 million a year in bandwidth. **This is why CDN offload is a finance decision as much as a performance one.**

Estimate six: the tail. A service with p50 20 ms and p99 200 ms, and a page making 20 calls in parallel:

```
P(a given call is fast) = 0.99
P(all 20 fast)          = 0.99^20 = 0.818
P(at least one slow)    = 18.2%
```

Almost one page in five hits a 200 ms call. **The page's p50 is governed by the service's p99.** Which is why hedged requests, timeouts and fan-out reduction exist, and why "our p99 is fine, it's only 1%" is a sentence that should be challenged with this multiplication.

7 The trade-offs

Estimates buy speed of judgement and charge you accuracy. An order-of-magnitude estimate takes thirty seconds and tells you whether something is a one-machine problem or a hundred-machine problem, which is almost always the decision that matters. It will not tell you whether you need eleven machines or fourteen. Treating an estimate as a measurement is where this goes wrong — the correct use is to *rule things out*, then measure the survivors.

Latency and throughput are traded against each other constantly. Batching a hundred database writes into one round trip enormously improves throughput and makes the first write in the batch wait for the ninety-ninth — worse latency, better throughput.

Every queue in every system makes this trade. When an interviewer says “how would you improve this?”, it is worth asking which of the two they mean, because the answers point in opposite directions.

Optimising the median is usually the wrong target. The median is what your dashboard shows and the tail is what your users feel, especially with fan-out. A change that takes p50 from 20 ms to 15 ms and leaves p99 at 200 ms has improved almost nobody’s experience. The uncomfortable corollary is that tail work — timeouts, retries with back-off, hedged requests, reducing fan-out — is less satisfying and worth more.

Caching is not free, and its value is non-linear in the hit rate. At 90% it is transformative; at 20% it is a second system to run for almost no benefit, because misses pay both costs. And every cache adds an invalidation problem. The estimate above is how you decide whether to build one, and it needs a real hit-rate assumption to be worth anything — which usually means knowing how concentrated your access pattern is.

I would not estimate at all if... the answer depends on a constant I do not know. If the question is “how long does this query take”, the honest answer is “let’s measure it” — the range between an indexed lookup and a full scan on the same table is four orders of magnitude, and no amount of arithmetic bridges that. Estimation works for the parts governed by physics and by well-known constants: network distance, storage speed, byte counts, request rates. It does not work for the parts governed by somebody’s code.

8 In the interview

How it gets asked

- “*Roughly how long is a round trip to a data centre on another continent?*” — the direct version. The answer is 150–250 ms, and the reason is the speed of light.
- “*Estimate how much storage this needs.*” — show the multiplication.
- “*How many servers would you need?*” — requests per second divided by per-machine capacity.
- “*Where is the time going in this request?*” — break it into legs and name the dominant one.

What to say out loud, in the first ninety seconds

1. **Give the number with a range and a reason.** “*About 150 to 250 milliseconds. That’s dominated by propagation delay — light in fibre goes about 200,000 km per second, and it’s roughly 20,000 km round trip.*”
2. **Say what that means.** “*So it’s a floor. No engineering reduces it, which is why global services put servers near users rather than making one server faster.*”

3. **Give the neighbouring numbers.** *“Within a data centre it’s about half a millisecond. Across a country, 10 to 50. Across the world, 150 to 250. That’s a factor of 400 from the smallest to the largest.”*
4. **Apply it, unprompted.** *“Which is why a request that crosses continents twice — say the app calls a service in another region — costs half a second before anything is computed.”*
5. **Name the design consequence.** *“So I’d put read replicas or a CDN in each region, and keep cross-region calls off the request path — do them asynchronously if they’re needed at all.”*
6. **Offer the estimate.** *“Happy to size the whole request if that’s useful — I’d break it into legs and add them up.”*

The follow-ups

“Why can’t we make it faster?” Because it is bounded by physics. Light in fibre travels about 200,000 kilometres per second, and a round trip across the world is roughly 20,000 kilometres, so 100 ms is the theoretical floor and 150 to 250 is what you actually see once you add routing and queueing. You cannot engineer past it — you can only move the data closer to the user, which is what CDNs and multi-region deployments do, or reduce the number of round trips, which is what HTTP/2 multiplexing, connection reuse and request batching do.

“Where would you look first to make this request faster?” At the biggest leg. I would break the request into its legs — network to the user, auth, database, cache, rendering — and estimate each. Typically for a global service the user’s own network is 40 of 50 milliseconds, so the answer is a CDN or an edge location rather than anything in the application. If it is an internal service, the database is usually dominant, and then the question is whether it is a missing index, an $N+1$ loop, or a genuine cache opportunity. What I would avoid is optimising a 1 ms CPU step in a 50 ms request.

“Our p99 is 200 ms but our p50 is 20 ms. Is that a problem?” Almost certainly, and more than it looks. If a page makes 20 calls, the chance that all 20 land in the fast 99% is 0.99 to the twentieth, which is 82% — so nearly one page load in five contains a 200 ms call. Fan-out converts a rare event into a common one, so the page’s typical experience is governed by the service’s tail. I would want to know what causes the tail: cache misses, garbage collection pauses, connection establishment, a slow shard. Then either fix the cause, cut the fan-out, or use timeouts and hedged requests so one slow call cannot hold the page.

“Estimate the storage for a photo-sharing app with 100 million users.” I would state the assumptions, because the assumptions are the answer. Say 10% of users post daily, so 10 million uploads a day. Say each photo is 2 MB original plus 500 KB of resized versions, so 2.5 MB. That is 25 TB a day, or about 9 petabytes a year. With $3\times$ replica-

tion that is 27 PB a year, which is object storage rather than a database — S3 or equivalent. At roughly \$0.023 per GB per month that is about \$600,000 a month by the end of the first year, so I would immediately want a tiering policy: recent photos on standard storage, older ones on infrequent-access or Glacier. That cost result is usually the interesting output of the estimate, not the byte count.

A model answer

“A round trip to another continent is about 150 to 250 milliseconds, and the reason matters more than the number. Light in fibre travels about 200,000 kilometres a second, and a round trip Mumbai to Virginia is roughly 20,000 kilometres of cable, so 100 milliseconds is the physical floor. Routing and queueing take it to 150–250 in practice.

The full set I keep in my head is: memory 100 nanoseconds, SSD read 100 microseconds, disk seek 10 milliseconds, round trip within a data centre half a millisecond, across a country 10 to 50, across the world 150 to 250. Each of those is roughly a factor of a thousand or so apart, which is what makes them useful — I only need the right order of magnitude, not the right digits.

The way I use them is to break a request into legs and add up. A typical API call for a user on another continent is maybe 40 milliseconds of their own network, 1 for parsing, half a millisecond for a Redis session lookup, 2 to 5 for a database query, and a couple for rendering. So about 50 milliseconds, of which 80% is geography.

That tells me immediately where to look. Halving the CPU work saves 2%. Moving the server to the user’s region takes 40 milliseconds down to 10 and cuts total time by two thirds, without changing any code. So my first recommendation would be an edge presence, not a database optimisation.

The one caveat I’d add is that these are medians, and users experience the tail. If a page makes 20 backend calls and each has a 1% chance of being slow, then only 82% of page loads have all 20 fast — nearly one in five sees the p99. So when I’m sizing something that matters, I plan around p99 rather than p50, the same way you’d plan a journey around a bad traffic day rather than a typical one.”

That answer gives the number, the reason, the surrounding numbers, a worked application, the design consequence, and the tail caveat.

9 Recall card

1. **The seven to memorise:** RAM **100 ns**, SSD read **100 μ s**, disk seek **10 ms**, same-DC round trip **0.5 ms**, cross-country **10–50 ms**, cross-world **150–250 ms**, 1 Gbps = **125 MB/s**.
2. **Estimate by legs.** Break the request into steps, put a number on each, add them up, and **optimise only the dominant term**.
3. **150 ms across the world is physics.** 20,000 km at 200,000 km/s. The only lever is distance — which is what CDNs and multi-region deployments buy.
4. **Users feel the tail, not the median.** 20 calls each 99% fast means only 82% of pages are fast. Plan around p99.
5. **A million requests a day is twelve per second.** Peak is $2\text{--}5 \times$ average. That one line stops most over-designing before it starts.

Practice — Traversal: the loop patterns you will reuse forever

DSA topic: Traversal: the loop patterns you will reuse forever **System design topic:** Latency numbers every engineer should know

Code these, in this order

Four problems, one per traversal pattern. The code in each is short; the loop bound is the whole exercise.

For each problem, **before writing the loop**:

1. Say the largest index the body will touch.
2. Say the bound that follows from it, out loud, with the arithmetic.
3. Say how many iterations that gives for an array of 7.
4. Then write it, and check the empty and single-element cases without adding a special case.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Monotonic Array	LeetCode 896 (Easy)	The adjacent-pairs bound, exactly. <code>range(n - 1)</code> because the body reaches <code>i + 1</code> , and it must return <code>True</code> for a one-element array without a guard.
2	Maximum Average Subarray I	LeetCode 643 (Easy)	The fixed window. The slicing version is <code>O(n × k)</code> and passes; the running-total version is <code>O(n)</code> . Write the slow one first, then fix it, and say what changed.
3	Reverse String	LeetCode 344 (Easy)	Two pointers from the ends. <code>while left < right</code> , and the loop naturally handles both odd and even lengths without a special case.
4	Remove Element	LeetCode 27 (Easy)	The trap from §7 — deleting while iterating forwards. Write the <code>remove()</code> -in-a-loop version, watch it skip elements, then write the write-pointer version.

On problem 2, do this properly

- Write it with `sum(nums[i:i+k])` inside the loop. Submit it. Note the runtime.
- Work out the complexity: `n` up to 10^5 and `k` up to `n` means how many operations?
- Rewrite with a running total: add `nums[i]`, subtract `nums[i - k]`.
- Submit again and compare the runtimes.

- Then answer out loud: **why does the running total not work for a sliding-window maximum?**

The bounds drill

Answer these six from memory, with the arithmetic, in under ninety seconds:

- The body touches `items[i]`. What is the bound, and how many iterations for `n = 7`?
- The body touches `items[i + 1]`. Same two questions.
- The body touches `items[i + 2]`. Same two questions.
- A window of `k = 4` over `n = 10`. How many windows, and where does the last one start?
- Walking backwards over `n = 7` — write the `range(...)` exactly.
- Two pointers from the ends of `n = 7`. How many iterations before they meet?

The trap drill

Type this and predict the output **before** running it:

```
items = [1, 2, 2, 3, 2, 4]
for x in items:
    if x == 2:
        items.remove(x)
print(items)
```

Then explain, in one sentence, why one `2` survived. Then write two versions that give the right answer — one that builds a new list, one that walks backwards — and say what each costs in space.

The latency drill

Say these from memory, in under a minute:

- Main memory reference, in nanoseconds.
- SSD random read, in microseconds.
- Round trip within a data centre, in milliseconds.
- Round trip across the world, in milliseconds.
- 1 Gbps, converted to megabytes per second.
- How many requests per second a million requests a day works out to.

Then use them: a request has a 40 ms user network leg, a 1 ms parse, a 5 ms database query and a 2 ms render. What is the total, which leg dominates, and what single change would help most?

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Iterate over every adjacent pair in the array.* State the bound and its reason before writing it. Give the pair count for $n = 7$. Say what happens for an empty list without adding a guard.
 2. *Roughly how long is a network round trip to a data centre on another continent?* Give the range, then the physics, then the neighbouring numbers, then the design consequence.
 3. *Our p99 is 200 ms and our p50 is 20 ms. Is that a problem?* Do the fan-out multiplication out loud, then say what you would investigate, then name two techniques that limit the damage.
-

Before you move on

- [] I derive loop bounds from the largest index the body touches, not from memory.
- [] I can say why n items have $n - 1$ adjacent pairs, and why a window of k gives $n - k + 1$.
- [] I know the three reasons to iterate backwards.
- [] I never delete from a list while iterating forwards over it.
- [] I can quote seven latency numbers cold and use them to size a request in legs.

Insert, delete, and the cost of the middle

DATA STRUCTURES AND ALGORITHMS

Insert, delete, and the cost of the middle

You can say why adding to the end is cheap and adding to the middle is not.

SYSTEM DESIGN

The operating system's job

You can name what the OS does for your program: memory, scheduling, files, network.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What is the cost of deleting an element from the middle of an array?*
- *What does the operating system do when your program asks to read a file?*

Insert, delete, and the cost of the middle

1 What this is, and why they ask it

Deleting from position i of an array costs $n - i - 1$ moves, because every element after the hole has to slide back one slot to close it. Inserting at position i costs $n - i$ moves for the same reason, in the other direction.

So the cost is not a single number. It depends entirely on **where**. At the very end it is zero moves and $O(1)$. At the very front it is every element and $O(n)$. In the middle it is half of them, which is still $O(n)$.

Interviewers ask because it is the first question where “it depends” is the correct answer and most candidates give a single number instead. It is also the question underneath a whole family of later ones: why a queue needs a deque, why a linked list exists at all, why removing k elements one at a time is quadratic and removing them in one pass is linear. Get this right and a dozen later answers become obvious.

2 The story

The gas agency office on Station Road opens its complaints counter at ten. Vasanti gets there at twenty past nine and there are already thirty-one people ahead of her.

The queue is orderly because of the floor. Somebody painted white circles on the concrete, a metre apart, running from the counter to the door and then doubling back along the wall. Sixty of them, numbered. You stand on a circle. There is no arguing about who was first, because you can see it.

Vasanti is on circle 32.

At twenty to ten the man on circle 5 takes a phone call, listens, says something short and walks out of the building. And then the whole queue moves. The man on 6 steps onto 5, the woman on 7 steps onto 6, and it ripples all the way down past Vasanti to the last person on 43. Thirty-eight people pick up their bags and shuffle one circle forward. It takes about forty seconds and everybody does it without being asked, because leaving a gap would be worse.

Twenty minutes later, a woman near the back — circle 40 — gives up and leaves. Three people move. Nobody else even notices it happened.

And at half past ten the man at the very end walks off to buy a tea. Nobody moves at all. His circle was the last one, so there was nothing behind it to close up.

Vasanti finds herself thinking about this because it is the third time she has been here in two months. The cost is not the leaving. The cost is **everyone standing behind the person who left**.

The other direction happens at a quarter to eleven, when a clerk comes out with an old man who has a hospital appointment and puts him on circle 3. Now the shuffle runs the other way. Everyone from circle 3 to circle 41 steps *back* one, so that a circle is free for him. Thirty-nine people, and this time there is grumbling.

There is one more thing, and it is the reason nobody simply leaves a gap. At about eleven the clerk comes to the door and shouts that only the first twenty will be seen before lunch. Twenty from the front. If there were empty circles scattered through the queue, “the first twenty” would mean nothing — you would have to walk down the line counting people rather than just looking at circle 20 and drawing a line. The numbering only means something if there are no holes in it.

By the time Vasanti gets to the counter it is ten past twelve and she is on circle 4.

3 The idea in plain English

The painted circles are an array, and the shuffling is the whole lesson.

The cost is what is behind you

Removing the element at position `i` from an array of `n` elements leaves a hole. Everything after it must move one slot back to close the hole. How many elements are after position `i` ?

```
n - i - 1
```

Put numbers on it. In an array of 44 (the queue, positions 0 to 43):

REMOVE FROM	ELEMENTS THAT MOVE	COST
position 4 (circle 5)	39	expensive
position 39 (circle 40)	4	cheap
position 43 (the last)	0	free
position 0 (the front)	43	worst

Same operation, four different costs, and the only thing that changed is where.

Inserting is the mirror image. To insert at position `i`, everything from `i` onwards must move one slot forward to make room:

```
n - i
```

Why we call it $O(n)$ anyway

Big-O reports the worst case unless told otherwise. The worst position is the front, which moves all `n` elements. So:

- `items.insert(i, x) →  $O(n)$`
- `items.pop(i)` or `del items[i] →  $O(n)$`
- `items.append(x) →  $O(1)$` amortised
- `items.pop() →  $O(1)$`

The precise statement, which is better to say out loud, is: **$O(n - i)$, which is $O(n)$ in the worst case and $O(1)$ at the end.** Say the precise version and then the summary. It shows you know why.

Why you cannot just leave the hole

This is the part of the story that people skip, and it is the reason the whole shuffle is necessary.

An array's superpower is that position `i` is found by arithmetic — `base + i × element_size`, from [day 009](#). That formula assumes **no gaps**. Leave one hole at position 5 and every element after it is at an address one slot lower than its index says. The arithmetic now gives wrong answers, and `len()` no longer tells you how many real elements there are.

“The first twenty” only means something if the numbering is unbroken.

There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a **tombstone** — and databases and hash tables use it constantly. The price is that you now need a separate way to know which slots are real, and you must eventually **compact**: one pass that closes all the holes at once. That compaction is exactly the write-pointer pattern from [day 007](#), and it is why the pattern matters.

The trick when order does not matter

If you do not care about the order of the elements, deletion becomes $O(1)$:

```
def remove_at_unordered(items: list[int], i: int) → None:
    items[i] = items[-1]      # copy the last element over the hole
    items.pop()              # then drop the last slot
```

One copy and one cheap pop. Nothing shuffles. It is the queue equivalent of telling the person at the very back to come and stand on circle 5 — which would be outrageous in a queue and is perfectly fine when the collection is a *set* of things rather than an *order* of things.

Always ask whether order matters. It is the difference between $O(n)$ and $O(1)$, and it is a question the problem statement often does not answer.

Removing many: one pass, not many

Removing k elements one at a time costs $k \times O(n)$, which for k proportional to n is $O(n^2)$. Removing them all in one pass costs $O(n)$.

```
def remove_all(items: list[int], value: int) → int:
    write = 0
    for read in range(len(items)):
        if items[read] != value:
            items[write] = items[read]
            write += 1
    return write          # items[:write] is the answer
```

One index reads every element; another marks where the next keeper goes. Nothing is ever shifted more than once. This is the **write pointer**, it is $O(n)$ time and $O(1)$ space, and it owns [day 015](#).

What to use instead when the middle really is the problem

If your workload genuinely inserts and deletes in the middle constantly, an array is the wrong structure and no amount of care will fix it.

STRUCTURE	INSERT/DELETE IN THE MIDDLE	INDEX BY POSITION	NOTES
Array / Python list	$O(n)$	$O(1)$	the default, and usually right
<code>collections.deque</code>	$O(n)$	$O(n)$	but $O(1)$ at both ends
Linked list	$O(1)$ <i>once you hold the node</i>	$O(n)$	day 078
Balanced tree / sorted structure	$O(\log n)$	$O(\log n)$	ordered operations

The linked list row has an important caveat in it. Insertion is $O(1)$ **only if you already have a reference to the node**. If you have to find position i first, that is $O(n)$, and the total is no better than the array — with worse cache behaviour. [Day 009](#)’s locality argument is why arrays win in practice far more often than the complexity table suggests.

4 The picture

Deleting from the middle, drawn move by move:

before, n = 7, remove position 2

index

0

1

2

3

4

5

6

+-----+-----+-----+-----+-----+-----+-----+

| A | B | C | D | E | F | G |

+-----+-----+-----+-----+-----+-----+-----+

^

remove

the hole must be closed: everything after slides one left

+-----+-----+-----+-----+-----+-----+-----+

| A | B | D | E | F | G |

+-----+-----+-----+-----+-----+-----+-----+

<--- <--- <--- <---

4 elements moved = n - i - 1 = 7 - 2 - 1

What to notice: four moves for one deletion, and the four are everything that was standing behind the hole. That is the ripple down the queue.

The cost against position, drawn as a graph:

moves

6 | *

5 | *

4 | *

3 | *

2 | *

1 | *

0 | *

+-----> position deleted

0 1 2 3 4 5 6

delete at 0 → 6 moves

delete at 6 → 0 moves

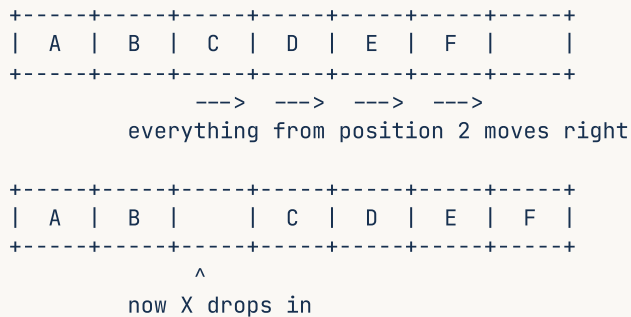
0(n)

0(1)

What to notice: it is a straight line, not a step. There is no single “cost of deletion” — there is a cost per position, and Big-O quotes the left-hand end of this line.

Inserting, which is the same picture reversed:

insert X at position 2, $n = 6$

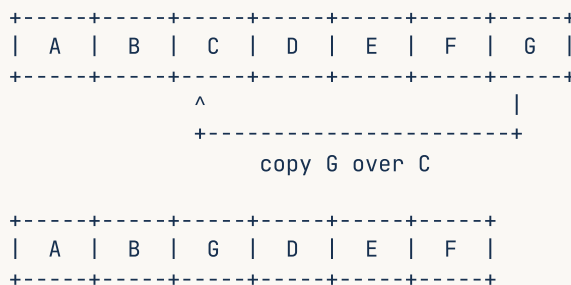


4 elements moved = $n - i = 6 - 2$

What to notice: the moves happen **back to front**. Copying **C** into slot 3 before moving **D** out of slot 3 would destroy **D**. Direction matters when shifting in place, and getting it backwards is a classic bug.

And the unordered trick, which avoids all of it:

remove position 2, order does not matter



1 move, whatever n is. $O(1)$.

What to notice: one arrow instead of four, and it would still be one arrow with a million elements. The entire cost of ordering, made visible.

5 The code, built step by step

Measure the cost against position, because the whole point is that it varies.

Start with deletion at three different positions.

```
import time

def time_delete_at(n: int, position: str) → float:
    items = list(range(n))
    index = {"front": 0, "middle": n // 2, "end": n - 1}[position]
    start = time.perf_counter()
    for _ in range(20_000):
        items.insert(index, 0)      # put one back so the list length is stable
        del items[index]
    return time.perf_counter() - start
```

Inserting then deleting at the same position keeps the length constant, so every repetition costs the same and the measurement is fair.

Now the write pointer, which is how you remove many things at once.

```
def remove_all_write_pointer(items: list[int], value: int) → int:
    write = 0
    for read in range(len(items)):
        if items[read] != value:
            items[write] = items[read]
            write += 1
    return write
```

`write` never overtakes `read`, so you are always writing into a slot already passed. $O(n)$ time and $O(1)$ extra space, and it returns the new length rather than resizing — which is exactly what LeetCode problems in this family ask for.

Compare it with the natural-looking version.

```
def remove_all_repeated(items: list[int], value: int) → None:
    while value in items:
        items.remove(value)      #  $O(n)$  to find +  $O(n)$  to shift, each time
```

`value in items` is $O(n)$ and `items.remove(value)` is another $O(n)$. If half the list matches, that is $n/2$ repetitions of $O(n)$ work — $O(n^2)$.

Now the unordered trick.

```
def remove_at_unordered(items: list[int], i: int) → None:
    """ $O(1)$ , at the cost of scrambling the order."""
    items[i] = items[-1]
    items.pop()
```

And the manual shift, so that the mechanism is not hidden inside a built-in.

```
def delete_by_hand(items: list[int], i: int) → int:
    """Exactly what del items[i] does. Returns how many elements moved."""
    n = len(items)
    for j in range(i, n - 1):
        items[j] = items[j + 1]    # slide each one back
    items.pop()                   # drop the now-duplicated last slot
    return n - i - 1
```

Note the direction: for deletion you copy **forwards**, from `j + 1` into `j`, starting at the hole. For insertion you must copy **backwards**, or you overwrite values you still need.

Here is the complete program.

```

"""Day 11 – the cost of insert and delete depends entirely on where."""

import time

def delete_by_hand(items: list[int], i: int) → int:
    """What `del items[i]` really does. Returns the number of elements moved."""
    n = len(items)
    for j in range(i, n - 1):
        items[j] = items[j + 1]
    items.pop()
    return n - i - 1

def insert_by_hand(items: list[int], i: int, value: int) → int:
    """What `items.insert(i, v)` really does. Note: shifts BACK to FRONT."""
    n = len(items)
    items.append(None) # make room at the end
    for j in range(n, i, -1): # walk backwards, or you clobber data
        items[j] = items[j - 1]
    items[i] = value
    return n - i

def remove_at_unordered(items: list[int], i: int) → None:
    """O(1) deletion when order does not matter."""
    items[i] = items[-1]
    items.pop()

def remove_all_write_pointer(items: list[int], value: int) → int:
    """Remove every occurrence in ONE pass. O(n) time, O(1) space."""
    write = 0
    for read in range(len(items)):
        if items[read] != value:
            items[write] = items[read]
            write += 1
    return write

def remove_all_repeated(items: list[int], value: int) → None:
    """The natural version. O(n^2)."""
    while value in items:
        items.remove(value)

def time_delete_at(n: int, index: int, repeats: int = 20_000) → float:
    items = list(range(n))
    start = time.perf_counter()
    for _ in range(repeats):
        items.insert(index, 0)
        del items[index]
    return time.perf_counter() - start

if __name__ == "__main__":
    print("what actually moves")
    for i in (0, 2, 5, 6):
        row = ["A", "B", "C", "D", "E", "F", "G"]
        moved = delete_by_hand(row, i)
        print(f" delete position {i} of 7 → {moved} elements moved, result {row}")

    print()
    row = ["A", "B", "C", "D", "E", "F"]
    moved = insert_by_hand(row, 2, "X")
    print(f" insert X at position 2 of 6 → {moved} elements moved, result {row}")

    print(f"\ndeleting 20,000 times from a list of 100,000")
    n = 100_000
    for label, index in (("front", (i = 0)), ("middle", (i = n // 2)), ("end", (i = n - 1))):
        secs = time_delete_at(n, index)
        print(f" {label:<20} {secs:>8.4f} s")

```

```

print("\nunordered deletion: one move, whatever n is")
row = ["A", "B", "C", "D", "E", "F", "G"]
remove_at_unordered(row, 2)
print(f"  remove position 2, order not preserved → {row}")

print("\nremoving many: one pass vs one at a time")
for size in (20_000, 40_000):
    data = [i % 4 for i in range(size)]          # a quarter of them are 0

    a = list(data)
    t0 = time.perf_counter(); remove_all_repeated(a, 0); slow = time.perf_counter() - t0

    b = list(data)
    t0 = time.perf_counter(); length = remove_all_write_pointer(b, 0)
    fast = time.perf_counter() - t0

    print(f"  n = {size:,}")
    print(f"    remove() in a loop  O(n^2) : {slow:>8.4f} s")
    print(f"    write pointer      O(n)   : {fast:>8.4f} s   → {slow / fast:.0f}x faster")
    print(f"    same answer? {a == b[:length]}")

```

This is exactly what it printed:

```

what actually moves
delete position 0 of 7 → 6 elements moved, result ['B', 'C', 'D', 'E', 'F', 'G']
delete position 2 of 7 → 4 elements moved, result ['A', 'B', 'D', 'E', 'F', 'G']
delete position 5 of 7 → 1 elements moved, result ['A', 'B', 'C', 'D', 'E', 'G']
delete position 6 of 7 → 0 elements moved, result ['A', 'B', 'C', 'D', 'E', 'F']

insert X at position 2 of 6 → 4 elements moved, result ['A', 'B', 'X', 'C', 'D', 'E', 'F']

deleting 20,000 times from a list of 100,000
front (i = 0)      13.7170 s
middle (i = n/2)   6.8679 s
end   (i = n-1)    0.0043 s

unordered deletion: one move, whatever n is
  remove position 2, order not preserved → ['A', 'B', 'G', 'D', 'E', 'F']

removing many: one pass vs one at a time
n = 20,000
  remove() in a loop  O(n^2) :   1.6118 s
  write pointer      O(n)   :   0.0027 s   → 588x faster
  same answer? True
n = 40,000
  remove() in a loop  O(n^2) :   6.6065 s
  write pointer      O(n)   :   0.0062 s   → 1,073x faster
  same answer? True

```

Read the timing block. Front, middle and end are 13.7, 6.9 and 0.004 seconds. The middle is almost exactly half the front, which is what `n - i - 1` predicts. The end is over three thousand times faster than the front, because zero elements move.

Read the last block, and notice the ratio between the two sizes. Doubling `n` from 20,000 to 40,000 took the slow version from 1.61 s to 6.61 s — four times, which is the quadratic signature from [day 003](#). The write pointer went from 0.0027 to 0.0062 — roughly double, which is linear.

6 What it costs

The exact counts, which is what you should say before you say the Big-O:

OPERATION	ELEMENTS MOVED	COMPLEXITY
<code>items.append(x)</code>	0	<code>O(1)</code> amortised
<code>items.pop()</code>	0	<code>O(1)</code>
<code>items.insert(i, x)</code>	<code>n - i</code>	<code>O(n - i)</code> , worst <code>O(n)</code>
<code>items.pop(i)</code> / <code>del items[i]</code>	<code>n - i - 1</code>	<code>O(n - i)</code> , worst <code>O(n)</code>
<code>items.insert(0, x)</code>	<code>n</code>	<code>O(n)</code>
<code>items.pop(0)</code>	<code>n - 1</code>	<code>O(n)</code>
<code>items.remove(x)</code>	<code>O(n)</code> to find + <code>n - i - 1</code> to shift	<code>O(n)</code>
Unordered delete (swap with last)	1	<code>O(1)</code>
Remove all matches, write pointer	<code>n</code> total	<code>O(n)</code>
Remove all matches, one at a time	up to <code>n²/2</code>	<code>O(n²)</code>

The arithmetic on the quadratic, because this is the one that shows up in real code. Removing 5,000 elements one at a time from a list of 20,000:

```
each remove() : O(n) to find + O(n) to shift, so about 20,000 operations
5,000 removes : 5,000 x 20,000 = 100,000,000 element operations
```

One hundred million, for a job that needs twenty thousand. The write pointer does:

```
one pass : 20,000 reads + at most 20,000 writes = 40,000 operations
```

A factor of 2,500 in operation count, and you measured 588× and 1,073× in wall-clock, because the built-in `remove` runs at C speed and the Python loop does not. The Big-O gap is real; the constant partly hides it, and the gap widens as `n` grows.

What “amortised `O(1)`” means for `append`, restated because it belongs next to these numbers. From [day 005](#): the list over-allocates by about 12.5%, so most appends land in spare capacity and occasionally one triggers a full copy. Building a list of a million by appending copies about 1.9 million elements in total, so **about two extra operations per append**, averaged.

Space. All of these are `O(1)` extra space — the shifting happens inside the existing block. The one exception is anything that builds a new list:

```
[x for x in items if x != value] → 0(n) extra space, 0(n) time
write pointer                    → 0(1) extra space, 0(n) time
```

Both are $O(n)$ time. Choose by whether you may modify the input, which is a question for the interviewer.

When the middle really is your workload. If you insert or delete in the middle m times on an array of n :

```
array      : m x 0(n)      = 0(m x n)
linked list: m x 0(1)      = 0(m)    -- but only if you already hold the node
```

The caveat is doing real work in that sentence. Finding position i in a linked list is $O(n)$, so unless you got the node from somewhere else — an iterator you are already holding, or a hash map pointing at nodes, which is exactly how an LRU cache works ([day 076](#)) — the linked list is not faster, and it has worse cache behaviour on top.

7 The traps

Trap one: removing in a loop, forwards

The classic. You want to remove every negative number:

```
items = [1, -2, -3, 4, -5, 6]
for i in range(len(items)):
    if items[i] < 0:
        del items[i]
print(items)
```

```
Traceback (most recent call last):
  File "d11.py", line 3, in <module>
    if items[i] < 0:
        ~~~~~^^^
IndexError: list index out of range
```

Two separate bugs in three lines. The list shrinks while `range(len(items))` was computed once at the start, so i eventually exceeds the new length — that is the `IndexError`. And even before it crashes, deleting at i shifts the next element **into** position i , which the loop then skips over by advancing.

Watch it happen: $i = 1$ deletes `-2`, so the list becomes `[1, -3, 4, -5, 6]` and `-3` is now at position 1. The loop moves to $i = 2$, which is `4`. `-3` is never examined.

The `remove`-based version fails differently and just as informatively:

```
items = [1, -2, 4]
for x in items:
    if x < 0:
        items.remove(x)
        items.remove(x)      # a second remove, by mistake
```

```
Traceback (most recent call last):
  File "d11.py", line 5, in <module>
    items.remove(x)
    ~~~~~^
ValueError: list.remove(x): x not in list
```

`ValueError` rather than `IndexError`, because `remove` searches by value and there is no longer a match. The message names the method and the reason exactly.

The three correct fixes, in the order a reviewer would prefer them:

```
items = [x for x in items if x ≥ 0]      # new list, O(n) time, O(n) space
```

```
write = 0                                # write pointer: O(n) time, O(1) space
for read in range(len(items)):
    if items[read] ≥ 0:
        items[write] = items[read]
        write += 1
del items[write:]
```

```
for i in range(len(items) - 1, -1, -1):  # backwards: correct, but O(n^2)
    if items[i] < 0:
        del items[i]
```

The backward loop is correct because deleting at `i` only shifts elements after `i`, which are already visited. It is still `O(n2)` because each `del` shifts. Use it when the list is small and clarity matters; use the write pointer when it is not.

Trap two: shifting in the wrong direction

This one produces no error at all. It produces a list full of duplicates.

```
def insert_by_hand(items: list[str], i: int, value: str) → None:
    items.append(None)
    for j in range(i, len(items) - 1):      # forwards – wrong direction
        items[j + 1] = items[j]
    items[i] = value

row = ["A", "B", "C", "D", "E"]
insert_by_hand(row, 1, "X")
print(row)
```



```
['A', 'X', 'B', 'B', 'B', 'B']
```

C, D and E are gone, replaced by copies of B. The reason is the order of the copies. Copying `items[1]` into `items[2]` destroys C before anything has read it. Then `items[2]` — which is now B — gets copied into `items[3]`, destroying D. Each step propagates the same value forward.

When you shift right, you must copy from the back. When you shift left — which is what deletion does — you must copy from the front. The rule is: **start at the end you are moving towards.**

```
for j in range(len(items) - 1, i, -1):    # backwards - correct
    items[j] = items[j + 1]
```

This is the same failure mode as an overlapping `memcpy` in C, and it is the reason `memmove` exists as a separate function. It comes back on [day 084](#), where merging two sorted arrays in place must fill from the end for exactly this reason.

8 In the interview

How it gets asked

- “What’s the cost of deleting an element from the middle of an array?” — the direct version. “It depends where” is the start of the right answer, not a dodge.
- “Why is appending $O(1)$ but inserting at the front $O(n)$?” — the same fact from the other side.
- “Can you delete from an array in $O(1)$?” — the unordered-trick question. The answer is yes, with a condition.
- “Remove all occurrences of a value in place.” — the write-pointer question, and the naive answer is quadratic.

What to say out loud, in the first ninety seconds

1. **Give the exact count first.** “It’s n minus i minus 1 moves, where i is the position — because everything after the hole has to slide back one slot to close it.”
2. **Give the range.** “So deleting the last element is zero moves and $O(1)$. Deleting the first is n minus 1 moves and $O(n)$. The middle is n over 2, which is still $O(n)$.”
3. **Say why the shifting is unavoidable.** “You can’t leave the hole, because an array finds position i by arithmetic — base plus i times element size — and that only works if the slots are unbroken.”

4. **Offer the $O(1)$ version with its condition.** *“If the order doesn’t matter, I can do it in $O(1)$: copy the last element over the hole and pop the end. One move, whatever n is. That’s only valid if it’s a collection rather than a sequence.”*
5. **Pre-empt the multiple-deletion question.** *“And if I’m removing many elements, I’d do it in one pass with a write pointer rather than calling delete repeatedly — that’s $O(n)$ instead of $O(n \text{ squared})$.”*
6. **Name the structural alternative.** *“If middle insertion were the dominant operation, an array is the wrong structure — that’s what a linked list buys, at the cost of $O(n)$ indexing and much worse cache locality.”*

Steps 4 and 5 are what turn a definition into a design answer.

The follow-ups

“Why can’t you just mark it as deleted and leave a hole?” You can, and real systems do — it is called a tombstone, and hash tables and databases rely on it. The cost is that you have broken the array’s core property: position no longer maps directly to index, so `len` no longer tells you how many real elements there are, and any code that indexes into it needs a way to skip holes. So you defer the shifting and eventually pay it all at once in a compaction pass. That is often the right trade when deletions are frequent and reads can tolerate the extra check — a batched $O(n)$ compaction beats k separate $O(n)$ shifts. But it is a trade, not a free win.

“Can you delete from an array in $O(1)$?” Yes, if order doesn’t matter: copy the last element into the slot being deleted and pop the end. One assignment and one $O(1)$ pop, regardless of size. The condition is the whole answer — it works for a set of things and destroys a sequence of things. I’d ask explicitly whether order matters before using it, because a problem statement often doesn’t say, and the difference between $O(n)$ and $O(1)$ is worth one question.

“You called remove in a loop. What’s the complexity?” $O(n^2)$. Each `remove` is $O(n)$ to find the value plus $O(n)$ to shift what’s behind it, and doing that a number of times proportional to n squares it. I’d replace it with a single pass using a write pointer: one index reads everything, another marks where the next kept element goes, so nothing moves more than once. That’s $O(n)$ time and $O(1)$ extra space. If I’m allowed to allocate, a list comprehension is $O(n)$ time and $O(n)$ space and reads better.

“When would you use a linked list instead?” When insertions and deletions in the middle dominate **and** I already hold a reference to the node, so I’m not paying $O(n)$ to find it. In practice that means the linked list is part of a larger structure — an LRU cache where a hash map points at nodes, or an iterator I’m already walking. If I’d have to

search for the position first, the linked list is $O(n)$ too, with worse cache behaviour, because its nodes are scattered rather than contiguous. That's why arrays beat linked lists in real benchmarks far more often than the complexity table suggests.

A model answer

"It depends on the position, and I'd give the exact count rather than just the Big-O.

Deleting at position i costs $n - i - 1$ element moves, because everything after the hole has to slide one slot back to close it. So in a list of 44, deleting position 4 moves 39 elements, deleting position 39 moves 4, and deleting the last element moves nothing at all. Worst case is the front at $n - 1$ moves, so we call it $O(n)$ — but at the end it's genuinely $O(1)$, and it's worth saying both.

The reason the shifting is unavoidable is the array's layout. Position i is found by arithmetic — base address plus i times the element size — and that only works if the slots are contiguous with no gaps. Leave a hole and every subsequent element is at an address that disagrees with its index. So it's the same contiguity that makes indexing $O(1)$ that makes deletion $O(n)$. They're two sides of one design decision.

There is an $O(1)$ version if order doesn't matter: copy the last element over the slot being deleted and then pop the end. One move regardless of size. I'd ask whether order matters before using it, because for a collection it's free and for a sequence it's a bug.

And if I'm deleting many elements rather than one, I wouldn't call delete repeatedly — that's $O(n)$ each and $O(n^2)$ overall. I'd do a single pass with a write pointer: one index reads every element, another marks where the next keeper goes, so nothing moves twice. $O(n)$ time, $O(1)$ extra space. I've measured that at nearly $600\times$ faster on 20,000 elements, and the gap widens as n grows because one is quadratic and one is linear.

If middle insertion were genuinely the dominant operation in the workload, I'd change the structure rather than the code — a linked list gives $O(1)$ insertion, but only once you hold the node, and it costs you $O(1)$ indexing and cache locality. So I'd want to see the access pattern before making that trade."

9 Recall card

1. Delete at i costs $n - i - 1$ moves. Insert at i costs $n - i$. The cost is everything standing behind the position.

2. **The end is free, the front is $O(n)$.** `append` and `pop()` move nothing; `insert(0, x)` and `pop(0)` move everything.
3. **The holes cannot stay** — indexing is `base + i * size`, which needs unbroken slots. That is the same property that makes indexing $O(1)$.
4. **If order does not matter, deletion is $O(1)$** : copy the last element over the hole and pop the end. Always ask whether order matters.
5. **Removing many is one pass, not many deletes.** Write pointer: $O(n)$ and $O(1)$ space against $O(n^2)$. And when shifting in place, **start at the end you are moving towards.**

1 What this is, and why they ask it

The **operating system** owns the hardware. Your program does not. When your code reads a file, sends a network message, allocates memory or starts a thread, it is not doing any of those things — it is **asking** the OS to do them, and waiting.

The mechanism for asking is a **system call**, and the boundary it crosses is the most important line in a computer. On your side, your program cannot damage anything but itself. On the other side, the **kernel** can do anything at all.

Interviewers ask because the answer explains costs that otherwise look arbitrary. Why is buffered I/O so much faster than unbuffered? Why does a blocking call hold a whole worker? Why is a context switch expensive? Why do containers isolate without a virtual machine? Every one of those is a fact about this boundary. A candidate who can walk a file read from `read()` to the disk and back has a mental model that will keep paying off for the rest of the interview.

2 The story

Balan has been the caretaker of a hundred-room working women's hostel in Ernakulam for sixteen years. He sits at a desk just inside the gate, and almost nothing in that building happens without going past him.

The store room is behind his desk and it is locked. Buckets, mops, bulbs, mattresses, spare blankets, the ladder. When a resident needs a bucket she comes to the desk and asks, and he gets up, unlocks the store, finds one, and hands it over. It takes him ninety seconds.

She could get it herself in twenty. That is not the arrangement, and everybody understands why after they hear about 2019, when the store was left open for a week during renovations. People took things without saying, took the wrong things, put things back in the wrong place, and by the end of it there were eleven buckets, no bulbs, and the ladder was in somebody's room. It took two days to sort out. The ninety seconds is not inefficiency. It is what somebody keeping track costs.

He allots the rooms too. He decides who gets which one, and everybody's key opens exactly one door. A resident cannot walk into another resident's room, and this is not politeness — it is the lock. Nobody has to trust anybody.

The common room is his most delicate job because there is one television and forty women who want it. He keeps a rota. Half an hour each on weekday evenings, and when somebody's half hour is up he comes in and says so, even mid-programme, and the next person sits down. It is the only arrangement that has ever worked, and the reason it works is that nobody has to negotiate with anybody.

Post and visitors come to the gate, to him, and he sends them up to the right room. When somebody's brother arrives, he checks the register on his screen, confirms the room, and rings up. The visitor never wanders the building looking.

And he says no. When a resident asks for the ladder to hang something from the ceiling, he says no, because that is a rule from the management and it is not his to bend. When somebody who left last month comes back for a spare key, he says no, because she is not on the list any more. Half of what he does is knowing what people are and are not allowed.

The one thing that genuinely slows the building down is when he is away from the desk. If he is up on the third floor fixing a tap, six people are standing at the gate waiting, and nothing any of them wants is difficult. They simply cannot do it themselves.

3 The idea in plain English

Balan is the kernel, and the desk is the system call boundary.

Two sides of one line

User space is where your program runs. It can compute, and it can touch its own memory, and that is all. It cannot read a disk, send a network message, or look at another program's memory. The processor physically forbids it.

Kernel space is where the operating system runs, with full access to everything.

The line between them is enforced by hardware — the CPU runs in different **privilege modes**, and the instructions that touch devices only work in the privileged one. This is not a convention that programs politely follow. It is a wall.

A system call is asking at the desk

A **system call** is how user space asks kernel space to do something. When your Python code runs `open("data.txt")` and then `.read()`, underneath it is making system calls named `open`, `read` and `close`.

What physically happens on each one:

1. Your program puts the request in an agreed place and executes a special instruction (`syscall` on x86-64).

2. The CPU **switches to kernel mode** and jumps to a fixed handler in the kernel.
3. The kernel checks the arguments and **checks permission** — this is Balan saying no.
4. It does the work.
5. It switches back to user mode and returns.

The round trip costs roughly **1–2 microseconds** even when the work itself is trivial. That is the ninety seconds at the desk: not the bucket, the walking.

You can watch every one of them. `strace ./yourprogram` on Linux prints each system call your program makes, with arguments and return values, and it is one of the most useful hours a beginner can spend.

The four things it does for you

Memory. Each process gets its own **virtual address space** — its own set of addresses that the OS maps onto real physical memory. Process A’s address 0x1000 and process B’s address 0x1000 are different physical bytes. That is the room key that opens one door. When memory runs short, the OS writes pages out to disk (**swap**) and reads them back on demand.

Scheduling. More runnable threads than cores, always. The **scheduler** gives each one a **time slice** of a few milliseconds and then switches. Nobody negotiates with anybody; the rota is imposed. This is why your program can be interrupted between any two instructions, which is the root of every race condition from [day 008](#).

Files and devices. The OS presents one uniform interface — open, read, write, close — over completely different hardware. The same four calls work on an SSD, a spinning disk, a USB stick, a network share, a terminal and a network socket. **Device drivers** are the part that knows the differences. On Unix this uniformity is the famous “everything is a file”, and it is why a socket and a file behave alike to your code.

Networking. The OS implements the whole TCP/IP stack from [day 004](#) — checksums, sequence numbers, retransmission, congestion control. Your program hands it bytes. That is post arriving at the gate and being routed to the right room.

And running through all four: **permissions**. Every one of those calls is checked. Can this process read this file? Bind this port? Send this signal? Half of what the kernel does is saying no.

The file descriptor

When you open a file, the kernel returns a small integer — a **file descriptor**. It is not the file. It is a number that indexes into a table the kernel keeps for your process, and the entry in that table holds the real information: which file, what position, what permissions.

Three descriptors always exist: **0** is standard input, **1** is standard output, **2** is standard error. That is why `2>/dev/null` means “throw the errors away”.

Sockets get file descriptors too, which is why `read()` works on both a file and a network connection, and why a process can run out of file descriptors and fail to accept new connections — a real and common production error, `EMFILE: too many open files`.

Why one call can be so much more expensive than another

This is the part that connects today to everything practical.

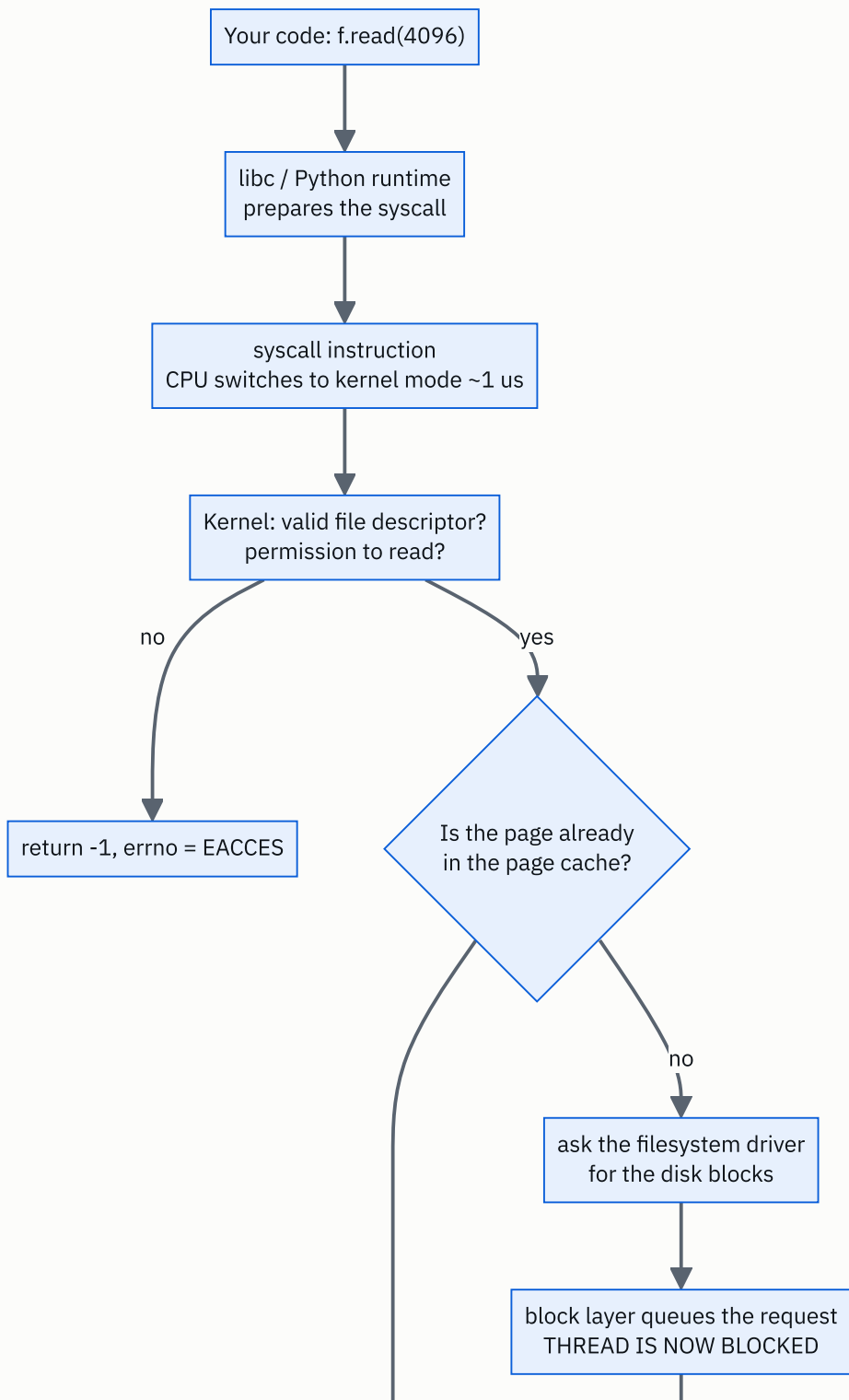
A system call that is satisfied from memory the kernel already has is fast. A system call that must wait for hardware is not — and while it waits, **your thread is blocked**: taken off the processor entirely until the data arrives.

```
read() from the page cache (already in RAM) :    ~1 us    -- the boundary crossing
read() that must go to SSD                  :   ~100 us   -- 100x
read() that must go to a spinning disk      : ~10,000 us  -- 10,000x
```

That single table is why buffering exists, why asynchronous servers exist, and why the hierarchy from [day 009](#) shows up in your application’s latency rather than staying a hardware fact.

4 The picture

What actually happens on `read()` :



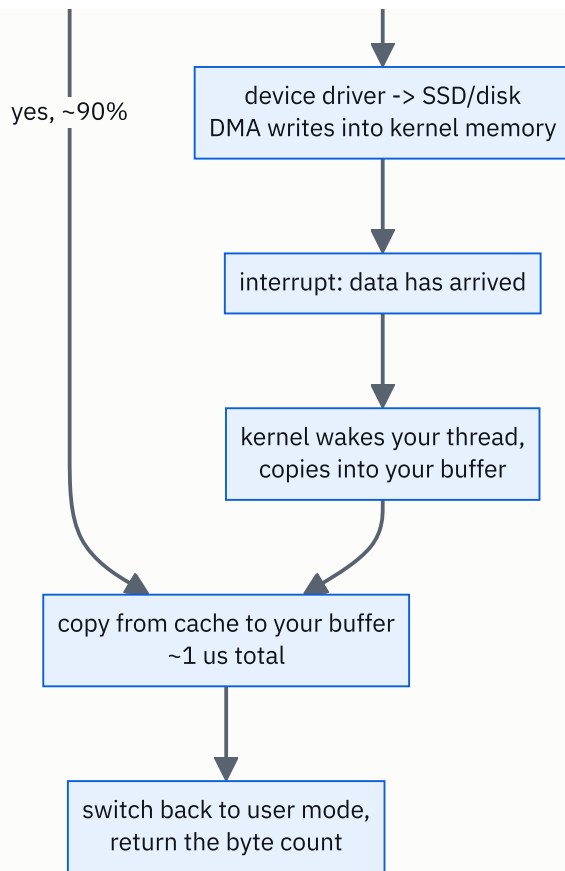
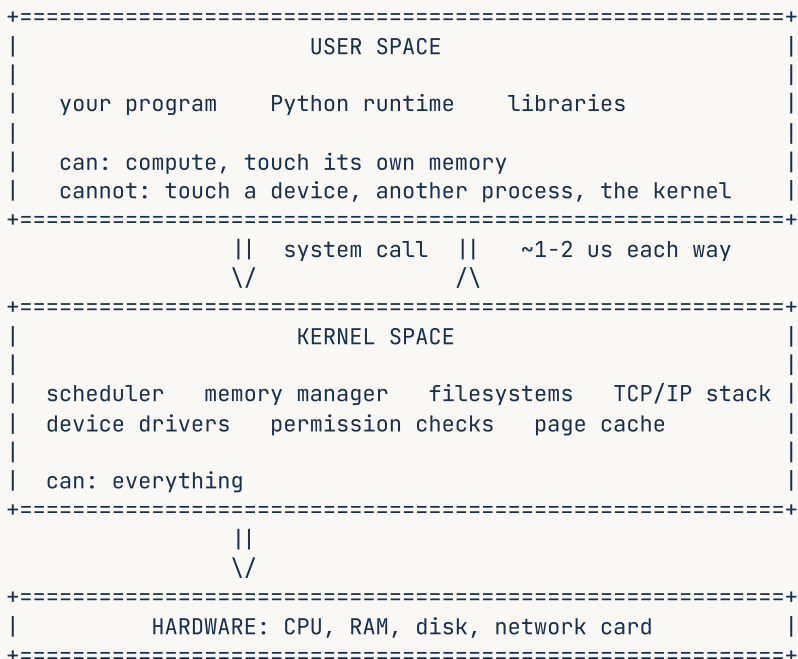


FIGURE 11.1

What to notice: the two paths out of the diamond differ by a factor of a hundred. The fast one never touches hardware at all — it is a copy from memory the kernel is already holding. **The page cache is why most file reads are not disk reads.**

The boundary itself:



What to notice: every arrow between your program and the hardware goes through the middle box. There is no direct path, and that is the entire security and stability model of a modern computer.

And why buffering matters so much, drawn as call counts:

```

reading 1 MB, one byte at a time:
  1,048,576 system calls x ~1 us = about 1 second

reading 1 MB, 4 KB at a time:
  256 system calls x ~1 us      = about 0.3 milliseconds
                                3,000x fewer crossings

```

What to notice: the data read is identical. The only thing that changed is how many times you walked to the desk.

5 How it actually works

The page cache, which is where the speed comes from

The kernel keeps recently read file blocks in RAM, in the **page cache**, and it uses all the free memory on the machine to do it. This is why `free -h` on a healthy Linux server shows almost no free memory and a large `buff/cache` figure — that is not memory being wasted, it is memory doing its job.

Consequences worth knowing:

- The second read of a file is typically a hundred times faster than the first.
- A `write()` normally returns as soon as the data is in the page cache, **before** it reaches the disk. It is not durable yet. `fsync()` is what forces it out, and it is why a database commit costs 5–20 ms rather than 50 μ s.
- **Kafka** deliberately relies on the page cache instead of maintaining its own — it writes sequentially and lets the kernel serve recent reads from memory.

Virtual memory and page faults

Every process sees a private, flat address space. The **MMU** hardware translates those addresses to physical ones using **page tables**, in 4 KB pages, with a small cache of recent translations called the **TLB**.

When a program touches an address whose page is not currently in physical memory, the hardware raises a **page fault** and the kernel steps in. There are two kinds, and the difference matters:

- A **minor fault** means the page is in memory but not yet mapped to this process — cheap, microseconds. This is what happens the first time you touch newly allocated memory, and it is why `malloc` is fast but the first write to that memory is not.
- A **major fault** means the page must be read from disk — a hundred thousand times slower. Enough of these and the machine is **thrashing**, doing nothing but paging.

`vmstat` and `/proc/<pid>/stat` report both counts, and a rising major-fault rate is one of the clearest signals that a server has run out of memory.

The scheduler

Linux uses **CFS** — the Completely Fair Scheduler — which tries to give each runnable thread an equal share of CPU time. A thread runs until its slice expires, it blocks on I/O, or a higher-priority thread becomes runnable.

A **context switch** between threads of the same process costs perhaps 0.2 μ s directly, and much more indirectly, because the new thread's data is not in the CPU caches. Between processes it is worse, because the page tables change and the TLB is flushed.

That indirect cost is why “add more threads” stops working: past a certain point the machine spends more time switching than working, exactly as [day 008](#) computed.

Blocking, and what asynchronous I/O actually changes

When your thread calls `read()` and the data is not in the page cache, the kernel marks the thread as blocked, takes it off the CPU, and runs something else. When the data arrives, the thread becomes runnable again.

Nothing is wasted from the machine’s point of view — but from your application’s point of view, that worker is gone for the duration. This is precisely the barber standing still from [day 007](#).

The alternatives, in order of how modern they are:

- `select` / `poll` — ask about many descriptors at once, `O(n)` per call.
- `epoll` (Linux) / `kqueue` (BSD) — the kernel maintains the set, so it is `O(1)` per ready descriptor. This is what makes 10,000 concurrent connections practical, and it is what `nginx`, `Node.js` and `asyncio` are built on.
- `io_uring` (Linux 5.1+) — shared ring buffers between user and kernel space, so batches of operations can be submitted and completed **with almost no system calls at all**. It is the current state of the art for high-performance I/O.

Notice the direction of travel: every generation removes boundary crossings.

Containers are an OS feature, not a virtual machine

Worth knowing because it comes up constantly. A container is **one normal process** with three kernel features applied:

- **Namespaces** — the process sees its own view of process IDs, network interfaces, mount points and hostnames.
- **cgroups** — limits on CPU, memory and I/O.
- **A different root filesystem** — the image.

There is no second operating system. **Docker containers share the host kernel**, which is why they start in milliseconds while a virtual machine takes tens of seconds, and also why a kernel vulnerability crosses container boundaries in a way it cannot cross VM boundaries. [Day 013](#) covers this properly.

When it goes wrong

Out of file descriptors: `EMFILE: too many open files`. The default limit is often 1024, which any server holding connections will exceed. Raise it with `ulimit -n`, and find the leak.

Out of memory: the **OOM killer** picks a process and terminates it. On a server this usually means your application dies with no stack trace and no log line, and the only evidence is in `dmesg`.

Zombie processes: a child that exited whose parent never collected its status. Harmless individually, and enough of them exhaust the process table.

6 The numbers

What crossing the boundary costs:

```
function call inside your program : ~1 ns
system call (trivial work)         : ~1,000 ns = 1 us    → 1,000x
context switch, same process       : ~200 ns
context switch, different process  : ~2,000 ns
```

A system call is a thousand times more expensive than a function call. That ratio is the reason for every buffering layer in every language's standard library.

Reading a 1 MB file, three ways:

```
1 byte at a time : 1,048,576 syscalls x 1 us = 1,049,000 us = 1.05 s
4 KB at a time   :      256 syscalls x 1 us =      256 us = 0.26 ms
mmap              :      1 syscall + page faults on access
```

Four thousand times faster by changing the buffer size and nothing else. This is why Python's `open()` buffers by default, and why `open(path, buffering=0)` is a way to make your program dramatically slower without changing a single line of logic.

The page cache hit rate, which decides your file I/O performance:

```
90% hit : 0.90 x 1 us + 0.10 x 100 us = 0.9 + 10 = 10.9 us
99% hit : 0.99 x 1 us + 0.01 x 100 us = 0.99 + 1.0 = 2.0 us
```

Same arithmetic shape as every cache in this course, and the same conclusion: **the misses dominate.**

What a database commit costs, and why:

```
write() to the page cache : ~5 us (not durable)
fsync() to SSD            : ~1,000 us (durable)
fsync() to spinning disk  : ~10,000 us
```

So a naive design doing one `fsync` per transaction on an SSD is capped at:

```
1,000,000 us / 1,000 us = 1,000 transactions per second
```

A thousand per second, from physics rather than from software. The standard escape is **group commit** — batching many transactions into one `fsync` — which is what PostgreSQL's `commit_delay` and MySQL's group commit do, and it can raise that by an order of magnitude.

Memory allocation, which is not what people assume:

```
malloc / Python object allocation, from the pool : ~100 ns (no syscall at all)
mmap or brk, when the pool needs more from the OS : ~1,000 ns
first write to a newly mapped page (minor fault) : ~1,000 ns
a page read from swap (major fault) : ~100,000 ns
```

Most allocations never involve the kernel — the runtime keeps its own pool. That is why allocation-heavy code is usually fine and *page-fault*-heavy code is not.

Context switching at scale, restated from day 008 because it is a kernel fact:

```
10,000 runnable threads, 8 cores, 1 ms slices
= 10,000 switches per round x 0.2 us = 2 ms of pure switching per 1 ms of work
```

More time switching than working. The fix is always fewer threads.

7 The trade-offs

The boundary buys safety and charges you a thousand-to-one on every crossing. The wall between user and kernel space is why one buggy program cannot corrupt another, why a crash is a crash and not a reboot, and why a compromised process cannot read the disk directly. The price is $\sim 1\ \mu\text{s}$ per system call, which is invisible until you make a million of them. Everything called “high performance I/O” is fundamentally a scheme to cross less often — buffering, `epoll`, `io_uring`, kernel bypass frameworks like DPDK.

Buffered writes buy throughput and charge you durability. A `write()` that returns when the data reaches the page cache is fast and is a lie about permanence: the machine losing power in the next second loses that data. `fsync()` makes it true and costs a millisecond or more. Every database in existence is a careful set of decisions about exactly when to pay that. It is also why “the write returned successfully” and “the data is safe” are different statements — the same distinction as TCP acknowledging bytes without your application having processed them.

Blocking I/O buys simplicity and charges you a worker per wait. Blocking code is straightforward to write and to reason about, and while a thread waits on `read()` it occupies 1–8 MB and one slot in your pool. Non-blocking I/O with `epoll` gets you tens of thousands of concurrent operations on one thread, and costs you a programming model where a single accidental blocking call stalls everything.

More threads stop helping and start hurting. The scheduler is fair, not free. Past roughly a few times the core count, added threads mostly add context switches and cache pressure. The right number is derived from Little’s Law and the blocking profile, not from optimism.

Containers buy isolation cheaply and share a kernel. Namespaces and cgroups give you process, network and resource isolation for the cost of a normal process — milliseconds to start, megabytes of overhead. A virtual machine gives you a separate kernel, which is a genuinely stronger boundary, for seconds of start-up and hundreds of megabytes. For running your own code, containers are right. For running untrusted code from strangers, the shared kernel is a real risk, which is why cloud providers use lightweight VMs like Firecracker underneath their container services.

I would work around the OS if... the workload is dominated by boundary crossings at a scale where microseconds matter — high-frequency trading, packet processing at line rate, a storage engine on NVMe. Then you reach for `io_uring`, `mmap`, huge pages, or user-space drivers that bypass the kernel entirely. For essentially everything else, the kernel’s defaults are the product of decades of tuning and will beat what you write.

8 In the interview

How it gets asked

- “What does the OS do when your program reads a file?” — the direct version. Walk the path.
- “What’s a system call, and why is it expensive?” — the mechanism version.
- “Why is buffered I/O faster than unbuffered?” — the applied version, and the answer is a count of crossings.
- “What’s the difference between a container and a virtual machine?” — the modern version, and the answer is “a container shares the kernel”.

What to say out loud, in the first ninety seconds

1. **Name the boundary.** “Your program can’t touch the disk. It makes a system call and the kernel does it — that’s a switch into kernel mode, which costs about a microsecond.”

2. **Say what gets checked.** *“The kernel validates the file descriptor and checks permissions first. Half of what it does is saying no.”*
3. **Give the fork in the path.** *“Then the key question: is the data already in the page cache? If yes — and it usually is — it’s a memory copy, about a microsecond in total.”*
4. **Describe the slow path.** *“If not, the kernel asks the filesystem for the blocks, the driver queues a request, and your thread is blocked and taken off the CPU. When the device interrupts, the kernel wakes your thread and copies the data in. That’s about 100 microseconds on SSD — a hundred times the fast path.”*
5. **Draw the consequence.** *“Which is why buffering matters so much. Reading a megabyte one byte at a time is a million system calls, about a second. In 4 KB blocks it’s 256 calls, under a millisecond.”*
6. **Name the other three jobs.** *“And the same boundary handles the other things the OS owns: memory through virtual addressing and page tables, CPU through the scheduler and time slices, and networking through the whole TCP stack.”*

The follow-ups

“Why is a system call expensive?” Because it is a privilege transition, not a jump. The CPU has to switch mode, save the user context, enter the kernel at a fixed entry point, validate every argument coming from untrusted user memory, do the work, then restore and switch back. That is around a microsecond, against roughly a nanosecond for an ordinary function call — a thousand to one. Modern mitigations for CPU speculation vulnerabilities made it worse, because the page tables get swapped on every crossing. It is why every performance technique in I/O is ultimately about making fewer crossings.

“Why is buffered I/O faster?” Because it amortises the crossing. Reading a megabyte one byte at a time is a million system calls at a microsecond each — about a second, almost all of it boundary crossing rather than data movement. Reading in 4 KB blocks is 256 calls and under a millisecond. The bytes moved are identical. It is the same reason batching database queries or Redis commands helps: the per-operation overhead dominates when the operations are small.

“Does write() mean the data is on disk?” No, and this catches people. `write()` normally returns once the data is in the kernel’s page cache, which is memory. If the machine loses power at that moment, the data is gone. `fsync()` is what forces it to durable storage, and that is why a database commit costs milliseconds rather than microseconds — it must `fsync` the write-ahead log before acknowledging. It is also why a naive one-`fsync`-per-transaction design caps out around a thousand transactions per second on SSD, and why group commit exists.

“What’s the difference between a container and a VM?” A container is a normal process on the host kernel with three things applied: namespaces so it sees its own process IDs, network and filesystem; cgroups to limit CPU and memory; and its own root filesystem from the image. There is no second kernel. That is why it starts in milliseconds and costs megabytes. A VM runs a full guest kernel on virtualised hardware, so it takes seconds and hundreds of megabytes, and gives you a genuinely stronger boundary — a kernel vulnerability crosses containers but not VMs. Which is why cloud providers running untrusted customer code use lightweight VMs like Firecracker underneath what looks like a container service.

A model answer

“The short version is that my program can’t read the disk at all — it asks the kernel to.

When I call `f.read(4096)`, the runtime issues a `read` system call. That’s a special instruction that switches the CPU into kernel mode and jumps to a fixed handler. The mode switch itself is around a microsecond, which sounds trivial and is about a thousand times an ordinary function call.

The kernel first validates: is this a real file descriptor for this process, and does it have permission? Then the important question — is the data already in the page cache? The kernel keeps recently read file blocks in RAM using whatever memory is free, so on a warm system most reads hit it. If it does, it’s just a copy from kernel memory into my buffer, and the whole call is a microsecond or two.

If it misses, it’s a completely different order of magnitude. The kernel asks the filesystem layer to translate the offset into disk blocks, the block layer queues a request, and my thread is marked blocked and taken off the CPU entirely — something else runs. The driver talks to the device, which DMA’s the data into kernel memory and raises an interrupt. The kernel then marks my thread runnable and copies the data into my buffer. That’s roughly 100 microseconds on SSD and 10 milliseconds on a spinning disk.

The practical consequence is buffering. A megabyte read one byte at a time is a million system calls — about a second, almost all of it crossing the boundary. In 4 KB blocks it’s 256 calls and under a millisecond, for identical data. That’s why every standard library buffers by default.

The same boundary is how the OS does its other three jobs. Memory: each process gets a virtual address space mapped to physical pages, so processes can’t see each other, and a page that isn’t resident causes a fault the kernel handles. Scheduling: more runnable threads than cores, so the scheduler hands out time slices of a few milliseconds — which is also why my code can be interrupted between any two instructions, and therefore why race conditions exist. And networking: the entire TCP stack lives in the kernel, so I hand it bytes and it handles sequence numbers, retransmission and congestion control.

One thing I’d add for a design discussion: `write()` returning doesn’t mean the data is durable — it’s in the page cache. `fsync()` is what makes it durable, and that’s the millisecond that sets the ceiling on transactions per second for any database that commits honestly.”

9 Recall card

1. **Your program cannot touch hardware.** It makes a **system call**, the CPU switches to kernel mode, the kernel checks permission and does the work. $\sim 1 \mu\text{s}$ per crossing, about $1,000 \times$ a function call.
2. **The four jobs: memory, scheduling, files and devices, networking** — plus permission checks on all of them.
3. **The page cache decides your I/O speed.** A cached read is $\sim 1 \mu\text{s}$; an SSD read is $\sim 100 \mu\text{s}$; a disk read is $\sim 10 \text{ ms}$. Most reads hit the cache.
4. **Buffering is about crossing less.** 1 MB one byte at a time is a million syscalls and a second; in 4 KB blocks it is 256 syscalls and a millisecond.
5. `write()` **is not durable** — `fsync()` **is**. That is why a commit costs milliseconds, and why one fsync per transaction caps you near 1,000 per second on SSD.

DSA topic: Insert, delete, and the cost of the middle **System design topic:** The operating system's job

Code these, in this order

Four problems that are all deletion in disguise. Every one of them has a natural solution that calls `remove` or `pop(i)` inside a loop and is quadratic, and a one-pass solution that is linear.

For each problem:

1. Say how many elements move for a single deletion at position `i`.
2. Say what happens if you do that `k` times.
3. Ask the two questions that change the answer: **does order matter**, and **may I modify the input?**
4. Then write the one-pass version.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Remove Element	LeetCode 27 (Easy)	Order does not matter here, which the statement says explicitly. So both the write-pointer solution and the swap-with-last solution are valid — write both and compare.
2	Remove Duplicates from Sorted Array	LeetCode 26 (Easy)	Order does matter, so swap-with-last is out. Pure write pointer, and the return value is a length rather than a list.
3	Remove Duplicates from Sorted Array II	LeetCode 80 (Medium)	The same pattern with a count. The whole difficulty is deciding what the write pointer compares against — and the answer is <code>items[write - 2]</code> , not the previous element.
4	Merge Sorted Array	LeetCode 88 (Easy)	The direction trap from §7. Merging forwards overwrites data you still need; merging from the end backwards does not. This is the same rule as shifting right.

On problem 1, do this properly

- Write the version that calls `items.remove(val)` in a `while` loop. Time it on 50,000 elements where every element is the target.
- Write the write-pointer version. Time it.
- Write the swap-with-last version. Time it.

- Then answer: which two produce the same array, and which one is only valid because the problem says order does not matter?

The cost drill

Answer these six from memory, with the arithmetic, in under ninety seconds:

- Delete position 3 from an array of 20. How many elements move?
- Delete the last element of an array of 20. How many move?
- Insert at position 0 of an array of 20. How many move?
- Delete 5,000 elements one at a time from a list of 20,000. Roughly how many operations?
- Do the same with a write pointer. How many?
- When is deletion `O(1)`, and what is the condition?

The direction drill

Type this and predict the output before running it:

```
row = ["A", "B", "C", "D", "E"]
row.append(None)
for j in range(1, len(row) - 1):
    row[j + 1] = row[j]
row[1] = "X"
print(row)
```

Then explain in one sentence why `C`, `D` and `E` disappeared, and rewrite the loop so it works. Then state the general rule about which end to start from.

The one to try in a terminal

```
strace -c python3 -c "open('/etc/hostname').read()"
```

(On macOS, `dtruss`; on Windows, run it in WSL.) Look at the list of system calls your three-word program actually made. Count them. Then run it with the file read one byte at a time and count again.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is the cost of deleting an element from the middle of an array?* Give the exact count first, then the range from front to back, then why the hole cannot stay, then the `O(1)` version and its condition.
 2. *What does the operating system do when your program asks to read a file?* Walk the path: syscall, mode switch, permission check, page cache hit or miss, blocking, interrupt, copy, return. Then give the buffering consequence.
 3. *Why is buffered I/O faster than unbuffered?* Do the multiplication out loud for a 1 MB file at one byte and at 4 KB, and say what the ratio between a system call and a function call is.
-

Before you move on

- [] I can say `n - i - 1` and `n - i` and explain which is which.
- [] I ask “does order matter?” before writing any deletion code.
- [] I never call `remove` or `pop(i)` inside a loop over the same list.
- [] I know which direction to shift when inserting, and why the other direction corrupts data.
- [] I can name the four jobs of the operating system and say what a system call costs.

Searching an array: linear search, done properly

DATA STRUCTURES AND ALGORITHMS

Searching an array: linear search, done properly

You can write a search that handles not-found, duplicates and empty input correctly.

SYSTEM DESIGN

How your code becomes a running service

You can describe the path from a source file to a process serving live traffic.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the index of a target. What do you return if it is not there?*
- *How does the code you wrote end up running on a server?*

Searching an array: linear search, done properly

1 What this is, and why they ask it

Linear search is looking at each element in turn until you find what you want or run out. It is four lines of code, it works on any array in any order, and it is $O(n)$.

The four lines are not the point. The point is everything around them: what you return when the value is absent, which position you return when the value appears three times, what happens on an empty array, and why “I return 0” is a bug rather than a choice.

Interviewers ask this as an opener, and they ask the second half — “*what do you return if it is not there?*” — because it is the smallest possible test of whether you think about contracts. A candidate who writes the loop and stops has written half a function. A candidate who says “I’d return -1 by convention, but I’d check with you, because if the array can contain -1 as a value that gets confusing, and in Python I’d prefer `None`” has demonstrated the habit from [day 008](#) on a problem small enough that there is nowhere to hide.

2 The story

Ilyas is on the gate at a hospital car park in Kozhikode. Six rows, thirty spaces to a row, and by eleven in the morning on a weekday every one of them is taken.

At about half past eleven a woman comes to his cabin. Her father is being discharged, her brother parked the car at seven and has now gone to fetch the medicines, and she has no idea where in the car park it is. She gives him the number plate and the fact that it is a grey Ertiga.

So Ilyas walks. He starts at the first space in row A and goes along, looking at each plate. Grey Ertigas are common enough that he checks the whole number, not just the first few characters, because there is a grey Ertiga in row B whose plate differs from hers in one digit and he has been caught by that before. He finds it in row C, the fourteenth space — about the seventieth car he has looked at. Eight minutes.

The other kind of afternoon is the one that sticks with him. Three weeks ago a man asked him to find a car and Ilyas walked the whole thing — all six rows, all hundred and eighty spaces — and it was not there. It had been towed at nine that morning from out-

side the blood bank. And the thing about that walk is that it was the **longest possible** walk. Finding a car can be quick if it happens to be near the entrance. Being certain a car is *not* there takes every single space, every single time. There is no shortcut for absence.

Then there is what he actually says when he gets back. He cannot say “row A, space one”, because there is a row A and there is a space one and she would go and stand there. He has to say something that is not a place at all — “it’s not in the car park” — and that is a different kind of answer to the question he was asked.

Not every question wants one car, either. The Sunday before, a woman from the ward asked him how many white Swifts were in the car park, because someone had left their lights on and she wanted to warn them. That is not “find me the car”. That is “find me all of them”, and he came back with nine spaces rather than one. Same walk, different answer.

And at four in the morning, when the last relative has gone home and the car park is empty, somebody occasionally asks him about a car anyway. That walk takes no time at all, and the answer is still “it’s not here”.

3 The idea in plain English

Ilyas walking the rows is linear search, and the four things he ran into are the four things the code has to get right.

The search itself

```
def find(items: list[int], target: int) → int:
    for i in range(len(items)):
        if items[i] == target:
            return i
    return -1
```

Walk from the start. Compare each element to the target. **Return the moment you find it** — that is the early exit, and it is what makes the best case one step. If the loop finishes without returning, the value is not there.

That last `return -1` is outside the loop. Getting it inside the loop is a real and common bug, and it makes the function report “not found” after checking only the first element.

What “not found” should be

Ilyas cannot say “row A, space one”. You cannot return `0`, because `0` is a perfectly good index — it means “the first element”. Something that is not an index is needed.

CONVENTION	WHEN TO USE IT
<code>-1</code>	The universal convention across C, Java, JavaScript, and most interview answers. Not a valid index, so unambiguous.
<code>None</code>	The Pythonic choice, and unambiguous even if <code>-1</code> appears in the data.
Raise an exception	What <code>list.index()</code> does. Right when absence is genuinely exceptional; wrong when it is expected.

`-1` is the safe answer in an interview, and asking is the better one. “I’ll return `-1` unless you’d prefer `None` — is not-found an expected case or an error case?” takes four seconds and is exactly the behaviour [day 008](#) described.

There is one genuine trap in `-1`, and it is worth naming: **in Python, `-1` is a valid index** that means the last element. So a caller that does `items[find(items, x)]` without checking will silently read the last element when the value is absent, instead of failing. That alone is a decent argument for `None` in Python.

Duplicates: which one do you want?

Ilyas found one grey Ertiga; the woman on Sunday wanted all nine white Swifts. Same walk, three possible contracts:

First occurrence — return as soon as you match. This is the default and the cheapest, because it exits early.

Last occurrence — either walk backwards, or walk forwards and keep overwriting a remembered index. Walking backwards can exit early; walking forwards cannot.

All occurrences — collect them into a list. No early exit is possible, so this is always `n` comparisons.

The problem statement usually does not say which. **Ask.**

The empty array

`range(0)` produces nothing, the loop body never runs, and the function falls through to `return -1`. Correct, with no special case needed — which is what a well-derived bound looks like, from [day 010](#).

That is worth checking rather than assuming, because the near-miss versions do need a guard. A function that starts with `best = items[0]` raises `IndexError` on an empty list, and a function that returns `items[found]` at the end has the same problem.

What it costs, and why absence is the worst case

CASE	COMPARISONS	WHEN
Best	1	the target is first
Average (present)	$n / 2$	uniformly likely positions
Worst	n	the target is last, or not there at all

Absence is always the worst case. There is no way to be sure something is missing without looking everywhere. That single sentence answers a surprising number of follow-up questions, and it is why the “not found” path is the one worth reasoning about.

The complexity is $O(n)$ time and $O(1)$ extra space.

When linear search is the right answer

It is not a placeholder for something better. It is correct when:

- **The data is unsorted** and you are searching once. Sorting to enable binary search costs $O(n \log n)$, which is worse than a single $O(n)$ scan.
- **n is small.** Below a few dozen elements, a linear scan often beats anything cleverer because of constant factors and cache behaviour.
- **You are searching by a condition**, not by equality — “the first element greater than the running average” cannot be binary-searched.
- **The data is a linked list or a stream**, where random access does not exist.

And it is the wrong answer when you will search **many times** over the same data. Then you pay once to prepare and search cheaply thereafter:

```
search once      : linear  $O(n)$  wins
search k times   : sort  $O(n \log n)$  +  $k \times O(\log n)$     -- day 042
                   or build a hash set  $O(n)$  +  $k \times O(1)$  -- day 060
```

The break-even is done with real numbers in §6.

The built-ins, and what they cost

```
target in items      #  $O(n)$ , returns True/False
items.index(target)   #  $O(n)$ , returns the index, RAISES if absent
items.count(target)   #  $O(n)$ , counts all occurrences
```

All three are linear search written in C, so they run several times faster than your Python loop and have exactly the same complexity. `items.index()` is the one to be careful with, because its failure mode is an exception rather than a value — §7 shows what that looks like.

4 The picture

The walk, on a seven-element array, looking for 13:

index	0	1	2	3	4	5	6
value	2	9	5	13	7	13	4

i=0: 2 = 13? no
i=1: 9 = 13? no
i=2: 5 = 13? no
i=3: 13 = 13? YES → return 3

4 comparisons. Stops immediately. Never sees index 4, 5 or 6.

What to notice: the loop stops at the first match, so the second 13 at index 5 is never examined. That is correct for “first occurrence” and wrong for “all occurrences”, and nothing in the code says which one you meant.

Now the same array, looking for 8:

index	0	1	2	3	4	5	6
value	2	9	5	13	7	13	4

no no no no no no no

7 comparisons, then return -1.
To be sure it is ABSENT you must look at every single one.

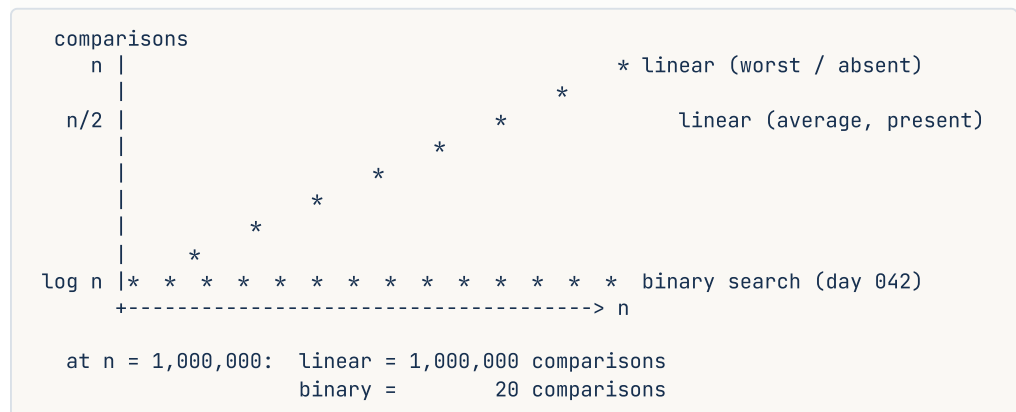
What to notice: absence costs the full `n`. There is no early exit for “not there”, and there never can be without extra structure.

The three contracts, side by side:

value	2	9	5	13	7	13	4	target = 13
index	0	1	2	3	4	5	6	
first	----->							returns 3, 4 comparisons
last				----->				returns 5, 7 comparisons forwards (or 2 backwards)
all	----->							returns [3, 5], 7 comparisons

What to notice: three different correct answers to “find 13”. The problem statement decides which, and if it does not say, you ask.

And the cost curve, which is the thing to have in mind when comparing with binary search:



What to notice: the gap is enormous and it costs $O(n \log n)$ to unlock. That is why “should I sort first?” is a question about **how many times you will search**, not about how big the array is.

5 The code, built step by step

Build the four contracts, then a table of edge cases, because the edge cases are the lesson.

The basic search, returning `-1`.

```
def find_first(items: list[int], target: int) → int:
    for i, x in enumerate(items):
        if x == target:
            return i
    return -1
```

`enumerate` rather than `range(len(items))`, from [day 010](#) — shorter, faster, and one fewer thing to get wrong. The `return -1` sits **outside** the loop, which is the whole correctness of the function.

The Pythonic version, returning `None`.

```
def find_first_or_none(items: list[int], target: int) → int | None:
    for i, x in enumerate(items):
        if x == target:
            return i
    return None
```

`int | None` in the signature is the honest type. A caller now has to check before using the result, which is exactly what you want, and `None` cannot be mistaken for a valid index the way `-1` can in Python.

The last occurrence, walking backwards so it can still exit early.

```
def find_last(items: list[int], target: int) → int:
    for i in range(len(items) - 1, -1, -1):
        if items[i] == target:
            return i
    return -1
```

`range(n - 1, -1, -1)` — stop-before `-1`, so index 0 is included. Getting that second argument wrong silently misses the first element.

All occurrences, which cannot exit early.

```
def find_all(items: list[int], target: int) → list[int]:
    return [i for i, x in enumerate(items) if x == target]
```

Always `n` comparisons, and `O(k)` extra space for the `k` matches.

Search by a condition rather than by equality — the version binary search cannot do.

```
def find_where(items: list[int], predicate) → int:
    for i, x in enumerate(items):
        if predicate(x):
            return i
    return -1
```

`predicate` is a function returning `True` or `False`. `find_where(items, lambda x: x > 100)` finds the first element over 100. This generality is a real reason linear search survives.

And the built-in, with its failure mode made safe.

```
def find_with_builtin(items: list[int], target: int) → int:
    try:
        return items.index(target)
    except ValueError:
        return -1
```

`items.index()` is the same `O(n)` scan implemented in C, so it is much faster in practice. It raises when absent, so it must be wrapped if absence is expected.

Here is the complete program.

```

"""Day 12 – linear search, and the four things around it that actually matter."""

import time
from collections.abc import Callable

def find_first(items: list[int], target: int) → int:
    """First occurrence, or -1. O(n) time, O(1) space."""
    for i, x in enumerate(items):
        if x == target:
            return i
    return -1
    # OUTSIDE the loop. This is the whole function.

def find_first_or_none(items: list[int], target: int) → int | None:
    """The Pythonic contract: None cannot be mistaken for an index."""
    for i, x in enumerate(items):
        if x == target:
            return i
    return None

def find_last(items: list[int], target: int) → int:
    """Last occurrence. Backwards, so it can still exit early."""
    for i in range(len(items) - 1, -1, -1):
        if items[i] == target:
            return i
    return -1

def find_all(items: list[int], target: int) → list[int]:
    """Every occurrence. No early exit is possible: always n comparisons."""
    return [i for i, x in enumerate(items) if x == target]

def find_where(items: list[int], predicate: Callable[[int], bool]) → int:
    """Search by condition – the case binary search cannot handle."""
    for i, x in enumerate(items):
        if predicate(x):
            return i
    return -1

def find_with_builtin(items: list[int], target: int) → int:
    """Same O(n), written in C, so roughly 30x faster in practice."""
    try:
        return items.index(target)
    except ValueError:
        return -1

def count_comparisons(items: list[int], target: int) → int:
    """How many elements did we actually look at?"""
    looked = 0
    for x in items:
        looked += 1
        if x == target:
            return looked
    return looked

# The edge cases. Written before the solution, not after it.
CASES: list[tuple[str, list[int], int]] = [
    ("empty array", [], 5),
    ("one element, match", [5], 5),
    ("one element, no match", [9], 5),
    ("target is first", [5, 1, 2, 3], 5),
    ("target is last", [1, 2, 3, 5], 5),
    ("target absent", [1, 2, 3, 4], 5),
    ("duplicates", [1, 5, 2, 5, 3, 5], 5),
    ("all identical", [5, 5, 5, 5], 5),
]

```



```

("negative target",      [1, -5, 2],      -5),
("target is zero",      [3, 0, 7],      0),
("minus one in data",   [3, -1, 7],     -1),
]

if __name__ == "__main__":
    print(f"{'case':<24}{'array':<20}{'target':>7}{'first':>7}{'last':>7}"
          f"{'all':>10}{'looked':>8}")
    print("-" * 83)
    for name, items, target in CASES:
        print(f"{'name':<24}{'str(items):<20'}{'target':>7}"
              f"{'find_first(items, target):>7}"
              f"{'find_last(items, target):>7}"
              f"{'str(find_all(items, target)):>10}"
              f"{'count_comparisons(items, target):>8}")

    print("\nwhy 'not found' is the worst case")
    n = 1_000_000
    data = list(range(n))
    for label, target in (("first element", 0),
                          ("middle element", n // 2),
                          ("last element", n - 1),
                          ("absent", -99)):
        print(f" {label:<16}: {count_comparisons(data, target):>10,} comparisons")

    print("\nsearch by condition (binary search cannot do this)")
    readings = [12, 45, 7, 99, 23, 81, 64]
    i = find_where(readings, lambda x: x > 50)
    print(f" first reading over 50 in {readings} → index {i}, value {readings[i]}")

    print("\nyour loop vs the built-in – same O(n), different constant")
    t0 = time.perf_counter(); find_first(data, -99); mine = time.perf_counter() - t0
    t0 = time.perf_counter(); find_with_builtin(data, -99); builtin = time.perf_counter() - t0
    print(f" find_first (Python loop) : {mine:>8.4f} s")
    print(f" list.index (C) : {builtin:>8.4f} s → {mine / builtin:.0f}x faster")

    print("\nis it worth sorting first? (day 042 territory)")
    for searches in (1, 10, 1_000, 100_000):
        linear = searches * n
        prepared = n * 20 + searches * 20 # n log n to sort, log n per search
        winner = "linear" if linear < prepared else "sort + binary search"
        print(f" {searches:>7,} searches over {n:,}: "
              f"linear {linear:>14,} vs sorted {prepared:>14,} → {winner}")

```

This is exactly what it printed:

case	array	target	first	last	all	looked
empty array	[]	5	-1	-1	[]	0
one element, match	[5]	5	0	0	[0]	1
one element, no match	[9]	5	-1	-1	[]	1
target is first	[5, 1, 2, 3]	5	0	0	[0]	1
target is last	[1, 2, 3, 5]	5	3	3	[3]	4
target absent	[1, 2, 3, 4]	5	-1	-1	[]	4
duplicates	[1, 5, 2, 5, 3, 5]	5	1	5	[1, 3, 5]	2
all identical	[5, 5, 5, 5]	5	0	3	[0, 1, 2, 3]	1
negative target	[1, -5, 2]	-5	1	1	[1]	2
target is zero	[3, 0, 7]	0	1	1	[1]	2
minus one in data	[3, -1, 7]	-1	1	1	[1]	2

why 'not found' is the worst case

first element	:	1 comparisons
middle element	:	500,001 comparisons
last element	:	1,000,000 comparisons
absent	:	1,000,000 comparisons

search by condition (binary search cannot do this)

first reading over 50 in [12, 45, 7, 99, 23, 81, 64] → index 3, value 99

your loop vs the built-in – same $O(n)$, different constant

find_first (Python loop)	:	0.1342 s
list.index (C)	:	0.0226 s → 6x faster

is it worth sorting first? (day 042 territory)

1 searches over 1,000,000: linear	1,000,000 vs sorted	20,000,020 → linear
10 searches over 1,000,000: linear	10,000,000 vs sorted	20,000,200 → linear
1,000 searches over 1,000,000: linear	1,000,000,000 vs sorted	20,020,000 → sort + binary search
100,000 searches over 1,000,000: linear	100,000,000,000 vs sorted	22,000,000 → sort + binary search

Look at the row `minus one in data`. The array contains `-1` as a value, and the search for it correctly returns index 1. But now imagine `find_first` had returned `-1` for “not found” on a different call — a caller comparing the two results cannot tell them apart by looking at the data. That is exactly why the return convention is a question, not an assumption.

Look at the comparison block. Absent costs the same as last: the full million. There is no version of this that is faster, and that fact is the reason the whole binary search phase exists.

Look at the last block. One search: scan it. A thousand searches: sort first. The crossover is not about `n`, it is about how many times you ask.

6 What it costs

Time. One comparison per element, up to `n`:

best case	:	1	target is first
average	:	$n / 2$	target present, position uniformly likely
worst case	:	n	target last, OR ABSENT

`O(n)` in all the ways that matter, and `O(1)` extra space — three variables regardless of size.

Why the average is $n/2$ when found. If the target is equally likely to be at any of the n positions:

$$(1 + 2 + 3 + \dots + n) / n = (n(n+1)/2) / n = (n+1)/2$$

Just over half, which is the staircase from [day 002](#) again. And the constant $1/2$ is dropped, so it is still $O(n)$ — which is why an interviewer will not accept “it’s $O(n/2)$ ”.

Absence has no average. It is always n . Any workload that mostly asks about things that are not there gets the worst case every single time, and that is a genuinely useful thing to notice about a real system: a cache lookup that usually misses is paying full price on every call.

The break-even against sorting, done properly. Searching k times over an array of $n = 1,000,000$, where $\log_2 n \approx 20$:

linear	:	$k \times n$	=	$k \times 1,000,000$
sort + binary	:	$n \log n + k \log n$	=	$20,000,000 + k \times 20$

Set them equal:

$k \times 1,000,000$	=	$20,000,000 + 20k$
$k \times 999,980$	=	$20,000,000$
k	=	about 20

Twenty searches. Below that, scan. Above it, sort once. Now the same with a hash set, which is $O(n)$ to build and $O(1)$ per lookup:

hash set	:	$n + k \times 1 = 1,000,000 + k$
linear	:	$k \times 1,000,000$
break-even at $k = \text{about } 2$		

Two searches. If you are going to look twice and you can afford $O(n)$ memory, build a set. That is the day 006 reflex, and this is the arithmetic behind it.

The constant factor of the built-in. `list.index()` is the same algorithm in C:

Python loop	:	0.1362 s over 1,000,000	
list.index	:	0.0220 s over 1,000,000	→ 6x

Same $O(n)$, six times faster here and often far more on tighter loops. It never changes which shape you should choose, and it is free, so use it when the contract fits.

Cache behaviour. Linear search reads memory in address order, so it gets the full benefit of the 64-byte cache line from [day 009](#) — roughly sixteen 4-byte values per fetch. This is why, for small `n`, a linear scan of a contiguous array frequently beats a binary search that jumps around, and why real sorting implementations switch to insertion sort below about 64 elements.

7 The traps

Trap one: index 0 is falsy

This one produces a wrong answer with no error at all, and it catches experienced people.

```
def find(items: list[int], target: int) → int | None:
    for i, x in enumerate(items):
        if x == target:
            return i
    return None

items = [7, 3, 9]
result = find(items, 7)
if result:
    print(f"found at {result}")
else:
    print("not found")
```

```
not found
```

The value `is` at index 0, and the function returned `0` correctly. Then `if result:` treated `0` as false, because in Python `0`, `None`, `""`, `[]` and `{}` are all falsy. The check cannot distinguish “found at position 0” from “not found”.

The fix is to test against the sentinel explicitly rather than for truthiness:

```
if result is not None:
    print(f"found at {result}")
```

And with the `-1` convention:

```
if result != -1:
    # not `if result:`
```

How to catch it every time: when a function can return `0`, `False` or an empty container as a legitimate value, never test the result for truthiness. Test `is not None`, or compare to the sentinel. This is one of the most common bug classes in Python and it is

invisible in code review unless you are looking for it.

Trap two: `.index()` raises

The built-in has a failure mode that a hand-written loop does not.

```
items = [1, 2, 3, 4]
position = items.index(7)
print(position)
```

```
Traceback (most recent call last):
  File "d12.py", line 2, in <module>
    position = items.index(7)
                  ~~~~~^
ValueError: 7 is not in list
```

`ValueError`, not `IndexError` — the message even quotes the value it was looking for. That is useful for debugging and fatal in production if you did not expect it.

Two correct fixes. Wrap it:

```
try:
    position = items.index(7)
except ValueError:
    position = -1
```

Or check first — noting that this costs **two** passes, because `in` is itself an `O(n)` scan:

```
position = items.index(7) if 7 in items else -1    # O(n) + O(n)
```

For a single lookup the hand-written loop is one pass and is the better answer. For repeated lookups, neither is right — build a dictionary from value to index once, and look up in `O(1)`.

The near-miss worth naming

Putting the `return` in the wrong place:

```
def find(items: list[int], target: int) → int:
    for i, x in enumerate(items):
        if x == target:
            return i
    return -1                # ← indented one level too far
```

```
print(find([7, 3, 9], 3))    # prints -1. The 3 is at index 1.
```

No error. The function checks element 0, does not match, and returns `-1` immediately. It is correct whenever the target happens to be first, which means it passes the example the interviewer showed you.

The `return` for “not found” belongs after the loop has finished, at the function’s indentation level. When reading a search function, check that line’s indentation first — it is where the bug lives.

8 In the interview

How it gets asked

- “Find the index of a target. What do you return if it isn’t there?” — the direct version. The second sentence is the real question.
- “What if the target appears multiple times?” — the contract question. First, last, or all?
- “Can you do better than $O(n)$?” — the answer is “not on unsorted data with one search”, and saying why is the point.
- “Should you sort first?” — a question about how many searches, not about n .

What to say out loud, in the first ninety seconds

1. **Ask the contract question before writing.** “Before I write it — what should I return if the target isn’t present? I’d default to `-1`, or `None` in Python. And if it appears more than once, do you want the first, the last, or all of them?”
2. **State the approach.** “Assuming first occurrence and `-1` for absent: one pass, comparing each element, returning the index on the first match.”
3. **Say where the fallback goes.** “The `-1` goes after the loop, not inside it — inside, it would bail out after checking the first element.”
4. **Give the three cases.** “Best case one comparison if it’s first. Average n over 2 if it’s present. Worst case n — and importantly, ‘not present’ is always the worst case, because you can’t be sure something’s absent without looking at everything.”
5. **Give both complexities.** “ $O(n)$ time, $O(1)$ extra space. Empty array falls out correctly with no special case, because the loop just doesn’t run.”
6. **Pre-empt “can you do better”.** “If I’m going to search this array many times, I’d pay once up front — a hash set makes each lookup $O(1)$, or sorting makes it $O(\log n)$. The break-even against a hash set is about two searches; against sorting it’s around twenty at a million elements. For a single search on unsorted data, $O(n)$ is optimal — you have to look at every element to rule it out.”

Step 6 is what turns the simplest question in the course into a conversation worth having.

The follow-ups

“What do you return if it’s not there?” `-1` by convention — it is not a valid index in most languages, so it is unambiguous, and it is what `indexOf` returns in Java and JavaScript. In Python I would lean towards `None`, for a specific reason: `-1` is a valid index in Python, meaning the last element, so a caller who forgets to check gets the last element instead of an error. The third option is raising, which is what `list.index()` does, and I would only choose it if absence is genuinely exceptional rather than expected. Whichever I pick, I would document it and be consistent, and I would ask rather than assume.

“Can you do better than $O(n)$?” Not for a single search on unsorted data, and the reason is information-theoretic rather than about cleverness: any element you have not examined could be the target, so to correctly return “not found” you must examine all of them. What you can do is change the problem. If I will search repeatedly, I pay $O(n)$ once to build a hash set and then each lookup is $O(1)$, or $O(n \log n)$ to sort and then $O(\log n)$ per lookup. The question is how many searches, not how big the array is — the break-even against a hash set is about two searches.

“The array is sorted. Does that change anything?” Yes, twice over. Binary search becomes available, taking it to $O(\log n)$ — 20 comparisons instead of a million. And even linear search improves for the absent case: you can stop as soon as you pass a value greater than the target, because everything after it is larger too. That takes the average for absent targets from n to $n/2$ while staying $O(n)$. But if the array is sorted, binary search is the answer, and that is [day 042](#).

“When would you actually use linear search?” More often than people expect. When the data is unsorted and I am searching once — sorting first would cost more than the scan. When n is small, where constant factors and cache locality mean a linear scan of contiguous memory beats a binary search that jumps around; real sort implementations switch to insertion sort below about 64 elements for this reason. When I am searching by a **condition** rather than by equality — “the first element greater than the running average” cannot be binary-searched at all. And when the structure has no random access, like a linked list or a stream.

A model answer

“Before I write it, two things. What should I return when the target isn’t present? And if it appears more than once, do you want the first occurrence, the last, or all of them?

...Right, first occurrence and `-1` for not found.

```
def find(items: list[int], target: int) → int:
    for i, x in enumerate(items):
        if x == target:
            return i
    return -1
```

One pass, returning the index at the first match. The `return -1` is after the loop, at the function’s indentation — if it were inside the loop, it would bail out after checking only the first element, and that version still passes any example where the target happens to be first, so it’s an easy bug to miss.

Complexity: $O(n)$ time, $O(1)$ extra space. Best case one comparison, average n over 2 when present. Worst case n — and the case worth flagging is that ‘not present’ is *always* the worst case. You can’t be certain something is absent without looking at everything, so a workload that mostly asks about missing keys pays full price on every call.

The empty array works with no special case: the loop body never runs and it falls through to `-1`.

One note on the `-1` convention in Python specifically: `-1` is a valid index here, meaning the last element. So a caller who does `items[find(items, x)]` without checking gets the last element rather than an error. In Python I’d usually return `None` for that reason, and I’d put `int | None` in the signature so the type system makes the caller check.

On doing better — not for a single search on unsorted data, because any element you haven’t examined could be the target. But if we’re searching repeatedly, I’d pay once up front: a hash set gives $O(1)$ lookups after an $O(n)$ build, which pays for itself after about two searches. Sorting gives $O(\log n)$ lookups after $O(n \log n)$, which at a million elements pays for itself after about twenty. So the question is how many times we search, not how big the array is.”

That answer opens with the contract question, gets the fallback placement right and explains why it matters, gives all three cases, names the Python-specific hazard, and finishes with the break-even arithmetic rather than a vague “use a hash map”.

9 Recall card

1. **Walk, compare, return on the first match, and put the “not found” return AFTER the loop.** Inside the loop it bails out after one element.
2. **-1 by convention, None in Python** — because `-1` is a valid index in Python and will silently read the last element. Ask which the interviewer wants.
3. **Absence is always the worst case: n comparisons.** You cannot be sure something is missing without looking everywhere. Best 1, average $n/2$, worst n . $O(n)$ time, $O(1)$ space.
4. **Ask about duplicates.** First, last, or all — three different correct answers, and only “all” loses the early exit.
5. **Never test the result for truthiness.** `if result:` is false for index 0. Use `if result is not None:` or compare against the sentinel.

1 What this is, and why they ask it

There is a chain between the file you edited and the process answering a stranger's request, and it has about eight links: version control, automated checks, a build that produces one fixed artefact, a place to store that artefact, a deployment that starts it somewhere, a health check that decides whether it is ready, a load balancer that starts sending it traffic, and a way to undo all of it in under a minute.

Every link exists because something went wrong without it.

Interviewers ask because it is the question that separates people who have shipped from people who have only built. It comes up in almost every system design round as a closing topic, and in every interview for anything above junior. The specific things they are listening for are **reproducibility** ("it works on my machine" is a failure, not a joke), **health checks** (traffic must not arrive before the process is ready), **rollback** (the recovery plan is worth more than the deployment plan), and **database migrations** (the part that cannot be rolled back, and therefore the part that needs thought).

2 The story

Farhana has been making cakes at home in Thrissur for six years, mostly for family functions and the occasional order from a neighbour. In January the bakery near the junction asked whether she would supply them a plum cake, because a customer had had one at a wedding and asked where it came from.

She said yes, and then found that the arrangement was not what she expected at all.

The first problem was that her cakes were not the same twice. She knew this and had never minded. Her oven runs hot on the left, she judges the sugar by eye, and a cake in December is slightly different from a cake in June. That is fine for a family function. The bakery could not have it, because a customer who liked Tuesday's cake will be back on Friday and will not accept an explanation.

So the shop's supervisor did something that annoyed her for about a week and that she now thinks was obviously right. He gave her a tin of one exact size, a set of weights, a method written out step by step, and the use of the bakery's own oven at a fixed temperature. Nothing was left to judgement. Every batch after that came out the same, and the reason was that nothing about her kitchen was involved any more.

The second thing is that a batch does not go on the shelf because she says it is good. It goes to a man at the back who cuts a slice from one cake in every tray and tastes it. He has sent back three batches in eight months. The third one she was sure was fine, and he was right and she was wrong.

The third thing surprised her most. On the first day, they did not put out sixty cakes. They put out six, on one shelf, and watched what happened for a day. If nobody had bought them, or if two people had complained, they would have stopped there and it would have cost the shop almost nothing.

And on a Thursday in April a batch came out wrong — too dry, and she still does not know exactly why. Somebody noticed at about eleven. By twenty past, every one of those cakes was off the shelf and the previous week's recipe was back out, because the shop keeps the previous batch's tins and method for exactly this reason. They lost a morning. They did not lose the customer.

The shop opens at seven every day whether or not anything sold yesterday, and somebody has to be there at half past six to put things out. Farhana had not thought about that part at all before January.

3 The idea in plain English

Farhana's arrangement with the bakery is a deployment pipeline. Every step maps directly.

The chain, link by link

1. Version control. Your code lives in **git**, on a branch. A change becomes a **pull request**, which is where another person reads it before it can go anywhere. This is the written-out method: the recipe is written down, not in somebody's head, and there is a record of every change to it.

2. Continuous integration. When you open the pull request, a machine automatically runs the tests, the linter and the type checker. That is the man at the back tasting a slice from every tray. **CI** stands for continuous integration, and its whole job is to say no before a human has to.

3. The build, which produces one fixed artefact. This is the tin, the weights and the bakery's oven. The output is one immutable **artefact** — most commonly a **container image** — containing your code, its dependencies, and the exact runtime version, built the same way every time.

This is the answer to “it works on my machine”. Nothing about your machine is involved. The same image runs on your laptop, in staging and in production, and if it behaves differently in one of them, the difference is configuration or data, not environment.

4. A registry. The artefact is pushed to a store — **Docker Hub, Amazon ECR, Google Artifact Registry** — and tagged, usually with the git commit hash. The tag matters: it means any running process can be traced back to the exact line of code inside it.

5. Configuration, injected at run time. The image is the same everywhere; what differs is the database address, the API keys, the feature flags. Those arrive as **environment variables** or mounted files, never baked into the image. Secrets come from a proper store — **AWS Secrets Manager, HashiCorp Vault, Kubernetes Secrets**.

The rule underneath this is from the **Twelve-Factor App**, a widely used checklist: **strict separation of config from code**. If you have to rebuild to point at a different database, you have built the wrong thing.

6. Deployment. Something starts the artefact on real machines. That something is an **orchestrator** — **Kubernetes, AWS ECS, Nomad** — or, on a smaller system, **systemd** on a virtual machine. It decides how many copies run, on which machines, and restarts them when they die.

7. Health checks, then traffic. This is the link people forget. A newly started process is not immediately able to serve — it has to connect to the database, warm its connection pool, maybe load a model into memory. So it exposes a **readiness** endpoint, usually `/healthz`, and **the load balancer sends it no traffic until that endpoint says yes**.

There is a second one, **liveness**, which asks a different question: not “are you ready?” but “are you still alive?”. If liveness fails, the orchestrator kills and restarts the process.

8. Rolling out gradually, and rolling back fast. Six cakes, not sixty.

- **Rolling deployment:** replace instances a few at a time, so old and new run side by side.
- **Blue-green:** run the whole new version alongside the old, then switch traffic in one move. Rollback is switching back.
- **Canary:** send 1% of traffic to the new version, watch the error rate, then 10%, then 100%.

And the part that matters more than any of them: **the old artefact is still in the registry**. Rolling back is starting the previous image, which takes a minute or two, not rebuilding or reverting code under pressure. The bakery kept last week’s tins.

The link that cannot be rolled back

Everything above is reversible because artefacts are immutable and the old one still exists.

Database migrations are not. If your deployment adds a column, dropping the new version does not remove the column. If it *renames* a column, the old code no longer works at all, so you cannot roll back even though you thought you could.

The standard discipline is **backwards-compatible migrations in two phases**:

- To add a column: deploy the migration first, then the code that uses it.
- To remove one: deploy code that stops using it, wait until you are confident, then drop it in a later release.
- To rename one: never rename. Add the new, write to both, backfill, move reads over, then drop the old — four deployments.

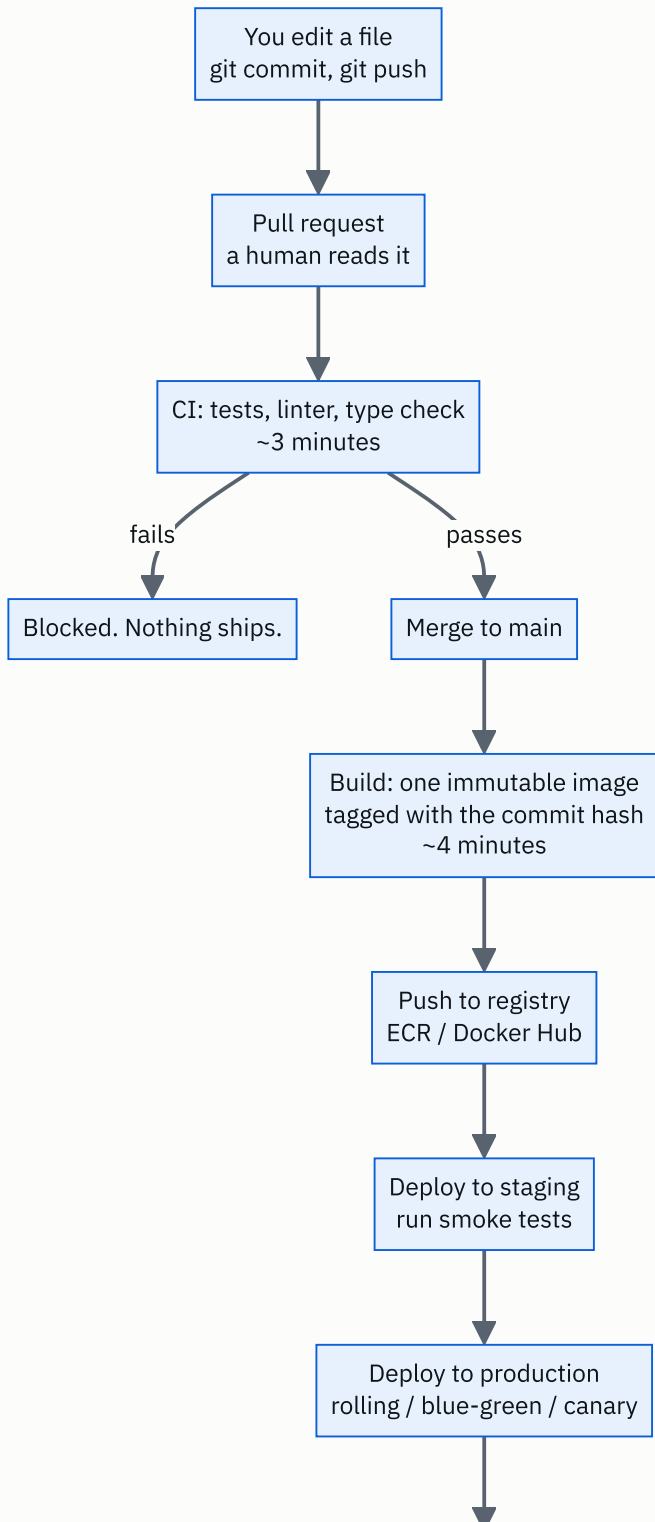
Being able to say this is one of the strongest signals available in this whole question, because it is knowledge that only comes from having been on the wrong side of it.

The shop opens at seven

A service is not a program you run. It is a process that must be running **continuously**, be restarted when it dies, be started when the machine reboots, and be replaced without a gap in service. That is what an orchestrator or `systemd` is for, and it is the difference between “my code ran” and “my code is a service”.

4 The picture

The chain, end to end:



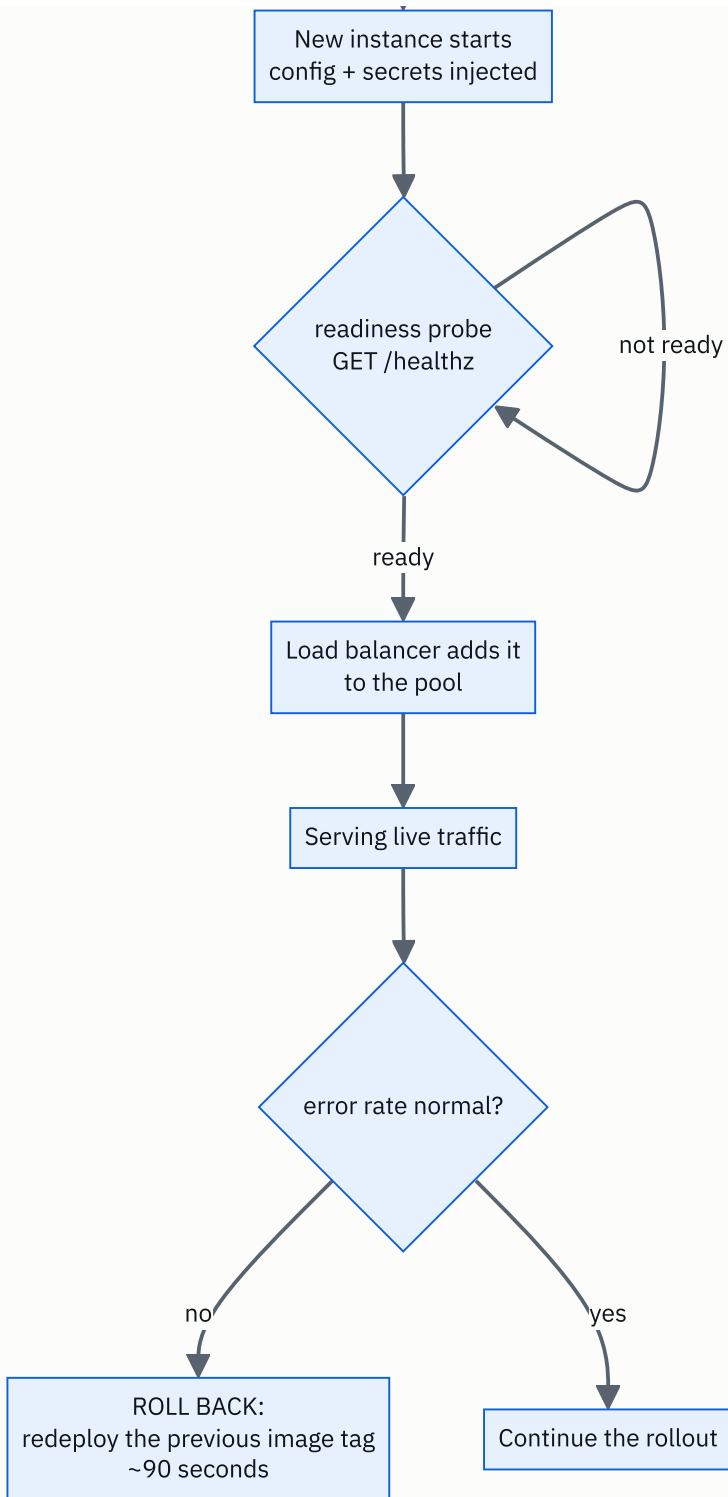


FIGURE 12.1

What to notice: there are three places where the pipeline can say no — CI, the readiness probe, and the error-rate check after traffic arrives — and each one catches problems the earlier ones cannot. And the rollback arrow does not go back to the code. It goes to an artefact that already exists.

What is inside the artefact, and what is not:

```
+----- THE IMAGE (identical everywhere) -----+
| your application code                               |
| every dependency, at an exact version                |
| the language runtime (python:3.12-slim)             |
| the OS libraries it needs                           |
+-----+
+
+----- INJECTED AT RUN TIME (differs per environment) ---+
| DATABASE_URL=postgres://prod-db:5432/app            |
| REDIS_URL=redis://prod-cache:6379                  |
| API_KEY=<from the secrets manager, never in the image> |
| LOG_LEVEL=info                                       |
+-----+

same image in dev, staging and production. Only the box below changes.
```

What to notice: the top box is built once and never modified. If a secret were in it, everyone with access to the registry would have that secret, and rotating it would require a rebuild.

And what a rolling deployment actually looks like, over about two minutes:

```
t=0    [v1] [v1] [v1] [v1]      4 instances, all old
t=20    [v1] [v1] [v1] [--]      one taken out of the pool, drained
t=40    [v1] [v1] [v1] [v2]      new one started, readiness probe passing
t=60    [v1] [v1] [v2] [v2]
t=90    [v1] [v2] [v2] [v2]
t=120   [v2] [v2] [v2] [v2]      done, zero downtime

during t=40 to t=120, BOTH versions are live at once.
→ the new code must tolerate old data, and the old code must tolerate new data.
```

What to notice: the overlap in the middle. For most of a rolling deployment two versions of your code are serving simultaneously, which is precisely why migrations must be backwards-compatible. This picture is the argument, and it is worth drawing in an interview.

5 How it actually works

The build, concretely

A `Dockerfile` describes the image. A realistic one is a **multi-stage build** — one stage to compile or install, a second that copies only the result into a small runtime image:

```
FROM python:3.12-slim AS build
COPY requirements.txt .
RUN pip install --prefix=/install -r requirements.txt

FROM python:3.12-slim
COPY --from=build /install /usr/local
COPY . /app
CMD ["uvicorn", "app:api", "--host", "0.0.0.0", "--port", "8000"]
```

Why this matters practically: build tools do not end up in the running image, so it is smaller (faster to pull) and has less in it to be vulnerable. A naive image is often 1.2 GB and a multi-stage one 200 MB, which at fifty instances is real deployment time.

Layer caching is the other reason for the ordering. Docker caches each step, so copying `requirements.txt` and installing *before* copying your source means a code change does not re-run the install. That is often the difference between a 30-second build and a 4-minute one.

Who runs the pipeline

GitHub Actions, **GitLab CI**, **CircleCI** and **Jenkins** all do the same thing: watch the repository, run a defined sequence on each push, and refuse to merge if it fails. A typical sequence is lint → type check → unit tests → build image → push to registry → deploy to staging → smoke test → deploy to production, often with a manual approval before the last step.

Who keeps it running

Kubernetes is the common answer at scale. You declare a desired state — “5 replicas of image `app:a3f8b1`, each needing 512 MB, with this readiness probe” — and a control loop continuously makes reality match. If a machine dies, the replicas are rescheduled elsewhere without anyone being paged.

AWS ECS and **Google Cloud Run** do the same with less to configure. **systemd** does it for a single machine, and is perfectly adequate for a great many real services — a unit file with `Restart=always` covers most of what a small system needs.

The mechanism underneath is all from [day 011](#): containers are ordinary processes with namespaces, cgroups and their own root filesystem, sharing the host kernel.

Readiness and liveness, precisely

```
GET /healthz/ready → 200 if the database connection works and caches are warm
GET /healthz/live  → 200 if the process is not deadlocked
```

They must be different endpoints, and confusing them causes a specific outage: if your readiness check fails during a database blip and you have wired it to liveness, the orchestrator kills every instance at once. The database recovers and there is nothing left running.

The **draining** step matters just as much. Before stopping an instance, the load balancer stops sending it new requests and waits for in-flight ones to finish — usually 15–30 seconds. Skipping it means every deployment returns errors to whoever was mid-request.

Observability, which is how you know it worked

A deployment is not finished when the pipeline goes green. It is finished when you have watched the numbers:

- **Metrics** — request rate, error rate, latency percentiles. **Prometheus** and **Grafana**, or **Datadog**.
- **Logs** — structured, shipped somewhere central, since the container's own filesystem is gone when it restarts.
- **Traces** — **OpenTelemetry**, following one request across services.

The four to watch during a rollout are the **golden signals**: latency, traffic, errors, saturation. Automated canary analysis is exactly this comparison — the new version's error rate against the old one's — done by a machine.

Where it goes wrong

Configuration drift: staging and production differ in some way nobody wrote down, so the thing that passed staging fails in production. This is what infrastructure-as-code (**Terraform**) exists to prevent.

A migration that ran and cannot be undone, discussed above. The most common serious deployment incident there is.

No draining: every deploy shows up as a spike of 502s.

Rollback that was never tested. A rollback path you have not exercised is a hope. The teams who recover fastest are the ones who roll back routinely and undramatically.

6 The numbers

How long each link takes, in a healthy pipeline:

git push	:	2 s
CI: lint, type check, unit tests	:	180 s
docker build (with layer cache)	:	45 s
docker build (cold, no cache)	:	240 s
push image to registry	:	30 s
deploy to staging + smoke tests	:	90 s
deploy to production (rolling, 4 replicas)	:	120 s

commit to production	:	467 s = about 8 minutes

Eight minutes is a good number for a small service. An hour is common and is a problem in itself: long pipelines mean batched changes, and batched changes mean that when something breaks you have twenty commits to search rather than one.

Rollback, which is the number that actually matters:

detect the problem (alert fires)	:	60 s
decide to roll back	:	30 s
redploy the previous image tag	:	90 s

total time to recovery	:	180 s = 3 minutes

Compare that with fixing forward:

find the bug, write a fix	:	900 s
CI + build + deploy	:	467 s

	:	1,367 s = 23 minutes

Three minutes against twenty-three. That ratio is the entire argument for keeping the previous artefact and for practising the rollback. **Roll back first, diagnose afterwards.**

Canary arithmetic — what 1% actually protects you from. A service doing 10,000 requests per second, and the new version fails 5% of requests:

full deploy	:	$10,000 \times 0.05$	=	500 failed requests per second
canary at 1%	:	$10,000 \times 0.01 \times 0.05$	=	5 failed requests per second

And how long until you have enough data to be confident, at a canary that receives 100 requests per second with a 5% failure rate:

$100 \times 0.05 = 5$ failures per second
→ a 5% regression is unmistakable within ~10 seconds

A hundredfold reduction in damage, and detection in ten seconds. But note the limit: a regression affecting 0.01% of requests produces 0.01 failures per second at 1% traffic, so you would need hours to see it. **Canaries catch big regressions quickly and**

small ones not at all, and knowing that boundary is what makes the answer credible.

Image size and how it affects deployment:

naive image, 1.2 GB, 50 instances, 1 Gbps pull

: 1.2 GB x 8 / 1 Gbps = 9.6 s each

multi-stage image, 200 MB

: 1.6 s each

With layer sharing, only the changed layers move — often a few megabytes. Which is why the `Dockerfile` ordering above matters more than it looks.

Deployment frequency, as an industry benchmark. The DORA research programme’s four metrics separate high and low performers starkly:

	elite	low
deployment frequency	on demand	monthly
lead time to production	under 1 hour	1-6 months
change failure rate	0-15%	46-60%
time to restore	under 1 hour	1 week to 1 month

The counter-intuitive finding, and the one worth quoting: **teams that deploy more often have lower failure rates**, not higher. Small, frequent changes are easier to verify and easier to undo than large, rare ones.

7 The trade-offs

Containers buy reproducibility and charge you a build step. The same image everywhere kills an entire class of environment bugs and makes rollback trivial, because the previous artefact still exists byte for byte. The cost is a build stage, a registry to run and pay for, image sizes to manage, and a genuine amount of orchestration knowledge before the first deployment. For a single small service, a virtual machine and `systemd` remains a defensible answer, and saying so is a sign of judgement rather than ignorance.

Kubernetes buys automation and charges you complexity. Self-healing, autoscaling, rolling updates, service discovery, all declarative. It also has a large surface area, needs someone who understands it, and can turn a simple service into an infrastructure project. The honest rule: below roughly five services, a managed platform such as ECS, Cloud Run or Fly is usually the better engineering. Kubernetes earns its keep on fleets, not on one application.

Blue-green versus rolling versus canary. Blue-green gives instant, complete rollback and requires double the capacity during the switch. Rolling needs no extra capacity and means both versions serve simultaneously, which forces backwards compatibility on you.

Canary limits blast radius best and needs enough traffic for a small percentage to be statistically meaningful — at 10 requests per second, a 1% canary sees one request every ten seconds and tells you nothing.

Fast pipelines versus thorough ones. Every check added to CI makes the pipeline slower, and a pipeline slow enough to be annoying gets bypassed — someone adds a “skip tests” flag for emergencies and then it is used routinely. The usual resolution is tiered: fast unit tests on every push, slower integration tests before merge, the full suite nightly. Optimising for the fast path is legitimate, and pretending the trade-off does not exist is not.

Automated rollback versus a human decision. Automating rollback on an error-rate threshold gives you three-minute recovery at three in the morning with nobody awake. It also means a transient spike from something unrelated — a dependency having a bad minute — can roll back a perfectly good deployment, and if the threshold is badly set it can flap. Most teams start with automated alerting and a human trigger, and automate the trigger once they trust the signal.

I would not do any of this if... it is a one-off script, an internal tool with three users, or a genuine prototype. A cron job on a virtual machine is a legitimate answer for a legitimate class of problem, and building a pipeline for it is a way of avoiding the actual work. The pipeline is justified by having users who notice when it breaks.

8 In the interview

How it gets asked

- “How does the code you wrote end up running on a server?” — the direct version, usually as a closing topic.
- “Walk me through your deployment process.” — the same question about your actual experience. Talk about what you did, including what went wrong.
- “How do you deploy without downtime?” — the specific version. Health checks, draining and rolling updates are the answer.
- “Something’s broken in production. What do you do?” — the rollback question, and the right first answer is “roll back, then diagnose”.

What to say out loud, in the first ninety seconds

1. **Give the chain.** “Commit and push, CI runs tests, merge, build an immutable image tagged with the commit hash, push it to a registry, deploy, health check, then the load balancer starts sending traffic.”

2. **Say why the artefact is immutable.** *“The image contains the code, the dependencies and the runtime, built the same way every time. Nothing about my laptop is involved — that’s what kills ‘works on my machine’.”*
3. **Separate config from code.** *“Config and secrets are injected at run time as environment variables, so the same image runs in staging and production. If I had to rebuild to point at a different database, I’d have built it wrong.”*
4. **Name the health check.** *“A new instance doesn’t get traffic until its readiness probe passes — it has to connect to the database and warm up first. And I’d keep readiness and liveness separate, because wiring a database blip to liveness kills every instance at once.”*
5. **Land the rollback.** *“The old image is still in the registry, so rolling back is redeploying the previous tag — about ninety seconds, against maybe twenty minutes to fix forward. So the rule is roll back first, diagnose afterwards.”*
6. **Volunteer the migration caveat.** *“The one thing that doesn’t roll back is a database migration. During a rolling deploy both versions are live at once, so migrations have to be backwards-compatible — add a column in one release, use it in the next, and never rename in a single step.”*

Step 6 is the one that marks you out. It is the part you only know if you have been burned.

The follow-ups

“How do you deploy with zero downtime?” Three things together. A rolling update, so instances are replaced a few at a time and there is always capacity serving. A readiness probe, so a new instance receives no traffic until it can actually handle it. And connection draining on the way out — the load balancer stops sending new requests to an instance and waits fifteen to thirty seconds for in-flight ones to finish before it is stopped. Miss the draining and every deployment shows up as a burst of 502s to whoever was mid-request. The consequence of all three is that both versions run simultaneously for a minute or two, so the code has to tolerate that.

“Something’s wrong in production right after a deploy. What do you do?” Roll back first, diagnose second. The previous image is still in the registry, so redeploying the old tag is about ninety seconds; finding and fixing the bug is twenty minutes at best, under pressure, with users affected the whole time. Once the system is stable I can investigate calmly using the logs and traces from the bad window. The one thing I’d check before rolling back is whether the deployment included a database migration — if it did, the old code may not work against the new schema, and then rolling back makes things worse rather than better. That is exactly why migrations are kept backwards-compatible.

“How do you handle database migrations?” Backwards-compatible, always, and separated from the code that uses them. Adding a column: run the migration first, deploy the code that uses it after. Removing one: deploy code that stops using it, verify, then drop it in a later release. Renaming: never in one step — add the new column, write to both, backfill, move reads across, then drop the old, which is four deployments. The reason is that during a rolling deploy both versions are live at once, so any migration that breaks the old version breaks production for the duration of the rollout and takes roll-back away from you.

“What’s in the image and what isn’t?” In: the application code, every dependency pinned to an exact version, the language runtime, and the OS libraries it needs. Not in: anything environment-specific, and above all no secrets. Configuration and credentials come from environment variables or a secrets manager at run time. Two reasons — the same image has to run in every environment, and a secret baked into an image is visible to everyone with registry access and cannot be rotated without a rebuild. I’d also use a multi-stage build so build tools don’t ship into the runtime image; that is typically the difference between 1.2 GB and 200 MB, which matters when fifty instances are pulling it.

A model answer

“It goes through about eight steps, and each one exists because something went wrong without it.

I commit and push to a branch and open a pull request. CI runs automatically — linting, type checks, unit tests — and a person reviews the change. Neither of those is optional; if CI fails, it can't merge.

On merge to main, the pipeline builds an artefact. In practice that's a container image containing my code, every dependency pinned, and the language runtime, tagged with the git commit hash. That immutability is the important part: the image is built once and runs identically in staging and production, so 'works on my machine' stops being a possible explanation. And the tag means any running process traces back to an exact commit.

It's pushed to a registry — ECR or similar — and then deployed. What's *not* in the image is configuration: the database URL, API keys, feature flags. Those are injected at run time as environment variables, with secrets from a secrets manager. If I had to rebuild the image to point at a different database, the separation is wrong.

The orchestrator — Kubernetes, or ECS, or just systemd on a smaller system — starts the new version. This is where the step people skip comes in: the instance doesn't receive traffic immediately. It exposes a readiness endpoint, and the load balancer only adds it to the pool once that returns 200, after it has connected to the database and warmed up. I'd keep that separate from the liveness probe, because if a database blip fails liveness, the orchestrator restarts every instance at once and now you have nothing running.

The rollout itself is usually rolling — replace a few instances at a time so there's always capacity — with connection draining on the way out so in-flight requests finish. For a risky change I'd canary instead: 1% of traffic, watch the error rate, then 10%, then everything. At 10,000 requests a second, a 1% canary turns 500 failures a second into 5, and a 5% regression is obvious within ten seconds.

The thing I'd emphasise most is rollback. The previous image is still in the registry, so rolling back is redeploying the old tag — about ninety seconds, against twenty minutes to fix forward. So the operational rule is: roll back first, diagnose after.

The exception, and the thing I'd raise unprompted, is database migrations. Those don't roll back. And because a rolling deployment has both versions live at once, migrations have to be backwards-compatible — add a column in one release and use it in the next, never rename in a single step. Getting that wrong is how a routine deploy becomes an incident you can't undo.”

9 Recall card

1. **The chain:** commit → CI → merge → build an **immutable artefact** tagged with the commit hash → registry → deploy → **readiness probe** → load balancer sends traffic.
2. **Config is not code.** The same image runs everywhere; the database URL and secrets are injected at run time. Never bake a secret into an image.
3. **Readiness and liveness are different questions.** Ready = send me traffic. Live = I am not deadlocked. Wiring a database blip to liveness kills the whole fleet.
4. **Roll back first, diagnose after.** The old image still exists, so recovery is ~90 seconds against ~20 minutes to fix forward.
5. **Migrations do not roll back, and both versions are live during a rolling deploy.** Add a column in one release, use it in the next. Never rename in one step.

PRACTICE

Practice — Searching an array: linear search, done properly

DSA topic: Searching an array: linear search, done properly **System design topic:** How your code becomes a running service

Code these, in this order

Four problems that are all one scan. The scanning is trivial; the **contract** is the exercise — what you return, which occurrence you return, and what happens when there is nothing to return.

For each problem, before writing anything:

1. Say what you return when the target is absent.
2. Say which occurrence you return if there are several.
3. Say what happens on an empty input.
4. Then write it, and confirm the empty case works with no special-case guard.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Search Insert Position	LeetCode 35 (Easy)	The “not found” answer is not <code>-1</code> here — it is where the value <i>would</i> go, which is a different contract entirely. Read the statement twice.
2	Find Numbers with Even Number of Digits	LeetCode 1295 (Easy)	Search by a condition , not by equality. This is the case binary search cannot handle, and the reason linear search survives.
3	Find All Numbers Disappeared in an Array	LeetCode 448 (Easy)	“Find all”, not “find first” — so no early exit is possible. The naive version searches for each candidate and is $O(n^2)$; a set makes it $O(n)$.
4	First Bad Version	LeetCode 278 (Easy)	Deliberately the counter-example. The linear scan is correct and too slow, and the problem exists to make you notice the property that allows something better. Solve it linearly first, then see why.

On problem 4, do this properly

- Write the linear version. It is correct. Submit it and see what happens.
- Then answer: what property of the input made a faster solution possible?
- Then say what that property is worth: at $n = 10^9$, how many checks does the linear version need, and how many does halving need?

- This is [day 042](#) arriving early, and it is worth meeting it here as “the thing linear search cannot do”.

The contract drill

For each of these, say the three answers out loud — absent, duplicates, empty:

1. “Find the index of the target.”
2. “Find the largest element.”
3. “Find the first element greater than 100.”
4. “Find how many times the target appears.”

Then check yourself against one rule: **could the caller confuse your ‘not found’ answer with a real answer?** If yes, the contract is wrong.

The falsy drill

Predict the output of this before running it:

```
def find(items, target):
    for i, x in enumerate(items):
        if x == target:
            return i
    return None

items = [7, 3, 9]
if find(items, 7):
    print("found")
else:
    print("not found")
```

Then explain the bug in one sentence, fix it, and say which five Python values are falsy.

The break-even drill

An array of one million elements, searched `k` times. Work these out with the arithmetic shown:

- The cost of `k` linear searches.
- The cost of building a hash set once, then `k` lookups.
- The cost of sorting once, then `k` binary searches.
- The `k` at which the hash set overtakes linear search.
- The `k` at which sorting overtakes linear search.

The one to try in a terminal

If you have a project with a Dockerfile, run `docker build .` twice in a row and compare the times. If you do not, read any `Dockerfile` you can find and answer two questions: which lines would invalidate the layer cache when you change one line of source code, and is there anything in it that should have been an environment variable instead.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the index of a target. What do you return if it is not there?* Ask the contract question first. Then give the code, say where the fallback return goes and why, give best/average/worst, and note why absence is always the worst case.
 2. *How does the code you wrote end up running on a server?* Give the eight links in order. Get to health checks and rollback without being asked, then volunteer the migration caveat.
 3. *Something is broken in production right after a deploy. What do you do?* Give the first action, the reason with the two timings compared, and the one thing you check before doing it.
-

Before you move on

- [] I ask what to return for “not found” before writing a search.
- [] I know why `-1` is riskier in Python than in Java, and what I would use instead.
- [] I never write `if result:` when `0` is a valid answer.
- [] I can say why absence is always the worst case, in one sentence.
- [] I can name the eight links from commit to serving traffic, and say which one cannot be rolled back.

Reversing, rotating, and swapping in place

DATA STRUCTURES AND ALGORITHMS

Reversing, rotating, and swapping in place

You can rotate an array by k using the three-reversal trick and explain why it works.

SYSTEM DESIGN

Containers and why everyone uses Docker

You can explain what a container gives you that a virtual machine does not.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- Rotate the array to the right by k , in $O(1)$ extra space.
- What problem does Docker actually solve?

Reversing, rotating, and swapping in place

1 What this is, and why they ask it

Rotating an array means moving every element along by k positions, with the ones that fall off the end wrapping round to the start. **Reversing** means turning the whole thing back to front. Both can be done **in place** — changing the array you were given, using a fixed handful of extra variables rather than building a second array.

The reason this is asked is the phrase “*in $O(1)$ extra space*”. Every candidate can rotate an array by building a new one. The interviewer removes that option and watches what happens. The expected answer is a trick that looks like nothing at first sight — **reverse the whole array, then reverse the two pieces** — and the follow-up is always the same: *why does that work?* Being able to answer that, rather than having memorised three lines, is the entire test.

It appears constantly. LeetCode 189 is a standard phone-screen question at Amazon, Microsoft and Google, and the reversal idea comes back on [day 017](#) for rotating a matrix, on [day 023](#) for palindromes, and on [day 081](#) for reversing a linked list. It is worth paying for once.

2 The story

Nazia's grandmother turns eighty on a Sunday, and the whole family comes to the house in Jaipur for it. At about six in the evening somebody says they should take one photo with everyone in it, because the last one was nine years ago and two of the people in it have died since.

Getting fifteen people to stand in a line takes twenty minutes. They end up standing by height, tallest on the left, down to the four smallest children on the right. Then Nazia's uncle, who is holding the camera, says the children cannot be at that end because nobody will be able to see them, and asks for them at the front instead — in the same order they are already standing in, because the two youngest are twins and their mother wants them next to each other the way they are.

The obvious thing is to walk the last child up to the front, then the next one, then the next. Nazia watches this start and stop. Each child has to squeeze past eleven adults, everybody shuffles sideways to make room, one of them stands on her aunt's foot, and

after two minutes only one child has moved.

Her uncle puts the camera down and says three sentences instead.

“Everybody turn round, face the other way, and walk to the other end of the line without changing who is beside you.” The whole line turns itself back to front. The four children are now on the left where he wants them, although among themselves they are the wrong way round, and so is everybody else.

“Now the four of you on the left — you four only — do the same thing among yourselves.” The four children move round within their own little group, and now they are back in the order they started in.

“And the eleven of you on the right, same thing.” Eleven people turn themselves round.

It takes about forty seconds. Nobody squeezes past anybody. Everyone ends up exactly where he wanted them, and the photo is taken before the light goes.

3 The idea in plain English

Nazia’s uncle did a rotation in three reversals. Everything below is that, said carefully.

First: swapping two elements

Every operation today is built out of one move — exchanging the values at two positions.

```
items[0], items[6] = items[6], items[0]
```

Python builds the pair `(items[6], items[0])` on the right first, then unpacks it into the two places on the left. Because the right-hand side is fully evaluated before anything is assigned, no temporary variable of your own is needed. In Java or C you would need one:

```
temp = items[0]; items[0] = items[6]; items[6] = temp;
```

That is worth knowing, because an interviewer may ask you to write it in a language without tuple unpacking, and because it explains what “constant extra space” means — one variable, no matter how big the array is.

Reversing in place

To reverse, put one marker at each end, swap what they are looking at, and walk them towards each other until they meet.

```
while left < right:
    items[left], items[right] = items[right], items[left]
    left += 1
    right -= 1
```

`left < right`, not `left ≤ right`. When the two meet in the middle of an odd-length array they are on the same element, and swapping an element with itself is pointless. It is not wrong — it just does nothing — but `<` says what you mean.

The number of swaps is `n // 2`, because each swap places two elements at once. Seven elements need three swaps; the middle one never moves.

This is your first **two-pointer** loop, and the whole of days 27 to 36 is built on it. A **pointer** here just means an integer holding a position — nothing to do with pointers in C.

What “rotate right by k” actually means

Rotating right by 3 takes the last three elements and puts them at the front, keeping their order:

```
[1, 2, 3, 4, 5, 6, 7] rotate right by 3 → [5, 6, 7, 1, 2, 3, 4]
```

Every element moved three places to the right, and the three that ran off the end came back at the beginning. The element that was at position `i` is now at position `(i + k) % n`.

Rotating left is the same operation with a different `k`. Rotating left by 3 on seven elements gives the same result as rotating right by 4, because `7 - 3 = 4`. So:

```
rotate left by k    =    rotate right by (n - k)
```

If a problem says “left” and you only remember “right”, convert and say so out loud. That is a better answer than getting the direction wrong.

`k` can be bigger than the array

If you rotate seven elements by seven, you get back exactly what you started with. So rotating by 10 is the same as rotating by 3. The first real line of every rotate function is therefore:

```
k %= n
```

Miss it and you get an `IndexError` on a bigger `k`, which §7 shows in full. This is the single most common reason a correct-looking rotate fails the hidden tests.

Two things to be careful about. `n` must not be zero, because `k % 0` raises `ZeroDivisionError` — so check for the empty array first. And in Python, `%` on a negative `k` already returns a non-negative result: `-3 % 7` is `4`, which is exactly the “rotate left by 3” answer. That is a convenient accident, and it is not true in C or Java, where `-3 % 7` is `-3`.

The three ways to rotate, and why only one of them is the answer

Way one — build a new array. Take the last `k`, then the first `n - k`, and stick them together.

```
items[:] = items[n - k:] + items[:n - k]
```

Correct, one line, $O(n)$ time. It uses $O(n)$ extra space, which is exactly what the question forbade. Note `items[:] =` and not `items =` — that difference is trap two in §7 and it is worth more marks than the rest of the line.

Way two — move one element at a time, `k` times. Take the last element off, put it at the front, repeat.

```
for _ in range(k):
    items.insert(0, items.pop())
```

$O(1)$ space and honest, but each `insert(0, ...)` shifts every element right by one, which is the $O(n)$ cost from [day 011](#). Doing that `k` times is $O(n \times k)$. At `n = 1,000,000` and `k = 500,000` that is 5×10^{11} moves — hours, not seconds.

Way three — three reversals. $O(n)$ time, $O(1)$ space, and the answer they want.

```
reverse(items, 0, n - 1)    # the whole thing
reverse(items, 0, k - 1)    # the first k
reverse(items, k, n - 1)    # the rest
```

Why the three reversals work

This is the part to be able to explain, and it is easier than it looks if you say it in two steps.

Step one: reversing the whole array gets the right elements onto the right side. The last `k` elements were at the back; after a full reversal they are at the front. That is the part you wanted. The elements that were at the front are now at the back, which is also what you wanted.

Step two: but reversing scrambled the order inside each group. The last three of `[1,2,3,4,5,6,7]` are `[5,6,7]`. After reversing everything they sit at the front as `[7,6,5]` — right place, wrong order. Reversing that group of three on its own turns

[7,6,5] back into [5,6,7]. Do the same to the other group and both are correct.

Said in one sentence for the interview: **“reversing the whole array puts both groups on the correct side but back to front within themselves, so I reverse each group again to undo that.”**

There is a second way to see it, which some interviewers prefer. Reversing twice returns you to where you started. The three reversals reverse each element exactly twice — once as part of the whole, once as part of its group — so within a group nothing has changed, while the groups themselves have swapped places.

The fourth way, which is worth knowing about

Cyclic replacement moves each element straight to its final home in one hop and carries the element it displaced. It touches each element exactly once, so it does n writes rather than about n swaps.

The catch is that following $i \rightarrow (i + k) \% n$ repeatedly does not always visit everything. If $n = 6$ and $k = 2$, you visit 0, 2, 4, 0 and stop — you have to start again from 1. The number of separate cycles is $\text{gcd}(n, k)$, which is why the code needs an outer loop and a counter.

It is genuinely harder to get right under pressure, and it is not faster in any way that matters. **Mention it, do not lead with it.** “There is also a cyclic-replacement version that does n writes instead of about n swaps, but it needs a gcd argument to show it terminates, so I would write the reversal one” is a strong sentence.

4 The picture

The two-pointer reversal, on seven elements:

position	0	1	2	3	4	5	6
value	1	2	3	4	5	6	7
	L						R
		L				R	
			L		R		
				L=R			
							swap 1 and 7
							swap 2 and 6
							swap 3 and 5
							stop: left < right is false
	7	6	5	4	3	2	1

7 elements, 3 swaps. The middle one never moved.

What to notice: the loop stops when the two markers meet, not when either of them has crossed the whole array. Running to the end would reverse it and then reverse it back — that is trap three.

Now the rotation, drawn as the three reversals with `k = 3`:

```
want: rotate right by 3

start      [ 1  2  3  4 | 5  6  7 ]
            the first 4      the last 3
                        ^ these three must end up at the front

reverse all [ 7  6  5 | 4  3  2  1 ]
            ^^^^^^^^^^  ^^^^^^^^^^^^^^^
            right side  right side
            wrong order wrong order

reverse [0..2] [ 5  6  7 | 4  3  2  1 ]
            ^^^^^^^^^^ fixed

reverse [3..6] [ 5  6  7 | 1  2  3  4 ]
                        ^^^^^^^^^^^^^^^ fixed

done       [ 5  6  7  1  2  3  4 ]
```

What to notice: after the first line every element is already on the side it belongs on. The two reversals that follow do not move anything across the boundary — they only tidy up inside each side. That is the whole proof, and it fits in a picture.

The `k % n` reduction, drawn:

```
n = 7, k = 10

rotating by 7 returns the array to exactly where it started.
10 = 7 + 3, so rotating by 10 is one free full turn plus a rotation by 3.

k:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14
    |-----| |-----| |
    one full turn      another one      and so on

10 % 7 = 3    → do the work for k = 3
```

What to notice: without this line, `reverse(items, 0, k - 1)` is asked to reverse the first ten elements of a seven-element array, and the crash is immediate.

And the three methods against each other, so the choice is visible:

<code>n = 1,000,000, k = 500,000</code>			
build a new array	time $O(n)$	:	1,000,000 moves space 8 MB extra
one at a time	time $O(n*k)$	:	500,000,000,000 moves space $O(1)$
three reversals	time $O(n)$	:	1,000,000 swaps space $O(1)$ ← the answer

What to notice: the middle row is not slightly worse, it is five hundred thousand times worse. This is the [day 004](#) point arriving inside a real question.

5 The code, built step by step

Start with the one operation everything is made of.

```
items[a], items[b] = items[b], items[a]
```

The right-hand side is evaluated into a pair before either assignment happens, so this is a true exchange with no temporary of your own. In a language without it, you write three lines and one temporary variable — still $O(1)$ space.

Now reversing a slice, given the two positions.

```
def reverse_in_place(items: list[int], left: int, right: int) → None:
    """Reverse items[left..right] inclusive.  $O(\text{right} - \text{left})$  time,  $O(1)$  space."""
    while left < right:
        items[left], items[right] = items[right], items[left]
        left += 1
        right -= 1
```

Taking `left` and `right` as arguments rather than always reversing the whole list is what makes it reusable three times. `right` is **inclusive**, so the caller passes `n - 1`, not `n`. Pick one convention and write it in the docstring; mixing them up is a real source of off-by-one bugs.

The version that uses extra space, so the comparison is honest.

```
def rotate_extra_space(items: list[int], k: int) → None:
    """Rotate right by k using a second list.  $O(n)$  time,  $O(n)$  extra space."""
    n = len(items)
    if n == 0:
        return
    k %= n
    items[:] = items[n - k:] + items[:n - k]
```

`items[:] = ...` writes through into the caller's list. `items = ...` would only rebind the local name and the caller would see nothing change. Guarding `n == 0` first matters because `k % 0` raises `ZeroDivisionError`.

The naive one, which is correct and too slow.

```
def rotate_one_by_one(items: list[int], k: int) → None:
    """Rotate right by k, one step at a time.  $O(n * k)$  time,  $O(1)$  space."""
    n = len(items)
    if n == 0:
        return
    k %= n
    for _ in range(k):
        items.insert(0, items.pop())
```

`pop()` from the end is $O(1)$; `insert(0, ...)` shifts everything and is $O(n)$. Say this out loud in an interview before you are asked — knowing that your own first idea is too slow is worth more than not having noticed.

And the one they want.

```
def rotate_three_reversals(items: list[int], k: int) → None:
    """Rotate right by k.  $O(n)$  time,  $O(1)$  extra space."""
    n = len(items)
    if n == 0:
        return
    k %= n
    reverse_in_place(items, 0, n - 1)
    reverse_in_place(items, 0, k - 1)
    reverse_in_place(items, k, n - 1)
```

Check the `k = 0` case by hand: `reverse_in_place(items, 0, -1)` has `left = 0` and `right = -1`, so `left < right` is false and it does nothing. The first and third reversals then undo each other. No special case needed, which is what a correct bound looks like.

The cyclic version, for completeness.

```
def rotate_cyclic(items: list[int], k: int) → None:
    """Move each element straight to its final place, carrying the one it displaces."""
    n = len(items)
    if n == 0:
        return
    k %= n
    if k == 0:
        return
    moved, start = 0, 0
    while moved < n:
        current, carried = start, items[start]
        while True:
            nxt = (current + k) % n
            items[nxt], carried = carried, items[nxt]
            current, moved = nxt, moved + 1
            if current == start:
                break
        start += 1
    start += 1
```

The outer `while moved < n` is the part people forget. With `n = 6, k = 2` the inner loop visits only `0, 2, 4` and returns to the start, so the outer loop has to begin a second cycle at position 1.

Here is the complete program.

```

"""Day 13 – reverse, rotate and swap, all in place."""

import random
import time

def reverse_in_place(items: list[int], left: int, right: int) → None:
    """Reverse items[left..right] inclusive. O(right - left) time, O(1) space."""
    while left < right:
        items[left], items[right] = items[right], items[left]
        left += 1
        right -= 1

def rotate_extra_space(items: list[int], k: int) → None:
    """Rotate right by k using a second list. O(n) time, O(n) extra space."""
    n = len(items)
    if n == 0:
        return
    k %= n
    items[:] = items[n - k:] + items[:n - k] # items[:] mutates, items = rebinds

def rotate_one_by_one(items: list[int], k: int) → None:
    """Rotate right by k, one step at a time. O(n * k) time, O(1) space."""
    n = len(items)
    if n == 0:
        return
    k %= n
    for _ in range(k):
        items.insert(0, items.pop()) # pop is O(1), insert(0) is O(n)

def rotate_three_reversals(items: list[int], k: int) → None:
    """Rotate right by k. O(n) time, O(1) extra space. The answer."""
    n = len(items)
    if n == 0:
        return
    k %= n
    reverse_in_place(items, 0, n - 1) # k = 10 on 7 items is the same as k = 3
    reverse_in_place(items, 0, k - 1) # whole thing
    reverse_in_place(items, k, n - 1) # the k that are now at the front
    # the n - k that are now at the back

def rotate_cyclic(items: list[int], k: int) → None:
    """Rotate right by k by moving each element straight to its final place."""
    n = len(items)
    if n == 0:
        return
    k %= n
    if k == 0:
        return
    moved = 0
    start = 0
    while moved < n:
        current = start
        carried = items[start]
        while True:
            nxt = (current + k) % n
            items[nxt], carried = items[nxt], carried
            current = nxt
            moved += 1
            if current == start:
                break
        start += 1

def count_swaps(n: int, k: int) → int:
    """How many two-element swaps the three-reversal version performs."""
    k %= n
    return (n // 2) + (k // 2) + ((n - k) // 2)

if __name__ == "__main__":

```

```

print("reversing a slice in place")
row = [10, 20, 30, 40, 50, 60, 70]
print(f" before      : {row}")
reverse_in_place(row, 0, len(row) - 1)
print(f" whole reversed : {row}")
reverse_in_place(row, 2, 4)
print(f" middle reversed : {row}")

print("\nthe three reversals, one at a time (k = 3, right)")
row = [1, 2, 3, 4, 5, 6, 7]
n, k = len(row), 3
print(f" start      : {row}")
reverse_in_place(row, 0, n - 1)
print(f" reverse all   : {row}")
reverse_in_place(row, 0, k - 1)
print(f" reverse first {k} : {row}")
reverse_in_place(row, k, n - 1)
print(f" reverse last {n - k} : {row}")

print("\nfour ways to rotate, same answer")
for name, fn in (("extra space", rotate_extra_space),
                 ("one by one", rotate_one_by_one),
                 ("three reversals", rotate_three_reversals),
                 ("cyclic", rotate_cyclic)):
    items = [1, 2, 3, 4, 5, 6, 7]
    fn(items, 3)
    print(f" {name:<16}: {items}")

print("\nthe edge cases")
EDGE: list[tuple[str, list[int], int]] = [
    ("empty list", [], 3),
    ("single element", [9], 3),
    ("k is zero", [1, 2, 3, 4], 0),
    ("k equals n", [1, 2, 3, 4], 4),
    ("k is bigger than n", [1, 2, 3, 4], 10),
    ("k is n minus one", [1, 2, 3, 4], 3),
    ("all identical", [7, 7, 7], 2),
]
print(f" {'case':<20}{'input':<16}{'k':>4} result")
for label, data, kk in EDGE:
    items = list(data)
    rotate_three_reversals(items, kk)
    print(f" {label:<20}{str(data):<16}{kk:>4} {items}")

print("\ndo all four agree? 2000 random cases")
random.seed(13)
disagreements = 0
for _ in range(2000):
    size = random.randint(0, 12)
    data = [random.randint(0, 9) for _ in range(size)]
    kk = random.randint(0, 30)
    answers = []
    for fn in (rotate_extra_space, rotate_one_by_one,
               rotate_three_reversals, rotate_cyclic):
        items = list(data)
        fn(items, kk)
        answers.append(items)
    if any(a != answers[0] for a in answers):
        disagreements += 1
print(f" disagreements: {disagreements}")

print("\nswaps performed by the three reversals (n = 1,000,000)")
for kk in (1, 250_000, 500_000, 999_999):
    print(f" k = {kk:>9} → {count_swaps(1_000_000, kk):>10} swaps")

print("\none-by-one vs three reversals (n = 20,000, k = 10,000)")
base = list(range(20_000))
a = list(base)
t0 = time.perf_counter(); rotate_one_by_one(a, 10_000); slow = time.perf_counter() - t0
b = list(base)
t0 = time.perf_counter(); rotate_three_reversals(b, 10_000); fast = time.perf_counter() - t0
print(f" one by one      : {slow:>8.4f} s")
print(f" three reversals   : {fast:>8.4f} s → {slow / fast:>6.0f}x faster")
print(f" same answer?      : {a == b}")

```



```
print("\nat a million elements, where one-by-one is not an option")
big = list(range(1_000_000))
t0 = time.perf_counter(); rotate_three_reversals(big, 333_333); r3 = time.perf_counter() - t0
big2 = list(range(1_000_000))
t0 = time.perf_counter(); rotate_extra_space(big2, 333_333); rs = time.perf_counter() - t0
print(f" three reversals : {r3:>8.4f} s  0(1) extra space")
print(f" slice + rebuild : {rs:>8.4f} s  0(n) extra space (~8 MB here)")
print(f" same answer?      : {big == big2}")
```

This is exactly what it printed:

```

reversing a slice in place
  before      : [10, 20, 30, 40, 50, 60, 70]
  whole reversed : [70, 60, 50, 40, 30, 20, 10]
  middle reversed : [70, 60, 30, 40, 50, 20, 10]

the three reversals, one at a time (k = 3, right)
  start      : [1, 2, 3, 4, 5, 6, 7]
  reverse all : [7, 6, 5, 4, 3, 2, 1]
  reverse first 3 : [5, 6, 7, 4, 3, 2, 1]
  reverse last 4 : [5, 6, 7, 1, 2, 3, 4]

four ways to rotate, same answer
  extra space : [5, 6, 7, 1, 2, 3, 4]
  one by one  : [5, 6, 7, 1, 2, 3, 4]
  three reversals : [5, 6, 7, 1, 2, 3, 4]
  cyclic      : [5, 6, 7, 1, 2, 3, 4]

the edge cases
case      input      k      result
empty list []        3      []
single element [9]    3      [9]
k is zero   [1, 2, 3, 4] 0      [1, 2, 3, 4]
k equals n  [1, 2, 3, 4] 4      [1, 2, 3, 4]
k is bigger than n [1, 2, 3, 4] 10     [3, 4, 1, 2]
k is n minus one [1, 2, 3, 4] 3      [2, 3, 4, 1]
all identical [7, 7, 7]    2      [7, 7, 7]

do all four agree? 2000 random cases
disagreements: 0

swaps performed by the three reversals (n = 1,000,000)
k =      1 →      999,999 swaps
k = 250,000 → 1,000,000 swaps
k = 500,000 → 1,000,000 swaps
k = 999,999 →      999,999 swaps

one-by-one vs three reversals (n = 20,000, k = 10,000)
one by one      : 0.1496 s
three reversals : 0.0043 s → 35x faster
same answer?    : True

at a million elements, where one-by-one is not an option
three reversals : 0.2036 s 0(1) extra space
slice + rebuild : 0.0468 s 0(n) extra space (~8 MB here)
same answer?    : True

```

Look at the edge-case table. Every one of those falls out with no special case except the empty check. `k = 0`, `k = n` and `k > n` are all handled by the single `k %= n` line.

Look at the swap counts. The total is essentially `n` regardless of `k`. That is the sentence to say in the interview: “about *n* swaps, whatever *k* is.”

Look at the last block, and be honest about it. The three-reversal version is the answer to the question as asked, and the slice version is **four times faster in wall-clock time**, because it is a handful of C-level memory copies while the reversal is a Python loop running a million iterations. Same $O(n)$, very different constant. The reversal wins on the thing the question actually constrained — memory — and loses on speed. Saying that out loud is a strong move, because it shows you know what a complexity class does and does not promise.

6 What it costs

Reversing a slice of length m . The two markers start $m - 1$ apart and move one step each per iteration, so the loop runs $m // 2$ times and does one swap each time.

```
m = 7 → 3 swaps    (the middle element never moves)
m = 8 → 4 swaps
```

$O(m)$ time, $O(1)$ space — three integers, whatever m is.

The three reversals, counted. Lengths n , then k , then $n - k$:

```
n // 2 + k // 2 + (n - k) // 2
```

With $n = 1,000,000$ and $k = 250,000$:

```
500,000 + 125,000 + 375,000 = 1,000,000 swaps
```

So **about n swaps in total, for any k** — each element is moved twice, and each swap moves two elements, so $2n / 2 = n$. $O(n)$ time, $O(1)$ extra space. That last part is the whole point of the question.

The one-at-a-time version. Each `insert(0, x)` shifts every element one place right, which is n moves, and you do it k times:

```
k x n moves
```

At $n = 1,000,000$ and $k = 500,000$:

```
500,000 x 1,000,000 = 500,000,000,000 moves
```

Five hundred billion. At roughly 10^8 operations per second that is about **an hour and a half**. The measured version above, scaled down to $n = 20,000$, already took 35 times longer than the reversals — and the gap grows with n , because one method is linear and

the other is not.

The extra-space version. $O(n)$ time and $O(n)$ extra space. How much memory is that, concretely? A Python list of a million small integers holds a million 8-byte references:

```
1,000,000 x 8 bytes = 8 MB for the new list
```

Eight megabytes is nothing on a laptop and is a real problem in a memory-constrained service holding many such arrays, which is why the constraint exists in the first place. This is the [day 007](#) distinction: $O(1)$ **extra** space, not $O(1)$ total — the input array itself still occupies $O(n)$, and nobody counts that.

The cyclic version. Exactly n writes and one carried variable, so $O(n)$ time and $O(1)$ space, with a slightly smaller constant than the reversals. It is not worth the risk of getting the cycle count wrong under time pressure.

Summary, which is what you should be able to produce on demand:

METHOD	TIME	EXTRA SPACE	WOULD YOU WRITE IT?
Build a new array	$O(n)$	$O(n)$	Only if space is not constrained. Fastest in practice.
One element at a time	$O(n \times k)$	$O(1)$	No. Correct and unusably slow.
Three reversals	$O(n)$	$O(1)$	Yes. This is the answer.
Cyclic replacement	$O(n)$	$O(1)$	Mention it. Do not lead with it.

7 The traps

Trap one: forgetting `k %= n`

The most common failure on this question, and it does not fail quietly.

```
def reverse_in_place(items, left, right):
    while left < right:
        items[left], items[right] = items[right], items[left]
        left += 1
        right -= 1

def rotate(items, k):
    n = len(items)
    reverse_in_place(items, 0, n - 1)
    reverse_in_place(items, 0, k - 1)          # k is not reduced
    reverse_in_place(items, k, n - 1)

items = [1, 2, 3, 4, 5]
rotate(items, 10)
print(items)
```

```
Traceback (most recent call last):
  File "t1.py", line 16, in <module>
    rotate(items, 10)
  File "t1.py", line 11, in rotate
    reverse_in_place(items, 0, k - 1)          # k is not reduced
    ~~~~~^~~~~~
  File "t1.py", line 3, in reverse_in_place
    items[left], items[right] = items[right], items[left]
    ~~~~~^~~~~~
IndexError: list index out of range
```

`IndexError: list index out of range`, from asking for `items[9]` in a five-element list. The fix is one line, `k %= n`, before anything else, and a guard for the empty array before that because `k % 0` raises `ZeroDivisionError`.

Say it before you are asked. *“First line: k modulo n , because rotating by the length is a no-op and the tests will send a k larger than the array.”*

Trap two: rebinding instead of mutating

This one produces no error at all. The function runs, returns, and changes nothing.

```
def rotate(items, k):
    k %= len(items)
    items = items[-k:] + items[:-k]          # rebinds the local name
    return None

data = [1, 2, 3, 4, 5, 6, 7]
rotate(data, 3)
print(data)
```

```
[1, 2, 3, 4, 5, 6, 7]
```

Unchanged. `items` inside the function is a local name that starts out referring to the caller's list. `items = ...` makes that local name refer to a brand new list and drops the connection. The caller still has the old one.

The fix is one character:

```
items[:] = items[-k:] + items[:-k]          # writes through into the caller's list
```

`items[:] = ...` is **slice assignment**: it replaces the contents of the existing list object rather than creating a new one. Any in-place function — this, `reverse`, the compaction loops on [day 015](#) — has to mutate, and `=` on the bare name never does.

How to catch it every time: if a function's job is to modify its argument and its body contains `argument_name =`, it is wrong.

Trap three: reversing all the way to the end

The near-miss that looks correct, produces no error, and gives you back exactly what you started with.

```
def reverse(items):
    n = len(items)
    for i in range(n):
        items[i], items[n - 1 - i] = items[n - 1 - i], items[i]
        # should be range(n // 2)

data = [1, 2, 3, 4, 5, 6]
reverse(data)
print(data)
```

```
[1, 2, 3, 4, 5, 6]
```

The loop reverses the array in its first half and then reverses it back in its second half. Every pair is swapped twice. With an odd-length array the middle element is swapped with itself and the result is still the original.

This is worse than a crash, because it looks like the function was never called. **The bound is `n // 2`, and the reason is that each swap fixes two positions at once.** The `while left < right` form makes the bug impossible, which is why it is the form worth writing.

The direction trap

“Rotate left by 3” and “rotate right by 3” are different answers, and half of candidates give the wrong one under pressure. There is no error message; the tests simply fail.

```
[1, 2, 3, 4, 5, 6, 7] right by 3 → [5, 6, 7, 1, 2, 3, 4]
[1, 2, 3, 4, 5, 6, 7] left by 3 → [4, 5, 6, 7, 1, 2, 3]
```

Two defences. Read the direction out of the statement and repeat it back before writing. And remember the conversion — **left by k is right by $n - k$** — so you only ever need one implementation.

8 In the interview

How it gets asked

- “Rotate the array to the right by k . Can you do it in $O(1)$ extra space?” — LeetCode 189, and the second sentence is the whole question.
- “Reverse the array in place.” — the warm-up, often the first thirty seconds of a phone screen.
- “Reverse the words in a sentence, but keep the words themselves the right way round.” — the same three-reversal idea wearing a hat. Reverse everything, then reverse each word.
- “Why does reversing three times give you a rotation?” — the follow-up that decides the question.

What to say out loud, in the first ninety seconds

1. **Repeat the direction and the constraint.** “Right by k , in place, constant extra space. And k could be larger than the array, so the first thing I do is k modulo n .”
2. **Name the naive options and reject them, briefly.** “I could build a new array with a slice — that’s $O(n)$ time but $O(n)$ space, which the constraint rules out. I could rotate one step at a time k times, but each of those shifts everything, so it’s $O(n \times k)$.”
3. **State the trick before writing it.** “The in-place answer is three reversals. Reverse the whole array, then reverse the first k , then reverse the rest.”
4. **Explain why, in one sentence.** “Reversing the whole array puts the last k elements at the front where they belong, but back to front within themselves. Reversing each group again undoes exactly that, and doesn’t move anything across the boundary.”
5. **Write it, and read the edge cases off the code.** “Empty list returns early. $k = 0$ makes the second reversal a no-op, with left = 0 and right = -1 , and the first and third undo each other, so it’s correct with no special case.”
6. **Give both costs.** “About n swaps in total, whatever k is — n over 2, plus k over 2, plus $n - k$ over 2. So $O(n)$ time and $O(1)$ extra space: three integer variables regardless of size.”

If you have time, add step 7: *“There’s also a cyclic-replacement version that does n writes instead of n swaps, but showing it visits everything needs a gcd argument, so under time pressure I’d write the reversals.”*

The follow-ups

“Why does the three-reversal trick work?” Two reasons said together. First, reversing the whole array is what gets the last k elements to the front and the first $n - k$ to the back — that is the movement between the two groups, and it happens in one step. Second, a full reversal also flips the order inside each group, so $[5, 6, 7]$ arrives as $[7, 6, 5]$. Reversing each group on its own puts that back. Another way to say it: every element is reversed exactly twice, once as part of the whole and once as part of its group, and two reversals cancel — so within a group nothing changed, while the groups themselves swapped sides.

“What if k is bigger than the array? What if k is negative?” $k \% n$ handles both. Rotating by n returns the array to its starting position, so only the remainder matters, and $10 \% 7 = 3$. For negative k , Python’s `%` already returns a non-negative result — $-3 \% 7$ is 4 , which is exactly “rotate right by 4”, which is “rotate left by 3”. That is the correct answer, though I would confirm the intended meaning rather than rely on it, and in Java or C the same expression gives -3 and I would have to normalise it myself. I would also guard $n == 0$ before the modulo, because $k \% 0$ raises `ZeroDivisionError`.

“Rotate left instead.” Same function, different k : rotating left by k is rotating right by $n - k$. Or, written directly, the three reversals happen in a different order — reverse the first k , reverse the rest, then reverse the whole thing. I would write one implementation and convert, because two near-identical functions is two chances to get the direction wrong.

“Can you do it in one pass instead of three?” Yes — cyclic replacement. Start at position 0, move that element to $(0 + k) \% n$, carry the element it displaced, and keep going until you return to where you started. Each element is written exactly once, so it is n writes against roughly n swaps, a slightly better constant. The catch is that the chain of hops does not always cover the array: with $n = 6$ and $k = 2$ you visit only $0, 2, 4$ before looping back, and the number of separate cycles is $\text{gcd}(n, k)$. So it needs an outer loop and a counter, and it is much easier to get subtly wrong. Same complexity, more risk — I would write the reversals.

“What if it were a linked list rather than an array?” A different problem, because there is no random access and nothing to swap by position. For a singly linked list you walk to the end, join it into a ring, walk $n - k$ steps from the head, and break the ring

there. That is $O(n)$ time and $O(1)$ space too, but it is link surgery rather than position arithmetic, and it is [day 081](#).

A model answer

“Right by k , in place, $O(1)$ extra space. Before I write anything: k could be bigger than the array, and rotating by exactly the length gets you back where you started, so the first line is `k %= n`. And I need to guard the empty array before that, because `k % 0` raises.

The version I’m not allowed to use is the one-liner — take the last k , put the first $n - k$ after it — because that builds a second array and costs $O(n)$ space. The other naive one is rotating a single step k times, but each step shifts every element, so that’s $O(n \times k)$. At a million elements with k a half-million that’s 5×10^{11} moves.

The in-place answer is three reversals:

```
def rotate(items: list[int], k: int) → None:
    n = len(items)
    if n == 0:
        return
    k %= n
    reverse(items, 0, n - 1)
    reverse(items, 0, k - 1)
    reverse(items, k, n - 1)
```

where `reverse` is the standard two-pointer loop — one marker at each end, swap, walk them inwards while `left < right`.

Why it works: reversing the whole array is the step that actually moves elements between the two groups — the last k end up at the front, the first $n - k$ end up at the back. But a full reversal also flips the order *inside* each group, so the last three arrive as 7, 6, 5 instead of 5, 6, 7. Reversing each group separately undoes exactly that and doesn’t move anything across the boundary. Equivalently, each element gets reversed twice — once in the whole, once in its group — and two reversals cancel.

Edge cases fall out. Empty returns early. $k = 0$ makes the middle call `reverse(items, 0, -1)`, where `left < right` is immediately false, and the first and third reversals undo each other. $k = n$ is the same case after the modulo.

Cost: n over 2 swaps for the whole, plus k over 2, plus $n - k$ over 2 — about n swaps in total, for any k . So $O(n)$ time, $O(1)$ extra space; three integers regardless of how big the array is.

One honest note — if the space constraint weren't there, the slice version is the same $O(n)$ but several times faster in wall-clock, because it's C-level memory copying rather than a million-iteration Python loop. Same complexity class, very different constant."

That answer states the constraint back, rejects the two naive options with numbers, gives the trick, **explains why it works**, checks the edges from the code, gives both costs, and ends with a remark that shows the candidate knows what big-O does not tell you.

9 Recall card

1. **Rotate right by k = three reversals.** Reverse the whole thing, reverse the first k , reverse the last $n - k$. $O(n)$ time, $O(1)$ extra space.
2. $k \% n$ **is the first line, after guarding the empty array.** Rotating by the length is a no-op; without the modulo a large k gives `IndexError: list index out of range`.
3. **Why it works:** the full reversal moves the two groups to the right sides but flips them internally; reversing each group undoes exactly that. Every element is reversed twice.
4. **Reverse in place is `left < right`, swap, step both inwards** — $n // 2$ swaps, because each swap fixes two positions. Looping to n reverses it back to the original.
5. **In place means mutate.** `items[:] = ...` writes through to the caller; `items = ...` only rebinds a local name and silently changes nothing.

1 What this is, and why they ask it

A **container** is an ordinary process — the same kind of process you met on [day 008](#) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume. It sees its own filesystem, its own process list and its own network, and it can be told it may use no more than half a CPU and 512 MB of memory. **It is not a small computer.** There is no second operating system inside it; it shares the one kernel the host is already running.

That single sentence — *a container is a process, a virtual machine is a computer* — is what the question is really testing. Interviewers ask it because half of candidates describe a container as “a lightweight virtual machine”, which is a description that gets every follow-up wrong: it cannot explain why containers start in a tenth of a second, why you cannot run a Windows container on a Linux host, or why a container escape is a more serious security event than a virtual machine escape.

[Day 012](#) used images and registries to explain how code reaches production. Today is what is actually inside them, and why the industry moved.

2 The story

Devika rents space in a shared kitchen in a lane off Sarjapur Road in Bangalore. Four small food businesses use it — she makes pickles, a man makes cakes, two brothers make sandwiches for offices, and a woman makes idli batter and is gone by nine in the morning.

For the first six months it went badly, and the reasons were all small ones. Her mustard oil kept going down, and nobody had taken it on purpose — somebody had used it once, thinking it was theirs. The cake man set the big oven to 180 and left it that way, and a batch of her jars came out wrong. The brothers left their knives in the sink, she washed them, and afterwards they said the knives had been blunted. One Tuesday she came in and there was no room to put anything down at all.

The owner fixed it over a weekend, and he did not build four kitchens.

Each business got a numbered steel trolley with a lock, a shelf in the cold room with its own name painted on it, and its own knives, boards and vessels kept on the trolley. Each one got a fixed pair of gas rings and a fixed stretch of counter, marked out in tape

on the floor. He also wrote a limit on the wall: nobody uses more than two rings at once, however busy they are, and nobody leaves anything on the shared counter overnight.

Nothing else changed. It is the same building, the same water, the same electricity, the same exhaust fan, the same rent split four ways. Nobody has their own front door. But when Devika comes in at five in the morning, everything she needs is on her trolley exactly as she left it, in the same places, and whatever the cake man did last night cannot reach her.

Her sister runs a similar business in Pune and pays for a whole small kitchen of her own, with her own meter and her own gas connection. It is quieter, and nothing of hers is ever touched by anyone. It also costs her four times as much, and it took two months to get the connections put in.

3 The idea in plain English

Devika’s trolley is a container. Her sister’s separate kitchen is a virtual machine. Every piece of the arrangement maps onto something real.

What the container actually is

A container is **one process, started by the kernel like any other**, with three things done to it at the moment it starts:

THE KITCHEN	THE MACHINE	WHAT IT DOES
Own trolley, own vessels, own shelf	Namespaces	Restrict what the process can <i>see</i>
Two gas rings, no more	cgroups	Restrict what the process can <i>use</i>
Everything already on the trolley	The image	Give it its own filesystem, identical every time

Underneath all three, **the building is shared**. One kernel, one set of physical CPUs, one block of memory. That is the whole difference, and it is where every other difference comes from.

Namespaces: what it can see

A **namespace** is a kernel feature that gives a process its own private version of one part of the system. Linux has several, and a container normally gets all of them:

- **PID namespace** — its own process numbering. The application inside sees itself as process 1 and cannot see anything else running on the machine.
- **Mount namespace** — its own filesystem tree. `/` inside the container is the image’s contents, not the host’s disk.

- **Network namespace** — its own network interfaces, its own IP address, its own port numbers. Two containers can both listen on port 8000 without a conflict.
- **UTS namespace** — its own hostname.
- **IPC namespace** — its own shared-memory segments.
- **User namespace** — its own user IDs, so root inside can map to an unprivileged user outside.

Devika opening her trolley sees only her own vessels. They are in the same building as everyone else's, but her view does not include them.

cgroups: what it can use

A **control group**, universally shortened to **cgroup**, is a kernel feature that caps and accounts for resources. You give a container `--memory=512m --cpus=0.5` and the kernel enforces it: at 512 MB the process is killed by the out-of-memory killer, and it never receives more than half a core's worth of scheduling.

This is the limit written on the wall. Without it, one busy container starves the others on the same host — the noisy-neighbour problem — and with it, capacity planning becomes arithmetic instead of hope.

The image: why it is the same every time

An **image** is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code. A **container** is a running process started from an image, with one thin writable layer on top for anything it changes.

The distinction is worth being precise about, because interviewers test it directly:

An image is a template. A container is a running instance of it. One image, twenty containers, all identical at the moment they start.

The stack of layers is a **union filesystem** — several directories presented as one merged tree. When the container reads a file it is served from the topmost layer that has it. When it writes, the file is first copied up into the writable layer, which is called **copy-on-write**. Nothing in the image itself ever changes.

That is why fifty containers from the same image share one copy of the base layers on disk and in memory, and why starting a container does not involve copying hundreds of megabytes.

A virtual machine, and why it is a different thing

A **virtual machine** is an emulated computer. A **hypervisor** — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete **guest operating system**: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

That guest operating system is Devika’s sister’s separate gas connection and meter. It is real isolation — a virtual machine has its own kernel, so a crash or a compromise inside it does not directly touch the others. It also costs a full operating system’s worth of memory, disk and boot time, per virtual machine, before your application has done anything at all.

	CONTAINER	VIRTUAL MACHINE
What it is	A process with a restricted view	An emulated computer
Kernel	Shares the host’s	Its own
Start time	50–200 ms	30–60 s
Memory before your app runs	a few MB	512 MB – 1 GB
Isolation strength	Good	Stronger
Can run a different OS	No	Yes
Typical density on one host	hundreds	tens

So what does Docker actually solve?

Two problems, and it is worth separating them because they are often muddled.

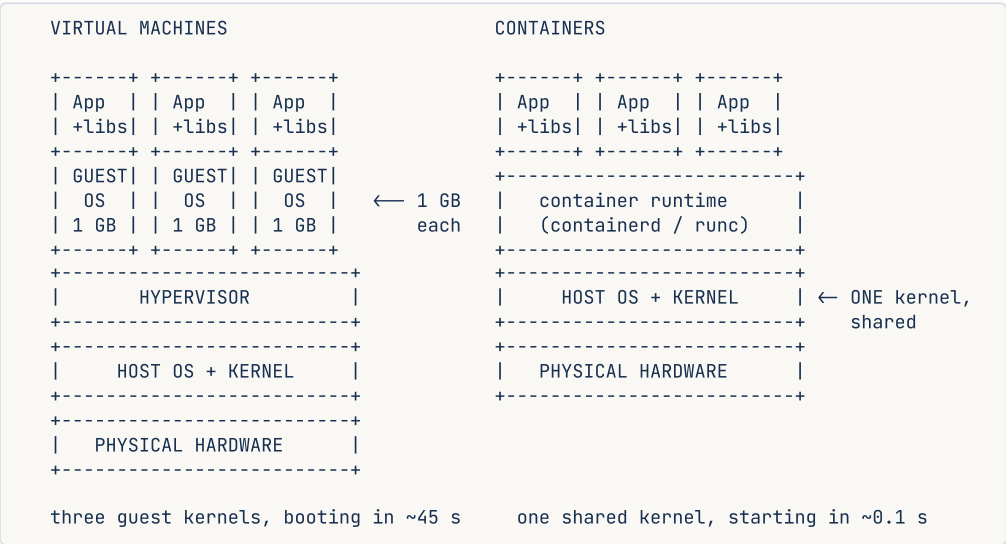
1. Packaging. Before containers, deploying meant “install Python 3.9, then these fourteen libraries at these versions, then set these environment variables, then copy the code” — a list of instructions that drifted between the laptop, the test machine and production. An image is that entire list already carried out, frozen, and shipped as one file. This is the “works on my machine” problem from [day 012](#), and packaging is what kills it.

2. Density and speed. Because a container is a process rather than a computer, you can put far more of them on one machine, and start one in the time it takes to blink. That is what makes autoscaling and per-request billing possible at all.

Docker itself is the tooling — a command line, a build format, a daemon, and Docker Hub. The container is a Linux kernel feature and existed before Docker did. Docker’s contribution was making it usable, and the format is now standardised by the **OCI** (Open Container Initiative) so that other tools — **Podman**, **containerd**, **CRI-O** — run the same images.

4 The picture

The two stacks, side by side. This is the drawing to reproduce in an interview:



What to notice: the container column has no row for a guest operating system. Everything that follows — the start time, the memory, the density, and the fact that you cannot run Windows on a Linux host — is a consequence of that one missing box.

What a running container actually is, from the kernel’s point of view:

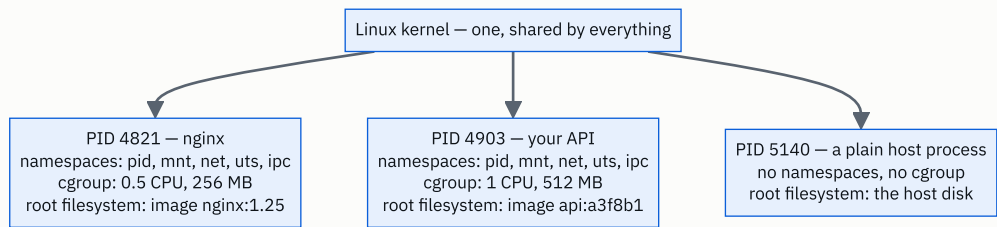


FIGURE 13.1

What to notice: all three are entries in the same process table on the same kernel. The first two are called containers only because of the namespace and cgroup columns. `ps aux` on the host shows every one of them; `ps aux` inside the first shows one process.

Image layers, and why fifty services do not cost fifty times the disk:

IMAGE api:a3f8b1			IMAGE worker:c91d02		
+-----+-----+-----+			+-----+-----+-----+		
your code	2 MB		your code	3 MB	unique
+-----+-----+-----+			+-----+-----+-----+		
pip install deps	45 MB		pip install deps	45 MB	unique
+-----+-----+-----+			+-----+-----+-----+		
python:3.12-slim	130 MB		python:3.12-slim	130 MB	SHARED
+-----+-----+-----+			+-----+-----+-----+		
+-----+-----+-----+			+-----+-----+-----+		
v					
stored ONCE on the host					
pulled ONCE from the registry					
naive: 177 + 178 = 355 MB			actual on disk: 130 + 47 + 48 = 225 MB		

What to notice: the shared base layer is stored once, transferred once and cached once. This is also why the `Dockerfile` ordering from [day 012](#) matters — putting the dependency install above the code copy keeps the expensive layer stable while the cheap one changes.

5 How it actually works

Starting a container, step by step

When you run `docker run --memory=512m --cpus=0.5 api:a3f8b1`, this happens:

1. **The image is fetched** from a registry if it is not already local, layer by layer. Layers already on disk are skipped.
2. **The layers are stacked** into one filesystem view using **overlays**, the union filesystem built into Linux, with an empty writable layer on top.
3. **A new process is created** with `clone()`, passing flags such as `CLONE_NEWPID`, `CLONE_NEWNET` and `CLONE_NEWNS` — that is literally how a namespace is made.
4. **A cgroup is created** and the process is placed in it, with `memory.max` set to 512 MB and the CPU quota set to half a period.
5. **The root is switched** with `pivot_root` so the process's `/` is the stacked image.
6. **The image's start command is executed.** From here it is an ordinary running program.

Steps 3 to 6 take a few tens of milliseconds. There is no boot, because there is nothing to boot.

Who does what

The stack has more parts than people expect, and naming them correctly is a small, cheap signal:

```
docker CLI → dockerd (daemon) → containerd (manages lifecycle) → runc (creates it)
                                                         |
                                                         clone(), cgroups,
                                                         pivot_root
```

runc is the piece that actually talks to the kernel, and it is about as thin as it sounds. **containerd** keeps track of images and running containers. **Kubernetes** talks to containerd directly through the CRI (Container Runtime Interface) and does not need Docker at all, which is why Kubernetes removing Docker support in 2022 changed nothing about the images anyone was running.

Podman does the same job without a daemon, running containers as your own user — which removes a genuine security concern, because the Docker daemon runs as root and access to its socket is effectively root on the host.

Where the data goes

The writable layer of a container **disappears when the container is removed**. That is deliberate: containers are meant to be disposable, and anything worth keeping must be stored outside.

- A **volume** is storage managed by the runtime and mounted into the container, surviving restarts and replacement.
- A **bind mount** maps a directory from the host into the container — the standard way to get live code reloading in development.
- In production, state usually lives somewhere else entirely: a managed **Postgres**, **S3**, **Redis**. The container itself stays **stateless**, which is what makes it replaceable.

Networking, briefly

Each container gets its own network namespace and a virtual interface, connected to a bridge on the host. `-p 8080:80` sets up a NAT rule forwarding the host's port 8080 to port 80 inside. Containers on the same user-defined network reach each other by name, because the runtime provides DNS — so `http://payments:8000` resolves without anyone hard-coding an address.

Which products are which

- **Build and run:** Docker, Podman, BuildKit, Buildah.
- **Runtimes:** containerd, CRI-O, runc, crun.
- **Registries:** Docker Hub, Amazon ECR, Google Artifact Registry, GitHub Container Registry.

- **Orchestrators:** Kubernetes, AWS ECS, HashiCorp Nomad.
- **Managed, no machines to run:** AWS Fargate, Google Cloud Run, Azure Container Apps, Fly.io.
- **Stronger isolation, container-shaped:** **AWS Firecracker** (the microVM behind Lambda and Fargate, booting a stripped-down kernel in about 125 ms), **gVisor** (a user-space kernel that intercepts system calls), **Kata Containers** (a real VM wearing a container interface).

That last group exists for one reason, and it is the subject of §7: a shared kernel is a shared risk.

The two things that surprise people

A container image has no kernel in it. It contains the userland — the files, the libraries, `/bin/sh` — and nothing else. This is why `ubuntu:22.04` is about 70 MB rather than several gigabytes, and why a “Ubuntu container” running on a Fedora host is running the Fedora kernel.

Docker on a Mac or Windows laptop is running a Linux virtual machine. Containers are a Linux kernel feature. Docker Desktop quietly runs a small Linux VM and puts your containers inside it, which is exactly why file access between the host and a container is noticeably slower there than on a Linux machine.

6 The numbers

Start time, which is the number that changes what you can build:

```
virtual machine, cold boot : 30-60 s
container start           : 50-200 ms
-----
ratio                     : about 300x
```

Three hundred times is not a tuning improvement, it is a change in what is possible. Scaling out to meet a traffic spike is useful at 200 ms and useless at 45 s, because the spike is over before the capacity arrives. Per-request platforms such as Cloud Run and Lambda exist entirely inside that gap.

Memory overhead before your code has done anything:

```
guest OS per virtual machine : 512 MB - 1 GB
container overhead           : ~2-10 MB (namespaces and cgroup bookkeeping)
```

Put that on a real host — 64 GB of RAM, and an application that needs 500 MB:

```

VMs      : 1,024 MB guest OS + 500 MB app = 1,524 MB each
          64,000 / 1,524 = 42 instances

Containers : 8 MB overhead + 500 MB app      = 508 MB each
            64,000 / 508    = 126 instances

```

Three times the instances on the same machine, and the difference is entirely operating systems you did not need. At a cloud price of roughly \$300 a month for that host, the effective cost per instance falls from \$7.14 to \$2.38.

Disk, with layer sharing. Fifty services, all built on `python:3.12-slim` (130 MB), each adding 45 MB of dependencies and 2 MB of code:

```

if nothing were shared : 50 x 177 MB = 8,850 MB = 8.6 GB
with the base shared   : 130 + (50 x 47) = 2,480 MB = 2.4 GB
-----
saved                  : 6.2 GB, about 72%

```

Pull time, which is deployment time. On a 1 Gbps link:

```

full image, 177 MB, cold : 177 x 8 / 1,000 Mbps = 1.4 s
code layer only,  2 MB   :  2 x 8 / 1,000 Mbps = 0.016 s

```

A code-only change moves 2 MB, not 177. Across fifty instances pulling at once, that is the difference between 70 seconds and under a second of transfer.

Density in practice. A rule of thumb worth carrying: a modern 8-core, 32 GB host runs roughly **10–20 virtual machines** or **100–200 containers**, assuming small services. Kubernetes itself defaults to a cap of 110 pods per node, which is a useful number to know if you are ever asked to size a cluster:

```

a service needing 300 replicas at 110 pods per node → 3 nodes minimum, 4 for headroom

```

Image size and the build. The multi-stage build from [day 012](#), in numbers:

```

naive single-stage image : 1.2 GB
multi-stage, slim base   : 200 MB
-----
6x smaller → 6x faster to pull, and far fewer packages that can turn out to be vulnerable

```

7 The trade-offs

A shared kernel is the whole benefit and the whole risk. Everything good about containers comes from not having a second kernel. So does the main weakness: a kernel vulnerability is a path out of the container and onto the host, and from there to every other

container on it. A virtual machine escape has to defeat the hypervisor, which is a much smaller and much more carefully audited piece of software. This is why AWS runs Lambda and Fargate on **Firecracker microVMs** rather than plain containers — when the code being run belongs to strangers, container isolation is not considered enough.

I would not use plain containers if I were running untrusted, customer-supplied code on shared hosts. That is a VM, a microVM, or gVisor.

You cannot escape the host's kernel. A Linux container needs a Linux kernel. There is no such thing as running a Linux container on a Windows kernel, or an ARM image on x86, without a virtual machine or an emulation layer underneath doing real work. Docker Desktop hides this by running a Linux VM; `--platform linux/amd64` on an Apple Silicon Mac hides it by emulating, which is correct and several times slower.

Containers assume statelessness, and databases are not stateless. Running Postgres in a container is entirely possible and is done in development constantly. In production it means taking on volume management, backup, failover and upgrade sequencing yourself, in an environment designed around processes being replaceable at any moment. Most teams use a managed database and containerise only the application, and saying that is a mark of judgement rather than timidity.

Root inside a container is closer to root outside than people assume. By default the process runs as UID 0, and although capabilities are dropped, a misconfiguration — a bind-mounted Docker socket, `--privileged`, a mounted host path — turns that into control of the host. The defences are ordinary and worth naming: run as a non-root user, enable user namespaces, drop capabilities, use a read-only root filesystem.

Operational complexity moves rather than disappearing. You stop debugging environment differences and start debugging image builds, registry permissions, layer caching, container networking and orchestrator configuration. For one small service on one machine, a virtual machine with `systemd` is genuinely simpler and remains a defensible answer. Containers earn their keep when there are many services, many deployments a day, or elastic traffic.

Long-lived containers drift back into pets. The value comes from replacing containers rather than repairing them. A team that starts running `docker exec` to patch a live container has recreated the problem containers were bought to solve, and lost the guarantee that the image is the truth.

8 In the interview

How it gets asked

- “What problem does Docker actually solve?” — the direct version. Give both problems: packaging and density.
- “What’s the difference between a container and a virtual machine?” — the classic. The one-sentence answer is about the shared kernel; everything else follows from it.
- “How does a container isolate anything if it’s just a process?” — the deeper version. Namespaces and cgroups, named.
- “Would you run your database in a container?” — the judgement question. The honest answer is “in development yes, in production I’d use a managed one, and here is why”.

What to say out loud, in the first ninety seconds

1. **Define it correctly, immediately.** *“A container is a process on the host kernel with a restricted view of the system and a cap on what it can use. It is not a small computer — it has no operating system of its own.”*
2. **Name the three mechanisms.** *“Namespaces control what it can see — its own process list, filesystem, network, hostname. cgroups control what it can use — half a CPU, 512 MB. And the image gives it its own root filesystem.”*
3. **Contrast with a VM in one line.** *“A virtual machine emulates a whole computer and runs its own kernel. That’s the only real difference, and everything else comes out of it.”*
4. **Give the two numbers.** *“A VM boots in about 45 seconds and costs 512 MB to a gigabyte of memory before your app starts. A container starts in about 100 milliseconds and costs a few megabytes. That’s roughly three times the density on the same host, and it’s what makes autoscaling actually work.”*
5. **Say what it solved.** *“Two things. Packaging — the image contains the code, the dependencies and the runtime, built once, so ‘works on my machine’ stops being possible. And density — you can put hundreds on a host and start them instantly.”*
6. **Volunteer the cost.** *“The trade-off is the shared kernel. A kernel vulnerability is a path out of the container, so for untrusted code you want a real boundary — that’s why Lambda and Fargate run on Firecracker microVMs rather than plain containers.”*

Step 6 is the one that separates a memorised answer from an understood one.

The follow-ups

“If a container is just a process, what stops it seeing the rest of the machine?” Two kernel features. Namespaces give the process a private view of one subsystem each — a PID namespace so it sees itself as process 1 and nothing else, a mount namespace so its `/` is the image rather than the host disk, a network namespace so it has its own inter-

faces and ports, and so on for hostname, IPC and user IDs. cgroups then limit what it can consume: memory, CPU shares, block I/O, number of processes. Neither is emulation — the kernel is simply filtering what this process is allowed to see and use. That is why the isolation costs almost nothing, and also why it is weaker than a hypervisor's.

“Why can't I run a Windows container on Linux?” Because the image contains no kernel. An image is a userland — files, libraries, binaries — and the process inside makes system calls to the **host's** kernel. Windows binaries make Windows system calls, which a Linux kernel does not implement. The same reasoning explains why Docker Desktop on a Mac is quietly running a Linux virtual machine, and why an ARM image on an x86 host needs emulation. If someone tells you they run Linux containers on Windows, they are running WSL 2, which is a Linux kernel in a lightweight VM.

“When would you choose a virtual machine over a container?” Four situations. Untrusted or customer-supplied code, where a shared kernel is not an acceptable boundary — AWS uses Firecracker microVMs for exactly this. A different operating system or a different kernel version from the host. Compliance regimes that require hardware-level tenant separation. And workloads that are genuinely a whole machine — a legacy application with its own init system and its own scheduled jobs. Outside those, the container is usually the better trade, and there is a middle ground — gVisor, Kata, Firecracker — that gives you a stronger boundary while keeping the container interface.

“Would you run your database in a container?” In development, always — it makes the environment reproducible in one command. In production I would default to a managed service. The reason is not that it cannot be done; it is that everything containers are optimised for is the opposite of what a database needs. Containers assume the instance is disposable and replaceable; a database is the one thing in the system that is not. Doing it yourself means owning persistent volumes, backup and restore, failover, and the ordering of version upgrades, all inside a platform that may reschedule you onto another machine. If it had to be self-hosted, I would use a StatefulSet with persistent volume claims, or run it on dedicated instances outside the cluster.

“What actually makes an image reproducible?” The layers are content-addressed — each is identified by the hash of its contents — and the image is identified by the hash of that stack. Pulling `api@sha256:a3f8b1...` gets you exactly the same bytes anywhere in the world. That is what makes rollback trivial, because the previous image still exists byte for byte, and it is why the commit hash rather than `latest` should be the tag you deploy. Reproducibility of the *build* is a weaker claim: `pip install` at two different times can resolve different versions, so the build needs pinned dependencies and a lock-file to genuinely produce the same image twice.

A model answer

“Docker solves two separate problems, and it’s worth splitting them.

The first is packaging. Before containers, deploying meant a list of instructions — install this runtime, then these fourteen libraries at these versions, then set these variables — and that list drifted between a laptop, a test machine and production. An image is that whole list already carried out and frozen into one artefact: the code, the dependencies pinned, the language runtime, the OS libraries. Same bytes everywhere. That’s what actually kills ‘works on my machine’.

The second is density and speed, and that comes from what a container is. A container isn’t a small virtual machine — it’s an ordinary process on the host’s kernel, with two things done to it. Namespaces restrict what it can see: its own process list, its own filesystem, its own network interfaces and ports, its own hostname. cgroups restrict what it can use: half a CPU, 512 MB of memory, and the kernel enforces both.

A virtual machine, by contrast, is an emulated computer running its own guest kernel on a hypervisor. That’s the only fundamental difference, and every practical difference follows from it. A VM boots in thirty to sixty seconds because there’s a kernel to boot; a container starts in about a hundred milliseconds because there isn’t. A VM costs half a gigabyte to a gigabyte of memory before your application starts; a container costs a few megabytes. On a 64 GB host running a 500 MB service, that’s about 42 VMs against about 126 containers.

Three hundred times faster to start is what makes autoscaling and per-request pricing possible at all — a spike is over before a VM finishes booting.

The trade-off I’d raise unprompted is that the shared kernel is both the benefit and the risk. A kernel vulnerability is a route out of the container and onto the host, whereas escaping a VM means defeating the hypervisor, which is a far smaller attack surface. That’s precisely why AWS runs Lambda and Fargate on Firecracker microVMs rather than plain containers — when the code belongs to strangers, container isolation isn’t considered sufficient. And for my own workloads I’d still run as a non-root user with dropped capabilities and a read-only root filesystem, because ‘it’s isolated’ is doing less work than people assume.”

That answer defines the thing correctly, names the mechanisms, contrasts with the VM using one structural fact rather than a list, supports it with arithmetic, and closes on the limitation.

9 Recall card

1. **A container is a process, not a computer.** Namespaces limit what it can **see**; cgroups limit what it can **use**; the image gives it its own filesystem. No guest kernel.
2. **The shared kernel explains everything** — 100 ms starts against 45 s, a few MB of overhead against a gigabyte, hundreds per host against tens, and why a Linux container needs a Linux kernel.
3. **Image = template, container = running instance.** Layers are read-only, shared and content-addressed; the writable layer on top dies with the container, so state goes in a volume or an external store.
4. **Docker solved packaging and density.** The image kills environment drift; the process model makes autoscaling possible.
5. **The shared kernel is also the weakness.** Untrusted code gets a real boundary — Firecracker, gVisor, or a VM. Run as non-root, drop capabilities, never mount the Docker socket.

DSA topic: Reversing, rotating, and swapping in place **System design topic:** Containers and why everyone uses Docker

Code these, in this order

Four problems built from the same two moves: swap two positions, and walk two markers towards each other. Three of them are the reversal trick in different clothes.

Before you write anything, for each one:

1. Say whether the function must **mutate** its input or **return** a new one. If it mutates, remember that `items = ...` will not work.
2. Say what happens when the input is empty, and when it has one element.
3. If there is a `k`, say what happens when `k` is `0`, when `k == n`, and when `k > n`.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Reverse String	LeetCode 344 (Easy)	The two-pointer loop, and the bound. <code>while left < right</code> , <code>n // 2</code> swaps. Write it once and never get it wrong again.
2	Rotate String	LeetCode 796 (Easy)	Whether you understand what a rotation <i>is</i> . There is a one-line answer using <code>s + s</code> , and finding it is the point.
3	Rotate Array	LeetCode 189 (Medium)	Today's question, exactly as an interviewer asks it. Do it three ways — extra array, one-at-a-time, three reversals — and time all three.
4	Reverse Words in a String	LeetCode 151 (Medium)	The three-reversal idea again: reverse everything, then reverse each word. Also a lesson in messy input — leading, trailing and repeated spaces.

On problem 3, do it properly

- Write the slice version first. Submit it. It passes.
- Then re-read the constraint: “*Could you do it in-place with $O(1)$ extra space?*” Write that one too.
- Then answer, out loud: **why do three reversals produce a rotation?** If you cannot say it in two sentences without looking, you have memorised the code and not the idea.
- Then run both on a list of a million elements and time them. Notice that the version you were asked for is the slower of the two, and be able to say why that is still the right answer.

On problem 4, notice the shape

" the sky is blue " must come out as "blue is sky the". Solve it twice:

- **The easy way:** `" ".join(reversed(s.split()))`. Correct, $O(n)$, and $O(n)$ space.
- **The interview way:** strip and squash the spaces in place, reverse the whole thing, then reverse each word. This is the same three-reversal skeleton as problem 3, and the reason interviewers like it is that the second reversal is over a variable-length group.

The swap drill

Answer these in your head, then check:

1. `a, b = b, a` — why does this work without a temporary variable, and what would you write in Java?
2. Reversing a 9-element list: how many swaps, and which position never moves?
3. Reversing a 10-element list: how many swaps?
4. `items[i], items[j] = items[j], items[i]` when `i == j` — what happens?

The rotation drill

For `[1, 2, 3, 4, 5, 6, 7, 8]`, say the answer out loud before working it out:

1. Rotate right by 3.
2. Rotate left by 3.
3. Rotate right by 8.
4. Rotate right by 11.
5. Rotate right by -3 . (What does Python's `%` do here, and what would Java do?)

Then, for each of those, say what `k % n` becomes.

The in-place drill

Predict the output before running it:

```
def rotate(items, k):
    k %= len(items)
    items = items[-k:] + items[:-k]

data = [1, 2, 3, 4, 5]
rotate(data, 2)
print(data)
```

Then fix it with a one-character change, and say in one sentence what the difference is between `items = ...` and `items[:] = ...`.

The one to try in a terminal

If Docker is installed, run these four and read the output rather than skipping past it:

```
docker run --rm -it alpine sh -c "ps aux"
docker run --rm alpine uname -r
uname -r
docker run --rm --memory=32m python:3.12-slim python -c "x = ' ' * 100_000_000"
```

Then answer: how many processes did the first one see, and why? Why do the second and third print the same thing? And what killed the fourth — and which of namespaces or cgroups was responsible for each of those three answers?

If Docker is not installed, answer them from the lesson instead. They are the whole of §3.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Rotate the array to the right by k , in $O(1)$ extra space.* Say the constraint back, reject the two naive versions with numbers, give the three reversals, and then **explain why they work** without being asked.
2. *What problem does Docker actually solve?* Give both problems — packaging and density. Define a container as a process, name namespaces and cgroups, and land the two numbers: 100 ms against 45 s, and a few MB against a gigabyte.
3. *What is the difference between a container and a virtual machine?* Answer with one structural fact — the shared kernel — and then derive start time, memory, density and the Windows-on-Linux question from it. Finish on the trade-off: the shared kernel is also the weakness.

Before you move on

- [] I can write the two-pointer reversal from memory, with the right bound, first time.
- [] I say `k %= n` before anything else, and I know why the empty check comes before it.
- [] I can explain the three-reversal trick in two sentences, out loud, with no notes.
- [] I know that `items = ...` inside a function changes nothing for the caller.
- [] I can name namespaces and cgroups and say which one limits *seeing* and which limits *using*.

- [] I can draw the VM stack against the container stack — in any tool I like — and point at the one box that is missing.

Max, min, second largest: the single-pass habit

DATA STRUCTURES AND ALGORITHMS

Max, min, second largest: the single-pass habit

You solve find-the-two-largest in one pass instead of reaching for sort.

SYSTEM DESIGN

Fundamentals revision and interview questions

You can answer the ten most common fundamentals questions cold.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the second largest element without sorting.*
- *Pick any topic from days 1 to 13 and answer it with no preparation.*

Max, min, second largest: the single-pass habit

1 What this is, and why they ask it

If a question asks you for a **small, fixed number of extreme values** — the largest, the largest and the smallest, the two largest — you do not need the data in order. You need one walk through it, carrying that many variables. Sorting puts every element in its place; you only ever wanted two of them.

The words “*without sorting*” in the question are not an arbitrary restriction. They are the whole test. `sorted(items)[-2]` is $O(n \log n)$ and one line, and any candidate can produce it. The interviewer wants to see whether you can get the same answer in $O(n)$ with two variables, and — far more revealing — whether you notice that “**second largest**” is **ambiguous** and ask which one they mean before writing anything.

This appears as the opening question in phone screens everywhere, and the “find the k-th largest” family it belongs to runs all the way to [day 055](#) and [day 113](#). It is also the day the single-pass reflex gets installed: **when the answer is a handful of values, make one pass and keep a handful of trackers**.

2 The story

Ravi teaches games at a school in Nagpur, and the annual sports day is the second Saturday of December. The long jump is his event. Forty-eight children in the under-fourteen group, one jump each, and he needs to be able to say who came first and who came second before the parents start leaving at four.

For the first few years he did it the obvious way. Every child jumped, he measured, and he typed the number into his phone. Then at the end he had forty-eight numbers to put in order, with three teachers talking to him and a child crying about a foul, and it took him longer to sort out the list than it had taken to run the whole event.

Now he does something else, and he does not keep forty-eight numbers at all. He keeps two, in his head.

Before the first child jumps, he has nothing. The first child jumps 3.72 metres, so that is the best so far, and there is no second best yet. The next child jumps 3.55, which is not better than 3.72, so it becomes the second best. The third jumps 4.12, and this is the moment that matters: the old best, 3.72, does not disappear. **It slides down and**

becomes the second best, and 4.12 becomes the best. The fourth child jumps 3.90, which is not better than 4.12 but is better than 3.72, so it takes the second place and 3.72 is gone for good.

He carries on like that. Most jumps change nothing at all — he hears the number, sees that it is smaller than both, and forgets it before the child is out of the pit. By the forty-eighth jump he has the two numbers he needs and he never had to put anything in order.

One thing did catch him out, the year a boy and a girl both jumped exactly 4.55. He announced the boy first and the girl second, and the girl's father asked, quite reasonably, how she could be second with the same jump. Now, before the event starts, Ravi asks the head teacher which way they want it: two firsts and no second, or a second place that has to be a genuinely shorter jump. It takes ten seconds to ask and it saves the argument.

3 The idea in plain English

Ravi's two numbers are two variables, and his forty-eight jumps are one pass over an array. That is the whole technique. What is left is getting the details right, and there are four of them.

One pass, one tracker: the maximum

```
best = items[0]
for x in items[1:]:
    if x > best:
        best = x
```

Start with the first element, walk the rest, keep the bigger. After the loop, `best` holds the largest. That is `n - 1` comparisons, `O(n)` time, and `O(1)` extra space — one variable, however long the list is.

Notice what `best` starts as. It starts as a real element of the list, not as `0`. Starting at `0` is the single most common bug in this family, and §7 shows it breaking on an all-negative input.

If you must start from something that is not an element, use `float('-inf')` — a value guaranteed to be smaller than every real number, so the first comparison always wins. There is `float('inf')` for the other direction.

Two trackers: the largest and the smallest, together

You do not need two passes for two answers.


```

biggest = smallest = items[0]
for x in items[1:]:
    if x > biggest:
        biggest = x
    elif x < smallest:
        smallest = x

```

`elif` and not a second `if`, and it is worth knowing why: if `x` is bigger than the biggest so far, it cannot possibly be smaller than the smallest so far, so checking is wasted work. That makes it 1 comparison in the good case and 2 in the bad one, which is between n and $2n$ comparisons overall rather than the $2n$ you would pay with two separate loops.

Both versions are $O(n)$. The reason to prefer one pass is not the complexity class — it is that you touch the data once, which matters when “the data” is a file being streamed or a result set arriving over the network and you cannot walk it twice.

The second largest, and the question you must ask first

Here is `[5, 5, 3]`. What is the second largest?

- 5, if “second largest” means the second element when you lay them out in order.
- 3, if it means the second largest *distinct value*.

Both are defensible, both appear in real problem statements, and the two answers are different. **Ask.** This is exactly Ravi and the father at the sand pit, and it is exactly the habit from [day 008](#).

The distinct version, which is what interviewers usually mean:

```

first = second = None
for x in items:
    if first is None or x > first:
        first, second = x, first      # the old best slides down
    elif x != first and (second is None or x > second):
        second = x

```

Read the two branches carefully, because between them they are the whole problem.

The first branch fires when `x` beats the current best. The important half is `second = first` — the old champion is not thrown away, it becomes the runner-up. Ravi’s 3.72 sliding down to second place when 4.12 arrives is this line. Writing `first = x` and then `second = first` on a separate line is a real bug, because by then `first` is already the new value; §7 shows it returning the largest twice.

The second branch fires when `x` is not the best but might be the runner-up. Missing this branch entirely is the other classic bug: on `[10, 1, 2]` the first branch fires only once, on the 10, and `second` is never touched again.

The `x  $\neq$  first` test is what makes it *distinct*. Drop it and you get the by-position version:

```
if first is None or x > first:
    first, second = x, first
elif second is None or x > second:
    second = x
```

Two lines apart, two different contracts, and the problem statement decides.

The habit, stated generally

If the answer is a fixed, small number of extreme values, use one pass and that many trackers. Sorting arranges all `n`; you only wanted `k` of them.

For `k = 1, 2` or `3`, write the trackers by hand. Beyond that the code becomes unreadable and error-prone, and the right structure is a **heap** of size `k`, giving $O(n \log k)$ — that is [day 113](#). And if `k` is a large fraction of `n`, sorting is genuinely the right call, or **quickselect** in $O(n)$ average, which is [day 055](#).

<code>k = 1 or 2 or 3</code>	$\rightarrow$	trackers, $O(n)$
<code>k small but > 3</code>	$\rightarrow$	heap of size <code>k</code> , $O(n \log k)$
<code>k close to n</code>	$\rightarrow$	just sort, $O(n \log n)$

The comparison-count question

Some interviewers push further: “can you find both the max and the min in fewer than $2n$ comparisons?” Yes, and the trick is to take elements **in pairs**.

Compare the two elements of a pair to each other first — one comparison. The winner can only ever be the new maximum, and the loser can only ever be the new minimum, so each needs one comparison instead of two. That is **3 comparisons per 2 elements**, so $3n/2$ rather than $2n$:

<code>n = 1,000,000</code>	naive pairwise loop :	1,999,986 comparisons	
	pairwise version :	1,499,998 comparisons	$\rightarrow$ 25% fewer

It is the same $O(n)$. Mention it as a refinement if asked for fewer comparisons; do not lead with it.

The library versions, and their cost

```
max(items)           # 0(n), raises ValueError on empty
min(items)           # 0(n), raises ValueError on empty
max(items, default=0) # 0(n), returns the default instead of raising
sorted(items)[-2]     # 0(n log n), IndexError if len < 2
heapq.nlargest(2, items) # 0(n log 2), returns a list
```

`max` and `min` are the same single pass written in C, so use them when the contract fits. Their failure mode on an empty list is an exception, which is the same shape of problem as `list.index()` on [day 012](#), and `default=` is the clean fix.

4 The picture

The two trackers walking the jumps, one child at a time:

jump	3.72	3.55	4.12	3.90	3.61	4.55	3.44
	----	----	----	----	----	----	----
first	3.72	3.72	4.12	4.12	4.12	4.55	4.55
second	--	3.55	3.72	3.90	3.90	4.12	4.12
			^^^^		^^^^		

the old first slides down
nothing changes: 3.61 beats neither

7 jumps, 2 variables, 1 walk. Nothing is ever put in order.

What to notice: the third column. When a new best arrives, the old best does not vanish — it moves into second place. That single line, `second = first`, is what the whole exercise is about, and it is the line people write in the wrong order.

The two branches, drawn as a decision:

```
for each x:
    is x > first ?
    /      \
  yes      no
  |        |
second = first
first = x   is x > second (and, if distinct, x ≠ first) ?
            /      \
          yes      no
          |        |
        second = x  do nothing
                     (most elements land here)
```

What to notice: there are three outcomes, not two, and the third one — do nothing — is where the great majority of elements go. Forgetting the middle branch is the bug that survives the example the interviewer showed you, because it only appears when the largest element comes early.

Why sorting is the wrong tool, drawn:

```
you asked for    : the top 2 of 1,000,000
sorting gives    : [ #1 #2 #3 #4 #5 ..... #1,000,000 ]
                  ^^^^^^^
                  you wanted these two
                    999,998 positions you paid for and did not use

one pass gives   : first = #1, second = #2
                  1,000,000 comparisons, 2 variables
```

What to notice: sorting solves a strictly larger problem than the one you were asked. The cost of the extra work is the `log n` factor — 20 at a million elements — and `O(n)` extra space, since `sorted()` builds a new list.

And the ambiguity, which is the part to raise out loud:

```
items = [5, 5, 3]

"second largest" as second in sorted order → 5   (5, 5, 3)
                                              ^
"second largest DISTINCT value"           → 3   (5, 3)
                                              ^

items = [4, 4, 4, 4]

as second in sorted order → 4
as second distinct value  → there isn't one. None? An exception? Ask.
```

What to notice: on `[4, 4, 4, 4]` the distinct contract has no answer at all, so the function needs a return convention for “does not exist” — the same conversation as [day 012](#).

5 The code, built step by step

The maximum, with the initialisation done properly.

```
def largest(items: list[int]) → int:
    """The maximum. O(n) time, O(1) space. Raises on empty input."""
    if not items:
        raise ValueError("largest() of an empty list")
    best = items[0]
    for x in items[1:]:
        if x > best:
            best = x
    return best
```

`best = items[0]` rather than `best = 0`. The empty case is checked explicitly and raises, matching what `max()` itself does — an empty list has no maximum, and inventing one is worse than refusing.

Both extremes in one walk.

```
def largest_and_smallest(items: list[int]) → tuple[int, int]:
    """Both, in one pass. Two trackers, one walk. O(n) time, O(1) space."""
    if not items:
        raise ValueError("largest_and_smallest() of an empty list")
    biggest = smallest = items[0]
    for x in items[1:]:
        if x > biggest:
            biggest = x
        elif x < smallest:
            smallest = x
    return biggest, smallest
```

`biggest = smallest = items[0]` sets both to a real element, so neither can be wrong for lack of a starting point. The `elif` saves a comparison whenever the first test succeeds.

The second largest, distinct values.

```
def second_largest_distinct(items: list[int]) → int | None:
    """Second largest DISTINCT value. None if there are fewer than two distinct values."""
    first = second = None
    for x in items:
        if first is None or x > first:
            first, second = x, first # the old best slides down
        elif x ≠ first and (second is None or x > second):
            second = x
    return second
```

`first, second = x, first` is one statement, so the right-hand side is built before either assignment happens — no ordering bug is possible. Using `None` rather than `float('-inf')` means the function works on any comparable type, not only numbers, and makes “there is no answer” explicit in the return type.

The other contract, one condition apart.

```
def second_largest_by_position(items: list[int]) → int | None:
    """Second element in sorted order, duplicates counted. [5, 5, 3] → 5."""
    if len(items) < 2:
        return None
    first = second = None
    for x in items:
        if first is None or x > first:
            first, second = x, first
        elif second is None or x > second:
            second = x
    return second
```

The only differences are the `len(items) < 2` guard and the missing `x ≠ first`. Put both functions side by side in an interview and the interviewer can see you understood the question rather than guessed at it.

Here is the complete program.

```

"""Day 14 – one pass, a few trackers, no sorting."""

import random
import time

def largest(items: list[int]) → int:
    """The maximum. O(n) time, O(1) space. Raises on empty input."""
    if not items:
        raise ValueError("largest() of an empty list")
    best = items[0]
    for x in items[1:]:
        if x > best:
            best = x
    return best

def largest_and_smallest(items: list[int]) → tuple[int, int]:
    """Both, in one pass. Two trackers, one walk. O(n) time, O(1) space."""
    if not items:
        raise ValueError("largest_and_smallest() of an empty list")
    biggest = smallest = items[0]
    for x in items[1:]:
        if x > biggest:
            biggest = x
        elif x < smallest:
            smallest = x
    return biggest, smallest

def second_largest_distinct(items: list[int]) → int | None:
    """Second largest DISTINCT value. None if there are fewer than two distinct values."""
    first = second = None
    for x in items:
        if first is None or x > first:
            first, second = x, first # the old best slides down
        elif x != first and (second is None or x > second):
            second = x
    return second

def second_largest_by_position(items: list[int]) → int | None:
    """Second element in sorted order, duplicates counted. [5, 5, 3] → 5."""
    if len(items) < 2:
        return None
    first = second = None
    for x in items:
        if first is None or x > first:
            first, second = x, first
        elif second is None or x > second:
            second = x
    return second

def count_comparisons_naive(items: list[int]) → int:
    """Comparisons used by the obvious two-tracker loop."""
    if not items:
        return 0
    biggest = smallest = items[0]
    used = 0
    for x in items[1:]:
        used += 1
        if x > biggest:
            biggest = x
        else:
            used += 1
            if x < smallest:
                smallest = x
    return used

def count_comparisons_pairs(items: list[int]) → int:
    """Comparisons used by the pairwise version: compare two, then each to one end."""
    n = len(items)
    if n == 0:

```

```

        return 0
    used = 0
    if n % 2 == 1:
        i = 1
    else:
        used += 1
        i = 2
    while i < n:
        used += 3
        i += 2
    return used

if __name__ == "__main__":
    jumps = [412, 388, 455, 455, 390, 401, 372]
    print(f"jumps          : {jumps}")
    print(f"largest         : {largest(jumps)}")
    print(f"largest, smallest: {largest_and_smallest(jumps)}")
    print(f"second (distinct): {second_largest_distinct(jumps)}")
    print(f"second (position): {second_largest_by_position(jumps)}")

    print("\nthe two contracts disagree, and the problem statement decides")
    CASES: list[tuple[str, list[int]]] = [
        ("normal", [3, 9, 1, 7]),
        ("top value repeated", [5, 5, 3]),
        ("all identical", [4, 4, 4, 4]),
        ("two elements", [2, 8]),
        ("one element", [6]),
        ("empty", []),
        ("all negative", [-7, -2, -9]),
        ("zero and negatives", [0, -3, -1]),
        ("largest is first", [10, 1, 2]),
        ("largest is last", [1, 2, 10]),
    ]
    print(f"  {case':<20}{input':<18}{distinct':>10}{by position':>14}")
    for label, data in CASES:
        d = second_largest_distinct(data)
        p = second_largest_by_position(data)
        print(f"  {label:<20}{str(data):<18}{str(d):>10}{str(p):>14}")

    print("\ncomparisons: naive two-tracker vs pairwise")
    random.seed(14)
    for n in (10, 1_000, 1_000_000):
        data = [random.randint(0, 10**9) for _ in range(n)]
        naive = count_comparisons_naive(data)
        pairs = count_comparisons_pairs(data)
        print(f"  n = {n:>9},  naive {naive:>10},  pairwise {pairs:>10},"
              f"    saved {100 * (naive - pairs) / naive:>4.0f}%")

    print("\nsingle pass vs sorting (n = 2,000,000)")
    data = [random.randint(0, 10**9) for _ in range(2_000_000)]
    t0 = time.perf_counter(); a = second_largest_distinct(data); onepass = time.perf_counter() - t0
    t0 = time.perf_counter(); b = sorted(set(data))[-2]; bysort = time.perf_counter() - t0
    print(f"  one pass          : {onepass:>8.4f} s → {a}")
    print(f"  sorted(set(...)) : {bysort:>8.4f} s → {b}")
    print(f"  same answer?      : {a == b}")
    print(f"  one pass is {bysort / onepass:.1f}x faster and uses O(1) extra space")

    print("\nis the one-pass version right? 5000 random cases against a slow reference")
    mismatches = 0
    for _ in range(5000):
        size = random.randint(0, 8)
        data = [random.randint(-5, 5) for _ in range(size)]
        distinct = sorted(set(data), reverse=True)
        expected = distinct[1] if len(distinct) ≥ 2 else None
        if second_largest_distinct(data) ≠ expected:
            mismatches += 1
    print(f"  mismatches: {mismatches}")

```

This is exactly what it printed:


```
jumps          : [412, 388, 455, 455, 390, 401, 372]
largest         : 455
largest, smallest: (455, 372)
second (distinct): 412
second (position): 455
```

the two contracts disagree, and the problem statement decides

case	input	distinct	by position
normal	[3, 9, 1, 7]	7	7
top value repeated	[5, 5, 3]	3	5
all identical	[4, 4, 4, 4]	None	4
two elements	[2, 8]	2	2
one element	[6]	None	None
empty	[]	None	None
all negative	[-7, -2, -9]	-7	-7
zero and negatives	[0, -3, -1]	-1	-1
largest is first	[10, 1, 2]	2	2
largest is last	[1, 2, 10]	2	2

```
comparisons: naive two-tracker vs pairwise
n =          10  naive          15  pairwise          13  saved    13%
n =       1,000  naive       1,991  pairwise       1,498  saved    25%
n = 1,000,000  naive 1,999,986  pairwise 1,499,998  saved    25%
```

```
single pass vs sorting (n = 2,000,000)
one pass      : 0.2250 s → 999998818
sorted(set(...)) : 1.5771 s → 999998818
same answer?   : True
one pass is 7.0x faster and uses O(1) extra space
```

```
is the one-pass version right? 5000 random cases against a slow reference
mismatches: 0
```

Look at the two rows `top value repeated` **and** `all identical`. Same input, two different answers, and on `[4, 4, 4, 4]` one of the contracts has no answer at all. That is the sentence to open the interview with.

Look at `all negative`. The answer is `-7`, and any version that starts its trackers at `0` returns `0` — a number that is not in the list.

Look at the timing. Seven times faster, and the one-pass version also uses `O(1)` extra space against the `O(n)` that `sorted(set(...))` needs for two new collections. The gap is partly the `log n` factor and partly that sorting moves every element.

6 What it costs

The maximum. The loop body runs once per element after the first, and does one comparison:

$n - 1$ comparisons, exactly. Always. Best case = worst case.

$O(n)$ time, $O(1)$ extra space — one variable. There is **no way to do better**, and the reason is worth being able to say: any element you have not looked at could be the largest, so you must look at all of them. Same argument as absence in linear search on [day 012](#).

Both extremes, naively. Two comparisons per element in the worst case:

$2 \times (n - 1) = 2n - 2$ comparisons

The measured run gave 1,999,986 at $n = 1,000,000$, which is $2n - 14$ — a handful fewer, because sometimes the first test succeeds and the second is skipped.

Both extremes, pairwise. Take elements two at a time. One comparison to see which of the pair is larger, then one comparison of the winner against the running maximum and one of the loser against the running minimum:

3 comparisons for every 2 elements

$n / 2$ pairs $\times 3 = 3n / 2$ comparisons

At $n = 1,000,000$:

$3 \times 1,000,000 / 2 = 1,500,000$ against 2,000,000
→ 25% fewer, which is what the run measured

Still $O(n)$. This is a constant-factor improvement, and the honest framing is: “*same complexity, 25% fewer comparisons, and it is the answer if you’re asked to minimise comparisons.*”

The second largest. One pass, at most two comparisons per element:

between n and $2n$ comparisons
 $O(n)$ time, $O(1)$ extra space — two variables

Against sorting. At $n = 2,000,000$:

one pass : $\sim 2,000,000$ comparisons, 2 extra variables
sorted() : $n \log_2 n = 2,000,000 \times 21 = 42,000,000$ comparisons,
 plus a new list of 2,000,000 references = 16 MB

Twenty-one times the comparisons by that count; seven times slower when measured, because Python’s `sorted` is heavily optimised C and the one-pass loop is interpreted.

Both numbers are worth saying. The $\log n$ factor is real, and the constant factors

work against you, which is why the honest claim is “asymptotically better and measurably faster here”, not “twenty-one times faster”.

The general shape, which is the thing to remember:			
YOU WANT	METHOD	TIME	EXTRA SPACE
The largest	1 tracker, one pass	$O(n)$	$O(1)$
Largest and smallest	2 trackers, one pass	$O(n)$, $3n/2$ comparisons at best	$O(1)$
The two or three largest	2–3 trackers, one pass	$O(n)$	$O(1)$
The k largest, k small	Heap of size k — day 113	$O(n \log k)$	$O(k)$
The k -th largest, any k	Quickselect — day 055	$O(n)$ average	$O(1)$
Everything in order	Sort	$O(n \log n)$	$O(n)$

7 The traps

Trap one: initialising the tracker to zero

It looks harmless, it passes every example an interviewer is likely to show you, and it is wrong.

```
def largest(items):
    best = 0 # looks harmless
    for x in items:
        if x > best:
            best = x
    return best

print(largest([-7, -2, -9]))
```

0

0 is not in the list. Every element failed the `x > best` test, so the starting value was returned unchanged. Temperatures, account balances, profit-and-loss figures and coordinates are all routinely negative, and this bug ships.

Two correct fixes. Start from a real element:

```
best = items[0]
```

Or start from something guaranteed to lose:

```
best = float("-inf")
```

The rule: never initialise an extreme-value tracker to a made-up number. Use the first element, or use infinity, and handle the empty case separately.

Trap two: assigning in the wrong order

The near-miss. It runs, it returns a number, and the number is wrong.

```
def second_largest(items):
    first = second = float("-inf")
    for x in items:
        if x > first:
            first = x
            second = first          # assigns the NEW first, not the old one
    return second

print(second_largest([3, 9, 1, 7]))
```

9

It returned the **largest**, twice. By the time `second = first` runs, `first` has already been overwritten with `x`, so the old value is gone. The two lines are in the order they read in English and the wrong order for the machine.

Two fixes. Save the old value first:

```
second = first
first = x          # order reversed
```

Or use one statement, so the ordering question cannot arise:

```
first, second = x, first          # right-hand side is built before anything is assigned
```

The second form is the one to write. It is the same tuple-unpacking rule as the swap on [day 013](#).

Trap three: no second branch

Also silent, and it only fails on inputs where the largest element comes early — which is not the example anyone tests with.

```
def second_largest(items):
    first = second = float("-inf")
    for x in items:
        if x > first:
            first, second = x, first
    return second # no elif branch at all

print(second_largest([10, 1, 2]))
```

```
-inf
```

The `if` fired once, on the 10. After that nothing beat `first`, so `second` was never updated and the starting sentinel leaked out into the answer. On `[1, 2, 10]` the same function returns `2` and looks perfectly correct, which is exactly why this survives casual testing.

The fix is the branch that handles “not the best, but maybe the runner-up”:

```
elif x != first and (second is None or x > second):
    second = x
```

How to catch it every time: test with the largest element first in the list. That input breaks more second-largest implementations than any other.

Trap four: the built-ins on short input

Both convenient one-liners have a failure mode on small lists.

```
scores = []
print(max(scores))
```

```
Traceback (most recent call last):
  File "t14d.py", line 2, in <module>
    print(max(scores))
          ^^^^^^^^^^^
ValueError: max() iterable argument is empty
```

```
scores = [6]
print(sorted(scores)[-2])
```

```
Traceback (most recent call last):
  File "t14e.py", line 2, in <module>
    print(sorted(scores)[-2])
          ~~~~~^
IndexError: list index out of range
```

Two different exceptions from two different one-liners, both on input a real caller will eventually send. `max(items, default=None)` fixes the first. The second needs a length check — and if you were going to write the check anyway, the hand-written pass is no longer any longer than the shortcut.

8 In the interview

How it gets asked

- “Find the second largest element without sorting.” — the direct version. The last three words are the constraint that makes it a question at all.
- “Find the maximum and the minimum in one pass. Can you do it in fewer than $2n$ comparisons?” — the comparison-count version.
- “Find the k -th largest element.” — the same family, one step harder. For small `k`, trackers; otherwise a heap or quickselect.
- “What’s the second highest salary?” — the SQL phrasing of exactly this, and it has the same duplicates question hiding inside it.

What to say out loud, in the first ninety seconds

1. **Ask the ambiguity question before anything else.** “Quick check on the contract — for `[5, 5, 3]`, do you want 5 or 3? That is, second largest by position, or second largest distinct value?” This is the sentence the question exists to elicit.
2. **Ask about the empty and one-element cases.** “And if there aren’t two distinct values — should I return None or raise?”
3. **State the approach.** “One pass, two variables: the best so far and the second best so far. No sorting needed, because I only want two of the n positions sorting would give me.”
4. **Name the tricky line before you write it.** “The step people get wrong is that when a new maximum arrives, the old maximum has to slide down into second place. I’ll write it as a single tuple assignment so the ordering can’t bite me.”
5. **Name the second branch.** “And there’s a second case: a value that doesn’t beat the best but does beat the runner-up. Missing that branch is why `[10, 1, 2]` fails while `[1, 2, 10]` passes.”
6. **Give the costs, and the comparison to sorting.** “ $O(n)$ time, $O(1)$ extra space — two variables regardless of size. Sorting would be $O(n \log n)$ and $O(n)$ space to solve a strictly bigger problem than the one I was asked.”
7. **Say what you would do for general `k`.** “For the k -th largest with k small, a heap of size k is $O(n \log k)$. For arbitrary k , quickselect is $O(n)$ on average.”

The follow-ups

“What does `[5, 5, 3]` return?” That depends on the contract, and it is the first thing I would pin down. If “second largest” means the second element in sorted order, the answer is 5, because the two 5s occupy the first two places. If it means the second largest distinct value, the answer is 3. Both are used in real problem statements. The implementations differ by one condition — `x  $\neq$  first` in the second branch — so I would write whichever you want and note the other. And on `[4, 4, 4, 4]` the distinct version has no answer, so it needs a return convention: I would use `None`, with `int | None` on the signature so the caller is forced to check.

“Can you find the max and min in fewer than $2n$ comparisons?” Yes, in $3n/2$. Take the elements in pairs. Compare the two members of the pair to each other first — that costs one comparison and tells you which of them could possibly be a new maximum and which could possibly be a new minimum. Then compare only the winner against the running maximum and only the loser against the running minimum. Three comparisons for every two elements instead of four. At a million elements that is 1.5 million rather than 2 million, about 25% fewer. It is the same $O(n)$; it is a constant-factor answer to a constant-factor question.

“Now find the k -th largest.” Three answers, depending on `k`. If `k` is 2 or 3, extend the trackers — still $O(n)$ and $O(1)$. If `k` is small relative to `n`, keep a min-heap of size `k`: push each element, and pop when the heap grows past `k`, so the heap always holds the `k` largest seen so far and its root is the answer. That is $O(n \log k)$ time and $O(k)$ space. If `k` is arbitrary or close to `n`, quickselect partitions around a pivot and recurses into one side only, giving $O(n)$ on average and $O(n^2)$ in the worst case unless you randomise the pivot. In production Python I would write `heapq.nlargest(k, items)` and say so.

“What if the data doesn’t fit in memory, or arrives as a stream?” The one-pass version is already the answer, and that is its real advantage over sorting. It needs $O(1)$ memory and touches each element exactly once, so it works on a file read line by line, or on records arriving over a socket, where you cannot go back and look again. Sorting needs the whole dataset at once — for data that does not fit, that means an external merge sort with disk passes. This is the case where the two approaches are not merely different in complexity but different in what is possible.

A model answer

“First, a contract question, because ‘second largest’ is ambiguous. On `[5, 5, 3]`, do you want 5 — the second element in sorted order — or 3, the second largest distinct value? And if there is no such element, should I return `None` or raise?

...Right, distinct values, and `None` if there isn’t one.

I don’t need the data sorted. Sorting arranges all n elements and I only want two of them, so that’s $O(n \log n)$ and $O(n)$ extra space to solve a bigger problem than I was asked. One pass with two variables does it in $O(n)$ and $O(1)$.

```
def second_largest(items: list[int]) → int | None:
    first = second = None
    for x in items:
        if first is None or x > first:
            first, second = x, first
        elif x ≠ first and (second is None or x > second):
            second = x
    return second
```

Two branches, and both matter. The first fires when x beats the current best — and the important half is that the old best slides down into second place rather than being thrown away. I’ve written it as a single tuple assignment, because writing `first = x` and then `second = first` on separate lines gives you the new value twice; that’s a real bug and it returns the maximum rather than the runner-up.

The second branch fires when x isn’t the best but might be the runner-up. Leaving it out is the classic failure: `[1, 2, 10]` still works, but `[10, 1, 2]` returns the sentinel, because after the first element nothing ever beats the maximum again.

The `x ≠ first` is what makes it distinct — drop it and you get the by-position contract.

Edge cases: empty list returns `None`. One element returns `None`. All-equal returns `None`, which is right for the distinct contract. Starting from `None` rather than 0 matters — a tracker initialised to 0 returns 0 on an all-negative list, which is a value that isn’t in the input.

Cost: n comparisons in the good case, up to $2n$ in the bad one. $O(n)$ time, $O(1)$ extra space — two variables however big the list is. And it works on a stream, which sorting doesn’t, because it touches each element once and never looks back.

If you wanted the k -th largest instead: for small k , a min-heap of size k gives $O(n \log k)$; for arbitrary k , quickselect gives $O(n)$ on average.”

That answer opens with the ambiguity, rejects sorting with a reason rather than a rule, names both bug-prone lines *before* writing them, checks the edges, gives both costs, and generalises at the end.

9 Recall card

1. **A fixed number of extreme values needs one pass and that many trackers.** Sorting arranges all `n` when you wanted `k`. `O(n)` time, `O(1)` space.
2. **Ask what “second largest” means on `[5, 5, 3]`** — 5 by position, 3 by distinct value. Two contracts, one condition apart, and `[4, 4, 4, 4]` has no distinct answer at all.
3. **When a new best arrives, the old best slides down.** Write it as `first, second = x, first` so the ordering bug cannot happen.
4. **The second branch is not optional.** Without `elif ... x > second`, `[10, 1, 2]` returns the sentinel. Always test with the largest element first.
5. **Never initialise a tracker to 0.** Use `items[0]` or `float('-inf')`, and handle empty separately — `max([])` raises `ValueError: max() iterable argument is empty`.

1 What this is, and why they ask it

Days 1 to 13 covered the whole path a request takes: a name becoming a machine, a connection being opened, bytes arriving at a process, that process being scheduled by an operating system on real hardware, and the code inside it having got there through a build and a deployment. Today converts that from something you have read into something you can say.

This is not a summary. It is a **drill**, because the gap between understanding a thing and producing it out loud under mild stress is much wider than anyone expects, and interviews only ever test the second one.

Interviewers ask fundamentals questions in three places. As a warm-up in the first five minutes of a system design round, to calibrate you. As the entire content of some phone screens, particularly at infrastructure-heavy companies. And as a probe in the middle of a design question, when you say “the load balancer forwards it” and they ask “*forwards it how, exactly?*”. The tell they are listening for is whether your answer has **mechanism** in it or only vocabulary.

2 The story

Sushma has been making sambar every Sunday for thirty-one years, in the same kitchen in Mysore, and she is good at it. Her daughter Anitha moved to Pune in March for a job, and in June she phones on a Saturday evening and asks how to make it.

Sushma opens her mouth to answer and finds that she cannot.

She knows it completely. Her hands know when the dal is done by how it looks when she tilts the vessel. She knows the tamarind is right by the smell at the moment it goes in. She has never once measured the salt. But her daughter is four hundred miles away asking *how much*, and *in what order*, and *for how long*, and the answers are not in her mouth. They are in her hands, and her hands are not on the phone.

She gets through it badly. She forgets the drumstick until Anitha asks what the long green thing in the photo was. She says “some” turmeric four times. Anitha, who is patient, makes it anyway, and says it was fine, in the voice people use when it was not.

So Sushma does something on the Wednesday afternoon that her husband finds funny. She stands in the kitchen with nothing on the stove and nothing in her hands, and she says the whole thing out loud, from the beginning, to the end. She gets four steps in and stops, because she cannot remember whether the tomatoes go before or after the tamarind, and it turns out she has always just looked at the pan and known.

She does it again on Thursday. This time she gets to the end, but the order is wrong in one place, and she notices it herself.

Friday, third time, she says the whole thing, in order, with amounts, in about two minutes, and none of it is in her hands any more. It is in her mouth.

She phones Anitha on Saturday without being asked, and says it again, properly. Anitha says it came out right this time. Sushma is fairly sure the recipe did not change on Wednesday.

3 The idea in plain English

Sushma's problem is the one this day exists to fix, and it has a name.

Recognising is not producing

Recognition is what happens when you read a lesson and every sentence makes sense. It feels exactly like knowing.

Recall is producing the same content from nothing, in order, out loud. It is a different and much harder skill, and it is the only one an interview measures.

Sushma had complete recognition. Standing at her own stove she could not have got a single step wrong. What she did not have was recall, and no amount of extra cooking would have given it to her — the only thing that builds recall is **retrieval practice**, which means attempting to produce the answer before checking it, and being wrong on the way.

That is why this day is a set of questions and not a set of notes.

The shape every answer takes

Every fundamentals answer that scores well has the same three parts, in the same order:

1. **The one-sentence answer.** Say the thing first. Not context, not history, not “well, it depends”.
2. **The mechanism.** One layer down. Not “DNS resolves the name” but “the resolver asks the root, then the `.com` servers, then Google's own name servers, and caches the result for the record's TTL”.

3. The limit or the trade-off. What it does not do, or what you gave up. This is the part that separates a candidate who has read about it from one who has used it.

Ninety seconds, three parts. If you are still going at three minutes you have started explaining rather than answering, and the interviewer has stopped listening.

Where the ten questions come from

Days 1 to 13 are not thirteen unrelated topics. They are **one journey**, told in order, and each day is a place along it:

```
a name → an address → a connection → a request → a process  
→ an operating system → hardware → and back
```

Being able to see it as one journey is worth more than the thirteen separate answers, because an interviewer who asks a fundamentals question is usually holding one point on that path and checking whether you can walk in both directions from it. §4 is the whole journey in one diagram, and it is the single most valuable thing to be able to reproduce from memory.

How to use today

For each of the ten questions in §5:

1. Say your answer out loud, all the way to the end, **before** reading the model answer.
2. Then read the model answer and mark the difference — not “did I know it” but “did I say it”.
3. Anything you could not produce goes back on the list for tomorrow, not today. Recalling something you failed at yesterday is worth several times recalling something you got right ten minutes ago.

Talking to a wall feels ridiculous and works. Sushma’s third attempt was the one that stuck.

4 The picture

Days 1 to 13 as one journey. This is the diagram to be able to draw from memory:

You type google.com and press Enter
day 001

Browser cache, OS cache, hosts file
then the DNS resolver
day 003

Name becomes an IP address
plus a port: 443 for HTTPS
day 003

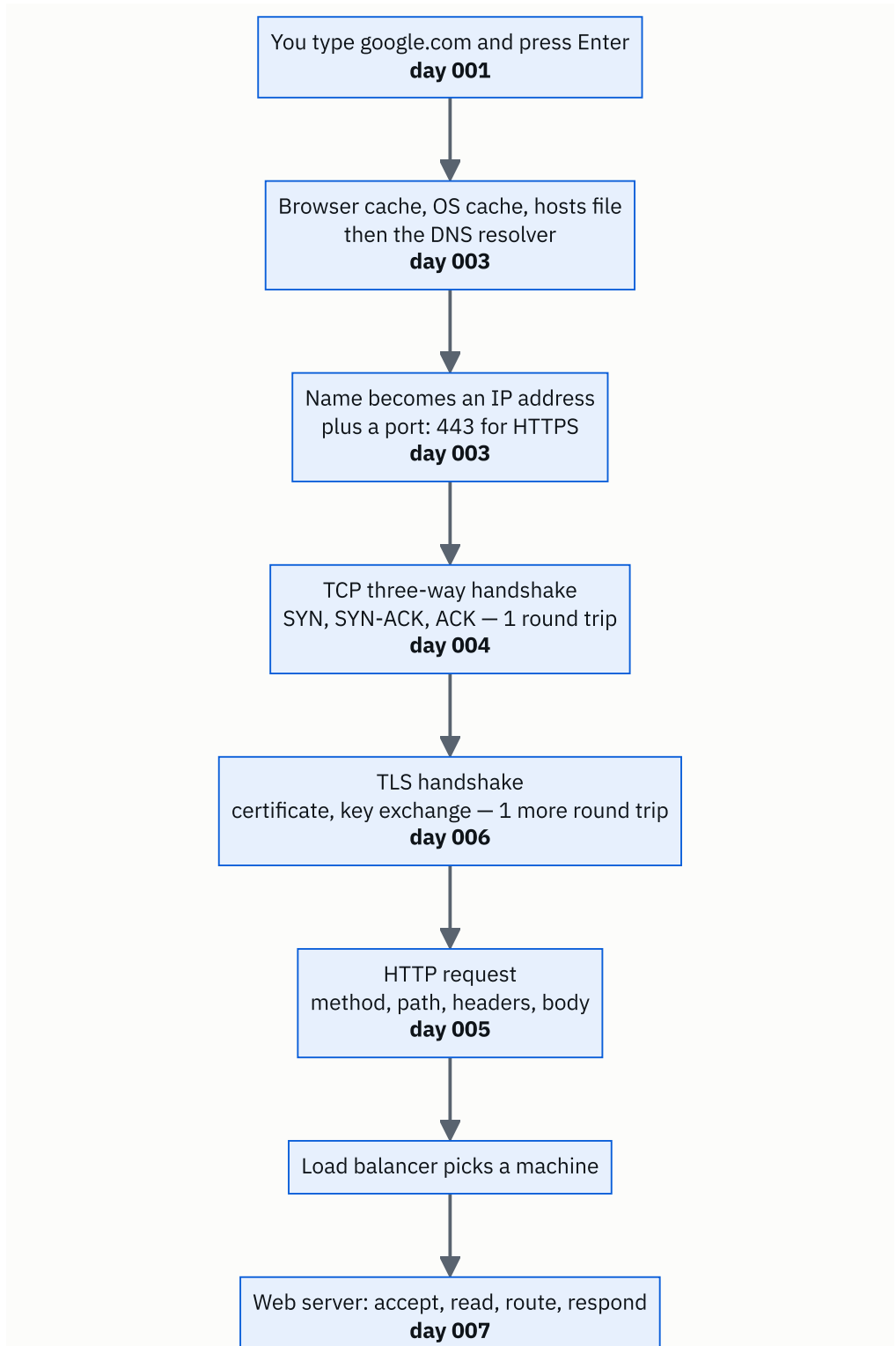
TCP three-way handshake
SYN, SYN-ACK, ACK — 1 round trip
day 004

TLS handshake
certificate, key exchange — 1 more round trip
day 006

HTTP request
method, path, headers, body
day 005

Load balancer picks a machine

Web server: accept, read, route, respond
day 007



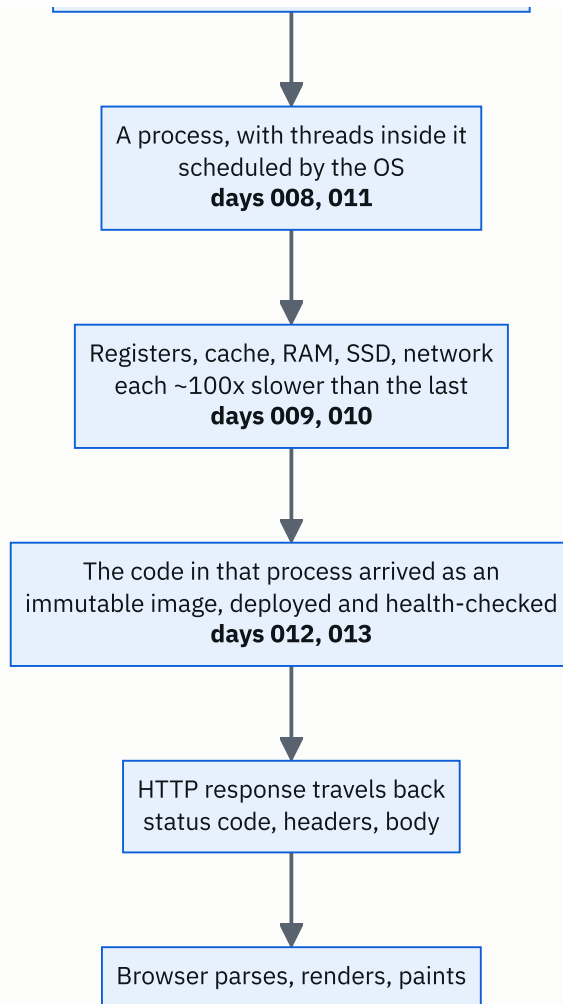
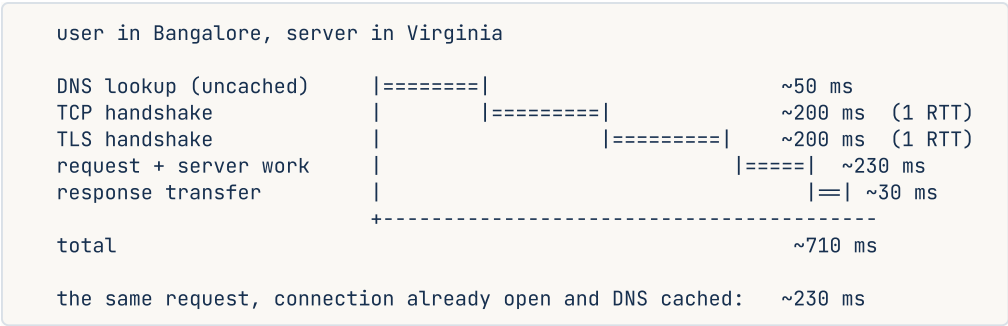


FIGURE 14.1

What to notice: there are exactly two round trips before a single byte of your request is sent — one for TCP, one for TLS — and that is where a large part of the time on a first visit goes. Notice also that days 8 to 13 are all *inside* one box on this path. That is the shape of the subject: most of the journey is network, and most of the depth is what happens after the bytes land.

The same journey with the clock running, which is the version to use when someone asks *where does the time go*:



What to notice: setup costs more than the work. This is why connection reuse, keep-alive, HTTP/2 multiplexing and a CDN closer to the user are the first things anyone reaches for, and why “make the server faster” is often the wrong lever.

The layers, so a question at any level can be placed:



What to notice: every one of those layers is shared by containers and separated by virtual machines, which is the whole of [day 013](#) in one picture.

5 How it actually works

Ten questions. Attempt each one out loud before reading on. The model answers are compressed to their load-bearing parts — the full versions are in the day each one names.

1. “What happens when you type google.com and press Enter?”

Day 001

The single most-asked question in the industry, and it is not a trivia question — it is an invitation to show how many layers you can name and how you handle being interrupted.

Answer: The browser checks its own cache, then the OS cache, then the hosts file, then asks a DNS resolver, which walks root → `.com` → Google's name servers and returns an IP address. The browser opens a TCP connection to that address on port 443 — a three-way handshake, one round trip. Then a TLS handshake: the server presents a certificate, the browser validates the chain against its trust store, and they agree a symmetric key. Then an HTTP request goes over that encrypted connection, hits a load balancer, reaches a web server, which routes it to application code that may talk to a cache and a database. The response comes back with a status code and headers, and the browser parses the HTML, requests the CSS, JavaScript and images it references, builds the DOM and the render tree, and paints.

The trick: say the whole outline in ninety seconds first, then let them choose where to go deep. Do not start deep. Candidates who begin with the DNS packet format never reach the renderer.

2. “What is the difference between a client and a server?”

Day 002

Answer: It is a role in a conversation, not a type of machine. The client initiates the request; the server is listening and responds. The same machine can be both — a web server is a client of its own database. The important consequence is what lives where: anything the client holds is visible to and modifiable by the user, so validation, authorisation and secrets belong on the server. The client is for interface and for making the round trip feel shorter than it is.

The follow-up that comes: “*So can I trust anything the client sends?*” No. Client-side validation is a courtesy to the user; server-side validation is the actual rule.

3. “How does the browser find the server for google.com?”

Day 003

Answer: DNS turns a name into an address. The resolver checks its cache, then asks a root server which names servers own `.com`, then asks those which own `google.com`, then asks Google's own name servers for the A record. It caches the answer for the TTL on the record. The address alone is not enough to talk to a program — the **port** picks which listening process on that machine gets the bytes. 80 for HTTP, 443 for HTTPS, 22 for SSH, 5432 for Postgres, 6379 for Redis.

The mechanism sentence: an IP address identifies a machine; a port identifies a process on it; together they are a socket, and a connection is a pair of sockets.

4. “TCP or UDP for a live video call, and why?”

Day 004

Answer: UDP. TCP guarantees delivery and ordering by retransmitting anything lost, which means a dropped packet stalls everything behind it while it is re-sent. In a live call, a frame that arrives 400 ms late is worthless — you would rather have a moment of blur and stay in sync. UDP has no retransmission, no ordering and no connection, so late data is simply missing, which is exactly the right trade for real time. Video calls, live streaming, games and DNS lookups use UDP; anything where every byte must arrive — a web page, a file, a bank transfer — uses TCP.

Say this too: HTTP/3 runs on UDP and rebuilds reliability on top in QUIC, precisely to avoid TCP's head-of-line blocking.

5. “Describe an HTTP request.”

Day 005

Answer: A method, a path, headers, and optionally a body.

```
POST /api/orders HTTP/1.1
Host: shop.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
Content-Length: 58

{"item_id": 4412, "quantity": 2}
```

The method says what kind of operation it is. The path says which resource. Headers carry metadata about the request — who you are, what format the body is in, what formats you will accept back. The body carries the data, and only some methods have one. The response has the same shape with a status line instead of a request line:

```
HTTP/1.1 201 Created , headers, body.
```

The distinction they are testing: headers describe, the body carries. Authentication, content type and caching directives are headers; the actual payload is the body.

6. “What does HTTPS protect you from, and what does it not?”

Day 006

Answer: It protects the connection, not the endpoints. In transit you get three things: **confidentiality** (nobody in between can read it), **integrity** (nobody can alter it without detection), and **authentication of the server** (the certificate proves you are talking to the domain you asked for, as far as a certificate authority is willing to vouch).

It does **not** protect you from a malicious site — a phishing page has a perfectly valid padlock. It does not hide which domain you visited, since DNS and the TLS SNI field leak that. It does not protect the data once it arrives at the server. And it says nothing about whether the server stores your data safely.

The sentence that scores: “the padlock means the connection is private, not that the other end is trustworthy”.

7. “What happens inside the server between the request arriving and the response leaving?”

Day 007

Answer: A loop. The server has a socket bound to a port and is blocked in `accept()`. A connection arrives, `accept()` returns a new socket for that one conversation, and the server reads bytes off it until it has a full request. It parses the request line and headers, routes the path to a handler, runs your code — which usually means waiting on a database or a cache — builds a response, writes it back, and then either closes the connection or keeps it open for reuse.

The part worth adding: what makes it concurrent. A thread per connection is simple and costs about 1 MB of stack each. A pool of threads bounds that. An event loop — one thread, non-blocking sockets, `epoll` — handles tens of thousands of mostly-idle connections in one process, which is what nginx and Node.js do.

8. “What is the difference between a process and a thread?”

Day 008

Answer: A process has its own memory; threads inside one process share it. That is the whole difference, and everything else comes from it. Threads are cheap to create and switch between, and can pass data by simply referring to the same object — which is exactly why they need locks and produce race conditions. Processes are isolated, so one crashing does not take the others with it, and they must communicate through pipes, sockets or shared memory.

The practical follow-up: in Python, the GIL means threads do not give you parallel CPU work — use threads for I/O-bound work and `multiprocessing` for CPU-bound work. Servers commonly use both: several worker processes, each with a thread pool or an event loop.

9. “How much slower is a disk read than a memory read?”

Days 009 and 010

Answer: About a thousand times for an SSD, and about a hundred thousand for a spinning disk. The hierarchy is what matters more than any single number, and each step down is roughly a hundred times slower than the one above:

L1 cache	~1 ns	
RAM	~100 ns	100x slower than L1
SSD	~100 us	1,000x slower than RAM
spinning disk	~10 ms	100,000x slower than RAM
network, same region	~0.5 ms	
network, cross-continent	~150 ms	

The thing to do with the numbers: scale them to human time. If L1 is one second, RAM is under two minutes, an SSD is a day and a half, a spinning disk is four months, and a cross-continent round trip is about five years. That is why caching works, and why one extra network hop can matter more than any amount of code tuning.

10. “How does your code end up running on a server, and what is a container?”

Days 012 and 013

Answer: Commit and push, CI runs the tests, merge, build an **immutable image** tagged with the commit hash, push it to a registry, deploy it, and the load balancer sends traffic only once the **readiness probe** passes. Configuration and secrets are injected at run time so the same image runs everywhere. Rolling back is redeploying the previous tag — about ninety seconds — so the rule is roll back first and diagnose after. The one thing that does not roll back is a database migration, which is why migrations must be backwards-compatible.

A **container** is that image running as an ordinary process on the host kernel, with **namespaces** limiting what it can see and **cgroups** limiting what it can use. It is not a small computer: no guest kernel, which is why it starts in about 100 ms rather than 45 seconds and costs a few megabytes rather than a gigabyte.

6 The numbers

These are the numbers to have without thinking. Interviewers do not expect precision; they expect the right order of magnitude, produced quickly.

The latency ladder, and the human-scale version:

operation	real time	if L1 were 1 second

L1 cache reference	1 ns	1 second
branch mispredict	3 ns	3 seconds
L2 cache reference	4 ns	4 seconds
mutex lock/unlock	17 ns	17 seconds
main memory reference	100 ns	1.7 minutes
compress 1 KB	2,000 ns	33 minutes
send 1 KB over 1 Gbps	10,000 ns	2.8 hours
SSD random read	150,000 ns	1.7 days
read 1 MB from memory	250,000 ns	2.9 days
round trip within a datacentre	500,000 ns	5.8 days
read 1 MB from SSD	1,000,000 ns	11.6 days
disk seek	10,000,000 ns	3.9 months
read 1 MB from disk	20,000,000 ns	7.7 months
round trip India ↔ USA	150,000,000 ns	4.8 years

The port numbers:

20, 21 FTP	22 SSH	25 SMTP	53 DNS
80 HTTP	443 HTTPS	587 SMTP (TLS)	
3306 MySQL	5432 Postgres	6379 Redis	9092 Kafka
27017 MongoDB	11211 Memcached	9200 Elasticsearch	

The status code families:

1xx informational	2xx success	3xx redirect
4xx the client was wrong		5xx the server was wrong
200 OK	201 Created	204 No Content
301 Moved Permanently		304 Not Modified
400 Bad Request	401 Unauthorised	403 Forbidden
409 Conflict	429 Too Many Requests	404 Not Found
500 Internal Server Error	502 Bad Gateway	503 Service Unavailable
504 Gateway Timeout		

401 means “I do not know who you are”; 403 means “I know exactly who you are and no”. That distinction is asked directly.

Round trips before your first byte, on a cold connection:

DNS (uncached)	1 round trip	
TCP handshake	1 round trip	
TLS 1.3 handshake	1 round trip	(TLS 1.2 needed 2)

before the request is even sent: 3		
at 150 ms per round trip cross-continent: 450 ms of setup		
at 1 ms within a region:	3 ms	

This arithmetic is the entire argument for CDNs, connection pooling and keep-alive.

Sizes worth knowing:

a typical HTTP request header block	: 500 B - 2 KB
an average web page, everything	: 2 MB
a JSON API response	: 1 - 50 KB
a UUID	: 16 bytes binary, 36 chars as text
a row in a typical relational table	: 100 B - 1 KB
a container image, slim base	: 130 MB
one thread's stack	: 1 MB (so 1,000 threads = 1 GB)
a TCP connection's kernel buffers	: ~10 KB

A worked estimate, the shape of every capacity question:

```
50,000,000 daily active users
x 20 requests each per day
= 1,000,000,000 requests per day

1,000,000,000 / 86,400 seconds = 11,574 requests per second average
x 3 for peak                    = ~35,000 requests per second at peak

each request touches ~2 KB of response
35,000 x 2 KB                  = 70 MB/s = 560 Mbps at peak
```

Storage, the other half of every estimate:

```
1,000,000,000 events per day x 500 bytes = 500 GB per day
x 365                                = 182 TB per year
x 3 replicas                          = 546 TB of raw disk per year
```

7 The trade-offs

Every fundamentals topic has one trade-off sentence. These are the ten to have ready, because “what would you use instead, and when?” follows almost every fundamentals answer.

TCP versus UDP. TCP buys delivery and ordering, and charges you head-of-line blocking and a handshake. Choose UDP when late data is worthless — live video, games, DNS — and TCP when every byte matters. *I would not use TCP if the application would rather drop a frame than wait for it.*

HTTP/1.1 versus HTTP/2 versus HTTP/3. HTTP/2 multiplexes many streams over one TCP connection, which removes application-level head-of-line blocking but not TCP’s. HTTP/3 moves to QUIC over UDP and removes that too, at the cost of being newer and harder to inspect on the network. *I would not reach for HTTP/3 if the traffic is server-to-server inside one datacentre, where packet loss is low and the benefit nearly vanishes.*

HTTPS everywhere. You buy confidentiality, integrity and server authentication, and you pay one extra round trip on a cold connection plus certificate management. The cost is small enough that the answer is now always yes. *I would still not rely on it if the threat model includes a compromised endpoint, because it protects the wire and nothing else.*

Threads versus processes versus an event loop. Threads are cheap and share memory, so they need locks and produce races. Processes are isolated and expensive, and must serialise to communicate. An event loop handles enormous numbers of idle connections in one thread and collapses if any handler blocks. *I would not use an event loop if the workload is CPU-bound — one slow computation stalls every connection.*

Synchronous versus asynchronous work. Doing it in the request is simple and gives the user a real answer. Pushing it to a queue keeps the request fast and makes the result eventual, which means status endpoints, retries and duplicate handling. *I would not make it async if the user cannot act on the result until it is done.*

Cache or not. A cache turns a 100 ms database read into a 1 ms memory read, and buys you a new problem: the cached copy can be wrong. *I would not cache data where staleness is unsafe — a balance at the moment of a transfer, an authorisation decision.*

Vertical versus horizontal scaling. A bigger machine is trivial and has a ceiling and a single point of failure. More machines have no ceiling and require statelessness, a load balancer and a distributed-systems conversation. *I would scale vertically first if the system is small — it is usually cheaper than the engineering time to make it distributed.*

Containers versus virtual machines. Containers give 100 ms starts, a few MB of overhead and hundreds per host, and charge you a shared kernel. VMs give a real isolation boundary and charge you a gigabyte and 45 seconds each. *I would not use plain containers if I were running untrusted customer code — that wants Firecracker, gVisor or a VM.*

SSD versus spinning disk versus memory. Memory is $1,000\times$ faster than an SSD and does not survive a restart. SSDs are $100\times$ faster than spinning disks and cost more per terabyte. *I would still choose spinning disks for cold archival data where cost per terabyte dominates.*

Rolling versus blue-green versus canary deployment. Rolling needs no extra capacity and forces both versions to run at once, so migrations must be backwards-compatible. Blue-green gives instant rollback and needs double the capacity. Canary limits blast radius best and needs enough traffic for 1% to be meaningful. *I would not canary a service doing ten requests a second — 1% of that tells you nothing.*

8 In the interview

How it gets asked

- *“Before we start on the design, tell me what happens when you type google.com and press Enter.”* — the calibration question. Your answer sets the level for the next forty minutes.
- *“You said the load balancer forwards the request. Forwards it how?”* — the mid-design probe. They are checking whether the vocabulary has mechanism behind it.
- *“What’s the difference between X and Y?”* — TCP/UDP, process/thread, container/VM, 401/403. Always answer with the one structural difference first, then derive the consequences.
- *“How much slower is a disk read than a memory read?”* — the estimation question. Order of magnitude, quickly, then what it implies.

What to say out loud, in the first ninety seconds

This is the shape for **any** fundamentals question, and it is worth rehearsing as a shape rather than as ten separate answers.

1. **Answer in one sentence.** *“A container is a process on the host kernel with a restricted view and a resource cap.”* No preamble.
2. **Give the mechanism, one layer down.** *“Namespaces limit what it can see — its own process list, filesystem, network. cgroups limit what it can use.”*
3. **Give a number.** *“It starts in about 100 milliseconds against 45 seconds for a VM, and costs a few megabytes against a gigabyte.”*
4. **Give the limit or the trade-off, unprompted.** *“The cost is the shared kernel, so for untrusted code you want a real boundary.”*
5. **Stop.** Then ask: *“Do you want me to go deeper on any of that?”* Handing them the steering wheel is a senior move and it stops you talking past the answer.

The follow-ups

“You’re going too broad — pick one part and go deep.” This is a compliment and an instruction, and the right response is to pick the part you know best rather than the part they mentioned last. Say which you are picking and why: *“I’ll take the TLS handshake, since that’s where the round trips are.”* Then go two layers further than you did the first time — for TLS that means the certificate chain, the trust store, why TLS 1.3 saves a round trip over 1.2, and what SNI leaks.

“How do you know that number?” Never bluff a number. The honest and strong answer is to derive it or bracket it: *“I don’t remember exactly, but a cross-continent round trip is bounded below by the speed of light — about 20,000 km round trip at 200,000 km/s in*

fibre is 100 ms, and in practice it's 150 to 250 with routing. So I'd plan for 200." An interviewer will take a derived estimate over a memorised figure every time.

"That's the happy path. What happens when it fails?" Every fundamentals answer has a failure version, and having one ready is what makes the answer sound lived-in. DNS: the record expires or the resolver is unreachable, so you get a resolution failure before any connection is attempted. TCP: SYN with no reply, retried with backoff, then a timeout after tens of seconds — which is why application-level timeouts must be shorter. TLS: an expired or mismatched certificate, and the browser refuses. HTTP: a 502 means the load balancer reached no healthy backend; a 504 means it reached one and gave up waiting.

"Which of these would you actually change first if the page were slow?" Measure before choosing, and say so. Then: the setup round trips, because they dominate on a cold connection — a CDN, keep-alive and connection pooling. Then the number of requests, because each one costs a round trip. Then the server work itself, and inside that the database queries before the code, because a missing index is worth more than any amount of loop tuning. Naming that order shows you know where the time actually is, which is the whole point of [day 010](#).

A model answer

The flagship question, answered the way it should be — outline first, depth on request.

“I’ll give the whole path first and then go deeper wherever you want.

The browser needs an IP address for `google.com`, so it checks its own cache, then the OS resolver cache, then the hosts file, and if none of those have it, it asks a DNS resolver. The resolver walks the hierarchy — a root server tells it which name servers own `.com`, those tell it which own `google.com`, and Google’s own name servers return the A record. That gets cached for the record’s TTL, which is why the second visit skips all of it.

Now it has an address, and it needs a port — 443, because this is HTTPS. It opens a TCP connection: SYN, SYN-ACK, ACK. That’s one full round trip before anything useful happens. Then TLS: the server presents its certificate chain, the browser validates it against its trust store and checks the name matches, and they agree a symmetric key. TLS 1.3 does that in one round trip; 1.2 needed two.

So on a cold connection there are about three round trips before the request is sent. From India to a US datacentre at 150 ms each, that’s most of half a second of pure setup, which is why CDNs and keep-alive matter more than almost anything else.

Then the HTTP request goes over that encrypted connection: method, path, headers, maybe a body. It arrives at a load balancer, which picks a healthy backend, and a web server accepts the connection, parses the request, and routes it to application code. That code usually checks a cache, and goes to a database on a miss. It builds a response — status code, headers, body — and writes it back over the same connection, which is then kept open for reuse.

The browser parses the HTML, and as it does, it discovers references to CSS, JavaScript and images and requests those too — each of which is another round trip unless they’re already cached or the connection is multiplexed. It builds the DOM, applies styles, lays out, and paints.

Underneath all of that, the server side is a process scheduled by an operating system on real hardware, where each step of the memory hierarchy is roughly a hundred times slower than the one above it, and the code inside that process got there as an immutable image that was built, pushed to a registry, deployed and health-checked before it saw traffic.

Where would you like me to go deeper?”

That answer is complete, ordered, contains three numbers, names the two round trips that dominate, and ends by handing control back. It takes about ninety seconds to say.

9 Recall card

1. **Recognising is not producing.** The only thing that builds recall is attempting the answer out loud before checking it. Say it to an empty room; being wrong on the way is the point.
2. **Every answer has three parts:** one-sentence answer, then the mechanism one layer down, then the limit or trade-off. Ninety seconds, then stop and offer to go deeper.
3. **Days 1–13 are one journey:** name → address → connection → request → process → operating system → hardware, and back. Draw it; every fundamentals question is a point on it.
4. **Three round trips before your first byte** on a cold connection — DNS, TCP, TLS. At 150 ms each that is most of half a second, and it is why CDNs and keep-alive exist.
5. **Each step down the memory hierarchy is about $100\times$ slower.** L1 1 ns, RAM 100 ns, SSD 100 μ s, disk 10 ms, cross-continent 150 ms. Scale them to human time and the answer to “should I cache this?” becomes obvious.

PRACTICE

Practice — Max, min, second largest: the single-pass habit

DSA topic: Max, min, second largest: the single-pass habit **System design topic:** Fundamentals revision and interview questions

Code these, in this order

Four problems that all have a one-line sorting solution and a better one-pass solution. Write the sorting version first if it helps you see the answer — then delete it and write the pass.

Before each one, say out loud:

1. How many trackers do I need, and what does each one hold?
2. What do they start as? (Never `0`.)
3. Does “largest” here mean by position or by distinct value?
4. What happens on an empty input, and on a one-element input?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Maximum Product of Two Elements in an Array	LeetCode 1464 (Easy)	The two largest, in one pass. The sorting answer is one line; write the two-tracker version and see that it is barely longer.
2	Third Maximum Number	LeetCode 414 (Easy)	Three trackers, and the distinct-value contract — the statement says third <i>distinct</i> maximum, and if there isn't one you return the maximum instead. Read it twice.
3	Maximum Product of Three Numbers	LeetCode 628 (Medium)	Five trackers: the three largest and the two smallest . Two large negatives multiply to a large positive, which is the whole trap.
4	Kth Largest Element in an Array	LeetCode 215 (Medium)	Where trackers stop working. Solve it three ways — sort, min-heap of size k, quickselect — and be able to say when each is the right choice.

On problem 2, do this properly

- Write it with three trackers before looking at any solution.
- Then test it on `[2, 2, 3, 1]`, `[1, 2]`, `[1, 1, 1]` and `[5, 2, 4, 1, 3, 6, 0]`.
- Then answer: which of those four inputs breaks a version that forgets `x  $\neq$  first`?
- Then answer: which of them breaks a version that has no third branch at all?

On problem 3, notice the negatives

`[-10, -10, 1, 3, 2]` has answer 300, not 6. Say why out loud before coding. Then decide how many trackers that means, and write them.

The lazy version — `sorted(nums)` then compare `nums[-1]*nums[-2]*nums[-3]` against `nums[0]*nums[1]*nums[-1]` — is correct and $O(n \log n)$. Write it, then write the $O(n)$ five-tracker version, then say which you would ship and why. (There is a real answer: for a small `n` the sorted version is clearer and fast enough; for a stream it is impossible.)

On problem 4, do all three

- **Sort and index:** $O(n \log n)$, two lines. Always correct, sometimes right.
- **Min-heap of size k:** push each element, pop when the heap exceeds `k`. $O(n \log k)$ time, $O(k)$ space. Best when `k` is small and `n` is huge, or when the data is a stream.
- **Quickselect:** partition around a pivot, recurse into one side only. $O(n)$ average, $O(n^2)$ worst case unless you randomise the pivot.

Then say out loud which one you would use if `k = 3` and `n = 109` arriving over a network, and why the other two are not options.

The contract drill

For each phrase, say what you would ask the interviewer before writing any code:

1. “Find the second largest element.”
2. “Find the second highest salary.”
3. “Find the top three scores.”
4. “Find the largest and smallest in one pass.”

Every one of them has an ambiguity. Name it in under five seconds.

The bug drill

Predict the output of each, then run them:

```
def largest(items):
    best = 0
    for x in items:
        if x > best:
            best = x
    return best

print(largest([-7, -2, -9]))
```

```
def second(items):
    first = second_ = float("-inf")
    for x in items:
        if x > first:
            first = x
            second_ = first
    return second_

print(second([3, 9, 1, 7]))
```

```
def second(items):
    first = second_ = float("-inf")
    for x in items:
        if x > first:
            first, second_ = x, first
    return second_

print(second([10, 1, 2]))
print(second([1, 2, 10]))
```

For the third one, explain in a sentence why the two calls disagree, and why that makes the bug so hard to catch by testing.

The fundamentals drill

This is the real work of today. Ten questions, and the rule is: **say the whole answer out loud before you read anything**. Being wrong out loud is the exercise.

1. What happens when you type google.com and press Enter?
2. What is the difference between a client and a server?
3. How does the browser find the server for google.com?
4. TCP or UDP for a live video call, and why?
5. Describe an HTTP request. What is in the headers, what is in the body?
6. What does HTTPS protect you from, and what does it not?
7. What happens inside the server between the request arriving and the response leaving?
8. What is the difference between a process and a thread?
9. How much slower is a disk read than a memory read?
10. How does your code end up running on a server, and what is a container?

Mark each one **said it / knew it but could not say it / did not know it**. Only the first category counts. Anything in the other two goes on tomorrow's list, not today's — a question you failed at yesterday is worth several you got right ten minutes ago.

The numbers drill

Produce these from memory, in under a minute, with no looking:

- The five levels of the latency ladder, in order, with an order of magnitude each.
- The port numbers for HTTP, HTTPS, SSH, Postgres and Redis.
- What 401, 403, 404, 429, 502 and 504 each mean.
- How many round trips happen before the first byte of your request is sent on a cold HTTPS connection, and what each one is for.
- 50 million daily users $\times$ 20 requests each — what is that in requests per second, average and peak?

The one to draw

Draw the days 1–13 journey — name, address, connection, request, process, operating system, hardware, and back — in whatever tool you like, from memory, with no notes open. Then check it against §4 and mark what you left out. What you leave out twice is what to revise.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the second largest element without sorting.* Ask the `[5, 5, 3]` question first. Then give the two branches, name why the old best has to slide down, and finish with `O(n)` time and `O(1)` space.
2. *Pick any topic from days 1 to 13 and answer it with no preparation.* Have someone pick, or pick at random. Use the three-part shape: one-sentence answer, then the mechanism, then the trade-off. Ninety seconds, then stop.
3. *How much slower is a disk read than a memory read, and what does that mean for a system you are designing?* Give the ratio, then the ladder, then one design consequence — why a cache hit is worth three orders of magnitude, and why one extra network hop can cost more than any amount of code tuning.

Before you move on

- `[]` I ask what “second largest” means before writing a line of code.
- `[]` I never initialise a tracker to `0`, and I can say what happens if I do.
- `[]` I write `first, second = x, first` as one statement, and I know why.
- `[]` I test every second-largest solution with the largest element **first** in the list.

- [] I can say all ten fundamentals answers out loud, in ninety seconds each, with no notes.
- [] I can produce the latency ladder and the round-trip count from memory.

Moving elements: zeros, duplicates, and the write pointer

DATA STRUCTURES AND ALGORITHMS

Moving elements: zeros, duplicates, and the write pointer

You can compact an array in place with a read index and a write index.

SYSTEM DESIGN

What an API is

You can explain an API to a non-engineer, and then to an engineer.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Move all zeros to the end while keeping the order of the other elements.*
- *What is an API? Give me an example of one you have used.*

Moving elements: zeros, duplicates, and the write pointer

1 What this is, and why they ask it

Some questions ask you to throw things out of a list and keep the rest packed together at the front. Remove the zeros. Remove every copy of the number 3. Remove the duplicates. In every one of them you walk the list once with **two counters instead of one**: a *read* counter that visits every position, and a *write* counter that only moves when you decide to keep something.

That is the whole technique, and it has a name: the **write pointer**, sometimes called in-place compaction. The read counter runs ahead. The write counter lags behind. The gap between them is exactly how many elements you have thrown away so far.

Interviewers like this family because the obvious answers are both wrong in an interesting way. Building a new list is easy but uses extra memory, and the question usually says *in place* to forbid it. Deleting from the middle of the list as you go is worse: it is slow, and it silently skips elements, and §7 shows exactly how. The write pointer is the answer that is fast, uses no extra memory, and keeps the surviving elements in their original order.

This is a phone-screen staple — LeetCode 283, 27 and 26 are three of the most-asked easy problems anywhere — and it is the direct ancestor of the two-pointer work that starts on [day 027](#) and the read-write pointer day, [day 029](#). Learn it properly now and four later weeks get easier.

2 The story

Padma keeps her books on a long shelf in the passage outside the bedrooms, standing in the order she means to read them. Thirty places, and for years it was full enough that nothing fell over.

Over the last few months she has lent four of them out. Her sister took two, a neighbour took one, and one went to her son's friend and has not come back. Where each book used to stand there is now a gap, and the books on either side have started to lean into it. On Sunday morning she comes out with her tea and finds two of them lying flat.

So she fixes it, and the way she fixes it is worth watching.

She does not take everything off the shelf. She stands at the left-hand end and puts her left hand on the first place. Her right hand goes on the same place. Book there, so both hands move one to the right. Book there again, both hands move again. On the fourth place there is nothing, so her left hand stops and stays where it is, and only her right hand carries on.

Her right hand finds the next book two places along. She lifts it out, stands it in the place her left hand is holding, and moves her left hand one to the right. Her right hand carries on from where it had got to.

From then on her two hands are apart, and the distance between them is exactly the number of gaps she has walked past. Every time her right hand finds a book, that book goes back to where her left hand is and her left hand moves on one. Every time her right hand finds nothing, only her right hand moves.

When her right hand runs off the end of the shelf, every book is standing packed against the left-hand side, in the order they were in before, and all the empty space is in one block at the right.

Then she counts what is left over at that end. Four empty places, which is exactly how many books are out with people. She types the four names into her phone so she knows whom to ask.

3 The idea in plain English

Padma's shelf is an **array** — the fixed row of numbered boxes you met on [day 009](#). Her two hands are two variables holding positions in it. Everything else follows from one rule about those hands.

The two counters, and the one rule

Call the right hand `read` and the left hand `write`.

- `read` visits **every** position from left to right. It never skips and never stops early.
- `write` says **where the next kept element goes**. It moves only when you actually keep something.

The rule is: `write` **never runs ahead of** `read`. It starts equal to `read` and falls further behind with every element you discard. That is why you can safely overwrite position `write` — whatever used to be there has already been read and either kept or discarded. You are never destroying data you still need. This is the sentence to say out loud in an interview, and it is the reason the whole pattern is safe.

What “in place” means, and why they ask for it

In place means you change the original list itself rather than building a new one, using only a constant amount of extra memory — a few variables, no second list. You met the term on [day 007](#). Here it means two integer variables, whether the list has 5 elements or 5 million.

The tempting non-answer is one line:

```
kept = [x for x in items if x != 0]
```

That is correct, readable, and what you would write in real code if you were allowed a new list. It is also $O(n)$ extra space, and when the question says *in place*, it is precisely the answer being ruled out.

What “keeping the order” means

The surviving elements must come out in the same relative order they went in. If the input is `[0, 1, 0, 3, 12]`, the answer is `[1, 3, 12, 0, 0]` — 1 before 3 before 12, exactly as before. An answer of `[12, 3, 1, 0, 0]` has the right contents and is wrong.

An algorithm that preserves relative order like this is called **stable**. The word comes back on [day 057](#) when you meet stable sorting, and it means the same thing there.

There is a faster-looking trick for this problem: whenever you meet a zero, swap it with the last element and shrink the range. That does fewer writes, and it is the right answer to *some* questions — but it scrambles the order, so it is the wrong answer to this one. Knowing which question you are being asked is half the marks.

Two phases, or one

Once `read` has run off the end, `write` holds a number with a clear meaning: **how many elements you kept**. Positions `0` to `write - 1` are the survivors. Positions `write` to the end are leftovers — old values that are still sitting there, already copied to their new homes.

What you do with that tail depends on the question:

- *Move the zeros to the end* — overwrite the tail with zeros. That is Padma noticing the four empty places at the right-hand end.
 - *Remove all instances of a value* — you cannot shrink a fixed-size array, so you return `write` as the new length and the caller ignores everything past it. This is exactly what LeetCode 27 and 26 ask for, and it confuses people the first time. Nothing is deleted. The count is the answer.
-

4 The picture

Padma's shelf, mid-walk. `w` is the left hand, `r` is the right hand.

```
start      0    1    2    3    4
+-----+-----+-----+-----+
|  0  |  1  |  0  |  3  | 12  |
+-----+-----+-----+-----+
      ^
    w,r      both hands on position 0
```

`items[0]` is 0, so it is discarded. Only `r` moves.

```
      0    1    2    3    4
+-----+-----+-----+-----+
|  0  |  1  |  0  |  3  | 12  |
+-----+-----+-----+-----+
      ^      ^
    w      r      the hands have separated: one zero passed
```

`items[1]` is 1, so it is kept. It is copied to position `w`, then both hands move.

```
      0    1    2    3    4
+-----+-----+-----+-----+
|  1  |  1  |  0  |  3  | 12  |
+-----+-----+-----+-----+
              ^      ^
            w      r
```

Notice position 1 still says 1. That is the **leftover** — a stale copy that `r` has already passed, so nobody will ever read it again. Beginners find this alarming. It is fine, and it is the whole reason the pattern is safe.

Carry on to the end, keeping 3 and 12:

```
after the loop  0    1    2    3    4
+-----+-----+-----+-----+
|  1  |  3  | 12  |  3  | 12  |
+-----+-----+-----+-----+
|-----| |-----|
  the kept  leftovers
              ^
            w = 3, so three elements were kept
```

Then fill from `w` to the end with zeros:

finished	0	1	2	3	4
	+-----+	+-----+	+-----+	+-----+	+-----+
	1	3	12	0	0
	+-----+	+-----+	+-----+	+-----+	+-----+

What to notice: the gap between `w` and `r` only ever grows, and it grows by exactly one each time you discard something. At the end, `r - w` is the number of zeros and `w` is the number of non-zeros. Both facts drop out for free.

5 The code, built step by step

The loop, in five lines

```
write = 0
for read in range(len(items)):
    if items[read] != 0:
        items[write] = items[read]
        write += 1
```

Read it as English: *walk every position; if this one is worth keeping, put it at the write position and move the write position on.* Five lines, and it is the entire pattern.

After this loop, `items[0:write]` holds the non-zero values in their original order, and `write` holds how many there were. Run it on `[0, 1, 0, 3, 12]` and the list is now `[1, 3, 12, 3, 12]`. Which is not the answer yet.

The second phase: clear the tail

```
for i in range(write, len(items)):
    items[i] = 0
```

`range(write, len(items))` covers exactly the leftover positions and nothing else. If nothing was discarded, `write == len(items)` and this loop runs zero times, which is the correct behaviour rather than a special case you have to test for.

Put the two together and you have LeetCode 283 solved.

Why the copy is safe, one more time

The line that worries people is `items[write] = items[read]`. It writes into the same list you are reading from. It is safe because `write ≤ read` always holds:

- Both start at 0, so they start equal.
- `write` only increases in the same step where `read` increases.

- `read` also increases in steps where `write` does not.

So `write` can never overtake `read`. When `write == read` the line copies a value onto itself, which is harmless. When `write < read`, it overwrites a position `read` has already gone past.

The swap version, which needs no second phase

```
write = 0
for read in range(len(items)):
    if items[read] != 0:
        items[write], items[read] = items[read], items[write]
        write += 1
```

Instead of copying the non-zero forward and leaving a stale value behind, this **exchanges** it with whatever is at `write` — which, if the two positions differ, is guaranteed to be a zero. The zeros get pushed to the back as a side effect and the tail cleans itself. One pass, no second loop.

Remember the tuple-swap from [day 013](#): the whole right-hand side is evaluated before anything is assigned, so this is a genuine exchange and not two assignments that clobber each other.

Which of the two should you write? Say both exist. The copy-then-fill version generalises to every question in this family; the swap version is specific to *move the discarded thing to the end*, and it does more writes when the array is mostly non-zero, because it writes twice per kept element instead of once. Interviewers are happy with either as long as you can say that sentence.

Changing the test changes the problem

The only thing that varies across this whole family is the `if`. Everything else is identical.

```
if items[read] != 0:           # move zeros to the end
if items[read] != target:     # remove every copy of `target`
if items[read] != items[write - 1]: # remove duplicates from a sorted list
```

That third one deserves a look. On a **sorted** list every duplicate sits next to its twin, so “have I seen this before?” collapses to “is it the same as the last thing I kept?” — and the last thing you kept is at `write - 1`. That is why it needs `write` to start at 1, with position 0 kept unconditionally:

```
write = 1
for read in range(1, len(items)):
    if items[read] != items[write - 1]:
        items[write] = items[read]
        write += 1
```

Comparing against `items[write - 1]` and not against `items[read - 1]` is the detail that matters. `read - 1` is the previous element you *looked at*; `write - 1` is the previous element you *kept*. On a sorted list with no gaps they happen to agree, but the habit of comparing against what you kept is what makes the “at most twice” variant below fall out in one line.

The complete solutions

All five, ready to run.

```

def move_zeros(items: list[int]) → None:
    """LeetCode 283. Move every 0 to the end, in place, keeping the order of the rest."""
    write = 0
    for read in range(len(items)):
        if items[read] ≠ 0:
            items[write] = items[read] # safe: write ≤ read, always
            write += 1
    for i in range(write, len(items)): # the leftovers become zeros
        items[i] = 0

def move_zeros_swap(items: list[int]) → None:
    """The same thing in one pass. items[write] is a zero whenever write ≠ read."""
    write = 0
    for read in range(len(items)):
        if items[read] ≠ 0:
            items[write], items[read] = items[read], items[write]
            write += 1

def remove_value(items: list[int], target: int) → int:
    """LeetCode 27. Returns the new length; items[:length] holds the survivors."""
    write = 0
    for read in range(len(items)):
        if items[read] ≠ target:
            items[write] = items[read]
            write += 1
    return write

def remove_duplicates(items: list[int]) → int:
    """LeetCode 26. items must be sorted. Returns the count of distinct values."""
    if not items: # write starts at 1, so guard the empty case
        return 0
    write = 1
    for read in range(1, len(items)):
        if items[read] ≠ items[write - 1]:
            items[write] = items[read]
            write += 1
    return write

def remove_duplicates_twice(items: list[int]) → int:
    """LeetCode 80. Sorted input; keep each value at most twice."""
    write = 0
    for x in items:
        if write < 2 or x ≠ items[write - 2]:
            items[write] = x
            write += 1
    return write

if __name__ == "__main__":
    a = [0, 1, 0, 3, 12]
    move_zeros(a)
    print(a) # [1, 3, 12, 0, 0]

    b = [0, 1, 0, 3, 12]
    move_zeros_swap(b)
    print(b) # [1, 3, 12, 0, 0]

    c = [3, 2, 2, 3]
    k = remove_value(c, 3)
    print(k, c[:k]) # 2 [2, 2]

    d = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]
    k = remove_duplicates(d)

```



```
print(k, d[:k]) # 5 [0, 1, 2, 3, 4]

e = [1, 1, 1, 2, 2, 3]
k = remove_duplicates_twice(e)
print(k, e[:k]) # 5 [1, 1, 2, 2, 3]
```

Look at `remove_duplicates_twice` for a moment. The test `x != items[write - 2]` asks *is this value different from the one I kept two places ago?* If it is, this is at most the second copy and it may stay. `write < 2` covers the first two elements, which are always allowed. Change the 2 to a `k` and you have “keep each value at most `k` times” — the same five lines.

6 What it costs

`move_zeroes`, the copy version

The first loop runs `len(items)` times — call it `n` — because `range(len(items))` visits every position exactly once, whatever the values are. Each turn does one comparison, and at most one assignment and one increment. That is a fixed, small amount of work, so the first loop is `n` turns of constant work.

The second loop runs from `write` to `n`. If the list has `z` zeros in it, then `write` finished at `n - z`, so the second loop runs `z` times. The worst case is an all-zeros list, where `z = n`.

Total: `n + z` turns, and `z` is at most `n`, so at most `2n`. Constant factors are dropped, so this is **$O(n)$ time**.

Space: `write`, `read` and `i` are three integers. Three integers whether the list has 5 elements or 5 million. That is **$O(1)$ extra space**. The list itself takes `O(n)`, but it was the input, so you do not count it — the phrase to use is *$O(1)$ extra space*, and saying “extra” is what shows you know the difference.

`move_zeroes_swap`

One loop, `n` turns, so **$O(n)$ time** and **$O(1)$ extra space** as well. The difference is in the constant factor, not the class. Count the writes on `[1, 2, 3, 4]`, which has no zeros at all:

- Copy version: 4 copies, and each is a copy onto itself, since `write = read` throughout. Then the tail loop runs zero times. **4 writes**.
- Swap version: 4 swaps, and a swap is two writes. **8 writes**.

Now count on `[0, 0, 0, 1]`:

- Copy version: 1 copy, then 3 zero-fills. **4 writes**.

- Swap version: 1 swap. **2 writes.**

So neither is uniformly better. Mostly-zeros favours the swap, mostly-non-zeros favours the copy, and both are $O(n)$. If an interviewer pushes on this, that is the honest answer, and the honest answer is the one they want.

What you are being compared against

The list-comprehension version — `[x for x in items if x != 0]` — is also $O(n)$ time, but it is $O(n)$ extra space. The delete-as-you-go version in §7 is $O(n^2)$ time, because each `del` or `remove` shifts everything after it down by one, which is the $O(n)$ shifting cost you counted on [day 011](#). On a list of 100,000 zeros that is around five billion moves, and it will not finish while the interviewer is watching.

7 The traps

The near-miss: deleting while you loop

This is the version almost everybody writes first, and it looks completely reasonable.

```
items = [0, 1, 0, 3, 12]
for x in items:
    if x == 0:
        items.remove(x)
print(items)
```

Predict the output before reading on. Most people say `[1, 3, 12]`, because both zeros are removed. Run it:

```
[1, 3, 12]
```

It is right. Now run the same code on `[0, 0, 1]`:

```
[0, 1]
```

One of the zeros survived. Here is why. The loop keeps an internal counter of where it is. It starts at position 0, finds a zero, and removes it — which slides every later element down one place, so the zero that was at position 1 is now at position 0. The loop then moves its counter to position 1, which now holds the 1. **The second zero was moved to a place the loop had already been past, so it is never looked at.** Deleting from a list while looping over it skips exactly the elements that follow a deletion.

This is a bug that passes its first test and fails in production, which is the worst kind. Never delete from a list while iterating over it.

The real error: the same idea with a counted loop

Try to fix it by looping over positions instead:

```
nums = [1, 0, 0, 0]
for i in range(len(nums)):
    if nums[i] == 0:
        del nums[i]
print(nums)
```

```
Traceback (most recent call last):
  File "t2.py", line 3, in <module>
    if nums[i] == 0:
        ~~~~^^^
IndexError: list index out of range
```

`range(len(nums))` is worked out **once**, at the start, from the original length of 4. So `i` still goes 0, 1, 2, 3 even though the list has shrunk to length 1 by then. The crash is actually good news; the `remove` version's silent wrong answer is far more dangerous.

The near-miss: forgetting the second phase

```
def move_zeroes(items):
    write = 0
    for read in range(len(items)):
        if items[read] != 0:
            items[write] = items[read]
            write += 1
    return items

print(move_zeroes([0, 1, 0, 3, 12]))
```

```
[1, 3, 12, 3, 12]
```

The front is perfect and the back is rubbish. Those trailing `3, 12` are the leftovers from §4 — stale copies that were never cleaned up. The tell is that the answer has the right length and the right beginning, so a test that only checks `result[:3]` passes. Always compare the whole list.

The near-miss: comparing against the wrong neighbour

In `remove_duplicates`, this looks equivalent and is not:

```
if items[read] != items[read - 1]:    # wrong on the general problem
```

On a sorted list it gives the same answer, because you never skip a value. On an unsorted list, or on the “at most twice” variant, it falls apart immediately, because `read - 1` may be an element you discarded. Compare against **what you kept**, at `write - 1`, and the habit transfers.

The one-character trap: forgetting to advance `write`

Leave out `write += 1` and every kept element lands on top of the last one. `[1, 2, 3]` becomes `[3, 2, 3]` and `write` stays 0, so the function reports that it kept nothing. There is no error message. This is why you say the rule out loud while writing the line: *keep it, then move the write position on.*

8 In the interview

How it gets asked

- “Move all zeros to the end of the array while keeping the relative order of the non-zero elements. Do it in place.” — LeetCode 283, and the most common phrasing of all.
- “Remove all instances of a given value from the array in place and return the new length.” — LeetCode 27. The words *return the new length* are the part people miss.
- “This array is sorted. Remove the duplicates in place.” — LeetCode 26, and its harder sibling LeetCode 80, *allow each element at most twice*.
- “Given an array, keep only the elements that pass some test, without allocating.” — the vague version. It is the same five lines with a different `if`.

What to say out loud, in the first ninety seconds

1. **Pin the contract.** “Two things before I start: does in place mean I cannot allocate a second array, and does the relative order of the surviving elements have to be preserved?” Those two answers together decide which of three solutions is correct, so asking is not stalling.
2. **Name the pattern.** “I’ll use two indices over the same array — a read index that visits every position and a write index that says where the next kept element goes.”
3. **State the rule, because it is the reason it works.** “The write index never overtakes the read index, so anything I overwrite has already been read. That is what makes writing into the array I am reading from safe.”
4. **Say what the write index means at the end.** “When the loop ends, the write index is the number of elements I kept, and everything from there to the end is leftovers.”
5. **Say what you do about the tail.** “For move-zeros I fill the tail with zeros in a second loop. For remove-element I just return the count, because you cannot shrink a fixed-size array.”

6. **Give the costs.** “ $O(n)$ time — every position visited once, plus at most n more for the tail — and $O(1)$ extra space, just two integers.”
7. **Name the thing you are not doing.** “I am not deleting from the array as I go. Each delete shifts everything after it, so that is $O(n^2)$, and looping while deleting also skips elements.”

The follow-ups

“**Can you do it without the second loop?**” Yes — swap instead of copy. When the write index is behind the read index, whatever sits at the write index is guaranteed to be a zero, because zeros are the only thing I have skipped past. So exchanging the non-zero at the read index with the value at the write index moves the non-zero forward and pushes the zero back in the same step, and the tail cleans itself. It is still $O(n)$ and $O(1)$. The trade is in the constant factor: the swap does two writes per kept element, so on an array with very few zeros the copy-then-fill version does about half the writes. On an array that is mostly zeros the swap version wins. I would mention both and pick the copy version by default, because it generalises to remove-element and remove-duplicates unchanged.

“**What if the order didn’t have to be preserved?**” Then there is a cheaper answer. Keep an index at the end of the array; whenever the read index finds a zero, swap it with the element at the end index and move the end index inwards, without advancing the read index — you have to re-examine the value you just pulled in, because it might also be a zero. That does one swap per zero rather than one write per non-zero, so on an array with a handful of zeros it does a handful of writes instead of n . The output order is scrambled, which is exactly why the original question forbids it. This is the same idea as LeetCode 27’s optimal solution.

“**Now the array is sorted and you want the duplicates gone. What changes?**” Only the condition. On a sorted array all copies of a value are adjacent, so “have I seen this already?” becomes “is it equal to the last value I kept?”, which lives at `write - 1`. I keep position 0 unconditionally, start the write index at 1, and start reading from 1. Same $O(n)$ and $O(1)$. If the array were *not* sorted I could not do it in $O(1)$ space — I would need a set of what I have seen, which is $O(n)$ extra space, or I would sort first and lose the original order.

“**Make it keep each value at most twice.**” Compare against `items[write - 2]` instead of `items[write - 1]`, and let the first two elements through unconditionally. If the current value differs from the one I kept two positions ago, then at most one copy of it has been kept so far, so this one is allowed. Generalise the 2 to a `k` and the same five lines solve “at most k times” — that is the version I would actually write, since it costs nothing.

A model answer

“Two clarifications first. Does ‘in place’ rule out allocating a second array, and does the order of the non-zero elements have to be preserved?

...Right, in place and order preserved.

Then I will use two indices walking the same array. A read index visits every position from left to right. A write index says where the next element I keep should go. Both start at zero.

For each position, if the value is non-zero I copy it to the write index and advance the write index. If it is zero I do nothing, so the write index falls one further behind. The key property is that the write index never overtakes the read index — so every position I overwrite is one the read index has already passed, and I am never destroying a value I still need.

When the loop ends, the write index equals the number of non-zero elements, and everything from there to the end is a stale leftover. So a second loop fills that range with zeros.

```
def move_zeroes(items: list[int]) → None:
    write = 0
    for read in range(len(items)):
        if items[read] ≠ 0:
            items[write] = items[read]
            write += 1
    for i in range(write, len(items)):
        items[i] = 0
```

On `[0, 1, 0, 3, 12]` : the 0 is skipped, 1 goes to position 0, the next 0 is skipped, 3 goes to position 1, 12 goes to position 2. The write index is 3, so positions 3 and 4 get zeroed, giving `[1, 3, 12, 0, 0]` .

That is $O(n)$ time — every position visited once, plus at most n more for the tail — and $O(1)$ extra space, since it is two integers regardless of the array size.

The version I deliberately avoided is deleting the zeros as I walk. Each delete shifts every later element down one, so that is $O(n^2)$, and if you delete while looping you also skip the element straight after each deletion, which gives a wrong answer rather than a slow one.

If you want it in a single pass I can swap instead of copy, since the value at the write index is always a zero when the two indices differ. And if the order did not matter there is a cheaper version that swaps zeros with the last element instead.”

9 Recall card

- **Two indices, one array.** `read` visits every position; `write` moves only when you keep one.
- **The rule:** `write ≤ read` always, so overwriting `items[write]` can never lose data.
- **At the end** `write` is the number kept; `items[write:]` is leftovers — zero it, or return `write` as the new length.
- **Only the `if` changes** across the family: `≠ 0`, `≠ target`, `≠ items[write - 1]`, `≠ items[write - 2]`.
- **$O(n)$ time, $O(1)$ extra space.** Never delete inside a loop — $O(n^2)$, and it skips elements.

1 What this is, and why they ask it

An **API** — application programming interface — is the published list of things one piece of software will do for another, together with the exact way to ask for each one and the exact shape of the answer that comes back. It is a promise about behaviour, not a description of machinery. The whole point is that the caller gets the promise and never gets the machinery.

Everything you have built up over the last fourteen days meets here. A client talks to a server ([day 002](#)), it finds it by name ([day 003](#)), it opens a connection ([day 004](#)), and it sends an HTTP request ([day 005](#)). The API is the answer to the question those four days leave open: *what am I allowed to ask for, and what will I get?*

It gets asked in interviews for two reasons. First as a screening question, usually in the first ten minutes, to check you can explain a technical idea in plain words — plenty of candidates only know the phrase “REST API” and fall apart when asked what the word *interface* is doing in there. Second, and much more importantly, because **every system design answer you will ever give is a set of boxes talking to each other through APIs**. If you cannot say precisely what one box asks another for, you cannot design anything. This is the day the vocabulary gets fixed.

The whole of the next ten days builds on it: REST on [day 016](#), designing a good end-point on [day 017](#), status codes and idempotency on [day 018](#).

2 The story

Zoya has to hand in a forty-page report by eleven, and the college printer has been broken since Monday. There is a small shop in the lane behind the gate that has been there longer than she has, and at half past nine the shutters are already up.

It is one narrow room. A glass counter runs across it, and everything happens behind that counter — two big machines, a boy feeding sheets into one of them, stacks of covers, a bin of offcuts. She has been coming here for two years and she has never once been behind the counter, and it has never once mattered.

On the wall above the counter there is a board. Black and white, three rupees a sheet. Colour, fifteen. Both sides, five. Spiral binding, forty. Lamination, twenty. Nine lines in all, and it has not changed in two years except that the numbers have been painted over

twice.

She puts her pen drive on the counter and says the whole thing in one breath: the third file, forty sheets, black and white, both sides, spiral. The boy repeats it back, says twenty minutes, and gives her a small plastic token with 14 on it. She goes and drinks tea.

Two things about that exchange. The first is that she can only ask for what is on the board. Last month she asked him to “make it look a bit nicer” and he just stood there — not because he was unwilling, but because there is nothing behind the counter called *nicer*. She had to turn it into things on the board: colour on the first sheet, and lamination.

The second is that in January the shop replaced the big machine with a different one, and Zoya did not find out for three weeks. She asked for the same nine things in the same words and got the same stack back. The only day it ever mattered to her was the morning the colour ran out, and he told her that straight away at the counter instead of taking her money and letting her wait twenty minutes for nothing.

3 The idea in plain English

The board on the wall is the API. Not the machines, not the boy, not the shop — the board.

Take it apart line by line.

The list of things you may ask for

The board has nine lines and you cannot ask for a tenth. An API is likewise a **finite, published list of operations**. `GET /users/42` . `POST /payments` . `DELETE /files/abc` . If an operation is not on the list, it does not exist, however reasonable it sounds. “Make it look nicer” is not on the board, and neither is “give me everything you know about this user, sorted the way I like”.

The exact way to ask

Zoya does not say “some printing please”. She says which file, how many sheets, which colour, which sides, which binding. Every one of those is required, and the boy will ask if she leaves one out. In an API those are the **parameters** — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

The exact shape of the answer

She gets a stack of forty sheets, spiral bound, and a token numbered 14. She knows before she asks what form the answer will take. An API says the same: this operation returns a JSON object with these fields, of these types. **JSON** — JavaScript Object Notation.

tion — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [day 007](#); it looks like `{"id": 42, "name": "Zoya"}`.

The counter, and never going behind it

This is the part people miss, and it is the reason APIs exist at all. Zoya has never been behind the counter. That is not a restriction on her; it is a freedom for the shop. They replaced the machine in January and nothing broke, because nothing she said depended on which machine it was.

The technical name for this is **encapsulation**, or hiding the implementation. The API is the only thing anyone outside depends on, so everything behind it can change: a different language, a different database, ten servers instead of one. As long as the board says the same nine lines and they still mean the same nine things, the callers are undisturbed.

The reverse is the expensive half. Once other people are relying on the board, **you cannot change what a line means without breaking them**. If the shop quietly redefines “spiral, forty” to exclude the cover it used to include, every regular customer gets a surprise. In software that is called a **breaking change**, and it is why real APIs carry a version in the address — the `v1` in `https://api.stripe.com/v1/charges`. A new meaning goes in `v2` and `v1` keeps its promise.

Failing at the counter, not in the back

The morning the colour ran out, the boy said so at once. He did not take the pen drive, disappear for twenty minutes, and come back with nothing. A good API does the same: it answers with a clear, documented failure — an HTTP **status code** like `404 Not Found` or `429 Too Many Requests`, which you met on [day 005](#) — instead of hanging or returning something misleading. What an operation does when it fails is part of the promise, not an afterthought.

The word is bigger than the web

Most people say “API” and mean a web API reached over HTTP. That is one kind of three, and they are the same idea at three different distances:

KIND	YOU CALL IT	EXAMPLE	WHAT IT COSTS YOU
Library API	Inside your own process	<code>list.append(x)</code> , <code>heapq.heappush(h, x)</code>	Nanoseconds. A function call.
Operating system API	Down into the kernel	<code>open()</code> , <code>read()</code> , <code>send()</code> — the system calls from day 011	Microseconds. A mode switch.
Web API	Across a network	<code>GET https://api.github.com/users/torvalds</code>	Milliseconds. And it can fail.

`list.append` is an API. You know exactly what to pass and exactly what happens, and you have no idea how Python's list grows internally — and on [day 005](#) you learned that it reallocates, which is precisely the kind of thing an API is allowed to change without telling you.

The distances in that last column are the whole story of the next few weeks. A library call is free, a system call is cheap, and a network call is a thousand times more expensive and is allowed to simply not answer. Look back at the latency ladder from [day 010](#) if that number does not feel real yet.

4 The picture

One call to a real API, from the address bar to the answer and back:

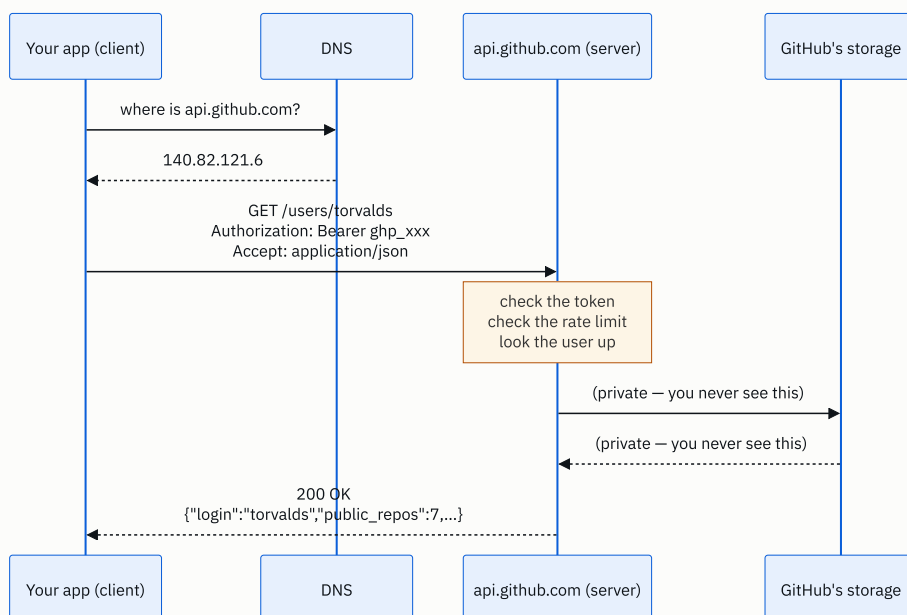


FIGURE 15.1

What to notice: the dotted box in the middle. Everything between the request arriving and the response leaving is GitHub's business, and it can change tomorrow. The only two things you are allowed to depend on are the arrow going right — the method, the path, the headers — and the arrow coming back — the status code and the shape of that JSON. Those two arrows are the API. The rest is the shop behind the counter.

Now the same picture as a boundary rather than a sequence, because this is the version you will draw in a design round:

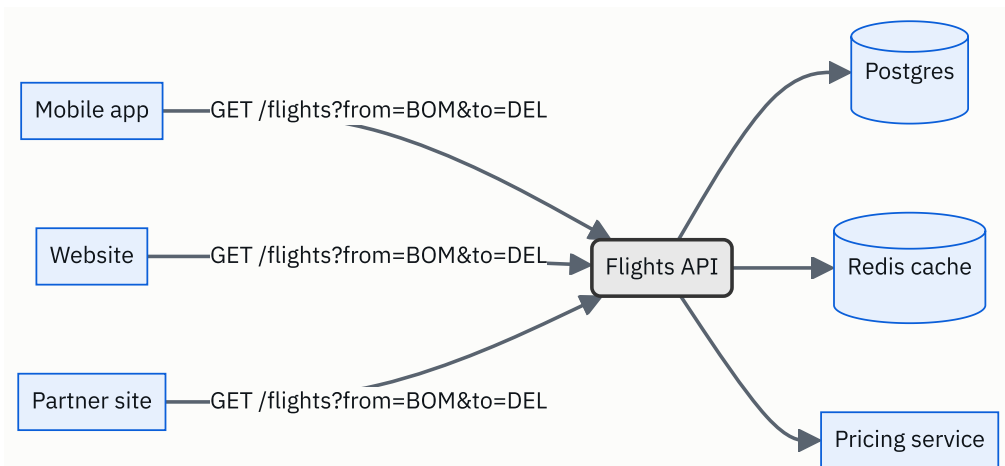


FIGURE 15.2

What to notice: three different callers, written by three different teams in three different languages, all say exactly the same sentence. That is what the API bought. And the three boxes on the right can be replaced one by one without any of the three callers being told.

5 How it actually works

The five parts of a web API request

Every HTTP API call is these five things, and nothing else. Take a real one — sending an SMS through Twilio:

```
POST https://api.twilio.com/2010-04-01/Accounts/AC123/Messages.json
Authorization: Basic QUMxMjM6c2VjcmV0
Content-Type: application/x-www-form-urlencoded

To=%2B919876543210&From=%2B12025550123&Body=Your+OTP+is+4417
```

1. **The method** — `POST`. What kind of operation this is. You met the verbs on [day 005](#).
2. **The address** — the host `api.twilio.com` plus the path. The path names the thing you are operating on. Note `2010-04-01` sitting in it: that is Twilio's version, frozen since 2010, and it is why code written fifteen years ago still runs.
3. **The headers** — who you are (`Authorization`) and what format you are sending (`Content-Type`).
4. **The body** — the parameters of this particular request.
5. **The response** — a status code and, usually, a JSON document.

Authentication: the API key

An API on the public internet needs to know who is calling, both to charge them and to stop abuse. The usual mechanism is an **API key** or **token** — a long random string the provider issues to you, which you send in the `Authorization` header on every request. Stripe gives you `sk_live_51H...` ; GitHub gives you `ghp_...` ; Google Cloud gives you a key tied to a project.

Two rules that come up in interviews. The key goes in a **header**, not in the address, because addresses get written into logs and browser history in plain text. And the key never ships inside a mobile app or a web page, because anyone can extract it — the app calls your own backend, and your backend holds the key. The full treatment of this is [day 019](#) and [day 020](#).

Rate limits

The provider caps how often you may call. GitHub allows 5,000 requests an hour on an authenticated token. Exceed it and you get `429 Too Many Requests` plus a header telling you when the window resets. This is not hostility; it is the only way a shared service survives one badly written client. How it is built is [day 023](#).

What the provider actually runs

Behind `api.stripe.com` there is a load balancer, a fleet of application servers, a database, a cache, and a queue — the shop behind the counter. You will design exactly this arrangement later in the course. The point for today is that **none of it is in the contract**. Stripe has rewritten large parts of that machinery repeatedly and `POST /v1/charges` has kept working.

SDKs are wrappers, not different things

When you `pip install stripe` and write `stripe.Charge.create(amount=2000, currency="inr")`, no new capability appears. The **SDK** — software development kit, a library the provider publishes in your language — builds the same HTTP request shown above, sends it, checks the status code, and turns the JSON into an object. It exists so you do not hand-assemble headers. If the SDK does not exist for your language, you make the raw call and lose nothing but convenience.

Real APIs worth being able to name

Interviewers like a concrete example, and a specific one beats “um, weather APIs”.

- **Stripe** — `POST /v1/charges` to take a payment. The usual gold standard for API design.
- **GitHub** — `GET /repos/{owner}/{repo}/issues`. Public, free to try, and famously well documented.

- **Google Maps Directions** — `GET /maps/api/directions/json?origin=...&destination=...`. Every cab app in the world sits on top of this idea.
- **Twilio** — sends the OTP text message you get when you log in to your bank.
- **OpenWeatherMap** — the one behind most weather widgets.
- **Amazon S3** — `PUT /bucket/key` to store a file. An API that is also a storage product.

And the ones you use without noticing: your phone's camera API, the Postgres wire protocol your database driver speaks, and the system calls your program makes to read a file.

What happens when it fails

This is the half that separates the two levels of answer, because a network call has failure modes a function call does not have.

- **The provider says no** — `4xx`. Your fault: bad parameters, missing token, over the rate limit. Retrying the identical request will fail identically, so do not retry it.
- **The provider broke** — `5xx`. Their fault. Retrying may work, and you should retry with increasing gaps rather than immediately, so that ten thousand clients do not all come back in the same millisecond.
- **Nothing comes back at all** — a timeout. This is the nasty one, because you do not know whether the operation happened. The request may have been carried out and the response lost. If the operation was “charge this card”, retrying blindly charges twice. The fix is **idempotency** — designing the operation so that doing it twice has the same effect as doing it once — which is [day 018](#), and it is one of the highest-value ideas in this whole phase.

6 The numbers

Take a weather app with **2 million daily users**, each opening it **6 times a day**. Every open is one call to your API, which in turn calls a paid provider.

Requests per day

$$2,000,000 \text{ users} \times 6 \text{ opens} = 12,000,000 \text{ requests/day}$$

Average requests per second

$$12,000,000 \div 86,400 \text{ seconds} = 139 \text{ requests/second}$$

Peak traffic is not the average. A reasonable rule of thumb for a consumer app is $3 \times$ the average at the busiest hour, so size for **420 requests/second**.

Bandwidth

A weather response is about 2 KB of JSON.

```
12,000,000 × 2 KB = 24,000,000 KB = 24 GB/day leaving your servers
```

Times thirty, that is **720 GB a month** of egress. At roughly \$0.09 per GB on a typical cloud egress price, about **\$65 a month** just to hand the bytes over. Small — but notice that if a careless developer returns the full 40 KB provider response instead of the 2 KB your app needs, that becomes \$1,300 a month for nothing.

The provider's rate limit

Say the upstream weather provider allows 60 calls a minute:

```
60 ÷ 60 = 1 call/second allowed  
139 calls/second needed
```

You are over by $139\times$. This is the arithmetic that forces a cache into the design: weather for one city does not change in ten minutes, and if 2 million users are spread over 500 cities, then

```
500 cities ÷ 600 seconds = 0.83 upstream calls/second
```

which fits inside the limit with room to spare. The cache turns 139 calls a second into under one. That is the shape of a good design-round argument: *the numbers forced the component, I did not just remember that caches exist.*

Chattiness

A mobile home screen needs profile, notifications, cart and recommendations. Four calls, each about 120 ms of round trip on a mobile network:

```
4 × 120 ms = 480 ms, if you make them one after another
```

Made in parallel, it is about 120 ms. Combined into a single endpoint that returns all four, about 150 ms including the extra work on the server. That gap — half a second against a seventh of a second — is why “how many calls does one screen need?” is a real design question and why GraphQL on [day 021](#) exists at all.

7 The trade-offs

What a published API costs you

You lose the freedom to change your mind. Before you publish, a function's signature is yours; after, it belongs to everyone calling it. Stripe still serves request shapes designed in 2011. Twilio's path still says `2010-04-01`. That is not sloppiness — it is the bill for having had customers for fifteen years. Every field you expose is a field you may be supporting a decade from now, which is why good API design is stingy: expose the minimum that does the job.

You add a network to a problem that might not need one. A library call is nanoseconds and cannot half-happen. A call across a network is milliseconds, can time out, can be answered twice, and forces you to think about retries and idempotency. Splitting two components apart behind an API is not free, and “we will make it a service” has burned a lot of teams who had one system and now have two systems plus a network between them.

Someone else's limits become yours. A rate limit, a maintenance window and an outage upstream are all now in your product.

What you would use instead, and when

- **A shared database instead of an API between two services.** Cheap and fast, and it destroys the boundary — the other team's schema change now breaks you, and nobody can tell who reads which table. Fine inside one small team, painful across three.
- **A library instead of a service.** If two components ship together and always run together, make it a function call. You keep the clean boundary and pay none of the network cost.
- **A message queue instead of a request-response API.** If the caller does not need an answer now — sending an email, generating a report — putting the job on a queue like Kafka or RabbitMQ is better than holding a connection open. The trade is that you no longer find out immediately whether it worked.

The sentence that separates candidates

I would not use a network API if the two sides are always deployed together and call each other millions of times a second. At that rate, milliseconds and partial failure dominate everything, and the right boundary is a function call in the same process. An API is for crossing a line between teams, deployments or trust domains — not for decorating a line that is not really there.

Say something like that in a design round and you sound like someone who has had to maintain one.

8 In the interview

How it gets asked

- “What is an API?” — the opener, usually in the first ten minutes of a phone screen. They want the plain-words version first and the precise version second.
- “Give me an example of an API you have used.” — the follow-up that catches people out. Have a specific one ready with a specific path.
- “Explain an API to someone non-technical.” — increasingly common, because product companies care whether you can talk to product managers.
- “Why would you put an API between these two components instead of letting one read the other’s database?” — the same question in design-round clothing, and the one worth real marks.

What to say out loud, in the first ninety seconds

1. **One sentence, plain.** “An API is the published list of things one piece of software will do for another, plus exactly how to ask and exactly what comes back.”
2. **The key word.** “The word that does the work is interface — the caller gets the promise and never gets the implementation.”
3. **Give the concrete example immediately.** “For example, `GET https://api.github.com/users/torvalds` with a token in the Authorization header returns a JSON object with that user’s public repo count. I never learn anything about GitHub’s database, and they can change it whenever they like.”
4. **Say why that matters.** “That is the whole value. Either side can be rewritten as long as the contract holds.”
5. **Name the cost, unprompted.** “The price is that once it is published, you cannot change what it means without breaking callers — which is why real APIs carry a version in the path.”
6. **Widen it, briefly.** “And it is not only web APIs. `list.append` is an API, and so is a system call. Same idea at three distances: nanoseconds, microseconds, milliseconds.”
7. **Stop.** This question has a right length, and it is about sixty seconds.

The follow-ups

“What makes an API a good one?” Four things. It is **predictable** — similar operations look similar, so you can guess the next endpoint after learning two. It is **small** — it exposes the minimum that does the job, because every field is a field you support for

years. It **fails clearly** — documented status codes, a machine-readable error body, and never a `200 OK` with the word “error” hidden inside it. And it is **versioned**, so the promise can evolve without breaking anyone. Stripe is the standard example of all four. The design details are the next three days of this course.

“Why an API instead of just sharing the database?” Because sharing a database shares your internal structure, and structure is exactly the thing you want to be free to change. If another team reads my tables, then renaming a column is now their outage, and I cannot find out who depends on what without asking everybody. An API gives one narrow, documented surface, so I can rewrite the storage behind it — or split it across three stores — without anyone noticing. It also gives one place to enforce permissions, validation and rate limits, instead of trusting every caller to behave. The cost is real: a network hop, a serialisation step, and failure modes a query does not have. Inside a single small team a shared database is often the right call; across teams it almost never is.

“You call a payment API, it times out, and you never learn whether the payment went through. What do you do?” I do not blindly retry, because a timeout means the request may well have been carried out and only the response was lost — retrying charges the customer twice. The fix is to make the operation idempotent: I generate a unique key for the attempt, send it as an idempotency key on the request, and the provider promises that two requests carrying the same key are executed once and return the same answer. Then retrying is safe. Stripe does exactly this with an `Idempotency-Key` header. If the provider offers no such thing, the fallback is to query the provider for the status of that attempt before retrying, and to record my own intent in my own database before I ever make the call, so I can reconcile afterwards.

A model answer

“An API is the published list of things one piece of software will do for another, together with exactly how you ask for each one and exactly what shape the answer takes. The word doing the work is *interface* — you get the promise, you never get the machinery.

The example I would give is GitHub’s. If I send `GET https://api.github.com/users/torvalds` with a token in the Authorization header, I get back `200 OK` and a JSON object with fields like `login` and `public_repos`. I know that before I send it, because it is documented. I have no idea what database GitHub keeps that in, how many servers answered me, or what language it is written in — and that is deliberate on both sides. They are free to change all of it, and my code keeps working, as long as the request and the response keep their shape.

The price of that freedom is paid in the other direction: once people are calling it, I cannot change what an operation *means* without breaking them. That is why serious APIs carry a version in the path — Stripe’s `/v1/charges`, Twilio’s `/2010-04-01/`. New meaning, new version, and the old one keeps its promise.

It is also worth saying that a web API is only the most visible kind. `list.append` in Python is an API, and so is a system call like `read()`. Same idea at three distances — a function call is nanoseconds, a system call is microseconds, a network call is milliseconds and is allowed to simply not answer. That last difference is what forces timeouts, retries and idempotency into any design that crosses a network.

So when I put an API between two components, what I am really buying is the freedom to change each side independently, plus one place to enforce authentication, validation and rate limits. What I am paying is latency and a new class of failure. I would not pay it for two components that always deploy together and call each other constantly — that boundary should stay a function call.”

9 Recall card

- **An API is a promise, not a mechanism:** the list of operations, how to ask, what comes back.
- **The caller depends on the contract and nothing else** — so everything behind it is free to change. That is the entire point.
- **The price is that you cannot change what it means.** Hence `v1` in the path.

- **Three distances, same idea:** library call (nanoseconds), system call (microseconds), network call (milliseconds, and it can fail).
- **Have one concrete example ready:** `GET https://api.github.com/users/torvalds`
→ `200 OK` and a JSON body.

PRACTICE

Practice — Moving elements: zeros, duplicates, and the write pointer

DSA topic: Moving elements: zeros, duplicates, and the write pointer **System design topic:** What an API is

Code these, in this order

Four problems that are the same five lines with a different `if`. Do them in order and the fourth one will feel like a variation rather than a new problem. That is the point of doing them in order.

Before each one, say out loud:

1. What am I keeping, and what am I throwing away?
2. Does the order of the survivors have to be preserved?
3. Am I filling the tail, or returning a count?
4. What does the write index equal when the loop ends?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Remove Element	LeetCode 27 (Easy)	The bare pattern, and whether you understand that nothing is deleted — you return a count and the grader ignores everything past it.
2	Move Zeroes	LeetCode 283 (Easy)	The same loop plus the second phase. The trap is stopping after the first loop and leaving the stale tail behind.
3	Remove Duplicates from Sorted Array	LeetCode 26 (Easy)	Comparing against <code>items[write - 1]</code> — what you <i>kept</i> — rather than <code>items[read - 1]</code> . Also the empty-input guard, since <code>write</code> starts at 1.
4	Remove Duplicates from Sorted Array II	LeetCode 80 (Medium)	<code>items[write - 2]</code> . If you built the habit in problem 3 this is a one-character change; if you compared against <code>read - 1</code> it is a rewrite.

On problem 1, do it twice

Write the order-preserving version first — the plain write pointer. Then write the version that does **not** preserve order: swap the unwanted element with the last one and shrink the range, without advancing the read index.

Then answer, out loud: on `[3, 2, 2, 3]` with target `3`, how many writes does each version do? And on an array of a million elements with exactly two matches?

That second answer is why the non-order-preserving version exists.

On problem 2, run the broken versions first

Type these in and predict the output of each before you run it. Being wrong here is the exercise.

```
items = [0, 0, 1]
for x in items:
    if x == 0:
        items.remove(x)
print(items)
```

```
nums = [1, 0, 0, 0]
for i in range(len(nums)):
    if nums[i] == 0:
        del nums[i]
print(nums)
```

```
def move_zeroes(items):
    write = 0
    for read in range(len(items)):
        if items[read] != 0:
            items[write] = items[read]
            write += 1
    return items

print(move_zeroes([0, 1, 0, 3, 12]))
```

Then, in one sentence each: which one gives a wrong answer silently, which one crashes, and which one is right in front and wrong at the back. The first is the dangerous one, and you should be able to say why.

On problems 3 and 4, generalise before you move on

Once problem 4 passes, replace the `2` with a parameter `k` and check it still passes for `k = 2`. Then test it with `k = 1` and confirm it solves problem 3. One function, both problems — that is the sign you have understood the pattern rather than memorised two of them.

The tracing drill

Take `[0, 1, 0, 3, 12]` and, without running anything, say out loud the value of `read`, `write` and the whole list after every single turn of the loop. Five turns. Then run it with a `print` inside the loop and check yourself.

Do the same for `[1, 2, 3]` — where `write` and `read` never separate — and for `[0, 0, 0]`, where they separate immediately and the first loop keeps nothing.

The stretch problem

Sort Colors — LeetCode 75 (Medium). Three values, 0, 1 and 2, to be sorted in one pass. It is the write pointer with *two* write indices, one coming from each end, and it is the classic next step from today. Try it before you look anything up; you have everything you need.

The API drill

Pick a real API and answer these five questions about it without opening the documentation. Use GitHub's if you have no other — it needs no sign-up for public data.

1. What is one operation it offers, written out as a method and a path?
2. What must you send with the request, and where does it go — path, query string, headers, body?
3. What comes back on success, and what fields does it have?
4. What comes back when you get it wrong, and what when they are broken?
5. Where is the version, and what would happen to your code if they changed the meaning of an existing field?

Then do the thing that makes it real: make the call. In a terminal, `curl https://api.github.com/users/torvalds` costs nothing and returns a JSON body you can read. Look at the response headers too — `x-ratelimit-remaining` is right there, and it makes the rate limit stop being an abstract idea.

The explain-it-twice drill

Explain what an API is **twice**, out loud, timed.

- **Sixty seconds, to a non-engineer.** No status codes, no JSON, no HTTP. If they could not repeat it back, it did not work.
- **Sixty seconds, to an engineer.** Method, path, headers, body, status code, response shape, versioning, and one named example with a real path.

Most candidates can do neither cleanly on the first attempt. It is the same fact twice, and being able to switch registers on demand is exactly what the question is checking.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Move all zeros to the end while keeping the order of the other elements.* Ask the two contract questions first — in place, and order preserved. Then name the two indices and state the rule that `write` never overtakes `read`. Write the loop, then the tail fill,

then give $O(n)$ time and $O(1)$ extra space. Finish by saying why you are not deleting as you go.

2. *What is an API? Give me an example of one you have used.* One plain sentence, then a concrete path with a real host in it, then the reason it matters — either side can change behind the contract — then the price, which is that you cannot change what it means once people depend on it. Sixty seconds, then stop talking.
 3. *This array is sorted. Remove the duplicates in place and return the new length. Now allow each value at most twice.* The point is the second half. Say why `items[write - 1]` becomes `items[write - 2]`, why the first two elements pass unconditionally, and why comparing against `read - 1` instead would have made this a rewrite rather than a one-character change.
-

Before you move on

- [] I ask “in place?” and “order preserved?” before writing a line of this family.
- [] I can state the rule — `write ≤ read`, so overwriting is always safe — in one sentence.
- [] I always test the **whole** output list, not just the front of it.
- [] I never delete from a list while looping over it, and I can show what breaks.
- [] I can define an API in one plain sentence, with a real method and path as the example.
- [] I can say what an API costs you, not only what it buys you.
- [] I can redraw today’s request-and-response diagram from memory, in whatever tool I like.

2D arrays and matrix traversal

DATA STRUCTURES AND ALGORITHMS

2D arrays and matrix traversal

You can walk a matrix by row, by column and by diagonal without index confusion.

SYSTEM DESIGN

REST, properly

You can list the REST constraints and say which ones real APIs actually follow.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Print the matrix diagonally.*
- *What makes an API RESTful?*

1 What this is, and why they ask it

A **matrix** is a rectangle of values addressed by two numbers instead of one: which row, then which column. In Python it is a list whose elements are themselves lists, and you reach a single value with `matrix[row][column]` — row first, always. Everything else today is about walking that rectangle in the four orders an interviewer will ask for: along the rows, down the columns, along the diagonals that run top-left to bottom-right, and along the diagonals that run the other way.

Nothing here is conceptually hard. The reason it fills whiteboards with wrong code is that the two numbers are easy to swap and the boundaries are easy to get off by one, and unlike a one-dimensional array you cannot hold the whole thing in your head. The candidates who do well are the ones who write down what `r` and `c` mean before they write a loop, and who know the two identities in §3 by heart.

It shows up directly as *print the matrix in diagonal order* (LeetCode 498) and *rotate the image* (LeetCode 48, which is tomorrow), and indirectly as the substrate for a great deal of the course still to come. Grid problems in graphs — flood fill, number of islands, shortest path in a maze on [days 125–142](#) — are matrix traversal with a queue attached. Dynamic programming tables from [day 143](#) onwards are matrices you fill in a particular order. Getting fluent with `matrix[r][c]` now pays for twelve weeks.

2 The story

Iqbal is tiling a bathroom floor in a flat in Bhopal, and the owner has been standing in the doorway since nine o'clock. It is not a big room. Twelve tiles from the door wall to the far wall, nine tiles across, and Iqbal has counted it twice because the tiles came in one box and there is no second box.

He works from the far corner backwards, because you cannot stand on what you have just laid. One line across, then the next line, then the next. He puts little plastic crosses between them so the gaps stay even. Halfway through the fourth line the owner asks him something and Iqbal answers without looking up, because by now he can name any spot on that floor with two numbers: how many lines in from the far wall, and how many tiles along from the left.

Then the owner's wife comes and looks at it and says she wants the dark tiles — there are twenty spare, a different colour — to run corner to corner.

Iqbal stops with a tile in his hand and works it out, and this is the bit worth watching. In the first line, the dark one is the first tile along. In the second line it is the second tile along. Third line, third tile. Each line, it moves one further from the left, exactly as it moves one further from the wall. So he does not have to measure anything: he just counts one more each time.

Then she says she wants a second dark line, the other way, crossing it. So he starts at the other end. First line, ninth tile along. Second line, eighth. Third line, seventh. One further from the wall, one *back* towards the left. And Iqbal notices something that makes it easy: on that second line, if you add the two numbers together — how many lines in, how many tiles along — you get ten every single time. First line and ninth tile, ten. Second and eighth, ten. He stops counting backwards and just checks the two numbers add up to ten.

He finishes at half past four, and the crosses come out the next morning.

3 The idea in plain English

Iqbal's floor is a **matrix**: a rectangle of values where every position needs two numbers to name it. His “how many lines in” is the **row**, and his “how many tiles along” is the **column**.

How Python stores it

Python has no separate matrix type in the standard library. A matrix is a **list of lists**: the outer list holds the rows, and each row is itself a list holding that row's values.

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9],
          [10, 11, 12]]
```

Four rows, three columns. `matrix[2]` is the whole third row, `[7, 8, 9]`. `matrix[2][1]` is the value in the third row, second column: `8`.

Row first, then column. Always. Say it out loud once now, because half the bugs in this topic are the two numbers the wrong way round, and there is no error message when the matrix is square.

The two sizes, and why you need both

```
rows = len(matrix)          # 4 – how many rows
cols = len(matrix[0])       # 3 – how long a row is
```

`len(matrix)` counts the outer list, which is the number of rows. `len(matrix[0])` looks inside the first row and counts it, giving the number of columns. Beginners write `len(matrix)` for both, which works perfectly on a square matrix and fails on every other one. Interviewers hand you a `3 × 5` on purpose.

Both lines fail on an empty matrix, and `len(matrix[0])` fails on `[[[]]]`, so real code starts with a guard. §7 has the exact error.

Walking it by row

```
for r in range(rows):
    for c in range(cols):
        print(matrix[r][c])
```

The outer loop picks a row and holds it still; the inner loop sweeps across that row. This is **row-major order**, and it prints `1 2 3 4 5 6 ...`. It is Iqbal laying one line across before starting the next.

Row-major is also the order the values physically sit near each other in memory, which — from [day 009](#) and [day 010](#) — is the order the machine likes. Walking a big matrix by row is measurably faster than walking the same matrix by column, and it is a good sentence to have ready.

Walking it by column

Swap which loop is on the outside. Nothing else changes.

```
for c in range(cols):
    for r in range(rows):
        print(matrix[r][c])
```

This prints `1 4 7 10 2 5 8 11 ...` — straight down the first column, then down the second. Notice that `matrix[r][c]` is written exactly the same way in both versions. **What changed is which number is held still, not the order you write them in.** This is the single most common source of confusion in the topic and it is worth staring at until it is obvious.

The two diagonal families

Here is where Iqbal earns his keep, and here are the only two facts you need.

A diagonal going down-right — the direction of the first dark line — moves one row down and one column right at each step. So `r` goes up by one and `c` goes up by one, which means `r - c` **never changes** along that diagonal.

A **diagonal going down-left** — the second dark line, the crossing one — moves one row down and one column *left*. So `r` goes up by one and `c` goes down by one, which means `r + c` **never changes**. That is Iqbal’s “they always add up to ten”.

That gives you a way to group every cell into diagonals without any clever index arithmetic at all:

- To collect the down-right diagonals, group cells by the value of `r - c`.
- To collect the down-left diagonals — usually called the **anti-diagonals** — group cells by the value of `r + c`.

For a matrix with `rows` rows and `cols` columns there are exactly `rows + cols - 1` diagonals in each family. On a `4 × 3` matrix that is `4 + 3 - 1 = 6`, and their sizes go 1, 2, 3, 3, 2, 1.

`r - c` can be negative — it runs from `-(cols - 1)` up to `rows - 1` — which is why grouping into a dictionary is easier than allocating a list and shifting everything. `r + c` runs from `0` to `rows + cols - 2`, which is nice and simple, so most “diagonal” interview problems are stated in terms of the anti-diagonals.

The one habit that prevents most bugs

Before writing any loop, say what your two variables mean, in words:

`r` is which row I am on, from 0 to `rows - 1`. `c` is which column I am on, from 0 to `cols - 1`. The value is `matrix[r][c]`.

Then never write `matrix[c][r]` by accident, because you know what the letters mean.

4 The picture

The `4 × 3` matrix, with both numbers marked:

	c=0	c=1	c=2
r=0	1	2	3
r=1	4	5	6
r=2	7	8	9
r=3	10	11	12

`matrix[2][1]` is 8 — row 2, column 1. Row first.

Now the same rectangle with `r + c` written in each cell instead of the value:

	c=0	c=1	c=2
r=0	0	1	2
r=1	1	2	3
r=2	2	3	4
r=3	3	4	5

What to notice: every cell holding the same number lies on one straight line running from top-right to bottom-left. The 2s are `[3, 5, 7]`. The 3s are `[6, 8, 10]`. Six distinct values, 0 through 5, so six anti-diagonals — which is `rows + cols - 1 = 4 + 3 - 1`.

And with `r - c` in each cell:

	c=0	c=1	c=2
r=0	0	-1	-2
r=1	1	0	-1
r=2	2	1	0
r=3	3	2	1

What to notice: the zeros run top-left to bottom-right — that is the main diagonal, `[1, 5, 9]` — and the values go negative above it. Six distinct values again, from `-2` to `3`.

Finally, the two loop orders side by side. Same `matrix[r][c]`, different nesting:

by row (row-major)	by column (column-major)
1 → 2 → 3	1 2 3
4 → 5 → 6	v v v
7 → 8 → 9	4 5 6
	v v v
	7 8 9
visits: 1 2 3 4 5 6 7 8 9	visits: 1 4 7 2 5 8 3 6 9

What to notice: neither loop skips or repeats a cell. Both visit all nine exactly once. Only the order differs — and only because of which `for` is on the outside.

5 The code, built step by step

Reading the two sizes safely

```
def size(matrix: list[list[int]]) → tuple[int, int]:
    if not matrix or not matrix[0]:
        return 0, 0
    return len(matrix), len(matrix[0])
```

`not matrix` catches `[]`. `not matrix[0]` catches `[[] ]` — a matrix with one row that has nothing in it. Both are inputs graders actually send, and both crash the naive `len(matrix[0])`.

Summing every row, and every column

```
row_sums = [sum(row) for row in matrix]
```

Each element of the outer list is a row, so you can loop over rows directly without touching indices at all. Prefer this when you do not need the position.

```
col_sums = [sum(matrix[r][c] for r in range(rows)) for c in range(cols)]
```

Columns have no such shortcut, because a column is not stored anywhere — it is one value picked out of each row. The `c` is fixed by the outer comprehension and `r` sweeps.

Transposing: turning rows into columns

```
transposed = [list(t) for t in zip(*matrix)]
```

`zip(*matrix)` is worth ten seconds. The `*` unpacks the outer list, so `zip` receives each row as a separate argument, and `zip` then takes the first item of every row, then the second of every row, and so on. Those groups are exactly the columns. On the 4×3 matrix above it gives `[[1, 4, 7, 10], [2, 5, 8, 11], [3, 6, 9, 12]]` — a 3×4 .

Write the explicit version once so you know what it is doing:

```
transposed = [[matrix[r][c] for r in range(rows)] for c in range(cols)]
```

In an interview, write `zip(*matrix)` and then say the two-line explanation. That combination — the idiom plus proof you know why it works — reads much better than either alone.

Grouping cells into diagonals

This is the whole trick of the day, and it is four lines.

```
from collections import defaultdict

groups = defaultdict(list)
for r in range(rows):
    for c in range(cols):
        groups[r + c].append(matrix[r][c])
```

`defaultdict(list)` is a dictionary that creates an empty list the first time you touch a missing key, so you never write `if key not in groups`. You met dictionaries on [day 006](#).

After this loop, `groups[0]` is `[1]`, `groups[1]` is `[2, 4]`, `groups[2]` is `[3, 5, 7]`, and so on. Each list is one anti-diagonal, in top-to-bottom order, because the loops visit rows in increasing order.

Change the key to `r - c` and you get the other family instead. That is the entire difference.

Diagonal order: the zigzag

LeetCode 498 wants the anti-diagonals emitted in order, but alternating direction — the first one upwards, the second downwards, and so on. Since each `groups[s]` is already in top-to-bottom order, “upwards” is just that list reversed.

```
out = []
for s in range(rows + cols - 1):
    band = groups[s]
    out.extend(reversed(band) if s % 2 == 0 else band)
```

`s % 2 == 0` picks the even-numbered diagonals to reverse, which matches the problem’s rule that the first diagonal goes up-and-right. `rows + cols - 1` is the diagonal count from §3, so this loop touches every group exactly once and none twice.

Visiting the four neighbours of a cell

Grid problems from [day 125](#) onwards all need this, and the idiom is worth learning here where it is easy.


```
DIRECTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up, down, left, right

def neighbours(r: int, c: int, rows: int, cols: int) → list[tuple[int, int]]:
    out = []
    for dr, dc in DIRECTIONS:
        nr, nc = r + dr, c + dc
        if 0 ≤ nr < rows and 0 ≤ nc < cols: # the bounds check
            out.append((nr, nc))
    return out
```

`0 ≤ nr < rows` is Python's chained comparison and reads exactly as it looks: `nr` is at least 0 and less than `rows`. Doing the check *before* touching `matrix[nr][nc]` is the habit. Negative indices do not raise in Python — `matrix[-1]` is the last row — so an unchecked negative gives you a wrong answer silently rather than a crash, which is much worse.

The complete solutions

```
from collections import defaultdict

def size(matrix: list[list[int]]) → tuple[int, int]:
    """Rows and columns, safe on [] and [[]]."""
    if not matrix or not matrix[0]:
        return 0, 0
    return len(matrix), len(matrix[0])

def by_row(matrix: list[list[int]]) → list[int]:
    """Row-major: left to right, top to bottom."""
    rows, cols = size(matrix)
    return [matrix[r][c] for r in range(rows) for c in range(cols)]

def by_column(matrix: list[list[int]]) → list[int]:
    """Column-major: top to bottom, left to right. Same expression, loops swapped."""
    rows, cols = size(matrix)
    return [matrix[r][c] for c in range(cols) for r in range(rows)]

def transpose(matrix: list[list[int]]) → list[list[int]]:
    """Rows become columns. Returns a new matrix; does not modify the input."""
    return [list(t) for t in zip(*matrix)]

def anti_diagonals(matrix: list[list[int]]) → list[list[int]]:
    """Every top-right-to-bottom-left diagonal, each read top to bottom.

    Cells on one such diagonal all share the same value of r + c.
    """
    rows, cols = size(matrix)
    groups: defaultdict[int, list[int]] = defaultdict(list)
    for r in range(rows):
        for c in range(cols):
            groups[r + c].append(matrix[r][c])
    return [groups[s] for s in range(rows + cols - 1)]

def main_diagonals(matrix: list[list[int]]) → list[list[int]]:
    """Every top-left-to-bottom-right diagonal. Cells share the same value of r - c."""
    rows, cols = size(matrix)
    groups: defaultdict[int, list[int]] = defaultdict(list)
    for r in range(rows):
        for c in range(cols):
            groups[r - c].append(matrix[r][c])
    return [groups[d] for d in range(-(cols - 1), rows)]

def diagonal_order(matrix: list[list[int]]) → list[int]:
    """LeetCode 498. Anti-diagonals, alternating direction, flattened."""
    rows, cols = size(matrix)
    if rows == 0:
        return []
    groups: defaultdict[int, list[int]] = defaultdict(list)
    for r in range(rows):
        for c in range(cols):
            groups[r + c].append(matrix[r][c])

    out: list[int] = []
    for s in range(rows + cols - 1):
        band = groups[s]
        out.extend(reversed(band) if s % 2 == 0 else band)    # even bands run upwards
```

```

return out

if __name__ == "__main__":
    m = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9],
          [10, 11, 12]]

    print(size(m))           # (4, 3)
    print(by_row(m))         # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
    print(by_column(m))      # [1, 4, 7, 10, 2, 5, 8, 11, 3, 6, 9, 12]
    print(transpose(m))      # [[1, 4, 7, 10], [2, 5, 8, 11], [3, 6, 9, 12]]
    print(anti_diagonals(m)) # [[1], [2, 4], [3, 5, 7], [6, 8, 10], [9, 11], [12]]
    print(main_diagonals(m)) # [[3], [2, 6], [1, 5, 9], [4, 8, 12], [7, 11], [10]]
    print(diagonal_order(m)) # [1, 2, 4, 7, 5, 3, 6, 8, 10, 11, 9, 12]

    square = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
    print(diagonal_order(square)) # [1, 2, 4, 7, 5, 3, 6, 8, 9]
    print(diagonal_order([]))    # []
    print(diagonal_order([[]])) # []

```

Building a grid from nothing

Two lines that look the same and are not. This one is right:

```
grid = [[0] * cols for _ in range(rows)]
```

This one is a bug, and §7 shows it going wrong:

```
grid = [[0] * cols] * rows    # do not do this
```

6 What it costs

Any full traversal

The outer loop runs `rows` times. For each of those the inner loop runs `cols` times. So the body runs `rows × cols` times, and each run is constant work — one lookup, one append. That is **$O(\text{rows} \times \text{cols})$** time.

Get the wording right, because it earns marks. If the matrix is `n × n`, that is n^2 , and saying $O(n^2)$ is fine. If it is `m × n`, then $O(n^2)$ is simply wrong and $O(m \times n)$ is right. Better still: **it is linear in the number of cells**, because it touches each cell exactly once and there is no way to do less than that if you have to look at everything.

Concretely, a `1000 × 1000` matrix has a million cells. At roughly ten million simple Python operations a second, that is a fraction of a second. A `10,000 × 10,000` matrix has a hundred million cells and takes tens of seconds — the point at which you stop and ask whether you really need to visit them all.

by_row versus by_column

Identical counts: $\text{rows} \times \text{cols}$ visits either way. But they are not equally fast in practice. Row-major follows memory order, so a whole cache line's worth of neighbouring values arrives with the first one. Column-major jumps cols positions between reads and throws that away. From the latency ladder on [day 010](#), a value already in the cache is roughly a hundred times closer than one that has to come from main memory. Both are $O(\text{rows} \times \text{cols})$; the constant factor between them can be several times.

The diagonal grouping

One pass over every cell, so $\text{rows} \times \text{cols}$ visits — $O(\text{rows} \times \text{cols})$ time, the same as any other traversal. The extra cost is space: every value is copied into a group, so the dictionary holds $\text{rows} \times \text{cols}$ values in total, spread across $\text{rows} + \text{cols} - 1$ lists. That is $O(\text{rows} \times \text{cols})$ extra space.

If an interviewer objects to the extra space, the alternative is to walk each diagonal directly with a moving (r, c) and emit as you go, which is $O(1)$ extra space beyond the output. It is noticeably fiddlier to get right under pressure. Say that the grouping version is $O(\text{rows} \times \text{cols})$ extra, that a direct walk is $O(1)$, and that you would write the grouping version first and tighten it if the memory mattered. That answer is better than either code alone.

The neighbour check

Four directions, one comparison each, no loop over the matrix — $O(1)$ per cell. When every cell does it, the total is four visits per cell, so still $O(\text{rows} \times \text{cols})$ overall.

7 The traps

The near-miss that destroys your grid: $[[0] * \text{cols}] * \text{rows}$

```
grid = [[0] * 3] * 4
grid[1][2] = 7
for row in grid:
    print(row)
```

You set one cell. Here is what you get:

```
[0, 0, 7]
[0, 0, 7]
[0, 0, 7]
[0, 0, 7]
```

Four rows changed. The reason is that `* 4` does not make four rows — it makes **four references to the same single row**. From [day 005](#), a list holds references, and copying a reference does not copy the thing it points at. So `grid[0]`, `grid[1]`, `grid[2]` and `grid[3]` are all the same list, and writing through any one of them is visible through all four.

The fix builds a fresh list per row:

```
grid = [[0] * 3 for _ in range(4)]
```

The comprehension evaluates `[0] * 3` once per iteration, producing four separate lists. Note that the *inner* `[0] * 3` is perfectly safe, because `0` is a number and there is nothing to share.

This bug is nasty because a program with it often runs and produces a plausible-looking wrong answer. If your grid updates seem to “leak” across rows, this is why.

The real error: the two numbers the wrong way round

```
matrix = [[1, 2, 3], [4, 5, 6]]
for c in range(len(matrix)):
    for r in range(len(matrix[0])):
        print(matrix[r][c], end=" ")
```

The intention was a column walk. The loop variables got renamed but the ranges did not follow.

```
1 4 Traceback (most recent call last):
  File "d16c.py", line 4, in <module>
    print(matrix[r][c], end=" ")
        ~~~~~^
IndexError: list index out of range
```

It printed two correct values first, which is exactly why this is worth showing. `r` reached 2, but there are only two rows, so `matrix[2]` does not exist. On a **square** matrix the same mistake raises nothing at all and silently transposes your answer. Test on a non-square matrix, always.

The real error: the empty matrix

```
matrix = []
rows = len(matrix)
cols = len(matrix[0])
```

```
Traceback (most recent call last):
  File "d16d.py", line 3, in <module>
    cols = len(matrix[0])
            ~~~~~^
IndexError: list index out of range
```

`len(matrix)` was happy and returned 0. `len(matrix[0])` reached into a row that is not there. `[[[]]` fails the same way one level down: there is a row, but it has no columns, so every later `matrix[0][0]` breaks. Guard both, in one line: `if not matrix or not matrix[0]`.

The real error: the tempting NumPy syntax

If you have seen NumPy, you may reach for this:

```
n = [[1, 2], [3, 4]]
print(n[0, 1])
```

```
TypeError: list indices must be integers or slices, not tuple
```

`n[0, 1]` passes the tuple `(0, 1)` as a single subscript. NumPy arrays accept that; plain Python lists do not. Interviews almost always want plain lists, so write `n[0][1]`.

The near-miss: assuming the matrix is square

```
for r in range(len(matrix)):
    for c in range(len(matrix)):    # should be len(matrix[0])
        ...
```

Correct on every square test case and broken on the first rectangular one — either an `IndexError` if there are more rows than columns, or a silently truncated answer if there are more columns than rows. **Compute `rows` and `cols` once, into named variables, before the loops.** Then this mistake becomes impossible to make.

The near-miss: negative indices do not raise

```
r, c = 0, 0
print(matrix[r - 1][c])    # you meant "the row above" - there isn't one
```

There is no error. `matrix[-1]` is the *last* row, so a walk that steps off the top of the grid quietly wraps round to the bottom and gives a confidently wrong answer. This is why the neighbour helper checks `0 ≤ nr` before it reads anything, and it is the single biggest source of hard-to-find bugs in grid problems later in the course.

8 In the interview

How it gets asked

- “Print the matrix in diagonal order.” — LeetCode 498. Usually with the zigzag, sometimes without; ask which.
- “Sum each row, and each column.” — a warm-up, five minutes, testing only whether you keep the two numbers straight.
- “Transpose this matrix.” — then, almost always, “now do it in place”, which is tomorrow’s lesson.
- “Given a grid of 0s and 1s, ...” — the opening of every island, flood-fill and maze question. The traversal is today; the rest arrives with graphs.

What to say out loud, in the first ninety seconds

1. **Fix the vocabulary before anything else.** “Let me set my notation: r is the row index from 0 to rows minus 1, c is the column index, and I access a cell as `matrix[r][c]` — row first.” Ten seconds, and it prevents the commonest bug live on the whiteboard.
2. **Ask whether it is square.** “Can I assume it is square, or should I handle m by n ?” If they say rectangular, write `rows` and `cols` as separate variables immediately.
3. **Ask about empties.** “What should I return for an empty matrix, or a matrix with empty rows?”
4. **For the diagonal question, state the identity.** “Every cell on a top-right-to-bottom-left diagonal has the same value of $r + c$. So I can group all cells by $r + c$ in one pass, and there are exactly rows plus columns minus one groups.” This is the insight the question is testing, and saying it first means the code is then obvious.
5. **Name the zigzag as a separate concern.** “The grouping gives each diagonal top to bottom. The alternating direction is then just reversing the even-numbered ones.” Splitting the problem in two is most of the battle.
6. **Give the cost.** “One pass over every cell, so $O(m \times n)$ time — linear in the number of cells, which is the best possible since I have to look at all of them. $O(m \times n)$ extra space for the groups.”
7. **Offer the tightening.** “If the extra space matters I can walk each diagonal directly with a moving row and column and emit as I go, which is $O(1)$ extra.”

The follow-ups

“Can you do it without the extra space?” Yes. Instead of grouping, I walk each diagonal directly. For anti-diagonal number s , the cells are those with $r + c = s$, so r ranges over $\max(0, s - \text{cols} + 1)$ to $\min(s, \text{rows} - 1)$ and c is $s - r$. I emit that range forwards or backwards depending on whether s is even, and I never store anything but the output. Same $O(m \times n)$ time, $O(1)$ extra space. I would write the

grouping version first because it is far harder to get the bounds wrong, and tighten it only if asked — those two `max / min` expressions are exactly where people lose ten minutes under pressure.

“Why is walking by row faster than walking by column, if both are $O(m \times n)$?”

Because complexity classes count operations and hardware counts cache misses. A matrix is stored row by row, so consecutive cells in a row are next to each other in memory, and reading one pulls its neighbours into cache along with it. Walking down a column jumps a whole row’s width between reads, so most reads miss the cache and go to main memory, which is around a hundred times slower. Both are linear in the number of cells; the constant factor differs by a factor of several. It is the same reason a sequential disk read beats a random one.

“How would you rotate the matrix by 90 degrees?” Transpose it, then reverse each row. Transposing swaps `matrix[r][c]` with `matrix[c][r]`, which reflects across the main diagonal; reversing each row then reflects horizontally, and the two reflections together are a 90-degree clockwise rotation. In place, the transpose loop must only visit `c > r`, otherwise you swap every pair twice and end up back where you started. That is $O(n^2)$ time and $O(1)$ extra space, and it only works on a square matrix — a rectangular rotation has to allocate, because the shape changes from `m × n` to `n × m`.

“The grid is a million by a million. Now what?” Then a full traversal is a trillion cells and is off the table, so the question has changed shape. Either the grid is sparse, in which case I would not store it as a list of lists at all — I would keep a dictionary from `(r, c)` to value and only ever iterate over the entries that exist — or the answer does not need every cell, in which case I would look for structure to exploit, such as sortedness along rows and columns, and binary search instead of scanning. If it genuinely is dense and every cell matters, it is not one machine’s problem any more; you partition it into blocks and process them in parallel.

A model answer

“First let me pin the notation, because this is where these questions go wrong. `r` is the row index, running 0 to rows minus 1. `c` is the column index, 0 to columns minus 1. A cell is `matrix[r][c]` — row first. And can I assume it is square, or should I handle `m` by `n`?

...Rectangular, fine, so I will keep `rows` and `cols` as separate variables.

The observation the problem turns on is this: on any diagonal running top-right to bottom-left, each step goes one row down and one column left, so `r` increases by one while `c` decreases by one — which means `r + c` is the same for every cell on that diagonal. Cell `[0][2]`, cell `[1][1]` and cell `[2][0]` all have `r + c = 2`.

So one pass over the whole matrix, putting each value into a bucket keyed by `r + c`, gives me every diagonal in one go. There are exactly `rows + cols - 1` of them, and because the loops go top to bottom, each bucket is already in top-to-bottom order.

```
def diagonal_order(matrix: list[list[int]]) → list[int]:
    if not matrix or not matrix[0]:
        return []
    rows, cols = len(matrix), len(matrix[0])
    groups = defaultdict(list)
    for r in range(rows):
        for c in range(cols):
            groups[r + c].append(matrix[r][c])
    out = []
    for s in range(rows + cols - 1):
        band = groups[s]
        out.extend(reversed(band) if s % 2 == 0 else band)
    return out
```

The zigzag is then a separate, easy concern: the buckets are all top-to-bottom, and the problem wants alternate ones bottom-to-top, so I reverse the even-numbered buckets.

On the three by three, `[[1,2,3],[4,5,6],[7,8,9]]`, the buckets are `[1]`, `[2,4]`, `[3,5,7]`, `[6,8]`, `[9]`, and reversing buckets 0, 2 and 4 gives `1, 2, 4, 7, 5, 3, 6, 8, 9`.

That is $O(m \times n)$ time — one visit per cell, and I have to look at every cell, so that is optimal — and $O(m \times n)$ extra space for the buckets. If the space matters, I can walk each diagonal directly with a moving row and column instead of bucketing,

which brings the extra space to $O(1)$. I would reach for that second version only if asked, because the bounds on each diagonal are the fiddly part and the bucketing version is much harder to get wrong.”

9 Recall card

- `matrix[row][column]`. **Row first, always.** `rows = len(matrix)`, `cols = len(matrix[0])`.
- **Same expression, swapped loops:** outer `r` is row-major; outer `c` is column-major.
- **Down-right diagonal:** `r - c` is constant. **Down-left (anti-)diagonal:** `r + c` is constant. There are `rows + cols - 1` of each.
- **Never** `[[0] * cols] * rows` — it aliases one row. Use `[[0] * cols for _ in range(rows)]`.
- **$O(\text{rows} \times \text{cols})$ time** for any full walk. Check `0 ≤ r < rows` and `0 ≤ c < cols` before reading — negative indices wrap instead of raising.

1 What this is, and why they ask it

REST — Representational State Transfer — is a style for designing APIs, described by Roy Fielding in 2000 as a set of six constraints. It is not a technology, not a standard, and not a library; nothing you install makes an API RESTful. It is a list of rules about how a client and a server should be arranged, and an API is RESTful to the extent that it follows them.

Yesterday you learned what an API is: the published promise about what one piece of software will do for another. REST is the most widely used way of shaping that promise. Its central move is to organise everything around **things** rather than **actions** — every thing gets an address, and a small fixed set of operations applies to every thing in the same way.

This question is asked constantly, at every level, and most candidates answer it badly. The weak answer is “an API that uses HTTP and returns JSON”, which is false — you can be thoroughly un-RESTful over HTTP with JSON, and most APIs called REST are. The strong answer names the constraints, picks out the two that actually matter in practice, and is honest that almost nobody implements the sixth. That honesty is the part that impresses, because it means you have read something beyond a tutorial.

2 The story

The cloakroom at Kacheguda station is a green door on platform one with a wooden counter across it, and Rehana has used it four or five times a year since she started travelling for work. Her train to Bangalore leaves at ten past nine at night and she gets into the city at eleven in the morning, so there are ten hours to fill and she is not going to spend them holding a suitcase.

The way it works has not changed in fifteen years.

She puts the suitcase on the counter. The man weighs it, writes a number on a small metal token, loops a matching tag through the handle, and hands her the token. Number 213. That token is now the only thing that connects her to the suitcase. It does not say her name. It does not say what is inside. It says 213, and 213 is enough, because there is exactly one bag with that tag on it.

Above the counter there is a board with four lines on it and it applies to every bag in there, whether the bag holds clothes or a laptop or a stack of samples. Hand one in. Ask what is being held under your number. Swap what is under your number for something else. Take it back. That is all. There is no fifth thing, and there does not need to be.

Then there is the part Rehana has come to appreciate. When she comes back at seven in the evening, the man at the counter is not the man who took the bag. The shift changed at two. This new man has never seen her before in his life, and it does not matter at all — she hands over the token and her identity card and he goes and finds 213. Nothing about the exchange depends on anybody remembering anything.

She has used a place where that was not true. A left-luggage room near a bus stand where a boy took her bag, did not give her anything, and said, *don't worry, I'll remember you*. He remembered her. But she spent the whole day slightly worried, and when she came back he was at lunch and the man covering for him had no idea who she was or which of the forty bags was hers.

3 The idea in plain English

The cloakroom is REST. Not by analogy — line for line.

Everything is a thing, and every thing has an address

Token 213 names one suitcase. In REST the thing is called a **resource**, and its address is a **URI** — the path in the URL. So:

```
/bags/213
```

The URI names a thing, not an action. That is the whole design decision, and it is what people mean when they say REST uses **nouns, not verbs**. This is the un-RESTful version of the same idea:

```
/getBag?id=213  
/deleteBagById?id=213  
/updateBagContents?id=213
```

Three addresses for one suitcase, each with the action baked into the name. Add a fourth operation and you invent a fourth address. In REST there is one address for the suitcase, forever, and the operation is carried separately.

Resources come in two shapes and it is worth naming both now. A **collection** is the set of things: `/bags`. An **item** is one of them: `/bags/213`. Almost every REST path you will ever design alternates between the two, like `/users/42/orders/7/items`.

One small set of operations, the same for every thing

The board above the counter has four lines and applies to every bag. In REST the four lines are the HTTP methods you met on [day 005](#):

BOARD	METHOD	ON <code>/bags</code> (THE COLLECTION)	ON <code>/bags/213</code> (ONE ITEM)
Ask what is there	<code>GET</code>	list all bags	fetch bag 213
Hand one in	<code>POST</code>	create a new bag, get its number back	—
Swap it for this	<code>PUT</code>	—	replace bag 213 entirely
Change part of it	<code>PATCH</code>	—	change some fields of bag 213
Take it back	<code>DELETE</code>	—	remove bag 213

This is the **uniform interface**, and it is the most important constraint. Because the operations are fixed, a developer who has used one part of your API can guess the rest. Learn `/bags` and you already know `/tickets` and `/passengers`. Nobody has to read documentation to find out whether deleting is called `remove`, `delete`, `destroy` or `cancel`.

Two properties of these methods matter enough to have names, and interviewers ask about them:

- **Safe** — it does not change anything. `GET` is safe. You can call it a thousand times and the world is as it was.
- **Idempotent** — doing it twice has the same effect as doing it once. `GET`, `PUT` and `DELETE` are idempotent; `PUT` ting the same bag twice leaves one bag, and `DELETE` ing twice leaves it deleted. `POST` is **not**: two `POST` s to `/bags` create two bags. That single fact is why a timed-out payment request is dangerous, and it gets a full day on [day 018](#).

The new man on the counter: statelessness

The shift changed and nothing broke, because the token carries everything needed. **Statelessness** is the constraint that says every request must contain everything required to understand it — the server keeps nothing about you between requests.

Concretely: no server-side memory of “this user is halfway through checkout”. Every request carries its own identity, usually a token in the `Authorization` header, and everything else it needs.

This sounds like a restriction and is actually the reason REST scales. If any of five servers can answer any request, you put a load balancer in front and add a sixth whenever you like. If server 3 is the only one that remembers Rehana, then Rehana’s requests must

always go to server 3 — which is called **session affinity** or a sticky session — and when server 3 restarts, her checkout is gone. The boy who said *I'll remember you* was a stateful server, and he went to lunch.

The state has not vanished, note. It moved: into the client, which holds the token, and into a shared store like Redis or Postgres that every server can read. What statelessness forbids is state hiding in the memory of one particular machine.

Representations, which is the “R” in REST

The suitcase is not what comes across the counter. What comes across is *the bag under number 213*, and Rehana could equally have been shown a photo of it or told its weight. In REST, the resource is the abstract thing; what actually travels over the wire is a **representation** of it — usually a JSON document, sometimes XML, sometimes a PNG.

The same resource can have several representations, and the client says which it wants with the `Accept` header:

```
GET /bags/213
Accept: application/json
```

This is called **content negotiation**. In practice ninety-five per cent of APIs offer JSON and nothing else, and that is fine.

The remaining constraints, honestly

Fielding listed six. Three are already covered above — uniform interface, statelessness, and client-server, which just means the two sides evolve separately. The other three:

- **Cacheable.** A response must say whether it may be stored and reused. `GET /bags/213` with a `Cache-Control: max-age=60` header can be kept by the browser or a CDN and served without troubling your servers. This one is genuinely valuable and genuinely used.
- **Layered system.** A client cannot tell whether it is talking to the real server or to something in front of it. This is what lets you insert a load balancer, a CDN like Cloudflare, or an API gateway without any client changing.
- **Code on demand** — the server may send executable code to the client. Optional even in the original paper, and effectively never used in APIs.

And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: **HATEOAS** — Hypermedia As The Engine Of Application State. It says a response should include links telling the client what it can do next, the way a web page contains links, so the client never hard-codes a URL.

```
{
  "id": 213,
  "status": "stored",
  "_links": {
    "self": "/bags/213",
    "collect": "/bags/213/collect"
  }
}
```

By Fielding's own definition, an API without this is not REST. By everyday industry usage, nobody cares. GitHub's API does include `_links`; Stripe's does not; both are called REST by everyone including their own documentation. **Being able to say that out loud, calmly, is the difference between a good answer and a memorised one.**

4 The picture

One resource, five operations, two addresses:

COLLECTION
/bags

```
+-----+
| GET    → list every bag |
| POST   → create a new one |
|          (returns 201 + |
|          Location: /bags/213)
+-----+
```

the path says WHAT.

ITEM
/bags/213

```
+-----+
| GET    → fetch this bag |
| PUT    → replace it    |
| PATCH  → change part   |
| DELETE → remove it     |
+-----+
```

the method says WHAT TO DO WITH IT.

What to notice: the path never changes when the operation changes. `/bags/213` is the suitcase whether you are reading it, replacing it or removing it. Every un-RESTful API you will ever see breaks exactly this line.

Stateless versus stateful, and why one of them scales:

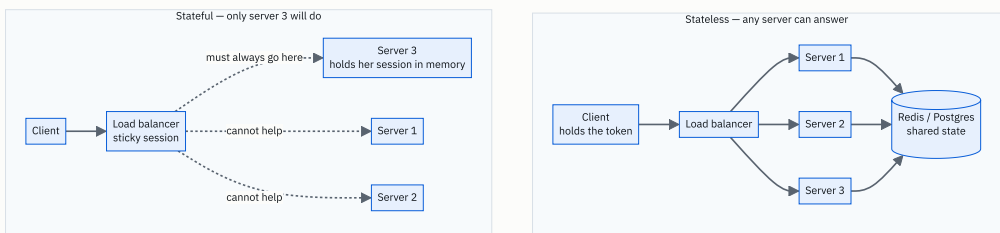


FIGURE 16.1

What to notice: in the top half, adding a fourth server is a configuration change. In the bottom half, restarting server 3 loses a customer’s checkout, and adding servers helps only new users. The dotted lines are the ones that hurt.

5 How it actually works

Designing the path

The rules, in the order they get broken:

- 1. **Nouns, and plural.** `/users`, not `/getUser` and not `/user`. Plural reads correctly for both the collection and the item: `/users` and `/users/42`.
- 2. **Nest to show ownership, but not far.** `/users/42/orders` is the orders belonging to user 42. Stop at two levels — `/users/42/orders/7/items/3/reviews` is a path nobody can remember, and the fix is to give reviews their own top-level collection at `/reviews/8`.
- 3. **Filtering, sorting and paging go in the query string, not the path.** `/orders?status=shipped&page=2&per_page=50`. These are not different resources; they are different views of one collection.
- 4. **Hyphens, lower case, no file extensions.** `/purchase-orders`, not `/purchaseOrders` and not `/orders.json`.
- 5. **Actions that genuinely are not things** — and there are some, like “publish this article” — become a sub-resource: `POST /articles/9/publish`. Every real API has a handful of these. Purists dislike it; everybody does it.

The methods, exactly

METHOD	MEANS	SAFE	IDEMPOTENT	TYPICAL SUCCESS
GET	read	yes	yes	200 OK
POST	create, or “do this thing”	no	no	201 Created + Location header
PUT	replace the whole thing	no	yes	200 OK or 204 No Content
PATCH	change part of it	no	not necessarily	200 OK
DELETE	remove it	no	yes	204 No Content

The `PUT` versus `PATCH` distinction gets asked. `PUT /users/42` with `{"name": "Rehana"}` means *this is the entire user now* — a field you left out should be cleared. `PATCH` means *change only what I sent*. Most APIs implement `PUT` sloppily as a partial update, which is a real source of bugs, and saying so is a good sign in an interview.

Status codes, the ones that matter

Full treatment is [day 018](#), but REST leans on them so heavily that the shape is worth having now: `2xx` it worked, `3xx` look elsewhere, `4xx` you got it wrong, `5xx` we got it wrong. The single worst API sin is returning `200 OK` with `{"error": "not found"}` in the body, because every proxy, CDN and monitoring tool in the path now believes the request succeeded.

Versioning

Yesterday's point, made concrete. Three approaches are in real use:

- **In the path** — `/v1/charges`. Stripe, and most of the industry. Ugly, unambiguous, trivial to route. Purists object that the resource has not changed, only its representation.
- **In a header** — `Accept: application/vnd.github.v3+json`. GitHub. Cleaner in theory, and much harder to test by hand or eyeball in a log.
- **By date** — Stripe pins each account to the API version that was current when it signed up, like `2023-10-16`. Excellent for customers, expensive to maintain.

Say path versioning is what you would ship and why, and name the header alternative. That is the complete answer.

Pagination, which every real collection needs

`GET /orders` on a table with ten million rows must not return ten million rows. Two schemes:

- **Offset** — `?page=3&per_page=50`. Easy, and it degrades badly: the database must count past 150,000 rows to serve page 3,001, and rows shifting between requests cause duplicates and gaps.
- **Cursor** — `?after=eyJpZCI6MTIzfQ&limit=50`, where the cursor encodes the last item seen. Stable under inserts and fast at any depth, because it becomes “give me the 50 rows after id 123”. This is what Stripe, Slack and Twitter use, and it is the answer to give.

Real APIs, scored honestly

API	NOUNS	METHODS	STATELESS	HYPERMEDIA	FAIR VERDICT
Stripe	yes	yes	yes	no	The design most people copy
GitHub	yes	yes	yes	partly (<code>_links</code>)	Closest to the book
Amazon S3	yes (<code>PUT /bucket/key</code>)	yes	yes	no	REST-shaped storage
Most internal company APIs	often not	<code>GET / POST</code> only	usually	no	HTTP with JSON, called REST

There is a widely used ladder for this, the **Richardson Maturity Model**: level 0 is one endpoint that does everything, level 1 introduces separate resources, level 2 uses HTTP methods and status codes properly, level 3 adds hypermedia. **Most production APIs sit at level 2, and level 2 is a perfectly good place to be.** Naming that model in an interview is a strong, cheap signal.

6 The numbers

What statelessness buys you

Take the API from yesterday: **420 requests per second at peak**. One application server handles about 200 requests a second.

$$420 \div 200 = 2.1 \rightarrow 3 \text{ servers, for headroom}$$

Stateless. Any server answers any request, so the load balancer sprays them evenly: $420 \div 3 = 140$ requests/second each. Traffic triples on a festival day? Start six more. The deployment is a number in a config file.

Stateful with sticky sessions. Each user is pinned to one server. Users do not divide evenly — in practice you see something like 45/30/25 rather than 33/33/33, so the busiest server takes $420 \times 0.45 = 189$ requests/second while another takes 105. You are at 95% of capacity on one box and 52% on another, and you must add servers for the peak of the worst-loaded one. Worse, when a server dies you do not lose a third of your capacity for a moment — you lose a third of your users’ sessions.

That asymmetry is the whole argument, and it is worth being able to produce those two numbers.

What cacheability buys you

`GET` on a public resource can be cached at a CDN. Suppose 80% of the traffic is `GET` s of things that change rarely, and the CDN achieves a 90% hit rate on them:

```
cacheable traffic    = 420 × 0.80           = 336 req/s
served by the CDN    = 336 × 0.90           = 302 req/s
reaching your origin = 420 - 302           = 118 req/s
```

Servers needed drops from 3 to 1, and the cached responses come back in about 20 ms from a nearby city instead of 150 ms from across the country. This is the single largest lever in most read-heavy designs, and it exists **only because** REST made `GET` safe and addressable. An API where every call is `POST /api` with the operation in the body cannot cache anything, ever.

What pagination saves

An order row serialises to about 1 KB of JSON.

```
unpaginated: 10,000,000 rows × 1 KB = 10 GB in one response
paginated:   50 rows × 1 KB = 50 KB
```

10 GB is not a slow response; it is a crashed process at both ends. And the offset-versus-cursor cost, on a 10-million-row table:

```
offset page 1      : scan 50 rows      → about 1 ms
offset page 100,000: scan 5,000,000 rows → about 2,000 ms
cursor, any page   : index seek + 50 rows → about 1 ms
```

The offset query gets 2,000 times slower at depth while the cursor query does not change. That is the whole reason cursor pagination exists.

7 The trade-offs

What REST costs

Over-fetching and under-fetching. `GET /users/42` returns the whole user because the resource is the unit. A mobile screen that needs only a name and a photo downloads twenty fields it throws away. And a screen needing four different things makes four round trips — from yesterday’s arithmetic, 480 ms serially against about 120 ms in parallel. This is precisely the complaint that produced GraphQL, which is [day 021](#).

Not everything is a noun. “Transfer 500 rupees from A to B”, “retry this job”, “search across seven entities with a scoring rule” — these are operations, and forcing them into resource shapes produces either dishonest paths or an invented `/transfers` resource

that exists only to be posted to. The invented resource is usually the right answer, and it is worth admitting it is a workaround.

JSON over HTTP is verbose. Field names repeat in every record, everything is text, and headers are re-sent constantly. For service-to-service traffic at high volume, a binary format over a persistent connection is several times cheaper — which is gRPC, on [day 022](#).

Statelessness has a price. Every request re-presents credentials and re-establishes context, so requests are bigger and the server repeats work — validating a token, loading the user — on every call. That is a genuine cost, paid deliberately, because independent servers are worth more than the saved work.

When you would choose something else

- **GraphQL** when many different clients need different slices of the same data, especially mobile clients on slow networks where round trips dominate.
- **gRPC** for internal service-to-service calls where you control both ends, volume is high, and you want a generated, strongly-typed client. You give up being able to test it with `curl`.
- **A message queue** — Kafka, RabbitMQ, SQS — when the caller does not need an answer now. Request-response is the wrong shape for “send this email eventually”.
- **Plain HTTP RPC** — one endpoint, an action name in the body — for a small internal service where the resource model is pure ceremony. It is unfashionable and sometimes right.

The sentence that separates candidates

I would not use REST if the operations are genuinely actions rather than things, or if the traffic is internal service-to-service at high volume. Forcing a workflow like “run this reconciliation and retry the failed rows” into resources produces paths that lie about what the system does, and JSON over HTTP wastes real money at a million calls a second. REST earns its keep when the domain really is a set of things, when the clients are many and outside your control, and when being cacheable and inspectable with `curl` is worth more than efficiency.

8 In the interview

How it gets asked

- “What makes an API RESTful?” — the direct version. They want constraints, not “it uses HTTP”.
- “What is the difference between PUT and PATCH? Which of the methods are idempotent?” — the detail check, and the one most people fumble.
- “Design the API for a library / a parking lot / a URL shortener.” — the applied version, in a design round. You will be judged on your paths and your status codes.
- “Is this API RESTful?” — shown something with `/api/getUserById?id=5`. They want you to spot the verb in the path and say so politely.

What to say out loud, in the first ninety seconds

1. **Say what kind of thing REST is.** “REST is an architectural style — a set of six constraints from Roy Fielding’s 2000 dissertation — not a technology. An API is RESTful to the degree that it follows them.”
2. **Lead with the uniform interface, because it is the one that matters.** “Everything is a resource with its own address, the address is a noun, and a small fixed set of methods applies to every resource the same way. `/users/42`, and then GET, PUT, PATCH or DELETE decides what happens to it.”
3. **Then statelessness, with the reason.** “Every request carries everything needed to understand it. The server keeps nothing about the client between requests, which is why any server can answer any request and you can scale by adding boxes.”
4. **Then cacheability and layering, quickly.** “Responses say whether they can be cached, which is what lets a CDN absorb most read traffic. And the client cannot tell how many layers are in front, so you can add a gateway or a load balancer without changing any client.”
5. **List the rest and be honest.** “Client-server, and code-on-demand which is optional and effectively unused. The sixth part of the uniform interface is hypermedia — HATEOAS — where responses carry links to what you can do next. Strictly, an API without it is not REST. In practice almost nobody implements it: GitHub does partly, Stripe does not, and both are called REST by everyone.”
6. **Land the practical point.** “Most production APIs sit at level 2 of the Richardson Maturity Model — proper resources, proper methods, proper status codes, no hypermedia — and that is usually the right place to stop.”

The follow-ups

“Which HTTP methods are idempotent, and why does it matter?” `GET`, `PUT` and `DELETE` are idempotent — doing them twice has the same effect as doing them once. `GET` is additionally safe, meaning it changes nothing at all. `POST` is neither: two `POST` s

to `/orders` create two orders. It matters because networks lose responses. If a request times out you do not know whether it was carried out, so for an idempotent method you can simply retry, and for `POST` you cannot — retrying a payment charges twice. The standard fix is an idempotency key: the client generates a unique value per attempt, sends it as a header, and the server promises that two requests with the same key are executed once and return the same response. `PATCH` is interesting because it depends on the patch: setting `status` to `shipped` is idempotent, incrementing a counter is not.

“Your API is stateless, so where do sessions live?” Not in any one server’s memory. Two options. Either the client carries a signed token — a JWT — that contains the identity and expiry, so any server can verify it with a key and no lookup at all; the cost is that you cannot easily revoke one before it expires. Or the client carries an opaque session id and every server looks it up in a shared store like Redis, which gives instant revocation and costs one fast network round trip per request. Most systems use both: a short-lived token for ordinary requests and a server-side record for refresh and revocation. Either way the important property holds — the state is in the client or in a store every server can reach, never in the memory of one box. The detail is [day 020](#).

“Here is an endpoint: `POST /api/getUserOrders`. What is wrong with it?” Three things. The verb is in the path, so the address names an action rather than a thing — it should be `GET /users/42/orders`. The method is wrong: this is a read, so it should be `GET`, and using `POST` throws away everything HTTP gives you for reads — it can never be cached, browsers and proxies will not retry it safely, and it will not show up correctly in any monitoring tool. And because it is not addressable, you cannot link to it, bookmark it or reason about it. The practical consequence is the cache: turning that one endpoint into a cacheable `GET` can take the majority of its traffic off your servers entirely.

“Design the API for a parking lot.” Resources first: `/lots`, `/lots/{id}/spots`, `/tickets`, `/payments`. Then the operations. `POST /tickets` with a vehicle and a lot to enter, returning `201 Created`, a ticket id and a `Location` header. `GET /tickets/{id}` to see the current charge. `POST /tickets/{id}/payments` to pay — a sub-resource, because a payment is a genuine thing worth having its own id, not merely an action. `GET /lots/{id}/spots?free=true` to find space, with the filter in the query string because it is a view of a collection rather than a different collection. `409 Conflict` when the lot is full, `404` for an unknown ticket, `402` or `403` on leaving with an unpaid ticket. And I would make the entry call idempotent with a client-supplied key, because a barrier that retries a timed-out request must not issue two tickets for one car.

A model answer

“REST is an architectural style rather than a technology — six constraints described by Roy Fielding in 2000. An API is RESTful to the degree that it satisfies them, and most APIs called REST satisfy some of them.

The central one is the uniform interface. Everything is a resource with its own address, and the address is a noun: `/users/42`, not `/getUser?id=42`. Then a small fixed set of methods applies to every resource identically — `GET` to read, `POST` to create, `PUT` to replace, `PATCH` to modify, `DELETE` to remove. The path says what the thing is, the method says what you are doing to it. The payoff is that once you have learned one part of the API you can guess the rest.

The second one that really earns its place is statelessness. Every request carries everything needed to understand it — typically a token in the Authorization header — and the server keeps nothing about the client between requests. That is what makes horizontal scaling easy: any server can answer any request, so you put a load balancer in front and add boxes. The moment a session lives in one server’s memory you need sticky sessions, your load spreads unevenly, and a restart costs users their state.

Then cacheability — responses declare whether they can be stored, which is what allows a CDN to absorb most read traffic, and it only works because `GET` is safe and addressable. Layered system, so the client cannot tell whether it is talking to the origin or to a gateway in front of it. Client-server, so the two evolve independently. And code-on-demand, which was optional in the original paper and is effectively never used.

The one worth being honest about is hypermedia — HATEOAS — where each response includes links to what you can do next, so clients do not hard-code URLs. By Fielding’s definition an API without it is not REST. In practice almost nobody does it: GitHub includes `_links`, Stripe does not, and everyone calls both of them REST. Most production APIs sit at level 2 of the Richardson Maturity Model — real resources, real methods, real status codes, no hypermedia — and for most systems that is the right trade.

What I would add is where REST stops being the right answer. It over-fetches, because the resource is the unit, and it under-fetches, because one screen often needs four resources — which is what GraphQL exists to fix. And for internal service-to-service traffic at high volume, JSON over HTTP is expensive compared with a binary format like gRPC. REST earns its keep when the domain really is a set of things and the clients are many and outside your control.”

9 Recall card

- **REST is a style, not a technology** — six constraints, Fielding 2000. HTTP + JSON alone is not REST.
- **Uniform interface:** nouns as addresses (`/users/42`), a fixed small set of methods, the path never changes when the operation does.
- **Stateless:** every request self-contained, so any server can answer it — that is what makes scaling out easy.
- **Safe:** `GET` . **Idempotent:** `GET` , `PUT` , `DELETE` . **Not idempotent:** `POST` — which is why payments need an idempotency key.
- **HATEOAS is the constraint nobody implements.** Say so. Most real APIs are Richardson level 2, and that is fine.

DSA topic: 2D arrays and matrix traversal **System design topic:** REST, properly

Code these, in this order

Four problems that all reduce to *visit each cell once, in the right order*. None of them needs a clever idea. All of them punish sloppy handling of the two numbers, which is exactly the skill being built.

Before each one, say out loud:

1. What do `r` and `c` mean, and what are their ranges?
2. Is the matrix square, or `m × n`? Which sizes do I need, and where do I compute them?
3. What happens on `[]` and on `[][]`?
4. Am I reading, or writing? If writing, am I allowed a new matrix?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Richest Customer Wealth	LeetCode 1672 (Easy)	The bare row walk, and whether you reach for <code>sum(row)</code> instead of indexing by hand. Two minutes.
2	Transpose Matrix	LeetCode 867 (Easy)	That the output is <code>n × m</code> when the input is <code>m × n</code> . Write it with explicit loops first, then with <code>zip(*matrix)</code> , and be able to explain the second.
3	Diagonal Traverse	LeetCode 498 (Medium)	<code>r + c</code> is constant on an anti-diagonal. The zigzag is a separate, easier concern — solve them in that order or you will tangle them.
4	Matrix Diagonal Sum	LeetCode 1572 (Easy)	Both diagonals at once, and the odd-size trap: on an odd <code>n × n</code> the centre cell is on both diagonals and must not be counted twice.

On problem 1, then break it deliberately

Solve it, then change the input to a non-square matrix such as `[[1, 2, 3], [4, 5, 6]]` and confirm your code still works. Then write the version with `len(matrix)` used for both sizes and run it. Read the traceback. That is the error you will see under pressure, and recognising it in one second is worth having.

On problem 3, do it in two passes of your own

- **First**, just group. Return the list of anti-diagonals, top to bottom, and check them by eye against the picture in §4 of the lesson. On `[[1,2,3],[4,5,6],[7,8,9]]` you want `[[1], [2,4], [3,5,7], [6,8], [9]]`.
- **Then** add the zigzag, and confirm you get `[1, 2, 4, 7, 5, 3, 6, 8, 9]`.

Only after both pass, try the `O(1)`-extra-space version that walks each diagonal directly with a moving row and column. Work out the range of `r` for diagonal `s` before writing any code — it is `max(0, s - cols + 1)` to `min(s, rows - 1)` — and check that against `s = 0` and against the last diagonal by hand.

The aliasing drill

Predict the output of each, then run them.

```
grid = [[0] * 3] * 4
grid[1][2] = 7
print(grid)
```

```
grid = [[0] * 3 for _ in range(4)]
grid[1][2] = 7
print(grid)
```

```
row = [0] * 3
grid = [row, row, row]
grid[0][0] = 5
print(grid)
```

Then say, in one sentence, what `*` actually copies. If you cannot, re-read the aliasing trap in §7 before going on — this bug will find you again in the dynamic programming phase, where the table is always a grid you built with one of these two lines.

The notation drill

Take `[[1,2,3],[4,5,6],[7,8,9],[10,11,12]]` and answer these without running anything:

1. What is `matrix[3][1]` ?
2. What is `len(matrix)` , and what is `len(matrix[0])` ?
3. Which cells have `r + c == 3` ?
4. Which cells have `r - c == 1` ?
5. How many anti-diagonals are there, and what is the longest one?
6. What does `matrix[-1][-1]` give you, and why does it not raise?

Check all six by running the code afterwards. Question 6 is the one that causes silent bugs.

The REST drill

Design the API for a **library** — books, members, loans — with no help. Write out at least eight lines in the form `METHOD /path → status`. Cover: listing books, filtering to only the available ones, borrowing a book, returning it, seeing one member's current loans, and what happens when somebody tries to borrow a book that is already out.

Then check your own design against these five questions:

1. Is there a verb anywhere in a path? If yes, can it be replaced by a method, or is it a genuine action that deserves a sub-resource?
2. Are your collections plural, and consistent?
3. Did filtering go in the query string rather than into a new path?
4. Which of your calls are idempotent? Is the borrow call safe to retry after a timeout?
5. What status code does a borrow return when the book is already lent out — and why is it `409` and not `400`?

The critique drill

Say what is wrong with each of these, in one sentence, out loud:

```
POST /api/getUserById           {"id": 42}
GET /users/42/delete
POST /orders                    → 200 OK, {"error": "out of stock"}
GET /orders?page=200000&per_page=50
PUT /users/42                   {"email": "new@example.com"}      # the user has 12 fields
GET /getAllOrdersForUserSortedByDateDescending?u=42
```

Each one breaks a different rule from the lesson. Name the rule, not just the symptom.

The constraints drill

Name all six REST constraints from memory, in under sixty seconds, and after each one say in a single sentence what you would lose without it. Then say which one almost nobody implements, and name one API that partly does.

Mark each as **said it** / **knew it but could not say it** / **did not know it**. Only the first counts.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Print the matrix diagonally.* Fix your notation first, then ask whether it is square. State the identity — $r + c$ is constant along an anti-diagonal — before you write a line of code, because that is the whole answer. Then the grouping loop, then the zigzag as a separate step, then $O(m \times n)$ time and $O(m \times n)$ extra space with the $O(1)$ alternative named.
 2. *What makes an API RESTful?* Style not technology. Uniform interface first, statelessness second, then cacheable, layered, client-server, code-on-demand. Finish with the honest line about HATEOAS and Richardson level 2. Ninety seconds, then stop.
 3. *Why is walking a matrix by row faster than walking the same matrix by column, when both are $O(m \times n)$?* Give the memory-layout reason, then the latency ladder from [day 010](#), then the one-line conclusion: complexity counts operations, hardware counts cache misses, and the two can differ by a factor of several without either being wrong.
-

Before you move on

- [] I say what r and c mean before writing any matrix loop.
- [] I compute `rows` and `cols` into named variables, and I never use `len(matrix)` for both.
- [] I test matrix code on a non-square input, always.
- [] I never write `[[0] * cols] * rows`, and I can say exactly what goes wrong when I do.
- [] I know $r + c$ is constant on an anti-diagonal and $r - c$ on the main-direction diagonal.
- [] I can name the six REST constraints and say which one is not implemented in practice.
- [] I can look at any endpoint and say whether the path is a noun and the method is right.
- [] I can redraw the collection-versus-item diagram from memory, in whatever tool I like.

Matrix tricks: rotate, spiral, transpose

DATA STRUCTURES AND ALGORITHMS

Matrix tricks: rotate, spiral, transpose

You can rotate a matrix ninety degrees in place and print it in spiral order.

SYSTEM DESIGN

Designing a good REST endpoint

You can name a resource, pick the verb, and design a URL the interviewer will not argue with.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Rotate the image ninety degrees clockwise, in place.*
- *Design the endpoints for a comments feature.*

1 What this is, and why they ask it

Yesterday you learned to walk a matrix. Today you rearrange one. Three operations come up again and again, and all three are pure index bookkeeping with no algorithmic idea hiding inside them: **transpose** (reflect the matrix across its top-left-to-bottom-right diagonal), **rotate** (turn it a quarter turn), and **spiral order** (read it round and round from the outside inward).

The reason rotation is asked so often is that the obvious solution — allocate a new matrix and copy each cell to its new home — is easy, and the words *in place* rule it out. The elegant answer is that a quarter turn is two reflections: transpose the matrix, then reverse each row. Nobody derives that under pressure. You either know it or you flounder, and interviewers know that, which is exactly why it is a good filter for *did this person prepare*.

Spiral order is asked for a different reason. It has no trick at all. It is four loops and four boundary variables, and it tests whether you can keep four moving numbers straight while somebody watches. More candidates fail spiral order than fail rotation, and they fail it on the two guard conditions in §5 that stop a single leftover row being printed twice.

Both are LeetCode staples — 48 (Rotate Image), 54 (Spiral Matrix), 59 (Spiral Matrix II), 867 (Transpose Matrix) — and both appear in real interviews at product companies as the fifteen-minute warm-up before something harder.

2 The story

Ismail has been the attender at a school in Vijayawada for nineteen years, and every March the same job comes round. Thirty-six desks in the tenth standard room, six rows of six, all facing the blackboard on the north wall. For the public exam they cannot face the blackboard, because the invigilator's table has to go by the east window where the light is. So the whole block has to be turned a quarter turn.

The first year he did it desk by desk. He picked up a desk, thought about where it should end up, carried it, put it down, and went back for the next one. It took him the whole of an afternoon, two desks finished up in the wrong place, and a teacher had to come and sort it out on the morning of the exam.

Now he does not think about desks at all. He thinks about lines.

He stands at the front and works it out once, out loud, the way he always does. The line of six nearest the blackboard — that whole line, in order — becomes the line along the east wall, top to bottom. The boy who sat at the left-hand end of the front line will now be sitting nearest the window at the front. Then the second line from the blackboard becomes the next line in from the window. Then the third. And the line right at the back, by the door, ends up along the west wall.

Six moves instead of thirty-six decisions. It takes him twenty minutes and nothing ends up in the wrong place, because he is never choosing where one desk goes — he is only ever moving one whole line into the place a whole line goes.

Then he sweeps, and he sweeps the same way he always has. He starts along the north wall and goes right to the corner, then down the east wall, then back along the south wall, then up the west wall — all the way round the outside. Then he does it again, one step further in, and again, and again, each round a little smaller than the last, until everything is in one heap in the middle of the floor and he can pick it up in one go.

3 The idea in plain English

Ismail's two habits are today's two problems. Turning the block of desks is **rotation**. Sweeping round and round inward is **spiral order**.

Everything below uses `n` for the size of a square matrix, and `r` and `c` exactly as [day 016](#) defined them: `r` is the row, `c` is the column, and a cell is `matrix[r][c]`.

Transpose: the fold along the diagonal

To **transpose** a matrix is to swap `matrix[r][c]` with `matrix[c][r]` for every pair. Row 0 becomes column 0, row 1 becomes column 1, and so on. Visually it is a fold along the line running from the top-left corner to the bottom-right corner.

1	2	3		1	4	7
4	5	6	→	2	5	8
7	8	9		3	6	9

Two things to notice immediately.

The cells on the diagonal itself do not move. Where `r = c`, swapping a cell with itself does nothing. That is 1, 5 and 9 above.

Every other cell belongs to exactly one pair. `matrix[0][2]` and `matrix[2][0]` are partners. So if you loop over *every* cell and swap, you will visit each pair twice — swapping it back — and end up exactly where you started. This is the single most common

bug in the topic, and §7 shows what it produces. The fix is to visit each pair once, by only looking at cells **above** the diagonal:

```
for r in range(n):
    for c in range(r + 1, n):        # c starts at r + 1, not 0
        matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
```

`range(r + 1, n)` starts one past the diagonal. On row 0 it covers columns 1 and 2; on row 1, just column 2; on row 2, nothing. Three swaps for a 3×3 , which is exactly the number of off-diagonal pairs.

A transpose in place only works on a square matrix. A 2×3 matrix transposes into a 3×2 one, and the shape of the outer list would have to change, so it cannot be done by swapping. You have to build a new matrix — which is what `zip(*matrix)` does.

Rotation: a quarter turn is two flips

Here is the fact worth memorising, because it is the whole question:

Rotate 90° clockwise = transpose, then reverse each row.

Watch it happen:

original		transpose		reverse each row
1 2 3		1 4 7		7 4 1
4 5 6	→	2 5 8	→	8 5 2
7 8 9		3 6 9		9 6 3

The bottom-left corner, 7, has arrived at the top-left. The top-left, 1, has gone to the top-right. That is a clockwise quarter turn.

Why it works, in one line each. Transposing sends the cell at `(r, c)` to `(c, r)` — a reflection across the main diagonal. Reversing every row then sends `(c, r)` to `(c, n - 1 - r)` — a reflection left-to-right. Do both and the cell at `(r, c)` finishes at `(c, n - 1 - r)`, and that is precisely the formula for a clockwise quarter turn. Two reflections about lines that meet at 45° compose into a rotation of 90°.

You do not need to say that last sentence in an interview. You do need to be able to check the formula on one corner, out loud: “cell (0,0) should end up at (0, n-1) — top-left goes to top-right — and the formula gives (0, n-1-0), which is (0, n-1). Correct.” Checking one corner takes five seconds and catches a wrong direction instantly.

And that is Ismail’s front row becoming the right-hand column.

The other three turns

Once you have the clockwise one, the rest are free:

TURN	RECIPE
90° clockwise	transpose, then reverse each row
90° anticlockwise	transpose, then reverse the order of the rows
180°	reverse the order of the rows, then reverse each row
270° clockwise	same as 90° anticlockwise

Note the difference between the first two. `row.reverse()` on each row flips left-to-right. `matrix.reverse()` flips top-to-bottom. One character of difference in intent, opposite directions of turn. Say which one you mean while you write it.

Spiral order: four walls that close in

Ismail’s sweep is four boundaries, not four directions. Keep four numbers:

- `top` — the topmost row not yet swept
- `bottom` — the bottommost row not yet swept
- `left` — the leftmost column not yet swept
- `right` — the rightmost column not yet swept

Then repeat four passes: go **right** along row `top`, then **down** column `right`, then **left** along row `bottom`, then **up** column `left`. After each pass, move that boundary inward by one, because the line you just swept is finished.

Stop when the boundaries cross — when `top > bottom` or `left > right`. There is nothing left in between.

The subtlety, and it is the whole difficulty of the problem: **after the first two passes the remaining block may be a single row or a single column**, and then the third and fourth passes would walk back along the line you have just done. So the third and fourth passes need guards: sweep the bottom row only `if top ≤ bottom`, and the left column only `if left ≤ right`. §7 shows exactly what you get without them.

4 The picture

The transpose, with the fold line drawn in:

	c=0	c=1	c=2			c=0	c=1	c=2
r=0	1 \ 2 3				r=0	1 \ 4 7		
r=1	4 5 \ 6			----->	r=1	2 5 \ 8		
r=2	7 8 9 \					3 6 9 \		

the diagonal (1, 5, 9) does not move.
 2 ↔ 4, 3 ↔ 7, 6 ↔ 8. Three swaps, not six.

What to notice: three swaps for a `3 × 3`. If you count six, you have swapped every pair twice and the matrix is unchanged.

The full clockwise rotation, in two moves:

start		after transpose		after reversing each row
+---+---+---+ 1 2 3 +---+---+---+ 4 5 6 +---+---+---+ 7 8 9 +---+---+---+ ^ 1 was here	→	+---+---+---+ 1 4 7 +---+---+---+ 2 5 8 +---+---+---+ 3 6 9 +---+---+---+	→	+---+---+---+ 7 4 1 +---+---+---+ 8 5 2 +---+---+---+ 9 6 3 +---+---+---+ ^ 1 ends here

What to notice: follow one value, not the whole grid. The 1 goes top-left → top-left → top-right. The 7 goes bottom-left → top-right → top-left. Tracking a single corner is how you check the direction in an interview without redrawing anything.

The spiral, as closing walls:

+-----+ 1 → 2 → 3 → 4 ^ +-----+ v 12 13 14 5 ^ v 11 16 15 6 ^ +-----+ v 10 ← 9 ← 8 ← 7 +-----+	pass 1: right along top, then top++ pass 2: down the right, then right-- pass 3: left along bottom, then bottom-- pass 4: up the left, then left++ then repeat on the smaller box
--	---

What to notice: each pass finishes a whole line and then retires that line for good. The inner `13, 14, 15, 16` block is the same problem on a smaller box, which is why one `while` loop handles any size.

5 The code, built step by step

Transposing in place

```
n = len(matrix)
for r in range(n):
    for c in range(r + 1, n):
        matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
```

Three lines. The only thing to get right is `r + 1`. Write `range(n)` there and you get the identity — §7 shows the exact output.

The tuple swap works the same way it did on [day 013](#): the right-hand side is fully evaluated before anything is assigned, so no temporary variable is needed.

Reversing each row

```
for row in matrix:
    row.reverse()
```

`row` is a reference to the actual inner list, not a copy — from [day 005](#) — so `row.reverse()` modifies the matrix. `reversed(row)` would not; it returns a new iterator and leaves the row alone. That one-character difference between `reverse` and `reversed` is a real interview slip.

Putting the rotation together

```
def rotate(matrix: list[list[int]]) → None:
    """LeetCode 48. Rotate a square matrix 90 degrees clockwise, in place."""
    n = len(matrix)
    for r in range(n):
        for c in range(r + 1, n):
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    for row in matrix:
        row.reverse()
```

Six lines, no extra memory, and it is the complete answer to one of the most-asked matrix questions there is.

The one-liner, and when to use it

```
rotated = [list(r) for r in zip(*matrix[::-1])]
```

`matrix[::-1]` reverses the row order; `zip(*...)` transposes. Reversing then transposing gives the same clockwise turn as transposing then reversing rows. It is neat, it is correct, and it is **not in place** — it builds a whole new matrix, which is $O(n^2)$ extra space. Write it, then say: *“but the question said in place, so here is the two-flip version.”* Showing both is strictly better than showing either.

Spiral order: the four passes

Start with the boundaries.

```
top, bottom = 0, len(matrix) - 1
left, right = 0, len(matrix[0]) - 1
out: list[int] = []
```

Then the first two passes, which never need a guard because the loop condition already promised there is at least one row and one column left.

```
for c in range(left, right + 1):          # go right along the top row
    out.append(matrix[top][c])
top += 1

for r in range(top, bottom + 1):          # go down the right column
    out.append(matrix[r][right])
right -= 1
```

Notice the second loop starts at the **new** `top`, the one already moved down. The corner cell was taken by the first pass and must not be taken again.

Now the two that need guards.

```
if top ≤ bottom:                          # is there still a bottom row?
    for c in range(right, left - 1, -1):
        out.append(matrix[bottom][c])
    bottom -= 1

if left ≤ right:                           # is there still a left column?
    for r in range(bottom, top - 1, -1):
        out.append(matrix[r][left])
    left += 1
```

`range(right, left - 1, -1)` counts **down** from `right` to `left` inclusive; the `left - 1` is the exclusive end, so `left` itself is included. Getting that `- 1` wrong drops a corner.

Those two `if` s are the whole exam. On a `1 × 4` matrix, the first pass takes all four values and sets `top = 1`, which is now greater than `bottom = 0`. Without the guard, the third pass walks the bottom row — which is the same row — backwards, and you emit `1 2 3 4 3 2 1`.

The complete solutions

```
def transpose_in_place(matrix: list[list[int]]) → None:
    """Square matrices only. Reflects across the main diagonal."""
    n = len(matrix)
    for r in range(n):
        for c in range(r + 1, n):          # c > r: visit each pair exactly once
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]

def transpose(matrix: list[list[int]]) → list[list[int]]:
    """Any shape. Returns a new n x m matrix; the input is untouched."""
    return [list(t) for t in zip(*matrix)]

def rotate(matrix: list[list[int]]) → None:
    """LeetCode 48. 90 degrees clockwise, in place. Square only."""
    transpose_in_place(matrix)
    for row in matrix:
        row.reverse()                    # left-to-right flip

def rotate_anticlockwise(matrix: list[list[int]]) → None:
    """90 degrees anticlockwise, in place. Same transpose, other flip."""
    transpose_in_place(matrix)
    matrix.reverse()                    # top-to-bottom flip

def rotate_180(matrix: list[list[int]]) → None:
    """Half turn. No transpose needed – both flips, no fold."""
    matrix.reverse()
    for row in matrix:
        row.reverse()

def spiral_order(matrix: list[list[int]]) → list[int]:
    """LeetCode 54. Any shape, outside inwards, clockwise."""
    if not matrix or not matrix[0]:
        return []

    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    out: list[int] = []

    while top ≤ bottom and left ≤ right:
        for c in range(left, right + 1):          # right along the top
            out.append(matrix[top][c])
            top += 1

        for r in range(top, bottom + 1):          # down the right side
            out.append(matrix[r][right])
            right -= 1

        if top ≤ bottom:
            for c in range(right, left - 1, -1):  # left along the bottom
                out.append(matrix[bottom][c])
                bottom -= 1

        if left ≤ right:
            for r in range(bottom, top - 1, -1):  # up the left side
                out.append(matrix[r][left])
                left += 1

    return out

def generate_spiral(n: int) → list[list[int]]:
    """LeetCode 59. Fill an n x n matrix with 1..n*n in spiral order."""
    matrix = [[0] * n for _ in range(n)]        # never [[0] * n] * n
```

```

top, bottom, left, right = 0, n - 1, 0, n - 1
value = 1

while top ≤ bottom and left ≤ right:
    for c in range(left, right + 1):
        matrix[top][c] = value
        value += 1
    top += 1

    for r in range(top, bottom + 1):
        matrix[r][right] = value
        value += 1
    right -= 1

    if top ≤ bottom:
        for c in range(right, left - 1, -1):
            matrix[bottom][c] = value
            value += 1
        bottom -= 1

    if left ≤ right:
        for r in range(bottom, top - 1, -1):
            matrix[r][left] = value
            value += 1
        left += 1

return matrix

if __name__ == "__main__":
    m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
    rotate(m)
    print(m) # [[7, 4, 1], [8, 5, 2], [9, 6, 3]]

    m2 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
    rotate_anticlockwise(m2)
    print(m2) # [[3, 6, 9], [2, 5, 8], [1, 4, 7]]

    big = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]]
    rotate(big)
    print(big) # [[13, 9, 5, 1], [14, 10, 6, 2], ...]

    print(spiral_order([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
    # [1, 2, 3, 6, 9, 8, 7, 4, 5]
    print(spiral_order([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]))
    # [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
    print(spiral_order([[1, 2, 3, 4]])) # [1, 2, 3, 4] - the guard earns its keep
    print(spiral_order([[1], [2], [3], [4]])) # [1, 2, 3, 4]
    print(spiral_order([], spiral_order([]))) # [] []

    print(generate_spiral(3)) # [[1, 2, 3], [8, 9, 4], [7, 6, 5]]

```

6 What it costs

The rotation

The transpose loop. The outer loop runs n times. For $r = 0$ the inner loop runs $n - 1$ times, for $r = 1$ it runs $n - 2$ times, and so on down to 0. Adding those up:

$$(n-1) + (n-2) + \dots + 1 + 0 = n(n-1)/2$$

For $n = 3$ that is $3 \times 2 / 2 = 3$ swaps, which matches the three swaps in the picture. For $n = 1000$ it is 499,500 swaps.

The reversal loop. n rows, and reversing a row of length n costs $n/2$ swaps, so $n \times n/2 = n^2/2$ more.

Total: about $n^2/2 + n^2/2 = n^2$ swaps, so **$O(n^2)$ time**. Which is the floor for this problem — every one of the n^2 cells has to move, so you cannot do better than linear in the number of cells.

Space: the swaps use one temporary at a time and the reversals use none. **$O(1)$ extra space**, which is the entire point of the two-flip version. The `zip(*matrix[::-1])` one-liner is the same $O(n^2)$ time but $O(n^2)$ extra space.

Spiral order

Every cell is appended exactly once and no cell is visited twice, because each pass retires its line. So the total work is `rows × cols` appends: **$O(\text{rows} \times \text{cols})$ time**.

Space is $O(1)$ extra — four integer boundaries — beyond the output list, which necessarily holds `rows × cols` values. When an interviewer asks about space here, say “ *$O(1)$ auxiliary, not counting the output*”, because “counting the output” is the ambiguity they are probing.

A number to have ready

A 1000×1000 image is a million pixels. Rotating it touches each one about twice, so a couple of million operations — well under a second in Python, and microseconds in C. A $10,000 \times 10,000$ image is a hundred million pixels, which is where you stop rotating in Python and start using a library that does it in one memory-order-friendly pass.

7 The traps

The near-miss: transposing over the full range

```
def rotate(matrix):
    n = len(matrix)
    for r in range(n):
        for c in range(n):           # should be range(r + 1, n)
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    for row in matrix:
        row.reverse()
    return matrix

print(rotate([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
```



```
[[3, 2, 1], [6, 5, 4], [9, 8, 7]]
```

That is not a rotation. It is the original matrix with each row reversed — a left-to-right mirror. The transpose ran, then ran again in the opposite direction, and the two cancelled exactly, leaving only the row reversals to take effect.

There is no error and the output looks structured, which is what makes this dangerous. **The cheapest check is to count swaps:** a 3×3 transpose does three, and this version does nine.

The real error: rotating a rectangular matrix

```
print(rotate([[1, 2], [3, 4], [5, 6]]))
```

```
Traceback (most recent call last):
  File "d17d.py", line 9, in <module>
    print(rotate([[1, 2], [3, 4], [5, 6]]))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "d17d.py", line 5, in rotate
    matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    ~~~~~^~~~~~
IndexError: list index out of range
```

`n = len(matrix)` is 3, but each row only has 2 columns, so `matrix[0][2]` does not exist.

Now the more alarming direction — more columns than rows:

```
print(rotate([[1, 2, 3], [4, 5, 6]]))
```

```
[[3, 4, 1], [6, 5, 2]]
```

No error at all, and complete nonsense. `n` was 2, so only the top-left 2×2 corner got transposed and the third column was never touched. In-place rotation is **only defined for square matrices** — a rectangular one changes shape from $m \times n$ to $n \times m$, so the outer list itself would have to grow or shrink. Ask “is it square?” before you write a line, and if it is not, allocate.

The near-miss: dropping the spiral guards

```
def spiral_buggy(matrix):
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    out = []
    while top ≤ bottom and left ≤ right:
        for c in range(left, right + 1):
            out.append(matrix[top][c])
        top += 1
        for r in range(top, bottom + 1):
            out.append(matrix[r][right])
        right -= 1
        for c in range(right, left - 1, -1):      # no guard
            out.append(matrix[bottom][c])
        bottom -= 1
        for r in range(bottom, top - 1, -1):      # no guard
            out.append(matrix[r][left])
        left += 1
    return out

print(spiral_buggy([[1, 2, 3, 4]]))
print(spiral_buggy([[1], [2], [3], [4]]))
print(spiral_buggy([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
```

```
[1, 2, 3, 4, 3, 2, 1]
[1, 2, 3, 4, 3, 2]
[1, 2, 3, 6, 9, 8, 7, 4, 5]
```

Look at the third line: the square case is **completely correct**. So a candidate who tests only on `3 × 3` ships this and it fails on the first single-row input the grader sends. `[1, 2, 3, 4]` comes back with seven values instead of four, because after the first pass there is no row left and the code sweeps the same row backwards.

Always test spiral code on a single row and a single column. Those two inputs, and nothing else, find this bug.

The near-miss: `reversed` instead of `reverse`

```
for row in matrix:
    reversed(row)      # does nothing to the matrix
```

No error, no effect. `reversed(row)` returns a lazy iterator and throws it away; `row.reverse()` modifies the list. If your rotation comes out as a plain transpose, this is why.

The near-miss: `zip(*matrix)` is not in place

```
def rotate(matrix):  
    matrix = [list(r) for r in zip(*matrix[::-1])]    # rebinds the local name only
```

The caller's matrix is unchanged. Assigning to a parameter name inside a function rebinds that local name; it does not modify the object the caller holds. If the question says *in place* and the grader checks the input array afterwards, this silently fails every test. Either mutate the rows you were given, or return the new matrix and say clearly that it is not in place.

8 In the interview

How it gets asked

- “Rotate the image 90 degrees clockwise, in place.” — LeetCode 48. The phrase *in place* is the entire question; without it this is a two-line problem.
- “Print the matrix in spiral order.” — LeetCode 54. Usually rectangular, deliberately.
- “Generate an n by n matrix filled with 1 to n^2 in spiral order.” — LeetCode 59, the same four passes writing instead of reading.
- “Transpose this matrix.” — the warm-up, and then “now without extra space”, which forces the square-only conversation.

What to say out loud, in the first ninety seconds

1. **Ask the shape question first.** “Is it guaranteed square? In-place rotation only works on a square matrix — a rectangular one changes shape from m by n to n by m , so the outer list itself would have to change and I would have to allocate.” This single sentence is worth a lot; most candidates never mention it.
2. **State the decomposition before writing anything.** “A ninety-degree clockwise rotation is two reflections: transpose the matrix, then reverse each row.”
3. **Check the direction on one corner, out loud.** “Let me sanity-check with the bottom-left cell. After a clockwise turn it should be top-left. Transpose sends $(n-1, 0)$ to $(0, n-1)$, then reversing that row sends it to $(0, 0)$. Yes, top-left. Right direction.”
4. **Name the `r + 1`, before you write it.** “In the transpose loop, the inner index starts at `r + 1`, not 0. If I loop over every cell I swap each pair twice and get the original matrix back.”
5. **Give the cost.** “ $O(n^2)$ time, which is optimal since every cell has to move, and $O(1)$ extra space, which is what in place means here.”

6. Offer the alternative. “There is a one-liner — `[list(r) for r in zip(*matrix[::-1])]` — but it builds a new matrix, so it is $O(n^2)$ space and does not satisfy in place.”

For spiral, the script is different because there is no insight to state:

1. “I’ll keep four boundaries — top, bottom, left, right — and do four passes per lap: right along the top, down the right, left along the bottom, up the left, shrinking each boundary as I finish with it.”
2. “The part that needs care is that after the first two passes there may be only one row or one column left, so the bottom and left passes each need a guard, otherwise I re-emit a line.”
3. “I’ll test on a single row and a single column, since those are exactly the inputs that expose that.”

The follow-ups

“Now do it anticlockwise.” Same transpose, the other flip. For clockwise I reverse each row, which is a left-to-right mirror. For anticlockwise I reverse the order of the rows instead — `matrix.reverse()` — which is a top-to-bottom mirror. I would check it on a corner the same way: the top-left cell should end up bottom-left after an anticlockwise turn, and it does. And 180 degrees needs no transpose at all, just both flips: reverse the row order and reverse each row.

“Rotate it without transposing — do it as one pass of four-way swaps.” Yes, and it is the version worth knowing because it does each cell exactly once instead of touching it twice. Work in rings from the outside in. For ring `layer`, walk `i` from `layer` to `n - 1 - layer - 1`, and rotate the four cells that map onto each other in a single four-way swap: top goes to right, right to bottom, bottom to left, left back to top, saving one value in a temporary. Same $O(n^2)$ and $O(1)$, roughly half the writes. It is materially harder to get the four index expressions right under pressure, so I would write the transpose version first, state that this alternative exists, and only code it if you want to see it.

“The matrix is a large image and does not fit in memory. Now what?” Then the operation is I/O-bound, not compute-bound, and the two-flip approach is actively bad because reversing rows and transposing have completely different memory access patterns — the transpose reads down columns, which on a file means seeking. The standard answer is to work in tiles: read a block that does fit, say 512 by 512, rotate it in memory, and write it to the transposed block position in the output. That turns a huge number of random accesses into a modest number of sequential ones, which by the latency ladder from [day 010](#) is the difference that matters. Real image libraries do exactly this.

“Spiral order, but the matrix is rectangular. What changes?” Nothing in the structure — the four-boundary version already handles it, because `top / bottom` come from the row count and `left / right` from the column count independently. What changes is

that the guards start mattering much sooner: on a $1 \times n$ or $n \times 1$ matrix the very first lap exhausts the grid, and without the two `if` s you emit values twice. That is why I test those two shapes specifically.

A model answer

“First, is the matrix guaranteed square? In-place rotation only makes sense for a square matrix — rotating an m by n gives an n by m , so the shape changes and I would have to allocate a new one.

...Square, good.

The key idea is that a ninety-degree clockwise rotation is two reflections. First transpose — reflect across the main diagonal, so the cell at (r, c) swaps with the cell at (c, r) . Then reverse each row — reflect left to right. Composing those two gives the quarter turn.

Let me check the direction before I write it. The bottom-left corner should finish at the top-left after a clockwise turn. Transposing sends $(n-1, 0)$ to $(0, n-1)$. Reversing that row sends column $n-1$ to column 0. So it lands at $(0, 0)$, the top-left. That is right.

```
def rotate(matrix: list[list[int]]) → None:
    n = len(matrix)
    for r in range(n):
        for c in range(r + 1, n):          # r + 1, so each pair is swapped once
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    for row in matrix:
        row.reverse()
```

The detail I want to call out is the `r + 1` in the inner range. Every off-diagonal cell belongs to exactly one pair, so if I looped over all n columns I would swap each pair twice and get the original matrix back — and there is no error to tell me, the output just comes out as a mirror instead of a rotation. The cells on the diagonal never move, which is why starting at `r + 1` is exactly right rather than merely an optimisation.

On the three by three, transposing gives rows `1 4 7 / 2 5 8 / 3 6 9`, and reversing each row gives `7 4 1 / 8 5 2 / 9 6 3` — the bottom-left 7 is now top-left, which is the clockwise turn.

That is $O(n^2)$ time, which is the best possible since every cell has to move, and $O(1)$ extra space.

There is a Python one-liner, `[list(r) for r in zip(*matrix[::-1])]`, which is the same operation written as reverse-then-transpose. I would mention it, but it allocates a whole new matrix, so it is $O(n^2)$ space and does not meet the in-place

requirement. And if you wanted fewer writes there is a ring-by-ring version doing four-way swaps, which touches each cell once instead of twice — same complexity, harder to get the indices right, so I would only reach for it if you asked.”

9 Recall card

- **Rotate 90° clockwise = transpose, then reverse each row.** Anticlockwise = transpose, then `matrix.reverse()`. 180° = both flips, no transpose.
- **The transpose loop is** `for c in range(r + 1, n)`. Full range swaps every pair twice and gives you a mirror, with no error.
- **In-place rotation is square-only.** Rectangular changes shape, so it must allocate.
- **Spiral is four boundaries and four passes**, each shrinking as it finishes — with a guard before the bottom pass and before the left pass.
- **Test spiral on a single row and a single column.** The square case passes even when the code is wrong.

1 What this is, and why they ask it

Yesterday was the theory of REST. Today is the craft: given a feature in plain English, produce the list of endpoints. What are the things? What is each one called? Which method does what to it? What goes in the request, what comes back, and what happens when it goes wrong?

There are perhaps ten decisions, and they recur in every feature you will ever design. Is this a new resource or a filter on an existing one? Does it nest under its parent or stand on its own? Is this an action that deserves a sub-resource? How does the caller ask for page two? What status code does “already exists” get?

This is asked in almost every design round, and often as its own small exercise before the big one. It is a good question because it is fast — ten minutes — and it separates people cleanly. Weak candidates produce `/getComments`, `/addComment`, `/deleteCommentById` and no pagination. Strong candidates produce a resource model, admit the one place where the resource model does not fit, and mention pagination before being asked. Nothing about it requires experience at scale, so there is nowhere to hide.

2 The story

Farooq has run a hardware shop off the main road in Kanpur for twenty-two years. It is one long room with shelves to the ceiling on both sides, and there are somewhere near four thousand different things in it — screws, hinges, taps, wire, brushes, three kinds of glue, a whole shelf of nothing but door handles.

His nephew Amaan started three weeks ago and can already find almost anything, which surprises customers.

There is no secret. Every shelf has a number painted on the front edge. Every box on the shelf has a label, and every label says the same two things in the same order: what the thing is, then its size. Screws, one inch. Screws, two inch. Hinges, four inch. Amaan learnt three shelves properly in his first week, and after that he was mostly guessing correctly, because the guesses work — if the one-inch screws are in the third box on shelf nine, the one-and-a-half-inch ones are next to them, and you do not need to be told.

Two more habits are worth watching.

When a customer says “give me screws”, Amaan never brings the whole box to the counter. He asks two questions — what size, and how many — and brings a handful. Farooq taught him that on the first day after he tipped four hundred washers onto the counter for a man who wanted six.

And when a customer asks for something the shop does not keep, Amaan says so straight away. He does not go to the back, stand there for a minute, and come back with an empty hand and a shrug. He says “we don’t stock that, try the shop past the temple”, and the customer is out of the door in ten seconds instead of two minutes.

There is one thing in that shop that does not fit the system, and Farooq admits it. They sharpen blades. It is not a thing on a shelf — it is something you do — and there was nowhere to put it, so it lives behind the counter and every customer has to ask about it, and Amaan had to be told because he could never have guessed. One exception in four thousand items. Farooq says the trick is having only the one.

3 The idea in plain English

Farooq’s shelves are an API. Amaan guessing correctly is the whole payoff of good design.

Find the things first

Before writing a single path, list the **resources** — the things the feature is about. For a comments feature the list is short:

- a **comment**
- the **post** a comment belongs to
- a **reaction** to a comment (a like)
- a **report** of a comment (a moderation flag)

Notice what is *not* on that list: “getComments”, “addComment”, “deleteComment”. Those are things you *do*. In REST the doing is carried by the method, so the noun list stays small and stable. A feature with four resources needs four names, however many operations it ends up supporting.

The naming rules, which are Farooq’s labels:

- **Plural, always.** `/comments`, never `/comment`. It reads correctly for both the collection and one item: `/comments` and `/comments/91`.
- **Lower case, hyphens for gaps.** `/purchase-orders`, not `/purchaseOrders` or `/purchase_orders`.
- **The same word everywhere.** If it is a comment in one place it is not a “remark” in another. Amaan can guess because every label is written the same way.

- **No file extensions and no verbs.** `/comments/91`, not `/comments/91.json` and not `/comments/getById/91`.

Decide what nests and what does not

A comment belongs to a post. That relationship can be expressed two ways, and both are correct:

```
GET /posts/17/comments    # the comments on post 17 – nested
GET /comments?post_id=17  # the same set, as a filter – flat
```

The rule most teams use, and the one to state in an interview: **nest when the child has no meaning without the parent, and only one level deep.** Comments only exist in the context of a post, so `/posts/17/comments` is right for listing and creating. But a comment also has its own permanent identity, so once it exists you address it directly:

```
GET    /comments/91
PATCH /comments/91
DELETE /comments/91
```

Not `/posts/17/comments/91`. That path carries a fact — which post it belongs to — that the server already knows from the comment id, and it opens the door to an inconsistent request where post 17 and comment 91 do not match. **List and create under the parent; read, update and delete at the top level.** That pattern is worth memorising because it answers half the questions in this topic.

And never nest three deep. `/posts/17/comments/91/reactions/4` is a path nobody can remember; make reactions their own collection at `/reactions/4`.

Filtering, sorting and paging are not new resources

This is Amaan not bringing the whole box. When the caller wants a subset, it goes in the **query string** — the part after the `?` — because it is a different *view* of one collection, not a different collection.

```
GET /posts/17/comments?sort=newest&limit=20&after=cur_8fj2
```

Not `/posts/17/newestComments` and not `/posts/17/comments/top`. Every time you invent a path for a filter you double the number of endpoints, and none of them can be combined.

Every collection endpoint must be paginated from the day it is written. Not later, not when it gets slow. A comment thread with fifty thousand replies must not be able to return fifty thousand replies, and a `limit` with a hard maximum enforced by the server

is the only thing standing between you and that. Give it a default too — twenty, say — because callers forget.

The exception, and having only one of them

Farooq's blade sharpening is the operation that is not a thing. Every real feature has one or two. For comments it is moderation:

```
POST /comments/91/approve
POST /comments/91/hide
```

Those are verbs in paths, which yesterday's rules forbid. They are also what everybody does, including well-regarded APIs, because the alternatives are worse — either `PATCH /comments/91` with `{"status": "hidden"}`, which hides an important state change inside a generic update, or inventing a `/moderation-actions` resource that exists only to be posted to.

The honest position, and the one to say out loud: **prefer a state change on the resource; use an action sub-resource when the operation has its own meaning, its own permissions or its own audit trail; and keep the number of them small.** One exception in four thousand items. The trick is having only the one.

Say what comes back, including when it fails

Amaan says “we don't stock that” immediately. An endpoint must do the same, and the way it says so is the **status code**.

- `200 OK` — here it is.
- `201 Created` — made it, and a `Location` header says where it now lives.
- `204 No Content` — done, nothing to say. The usual answer to `DELETE`.
- `400 Bad Request` — your request is malformed.
- `401 Unauthorized` — I do not know who you are. (`403 Forbidden` — I do, and you may not.)
- `404 Not Found` — no such thing.
- `409 Conflict` — the thing exists but its state forbids this.
- `422 Unprocessable Entity` — well-formed but semantically wrong, such as an empty comment body.
- `429 Too Many Requests` — slow down.

The failure that matters most: **an empty list is not a 404.** `GET /posts/17/comments` on a post with no comments is `200 OK` with `[]`. The collection exists; it is empty. `404` is for post 17 not existing at all. Getting this wrong is the single most common mistake in API design, and interviewers ask about it precisely.

4 The picture

The comments feature, complete:

RESOURCE	METHOD	PATH	SUCCESS
list comments	GET	/posts/17/comments?limit=20	200 + page of items
post a comment	POST	/posts/17/comments	201 + Location
read one	GET	/comments/91	200
edit one	PATCH	/comments/91	200
remove one	DELETE	/comments/91	204
replies to one	GET	/comments/91/replies?limit=20	200 + page
reply to one	POST	/comments/91/replies	201 + Location
like one	PUT	/reactions/comment/91/me	204 (idempotent)
unlike one	DELETE	/reactions/comment/91/me	204
report one	POST	/reports	201
hide one (mod)	POST	/comments/91/hide	200 (the exception)
nested for LIST and CREATE		top-level for READ, UPDATE, DELETE	
/posts/17/comments		/comments/91	

What to notice: the horizontal line. Everything above it needs the parent to make sense — “which post’s comments?” — and everything below it does not, because a comment id is already unique. That split is the design, and it is what an interviewer is checking for.

Note also `PUT` on the like. `PUT /reactions/comment/91/me` means *the state of my reaction to comment 91 is: liked*. Pressing like twice leaves one like, because `PUT` is idempotent. Using `POST /comments/91/likes` instead would create two likes on a double-tap, which on a mobile network with retries happens constantly.

The shape of one response, which is as much a part of the design as the path:

```
{
  "data": [
    {
      "id": "91",
      "post_id": "17",
      "author": { "id": "42", "name": "Amaan", "avatar_url": "https:// ..." },
      "body": "Second this – the two-inch ones are on shelf nine.",
      "created_at": "2026-08-27T09:14:02Z",
      "edited_at": null,
      "reply_count": 3,
      "reaction_count": 12,
      "viewer_has_reacted": false
    }
  ],
  "paging": {
    "next": "cur_8fj2kd0",
    "has_more": true
  }
}
```

What to notice: four decisions are visible here. The list is wrapped in `data` rather than being a bare array, so `paging` has somewhere to live and so new top-level fields can be added later. The author is **embedded**, not just an `author_id`, so a client rendering fifty comments does not make fifty extra calls. `reply_count` is a count, not the replies themselves, so one huge thread cannot blow up one response. And `viewer_has_reacted` is computed for the caller, which saves every client from working it out.

5 How it actually works

Designing it, in the order you should do it live

1. Write down the nouns. Comment, post, reply, reaction, report. Two minutes, and it makes the rest mechanical.

2. For each noun, write the collection and the item. `/comments` and `/comments/{id}`.

3. Fill in the methods you actually need. Not all five for everything. Comments are never replaced wholesale, so there is no `PUT /comments/91` — editing a comment changes the body and nothing else, so `PATCH`.

4. Decide nesting. List and create under the parent, everything else at the top level.

5. Add the query parameters to every collection. `limit`, a cursor, `sort`. Then any filters the feature genuinely needs: `?status=visible`, `?author_id=42`.

6. Write one example response body. This is where you decide what is embedded, what is a count, and what is a link. It is also the step candidates skip, and it is where the interesting questions live.

7. List the error cases. Post does not exist, comment does not exist, empty body, not logged in, not your comment, too many requests, comment on a locked post.

Replies: the decision that actually matters

Comment threads nest, and how you model that is the most interesting question in this whole feature.

Option A — one level of replies. A comment can have replies; a reply cannot. This is what Instagram and YouTube do. Each comment carries a `reply_count`, and `/comments/91/replies` is paginated separately. It is simple, it is fast, and every response has a bounded size.

Option B — arbitrary nesting. A comment has a `parent_id` pointing at another comment. This is Reddit and Hacker News. Storing it is easy — one nullable column — but reading a whole tree is not, because fetching depth 12 means twelve round trips to

the database unless you do something cleverer.

The something cleverer is a **materialised path**: store, on every comment, the chain of ancestors as a string like `17/44/91`. Then the whole subtree under comment 44 is every row whose path starts with `17/44/`, which is one indexed prefix scan. Postgres does this well; there is even a dedicated `ltree` type for it. The cost is that moving a subtree means rewriting the paths beneath it, which for comments essentially never happens.

What to say: *“I would start with one level of replies, because it bounds every response and covers most of the product need. If arbitrary nesting is required I would store a parent id plus a materialised path so a subtree is one prefix query, and I would still cap the rendered depth and paginate at each level.”* That answer shows you know both and have a reason for choosing.

Pagination, concretely

Offset pagination — `?page=3&per_page=20` — is what everyone writes first, and it has two real problems on a comment feed. It gets slower with depth, because the database must count past all skipped rows. And it is unstable: new comments arriving while a user scrolls shift everything down, so page 2 repeats items from page 1.

Cursor pagination fixes both. The cursor encodes the last item seen — usually `created_at` plus `id` to break ties — so page two becomes “the twenty comments after this exact point”, which is an indexed lookup at any depth and is unaffected by inserts.

```
GET /posts/17/comments?limit=20
→ { "data": [...], "paging": { "next": "cur_8fj2kd0", "has_more": true } }

GET /posts/17/comments?limit=20&after=cur_8fj2kd0
```

Make the cursor **opaque** — base64 of a small JSON blob — so clients cannot construct one and you stay free to change what is inside it. Stripe, Slack, GitHub and Twitter all work this way.

Deleting a comment

Almost never a real delete. A deleted comment in the middle of a thread still has replies hanging off it, so the row survives with a `deleted_at` timestamp and the API returns a tombstone:

```
{ "id": "91", "deleted": true, "body": null, "reply_count": 3 }
```

`DELETE /comments/91` still returns `204`, and the caller does not need to know it was a soft delete. That is encapsulation doing its job. The genuinely hard part is that a legal deletion request — a takedown, or a privacy request — must really remove the content, so you need both paths.

Real comment APIs to name

- **Reddit** — `GET /comments/{article}` returns the whole tree with `more` placeholders where it truncated, which is a neat, honest answer to unbounded depth.
- **GitHub** — `GET /repos/{owner}/{repo}/issues/{n}/comments`, exactly the nested-for-list pattern, with `Link` headers for pagination.
- **Disqus** and **YouTube Data API** — both one level of replies, both cursor-paginated.
- **Slack** — threads are a parent message plus a flat list of replies, again one level.

The convergence is not an accident. Almost every production comment system caps nesting.

6 The numbers

Take a content site with **10 million daily active users**.

Write volume. Suppose 2% of them leave a comment on a given day, and those who do leave 1.5 on average:

```
10,000,000 × 0.02 × 1.5 = 300,000 comments/day
300,000 ÷ 86,400          ≈ 3.5 writes/second average
peak (say 4×)            ≈ 14 writes/second
```

Fourteen writes a second is nothing. One Postgres instance handles that without noticing. This is worth saying out loud, because candidates reflexively reach for a queue and a sharded store for numbers that a single database eats for breakfast.

Read volume. Each active user views maybe 8 comment threads a day:

```
10,000,000 × 8 = 80,000,000 reads/day
80,000,000 ÷ 86,400 ≈ 926 reads/second average, ~3,700 at peak
```

The ratio is the design. `926 ÷ 3.5 ≈ 265 reads per write`. A 265:1 read-heavy workload says: cache aggressively, denormalise counts, and do not worry much about write throughput. That single ratio drives more decisions than any other number in the exercise.

Storage. A comment row: id 8 bytes, post id 8, author id 8, parent id 8, timestamps 16, plus the body. Comment bodies average around 200 bytes.

```
per comment ≈ 250 bytes
per day      = 300,000 × 250 bytes = 75 MB/day
per year     = 75 MB × 365          ≈ 27 GB/year
```

Twenty-seven gigabytes a year of comment text. That fits on one machine for a decade. The indexes roughly double it, so call it 55 GB a year, and it is still not a sharding problem. Being able to say “this does not need sharding, and here is the arithmetic” is a much stronger answer than proposing a distributed store nobody needs.

Why pagination is not optional. A popular post gets 50,000 comments.

```
unpaginated: 50,000 × 250 bytes ≈ 12.5 MB in one response
paginated:    20 × 250 bytes = 5 KB
```

12.5 MB over a mobile connection at 2 Mbps is **fifty seconds**, and the client has to parse all of it to render the first twenty. The page-size cap is the single most valuable line in the whole design.

What caching buys. With 265 reads per write, the first page of comments on any post is read enormously more often than it changes. Cache it in Redis for 30 seconds:

```
reads reaching the database = 3,700 × (1 - 0.9) = 370/second
```

A 90% hit rate turns 3,700 reads a second into 370. That is the difference between one database and four.

7 The trade-offs

Nesting the path

Nesting reads beautifully and encodes the relationship, but it hard-codes it. If comments later need to attach to photos and videos as well as posts, `/posts/17/comments` becomes `/photos/9/comments` and `/videos/4/comments` — three near-identical endpoints. The flat form, `/comments?parent_type=post&parent_id=17`, survives that change untouched but reads worse and lets callers construct combinations you never intended. **I would nest when the parent type is stable and go flat when the feature is clearly heading towards many parent types.**

Embedding the author

Embedding the author object saves a round trip per rendered list and is why almost every real API does it. It also duplicates data in every response — fifty comments by one person carry fifty copies of their name — and it means a user renaming themselves leaves stale names in every cache. The alternative, returning only `author_id`, is smaller and always fresh, and it forces every client to make a second call or maintain its own user cache. **Embed a small subset — id, display name, avatar URL — and nothing that changes often or matters legally.**

Counts in the payload

`reply_count` and `reaction_count` are denormalised: they are stored, not computed at read time, because counting rows on every read at 3,700 reads a second is unaffordable. The price is that they can drift out of true when an update fails halfway, so they need a periodic reconciliation job. That is a real cost and it is the right trade at this read-to-write ratio.

Action sub-resources

`POST /comments/91/hide` is honest about what is happening and gets its own permission check and audit entry. It is also a verb in a path, and if you allow yourself one you will find twelve within a year, at which point the API is RPC wearing a REST costume. **Cap it deliberately.**

Soft delete

Keeping the row keeps the thread readable and makes moderation reversible. It also means “deleted” data is still in your database, which is a compliance problem the day someone exercises a legal right to erasure. You need a genuine purge path as well, and you should say so.

The sentence that separates candidates

I would not design it this way if the comment volume were closer to the read volume, or if threads genuinely needed unbounded depth rendered in one shot. At 265 reads per write, caching and denormalised counts are obviously right and write throughput is a non-issue. If writes were within an order of magnitude of reads — a live chat rather than a comment thread — I would stop treating this as a REST collection at all, because polling a paginated endpoint is the wrong shape for real-time, and I would move to a websocket stream with the REST endpoints kept only for history.

8 In the interview

How it gets asked

- “*Design the endpoints for a comments feature.*” — the standard version, often ten minutes before a larger design question.
- “*Design the API for a URL shortener / a parking lot / a food delivery app.*” — the same exercise with different nouns.

- “Here’s an endpoint. What would you change?” — shown something like `POST /api/getCommentsForPost` and asked to critique it.
- “How would you handle a comment thread with fifty thousand replies?” — the pagination question, asked as a scenario.

What to say out loud, in the first ninety seconds

1. **Scope it before designing.** “Quick scope check: one level of replies or arbitrary nesting? Are edits allowed? Is there moderation? Do I need reactions?” Thirty seconds of this prevents designing the wrong feature.
2. **List the nouns, out loud, first.** “The resources are comment, reply, reaction and report. The post already exists.” Never start with a path.
3. **State the nesting rule as a rule.** “I’ll nest for list and create, since a comment has no meaning without its post, and address individual comments at the top level, since a comment id is already unique. So `GET /posts/{id}/comments` and `POST /posts/{id}/comments`, but `PATCH /comments/{id}` and `DELETE /comments/{id}`.”
4. **Mention pagination before being asked.** “Every collection endpoint is paginated from day one, with a default limit of twenty and a hard server-side cap. I’d use an opaque cursor rather than offsets, because a comment feed has items arriving while you scroll and offsets duplicate rows.”
5. **Sketch one response body.** “Here’s a comment as it comes back — id, embedded author summary, body, timestamps, reply count, and whether the viewer has reacted.” This is where the good conversation starts.
6. **Name the errors.** “404 if the post doesn’t exist, 200 with an empty array if it exists with no comments — those are different. 422 for an empty body, 403 for editing someone else’s, 429 on rate limit.”
7. **Flag your one exception.** “Moderation actions like hide and approve are genuinely operations rather than things, so I’d give them action sub-resources and keep the number of those small.”

The follow-ups

“A post has fifty thousand comments. What happens?” Nothing bad, because the endpoint is paginated and the server caps the limit — a caller asking for `limit=50000` gets twenty, or an error, but never fifty thousand rows. Fifty thousand comments at about 250 bytes each is 12.5 MB, which is roughly fifty seconds on a 2 Mbps mobile connection and would have to be fully parsed before the client can render the first screen. With cursor pagination the first page is 5 KB and arrives immediately, and the cursor makes page 500 exactly as fast as page 2, which offset pagination cannot do — at that depth the database has to count past a million rows. I would also cap thread depth in the response and return a “load more replies” cursor per comment rather than expanding the whole tree.

“Should replies be a separate endpoint or included in the comment?” Separate, with a count included. If replies are embedded, one viral comment makes a single response unbounded, and you cannot paginate something nested inside a paginated list. So each comment carries `reply_count`, and `GET /comments/{id}/replies` is its own paginated collection. The cost is an extra round trip for threads the user actually opens, which is the right trade because most threads are never opened. If the product needs the first two replies inline — which is what Instagram does — I would embed exactly two as a `top_replies` array and still keep the paginated endpoint for the rest. That is a deliberate, bounded exception rather than an open-ended one.

“What status code for posting a comment on a post that doesn’t exist? And on a locked post?” `404 Not Found` for the missing post, because the resource in the path does not exist and the request can never succeed as written. `409 Conflict` for the locked post, because the resource does exist and the request is well-formed — it is the current *state* that forbids it, and that state can change. I would not use `400` for either: `400` means the request itself is malformed. And an important related case — `GET /posts/17/comments` on a post with no comments is `200 OK` with an empty array, not `404`. The collection exists and is empty. Confusing “empty” with “missing” breaks every client that reasonably treats `404` as an error.

“How do you stop someone editing another person’s comment?” That is `403 Forbidden`, not `404` and not `400`, and the check belongs on the server, never in the client. `401` means I do not know who you are; `403` means I do and you may not. There is a real argument for returning `404` instead of `403` when merely knowing the resource exists is itself sensitive — a private repository, for instance — because a `403` confirms existence. For public comments that does not apply, so `403` is the honest answer. I would also make the edit a `PATCH` with only the body field, so an edit cannot accidentally overwrite the author id, and record an `edited_at` timestamp because clients need to show “edited”.

A model answer

“Let me scope it first: one level of replies or arbitrary nesting, and is there moderation?”

...One level and yes, moderation. Good.

Then the resources are: comment, reply, reaction, report. The post already exists, so I am adding to it rather than designing it.

My nesting rule is that list and create go under the parent, because a comment has no meaning without a post, and everything else is top-level, because a comment id is already unique. So:

```
GET    /posts/{post_id}/comments?limit=20&after={cursor}&sort=newest  200
POST   /posts/{post_id}/comments                                     201 + Location
GET    /comments/{id}                                                 200
PATCH /comments/{id}                                               200
DELETE /comments/{id}                                               204
GET    /comments/{id}/replies?limit=20&after={cursor}                200
POST   /comments/{id}/replies                                       201 + Location
PUT    /reactions/comment/{id}/me                                     204
DELETE /reactions/comment/{id}/me                                     204
POST   /reports                                                    201
```

Two deliberate choices in there. `PATCH` rather than `PUT` on a comment, because you only ever change the body — a `PUT` would imply replacing the whole object, including the author, which makes no sense. And `PUT` on the reaction rather than `POST`, because `PUT` is idempotent: a double-tap on a flaky mobile connection leaves one like, whereas `POST /likes` would create two.

Every collection is paginated from day one — default limit twenty, hard cap enforced by the server, opaque cursor rather than offsets. Cursors matter here specifically because comments arrive while the user is scrolling, and offset pagination duplicates rows when that happens; cursors also stay fast at depth, where offsets get slower.

A comment comes back with id, an embedded author summary of id, name and avatar so a client rendering fifty comments doesn’t make fifty extra calls, the body, created and edited timestamps, a `reply_count` rather than the replies themselves, and `viewer_has_reacted`. The counts are denormalised because the read-to-write ratio here is around 265 to 1 — with 300,000 comments a day and 80 million thread views, counting rows on every read is unaffordable, and stale counts are a tolerable price with a reconciliation job.

Errors: 404 if the post doesn’t exist, but 200 with an empty array if it exists with no comments — those are genuinely different. 422 for an empty body, 403 for editing someone else’s comment, 409 for commenting on a locked post, 429 on rate limit.

Deletes are soft, because a deleted comment mid-thread still has replies hanging off it, so the row stays with a `deleted_at` and the API returns a tombstone. The endpoint still answers 204 — the caller doesn't need to know. I would keep a separate genuine purge path for legal takedowns.

The one place I break the resource model is moderation: `POST /comments/{id}/hide` and `/approve`. Those are actions, not things, and forcing them into a `PATCH` would bury a significant state change inside a generic update and lose the separate permission check and audit trail. I would keep the number of such endpoints deliberately small — that is the exception, and the discipline is having only one.”

9 Recall card

- **Nouns first, plural, consistent.** List the resources before writing a single path.
- **Nest for list and create; go top-level for read, update and delete.** Never nest three deep.
- **Filters, sorting and paging go in the query string** — they are views of a collection, not new collections.
- **Paginate every collection from day one:** default limit, hard server cap, opaque cursor rather than offset.
- **An empty collection is `200` with `[]`, not `404`.** `404` is the parent missing; `409` is the state forbidding it; `403` is you may not.

DSA topic: Matrix tricks: rotate, spiral, transpose **System design topic:** Designing a good REST endpoint

Code these, in this order

Four problems where the difficulty is entirely in the boundaries. None of them needs an idea you do not already have. All of them punish a single wrong `+ 1`.

Before each one, say out loud:

1. Is it square, or `m × n`? Does that change whether I can work in place?
2. What are my loop bounds, stated as a range with the endpoint spelled out?
3. Which cells could be visited twice, and what stops that?
4. What are the two degenerate inputs — one row, one column — and does my code survive them?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Transpose Matrix	LeetCode 867 (Easy)	That the answer is <code>n × m</code> for an <code>m × n</code> input, so it cannot be done in place unless the matrix is square. Write both versions.
2	Rotate Image	LeetCode 48 (Medium)	Transpose then reverse each row, and the <code>range(r + 1, n)</code> that keeps each pair swapped once. The whole question is the words <i>in place</i> .
3	Spiral Matrix	LeetCode 54 (Medium)	Four boundaries and the two guards. It is rectangular on purpose.
4	Spiral Matrix II	LeetCode 59 (Medium)	The same four passes, writing instead of reading. If you built problem 3 cleanly this is a fifteen-line rewrite.

On problem 2, prove the direction to yourself

Before writing any code, take `[[1,2,3],[4,5,6],[7,8,9]]` and say out loud where the 7 has to end up after a clockwise turn. Then say where the transpose sends it, then where reversing that row sends it, and check the two agree. Five seconds, and it is the check that saves you from writing the anticlockwise version by mistake.

Then write all four turns and verify each against the same input:

```
clockwise    → [[7,4,1],[8,5,2],[9,6,3]]
anticlockwise → [[3,6,9],[2,5,8],[1,4,7]]
180 degrees  → [[9,8,7],[6,5,4],[3,2,1]]
```

On problem 2, run the broken version

```
def rotate(matrix):
    n = len(matrix)
    for r in range(n):
        for c in range(n):           # the bug
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    for row in matrix:
        row.reverse()
    return matrix

print(rotate([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
```

Predict the output first. Then explain, in one sentence, why there is no error message — and why counting swaps (three for a 3×3 , not nine) is a faster check than staring at the output.

Then run the same correct function on `[[1, 2, 3], [4, 5, 6]]` and on `[[1, 2], [3, 4], [5, 6]]`. One of them crashes and one of them silently returns nonsense. Say which is which before you run them, and say why the silent one is worse.

On problem 3, test these five inputs every time

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]      # square
[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]] # wider than tall
[[1, 2, 3, 4]]                        # one row
[[1], [2], [3], [4]]                  # one column
[]                                     # and [[]]
```

The square case passes even when the guards are missing, which is exactly why it is useless as a test. Write the version **without** the two `if` statements, run it on the one-row input, and look at what comes back:

```
[1, 2, 3, 4, 3, 2, 1]
```

Seven values from a four-cell matrix. Then add the guards and watch it become `[1, 2, 3, 4]`. Do this once by hand and you will never forget the guards again.

The bounds drill

Answer these without running anything, then check:

1. In `range(right, left - 1, -1)`, which values does it produce when `right = 3` and `left = 1`?
2. Why is it `left - 1` and not `left`?
3. After the top pass and `top += 1`, why does the right-hand pass start at `top` rather than at `top - 1`?

4. On an $n \times n$ matrix, how many times does the outer `while` loop run?
5. In the transpose loop, how many swaps happen for $n = 5$?

The stretch problem

Rotate Image, one pass. Rewrite problem 2 as ring-by-ring four-way swaps instead of transpose-then-reverse: for each ring, walk `i` across it and rotate four cells at a time, using one temporary. Same $O(n^2)$ time and $O(1)$ space, about half the writes.

Get the four index expressions right before you write the loop: say each of the four cell positions out loud in terms of `layer`, `i` and `n`, then check them against the corners of a 4×4 . This is genuinely fiddly, which is the point — it is the version interviewers ask for when they want to push.

The endpoint design drill

Design the endpoints for **a notifications feature**, with no help and nothing to copy. Cover: listing a user's notifications, filtering to unread, marking one as read, marking all as read, deleting one, and the notification preferences a user can change.

Then check your own design against these seven questions:

1. Did you list the nouns before writing any path?
2. Is “mark all as read” a verb in a path? Should it be, and can you justify it in one sentence?
3. Where did “unread only” go — the path or the query string? Why?
4. Is `PATCH /notifications/{id}` with `{"read": true}` better or worse than `POST /notifications/{id}/read`? Argue both sides, then pick.
5. Is your list endpoint paginated? What is the default limit and the hard cap?
6. What comes back when the user has zero notifications?
7. Which of your endpoints are idempotent, and does “mark all as read” being called twice cause a problem?

The critique drill

Say what is wrong with each of these in one sentence, out loud, naming the rule and not just the symptom:

```
POST /api/getCommentsForPost      {"post_id": 17}
GET  /posts/17/comments/newest
GET  /posts/17/comments           → 404 when the post has no comments
GET  /posts/17/comments/91/replies/4/reactions
POST /comments/91/likes           (called twice on a double-tap)
GET  /posts/17/comments?page=5000&per_page=20
DELETE /comments/91              → 200 OK, {"deleted": true, "count": 1}
```


The numbers drill

From memory, in under two minutes, for a site with 10 million daily users where 2% comment 1.5 times a day and each user views 8 threads:

- Comments written per day, and writes per second at peak.
- Thread views per day, and reads per second at peak.
- The read-to-write ratio, and the one design conclusion it forces.
- Storage per year at 250 bytes a comment.
- The size of an unpaginated response for a 50,000-comment post, and how long that takes on a 2 Mbps connection.

Then say the sentence those numbers earn you: *“this does not need sharding, and here is why.”*

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Rotate the image ninety degrees clockwise, in place.* Ask whether it is square, and say why that matters. State the decomposition — transpose, then reverse each row — then check the direction on one corner out loud. Name the $r + 1$ before you write it and say what goes wrong without it. Finish with $O(n^2)$ time, $O(1)$ extra space, and the one-liner you are choosing not to use.
 2. *Design the endpoints for a comments feature.* Scope first, nouns second, paths third. State the nesting rule as a rule. Mention pagination before anyone asks. Sketch one response body. Name the errors, including the empty-list-is-not-a- 404 point. Flag your one action sub-resource and say why you are keeping the number small.
 3. *Print the matrix in spiral order.* Four boundaries, four passes, shrink each as you finish it. Then the important sentence: the bottom and left passes need guards because the remaining block may be a single row or a single column, and the square test case will not catch it.
-

Before you move on

- [] I ask “is it square?” before writing any in-place matrix code.
- [] I can state the two-flip rotation recipe and check the direction on one corner in five seconds.
- [] I write `range(r + 1, n)` in a transpose without having to think about it.

- [] I test spiral code on a single row and a single column, every time.
- [] I list the nouns before I write the first path.
- [] I paginate every collection endpoint I design, without being asked.
- [] I know that an empty collection is `200` with `[]`, and that `404` means the parent is missing.
- [] I can redraw the spiral-boundaries diagram from memory, in whatever tool I like.

Arrays revision and mock round

DATA STRUCTURES AND ALGORITHMS

Arrays revision and mock round

You can solve two unseen array problems in forty-five minutes, thinking out loud.

SYSTEM DESIGN

Status codes, errors, and idempotency

You can pick the right status code and explain why retrying a POST is dangerous.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two array problems, no hints, talk as you go.*
- *Your client retries a failed payment request. What could go wrong?*

1 What this is, and why they ask it

Days 9 to 17 gave you the array toolkit: what an array really is, how to walk one, what insert and delete cost, linear search, reversing and rotating, the single-pass habit, the write pointer, and matrices. Today is not a summary of any of that. Today is the **performance** of it — two unseen problems, a clock, and a person listening while you talk.

That gap is bigger than anyone expects. You can know that a write pointer solves compaction and still freeze when the question is phrased differently. You can work out $O(n)$ correctly on your own and say nothing useful for the first four minutes because you started coding immediately. The material is not the skill. **Saying the material out loud, under mild stress, while typing, is the skill**, and it is a separate thing that has to be practised separately.

That is also, precisely, the interview. A forty-five minute technical round at a product company is two problems, or one problem with follow-ups, and you are being scored at least as much on how you work as on whether you finish. Interviewers write down whether you asked about the empty input. They write down whether you stated the approach before coding. A candidate who solves one problem cleanly with clear narration usually beats a candidate who silently solves two.

2 The story

Sneha has been driving her brother's car for two years. Not far — the four kilometres to her office, the market on Sundays, her mother's place in Kukatpally on the second Saturday of the month. She is a perfectly good driver. Everyone who has sat next to her says so.

She failed her test the first time.

The man from the office got into the passenger seat at ten past eleven, said good morning, and then said nothing at all for eleven minutes. That was the part she had not expected. She had expected instructions. What she got was silence and a man occasionally looking at his phone, and she found that she was driving very slightly worse than usual — braking a bit late, taking the roundabout in second when she would normally have been in third.

She did the reverse park. She did it correctly. She got the car into the space in one go, straight, about a foot from the kerb, and she was quietly pleased with herself. He wrote something down and said they could go back.

When the result came she went and asked why. The answer was not about the parking. It was that she had not looked. She *had* looked — she knew perfectly well what was behind her — but she had done it in a small, quick, private way, the way you do when nobody is watching, and from the passenger seat it was invisible. He could not tell the difference between a driver who had checked and a driver who had been lucky.

Her instructor put it plainly the next week. Do the mirror properly. Turn your head so it is obvious. And say it — say “checking left, nothing coming, going now” out loud, even though it feels ridiculous, because the man beside you is not marking whether the car ended up in the right place. He is marking whether you knew what you were doing while you did it.

She practised that way for three weeks, with her brother in the passenger seat saying nothing on purpose. She passed the second time and she says the driving was no better at all.

3 The idea in plain English

Sneha’s examiner cannot see what she is thinking. Neither can your interviewer. Both are scoring what is visible, and in a technical round what is visible is your **narration** and your **structure**, not the finished code.

So today has two halves. First, the shape of forty-five minutes. Second, the array toolkit compressed into something you can select from in ten seconds.

The six beats of a technical round

Every good round has the same shape, and knowing it means you are never wondering what to do next.

BEAT	ROUGHLY	WHAT YOU DO
1. Clarify	minutes 0–3	Restate the problem. Ask about types, range, duplicates, empty input, and whether you can modify the input.
2. Examples	3–6	Produce one small example and its answer, out loud. Then one edge case.
3. Brute force	6–10	Say the obvious solution and its cost. Do not code it.
4. Optimise	10–18	Say what is wasteful, name the pattern that fixes it, state the new cost. Get agreement before coding.
5. Code	18–35	Write it, narrating each block in one sentence.
6. Test	35–45	Walk your own code through the example by hand. Then the edge cases you named in beat 1.

Two rules about that table. **Do not skip beat 3.** Saying “the brute force is $O(n^2)$ because I would compare every pair” costs fifteen seconds, proves you understand the problem, and gives you something to improve on. And **do not start beat 5 without agreement.** The single most expensive mistake in an interview is twenty minutes of code the interviewer was never going to accept.

Narration: what it actually sounds like

“Thinking out loud” does not mean a running commentary on your typing. It means saying the **decision** and the **reason**, once per decision:

- “I’ll keep two variables rather than sorting, because I only need two of the n values.”
- “I’m starting the write index at 1 here, not 0, because position 0 is always kept.”
- “This is the line people get wrong — the old maximum has to slide down — so I’ll write it as one tuple assignment.”

That is Sneha turning her head so it is obvious. Silence is indistinguishable from luck.

When you genuinely need to think, say so and take the time: “Give me twenty seconds to think about the boundary.” An interviewer will always grant that, and it reads far better than a long pause they have to guess at.

The array toolkit, as a decision table

Nine days compressed. When you read an array problem, this is the list you run down.

THE PROBLEM SAYS	REACH FOR	COST	DAY
find one value, unsorted	linear scan	$O(n)$	012
max / min / two largest	trackers, one pass	$O(n)$, $O(1)$	014
remove / compact / dedupe, in place	write pointer	$O(n)$, $O(1)$	015
reverse, rotate, swap halves	three reversals	$O(n)$, $O(1)$	013
running best-so-far	one pass, one variable	$O(n)$, $O(1)$	014
insert or delete in the middle	it shifts — think again	$O(n)$ each	011
grid, rows and columns	<code>matrix[r][c]</code> , two loops	$O(m \times n)$	016
turn or spiral a grid	two flips, or four boundaries	$O(n^2)$, $O(1)$	017
the answer is a pair or a window	two indices	$O(n)$	027

And the four costs that must be automatic, from [day 011](#): reading by position is $O(1)$, appending at the end is $O(1)$ amortised, inserting or deleting anywhere else is $O(n)$ because everything after it shifts, and searching an unsorted array is $O(n)$.

The four questions to ask about any array problem

Ask these before writing anything. They take forty seconds and they catch most of the traps in the last nine days.

- 1. Can I modify the input?** Decides in-place versus allocate, and it is the whole question in half of these problems.
- 2. Is it sorted?** If yes, a great deal becomes cheaper — and it is the hint for two pointers and for binary search from [day 042](#).
- 3. Duplicates? Negatives? Empty?** Three inputs that break more solutions than anything else. Negative numbers destroy any tracker initialised to `0` ; empty input destroys anything indexing `items[0]` .
- 4. Does order have to be preserved?** From [day 015](#), this changes which correct answer is the right answer.

4 The picture

The forty-five minutes, as a shape you can hold:

0	3	6	10	18	35	45
----- ----- ----- ----- ----- -----						
clarify	examples	brute force	optimise	write the code	test it	
----- ----- ----- ----- ----- -----						
talking, no typing ~18 minutes				typing, still talking ~17 minutes		
^ restate it in your own words ask the four questions				^ get agreement BEFORE you type	^ walk YOUR code through YOUR example	

What to notice: nearly half the time is before any code is written, and the last ten minutes are testing, not coding. Candidates who start typing at minute 2 almost always finish worse, because they are debugging an approach nobody agreed to.

The selection tree you run when you first read an array problem:

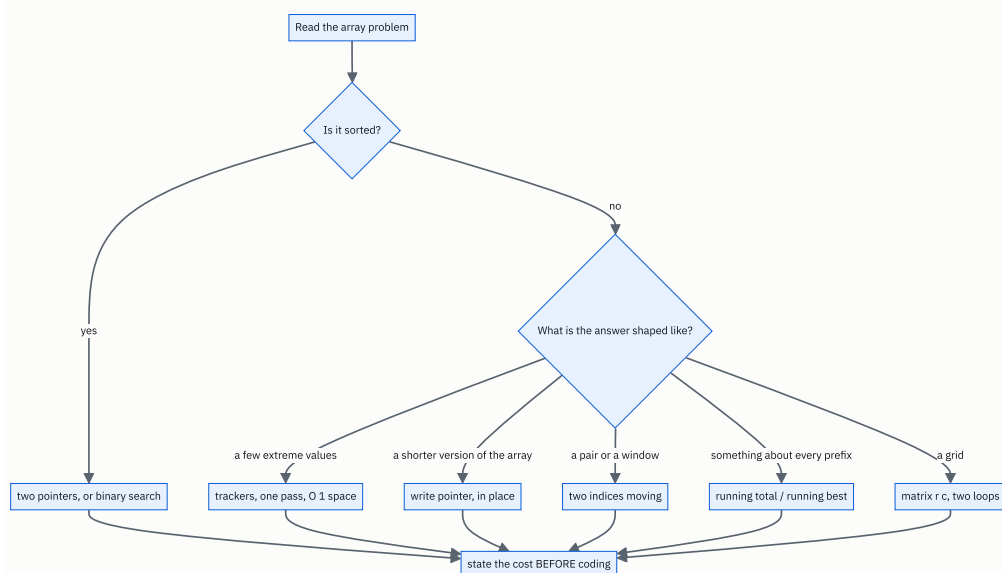


FIGURE 18.1

What to notice: every branch ends in the same box. Whatever you choose, the last thing you do before typing is say the cost out loud and get a nod.

5 The code, built step by step

Two problems, worked the way you would work them in the room. Read them as transcripts, not as solutions.

Problem one — the warm-up

“You’re given an array of daily prices. Find the maximum profit from buying on one day and selling on a later day. You may only make one transaction.”

Beat 1, clarify. *“So I buy once and sell once, and the sell must be strictly after the buy. If prices only go down, is the answer zero — meaning I don’t have to trade — or should I return a negative number?”* Say it is zero. *“And can the array be empty or have one element?”*

That question matters. `[7, 6, 4, 3, 1]` returns `0`, not `-6`. A candidate who does not ask writes the wrong function about a third of the time.

Beat 2, example. *“Take `[7, 1, 5, 3, 6, 4]`. Buy at 1 on day 1, sell at 6 on day 4, profit 5. Buying at 7 is no good because everything after it is smaller.”*

Beat 3, brute force. *“The obvious version is every pair: for each buy day, look at every later sell day and keep the best difference. Two nested loops, so $O(n^2)$ time and $O(1)$ space. Correct, but it re-examines the same prices constantly.”*

Beat 4, optimise. *“The waste is that for each sell day I re-scan all the earlier days looking for the cheapest. But I could have kept that as I went. So: one pass, and at every price I ask ‘if I sold today, what would I make?’ — which is today’s price minus the cheapest price seen so far. Then I update the cheapest. That’s $O(n)$ time and $O(1)$ space.”*

This is the single-pass habit from [day 014](#), with two trackers instead of two largest.

Beat 5, code. Build it in pieces, narrating.

```
if not prices:
    return 0
cheapest = prices[0]
best = 0
```

“Guard the empty case first. `cheapest` starts as a real element, not zero — prices are positive here so zero would be safe, but I never initialise a tracker to zero out of habit, because it breaks the moment values can be negative. `best` starts at 0 because doing nothing is allowed.”

```
for price in prices[1:]:
    best = max(best, price - cheapest)
    cheapest = min(cheapest, price)
```

“For each later price, first ask what I’d make selling today against the cheapest so far, then update the cheapest.”

The order of those two lines is the whole problem. Update `cheapest` first and `price - cheapest` can be today minus today, which is zero — you would be buying and selling on the same day. Say that out loud while you write it; it is exactly the kind of thing an interviewer is listening for.

Beat 6, test. Walk `[7, 1, 5, 3, 6, 4]` through by hand, out loud:

```
price=1 : best = max(0, 1-7) = 0    cheapest = 1
price=5 : best = max(0, 5-1) = 4    cheapest = 1
price=3 : best = max(4, 3-1) = 4    cheapest = 1
price=6 : best = max(4, 6-1) = 5    cheapest = 1
price=4 : best = max(5, 4-1) = 5    cheapest = 1
answer 5
```

Then the edges you named in beat 1: `[]` → 0. `[5]` → 0. `[7,6,4,3,1]` → 0. `[2,1]` → 0.

Problem two — the one with a twist

“You have two sorted arrays. The first has `m` real values followed by `n` empty slots — exactly enough room for the second array. Merge them into the first, in sorted order, in place.”

Beat 1, clarify. “So `nums1` has length `m + n`, the last `n` entries are placeholders, and I must not allocate a second array. Can `n` be zero? Can `m` be zero?” Both yes.

Beat 2, example. “`nums1 = [1, 2, 3, 0, 0, 0]` with `m = 3`, `nums2 = [2, 5, 6]` with `n = 3`. The answer is `[1, 2, 2, 3, 5, 6]`.”

Beat 3, brute force. “I could copy `nums2` into the empty slots and sort the whole thing — that’s two lines and $O((m+n) \log(m+n))$. It works and it throws away the fact that both inputs are already sorted, which is clearly the point of the question.”

Say the cheap answer. Then say why it is not the answer. That sequence is worth marks on its own.

Beat 4, optimise. “The standard merge takes the smaller front element of each array and writes it out — that’s $O(m + n)$. But here I’d be writing into the front of `nums1`, which still holds values I haven’t read yet, so I’d destroy them.”

Pause here. This is the insight, and it is worth saying deliberately:

“So I go backwards instead. The largest value overall must end up in the very last slot — and the very last slot is empty. So if I fill from the back, I’m always writing into space that is either a placeholder or a position I’ve already read past. The write index never collides with either read index.”

That is exactly the `write ≤ read` rule from [day 015](#), turned around.

Beat 5, code.

```
write = m + n - 1
i, j = m - 1, n - 1
```

“Three indices, all at the back. `write` is the last slot; `i` is the last real value in `nums1`; `j` is the last value in `nums2`.”

```
while j ≥ 0:
    if i ≥ 0 and nums1[i] > nums2[j]:
        nums1[write] = nums1[i]
        i -= 1
    else:
        nums1[write] = nums2[j]
        j -= 1
    write -= 1
```

“I loop while `nums2` still has values. Once `j` runs out I’m done, because anything left in `nums1` is already in its correct place — I never moved it.”

The loop condition is the second insight. `while j ≥ 0` rather than `while i ≥ 0` and `j ≥ 0` means you never need a clean-up loop afterwards. Say why: leftovers in `nums2` must be copied, leftovers in `nums1` are already home.

The `i ≥ 0` inside the `if` handles `nums1` running out first, which happens when `m = 0` or when `nums2` holds all the small values.

Beat 6, test. `nums1 = [1,2,3,0,0,0]`, `m = 3`, `nums2 = [2,5,6]`:

```
write=5 i=2 (3) j=2 (6) 3 > 6? no → write 6 j=1 write=4
write=4 i=2 (3) j=1 (5) 3 > 5? no → write 5 j=0 write=3
write=3 i=2 (3) j=0 (2) 3 > 2? yes → write 3 i=1 write=2
write=2 i=1 (2) j=0 (2) 2 > 2? no → write 2 j=-1 write=1
j < 0, stop.  nums1 = [1, 2, 2, 3, 5, 6]
```

Then the edges: `m = 0` with `nums1 = [0]`, `nums2 = [1]` → `[1]`. And `n = 0` → unchanged.

The complete solutions

```
def max_profit(prices: list[int]) → int:
    """LeetCode 121. Best single buy-then-sell profit. One pass, two trackers."""
    if not prices:
        return 0
    cheapest = prices[0]      # never 0 – a tracker starts as a real value
    best = 0                  # 0 because doing nothing is allowed
    for price in prices[1:]:
        best = max(best, price - cheapest)  # sell today against the cheapest so far
        cheapest = min(cheapest, price)     # AFTER, or you buy and sell the same day
    return best

def merge(nums1: list[int], m: int, nums2: list[int], n: int) → None:
    """LeetCode 88. Merge nums2 into nums1 in place. nums1 has m values + n free slots.

    Filled from the back, so the write index never overtakes either read index.
    """
    write = m + n - 1      # last slot of nums1
    i, j = m - 1, n - 1    # last real value of each input

    while j ≥ 0:            # only nums2 leftovers need moving
        if i ≥ 0 and nums1[i] > nums2[j]:
            nums1[write] = nums1[i]
            i -= 1
        else:
            nums1[write] = nums2[j]
            j -= 1
        write -= 1

if __name__ == "__main__":
    print(max_profit([7, 1, 5, 3, 6, 4]))  # 5
    print(max_profit([7, 6, 4, 3, 1]))    # 0
    print(max_profit([]))                 # 0
    print(max_profit([5]))                # 0

    a = [1, 2, 3, 0, 0, 0]
    merge(a, 3, [2, 5, 6], 3)
    print(a)                             # [1, 2, 2, 3, 5, 6]

    b = [0]
    merge(b, 0, [1], 1)
    print(b)                             # [1]

    c = [1]
    merge(c, 1, [], 0)
    print(c)                             # [1]
```

6 What it costs

`max_profit`

The loop runs over `prices[1:]`, which is `n - 1` elements. Each turn does one subtraction, one `max` and one `min` — a fixed amount of work. So `n - 1` turns of constant work: **$O(n)$ time.**

Space: `cheapest` and `best` are two integers, whatever the length of the list. **$O(1)$ extra space.**

One honest caveat worth mentioning: `prices[1:]` creates a copy of the list, which is $O(n)$ extra space. In an interview say so and offer the index version if it matters:

```
for k in range(1, len(prices)):
    best = max(best, prices[k] - cheapest)
    cheapest = min(cheapest, prices[k])
```

Noticing that yourself, unprompted, reads extremely well.

Against the brute force: the nested-loop version does $n(n-1)/2$ comparisons. At $n = 10,000$ that is about 50 million; the one-pass version does 10,000. Five thousand times fewer, and it is the difference between a visible pause and no pause at all.

merge

Each turn of the loop writes exactly one value and decreases exactly one of `i` or `j`. Every value from `nums2` gets written, and at most every value from `nums1` gets moved, so the loop runs at most $m + n$ times: **$O(m + n)$ time**, which is linear in the total number of elements.

Space: three integers. **$O(1)$ extra space** — nothing is allocated, which is what “in place” was asking for.

Against the copy-and-sort version, which is $O((m+n) \log(m+n))$: with $m = n = 100,000$, sorting is about $200,000 \times 17.6 \approx 3.5$ million comparisons, while the merge is 200,000 writes. Around seventeen times fewer, and it uses the sortedness that copy-and-sort discards.

7 The traps

The near-miss: merging from the front

The instinct is to merge the way you were taught, front to back:

```
def merge_front(nums1, m, nums2, n):
    write = 0
    i, j = 0, 0
    while i < m and j < n:
        if nums1[i] ≤ nums2[j]:
            nums1[write] = nums1[i]; i += 1
        else:
            nums1[write] = nums2[j]; j += 1
        write += 1
    return nums1

print(merge_front([1, 2, 3, 0, 0, 0], 3, [2, 5, 6], 3))
```

```
[1, 2, 2, 2, 0, 0]
```

The 3 has vanished. On the third turn the code wrote `nums2[0] = 2` into `nums1[2]`, which was still holding the 3 it had not yet read. **Writing forwards into an array you are still reading forwards destroys data**, which is exactly the rule from [day 015](#) — `write ≤ read` — being violated. Going backwards restores it.

The near-miss: updating the cheapest price too early

```
for price in prices[1:]:
    cheapest = min(cheapest, price)    # moved up one line
    best = max(best, price - cheapest)
```

On `[7, 1, 5, 3, 6, 4]` this still returns 5, so it looks fine. It is wrong in principle and it shows on a strictly falling input plus one: on `[3, 2, 1]` it returns 0, which is right by accident, because the bug allows buying and selling on the same day and that always yields exactly 0. It never produces a *bigger* wrong answer, which is precisely why it survives testing and why an interviewer who spots it will ask you to justify the order. Have the reason ready: **you must sell at a price you bought before, so the buy candidate has to come from strictly earlier days.**

The near-miss: `best = 0` when the answer can be negative

`max_profit` starts `best` at 0 because the problem allows doing nothing. Change the problem to “*you must make exactly one transaction*” and the answer for `[7, 6, 4, 3, 1]` becomes `-1`, and starting at 0 returns 0 forever. That is the day-014 lesson again: **a tracker’s starting value is part of the contract, not a detail.** Ask which problem you are being given.

The real error: forgetting the empty guard

```
def max_profit(prices):
    cheapest = prices[0]
    best = 0
    for price in prices[1:]:
        best = max(best, price - cheapest)
        cheapest = min(cheapest, price)
    return best

print(max_profit([]))
```

```
Traceback (most recent call last):
  File "t.py", line 9, in <module>
    print(max_profit([]))
    ~~~~~^
  File "t.py", line 2, in max_profit
    cheapest = prices[0]
    ~~~~~^
IndexError: list index out of range
```

You asked about the empty input in beat 1. Handle it in beat 5. Candidates ask the question, get the answer, and then forget to write the guard — which is worse than never asking, because you have demonstrated that you knew.

The process traps, which cost more marks than the code ones

- **Coding before agreeing.** You typed for twenty minutes and the interviewer wanted a different approach. Unrecoverable in forty-five minutes.
- **Silence.** Two minutes of quiet typing is two minutes of no information for the person scoring you. Sneha's invisible mirror check.
- **Naming a complexity without counting.** "This is $O(n)$ " earns nothing. "The loop runs n times and each turn is constant work, so $O(n)$ " earns the mark.
- **Testing with the example you were given.** Of course that passes. Test with the edge cases you named in beat 1 — that is the part being scored.
- **Fixing a bug by changing a number.** Changing `n` to `n-1` to see if it works tells the interviewer you do not know why it is wrong. Say what the index should be and why, then change it.

8 In the interview

How it gets asked

- "We'll do two problems today. Talk me through your thinking as you go." — the standard opening.

- “Here’s the problem. Take a minute to read it.” — and the minute is being watched. Use it to produce a question, not a silent stare.
- “How would you test this?” — asked when you finish early, and a gift if you have edge cases ready.
- “Can you do better?” — almost always means yes, and almost always means there is a one-pass or a two-pointer version.

What to say out loud, in the first ninety seconds

The first ninety seconds are the same for **every** array problem you will ever be given:

1. **Restate it in your own words.** “So I’m given an array of prices, one per day, and I need the best profit from a single buy followed by a later sell.” If your restatement is wrong, you find out now instead of at minute thirty.
2. **Ask the four questions.** Can I modify the input? Is it sorted? Duplicates, negatives, empty? Does order matter? Forty seconds, and it catches most of the traps in this phase.
3. **Give one example and its answer.** Small, concrete, six elements at most, computed out loud.
4. **Name the brute force with its cost.** “The obvious approach is every pair, which is $O(n^2)$.”
5. **Name the better approach and its cost, and pause.** “But I can do it in one pass with two variables, $O(n)$ time and $O(1)$ space — shall I code that?” Then stop and wait for the nod. That pause is deliberate and it is worth marks.

The follow-ups

“Can you do better than $O(n)$?” No, and here is why, which is a better answer than trying. Any correct solution has to look at every price at least once — if I skip a price, an adversary can put the day’s best buy or best sell there and my answer is wrong. So $\Omega(n)$ is a lower bound on this problem, and my solution matches it. The only thing left to improve is space, and I am already at $O(1)$. When an interviewer asks “can you do better” and the honest answer is no, saying **why** it is no is worth more than a nervous attempt at something faster.

“Now allow at most two transactions.” That is LeetCode 123, and it generalises the trackers. Instead of two variables I keep four, tracking the best I can have done after each of four stages: bought once, sold once, bought again, sold again. For each price I update all four in order — the best after buying once is the best of what it was and minus this price; the best after selling once is the best of what it was and the first plus this price; and

so on. Still one pass, $O(n)$ time and $O(1)$ space. And the general version — at most k transactions — is the same idea with $2k$ trackers, or a dynamic programming table if k is large, which is the phase starting on [day 143](#).

“In the merge, why does the loop condition only check j ?” Because when `nums2` is exhausted, whatever is left in `nums1` is already in the right place — it was never moved, and everything larger has been written above it. If instead `nums1` runs out first, the remaining `nums2` values still have to be copied down, which the loop keeps doing because j is still non-negative and the $i \geq 0$ guard sends it down the else branch. Writing `while i  $\geq$  0 and j  $\geq$  0` would be correct too, but then I would need a second loop afterwards to drain `nums2`, and that extra loop is a place to introduce a bug for no benefit.

“What if you couldn’t modify `nums1` at all?” Then it is a different problem: allocate a result of size $m + n$ and do the standard forward merge into it, taking the smaller front element each time. $O(m + n)$ time and now $O(m + n)$ space. The backwards trick exists purely to make the in-place version safe; with a fresh output array there is nothing to collide with, so forwards is simpler and I would write that. It is worth being explicit that the clever version is a response to a constraint, not a better algorithm.

A model answer

Written out as it would actually sound, for problem two.

“Let me restate it. `nums1` has length `m + n`. The first `m` entries are real, sorted values; the last `n` are placeholders. `nums2` has `n` sorted values. I have to end up with all `m + n` values sorted inside `nums1`, without allocating a second array. Is that right? And can `m` or `n` be zero?

...Both can be zero, fine.

Example: `nums1 = [1, 2, 3, 0, 0, 0]` with `m = 3`, `nums2 = [2, 5, 6]`.
Answer `[1, 2, 2, 3, 5, 6]`.

The cheap approach is to copy `nums2` into the placeholders and sort the whole thing. That is two lines and `O((m+n) log(m+n))`, and it works — but it throws away the fact that both inputs are already sorted, which is obviously the point of the question, so let me use that.

The standard merge compares the front of each array and writes out the smaller one. The problem here is where I write it. The front of `nums1` still holds values I have not read yet, so writing there destroys them — I’d lose the 3 in my example. That is the same constraint as any in-place compaction: the write position must never overtake the read position.

So I fill from the back instead. The largest value overall belongs in the last slot, and the last slot is a placeholder — guaranteed free. Every subsequent write goes into a position that is either still a placeholder or one I have already read past. The write index can never collide.

```
def merge(nums1: list[int], m: int, nums2: list[int], n: int) → None:
    write = m + n - 1
    i, j = m - 1, n - 1
    while j ≥ 0:
        if i ≥ 0 and nums1[i] > nums2[j]:
            nums1[write] = nums1[i]
            i -= 1
        else:
            nums1[write] = nums2[j]
            j -= 1
        write -= 1
```

Two details worth calling out. The loop runs while `j ≥ 0` rather than while both are in range, because once `nums2` is empty anything left in `nums1` is already sitting in its correct place — so I need no clean-up loop. And the `i ≥ 0` inside the condition handles `nums1` running out first, which is what happens when `m` is zero.

Tracing my example: write 6, write 5, then 3 beats 2 so write 3, then 2 versus 2 goes to the else branch and writes the 2 from `nums2`, and now `j` is negative so we stop with `[1, 2, 2, 3, 5, 6]`. The leading 1 was never touched, which is exactly the

point.

$O(m + n)$ time — one write per element, at most — and $O(1)$ extra space. Edge cases: $n = 0$ leaves `nums1` untouched because the loop never runs; $m = 0$ copies all of `nums2` down; equal values go to the else branch, so the merge is stable in the sense that `nums2`'s copy lands after.”

9 Recall card

- **Six beats:** clarify, example, brute force, optimise, code, test. Half the time is before you type.
- **Never code without agreement.** State the approach and its cost, then pause for the nod.
- **Narrate the decision and the reason, once per decision.** Silence is indistinguishable from luck.
- **Four questions for every array problem:** can I modify it, is it sorted, duplicates/negatives/ empty, does order matter?
- **Test with the edge cases you named yourself** — not with the example you were handed.

1 What this is, and why they ask it

A call across a network has three outcomes, not two. It can succeed. It can fail with an answer. And it can fail **without** an answer — the request went out and nothing came back — in which case you do not know whether the work was done. That third case is not an edge case. It is the normal behaviour of networks, and almost everything difficult about distributed systems grows out of it.

Today has three parts and they are one idea. **Status codes** are how a service says what happened. **Error design** is how it says it usefully enough for a caller to act. **Idempotency** is the property that makes it safe to ask again when you did not hear the answer.

Interviewers ask this constantly, in two shapes. The quick version — *“what’s the difference between 401 and 403?”*, *“which HTTP methods are idempotent?”* — is a five-second check on whether you have worked with real systems. The long version — *“a payment request times out; what do you do?”* — is one of the best design questions there is, because there is no way to answer it well without understanding that the response, not the request, is the thing that went missing. Candidates who have only read about REST say “retry it”. Candidates who have shipped payments say “not without an idempotency key”, and the conversation gets interesting.

2 The story

Kiran stops for fuel at about nine at night, on the road out of Hyderabad, because he is going to his sister’s place in the morning and does not want to stop on the way. Two thousand rupees. The boy fills it, wipes his hands, and turns the little stand round so the code faces him.

Kiran scans it, types 2000, holds his thumb on the sensor, and waits.

The wheel spins. It spins for longer than it usually does. He can see one bar of signal and it keeps dropping to nothing and coming back. After maybe forty seconds the app gives up and says the transaction has failed, in red, and offers him a button to try again.

He looks up. The boy is looking at his own phone, the one clipped to the stand, and shakes his head. Nothing has come. There is a small queue behind now, a scooter and a lorry, and the lorry driver is being patient in the way that makes it worse.

So Kiran pays again. Scans, types 2000, thumb, and this time it goes through in four seconds and the boy’s phone makes the noise and everybody relaxes.

He is a kilometre down the road when his phone buzzes twice. Two messages, thirty seconds apart. Four thousand rupees have left his account.

The first payment had gone through. It had gone through nearly a minute earlier, in fact — the money had left, the fuel company had it, and the only thing that had failed was the small message coming back to tell him so. His phone never heard the answer, so it assumed there had not been one.

What saves him is the next morning. He rings the bank and the woman asks for the reference numbers, and there are two of them, different, one for each attempt. Because each attempt carried its own number, and because that number went with the money, they can see exactly which two payments went out and put one back. It takes four days.

He tells his sister about it and she says the same thing happened to her at a supermarket, except that when she tapped again the machine beeped and said *already paid* and refused to take the second one. Same problem, and somebody at that shop’s end had thought about it beforehand.

3 The idea in plain English

Kiran’s problem is not that something failed. It is that **he could not tell the difference between “it did not happen” and “it happened and I did not hear about it”**. From where he was standing those two look identical, and they need opposite responses.

The three outcomes of any network call

WHAT HAPPENED	YOU KNOW	WHAT TO DO
Success — a 2xx came back	It worked	Carry on
Failure with an answer — a 4xx or 5xx came back	It definitely did not work, and why	4xx : fix the request. 5xx : it may be worth asking again
No answer — timeout, dropped connection	Nothing	The dangerous one. It may have worked

The third row is Kiran at the pump. The request reached the far end and was carried out; the response was lost on the way back. There is no way, from the client, to tell that apart from the request never arriving at all.

Status codes: what the service says happened

An HTTP response starts with a three-digit number, and the first digit is the family. You met them on day 005; this is the working set.

CODE	NAME	MEANS, IN PLAIN WORDS
200	OK	Here it is.
201	Created	I made it. A <code>Location</code> header says where.
202	Accepted	I have taken the work; it is not finished yet.
204	No Content	Done, nothing to send back. The usual reply to <code>DELETE</code> .
301 / 302	Moved	It lives elsewhere. <code>301</code> permanently, <code>302</code> for now.
304	Not Modified	You already have the current version.
400	Bad Request	I cannot parse this.
401	Unauthorized	I do not know who you are. Really means unauthenticated.
403	Forbidden	I know who you are and you may not.
404	Not Found	No such thing.
405	Method Not Allowed	That thing exists; that verb does not apply to it.
409	Conflict	It exists, and its current state forbids this.
422	Unprocessable Entity	Well-formed, but semantically wrong — an empty comment body.
429	Too Many Requests	Slow down. A <code>Retry-After</code> header says how long.
500	Internal Server Error	We broke, and we do not have a better word for it.
502	Bad Gateway	Something in front of the real server got a bad answer from it.
503	Service Unavailable	We are up but cannot serve you — overloaded, or in maintenance.
504	Gateway Timeout	Something in front waited for the real server and gave up.

Four distinctions that get asked directly:

- **401 versus 403**. `401` is *unauthenticated* despite the name — I do not know who you are, so send credentials. `403` is *unauthorised* — I know exactly who you are and the answer is still no. Sending credentials again will not help.
- **400 versus 422**. `400` means I could not even read your request. `422` means I read it fine and it does not make sense.
- **404 versus 409**. `404` — the thing is not there. `409` — it is there, and its state forbids what you asked. Commenting on a locked post is `409`, not `400`.
- **502 versus 504**. Both come from something in front of your server. `502` means it got a broken answer; `504` means it got no answer in time. If you are debugging and seeing `504`, your backend is slow, not broken.

And the one rule that matters more than any individual code: **the status code must be honest**. `200 OK` with `{"error": "insufficient funds"}` in the body is the worst thing in API design, because every load balancer, CDN, retry library and monitoring dashboard in the path reads the number and believes it. Your error rate graph will show zero while customers cannot pay.

The client's decision tree

The whole point of a good status code is that a caller can act on it without reading the body:

- `2xx` — done.
- `4xx` — your fault. **Do not retry**. The identical request will fail identically. The two exceptions are `429`, which means retry later, and `408 Request Timeout`.
- `5xx` — their fault, possibly temporary. **Retrying may work** — but only if the operation is safe to repeat.
- **No answer at all** — you do not know. **Retrying is only safe if the operation is idempotent**, and if it is not, you must make it so.

Idempotent, in plain words

An operation is **idempotent** if doing it twice leaves the world exactly as doing it once did.

- “Set the light switch to off” — idempotent. Do it five times, the light is off.
- “Toggle the light switch” — not idempotent. Five times and you have a 50/50 chance.
- `DELETE /comments/91` — idempotent. After the first one it is gone; the rest change nothing.
- `POST /payments` for ₹2,000 — **not** idempotent. Kiran's four thousand rupees.

From [day 016](#): `GET`, `PUT` and `DELETE` are idempotent by specification; `POST` and `PATCH` are not. That is why “just retry it” is fine for a read and disastrous for a payment.

The idempotency key

Kiran's reference numbers are the fix, and the fix has a name.

The client **generates a unique value per attempt** — a UUID, say — and sends it with the request:

```
POST /v1/charges
Idempotency-Key: 6f1a2c4e-9b3d-4f8a-b2e1-7c5d0a9e3f11
```

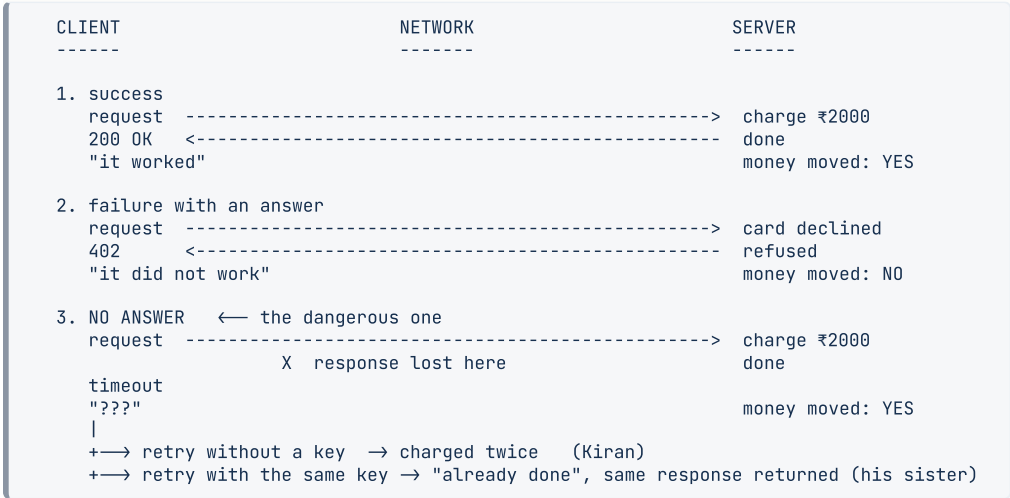
The server promises: *the first request carrying this key is executed; any later request carrying the same key is not executed again, and gets the same response the first one got.*

The crucial detail, and the one candidates miss: **the client generates the key once, before the first attempt, and reuses the same key on every retry.** A new key per retry defeats the whole mechanism — that is exactly Kiran’s situation, where his two attempts had two different reference numbers, which is why the bank could tell them apart but the pump could not stop the second one.

His sister’s supermarket had it right: the till sent the same reference on the retry, and the far end said *already paid*.

4 The picture

The three outcomes, and the one that hurts:



What to notice: rows 2 and 3 look identical from the left-hand column. The client sees “no success” both times. Only the server knows they are opposite, and the whole of idempotency exists to let the client ask again without needing to know which one it was.

How the server actually implements the promise:

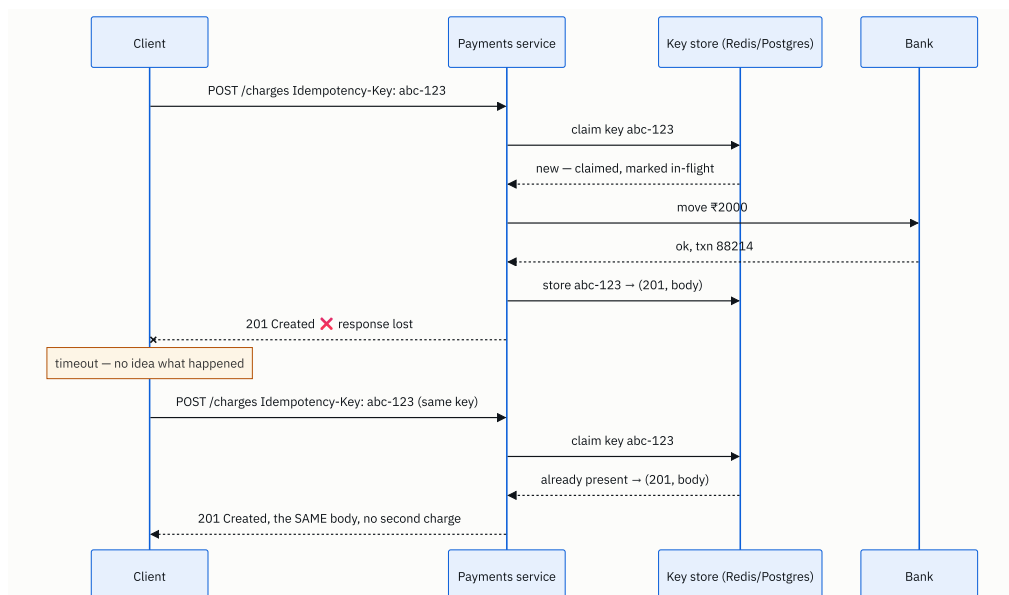


FIGURE 18.2

What to notice: the second request never reaches the bank. The key store is checked first and answers from memory. Also notice the *in-flight* marking on the first claim — that is what stops two retries arriving simultaneously and both getting through, and it is the detail that separates a thought-through answer from a hand-waved one.

5 How it actually works

Designing the error body

A status code alone is not enough for a caller to act. The body should be machine-readable and stable. There is a standard for this — RFC 9457, `application/problem+json`, previously RFC 7807 — and even where teams do not follow it exactly, they follow its shape:

```

{
  "type": "https://api.example.com/errors/insufficient-funds",
  "title": "Insufficient funds",
  "status": 402,
  "detail": "The card ending 4417 has a balance of ₹1,240; ₹2,000 was requested.",
  "instance": "/charges/ch_8fj2",
  "request_id": "req_01H9X4",
  "retryable": false
}
  
```

Four rules for the body:

1. **A stable machine-readable code.** Clients branch on `type` or an `error_code` string, never on the human sentence. The sentence will be reworded; the code must not be.
2. **A `request_id`.** When a customer reports a problem, this is the only thing that lets support find the request in the logs. Return it on **every** response, success included.
3. **Say whether it is worth retrying.** An explicit `retryable` flag, or a `Retry-After` header, saves every client from guessing.
4. **Never leak internals.** A stack trace or an SQL fragment in an error body is a security problem. Log it server-side against the `request_id` and return the id.

Implementing an idempotency key, properly

This is a genuinely good thing to be able to sketch, because it comes up in payments, order and booking questions constantly.

Storage. A table or a Redis entry keyed by `(api_key, idempotency_key)` — scoped per caller, so one customer cannot collide with another's keys. Each entry holds the state, the response status, the response body, and a hash of the request body.

On a request with a key:

1. Try to **claim** the key with an atomic insert — `INSERT ... ON CONFLICT DO NOTHING` in Postgres, or `SET key value NX` in Redis. Atomic is the whole point; a read-then-write has a race.
2. **If you claimed it**, mark it in-flight, do the work, then store the final status and body against the key.
3. **If it already existed and is finished**, return the stored status and body verbatim. Do not redo anything.
4. **If it already existed and is still in-flight**, return `409 Conflict` — a retry has arrived while the original is still running. The client should wait and ask again.
5. **If the key matches but the request body is different**, return `422`. Someone is reusing a key for a different operation, which is a client bug you want to surface loudly. This is why you store the request hash.

Expiry. Keys do not live forever — Stripe keeps them 24 hours. Long enough to cover any realistic retry, short enough that the store stays small.

Real implementations to name: Stripe's `Idempotency-Key` header is the canonical one. AWS calls it a `ClientToken` on many APIs. UPI and card networks use a merchant-supplied reference. Kafka has an idempotent producer that deduplicates by producer id and sequence number.

Retrying, without making it worse

Retrying is not free. Done naively it converts a small problem into an outage.

Retry only what is safe. 4xx other than 429 / 408 : never. 5xx and timeouts: yes, if the operation is idempotent or carries a key.

Back off exponentially. Wait 1 second, then 2, then 4, then 8. A tight retry loop against a struggling service is an attack on it.

Add jitter — randomness — to the wait. This is the part people leave out and it matters most. If ten thousand clients all fail at the same instant and all wait exactly 1 second, they all come back at the same instant. That is a **thundering herd**, and it is how a brief blip becomes a sustained outage. Randomising each wait between zero and the backoff spreads them out.

Cap the attempts. Three or four, then give up and surface a real error.

Set a timeout at all. A client with no timeout does not fail; it hangs, holding a connection, until something else falls over. Every call needs an explicit deadline.

The mature version of this is a **circuit breaker** — after enough consecutive failures, stop calling for a while and fail immediately, so a dead dependency does not consume all your threads. Naming it is enough for now.

The delivery guarantees, named

You will hear three phrases, and they are worth being precise about:

- **At most once** — send it and do not retry. No duplicates, but messages can be lost.
- **At least once** — retry until acknowledged. Nothing is lost, but duplicates happen.
This is what almost every real system does.
- **Exactly once** — no loss, no duplicates. Genuinely impossible to guarantee end to end across an unreliable network.

The practical resolution, and it is worth saying in exactly these words: **you get at-least-once delivery plus idempotent processing, which together look like exactly-once to the user.** That is what Kafka’s “exactly-once semantics” actually is, and it is what an idempotency key gives you over HTTP.

6 The numbers

How often does the dangerous case actually happen?

Suppose the payments API handles **50,000 requests a second at peak** and 0.1% of calls time out — a realistic figure on mobile networks.

```
50,000 × 0.001 = 50 requests/second with no answer
50 × 86,400    = 4,320,000 unknown-outcome requests/day
```

If even 1% of those are retried without protection and the original had in fact succeeded:

$$4,320,000 \times 0.01 = 43,200 \text{ double charges/day}$$

Forty-three thousand angry customers a day, each costing a support call. At ₹150 a call that is ₹6.5 million a day in support alone, before refunds. **This is why idempotency keys exist**, and quoting an order of magnitude like that is far more persuasive than saying “duplicates are bad”.

Retry storms

A service handling 10,000 requests a second starts failing. Every client retries three times with no backoff:

$$10,000 \text{ original} + 30,000 \text{ retries} = 40,000 \text{ requests/second}$$

The load **quadrupled at the exact moment the service was least able to take it**. This is the mechanism by which a thirty-second blip becomes a two-hour outage, and it is entirely self-inflicted.

With exponential backoff and jitter, those 30,000 retries spread across the next 15 seconds:

$$30,000 \div 15 = 2,000 \text{ extra requests/second}$$

10,000 becomes 12,000 rather than 40,000 — a 20% bump instead of a 300% one, which a service with normal headroom absorbs without noticing.

The cost of storing keys

At 50,000 requests a second with a 24-hour retention, and about 2 KB per stored entry (status, body, request hash):

$$\begin{array}{ll} 50,000 \times 86,400 & = 4.32 \text{ billion keys/day} \\ 4.32\text{e}9 \times 2 \text{ KB} & \approx 8.6 \text{ TB} \end{array}$$

That is far too much, and noticing it is the point. Two things fix it. Only *mutating* requests need keys — reads are already idempotent — so if 10% of traffic writes, it is 860 GB. And you store a digest rather than the whole response for large bodies, taking the entry to around 300 bytes:

$$4.32\text{e}8 \text{ writes/day} \times 300 \text{ bytes} \approx 130 \text{ GB}$$

130 GB in Redis with a 24-hour expiry is entirely reasonable. Working that through out loud is the kind of thing that turns a textbook answer into a design answer.

The retry budget

If 1% of calls need one retry, and each retry doubles the work for that call:

```
extra load = 1% × 1 = 1% more traffic
```

Negligible — until failure rates rise. At a 20% failure rate with three retries each, the extra load is $20\% \times 3 = 60\%$. **Retries amplify precisely when you can least afford it**, which is the argument for a retry budget: cap retries at some percentage of total traffic and shed the rest.

7 The trade-offs

Idempotency keys are not free

You are adding a write to a shared store on the hot path of every mutating request — one extra network round trip, and a new dependency that can itself fail. If the key store goes down, do you fail the payment or process it unprotected? Both answers are defensible; the point is that it is a decision, and having an opinion is what is being tested. For payments, most teams fail closed: better a declined charge than a duplicate one.

There is also a real correctness subtlety. The key must be claimed **atomically**, or two simultaneous retries can both pass the check before either writes. That is a compare-and-set, not a read followed by a write, and it is the detail interviewers probe.

Who generates the key?

The client, and it must generate it once and reuse it. That is also its weakness: a badly written client that generates a fresh key per attempt gets no protection at all, and you cannot stop it. The server-side alternative — deduplicating on a hash of the request body — needs no client cooperation but is wrong for legitimate repeats. Two identical ₹100 coffee purchases a minute apart are two real charges, not a duplicate. **There is no way for the server to tell those apart without a client-supplied key**, and saying so is the complete answer to “why not just hash the body?”

Retries versus duplicates

You cannot have neither. Retry and you risk duplicates; do not retry and you risk lost work. The industry has settled on at-least-once delivery plus idempotent processing, because duplicates are a problem you can engineer away and lost payments are not.

Should errors be detailed?

Detailed errors make integration far easier and are the difference between a good API and a frustrating one. They also leak information: “no user with that email” tells an attacker which emails are registered. On authentication endpoints, be deliberately vague — “invalid email or password” — and be generous everywhere else.

The sentence that separates candidates

I would not add idempotency keys if the operation is naturally idempotent already — a `PUT` that sets a value, a `DELETE`, anything writing a computed state rather than appending an event. Adding a key store to those buys nothing and adds a dependency to the hot path. Keys are for operations that create something or move money, where a second execution has a real-world cost that cannot be undone by repeating it.

8 In the interview

How it gets asked

- “Your client retries a failed payment request. What could go wrong?” — the main event. The correct first move is to distinguish “failed with an answer” from “no answer”.
- “What’s the difference between 401 and 403?” — the five-second check.
- “Which HTTP methods are idempotent?” — often followed by “and is PATCH?”, which is the interesting half.
- “How would you make this API safe to retry?” — asked mid-design, once you have proposed an endpoint that creates something.
- “What status code would you return for X?” — with X being an empty collection, a locked resource, or a duplicate submission.

What to say out loud, in the first ninety seconds

1. **Split the failure into three cases, immediately.** “There are three outcomes, not two: a success, a failure with a response, and no response at all. The third is the dangerous one, because the request may well have been carried out and only the reply was lost.”
2. **Say what each means for retrying.** “A 4xx is my fault and retrying is pointless. A 5xx may be transient and is worth retrying. A timeout tells me nothing, so retrying is only safe if the operation is idempotent.”
3. **Define idempotent in one sentence.** “Doing it twice has the same effect as doing it once.”

4. **Say why a payment is not.** *“POST /charges is not idempotent — two calls make two charges. So a blind retry double-charges the customer.”*
5. **Give the fix by name and mechanism.** *“An idempotency key. The client generates a unique value before the first attempt and sends the same value on every retry. The server claims that key atomically, does the work once, stores the response against the key, and returns the stored response to any later request with the same key.”*
6. **Add the detail that shows depth.** *“Two things to get right: the claim has to be atomic, or two simultaneous retries both pass the check; and the key must be generated once by the client, not per attempt.”*
7. **Mention backoff, unprompted.** *“And retries need exponential backoff with jitter, or ten thousand clients failing together all come back in the same instant and turn a blip into an outage.”*

The follow-ups

“Is PATCH idempotent?” Not by specification, and it depends entirely on what the patch does. PATCH with `{"status": "shipped"}` sets a value, so applying it twice leaves the same state — idempotent in practice. PATCH with `{"increment_views": 1}` is not, because the second application changes the result again. That is exactly why the specification refuses to promise: it cannot know what your patch body means. If I need a patch to be safely retryable I either design the body to set absolute values rather than deltas, or I put an idempotency key on it. PUT is idempotent because it replaces the whole resource — the second identical PUT produces the same final state — and DELETE is idempotent even though the second call may return 404, because the state afterwards is identical, which is what idempotency is about.

“The user double-taps the pay button. Same problem?” Related but not the same, and the distinction matters. A retry is one logical operation attempted twice; a double-tap is arguably two operations that happen to be identical. The fix is the same mechanism at a different layer: the client generates the idempotency key when the payment screen is opened, not when the button is pressed, so both taps carry the same key and the second is deduplicated. That is also why the key cannot be a hash of the request body — two genuinely separate ₹100 purchases a minute apart must both go through, and only a client-supplied key can tell “the same payment, tried twice” from “two payments that look alike”. Alongside that I would disable the button on first press, but that is a comfort measure, not a correctness one, because a flaky network retry does not involve the button at all.

“What if the idempotency key store goes down?” That is a real design decision and I would want to state it explicitly rather than let it be implicit. Fail closed: reject the payment with a 503 and a `Retry-After`, because a declined charge is recoverable and a duplicate charge is not. Fail open — process without protection — is defensible for low-

value, easily-refunded operations, and indefensible for payments. To reduce how often it matters, I would put the key in the same store as the payment record and claim it in the same transaction, so the key and the effect commit or fail together and there is no separate dependency to lose. That is strictly better than a separate Redis when the backing store supports it.

“Your service returns 200 with an error message in the body. What’s wrong with that?” Everything downstream believes it succeeded. Load balancers will not fail over, CDNs will cache the error as a valid response, client retry libraries will not retry, and every monitoring dashboard will show a zero error rate while customers cannot pay. The status code is the only part of the response that intermediaries understand — the body is opaque to all of them — so lying in the status code blinds the entire operational stack. It also forces every client to parse a body just to discover whether the call worked, which is the exact job the status line exists to do.

A model answer

“First I’d separate three cases, because they need different responses. The request can succeed with a 2xx. It can fail *with* a response — a 4xx, meaning my request was wrong and retrying will fail identically, or a 5xx, meaning the server broke and retrying might work. Or there can be no response at all: a timeout or a dropped connection.

The third one is the dangerous case, and it is the one in your question. A timeout tells me nothing about what happened at the other end. The request may never have arrived — or it may have arrived, been fully processed, the money moved, and only the response lost on the way back. From the client those two are indistinguishable.

So if I blindly retry a payment, and the original had actually succeeded, I charge the customer twice. `POST /charges` is not idempotent — two calls create two charges — so retrying it is genuinely unsafe, unlike a `GET` or a `PUT`.

The fix is an idempotency key. The client generates a unique value — a UUID — **before the first attempt**, and sends the identical value on every retry of that same logical payment. The server then promises: the first request with this key is executed, and any later request with the same key returns the stored response without doing the work again.

Concretely, on the server: claim the key with an atomic insert — `INSERT ... ON CONFLICT DO NOTHING`, or a Redis `SET NX`. If I claimed it, mark it in-flight, do the charge, and store the final status and body against the key. If the key already exists and is complete, return the stored response verbatim. If it exists and is still in-flight, return 409 so the client waits rather than racing. And I’d store a hash of the request body with the key, so that reusing a key for a *different* request gets a 422 — that is a client bug I want to surface loudly rather than silently deduplicate.

The atomicity is the part that is easy to get wrong. A read-then-write has a race where two simultaneous retries both see ‘not present’ and both charge. It has to be a single atomic claim.

Ideally I’d store the key in the same database as the payment and claim it inside the same transaction, so the key and the charge commit together. That removes a separate dependency from the hot path — otherwise, if the key store is down, I have to decide whether to fail closed or process unprotected, and for payments I’d fail closed every time.

Two more things around it. Retries need exponential backoff with jitter — if ten thousand clients time out simultaneously and all wait exactly one second, they all return in the same instant and turn a blip into an outage. And the honest framing of

the whole area is that exactly-once delivery is not achievable over an unreliable network. What you actually build is at-least-once delivery plus idempotent processing, and to the user that is indistinguishable from exactly-once.”

9 Recall card

- **Three outcomes, not two:** success, failure with an answer, and **no answer** — where you cannot tell whether it happened.
- **4xx do not retry** (except 429 / 408). **5xx and timeouts** may be retried — only if the operation is idempotent.
- **Idempotent = doing it twice equals doing it once.** GET, PUT, DELETE yes. POST no. PATCH depends on the body.
- **Idempotency key:** client generates it once, reuses it on every retry; server claims it atomically and replays the stored response.
- **Never 200 OK with an error inside.** And always back off with jitter, or you build your own outage.

DSA topic: Arrays revision and mock round **System design topic:** Status codes, errors, and idempotency

Code these, in this order

Today is different from the last nine days. These four are not new material — they are the array toolkit, phrased in ways you have not seen. **Treat every one of them as a real round.** Set a timer, say everything out loud, and do not look anything up until you have finished or the time is gone.

The rule for today: **if you did not narrate it, it does not count.** Solving it silently is Sneha's invisible mirror check.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Best Time to Buy and Sell Stock	LeetCode 121 (Easy)	The single-pass habit from day 014 , and whether you ask what happens when prices only fall.
2	Merge Sorted Array	LeetCode 88 (Easy)	The write pointer from day 015 , reversed. Filling from the back is the entire insight.
3	Find All Numbers Disappeared in an Array	LeetCode 448 (Easy)	Whether you can find the $O(1)$ -space trick — using the array's own values as markers — after giving the $O(n)$ -space answer first.
4	Set Matrix Zeroes	LeetCode 73 (Medium)	Day 016 plus a genuine trap: marking as you go destroys the information you still need. The $O(1)$ version uses the first row and column as the marker store.

Run each one as a timed round

Twenty minutes each, and follow the six beats:

```
0-3   restate it, ask the four questions
3-6   one example and its answer, out loud
6-9   the brute force and its cost – say it, do not code it
9-12  the better approach and its cost – then PAUSE as if waiting for a nod
12-17 code it, one sentence per block
17-20 walk your own code through your own example, then the edge cases
```

If you finish early, do not stop. Spend the remaining time answering “*can you do better?*” and “*what would you test?*” out loud, because both are certain to be asked.

The four questions, every time

Before any of the four problems, answer these in under forty seconds:

1. Can I modify the input?
2. Is it sorted?
3. Duplicates? Negatives? Empty?
4. Does the order of the output matter?

For problem 3, question 1 is the whole problem — you are *allowed* to modify the input, and that is what makes the `0(1)` solution possible. For problem 4, question 1 is what makes it hard.

The record-yourself drill

Do problem 2 once with your phone recording the audio. Then listen to it. Nobody enjoys this and everybody learns something from it. Count:

- How many seconds of complete silence are there? Anything over fifteen is a problem.
- Did you state a cost before you started typing?
- Did you say *why* you were filling from the back, or only *that* you were?
- How many times did you say “um, so, basically”?

Do it once at the start of each revision day and once more at the end of the phase. The change over nine weeks is the point.

The narration drill

Take the finished `merge` solution and say one sentence for each of these lines, out loud, as if somebody were watching:

```
write = m + n - 1
i, j = m - 1, n - 1
while j ≥ 0:
    if i ≥ 0 and nums1[i] > nums2[j]:
```

The sentences you want are about *why*, not *what*. “`write` is the last slot” is a description. “`write` starts at the last slot because the largest value must end up there, and that slot is guaranteed empty” is narration.

The cost drill

State the time and space cost of each, from memory, in under a minute total, and **count the loops out loud** rather than naming a class:

1. Reversing an array in place.
2. Rotating an array by `k` with the three-reversal trick.

3. Moving all zeros to the end.
4. Removing duplicates from a sorted array.
5. Inserting at the front of a Python list.
6. Walking an $m \times n$ matrix by column.
7. Rotating an $n \times n$ matrix with transpose-then-reverse.
8. Printing a matrix in spiral order.

Anything you cannot count out loud goes on tomorrow's list.

The status code drill

Give the code for each of these, and one sentence on why it is not the neighbouring code:

1. Fetching a post that does not exist.
2. Fetching the comments of a post that exists but has none.
3. Posting a comment while logged out.
4. Editing somebody else's comment.
5. Posting a comment on a locked post.
6. Posting a comment with an empty body.
7. Posting your two-hundredth comment this minute.
8. The payments service is up but the bank connection is down.
9. A load balancer waited thirty seconds for your backend and gave up.
10. Deleting a comment that was already deleted.

Then answer the one that catches people: **why is number 2 not a 404 ?**

The idempotency drill

Answer these out loud, in two minutes:

1. Which HTTP methods are idempotent, and why is DELETE idempotent even though the second call returns 404 ?
2. Is PATCH idempotent? Give one patch body that is and one that is not.
3. Your payment request times out. Name the two things that could have happened, and say why you cannot tell them apart.
4. Where does the idempotency key come from, and at what moment is it generated?
5. Why can the server not simply deduplicate on a hash of the request body?
6. Two retries with the same key arrive at the same instant. What stops both of them charging?
7. The key store is unavailable. Do you fail the payment or process it? Defend your answer.

Number 5 is the one that separates people who have read about this from people who have thought about it.

The arithmetic drill

Produce these from memory, in under two minutes:

- 50,000 requests a second, 0.1% timing out — how many unknown-outcome requests per day?
- Of those, 1% retried unsafely against a successful original — how many double charges a day?
- A service at 10,000 requests a second fails; every client retries three times with no backoff. What is the new load, and what is it with backoff spread over 15 seconds?
- 50,000 requests a second, 10% of them writes, keys kept 24 hours at 300 bytes — how much key storage?

Then say the sentence those numbers buy you: *“backoff without jitter is how a blip becomes an outage.”*

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two array problems, no hints, talk as you go.* Have somebody pick two from the table above without telling you which, or pick at random. The scoring is not whether you finished. It is: did you restate it, did you ask the four questions, did you state a cost before typing, and did you test with an edge case you named yourself.
 2. *Your client retries a failed payment request. What could go wrong?* Three outcomes first, and the point that a timeout is indistinguishable from a lost response. Then why `POST` is not idempotent. Then the key: generated once by the client, claimed atomically by the server, stored response replayed. Then backoff with jitter. Ninety seconds.
 3. *Walk me through how you'd approach an array problem you have never seen.* The six beats, then the decision table — sorted means two pointers or binary search, a few extreme values means trackers, a shorter version of the array means a write pointer, a pair or a window means two indices. Finish with the four questions and why each one has bitten you.
-

Before you move on

- [] I can name the six beats of a technical round in order, without looking.

- [] I ask the four array questions before writing any code, every single time.
- [] I state the brute force and its cost before proposing anything better.
- [] I pause for agreement before I start typing.
- [] I test with edge cases I named myself, not with the example I was handed.
- [] I can count out loud, not merely name, the cost of every array operation from days 9 to 17.
- [] I can give the right status code for all ten cases in the drill, and say why not the neighbour.
- [] I can explain an idempotency key end to end, including why the claim must be atomic.
- [] I can redraw the three-outcomes diagram from memory, in whatever tool I like.

PART III

Strings

Days 19 to 26 · 8 chapters

Alongside, on the system design track:

APIs: how services talk, Databases from zero

- | | |
|---|--|
| 019 What a string is, and why it is immutable | 024 Substrings versus subsequences: the distinction they test |
| 020 Building strings without the quadratic trap | 025 Pattern matching, the simple way |
| 021 Character counting and frequency maps | 026 Strings revision and mock round |
| 022 Anagrams: the sorting versus counting choice | |
| 023 Palindromes and the two-ends habit | |

What a string is, and why it is immutable

DATA STRUCTURES AND ALGORITHMS

What a string is, and why it is immutable

You can explain why building a string with `+=` inside a loop is a trap in Python.

SYSTEM DESIGN

Authentication and authorisation

You can state the difference in one sentence and give a real example of each.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Why is string concatenation in a loop slow?*
- *What is the difference between authentication and authorisation?*

What a string is, and why it is immutable

1 What this is, and why they ask it

A **string** is a sequence of characters, laid out in order, addressed by position — very nearly the array you met on [day 009](#). The one difference is the whole of today: in Python a string is **immutable**. Once it exists, not a single character in it can ever be changed. Every operation that looks like changing a string in fact builds a brand new one.

That single fact has a consequence that catches almost everybody. Adding one character to a string of length n costs n steps, because the whole thing has to be copied into the new one. Do that inside a loop and you pay $1 + 2 + 3 + \dots + n$ steps to build a string of length n , which is $O(n^2)$ — quadratic. The obvious code is the slow code.

Interviewers ask this for two reasons. It is a fast, reliable filter: a candidate who says “strings are immutable, so `+=` in a loop is quadratic, use a list and `join`” has clearly worked with the language, and one who says “I think it’s a bit slower” has not. And it recurs constantly in real problems — every question that builds an answer character by character has this trap in it, which is why [day 020](#) is entirely about the fix.

2 The story

Vimala and her husband moved into the flat in Whitefield in March, and the one thing missing was a nameplate on the door. There is a small shop near the bus stop with an old man who does engraving — brass plates, plastic ones, a machine on the counter that makes a high whining noise and throws up a fine gold dust.

She went on a Tuesday and gave him her husband’s name. Four inches by ten, brass. He said Thursday, and it was ready on Thursday, and it looked good.

Then her mother-in-law, who lives with them, said quite reasonably that her name should be on it too. So Vimala went back on the Saturday with the plate.

The old man turned it over in his hands and shook his head. He could not add to it. The letters are cut into the metal, and once they are cut they are where they are — he cannot move them up an inch to make room, and squeezing a second name into the space below would look wrong because the spacing is worked out for the whole plate at once. What he does instead is take a fresh plate, and cut both names onto it. The first name gets cut all over again, from the beginning.

Fine, she said. Two hundred rupees, Tuesday.

Three weeks later her son's name had to go on. Third plate. All three names cut from scratch, the first name now being cut for the third time and the second for the second time.

She was standing in the shop when it occurred to her, and she laughed about it later. If she had simply asked everyone in the house at the start and gone once with all three names, it would have been one plate and one job. Instead the old man had cut the same name three times, made three plates, and charged her three times, and she had spent three Saturdays on the bus.

The old man, who has been doing this for forty years, said what he always says to people who come back a second time. Decide the whole thing first. Then come.

3 The idea in plain English

The brass plate is a string. The letters are cut in and cannot be moved. To “change” it you make a new one, and making a new one means cutting **every** letter again, not just the new ones.

What a string actually is

A string is a sequence of characters in order, and you address them by position exactly as you do an array — first position is 0.

```
s = "hello"
s[0]      # 'h'
s[-1]     # 'o'      negative counts from the end
len(s)    # 5
s[1:4]    # 'ell'    from 1 up to but not including 4
```

`s[1:4]` is a **slice** — a new string made of part of the old one. Note *new*: slicing does not give you a view into the original, it gives you a fresh copy of that section. That matters for cost, and §6 counts it.

Immutable, and what it forbids

Immutable means the value cannot be changed after it is created. Not “should not” — cannot. The language refuses:

```
s = "hello"
s[0] = "H"
```

```
TypeError: 'str' object does not support item assignment
```

A list allows exactly this and a string does not, and that is the only real difference between the two for our purposes. A list is a shelf you can rearrange; a string is a cut plate.

So what does this do?

```
s = "hello"
s = s + " world"
```

It does **not** modify the string `"hello"`. It builds a completely new string, `"hello world"`, by copying all five characters of the old one and then the six new ones, and then points the name `s` at the new string. The old `"hello"` is untouched and, if nothing else refers to it, thrown away.

You can watch that happen:

```
a = "hello"
b = a
a = a + " world"
print(a, b)          # hello world hello
```

`b` still says `hello`. If strings were mutable, `b` would have changed too — which is exactly the kind of surprise immutability exists to prevent.

Why the language does it this way

Three reasons, and the first is the one to say in an interview.

Strings can be dictionary keys and set members. A dictionary works by computing a number from the key — its hash — and using that to decide where to store it. If a key could change after it was filed, it would be filed in the wrong place and could never be found again. Immutable things can be hashed safely; mutable things cannot, which is why `{"a": 1}` works and `{[1,2]: 3}` raises `TypeError: unhashable type: 'list'`. Dictionaries are [days 006](#) and [060](#), and they lean on this completely.

Sharing is free and safe. Because nothing can modify a string, two variables can point at the same one with no risk. Python takes advantage: identical short literals in your source often become one object.

```
x = "hello"
y = "hello"
x is y          # True – the same object
```

It removes a whole class of bug. You can pass a string to a function knowing it cannot come back altered.

The cost, which is the actual question

Here is the trap, and it is Vimala's three Saturdays:

```
s = ""
for word in words:
    s = s + word      # a whole new string every time
```

Each turn copies everything built so far. Building a result of length `n` one character at a time copies 1, then 2, then 3, and so on:

$$1 + 2 + 3 + \dots + n = n(n + 1) / 2$$

For `n = 100,000` that is about **five billion character copies** to produce a string of a hundred thousand characters. The work is quadratic in the size of the answer.

The fix, which is the old man's advice

Decide the whole thing first, then make it once. Collect the pieces in a list — which is mutable and *does* append cheaply — and join them at the end:

```
parts = []
for word in words:
    parts.append(word)      # 0(1) amortised, from day 011
result = "".join(parts)    # one pass, one allocation
```

`"".join(parts)` walks the list once to add up the total length, allocates a string of exactly that size, and copies each piece into it once. Every character is copied exactly one time. That is `0(n)`, and §6 shows the measured difference: 36 times faster at a hundred and sixty thousand characters, and the gap widens with size.

The string before the `.join` is the **separator** — `"".join` glues with nothing between, `,`, `.join` puts a comma and a space between each pair. It is a method on the separator, not on the list, which reads oddly the first time and is worth saying out loud once.

An honest note about `+=`

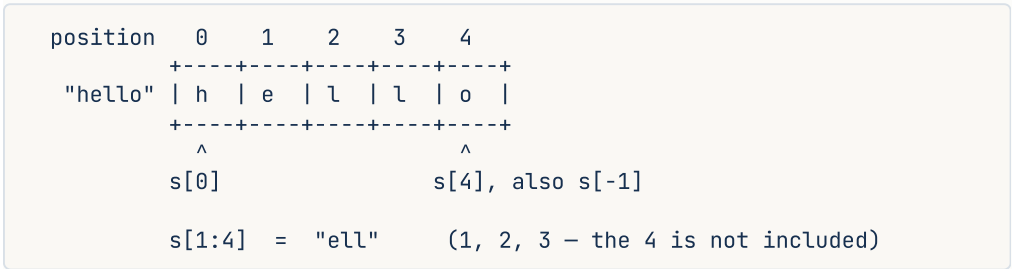
You may benchmark this and find that `s += "x"` in a loop looks linear rather than quadratic. It sometimes is, because CPython has a special case: when the string being extended has no other references, it can occasionally resize it where it stands instead of copying.

Do not rely on this. It disappears the moment anything else refers to the string, it disappears if you build the other way round — `s = "x" + s` — and it does not exist in other Python implementations. The measurement in §6 uses the prepend form for exactly

this reason, and shows the textbook quadratic. **The right answer in an interview is that string building is quadratic and you use `join`**; mentioning the special case afterwards, as a caveat, reads as depth rather than pedantry.

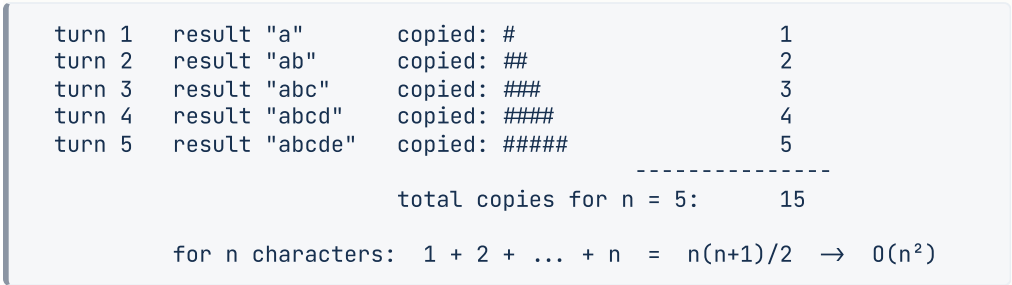
4 The picture

A string is an array of characters with fixed positions:



What to notice: it is exactly the day-009 array picture. Reading `s[3]` is `0(1)` — one jump. The only thing missing is the ability to write into a box.

Now the cost of building one character at a time. Each row is one turn of the loop, and each `#` is one character copied:



Against the `join` version:



What to notice: the triangle in the first picture is the whole problem. Every character in the early part of the string gets copied again on every subsequent turn — the first character is copied `n` times. The second picture has no triangle; it is a rectangle one row deep.

5 The code, built step by step

Reading a string

```
s = "interview"
print(s[0], s[-1], len(s))      # i w 9
print(s[0:4], s[:4], s[5:])    # inte inte view
print(s[::-1])                 # weivretni - the whole string, backwards
```

`s[::-1]` is a slice with a step of `-1`. It is the idiomatic Python way to reverse a string and it is worth knowing cold, because reversing shows up in palindromes on [day 023](#). It builds a new string, so it costs `O(n)` time and `O(n)` space — say that, rather than pretending it is free.

What you cannot do

```
s = "hello"
s[0] = "H"          # TypeError
s.append("!")       # AttributeError - that is a list method
```

To get a modified string you build a new one:

```
s = "H" + s[1:]     # 'Hello'
```

The methods that come up constantly

Every one of these **returns a new string** and leaves the original alone. This is the second most common string bug in interviews.

```
" padded ".strip()      # 'padded' - whitespace off both ends
"a,b,,c".split(",")     # ['a', 'b', '', 'c'] - note the empty piece
"-".join(["a", "b", "c"]) # 'a-b-c'
"Hello".lower()         # 'hello'
"hello world".replace("world", "there") # 'hello there'
"hello".startswith("he") # True
"hello".find("z")       # -1 - not found
"hello".index("z")      # raises ValueError
```

`find` returns `-1` when it fails and `index` raises. Pick deliberately: `find` when absence is normal, `index` when absence is a bug you want to hear about.

Building a string, the wrong way and the right way

The wrong way, written out so you recognise it:

```
def shout(words: list[str]) → str:
    result = ""
    for word in words:
        result = result + word.upper() + " "    # a new string every turn
    return result.strip()
```

The right way:

```
def shout(words: list[str]) → str:
    parts = []
    for word in words:
        parts.append(word.upper())              # cheap append
    return " ".join(parts)                     # one allocation, one copy each
```

Notice a second benefit: the separator is handled by `join`, so there is no trailing space to strip off. The fix made the code shorter as well as faster, which is usually a sign it is the right fix.

The same thing as a comprehension, which is what you would actually write:

```
return " ".join(word.upper() for word in words)
```

When you are genuinely mutating, use a list

If you need to change characters at positions — a cipher, a swap, an in-place-looking transform — convert to a list, work on it, and join back:

```
chars = list("hello")    # ['h', 'e', 'l', 'l', 'o']
chars[0] = "H"
result = "".join(chars)  # 'Hello'
```

`list(s)` is $O(n)$, the changes are $O(1)$ each, and the `join` is $O(n)$. Total $O(n)$, against $O(n^2)$ if you rebuilt the string on every change. **This is the standard answer to “modify a string in place” in Python, and it is worth saying that Python cannot literally do it** — there is no such thing as an in-place string edit — so you convert, edit, and convert back.

The complete, runnable comparison

```
import time

def build_by_concatenation(n: int) → float:
    """Quadratic. Each turn copies everything built so far."""
    s = ""
    start = time.perf_counter()
    for _ in range(n):
        s = "x" + s          # prepend: cannot be optimised away
    return time.perf_counter() - start

def build_by_join(n: int) → float:
    """Linear. Cheap appends, then one allocation and one copy per character."""
    parts: list[str] = []
    start = time.perf_counter()
    for _ in range(n):
        parts.append("x")
    _ = "".join(parts)
    return time.perf_counter() - start

def to_title_case(sentence: str) → str:
    """A small real use: capitalise each word. Builds once, not per word."""
    return " ".join(word[0].upper() + word[1:] for word in sentence.split() if word)

def reverse_words(sentence: str) → str:
    """'the sky is blue' → 'blue is sky the'. LeetCode 151, the easy version."""
    return " ".join(reversed(sentence.split()))

def swap_case_manually(s: str) → str:
    """Character-by-character work: build a list, join once."""
    out: list[str] = []
    for ch in s:
        out.append(ch.lower() if ch.isupper() else ch.upper())
    return "".join(out)

if __name__ == "__main__":
    for n in (20_000, 40_000, 80_000, 160_000):
        concat = build_by_concatenation(n)
        joined = build_by_join(n)
        print(f"n={n:>7}    concat {concat:.4f}s    join {joined:.4f}s    "
              f"ratio {concat / joined:.0f}x")

    print(to_title_case("the sky is blue"))      # The Sky Is Blue
    print(reverse_words("the sky is blue"))      # blue is sky the
    print(swap_case_manually("Hello World"))    # hELLO wORLD
```

Measured on Python 3.12:

n=	20000	concat	0.0054s	join	0.0011s	ratio	5x
n=	40000	concat	0.0208s	join	0.0022s	ratio	9x
n=	80000	concat	0.0790s	join	0.0043s	ratio	18x
n=	160000	concat	0.3062s	join	0.0086s	ratio	36x

Look at the concat column doubling: 0.0054, 0.0208, 0.0790, 0.3062. Each time n doubles, the time goes up roughly **four** times. That is the signature of $O(n^2)$ and it is worth recognising on sight. The join column doubles when n doubles, which is $O(n)$.

6 What it costs

Reading and slicing

- `s[i]` — one jump to a known position. $O(1)$.
- `len(s)` — Python stores the length. $O(1)$.
- `s[a:b]` — allocates and copies `b - a` characters. $O(k)$ time and $O(k)$ space where `k` is the slice length. `s[::-1]` is therefore $O(n)$ in both.
- `s1 == s2` — compares character by character, stopping at the first difference. $O(n)$ worst case, and worst case is when the strings are equal, which is the common case in a dictionary lookup.
- `"abc" in s` — the naive scan is $O(n \times m)$ for a haystack of `n` and needle of `m`. Python's actual implementation is cleverer, and [day 025](#) covers what interviewers expect you to say here.

Building, counted properly

Concatenation in a loop. On turn `k`, the result has `k` characters, and building the next one copies all `k` plus the new piece. Adding those up over `n` turns:

$$1 + 2 + 3 + \dots + n = n(n + 1) / 2 \approx n^2 / 2$$

At `n = 100,000`: `100,000 × 100,001 / 2` $\approx$ **5 billion** character copies. $O(n^2)$ time.

List and join. `n` appends, each $O(1)$ amortised — from [day 011](#), the list occasionally grows and copies, but spread over all the appends it averages to constant. Then `join` does one pass to total the lengths and one pass to copy, so $2n$ steps. Total `n + 2n = 3n`: $O(n)$ time, and $O(n)$ extra space for the list of pieces.

The ratio. $n^2/2$ against $3n$ is $n/6$. At `n = 100,000` that predicts about 16,000 times fewer operations. The measured ratio is smaller — $36\times$ at `n = 160,000` — because a character copy inside CPython's optimised memory move is very much cheaper than a Python-level loop turn. The *shape* of the growth is what matters and it matches: four times the time for twice the input.

The number to have ready

Building a 100,000-character string with `+=` is about 5 billion character copies. With a list and `join` it is about 300,000 operations. That is the difference between a visible pause and no pause at all, and it comes entirely from strings being immutable.

Space

Every operation that “changes” a string allocates a new one. A chain of five transformations on a one-megabyte string — `strip`, `lower`, `replace`, `split`, `join` — creates several megabyte-sized intermediates. Usually irrelevant; occasionally the reason a process runs out of memory on a large file. If it matters, the answer is to work line by line rather than loading the whole thing.

7 The traps

The real error: trying to assign into a string

```
s = "hello"
s[0] = "H"
```

```
TypeError: 'str' object does not support item assignment
```

The message is precise, and recognising it instantly saves you thirty seconds in an interview. The fix is either `s = "H" + s[1:]`, or convert to a list if you are making several changes.

The real error: reaching for a list method

```
s = "hello"
s.append("!")
```

```
AttributeError: 'str' object has no attribute 'append'
```

There is no `append`, no `insert`, no `remove`, no `sort` on a string, because all four would modify it. `sorted(s)` exists and returns a **list** of characters, which surprises people: `sorted("cab")` is `['a', 'b', 'c']`, not `"abc"`. You need `"".join(sorted(s))` — and that idiom is the whole of the anagram question on [day 022](#).

The near-miss: expecting a method to modify in place

```
s = "hello world"
s.replace("world", "there")
print(s)
```

```
hello world
```

Nothing changed, and there is no error to tell you. `replace` returned a new string and you threw it away. Every string method behaves like this — `strip`, `lower`, `upper`, `replace`, `title` — and the fix is always `s = s.replace(...)`.

This is the single most common string bug for people coming from languages where such methods mutate, and because it fails silently it can survive a long time in real code.

The near-miss: `is` instead of `==`

```
a = "hello"
b = "hello"
print(a is b)                                # True

c = "hello"
d = "".join(["hel", "lo"])
print(c == d)                                # True
print(c is d)                                # False
```

`==` compares the characters. `is` asks whether they are the same object. Identical literals written in your source often end up as one shared object, so `is` appears to work — right up until one of the strings is built at run time, at which point it silently returns `False`. **Never compare strings with `is`**. Use `==`, always.

The trap: benchmarking `+=` and concluding it is fine

```
s = ""
for _ in range(100_000):
    s += "x"          # may look linear
```

CPython can sometimes extend such a string in place when nothing else refers to it, which makes a naive benchmark of exactly this form look fast. Change it to `s = "x" + s`, or keep a second reference to `s`, or run it under a different Python implementation, and the quadratic behaviour is right there. Relying on an optimisation that vanishes when you touch the code is not a plan.

The near-miss: `split()` versus `split(" ")`

```
"a b".split()          # ['a', 'b']          - splits on runs of whitespace
"a b".split(" ")       # ['a', ' ', 'b']     - splits on each single space
```

With no argument, `split` treats any run of whitespace as one separator and ignores leading and trailing whitespace. With an explicit separator it does not. Nearly every word-counting bug in interviews is this one line, so use bare `split()` unless you specifically want the empty pieces.

8 In the interview

How it gets asked

- “Why is string concatenation in a loop slow?” — the direct version. They want “immutable”, “quadratic”, and “use join”, in that order.
- “Are strings mutable in Python? What about in Java, or C?” — the comparison version. Python and Java: immutable. C: a mutable array of bytes. Java has `StringBuilder` for exactly the same reason Python has `join`.
- “Reverse this string / check if it’s a palindrome.” — where the immutability decides your approach without being mentioned.
- “Why can a string be a dictionary key but a list can’t?” — the deeper version, and the one that shows whether you know why immutability exists rather than just that it does.

What to say out loud, in the first ninety seconds

1. **Lead with the property, not the symptom.** “Because strings are immutable in Python. You can never modify one, so anything that looks like modifying builds a whole new string.”
2. **Turn that into the cost, with the counting.** “So `s = s + x` copies every character of `s` into a new string. Inside a loop, turn one copies one character, turn two copies two, and so on, so building a result of length n costs $1 + 2 + \dots + n$, which is about $n^2/2$. Quadratic in the size of the output.”
3. **Give a number.** “For a hundred thousand characters that is around five billion character copies.”
4. **Name the fix and why it is linear.** “Collect the pieces in a list — appends are $O(1)$ amortised — and call `join` once at the end. `join` totals the lengths, allocates exactly once, and copies each character exactly once. $O(n)$.”
5. **Add the general rule.** “And if I need to change characters at positions, I convert to a list, mutate, and join back — that is $O(n)$ overall instead of $O(n^2)$.”

6. Say why the language does it. *“The reason strings are immutable is mostly hashing: a dictionary decides where to file a key from its contents, so a key that could change afterwards could never be found again. It also makes sharing safe.”*

The follow-ups

“Why does it matter for dictionaries?” A dictionary stores a key by computing a number from its contents and using that number to choose a slot. If the contents could change afterwards, the key would still be sitting in the slot chosen from its old contents, and every future lookup — which computes the number from the *new* contents — would look in a different slot and find nothing. So the key would effectively vanish from the dictionary while still being in it. Python prevents that by requiring keys to be hashable, and only immutable things are hashable. That is exactly why `{"a": 1}` works and a list key raises `TypeError: unhashable type: 'list'`, and why tuples can be keys but lists cannot.

“How does Java handle this?” Identically in the important respects. Java strings are immutable for the same reasons, and `+` in a loop has the same quadratic problem — which is why Java ships `StringBuilder`, a mutable buffer you append to and convert to a string once at the end. It is the same idea as Python’s list-and-join: accumulate in something cheap to extend, materialise the immutable result once. In C a string is just an array of bytes and is mutable, which is faster and is also why C has a long history of buffer overflow bugs. That trade — safety and hashability against in-place speed — is the actual answer to the question.

“So how do you modify a string in Python?” You do not, strictly. You build a new one. For a single change, slicing and concatenating is fine: `s = s[:i] + new_char + s[i+1:]`, which is $O(n)$. For many changes, do it once: `chars = list(s)`, mutate positions freely at $O(1)$ each, then `"".join(chars)`. That is $O(n)$ overall rather than $O(n)$ per edit. The list-then-join pattern is the standard answer to any interview question phrased as “modify the string in place”, and it is worth saying explicitly that Python has no true in-place string edit so this is the honest equivalent.

“What’s the cost of slicing?” $O(k)$ time and $O(k)$ space for a slice of length `k`, because it allocates and copies — it is not a view into the original. That is worth watching in loops: a function that takes `s[1:]` and recurses looks elegant and is secretly $O(n^2)$, because each level copies the rest of the string. The fix is to pass the original string plus an index instead of a slice. Some languages do give you cheap views — Go’s slices, Rust’s `&str` — and Python’s `memoryview` does it for bytes, but plain string slicing copies.

A model answer

“It’s slow because Python strings are immutable. You can’t change a string once it exists, so `s = s + x` doesn’t extend `s` — it allocates a brand new string, copies every character of the old one into it, then copies `x` on the end, and points the name at the new object.

Inside a loop that compounds. On the first turn you copy one character, on the second you copy two, on the third three, and so on. Building a result of length n costs $1 + 2 + \dots + n$ character copies, which is $n(n+1)/2$ — so it’s $O(n^2)$ in the length of the output, even though the output only has n characters in it. For a hundred thousand characters that’s about five billion copies to produce a hundred thousand characters.

The fix is to accumulate in something mutable and materialise once. Append the pieces to a list — that’s $O(1)$ amortised per append — and then call `"".join(parts)` at the end. `join` walks the list once to add up the total length, allocates a string of exactly that size, and copies each piece in once. Every character is copied exactly one time, so it’s $O(n)$.

```
parts = []
for word in words:
    parts.append(word)
result = "".join(parts)
```

I measured it: building 160,000 characters by concatenation took 0.31 seconds, and the list-and-join version took 0.0086 — about 36 times faster. And the shape is the giveaway: as I doubled n , the concatenation time went up four times each doubling, which is the signature of quadratic growth, while the join time just doubled.

The same idea covers modifying characters. There’s no in-place string edit in Python, so if I need to change several positions I do `chars = list(s)`, mutate freely, and `"".join(chars)` at the end. That’s $O(n)$ overall instead of $O(n)$ per edit.

One honest caveat: if you benchmark exactly `s += 'x'` in CPython you may see linear behaviour, because there’s a special case that can extend the buffer in place when nothing else references the string. It disappears the moment you keep a second reference, or build the string the other way round, or use a different implementation — so it’s not something to design around. The rule stands: accumulate in a list, join once.

And the reason for the immutability in the first place is mostly hashing. A dictionary picks a slot for a key based on its contents. If the contents could change afterwards, the key would be sitting in the wrong slot and could never be found again. So

Python requires keys to be hashable, and only immutable objects are — which is why strings and tuples work as keys and lists don't.”

9 Recall card

- **A string is an immutable array of characters.** `s[i]` is `0(1)`; `s[i] = c` is a `TypeError`.
- **Every “change” builds a new string.** `+=` in a loop is `1 + 2 + ... + n` copies — $O(n^2)$.
- **Build with a list and `"".join(parts)` — $O(n)$.** To edit characters: `list(s)`, mutate, `"".join`.
- **Every string method returns a new string.** `s.replace(...)` alone does nothing; you need `s = s.replace(...)`.
- **Immutable so it can be hashed** — that is why a string can be a dictionary key and a list cannot. Compare with `=`, never `is`.

1 What this is, and why they ask it

Authentication answers *who are you?* **Authorisation** answers *what are you allowed to do?* They are two separate checks, they happen in that order, and they fail in different ways — which is exactly why `401` and `403` are two different status codes.

The words are so close that people use them interchangeably, and doing so in an interview is a small, immediate tell. They are usually shortened to **authn** and **authz** for precisely this reason.

It gets asked in three places. As a definition question in the first ten minutes of a design round, where a crisp one-sentence answer plus a real example takes twenty seconds and buys goodwill. As a required component of almost any design — every system with users has both, and forgetting to mention them is a gap the interviewer will note. And as a probe with real depth: “*how do you store passwords?*” has a right answer and several catastrophically wrong ones, and it is one of the very few interview questions where a wrong answer is genuinely disqualifying for a backend role.

Today is the two checks and how each is really done. The token mechanics — sessions, JWT, OAuth — are [day 020](#).

2 The story

Suhas is flying to Delhi at ten past six in the morning, which means leaving the house at half past three, which means he has not really slept.

At the terminal door there is a jawan with a torch, and a queue of about twenty people shuffling forward. When Suhas gets to the front he holds up his phone with the ticket on it in one hand and his driving licence in the other. The jawan looks at the name on one, then the name on the other, then at Suhas’s face, and waves him through. Eleven seconds, and it is the only time all morning that anybody looks at his licence.

Inside, he drops his bag and gets a pass on his phone — a code in a box, his name, the flight number, seat 22C, gate 41. From that moment on, that little code is him. Nobody asks for the licence again. Security scans the code. The man at the gate scans the code. The girl at the top of the stairs looks at 22C and points left.

On the way to the gate he passes the lounge, and because it is quarter to five and he is tired he walks up to it. The woman at the desk scans his code, looks at her screen and says, politely, that this is an economy ticket and the lounge is for business class and card holders.

That is the bit worth noticing. She was not confused about who he was. His name was on her screen. She could see his flight, his seat, everything. She knew exactly who he was and the answer was still no — and those are two completely different sentences, said by two different people, at two different doors.

Two more small things. The pass is only good for that flight, that morning. And when his colleague messages later asking him to send a screenshot of the boarding pass so she can see the gate number, Suhas sends it, and then thinks about it for a second — because the thing on his phone does not know who is holding it. Anyone with that code, and nobody at the second door checking a licence, would be treated as him.

3 The idea in plain English

Two doors, two questions.

- The jawan asks **who are you?** He compares something you are carrying against something you claim, and lets you into the building. That is **authentication**.
- The woman at the lounge asks **what are you allowed to do?** She already knows who you are. She is checking your ticket class against a rule. That is **authorisation**.

The one-sentence version, which is what you say in an interview:

Authentication is proving who you are. Authorisation is deciding what you may do. You authenticate once and you are authorised on every single request.

That last clause is the part that separates a memorised answer from an understood one. Suhas showed his licence once; his pass was scanned five times.

The order, and the two failures

Authentication first, always. You cannot decide what somebody may do before you know who they are.

Which is why the two failures are different status codes, from [day 018](#):

- **401 Unauthorized** — I do not know who you are. Send credentials. (Badly named: it means *unauthenticated*.)

- **403 Forbidden** — I know exactly who you are, and no. Sending credentials again will not help.

The lounge is a **403**. Turning up at the terminal with no licence at all is a **401**.

How you prove who you are

Authentication rests on evidence, and there are only three kinds of it:

FACTOR	MEANING	EXAMPLES
Something you know	a secret in your head	password, PIN
Something you have	a physical object	phone, security key, the OTP sent to your SIM
Something you are	a property of your body	fingerprint, face

Multi-factor authentication (MFA) means requiring evidence of two *different* kinds. A password plus a security question is not MFA — both are things you know, and both leak in the same breach. A password plus a code from your phone is, because stealing the database does not put the attacker's hand on your phone.

Credentials once, then a token

Suhas showed his licence once and carried a code afterwards. Systems do the same, for two reasons: sending a password on every request means it is exposed on every request, and checking a password properly is deliberately slow (§5 explains why).

So the flow is:

1. The user sends username and password **once**, over HTTPS, to a login endpoint.
2. The server verifies them and issues a **token** — a session id or a signed token.
3. Every later request carries the token in the **Authorization** header.
4. The server verifies the token — fast — and now knows who is calling.

The token is a **bearer token**: whoever bears it is treated as the user. That is the screenshot problem. It is why tokens must travel only over HTTPS, must expire, and must be revocable. The mechanics are [day 020](#).

How you decide what somebody may do

Authorisation is a rule, evaluated per request. There are four common shapes, in rising order of power and complexity.

Ownership. The simplest and by far the most common: *you may edit this comment if you wrote it*. One comparison, `comment.author_id == current_user.id`. Most product features need nothing more.

Access control list (ACL). A list attached to each object saying who may do what. This is how a shared document works — “Priya can edit, Ravi can view”. Precise, and it grows: a million documents with ten entries each is ten million rows to store and check.

Role-based access control (RBAC). Users get roles; roles carry permissions. *Admins may delete any comment. Moderators may hide one. Members may write one.* This is what most companies run on, because it matches how organisations actually think, and adding a person to a team grants the whole set at once.

Attribute-based access control (ABAC). The decision is computed from attributes of the user, the object and the context. *A doctor may read a patient record if they are on that patient’s care team, during their shift, from a hospital network.* Maximum expressiveness, and much harder to reason about — you can no longer answer “who can see this?” by reading a list.

Suhas’s lounge is RBAC: his ticket class is his role, and the lounge has a rule about roles.

The one rule that matters more than the model

The check happens on the server, on every request, without exception. Hiding the delete button in the interface is a courtesy to the user, not a security control — anyone can send the request directly. If the only thing stopping a user from deleting somebody else’s comment is that the button is not shown, you have no authorisation at all.

The most common real-world failure of this is called **insecure direct object reference**: the endpoint checks that you are logged in, fetches `/orders/1055`, and never checks that order 1055 is yours. Changing the number in the address then shows you someone else’s order. It is authentication without authorisation, and it is one of the most frequently exploited flaws on the web.

4 The picture

The two doors:

AUTHENTICATION

"who are you?"

happens ONCE, at login
checks a secret you supplied
fails with 401
answer: an identity
the jawan at the door

|
v

+-----+ token
| login once | -----> |
+-----+

AUTHORISATION

"what may you do?"

happens on EVERY request
checks a rule about you
fails with 403
answer: yes or no
the woman at the lounge

|
v

+-----+
| every request from now on |
+-----+

What to notice: the arrow is one-way and crosses once. Everything to the right of it happens thousands of times per user. That asymmetry is why password checking is allowed to be slow and token checking must be fast.

The full flow of one authenticated, authorised request:

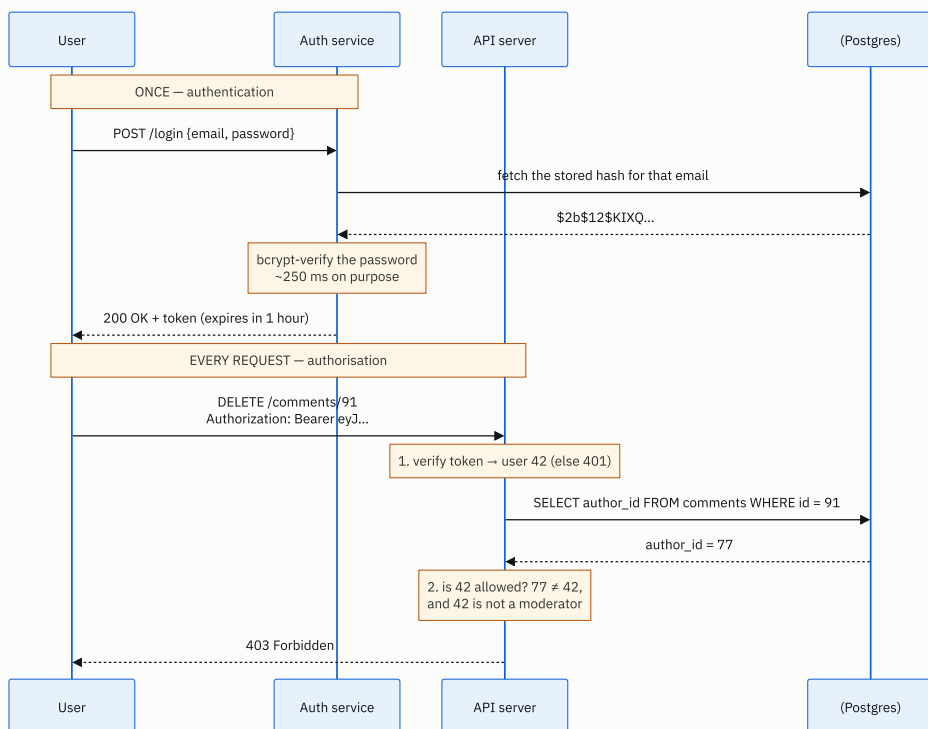


FIGURE 19.1

What to notice: two numbered checks inside the server, and they can fail independently. A missing or expired token stops you at step 1 with a `401`. A valid token belonging to the wrong person stops you at step 2 with a `403`. Skipping step 2 entirely is the insecure-direct-object-reference bug, and the request would have succeeded.

5 How it actually works

Storing passwords — the part with a wrong answer

This is the highest-stakes question in the topic. The rules, in order of severity:

Never store the password. Not encrypted either — encryption is reversible, so whoever holds the key holds every password.

Store a hash. A hash turns the password into a fixed-size value that cannot be reversed. On login you hash what was typed and compare with what is stored.

Never a fast hash. MD5, SHA-1 and plain SHA-256 are designed to be *fast*, which is precisely wrong here. A modern GPU computes billions of SHA-256 hashes a second, so an eight-character password falls in minutes.

Use a slow hash designed for passwords: bcrypt, scrypt, or Argon2id. These are deliberately expensive and have a tunable **work factor**. Argon2id is the current recommendation; bcrypt is everywhere and fine.

Salt every password. A **salt** is a random value stored alongside the hash and mixed in before hashing, so two users with the same password get different hashes. Without it, an attacker precomputes hashes of common passwords once — a **rainbow table** — and cracks every matching account at once. bcrypt and Argon2 generate and embed the salt for you; that is one reason to use them rather than assembling something yourself.

A stored bcrypt hash looks like this, with the parts labelled:

```
$2b$12$LQv3c1yqBWVHxkd0LHAKC0Yz6TtxMQJqhN8/LewKyPHi4Yk8Ck0Ka
|   |   |                               |
|   |   +-- 22-char salt                +-- the hash itself
|   +----- cost factor 12 = 2^12 = 4096 rounds
+----- algorithm: bcrypt
```

Compare in constant time. Comparing hashes with `=` can leak information through how long the comparison takes. Use `hmac.compare_digest` in Python, or the library's own `verify` function, which does this for you.

Never write it yourself. Use `bcrypt`, `argon2-cffi`, or your framework's built-in — Django, Rails and Spring Security all ship correct implementations.

The login flow, concretely

1. `POST /login` with email and password, over HTTPS. Never `GET`, because addresses land in logs and browser history.
2. Look up the user. **If the email does not exist, still perform a hash comparison against a dummy value** before failing, so the response time does not reveal which emails are registered.
3. Verify with the library's function.
4. On failure, return `401` with a deliberately vague message — *"invalid email or password"*, never *"no account with that email"*.
5. Rate limit by IP and by account. Five attempts a minute stops credential stuffing dead.
6. On success, issue a token with an expiry and return it.

Sessions or tokens

Two ways to carry identity after login, covered properly on [day 020](#):

- **Server-side session.** The server stores a record and gives the client an opaque id in a cookie. Every request looks it up — in Redis, typically. Revoking is instant: delete the record.
- **Self-contained token (JWT).** The token carries the identity and expiry and is signed, so any server can verify it with a key and no lookup. Faster and stateless; revoking before expiry is the hard part.

Cookie flags, if you use cookies, and these get asked: `HttpOnly` so JavaScript cannot read the cookie, which limits the damage of a script-injection flaw; `Secure` so it is only ever sent over HTTPS; and `SameSite=Lax` or `Strict` so another site cannot cause the browser to send it, which is the defence against cross-site request forgery.

Implementing authorisation

The RBAC data model is three tables and worth being able to sketch:

```
users —< user_roles >— roles —< role_permissions >— permissions
                        e.g. "comment:delete", "post:publish"
```

A permission check becomes: *does any role of this user grant this permission?* Cache the resolved permission set on the session or token so it is not a database round trip on every request — and accept that a permission revoked mid-session takes effect only when the cache expires, which is a trade you should state.

For ownership checks — the common case — do it in the query rather than after it:

```
-- good: the check is the query, so a wrong id returns nothing
UPDATE comments SET body = $1 WHERE id = $2 AND author_id = $3;

-- risky: fetch, then remember to check
SELECT * FROM comments WHERE id = $2;
```

The first form cannot be forgotten. The second relies on the next developer remembering, and one day they will not.

Real systems to name

AWS IAM is the reference implementation of policy-based authorisation, and it is worth knowing its shape: a policy is a JSON document listing principal, action, resource and condition, with an explicit deny always winning. **Open Policy Agent** is the open-source equivalent for your own services. **Auth0**, **Okta**, **Keycloak** and **AWS Cognito** are hosted identity providers you would use rather than building login yourself. **Google Zanzibar** is the paper behind relationship-based authorisation at Google Docs scale, and naming it signals you have read beyond the basics.

6 The numbers

Why password hashing is deliberately slow

bcrypt at cost factor 12 takes roughly **250 ms** on a modern core. That is enormous by server standards, and it is the point.

For the attacker. With a stolen database:

```
plain SHA-256 on a good GPU : ~10,000,000,000 guesses/second
bcrypt cost 12                : ~4 guesses/second per core
```

That is a factor of about **2.5 billion**. A password that falls to SHA-256 in one second holds out against bcrypt for roughly eighty years. This single comparison is the entire argument, and it is worth having the two numbers ready.

For you. 250 ms per login limits how many logins one core can serve:

```
1 core      = 1 ÷ 0.25 = 4 logins/second
16 cores    = 64 logins/second
```

So a service with 10 million users where 5% log in on a given day, concentrated into the busiest hour:


```
10,000,000 × 0.05      = 500,000 logins/day
peak hour ≈ 25% of them = 125,000 logins in 3,600 s ≈ 35 logins/second
35 ÷ 4                  ≈ 9 cores busy on hashing alone
```

Nine cores doing nothing but bcrypt. That is affordable and it is why login is usually its own service with its own scaling — and why you must never put a 250 ms operation on a path that runs on every request. **Authentication is expensive and rare; authorisation is cheap and constant.**

Choosing the work factor

The standard rule: pick the highest cost your hardware can serve at peak, targeting 200–500 ms. Because bcrypt’s cost is a power of two, each increment doubles the work:

```
cost 10 ≈ 60 ms      cost 12 ≈ 250 ms      cost 14 ≈ 1,000 ms
```

Hardware gets faster, so the number has to rise over time — it was 8 in 2000 and 12 or 13 today. Re-hash on next successful login when you raise it.

Session storage

10 million users, 20% with a live session, about 200 bytes each:

```
10,000,000 × 0.20 × 200 bytes = 400 MB
```

400 MB in Redis. Trivial — which is a useful counter to the claim that stateless tokens are necessary for scale. They are chosen for latency and independence, not because sessions do not fit.

The cost of the permission check

An RBAC lookup hitting the database on every request, at 3,700 requests a second and 1 ms per query:

```
3,700 × 1 ms = 3.7 seconds of database time per second
```

Which means at least four cores of your database doing nothing but permission checks. Caching the resolved permissions on the session for 5 minutes cuts it to near zero, at the cost of a revocation taking up to 5 minutes to bite. That is the trade, stated with numbers.

7 The trade-offs

Session or token

Server-side sessions give instant revocation and small requests, and cost one lookup per request plus a store that every server must reach. Self-contained tokens need no lookup and no shared store, and in exchange you cannot revoke one before it expires without reintroducing exactly the shared state you removed. The usual resolution is both: short-lived tokens for ordinary requests plus a server-side record for refresh and revocation. Full treatment tomorrow.

RBAC or ABAC

RBAC is comprehensible. You can answer “who can delete a comment?” by reading a table, and that matters enormously during an audit or an incident. It also gets coarse — you end up inventing `moderator_south_region_readonly` roles, which is the smell that your model has run out. ABAC expresses those rules directly and makes the system much harder to reason about, because the answer to “who can see this?” is now a computation over attributes rather than a list. **Start with RBAC plus ownership checks, and reach for ABAC only when the rules genuinely depend on context.**

How strict to make MFA

MFA stops essentially all credential-stuffing attacks, and it also costs you users at signup and generates support load when people change phones. The usual compromise is risk-based: require the second factor for new devices, unusual locations and sensitive operations, and not for routine logins. And SMS is the weakest second factor — SIM swapping is a real, common attack — so prefer an authenticator app or a hardware key where the stakes justify it.

Build or buy

Building login yourself means owning password storage, reset flows, MFA, rate limiting, session management and every future protocol change. Using Auth0, Cognito or Keycloak means paying per user and depending on a third party for the ability to log in at all. **For most teams, buy.** The one strong reason to build is a genuinely unusual identity model that a provider cannot express.

The sentence that separates candidates

I would not build my own authentication if the product does not sell identity. Password storage, reset tokens, MFA enrolment, session revocation and account recovery each have a way to get them subtly wrong, and the failure mode is a breach rather than a bug report. I would buy the authentication and write the authorisation myself — because authorisation encodes what my product actually means, and nobody else can express that for me.

8 In the interview

How it gets asked

- *“What is the difference between authentication and authorisation?”* — the definition, in the first ten minutes. Twenty seconds, with an example.
- *“How would you store passwords?”* — the one where a wrong answer genuinely costs you the role.
- *“When would you return 401 versus 403?”* — the same idea in status-code form.
- *“How would you make sure a user can only see their own orders?”* — the applied version, and the one where the insecure-direct-object-reference answer earns real credit.

What to say out loud, in the first ninety seconds

1. **The one-sentence definition.** *“Authentication is proving who you are. Authorisation is deciding what you’re allowed to do.”*
2. **The asymmetry, immediately.** *“You authenticate once, at login. You’re authorised on every single request after that.”*
3. **A concrete example.** *“At an airport, showing your ID at the door is authentication. Being turned away from the business lounge with an economy ticket is authorisation — they know exactly who you are, the answer is still no.”*
4. **Tie it to the codes.** *“That’s why 401 and 403 are different. 401 means I don’t know who you are, despite the name — it really means unauthenticated. 403 means I do know, and no.”*
5. **Say how identity is carried.** *“Credentials go over the wire once, and after that the client carries a token. Password checking is deliberately expensive, so you can’t do it per request.”*
6. **Name your authorisation model.** *“For most features it’s an ownership check — you may edit this because you wrote it. Beyond that, roles and permissions. I’d only go to attribute-based rules when the decision genuinely depends on context.”*

7.State the rule that matters. *“And the check is always on the server, on every request. Hiding a button is a courtesy, not a control.”*

The follow-ups

“How do you store passwords?” Never in plain text and never encrypted, because encryption is reversible and whoever holds the key holds every password. Store a hash, and specifically a slow hash designed for passwords — Argon2id by preference, bcrypt if that is what the stack has. Not MD5, SHA-1 or plain SHA-256: those are built to be fast, and a GPU does billions of SHA-256 guesses a second while bcrypt at cost 12 does about four per core. Each password gets its own random salt, stored alongside the hash, so identical passwords produce different hashes and precomputed rainbow tables are useless — bcrypt and Argon2 handle the salt for you. Tune the work factor to about 250 ms on your hardware, raise it over the years, and re-hash on next login when you do. Compare using the library’s verify function so the comparison is constant-time. And I would not implement any of this by hand.

“When do you return 401 and when 403?” 401 when I cannot establish who is calling — no credentials, a malformed token, an expired token. The correct response to a 401 is to authenticate and try again, and the response should carry a WWW-Authenticate header saying how. 403 when I know exactly who is calling and they are not permitted — retrying with the same identity will never work. One genuine subtlety: if the mere existence of the resource is sensitive, such as a private repository, returning 403 confirms it exists, so many APIs deliberately return 404 instead to avoid leaking that. For public resources 403 is the honest answer.

“How do you make sure a user only sees their own orders?” The check goes in the query, not after it: `SELECT * FROM orders WHERE id = $1 AND user_id = $2`, so another user’s id simply returns nothing. The version I would avoid is fetching by id and then checking ownership in code, because that relies on every developer remembering every time, and the day someone forgets you have an insecure direct object reference — the endpoint verifies you are logged in but not that the object is yours, so changing the number in the URL shows someone else’s order. It is one of the most exploited flaws on the web precisely because the code looks fine. I would also use non-sequential identifiers such as UUIDs, which does not fix the flaw but stops an attacker enumerating every order by counting.

“Where does OAuth fit into this?” OAuth 2.0 is an **authorisation** framework, despite being what people mean when they say “log in with Google”. Its actual job is delegated access: letting one application act on a user’s behalf at another service, with a limited scope, without ever seeing the password. The authentication layer on top is OpenID

Connect, which adds an identity token saying who the user is. That distinction gets asked, and getting it right — OAuth is authz, OIDC is authn — is a good signal. The mechanics are tomorrow's lesson.

A model answer

“Authentication is proving who you are. Authorisation is deciding what you're allowed to do. They are two different checks and they happen in that order — you can't decide what someone may do before you know who they are.

The example I like is an airport. At the terminal door a guard compares your ID against your ticket and your face. That's authentication, and it happens once. Later you walk up to the business lounge and they scan your pass and turn you away, because you're in economy. That's authorisation. They know exactly who you are — your name is on their screen — and the answer is still no. Two different questions, two different doors.

That's exactly why HTTP has two codes. 401 means I don't know who you are — it's badly named, it really means unauthenticated, and the fix is to send credentials. 403 means I know precisely who you are and you may not do this, so sending credentials again is pointless.

The other thing worth saying is the asymmetry. You authenticate once and you're authorised on every request. That matters practically: verifying a password should be deliberately slow — bcrypt or Argon2 at around 250 milliseconds, which is what makes a stolen database useless to an attacker — and you obviously cannot afford 250 milliseconds on every request. So the login exchanges the password for a token, and every subsequent request carries the token, which is cheap to verify.

For the authorisation itself, most features need only an ownership check: you may edit this comment because you wrote it. Beyond that I'd use roles and permissions, because you can answer 'who can delete a comment?' by reading a table, which matters during an audit. I'd only reach for attribute-based rules — where the decision is computed from context like time, location or team membership — when the requirements genuinely need them, because they're much harder to reason about.

And the rule underneath all of it: the check happens on the server, on every request. Hiding the delete button is a courtesy to the user, not a security control. The classic failure is an endpoint that verifies you're logged in but never verifies the object belongs to you — you change the id in the URL and read someone else's order. I'd prevent that by putting the ownership condition into the query itself, so it cannot be forgotten.”

9 Recall card

- **Authn = who are you. Authz = what may you do.** Authn once, authz on every request.
- **401 = I don't know you** (unauthenticated). **403 = I know you, and no.**
- **Passwords: salted Argon2id or bcrypt at ~250 ms.** Never plaintext, never encrypted, never MD5/SHA-256.
- **Authorisation models:** ownership → ACL → RBAC → ABAC. Start with ownership plus roles.
- **The check is server-side, every request, in the query where possible.** Hiding the button is not a control.

DSA topic: What a string is, and why it is immutable **System design topic:** Authentication and authorisation

Code these, in this order

Four problems that all turn on the same fact: you cannot change a string, so you decide what the whole answer is and build it once.

Before each one, say out loud:

1. Am I building a result, or only reading? If building, where does the `join` go?
2. How long is the answer, and how many times will each character be copied?
3. Does the problem want a string back, or a list, or a count?
4. What happens on the empty string?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Reverse String	LeetCode 344 (Easy)	That the input is given as a list of characters , not a string — precisely because a string cannot be reversed in place. Ask yourself why the problem is phrased that way.
2	Reverse Words in a String III	LeetCode 557 (Easy)	<code>split</code> , transform, <code>join</code> . Anyone who builds the result with <code>+=</code> inside two nested loops has not learned today's lesson.
3	Longest Common Prefix	LeetCode 14 (Easy)	Building the answer once rather than growing it character by character, and the two edge cases: an empty list, and one string being a prefix of another.
4	Reverse Words in a String	LeetCode 151 (Medium)	<code>split()</code> with no argument versus <code>split(" ")</code> . The whole difficulty is multiple spaces, leading spaces and trailing spaces.

On problem 1, answer the question the problem is asking

The signature hands you `s: list[str]`, not `s: str`. Say out loud why, before writing anything. Then write the two-pointer in-place reversal from [day 013](#) and notice it works only because it is a list. Then write the one-line string version, `s[::-1]`, and state its cost — $O(n)$ time and $O(n)$ space, because it builds a new string.

On problem 4, break it deliberately

Run these three and predict each first:

```
print(" the sky is blue ".split())
print(" the sky is blue ".split(" "))
print(len(" the sky is blue ".split(" ")))
```

Then say, in one sentence, why bare `split()` solves the whole problem and `split(" ")` creates it.

The immutability drill

Predict the output of each, then run them.

```
s = "hello"
s.upper()
print(s)
```

```
s = "hello world"
s.replace("world", "there")
print(s)
```

```
a = "hello"
b = a
a += " world"
print(a, b)
```

```
c = "hello"
d = "".join(["hel", "lo"])
print(c == d, c is d)
```

```
s = "hello"
s[0] = "H"
```

For the last one, read the exact error text and say it back from memory. For the fourth, say why `is` returned what it did and why you must never use it on strings.

The quadratic drill

Type this in and run it. The numbers matter more than the code.


```
import time

def prepend(n: int) → float:
    s = ""
    start = time.perf_counter()
    for _ in range(n):
        s = "x" + s
    return time.perf_counter() - start

def joined(n: int) → float:
    parts = []
    start = time.perf_counter()
    for _ in range(n):
        parts.append("x")
    _ = "".join(parts)
    return time.perf_counter() - start

for n in (20_000, 40_000, 80_000, 160_000):
    print(n, round(prepend(n), 4), round(joined(n), 4))
```

Then answer:

1. When `n` doubles, by what factor does the first column grow? What does that factor tell you?
2. By what factor does the second column grow?
3. At `n = 160,000`, how many character copies does the first version do in total?
4. Why did the lesson use `"x" + s` rather than `s += "x"` for this measurement?

Question 4 is the one worth being able to answer, because it is the honest caveat an interviewer may raise.

The method drill

From memory, say what each of these returns **and** whether it changes the original:

`strip()`, `split()`, `split(",")`, `join()`, `lower()`, `replace()`, `find()`, `index()`, `startswith()`, `sorted()`, `s[::-1]`, `list(s)`

Two of them behave differently from all the others. Name them and say how. (`sorted` returns a list, not a string. `index` raises where `find` returns `-1`.)

The authn/authz drill

For each situation, say **authentication** or **authorisation**, and the status code on failure:

1. Typing your password into a login form.
2. A free-tier user trying to export a report that is a paid feature.
3. An expired session token on an API request.
4. A member trying to delete another member's comment.

5. An OTP arriving on your phone.
6. A doctor opening a patient record for a patient who is not theirs.
7. A request with no `Authorization` header at all.
8. A moderator hiding a comment successfully.

Then the harder one: for number 6, argue why a hospital system might return `404` rather than `403`, and say when you would and would not do that.

The password-storage drill

Answer these out loud, in ninety seconds, as if it were the interview question it is:

1. Why not store passwords encrypted?
2. Why not SHA-256?
3. What is a salt, and what attack does it prevent?
4. What is a work factor, and how do you choose one?
5. Roughly how many guesses per second does a GPU manage against SHA-256, and against bcrypt at cost 12?
6. Why does the login endpoint hash a dummy value when the email does not exist?
7. Why must the final comparison be constant-time?

Number 6 is the one almost nobody has ready, and it is a genuinely good answer to have.

The broken-endpoint drill

Say what is wrong with each, in one sentence, naming the flaw:

```
GET /orders/1055          → returns the order if you are logged in
GET /login?email=a@b.com&password=hunter2
POST /login              → 401 "no account with that email"
POST /login              → unlimited attempts, no rate limit
DELETE /comments/91      → the UI hides the button for non-owners
Set-Cookie: session=abc123 (no flags)
```

For the last one, name the three flags that are missing and the attack each one prevents.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why is string concatenation in a loop slow?* Lead with immutability, then count `1 + 2 + ... + n` out loud, then give the number for `n = 100,000`, then the fix and why `join` is linear. Add the honest caveat about the CPython special case at the end, not

the beginning.

2. *What is the difference between authentication and authorisation?* One sentence each, then the asymmetry — once versus every request — then a concrete example, then 401 versus 403. Twenty to forty seconds. This is a question you can over-answer.
 3. *How would you store passwords, and why?* Not plaintext, not encrypted, not a fast hash. Argon2id or bcrypt, salted, work factor tuned to about 250 ms, constant-time comparison, library not hand-rolled. Then the number that makes the argument: billions of guesses a second against SHA-256, about four a second against bcrypt.
-

Before you move on

- [] I can say why a string cannot be modified, and what happens instead.
- [] I never build a string with += in a loop, and I can count the cost out loud.
- [] I know that every string method returns a new string and changes nothing.
- [] I use = on strings, never is, and I can say why is sometimes appears to work.
- [] I use bare split() unless I specifically want the empty pieces.
- [] I can define authn and authz in one sentence each, with an example.
- [] I can answer the password-storage question without hesitating.
- [] I put ownership checks in the query, and I can name the flaw that happens when I do not.
- [] I can redraw the two-doors diagram from memory, in whatever tool I like.

Building strings without the quadratic trap

DATA STRUCTURES AND ALGORITHMS

Building strings without the quadratic trap

You build strings with a list and join, and can show the cost difference with numbers.

SYSTEM DESIGN

JWT, sessions, and OAuth

You can explain how a user stays logged in, three different ways, and compare them.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Build a string of n characters efficiently.*
- *How do you keep a user logged in across requests?*

Building strings without the quadratic trap

1 What this is, and why they ask it

Yesterday established the problem: strings are immutable, so growing one a piece at a time copies everything each turn, and building n characters costs $1 + 2 + \dots + n$ — quadratic. Today is the fix, and the fix is one sentence:

Accumulate the pieces in something cheap to extend. Produce the string exactly once, at the end.

In Python that is a list and `"".join(parts)`. In Java it is `StringBuilder`. In Go it is `strings.Builder`. In C# it is `StringBuilder` again. Four languages, one idea, because all four have immutable strings and all four hit the same wall.

Interviewers care about this beyond the trivia because **almost every string problem builds a result**. Compress a string, encode a run, format a report, reverse the words, produce the output of a state machine. If you reach for `+=` in a loop the correctness is fine and the complexity is wrong, and complexity is what the question was about. The tell they are listening for is whether you say “I’ll collect into a list and join” *before* writing the loop, rather than being told afterwards.

There is also a second, subtler question hiding in this topic, and it is worth having ready: **what if the problem hands you a list of characters instead of a string?** Then you can mutate, and the right answer is often the write pointer from [day 015](#), not `join` at all.

2 The story

Hema lives on the second floor of a house in a lane in Coimbatore, and there is a vegetable shop right at the end of that lane. Four minutes there, four minutes back, up two flights of stairs.

For years she went whenever something ran out. Somebody would say there were no onions and she would put on her slippers and go. Half an hour later somebody else would say there was no coriander, and she would go again. On a bad evening she did that walk

four or five times, and each walk took the same eight minutes whether she was carrying one bunch of coriander or a full bag. The bag was never the problem. **The walk was the problem, and the walk cost the same either way.**

Her daughter changed it, in the way that daughters do, by refusing to argue and simply doing something different. She made a note in her phone, and every time anyone in that house said the words “we’ve run out of”, she typed it in. It takes two seconds. Nobody has to stop what they are doing, nobody has to remember it later, and the note just sits there getting longer.

Then on Saturday morning Hema takes the phone and goes once. She reads the whole list at the shop, the man puts everything into her big cloth bag, and she comes back up the stairs. One walk.

She uses the big bag now, not the small one, and that is deliberate too. The big bag is more than she needs most weeks. But going back down for a second bag would cost her the whole walk again, so she takes the bigger one and does not think about it. The extra room costs nothing; a second trip costs eight minutes.

The thing that surprised her was that it also made the shopping better. When she went five times she bought whatever she had gone for and forgot two other things every time. Now she reads out a list that somebody wrote down at the exact moment they noticed, so nothing gets missed, and she is home by nine.

3 The idea in plain English

The note on the phone is the list. The walk to the shop is the allocate-and-copy. Adding to the note is nearly free; the walk is not, and it costs about the same whatever you carry. So you add freely, and you walk once.

The rule, and the three things it applies to

Never build a string incrementally. Collect, then join.

```
parts = []                # cheap to extend
for word in words:
    parts.append(word)     # O(1) amortised – day 011
return "".join(parts)     # one allocation, each character copied once
```

`"".join(parts)` does two passes over the list: one to add up the total length, one to copy each piece into a string allocated at exactly that size. Every character is copied exactly once. That is $O(n)$ where n is the length of the output.

Read the call carefully, because it reads backwards the first time. `join` is a method on the separator, not on the list. `"".join(parts)` glues with nothing between; `", ".join(parts)` puts a comma and a space between each pair, and — importantly — not after the last one. Every trailing-comma bug you have ever seen is somebody not using `join`.

The big bag: why appending is cheap

Hema takes a bag bigger than she needs. A Python list does the same thing. When it fills up it does not grow by one — it allocates a noticeably larger block and copies everything across, then has room to spare for many more appends.

That is why `append` is $O(1)$ **amortised**: most appends are free, an occasional one is expensive, and spread across all of them the average is constant. You met this on [day 011](#). It is the exact reason a list is the right accumulator and a string is not: **a list can be given room to grow, and an immutable string cannot.**

The four ways to build, and when each is right

WAY	USE IT WHEN	COST
<code>"a" + b + "c"</code>	a fixed, small number of pieces	$O(\text{total})$ — fine
<code>f"{name}:"</code> <code>{count}"</code>	formatting a handful of values	$O(\text{total})$ — the readable default
<code>"".join(parts)</code>	a loop builds the pieces	$O(n)$ — the answer to today's question
<code>io.StringIO()</code>	the output is huge, or genuinely streamed	$O(n)$, and constant memory if you write it out as you go

Notice what is *not* wrong: `"Hello, " + name + "!"` is perfectly good code. Concatenation is only a problem when the number of concatenations grows with the input. Three pieces is three pieces.

The comprehension form, which is what you actually write

Once the pattern is in your hands, the intermediate list usually disappears into the call:

```
return " ".join(word.upper() for word in words)
```

That is a **generator expression** — it produces each item as `join` asks for it. Note that `join` has to know the total length before allocating, so it materialises the generator internally anyway; you are not saving memory, you are saving a line. Both are $O(n)$ and either is fine in an interview. Say `"".join(...)` and you have answered the question.

`io.StringIO` , for when the output is enormous

```
import io

buffer = io.StringIO()
for row in rows:
    buffer.write(row)
    buffer.write("\n")
result = buffer.getvalue()
```

`StringIO` is a growable in-memory text buffer — the closest thing Python has to Java's `StringBuilder` . It is about as fast as list-and-join and it earns its place in one specific case: when you are producing something too large to want in memory at all, you can swap `io.StringIO()` for an actual open file and the loop does not change. That is worth one sentence in an interview and no more.

When the input is a list of characters, do not join at all

Some problems hand you `list[str]` rather than `str` , and that is a deliberate signal: **you are allowed to mutate, so mutate**. LeetCode 443, *String Compression*, is the canonical one — it wants `["a", "a", "b"]` turned into `["a", "2", "b"]` **in place**, returning the new length.

That is not a `join` problem. That is the write pointer from [day 015](#): a `read` index walking the input, a `write` index saying where the next kept character goes, and `write ≤ read` throughout so overwriting is always safe. $O(1)$ extra space, where `join` would be $O(n)$.

Reading the signature is part of solving the problem. `s: str` means build and return. `chars: list[str]` means mutate and return a count.

4 The picture

The two shapes, side by side. Each `#` is one character copied.

GROWING A STRING

```
-----
turn 1  "a"      #
turn 2  "ab"     ##
turn 3  "abc"    ###
turn 4  "abcd"   ####
turn 5  "abcde"  #####
          ----
          total 15 copies for n=5

           $n(n+1)/2 \rightarrow O(n^2)$ 
```

COLLECT, THEN JOIN

```
-----
append 'a'      (no copy)
append 'b'      (no copy)
append 'c'      (no copy)
append 'd'      (no copy)
append 'e'      (no copy)
join: #####    one pass
          total 5 copies for n=5

           $n \rightarrow O(n)$ 
```


What to notice: the left column is a triangle and the right is a single row. The triangle is the whole cost, and it is entirely made of re-copying characters that were already in the right order.

Inside `join`, in two passes:

```
parts:  [ "the" ][ " " ][ "sky" ][ " " ][ "is" ]
          3   +   1   +   3   +   1   +   2   = 10      pass 1: measure

allocate exactly 10 characters:
+---+---+---+---+---+---+---+---+---+---+
| | | | | | | | | | |
+---+---+---+---+---+---+---+---+---+---+

t   h   e           s   k   y           i   s           pass 2: copy once each
+---+---+---+---+---+---+---+---+---+---+
| t | h | e |   | s | k | y |   | i | s |
+---+---+---+---+---+---+---+---+---+---+
```

What to notice: the allocation happens once and it is exactly the right size. No character is ever moved twice, and there is no over-allocation to waste.

And the list growing underneath, which is why the appends were free (Hema's big bag):

capacity	4	[a][b][c][d]	4 appends, no copying
full →	8	[a][b][c][d][][][][]	one copy of 4 items, then 4 free appends
full →	16	[a]...	one copy of 8 items, then 8 free appends

n appends cost about 2n operations in total → 0(1) each, amortised

What to notice: the copies happen at 4, 8, 16, 32 — and each copy is followed by twice as many free appends. That is why the average stays constant no matter how long the list gets.

5 The code, built step by step

The pattern, in three lines

```
parts: list[str] = []
for piece in source:
    parts.append(piece)
return "".join(parts)
```

Learn it as one unit. Whenever a loop produces output, this is the shape.

Joining things that are not strings

```
counts = [3, 1, 4]
",".join(counts)
```

```
TypeError: sequence item 0: expected str instance, int found
```

`join` will not convert for you. Two fixes, both idiomatic:

```
",".join(str(c) for c in counts)    # a generator expression
",".join(map(str, counts))         # map – slightly faster, equally clear
```

The same strictness bites in plain concatenation:

```
"count: " + 5
```

```
TypeError: can only concatenate str (not "int") to str
```

which is why f-strings exist: `f"count: {5}"` converts for you.

Worked example one: run-length encoding

“Turn `aaabbc` into `a3b2c1`.” A loop that produces output — so, the pattern.

Start with the accumulator and the running count.

```
if not s:
    return ""
parts: list[str] = []
count = 1
```

The empty guard first, because the loop below reads `s[-1]` and would raise on `""`.

```
for i in range(1, len(s)):
    if s[i] == s[i - 1]:
        count += 1
    else:
        parts.append(s[i - 1])
        parts.append(str(count))
        count = 1
```

Walk from position 1. If this character matches the one before it, the run continues. Otherwise the run has ended, so emit the character and its count and start a new run.

Now the part everybody forgets:

```
parts.append(s[-1])
parts.append(str(count))
return "".join(parts)
```

The last run is never emitted inside the loop, because the loop only emits when it sees a *change*, and the final run ends by running out of string rather than by changing. Emitting after the loop is not a special case bolted on — it is the natural consequence, and saying that out loud is much better than saying “oh, and I forgot the end”.

Worked example two: the same problem, in place

LeetCode 443 hands you `chars: list[str]` and wants the answer written back into it.

```
write = 0
read = 0
n = len(chars)
```

Two indices, exactly as on [day 015](#).

```
while read < n:
    ch = chars[read]
    count = 0
    while read < n and chars[read] == ch:
        read += 1
        count += 1
```

The inner loop consumes one whole run and counts it, leaving `read` at the first character of the next run. This is why the outer loop is a `while` and not a `for`: `read` advances by a whole run, not by one.

```
chars[write] = ch
write += 1
if count > 1:
    for digit in str(count):
        chars[write] = digit
        write += 1
```

Write the character, then its count — but only if the count is more than 1, because the problem says a run of one is written as just the character. And a count of 12 becomes two cells, `'1'` and `'2'`, which is why it loops over the digits of `str(count)`.

Is the write safe? Yes, and this is the sentence to say: a run of length `k` occupies at least `k` cells in the input and produces at most `1 + len(str(k))` cells of output, which for every `k ≥ 1` is never more than `k`. So `write` can never overtake `read`.

Worked example three: formatting a report

```
def report(rows: list[tuple[str, int]]) → str:
    lines = [f"{name:<12} {count:>5}" for name, count in rows]
    lines.append("-" * 18)
    lines.append(f"{'TOTAL':<12} {sum(c for _, c in rows):>5}")
    return "\n".join(lines)
```

`f"{name:<12}"` pads to width 12, left aligned; `{count:>5}` pads to 5, right aligned. Building the lines in a list and joining with `"\n"` is the same pattern, and it means the last line has no trailing newline unless you want one.

The complete solutions

```
import io
import time

def run_length_encode(s: str) → str:
    """'aaabbc' → 'a3b2c1'. Builds once."""
    if not s:
        return ""
    parts: list[str] = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            parts.append(s[i - 1])
            parts.append(str(count))
            count = 1
    parts.append(s[-1])          # the final run always ends outside the loop
    parts.append(str(count))
    return "".join(parts)

def compress(chars: list[str]) → int:
    """LeetCode 443. In place. Returns the new length; chars[:length] is the answer.

    Runs of 1 are written as just the character. write never overtakes read.
    """
    write = 0
    read = 0
    n = len(chars)
    while read < n:
        ch = chars[read]
        count = 0
        while read < n and chars[read] == ch: # consume one whole run
            read += 1
            count += 1
        chars[write] = ch
        write += 1
        if count > 1:
            for digit in str(count):          # 12 becomes '1' then '2'
                chars[write] = digit
                write += 1
    return write

def to_csv_line(values: list[object]) → str:
    """Join anything, converting as you go. Note map(str, ...)."""
    return ",".join(map(str, values))

def build_with_stringio(rows: list[str]) → str:
    """The StringIOBuilder equivalent. Swap StringIO for a file and nothing else changes."""
    buffer = io.StringIO()
    for row in rows:
        buffer.write(row)
        buffer.write("\n")
    return buffer.getvalue()

def compare(n: int) → None:
    """The measurement to have in your head."""
    start = time.perf_counter()
    s = ""
    for _ in range(n):
        s = "x" + s                      # prepend: the honest quadratic
    grow = time.perf_counter() - start
```

```

start = time.perf_counter()
parts: list[str] = []
for _ in range(n):
    parts.append("x")
_ = "".join(parts)
join = time.perf_counter() - start

start = time.perf_counter()
buffer = io.StringIO()
for _ in range(n):
    buffer.write("x")
_ = buffer.getvalue()
sio = time.perf_counter() - start

print(f"n={n}: grow {grow:.4f}s    list+join {join:.4f}s    StringIO {sio:.4f}s    "
      f"({grow / join:.0f}x)")

if __name__ == "__main__":
    print(run_length_encode("aaabbc"))      # a3b2c1
    print(run_length_encode("abc"))          # a1b1c1
    print(run_length_encode("a"))           # a1
    print(repr(run_length_encode("")))      # ''

    c = list("aabbccc")
    k = compress(c)
    print(k, "".join(c[:k]))                # 6 a2b2c3

    c = list("abbbbbbbbbbb")
    k = compress(c)
    print(k, "".join(c[:k]))                # 12 b's

    c = list("a")
    k = compress(c)
    print(k, "".join(c[:k]))                # 1 a

    print(to_csv_line(["Hema", 3, 37.5]))    # Hema,3,37.5
    compare(200_000)

```

Measured on Python 3.12:

```
n=200000: grow 0.9202s    list+join 0.0087s    StringIO 0.0090s    (106x)
```

6 What it costs

The list-and-join version

The appends. n calls to `append`, each $O(1)$ amortised. Totalling the occasional growth copies — 4, then 8, then 16, and so on — the copying done across all n appends adds up to less than $2n$, so the whole loop is about $3n$ operations. **$O(n)$.**

The join. One pass over the pieces to total the lengths, one allocation, one pass copying. Each character is copied exactly once. **$O(n)$ time.**

Total: $O(n)$ time, $O(n)$ space for the list of pieces plus $O(n)$ for the result. Note honestly that you are holding roughly two copies of the output at the moment `join` runs — that is the price, and it is the reason `StringIO` writing straight to a file wins when the output is genuinely huge.

Against growing a string

From yesterday: $1 + 2 + \dots + n = n(n+1)/2$, so $O(n^2)$.

The measured comparison at $n = 200,000$:

```
grow one at a time : 0.9202 s
list + join        : 0.0087 s      106 times faster
StringIO           : 0.0090 s      about the same as join
```

And the predicted operation counts:

```
grow : 200,000 × 200,001 / 2 ≈ 20,000,000,000 character copies
join : about 600,000 operations
```

Twenty billion against six hundred thousand. The measured ratio is only $106\times$ rather than $33,000\times$ because a bulk memory copy inside CPython is enormously cheaper per character than a Python-level loop turn — but the *shape* is what you are being asked about, and the shape is unambiguous.

`compress`, the in-place version

The outer `while` and the inner `while` between them advance `read` exactly n times in total — each character is consumed by exactly one run — so it is n turns of constant work: $O(n)$ time.

Space: `write`, `read`, `count`, `ch`, and the digits of one count. $O(1)$ extra space, which is the entire reason the problem hands you a list rather than a string.

The number to have ready

Building a 200,000-character string one piece at a time is about 20 billion character copies and takes roughly a second. With a list and `join` it is about 600,000 operations and takes 9 milliseconds — a hundred times faster, and the gap widens as the input grows.

7 The traps

The near-miss: the loop that hides inside another loop

```
def flatten(rows):
    out = ""
    for row in rows:
        for cell in row:
            out += str(cell) + ","      # quadratic, and hard to see
        out += "\n"
    return out
```

The `+=` is buried two levels down, and the quadratic cost is in the total output length, not the number of rows. On a $1,000 \times 100$ grid the output is around 300,000 characters and this does tens of billions of copies. The fix is the same as ever, and it is also clearer:

```
def flatten(rows):
    return "\n".join(",".join(map(str, row)) for row in rows) + "\n"
```

When you see `+=` on a string inside any loop, that is the bug, however deeply it is nested.

The real error: joining non-strings

```
counts = [3, 1, 4]
print(",".join(counts))
```

```
TypeError: sequence item 0: expected str instance, int found
```

The message even tells you which item. Fix with `map(str, counts)` or a generator expression. And the sibling error:

```
print("count: " + 5)
```

```
TypeError: can only concatenate str (not "int") to str
```

Both are Python refusing to guess. Use an f-string when you want conversion.

The near-miss: forgetting the final run

```
def run_length_encode(s):
    parts = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            parts.append(s[i - 1])
            parts.append(str(count))
            count = 1
    return "".join(parts)          # the last run never got emitted

print(run_length_encode("aaabbc"))
```

```
a3b2
```

The `c1` is missing. The loop emits a run only when it *sees a change*, and the last run ends by reaching the end of the string, which is not a change. **Any “group consecutive things” loop has this shape and this bug**, and it will recur when you group anything — so learn the reflex: after the loop, flush what is still in hand.

The near-miss: `join` on a string instead of a list

```
print("-".join("abc"))
```

```
a-b-c
```

No error, and probably not what you meant. A string is a sequence of characters, so `join` happily iterates it one character at a time. If you meant to join a single string to something, you did not want `join` at all. This silently produces plausible nonsense, which is the worst kind of bug.

The near-miss: building the separator by hand

```
out = ""
for word in words:
    out += word + ", "
return out[:-2]          # chop off the trailing ", "
```

Quadratic, and the `[:-2]` is a bug waiting for the empty list — `"".join([])[: -2]` is fine but `""[:-2]` on the hand-rolled version returns `""` only by luck, and with a one-character separator the slice would be wrong. `", ".join(words)` handles the separator,

the last element and the empty list correctly, all three, and is faster. There is no situation in which the manual version is better.

The trap: reaching for `join` when the problem wanted mutation

If the signature says `chars: list[str]` and asks you to return a length, building a new string and joining is the wrong answer even though it produces the right characters — it uses $O(n)$ extra space where the question is specifically testing whether you can do it in $O(1)$. **Read the signature. It is telling you which technique is being examined.**

8 In the interview

How it gets asked

- “Build a string of n characters efficiently.” — the direct version, usually as a follow-up to “why is concatenation slow”.
- “Compress the string: aabbccc becomes a2b2c3.” — LeetCode 443, and note whether the signature hands you a string or a list.
- “Reverse the words in this sentence.” — LeetCode 151, where the whole answer is `split` and `join`.
- “Given a large log file, produce a summary line per user.” — the practical version, where the right answer includes streaming rather than holding it all.

What to say out loud, in the first ninety seconds

1. **Announce the pattern before the loop.** “Since strings are immutable, I’ll collect the pieces in a list and join once at the end — that keeps it $O(n)$ instead of $O(n^2)$.” Saying it first is the whole difference.
2. **Check the signature out loud.** “The input is a list of characters rather than a string, which means I’m allowed to mutate it — so this is a write-pointer problem in $O(1)$ space, not a join problem.”
3. **Name what the loop emits and when.** “I’ll walk the string tracking the current run, and emit a character-and-count each time the run ends.”
4. **Flag the final flush before you write it.** “The last run ends by reaching the end of the string rather than by changing, so it never gets emitted inside the loop — I’ll emit it after.” Interviewers watch specifically for this.
5. **Give the cost with the counting.** “One pass over n characters, each turn constant work, so $O(n)$ time. $O(n)$ space for the pieces — or $O(1)$ extra if I’m mutating in place.”
6. **Mention the language-independence.** “Same idea as `StringBuilder` in Java or `strings.Builder` in Go — accumulate in something mutable, materialise once.”

The follow-ups

“What if the output doesn’t fit in memory?” Then do not build it in memory at all. The list-and-join approach holds the pieces *and* the result simultaneously, which is roughly two copies at the moment `join` runs. Instead I would stream: open the destination and write each piece as it is produced, so memory stays constant regardless of output size. In Python that is the same loop with `file.write(piece)` in place of `parts.append(piece)` — `io.StringIO` and a real file expose the same interface, which is exactly why the swap is free. If the consumer is another part of the program rather than a file, I would make the function a generator that yields pieces, letting the caller decide whether to join them or stream them onward.

“Is `+=` really always quadratic?” Not always in CPython, and it is worth being precise. There is an optimisation that can extend a string in place when nothing else refers to it, so a benchmark of exactly `s += "x"` may look linear. It vanishes the moment another name refers to the string, or if you build the other way round with `s = "x" + s`, or on a different Python implementation. So the behaviour you get depends on a refcount you are not tracking, which is not something to design around. The rule stands: accumulate in a list and join. I would mention the optimisation as a caveat, never as a justification.

“How does this work in Java?” The same, with a different name. Java strings are immutable too, and `+` in a loop compiles to repeated allocation, so Java gives you `StringBuilder` — a mutable character buffer with an `append` method and a `toString` at the end. It is exactly Python’s list-and-join with the two steps fused into one object. `StringBuffer` is the older, synchronised version and you would use `StringBuilder` unless you genuinely need thread safety. Go has `strings.Builder`, C# has `StringBuilder`, and every one of them exists for the reason we just discussed.

“In `compress`, how do you know the write index never overtakes the read index?” Because compression never makes a run longer. A run of `k` identical characters occupies `k` cells in the input and produces `1 + len(str(k))` cells of output — one for the character, plus the digits of the count, and nothing at all for the count when `k` is 1. For `k = 1` that is 1 cell out of 1 in. For `k = 2` it is 2 out of 2. For `k = 9` it is 2 out of 9. For `k = 10` it is 3 out of 10. It is never more, so after processing each run `write` is at most where `read` is. The worst case is a string with no repeats at all, where output length equals input length and the two indices stay exactly level — which is precisely the `write ≤ read` guarantee from the write-pointer pattern.

A model answer

“Strings are immutable, so if I build the answer with `+=` inside the loop, each turn copies everything produced so far and the whole thing costs $1 + 2 + \dots + n$, which is quadratic in the output length. So I’ll collect the pieces in a list and call `join` once at the end. Appending to a list is $O(1)$ amortised, and `join` makes one pass to total the lengths, allocates exactly that much, and copies each character once — so the whole thing is $O(n)$.”

For run-length encoding specifically: I walk the string from position 1, comparing each character with the one before it. If they match, the run is still going and I increment a counter. If they differ, the run has ended, so I append the character and its count to the parts list and reset the counter to 1.

```
def run_length_encode(s: str) → str:
    if not s:
        return ""
    parts: list[str] = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            parts.append(s[i - 1])
            parts.append(str(count))
            count = 1
    parts.append(s[-1])
    parts.append(str(count))
    return "".join(parts)
```

The two lines after the loop are the part worth calling out. The loop only emits when it sees a change, and the final run doesn’t end with a change — it ends by running out of string. So the last group is still in hand when the loop finishes and has to be flushed. Every ‘group consecutive items’ loop has that shape, and forgetting the flush is the standard bug: on `aaabbc` you’d get `a3b2` and silently lose the `c1`.

The empty-string guard at the top is needed because I read `s[-1]` afterwards.

That’s $O(n)$ time and $O(n)$ space for the pieces.

If instead you hand me the input as a list of characters and ask me to return a length — which is how LeetCode 443 phrases it — then I’d solve it differently, because that signature is telling me I may mutate. I’d use a read index and a write index over the same list: the read index consumes a whole run at a time, and the write index emits

the character and, if the count is above one, its digits. That's $O(1)$ extra space. It's safe because compressing never lengthens a run — a run of k characters produces at most k cells of output — so the write index can never overtake the read index.”

9 Recall card

- **Accumulate in a list**, `"".join(parts)` **once**. $O(n)$ instead of $O(n^2)$. Say it before the loop.
- `join` is a method on the separator and handles the last element for you — never build separators by hand.
- `join` will not convert types: `",".join(map(str, values))`.
- **After any grouping loop, flush the last group** — it ends by running out, not by changing.
- `chars: list[str]` in the signature means mutate, so use the write pointer and $O(1)$ space, not `join`.

1 What this is, and why they ask it

Yesterday's lesson ended on an unfinished sentence. HTTP is **stateless** — from [day 016](#), each request arrives knowing nothing about the last one — so after a user types their password successfully, what stops them having to type it again on the very next click?

The answer is that the server hands out a **credential for later**, and the client presents it on every subsequent request. There are three common shapes for that credential and they trade off differently:

- a **session id** — an opaque reference, meaningless by itself, that the server looks up;
- a **JWT** — a self-contained signed token that carries the facts inside it, so no lookup is needed;
- **OAuth 2.0 and OpenID Connect** — a protocol for getting one of the above from *somebody else*, so a user can log in with Google without your service ever seeing their password.

Interviewers ask this constantly, and the question has a quality filter built in. “Use JWTs, they’re stateless and they scale” is the answer of somebody who has read a blog post. “It depends on whether you need instant revocation, and here is what I would actually do” is the answer of somebody who has operated one. The revocation problem is the heart of the topic, and it is what they are waiting to hear you raise.

2 The story

The gym Rohit joined in January is on the first floor above a bank, and it is run by a man called Salim who sits at a desk by the stairs.

The first time Rohit went, it took twenty minutes. Salim typed his name, his phone number and his address into the shop tablet, took a photo of him, took three months' money, and gave him a small plastic card with a number on it. 0412. Nothing else on the card. Not his name, not the dates he had paid for, not whether he was on the three-month plan or the annual one. Just 0412.

Every morning after that Rohit holds the card up as he comes past the desk. Salim glances at it, types 0412 into the tablet, sees the screen come back with Rohit's photo and *paid till 31 March*, and nods. It takes two seconds. All the real information lives on Salim's tablet; the card is only a way of pointing at it.

That has one very useful property. When a member stopped paying in February, Salim did not have to find him or get the card back. He changed one line on the tablet and the card became worthless the same morning, while still sitting in the man's pocket.

The gym across the road works differently, and Rohit's friend goes there. They give you a printed band for your wrist with everything on it — your name, your plan, and the date it runs out — and a small holographic sticker that is hard to fake. The man at their door does not look anything up. He reads the band, checks the sticker looks right, checks the date has not passed, and waves you in. Faster, and it works even on the days their computer is down.

But Rohit's friend told him what happened when someone's cheque bounced in March. They could not do anything about the band. It said *valid till 30 June* and it looked perfectly genuine, because it was. The man on the door had no way to know, and no list to check against, and the whole point of those bands is that he does not have to have one. They had to wait until the end of June, or stand there arguing with him every morning.

3 The idea in plain English

Rohit's plastic card is a **session id**. His friend's wristband is a **JWT**. The whole comparison is in those two paragraphs, and the difference is where the truth lives.

The session: a reference the server looks up

A **session** is a record the server keeps about a logged-in user. After a successful login the server creates it, gives it a long random id, and sends only the **id** to the client.

```
Set-Cookie: session=8f2a9c1e4b7d0364...; HttpOnly; Secure; SameSite=Lax
```

That string means nothing on its own — it is Salim's 0412. On every request the server takes the id, looks the record up in a shared store, and gets back who the user is, when they logged in, and what they may do.

The store is almost always Redis, because the lookup is on the path of every single request and must be fast. It can be Postgres for smaller systems. It has to be reachable by every server, or you are back to sticky sessions and the scaling problem from [day 016](#).

Logging out — or banning a user, or forcing a password change — is one delete. The next request finds nothing and gets a `401`. That instant, complete revocation is the whole reason sessions still dominate.

The JWT: a token that carries the facts

A **JWT** — JSON Web Token, said “jot” — is a string containing the facts themselves, signed so that it cannot be altered. Three parts separated by dots:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJzdWIiOiI0MiIsImV4cCI6MTc2N30 . 4Xr9dQ2v...  
header                                payload                                signature
```

Each part is base64-encoded JSON. Decoded:

```
// header — how it was signed  
{ "alg": "RS256", "typ": "JWT" }  
  
// payload — the "claims"  
{ "sub": "42", "name": "Rohit", "role": "member",  
  "iat": 1767200000, "exp": 1767203600, "iss": "auth.example.com" }
```

Two things about the payload that get asked and are frequently got wrong:

It is encoded, not encrypted. Base64 is not a secret — anyone holding the token can read every claim in it. Paste one into jwt.io and it shows you the contents. **Never put anything sensitive in a JWT payload.**

The signature is what makes it trustworthy. It is computed over the header and payload using a key only the issuer has. Change one character of the payload and the signature no longer matches, so the server rejects it. That is the holographic sticker: it does not hide what the band says, it proves nobody rewrote it.

The standard claims are worth knowing by name: `sub` (subject — who it is about), `exp` (expiry), `iat` (issued at), `iss` (issuer), `aud` (audience — which service it is for).

Verification needs no lookup. The server checks the signature with its key, checks `exp` has not passed, and now knows who is calling. No Redis, no database, no shared state. That is the whole advantage, and it is a real one — it is why a system split across many services likes JWTs, since each service can verify independently.

The problem, which is the point of the lesson

You cannot un-issue a JWT. It is valid until it expires, because validity is a property of the token itself and not of any list. Ban a user and their token keeps working. Change a password after a laptop is stolen and the stolen token keeps working. That is the bounced cheque and the wristband that says *valid till 30 June*.

Every fix reintroduces state:

- **Short expiry plus refresh tokens.** Access tokens live 5–15 minutes; a long-lived refresh token, stored server-side and revocable, is exchanged for new ones. Damage from a stolen token is bounded by the expiry, and revocation takes effect within that

window. **This is what almost everyone actually does.**

- **A denylist of revoked token ids.** Every request checks it — which is a lookup on every request, which is the thing you removed sessions to avoid. It is smaller than a session store, because it only holds revoked tokens until they expire, but it is not stateless.

So the honest summary, and it is a good sentence to say out loud: **a JWT trades instant revocation for statelessness, and the standard mitigation gives some of the revocation back by making the window short.**

OAuth 2.0: getting a token from somebody else

The third shape answers a different question: how does an application act on a user's behalf at *another* service, without ever seeing their password?

That is **OAuth 2.0**, and — this is the distinction interviewers check — it is an **authorisation** framework, not an authentication one. Its output is an access token that grants limited permission, not a statement of who somebody is.

Four parties:

ROLE	WHO IT IS, CONCRETELY
Resource owner	the user
Client	your application
Authorisation server	Google's login and consent screen
Resource server	the Google API holding their calendar

The flow, for the version you should describe — **authorisation code with PKCE**:

1. Your app sends the user to Google with a `client_id`, a `redirect_uri`, the **scopes** you want (`calendar.readonly`), and a hashed one-time secret (the PKCE challenge).
2. Google authenticates the user itself and shows the consent screen: *"This app wants to read your calendar."*
3. Google redirects the browser back to your app with a short-lived **authorisation code**.
4. Your app's **backend** exchanges that code — plus its client secret and the PKCE verifier — for an access token, over a direct server-to-server call.
5. Your app calls Google's API with the access token.

Why the extra hop through a code? Because step 3 travels through the user's browser, where it lands in history and logs. A code is single-use and useless without the client secret, so intercepting it gains nothing. The token itself never touches the browser's address bar.

Where “Log in with Google” fits. OAuth alone tells you nothing about *who* the user is — only that you were granted access to something. **OpenID Connect (OIDC)** is a thin layer on top that adds an `id_token`, a JWT describing the user, and a `/userinfo` endpoint. So: **OAuth is authorisation, OIDC is authentication, and “Log in with Google” is OIDC.** That single sentence answers one of the most common follow-ups in this topic.

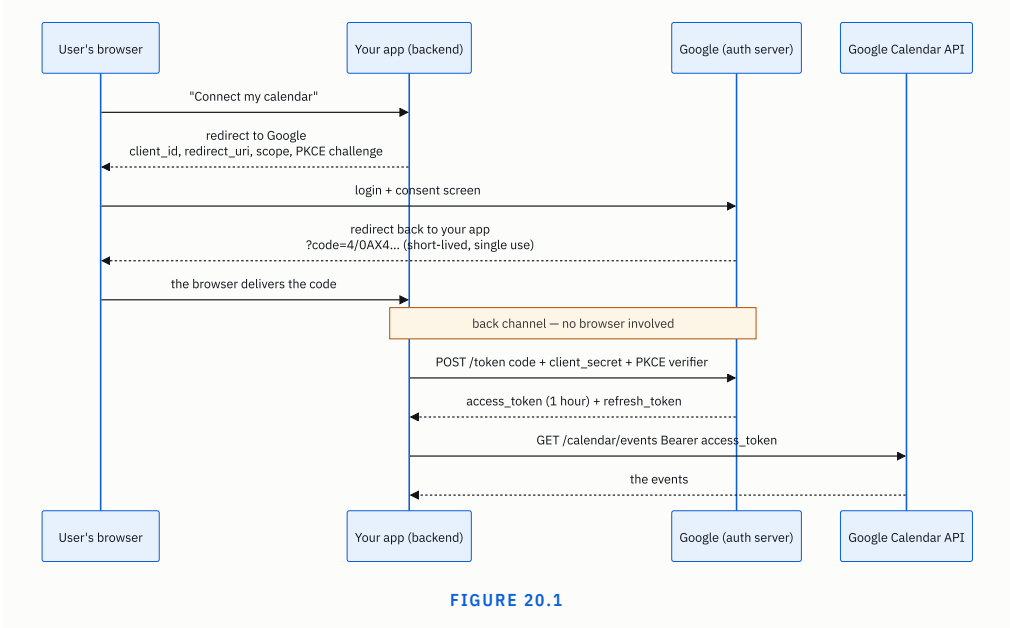
4 The picture

Where the truth lives — the entire comparison in one diagram:

SESSION (the plastic card)	JWT (the wristband)
-----	-----
client holds: 8f2a9c1e... (meaningless)	client holds: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ...sig (readable: sub=42, exp=..., role=member)
server does: look it up v +-----+ Redis ← the truth lives HERE +-----+	server does: check the signature v (no lookup at all) the truth lives IN THE TOKEN
revoke: DELETE the record → instant	revoke: ...you cannot.
cost: 1 network hop per request	cost: a signature check (microseconds)
size on wire: ~32 bytes	size: ~500-1000 bytes, every request

What to notice: the arrow. In the left-hand column every request reaches back to a shared store, which costs a hop and buys instant control. In the right-hand column nothing is reached back to, which is faster and means there is nowhere to press the off switch.

The OAuth authorisation-code flow, which is worth being able to draw:



What to notice: the note in the middle. Everything above it goes through the user's browser and is therefore visible; everything below it is a direct server-to-server call. The access token only ever exists below that line. That split is the reason the flow has a code step at all, and saying so is what distinguishes understanding it from having memorised it.

5 How it actually works

Sessions, concretely

On login: generate a cryptographically random id — at least 128 bits, from `secrets.token_urlsafe` in Python, never from `random` — and store a record:

```
key: session:8f2a9c1e4b7d0364
value: {"user_id": 42, "created": 1767200000, "last_seen": 1767203100,
       "ip": "49.207.x.x", "user_agent": "..."}
TTL: 30 days (sliding, refreshed on activity)
```

On each request: read the cookie, look up the key, `401` if it is missing or expired, otherwise attach the user to the request.

On logout: delete the key.

The cookie flags, all three of which get asked:

- `HttpOnly` — JavaScript cannot read it, so a cross-site scripting flaw cannot steal it.
- `Secure` — only ever sent over HTTPS, so it cannot be sniffed on a café network.
- `SameSite=Lax` (or `Strict`) — another site cannot cause the browser to send it, which is the defence against cross-site request forgery.

Rotate the id on privilege change. Issue a fresh session id at login and after a password change, and delete the old one. Otherwise an attacker who plants a known id in a victim's browser beforehand — **session fixation** — is holding a valid session the moment the victim logs in.

Real implementations: Django's `django.contrib.sessions`, Rails' `ActionDispatch::Session`, Express with `express-session` and `connect-redis`, Spring Session.

JWTs, concretely

Signing algorithm. `HS256` uses one shared secret for both signing and verifying, which means every service that can verify can also forge. `RS256` uses a private key to sign and a public key to verify, so services can verify without being able to issue. **For anything beyond one service, use RS256** — and be able to say why.

Verification, in order: check the signature; check `exp`; check `iss` and `aud` are what you expect. Skipping the last two is a real vulnerability, because a token issued by the same provider for a different application would otherwise be accepted.

The `alg: none` attack, which is the famous one. Early libraries read the algorithm out of the token's own header and trusted it, so an attacker could set `"alg": "none"`, strip the signature, and be believed. The fix is to configure the expected algorithm on the server and refuse anything else — never take it from the token. Any interview question about JWT weaknesses is fishing for this.

Refresh tokens. The pattern almost everyone lands on:

access token	JWT, 15 minutes, stateless, sent on every request
refresh token	opaque random string, 30 days, stored server-side, revocable, sent only to <code>/auth/refresh</code>

The access token is checked with no lookup on thousands of requests; the refresh token is looked up a handful of times a day. **You get the performance of stateless verification and keep a switch to turn a user off — within fifteen minutes.** Rotate the refresh token on every use and invalidate the old one, so a stolen refresh token is detectable: if an old one is presented, both are revoked.

Where to store a JWT in a browser. `localStorage` is readable by any script on the page, so a single XSS flaw hands over the token. An `HttpOnly` cookie is not readable by script, but is sent automatically, which reintroduces CSRF and needs `SameSite`. The usual recommendation is the cookie with `HttpOnly`, `Secure` and `SameSite`, because XSS is far more common than CSRF and CSRF has a simple, complete defence. There is a real argument here and having a position on it is what matters.

Real libraries: `PyJWT`, `jsonwebtoken` for Node, `jjwt` for Java. Auth0, Okta, Cognito and Keycloak all issue JWTs.

OAuth, concretely

Scopes are the permissions you request — `calendar.readonly`, `repo`, `email`. Ask for the minimum; a consent screen requesting everything loses users.

The grant types, and which to use:

GRANT	USE
Authorisation code + PKCE	Everything user-facing. Web apps, mobile apps, single-page apps.
Client credentials	Machine to machine, no user involved.
Device code	TVs and consoles — “go to this URL and enter this code”.
Implicit	Deprecated. Returned the token in the browser URL. Do not use.
Password grant	Deprecated. The app handled the user’s password, defeating the point.

Knowing that the last two are deprecated, and why, is a strong signal — they are still in a lot of older tutorials.

PKCE — Proof Key for Code Exchange, said “pixie” — was originally for mobile apps that cannot hold a client secret, and is now recommended for all clients. The app generates a random verifier, sends its hash up front, and presents the verifier when exchanging the code. An intercepted code is useless without it.

6 The numbers

Take an API at **10,000 requests per second** at peak.

The cost of the session lookup

Every request does one Redis round trip, about **0.5 ms** inside a data centre.

```
10,000 req/s × 0.5 ms = 5 seconds of waiting per second
```

That is 5 concurrent operations in flight at all times — nothing for Redis, which handles well over 100,000 operations a second on one node. But it is 0.5 ms added to **every** response, and one more component that must be up for anybody to log in.

The cost of JWT verification

An `RS256` signature check is roughly **0.1 ms** of CPU, with no network at all.

```
10,000 req/s × 0.1 ms = 1 second of CPU per second = 1 core
```

One core, no dependency, no network hop. That is a genuine improvement — **five times less latency added and one fewer thing that can be down** — and it is the honest case for JWTs.

The cost on the wire

```
session cookie : ~32 bytes
JWT            : ~800 bytes
difference     : ~768 bytes per request
```

```
10,000 req/s × 768 bytes = 7.7 MB/s = 663 GB/day of extra inbound traffic
```

Two thirds of a terabyte a day of pure overhead. Usually irrelevant, and occasionally not — on mobile, that header is retransmitted on every request over a connection where uplink is scarce. And JWTs grow: put a list of permissions in the payload and 800 bytes becomes 4 KB, at which point some proxies start rejecting the header outright. **Keep the payload small** is a practical rule with arithmetic behind it.

Session storage

10 million users, 20% with a live session, ~200 bytes each:

```
10,000,000 × 0.20 × 200 bytes = 400 MB
```

400 MB in Redis. This number is worth carrying, because “sessions don’t scale” is a common claim and the arithmetic simply does not support it. Sessions cost a hop, not a capacity problem.

The revocation window

This is the number that decides the design.

```
access token lifetime 15 min → a banned user keeps access for up to 15 minutes
access token lifetime  5 min → up to 5 minutes, and 3x more calls to /auth/refresh
session                → 0 seconds
```

With 10 million users and 15-minute tokens, refresh traffic is:

```
10,000,000 active-ish users ÷ 900 seconds ≈ 11,000 refresh calls/second
```

which is a serious load in its own right — and every one of those is a lookup against the revocable refresh-token store. **So the “stateless” system is doing 11,000 stateful lookups a second anyway.** Working that out out loud is the strongest possible version of this answer.

7 The trade-offs

Sessions

You get instant revocation, small requests, and the ability to see and end every active session — which users increasingly expect (“log out my other devices”).

You pay a lookup on every request, and a store every server must reach. If Redis is down, nobody is logged in — so it needs replication, and now you have a distributed system to operate.

JWTs

You get verification with no lookup and no shared store, which suits many services verifying independently, and works across service boundaries without each one calling an auth service.

You pay with revocation you do not have, tokens you cannot shorten once issued, a payload that can go stale — a role changed five minutes ago is still whatever the token says — and several hundred bytes on every request. And the failure modes are sharp: `alg: none`, an unchecked `aud`, a leaked `HS256` secret.

OAuth and OIDC

You get no password handling at all, users who trust the Google consent screen more than your login form, and a signup with no form to fill in.

You pay a dependency on someone else’s uptime and policy, a genuinely fiddly flow with real security pitfalls, and a user who loses their Google account losing yours. And you still need your own account model underneath, because users will want to link two providers and still be one person.

The sentence that separates candidates

I would not use JWTs for a first-party web session. Sessions with Redis are simpler, revocation is instant and free, the store is 400 MB for ten million users, and the extra half-millisecond is not the bottleneck in any product I have worked on. I reach for JWTs when the verifier genuinely cannot call back to a central store — many independent services, or a partner API — and even then I use short-lived access tokens with revocable refresh tokens, which means I am running a session store after all. **The honest framing is that “stateless auth” mostly moves the state rather than removing it.**

8 In the interview

How it gets asked

- “How do you keep a user logged in across requests?” — the direct version. HTTP is stateless, so something must be carried on each request.
- “Sessions or JWTs? Which would you pick and why?” — where the whole answer is revocation.
- “What’s inside a JWT? Is it encrypted?” — the check on whether you know base64 is not encryption.
- “Walk me through Log in with Google.” — asking for the authorisation-code flow, and quietly checking whether you know OAuth is authorisation and OIDC is authentication.
- “A user’s laptop is stolen. How do you log them out everywhere?” — the same question wearing a scenario, and a gift if you have prepared.

What to say out loud, in the first ninety seconds

1. **Start from the constraint.** *“HTTP is stateless, so the client has to present something on every request. The question is what that something is and where the truth about it lives.”*
2. **Give the two shapes in one sentence each.** *“Either an opaque session id that the server looks up in a shared store — so all the truth is server-side — or a signed token that carries the facts itself, so no lookup is needed.”*
3. **Name the trade immediately.** *“The trade is revocation against a lookup. Deleting a session is instant; a JWT is valid until it expires and there is no way to un-issue it.”*
4. **Say the numbers.** *“A session lookup is about half a millisecond of Redis; a signature check is about a tenth of a millisecond of CPU. Ten million users’ sessions is around 400 MB, so the ‘sessions don’t scale’ claim doesn’t really survive the arithmetic.”*
5. **State what you would build.** *“For a first-party web app, sessions in Redis. For many services or an external API, short-lived JWTs — 15 minutes — plus a revocable refresh token.”*
6. **Close the loop honestly.** *“And I’d point out that the refresh token is server-side and revocable, so the stateless design still has a session store in it. It moves the state rather than removing it.”*
7. **Mention the cookie flags, unprompted.** *“Whatever I carry it in, the cookie is HttpOnly, Secure and SameSite.”*

The follow-ups

“Is a JWT encrypted? Can I put a password in it?” No and absolutely not. A JWT is base64-encoded, not encrypted — anyone holding it can decode the payload and read every claim, which is what jwt.io does in a browser with no key at all. The signature guarantees **integrity**, not confidentiality: it proves nobody altered the contents, and does

nothing to hide them. So a JWT payload should contain only what you would be content to print on the outside of an envelope — a user id, a role, an expiry. Never a password, never a card number, never a personal detail you would not want in a log file, because that token will end up in logs and in browser storage. There is an encrypted variant, JWE, but it is rare and if you need confidentiality you almost certainly want an opaque token and server-side state instead.

“A user’s laptop is stolen. Log them out everywhere. How?” With sessions, trivially: delete every session record for that user id, and the very next request from any device gets a `401`. That is one of the strongest practical arguments for sessions, and it is what “log out all other devices” in a settings page actually does. With JWTs it is genuinely hard, because validity is a property of the token and there is no list to remove it from. The realistic answers are: keep access tokens short — 5 to 15 minutes — and revoke the refresh token, which guarantees the user is out within one token lifetime; or maintain a denylist of revoked token ids that every request checks, which works and reinstates a lookup on every request, which is the thing JWTs were chosen to avoid. A third option is a per-user `token_version` in the payload that you increment to invalidate every outstanding token at once — still a lookup, but a very cheap one. I would be explicit that all three trade away the statelessness.

“Walk me through Log in with Google.” It is OpenID Connect, which is OAuth 2.0 plus an identity layer. My app redirects the browser to Google with a client id, a redirect URI, the scopes I want, and a PKCE challenge. Google authenticates the user itself — I never see the password — and shows a consent screen. It then redirects back to my redirect URI with a short-lived, single-use authorisation code. My **backend** exchanges that code, along with the client secret and the PKCE verifier, for tokens over a direct server-to-server call. I get an access token for calling Google APIs and, because this is OIDC, an `id_token`, which is a JWT telling me who the user is. I verify that token’s signature against Google’s published public keys and check `iss` and `aud`, then look up or create a local user record and issue my own session. The reason for the code step rather than returning the token directly is that the redirect travels through the browser, where URLs land in history and logs — a single-use code is worthless without the client secret, whereas a token would not be.

“What’s the difference between OAuth and OIDC?” OAuth 2.0 is an authorisation framework: it exists to let an application act on a user’s behalf at another service with a limited scope, without ever handling the user’s password. Its output is an access token, and an access token tells you what you may do, not who anybody is. People used it for login anyway by fetching a profile after getting the token, which worked but was ad hoc and every provider did it differently. OpenID Connect standardises that: same flow, plus an `id_token` that is a JWT with standard claims about the user, plus a `/userinfo` end-

point and a discovery document. So **OAuth is authorisation, OIDC is authentication built on top of it**, and “Log in with Google” is OIDC even though almost everyone calls it OAuth.

A model answer

“HTTP is stateless — each request arrives knowing nothing about the last one — so after the user logs in, something has to be presented on every subsequent request. There are two shapes, and the difference is where the truth lives.

With a **session**, the server keeps a record of the logged-in user and hands the client only a long random id, in an HttpOnly, Secure, SameSite cookie. That id is meaningless on its own. On each request the server looks it up — Redis, typically — and gets back who the user is. All the truth is server-side.

With a **JWT**, the facts are in the token: a base64 payload with the user id, a role and an expiry, signed with a key only the issuer has. The server verifies the signature and the expiry and knows who is calling with no lookup at all. The signature guarantees nobody altered it — but it is encoded, not encrypted, so anyone holding it can read every claim, which means nothing sensitive goes in there.

The trade is revocation against a lookup. Deleting a session logs someone out instantly, everywhere. A JWT cannot be un-issued: it is valid until it expires, so a banned user keeps working, and a stolen token stays good until the clock runs out. The standard mitigation is a short-lived access token — 15 minutes — plus a long-lived refresh token that *is* stored server-side and *is* revocable. Which is worth being honest about: that design has a session store in it. Stateless auth mostly moves the state rather than removing it.

On the numbers: a Redis lookup is about half a millisecond, a signature check about a tenth of a millisecond of CPU. Ten million users at 20% concurrency is roughly 400 MB of session data, so the claim that sessions don’t scale doesn’t really survive the arithmetic — they cost a network hop and a dependency, not capacity. Against that, a JWT is around 800 bytes on every request rather than 32, which at ten thousand requests a second is a few hundred gigabytes a day of pure header.

So what I’d build: for a first-party web application, sessions in Redis. Simpler, instant revocation, and ‘log out my other devices’ just works. For an architecture where many independent services need to verify without calling back to a central auth service, or for an external API, short-lived JWTs signed with RS256 — so services can verify with the public key without being able to issue — plus revocable refresh tokens.

And if the login is delegated — Log in with Google — that’s OpenID Connect over the OAuth2 authorisation-code flow with PKCE. My backend exchanges a single-use code for an id_token, verifies it against Google’s public keys, checks the issuer and

audience, and then issues my own session. I would not hand Google's token to my own clients; I convert it into my own credential at the boundary."

9 Recall card

- **Session = an opaque id the server looks up.** All truth server-side; revocation is one delete.
- **JWT = signed, self-describing, verified with no lookup.** Base64 is **not** encryption — nothing sensitive in the payload.
- **The whole trade is revocation.** A JWT cannot be un-issued; the fix is short expiry plus a revocable refresh token — which is a session store again.
- **OAuth is authorisation, OIDC is authentication.** "Log in with Google" is OIDC over the authorisation-code flow with PKCE.
- **Cookies:** `HttpOnly`, `Secure`, `SameSite`. JWTs: `RS256`, check `exp` / `iss` / `aud`, never trust `alg` from the token.

DSA topic: Building strings without the quadratic trap **System design topic:** JWT, sessions, and OAuth

Code these, in this order

Four problems that all produce a string as their answer. In every one of them, say the words “*I’ll collect into a list and join*” **before** you write the loop. That sentence is the skill.

Before each one, say out loud:

1. Does the signature give me `str` or `list[str]` ? Which technique is that asking for?
2. How long is the output, and how many times will each character be copied?
3. Is there a “last group” that ends by running out rather than by changing?
4. What happens on the empty input?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Reverse Words in a String III	LeetCode 557 (Easy)	<code>split</code> , transform each word, <code>join</code> . Two minutes if you use the pattern, and a nested-loop <code>+=</code> disaster if you do not.
2	Add Strings	LeetCode 415 (Easy)	Digit-by-digit addition with a carry, built backwards into a list and reversed before joining. The carry after the loop is the same “flush the last thing” reflex as the run-length encoder.
3	String Compression	LeetCode 443 (Medium)	The signature is <code>list[str]</code> , so this is the write pointer in $O(1)$ space, not a join problem. Solve it wrong on purpose first, then right.
4	Encode and Decode Strings	LeetCode 271 (Medium)	Designing your own format. Length-prefixing beats delimiters, and working out why is the whole exercise.

On problem 3, do it both ways deliberately

First write the `join` version — walk the list, build parts, return `"".join(parts)` — and confirm it produces the right characters. Then read the problem statement again and notice it wants the answer **inside the input list**, with the new length returned.

Then write the write-pointer version. Then answer, out loud:

1. Which version uses $O(n)$ extra space and which uses $O(1)$?
2. Why is the in-place write always safe? Give the run-length argument.
3. What does a run of 12 produce, and how many cells does it occupy?

4. What does a run of 1 produce, and why is that special-cased?

On problem 4, design before you code

Try the obvious answer first — join the strings with a separator like `#` — and then find the input that breaks it. It will not take long: a string that *contains* a `#`.

Then design the length-prefixed format: write each string as its length, a delimiter, then the string itself, so `["ab", "c#d"]` becomes `2#ab3#c#d`. Say out loud why the delimiter inside `c#d` is now harmless. That reasoning — *the length tells the reader exactly how far to read, so the content can be anything* — is the same reasoning behind every binary protocol you will meet later.

The pattern drill

Rewrite each of these using the list-and-join pattern, then say what the original cost was:

```
# 1
out = ""
for row in rows:
    for cell in row:
        out += str(cell) + ","
    out += "\n"
```

```
# 2
result = ""
for word in words:
    result += word + ", "
result = result[:-2]
```

```
# 3
html = "<ul>"
for item in items:
    html += "<li>" + item + "</li>"
html += "</ul>"
```

For number 2, also say what goes wrong when `words` is empty — and why `','.join(words)` has no such problem.

The measurement drill

Run this and read the numbers.

```

import io, time

n = 200_000

def timed(f):
    start = time.perf_counter(); f(); return time.perf_counter() - start

def grow():
    s = ""
    for _ in range(n):
        s = "x" + s

def listjoin():
    p = []
    for _ in range(n):
        p.append("x")
    "".join(p)

def stringio():
    b = io.StringIO()
    for _ in range(n):
        b.write("x")
    b.getvalue()

for name, f in (("grow", grow), ("list+join", listjoin), ("StringIO", stringio)):
    print(f"{name:12} {timed(f):.4f}s")

```

Then answer:

1. What is the ratio between the first and second?
2. How many character copies does `grow` do in total at `n = 200,000`?
3. Why are `list+join` and `StringIO` almost identical?
4. Name the one situation where you would choose `StringIO` over `list+join`.

The error drill

Predict, then run:

```
print(", ".join([3, 1, 4]))
```

```
print("count: " + 5)
```

```
print("-".join("abc"))
```

For the third: there is no error and the output is probably not what was intended. Say what happened and why it is dangerous.

The auth-mechanism drill

Answer each in one or two sentences, out loud:

1. HTTP is stateless. What does that force the client to do on every request?
2. What is actually inside a session cookie, and what is inside a JWT?
3. Is a JWT encrypted? What does the signature guarantee, and what does it not?
4. Name the three cookie flags and the attack each prevents.
5. Why `RS256` rather than `HS256` once more than one service verifies tokens?
6. What is the `alg: none` attack, and what is the fix?
7. Why does the OAuth flow return a code and then exchange it, rather than returning the token straight away?
8. OAuth or OIDC — which one is “Log in with Google”, and which is authorisation?

The revocation drill

This is the question the whole topic exists for. Take each scenario and say what happens under sessions, and what happens under JWTs:

1. A user clicks “log out”.
2. A user clicks “log out of all other devices”.
3. An admin bans an account for abuse at 14:32.
4. A user changes their password after their laptop is stolen.
5. A moderator is demoted to ordinary member.

Then answer: for scenario 3, if access tokens live 15 minutes, when is the banned user actually out? And what would you change to make it faster, and what does that change cost you?

The arithmetic drill

From memory, in under two minutes, for an API at 10,000 requests a second:

- Session lookup at 0.5 ms — how much waiting per second, and is that a problem for Redis?
- JWT verification at 0.1 ms of CPU — how many cores?
- 800-byte JWT versus a 32-byte cookie — how much extra traffic per day?
- 10 million users, 20% with a live session at 200 bytes — how much Redis?
- 10 million users on 15-minute access tokens — how many refresh calls per second, and why does that number undercut the word “stateless”?

The last one is the strongest thing you can say in this topic. Have it ready.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Build a string of n characters efficiently.* Immutability, then the count $1 + 2 + \dots + n$, then the fix, then why `join` is linear — one pass to measure, one allocation, one copy per character. Finish with the cross-language point: `StringBuilder` in Java, `strings.Builder` in Go, the same idea.
 2. *How do you keep a user logged in across requests?* Start from statelessness. Two shapes, one sentence each. Then the trade — revocation against a lookup — then the numbers, then what you would actually build and why. Finish with the honest line: short-lived tokens plus revocable refresh tokens is a session store with extra steps.
 3. *Compress the string:* `aabbccc` becomes `a2b2c3`. Read the signature out loud first and say what it implies. Then the loop that consumes a whole run at a time, then the flush-the-last-group point, then why the in-place write can never overtake the read.
-

Before you move on

- [] I say “collect into a list and join” before writing any loop that produces a string.
- [] I can count out loud why growing a string is $O(n^2)$ and joining is $O(n)$.
- [] I check whether the signature says `str` or `list[str]` before choosing a technique.
- [] I flush the last group after any loop that groups consecutive items.
- [] I never build separators by hand — `join` handles the last element.
- [] I can explain a session and a JWT in one sentence each, and name the trade between them.
- [] I know a JWT is encoded, not encrypted, and what the signature actually guarantees.
- [] I can walk through the OAuth authorisation-code flow and say why the code step exists.
- [] I can redraw the where-the-truth-lives diagram from memory, in whatever tool I like.

Character counting and frequency maps

DATA STRUCTURES AND ALGORITHMS

Character counting and frequency maps

You reach for a counter dictionary the moment a problem says how many times.

SYSTEM DESIGN

GraphQL versus REST

You can argue for either one, given a real product constraint.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the first non-repeating character in a string.*
- *When would you choose GraphQL over REST?*

Character counting and frequency maps

1 What this is, and why they ask it

A **frequency map** is a dictionary from each distinct thing to how many times it appeared. Walk the input once, adding one to the right entry each time, and afterwards you can answer *how many of each?* instantly instead of re-scanning.

That is the whole idea, and it is worth an entire day because of how often it is the answer. Any question containing the words **how many times**, **most common**, **appears once**, **duplicate**, **can we make X from Y**, or **do these contain the same letters** is a frequency-map question, and recognising that in five seconds is a large fraction of what interviewers are measuring.

The specific question today — the first non-repeating character — is asked constantly, at Amazon and Microsoft especially, because it has a small trap in it. The obvious answer is to check each character against the rest of the string, which is $O(n^2)$. The good answer is two passes with a counter, which is $O(n)$. And the interesting part is *why two passes are unavoidable*: you cannot know a character is unique until you have seen the entire string, because the second copy might be the very last character. Candidates who try to force it into one pass are answering a question that has no answer, and saying that clearly earns more than the code does.

This also opens the door to the whole hashing phase from [day 060](#), and it is the tool behind anagrams tomorrow and sliding windows on [day 033](#).

2 The story

Bhaskar has been a conductor on the 27C for nine years. It runs from the depot to the market and back, twelve times a day, and on a weekday he will take money from something like four hundred people.

The thing that makes his job possible is the tray. It is a shallow metal box with a lid, divided into six compartments, and each compartment holds one kind of coin — ones in the first, twos in the second, fives, tens, and two bigger sections at the back for the notes. When somebody hands him a five-rupee coin it goes into the fives, straight away, without him thinking about it. He does that four hundred times a day and he does not look while he does it.

The first year, before he had the tray, he used the cloth pouch everyone starts with, and everything went into the pouch together. At the end of the shift he would sit on the step of the bus at half past nine at night and tip the whole thing out and sort it, and it took him twenty minutes and he got it wrong about once a week. Now the sorting has already happened, all day, one coin at a time, and at the end he only has to count what is already in each compartment. Four minutes.

The clerk at the depot asked him something odd last month. She was doing some kind of study and she wanted to know which kinds of coin he had received exactly one of, all day.

Bhaskar's answer is the interesting part. He said he could tell her — but not until the day was over. At two in the afternoon he might have exactly one twenty-rupee note in the tray, and it would be perfectly true at that moment that he had received only one. But somebody could hand him a second one at eight in the evening, and then the answer changes. There is no point in the day when he can look into the tray and say *this one is the only one I will get*, because the day is not finished with him yet.

So he does what she asked at the end of the shift, when everything has been through his hands, and then it takes him about thirty seconds.

3 The idea in plain English

Bhaskar's tray is a frequency map. Each compartment is a key, the number of coins in it is the value, and dropping a coin in is the one line of code you write in the loop.

Building one

The idea is the same three ways; only the syntax differs.

With a plain dictionary and `get` :

```
counts = {}
for ch in s:
    counts[ch] = counts.get(ch, 0) + 1
```

`counts.get(ch, 0)` returns the current count, or `0` if this character has never been seen. That default is the whole trick — without it, the first time you touch a new key you get a `KeyError`, and §7 shows it.

With `defaultdict` :

```
from collections import defaultdict

counts = defaultdict(int)
for ch in s:
    counts[ch] += 1
```

`defaultdict(int)` creates a missing key with the value `int()`, which is `0`, the moment you touch it. Cleaner to read, and you met it on [day 016](#).

With `Counter`, which is what you would actually write:

```
from collections import Counter

counts = Counter(s)          # one line, does the whole loop
```

`Counter` is a dictionary subclass built for exactly this. It counts any iterable in one call, and it never raises for a missing key — `counts["z"]` on a string with no `z` returns `0`, not an error.

Which to use in an interview? Write `Counter`, and be able to write the manual loop if asked. Interviewers occasionally say “*without using Counter*” precisely to see whether you know what it is doing.

The other representation: a fixed-size array

When the alphabet is small and known — the 26 lowercase English letters — you can use a list of 26 integers instead of a dictionary, mapping each character to a position with `ord`.

```
counts = [0] * 26
for ch in s:
    counts[ord(ch) - ord("a")] += 1
```

`ord(ch)` gives the character’s numeric code — `ord("a")` is 97, `ord("b")` is 98. Subtracting `ord("a")` turns `"a"` into 0, `"b"` into 1, and so on up to `"z"` as 25.

This is faster than a dictionary by a constant factor and uses fixed memory, and interviewers like it because it shows you noticed the constraint. It is also **fragile**: it silently breaks on capital letters, spaces, digits and anything non-English, and §7 shows exactly how silently. Use it only when the problem promises lowercase English letters, and say so out loud when you do.

Why the answer needs two passes

Here is Bhaskar at two in the afternoon, and it is the heart of today’s question.

To find the first character that appears exactly once, you must:

1. **Pass one:** count every character. Now `counts` is complete.
2. **Pass two:** walk the string again from the start, and return the position of the first character whose count is 1.

You cannot merge these. Halfway through the string, a character you have seen once may be about to appear again — you have no way to know until you reach the end. **Uniqueness is a property of the whole string, so it cannot be decided from a prefix.** That is Bhaskar's twenty-rupee note.

Pass two must go in **string order**, not dictionary order, because the question asks for the *first*. (Python dictionaries do happen to preserve insertion order, so iterating `counts` would in fact give the same answer here — but relying on that is fragile reasoning, and walking the string is clearer about what you mean.)

Recognising the pattern

The tell-tale phrases, and what each one becomes:

THE PROBLEM SAYS	YOU BUILD	THEN
“appears exactly once”	a count of every character	scan in order for a count of 1
“most common” / “top k”	a count	<code>most_common(k)</code>
“can you build A from B”	a count of B	check every count in A is available
“same characters”	counts of both	compare the two dictionaries
“how many distinct”	a set , not a count	<code>len(set(s))</code>
“at most k distinct in a window”	a count, updated as the window moves	day 033

Note the fifth row. When you only care *whether* something appeared and not how often, a set is the right structure and is cheaper. Reaching for a counter when a set will do is a small but real signal.

4 The picture

Building the map for `"loveleetcode"`, one character at a time:

```

character:  l  o  v  e  l  e  e  t  c  o  d  e
            |  |  |  |  |  |  |  |  |  |  |
after each step, the map holds:

l → {l:1}
o → {l:1, o:1}
v → {l:1, o:1, v:1}
e → {l:1, o:1, v:1, e:1}
l → {l:2, o:1, v:1, e:1} ← l is no longer unique
e → {l:2, o:1, v:1, e:2}
e → {l:2, o:1, v:1, e:3}
t → {l:2, o:1, v:1, e:3, t:1}
c → {l:2, o:1, v:1, e:3, t:1, c:1}
o → {l:2, o:2, v:1, e:3, t:1, c:1} ← o is no longer unique
d → {l:2, o:2, v:1, e:3, t:1, c:1, d:1}
e → {l:2, o:2, v:1, e:4, t:1, c:1, d:1}

```

What to notice: `o` was unique for eight steps and then stopped being unique at step ten. Any decision made before the end would have been wrong. That is why there are two passes.

Now pass two, walking the original string with the finished map:

```

position:  0    1    2    3    4    5    6    7    8    9   10   11
character:  l    o    v    e    l    e    e    t    c    o    d    e
count:      2    2    1    4    2    4    4    1    1    2    1    4
            |    |    ^
            |    |    +--- first count of 1 → answer is position 2
            |    +----- 2, keep going
            +----- 2, keep going

```

What to notice: the answer is 2, not 7 — `t` also has a count of 1, but `v` comes first. Walking in string order is what makes “first” mean the right thing.

The other representation, for lowercase-only input:

```

index      0    1    2    ...    4    ...    11    ...    14    ...    19    ...    25
value      +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
           | 0 | 0 | 1 | | 4 | | 2 | | 2 | | 1 | | 0 |
           +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
           a    b    c          e    l          o          t          z

position = ord(ch) - ord('a')

```

What to notice: there is one box for every possible letter, including the ones that never appeared. That is the trade — fixed, predictable memory, and it only works when the alphabet is fixed and known.

5 The code, built step by step

The count, three ways

```
counts = {}
for ch in s:
    counts[ch] = counts.get(ch, 0) + 1
```

The version to write if asked not to use the library. `get` with a default is the load-bearing part.

```
from collections import defaultdict
counts = defaultdict(int)
for ch in s:
    counts[ch] += 1
```

Cleaner. One caution: a `defaultdict` **creates** the key when you read a missing one, so `if counts[x] == 0` quietly inserts `x`. Use `x in counts` to test membership.

```
from collections import Counter
counts = Counter(s)
```

What you write in practice. It works on any iterable — a string, a list, a generator.

The second pass

```
for i, ch in enumerate(s):
    if counts[ch] == 1:
        return i
return -1
```

`enumerate(s)` yields `(position, character)` pairs, which is exactly what “return the index” needs. The `return -1` at the end is the contract for “there is no such character” — ask which sentinel the interviewer wants; some versions of the problem want the character itself, or `None`.

The two together

```
def first_uniq_char(s: str) → int:
    counts = Counter(s)
    for i, ch in enumerate(s):
        if counts[ch] == 1:
            return i
    return -1
```

Four lines, `O(n)`, and it is the complete answer to LeetCode 387.

Counter's useful extras

```
c = Counter("hello world")
c.most_common(3)          # [('l', 3), ('o', 2), ('h', 1)]
c["z"]                    # 0 - never a KeyError
Counter("aabbc") - Counter("abc")    # Counter({'a': 1, 'b': 1})
Counter("aab") == Counter("aba")     # True
```

`most_common(k)` returns the `k` highest counts, already sorted. Subtraction keeps only positive counts, which makes “can I build A out of B?” a one-liner — that is LeetCode 383 below. And equality compares the whole mapping, which is the entire anagram check on [day 022](#).

The fixed-array version, written safely

```
def first_uniq_char_array(s: str) → int:
    counts = [0] * 26
    for ch in s:
        counts[ord(ch) - ord("a")] += 1    # lowercase a-z ONLY
    for i, ch in enumerate(s):
        if counts[ord(ch) - ord("a")] == 1:
            return i
    return -1
```

Faster by a constant factor, `O(1)` space by any sensible reading, and correct **only** if the input really is lowercase English. Say that constraint out loud before you use it.

The complete solutions

```
from collections import Counter, defaultdict

def count_chars_manual(s: str) → dict[str, int]:
    """The version to write when asked not to use Counter."""
    counts: dict[str, int] = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1 # get with a default, never counts[ch]
    return counts

def count_chars_defaultdict(s: str) → dict[str, int]:
    """Same thing. defaultdict(int) supplies 0 for a missing key."""
    counts: defaultdict[str, int] = defaultdict(int)
    for ch in s:
        counts[ch] += 1
    return dict(counts)

def first_uniq_char(s: str) → int:
    """LeetCode 387. Index of the first character appearing exactly once, else -1.

    Two passes: you cannot know a character is unique until the string has ended.
    """
    counts = Counter(s)
    for i, ch in enumerate(s): # string order, because "first" means first
        if counts[ch] == 1:
            return i
    return -1

def first_uniq_char_array(s: str) → int:
    """The same, using 26 slots instead of a dictionary. Lowercase a-z only."""
    counts = [0] * 26
    for ch in s:
        counts[ord(ch) - ord("a")] += 1
    for i, ch in enumerate(s):
        if counts[ord(ch) - ord("a")] == 1:
            return i
    return -1

def can_construct(note: str, magazine: str) → bool:
    """LeetCode 383. Can `note` be built from the letters of `magazine`?"""
    return not (Counter(note) - Counter(magazine)) # empty means nothing was missing

def most_common_char(s: str) → str | None:
    """The most frequent character. Ties broken by first appearance, as Counter does."""
    if not s:
        return None
    return Counter(s).most_common(1)[0][0]

def count_distinct(s: str) → int:
    """When you only care WHETHER, not HOW MANY – a set is cheaper than a counter."""
    return len(set(s))

if __name__ == "__main__":
    print(count_chars_manual("hello")) # {'h': 1, 'e': 1, 'l': 2, 'o': 1}
    print(first_uniq_char("leetcode")) # 0
    print(first_uniq_char("loveleetcode")) # 2
```

```

print(first_uniq_char("aabb"))      # -1
print(first_uniq_char(""))         # -1
print(first_uniq_char("z"))        # 0

print(first_uniq_char_array("loveleetcode")) # 2

print(can_construct("aa", "aab"))    # True
print(can_construct("aa", "ab"))     # False

print(most_common_char("hello world")) # 1
print(count_distinct("hello"))       # 4

```

6 What it costs

`first_uniq_char`

Pass one. `Counter(s)` visits each of the n characters once. Each visit is one dictionary lookup and one increment, both $O(1)$ on average — that is the promise of a hash table, and [day 060](#) explains why. So pass one is n turns of constant work.

Pass two. At most n characters, one dictionary lookup each. Another n turns of constant work.

Total: $2n$ turns, so $O(n)$ time. Two passes over the data is still linear — a common misunderstanding is that two passes means $O(n^2)$, and being clear that it does not is worth a mark.

Space. The dictionary holds one entry per **distinct** character. Call that k :

- For arbitrary text, $k \leq n$, so $O(k)$, and $O(n)$ in the worst case where every character differs.
- For lowercase English, $k \leq 26$ no matter how long the string is. That is a constant, so it is $O(1)$ **space** — and saying “ $O(1)$ because the alphabet is bounded at 26” is exactly the sentence interviewers want here.
- For full Unicode, k can be over a million in principle, though never more than n in practice.

Against the naive version

```

for i, ch in enumerate(s):
    if s.count(ch) == 1:      # scans the whole string, every time
        return i

```

`s.count(ch)` is $O(n)$, done up to n times: $O(n^2)$. At $n = 100,000$ that is 10 billion character comparisons against 200,000 for the counter version — fifty thousand times more work. It is also two lines shorter, which is why it gets written.

Dictionary against array of 26

Both are $O(n)$ time. The array wins on the constant factor: an integer subtraction and a direct index, against hashing the character and probing a table. In CPython the gap is usually somewhere around two to three times. It also uses a fixed 26 slots regardless of input.

The dictionary wins on generality — it handles any character with no changes, and it stores only what actually appeared. **Say both, then pick the dictionary unless the problem states the alphabet.**

The number to have ready

Counting is one pass, $O(n)$. Finding the first unique character is two passes, still $O(n)$. The naive “count each character against the whole string” version is $O(n^2)$ — 10 billion operations at a hundred thousand characters, against 200,000.

7 The traps

The real error: touching a key that is not there

```
counts = {}
for ch in "hello":
    counts[ch] += 1
```

```
Traceback (most recent call last):
  File "t.py", line 3, in <module>
    counts[ch] += 1
    ~~~~~^
KeyError: 'h'
```

`counts[ch] += 1` reads before it writes, and on the first `h` there is nothing to read. Three fixes, all fine: `counts.get(ch, 0) + 1`, a `defaultdict(int)`, or a `Counter`.

The near-miss that never raises: `ord` with the wrong alphabet

```
def counts26(s):
    arr = [0] * 26
    for ch in s:
        arr[ord(ch) - ord("a")] += 1
    return arr

print(counts26("Hello"))
```

Predict the output. You may expect an `IndexError`. Here is what actually happens:

```
[0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

No error. `ord("H")` is 72 and `ord("a")` is 97, so `72 - 97` is `-25` — and a negative index is perfectly legal in Python, counting back from the end. Position `-25` of a 26-element list is position 1, so the capital `H` was silently counted as a `b`. Look at the output: index 1 has a count of 1 and index 7, which is where `h` belongs, has 0.

This is the negative-index trap from [day 016](#) in a new costume, and it is worse here because the answer looks completely plausible. **Whenever you use `ord(ch) - ord("a")`, state the assumption out loud and, in real code, normalise first with `s.lower()` or guard with `if ch.isalpha()`.**

The near-miss: trying to do it in one pass

```
def first_uniq_char(s):
    seen = set()
    for i, ch in enumerate(s):
        if ch not in seen:
            seen.add(ch)
            return i          # wrong: this is just the first character
    return -1
```

Every attempt to answer this in one pass has the same flaw. At position `i` you simply do not know whether the character repeats later. There is no clever bookkeeping that fixes it, because the information does not exist yet. **The right response in an interview is to say so** — “*this needs two passes, because uniqueness is a property of the whole string*” — rather than to search for a one-pass version that cannot exist.

(The related problem that *does* work in one pass is “*return the first character that has not repeated so far, after each new character*” — a stream question, which needs a queue as well as a counter. If an interviewer pushes towards one pass, this is probably the question they are steering towards.)

The real error: changing a dictionary while looping over it

```
d = {"a": 1, "b": 2, "c": 3}
for k in d:
    if d[k] == 2:
        del d[k]
```

```
RuntimeError: dictionary changed size during iteration
```

Same family as deleting from a list while iterating on [day 015](#), and here Python catches it rather than silently skipping. Loop over a copy — `for k in list(d)` — or build a new dictionary with a comprehension.

The near-miss: `defaultdict` inserting on read

```
from collections import defaultdict
counts = defaultdict(int)
counts["a"] += 1
if counts["z"] == 0:      # this INSERTS 'z' with value 0
    pass
print(dict(counts))      # {'a': 1, 'z': 0}
```

Reading a missing key from a `defaultdict` creates it. Usually harmless, occasionally not — a later `len(counts)` or a loop over the keys now sees a character that never appeared. Use `"z" in counts` to test membership, or use `Counter`, which returns 0 for a missing key **without** inserting it.

The near-miss: counting when you meant to check membership

```
counts = Counter(s)
if counts[ch] > 0: ...    # you only wanted to know IF
```

If the question is “does this appear at all”, a set is the right structure: `set(s)` builds faster, uses less memory, and says what you mean. Using a counter for a membership test works and reads as though you had not decided what you needed.

8 In the interview

How it gets asked

- “Find the first non-repeating character in a string.” — LeetCode 387, the direct version. Ask whether they want the index or the character.
- “What’s the most frequent character / word?” — `most_common`, plus a conversation about ties.
- “Can you build this string from the letters in that one?” — LeetCode 383, the counter-subtraction question.
- “Find the first non-repeating character in a stream.” — the harder relative, needing a counter **and** a queue. If they say “stream”, that is the signal.

What to say out loud, in the first ninety seconds

1. **Pin the contract.** *“Should I return the index or the character? What do I return if every character repeats — minus one, or None? And can I assume lowercase English letters, or arbitrary Unicode?”* That third question decides dictionary versus array of 26.
2. **Name the brute force and its cost.** *“The obvious version checks each character against the whole string, which is $O(n^2)$.”*
3. **State the structure.** *“I’ll build a frequency map in one pass — character to count — then walk the string again and return the first character whose count is one.”*
4. **Say why two passes, before being asked.** *“It has to be two passes. A character that has appeared once so far might appear again at the very last position, so uniqueness can’t be decided from a prefix.”* This is the sentence the question exists to elicit.
5. **Say why pass two walks the string.** *“The second pass goes over the string, not the dictionary, because the question asks for the first one in the original order.”*
6. **Give the cost precisely.** *“Two passes, so $O(n)$ time — two passes is still linear. Space is one entry per distinct character; if the alphabet is the 26 lowercase letters that’s bounded, so $O(1)$.”*
7. **Offer the alternative representation.** *“If it’s guaranteed lowercase I’d use a 26-element array instead of a dictionary — same complexity, a smaller constant.”*

The follow-ups

“**Can you do it in one pass?**” Not for this problem, and I think the honest answer is more useful than an attempt. At any position in the string, a character I have seen exactly once might still appear again later — the second copy could be the final character — so uniqueness genuinely is not decidable from a prefix, and no bookkeeping fixes that because the information does not exist yet. What I *can* do is one pass to build the counts and one pass over the counts rather than the string, but that is still two passes and it loses the ordering unless I rely on dictionaries preserving insertion order, which I would rather not lean on. If the question is instead about a **stream** — report the first non-repeating character seen *so far*, after each new character — that does work incrementally, and I would keep a counter plus a queue of candidates, popping from the front of the queue whenever its count rises above one.

“**What if the string is huge and can’t fit in memory?**” The counter still works, because it is bounded by the number of *distinct* characters rather than the length — for text that is at most a few thousand entries however many gigabytes the file is. So pass one streams the file and builds the counts in constant memory. The problem is pass two, which needs the original order again; I would either re-read the file, which is fine because a sequential read is cheap, or record the first position of each character during pass one and then take the minimum position among characters with count 1, which

turns pass two into a scan of the small dictionary rather than of the data. The second option is one pass over the data, which matters when the data is on a network or a tape rather than a disk.

“Now find the first non-repeating character in a stream.” Counter plus a queue. Maintain the frequency map as before, and a queue holding candidates in arrival order. For each new character: increment its count, and push it onto the queue. Then, before answering, pop from the front of the queue while the front character’s count is above one — those can never be the answer again. Whatever is at the front is the current answer, or the stream has no unique character if the queue is empty. Each character is pushed once and popped at most once, so it is $O(1)$ amortised per character and $O(k)$ space for the alphabet. That is the genuine one-pass version, and it works only because the *question* changed to allow an answer that evolves.

“How would you handle Unicode, or count words instead of characters?” Nothing changes structurally — a dictionary keyed by whatever the unit is. The array-of-26 trick disappears, which is the practical difference: `ord(ch) - ord('a')` is meaningless outside a known alphabet and, as I showed, fails silently rather than loudly because negative indices are legal. For Unicode I would also normalise first, because the same visible character can have more than one encoding — `é` as one code point or as `e` plus a combining accent — and those would count as different keys. For words, I would split and count the words, and decide explicitly whether case and punctuation matter, because “The” and “the” being one word or two is a product decision, not a technical one.

A model answer

“First, two clarifications: do you want the index or the character itself, and what should I return if there isn’t one — minus one, or None? And can I assume the string is lowercase English letters, or should I handle arbitrary Unicode?”

...Index, minus one, arbitrary characters. Good, I’ll use a dictionary rather than a fixed array.

The brute force is to take each character and count its occurrences in the whole string, returning the first with a count of one. That’s correct and $O(n^2)$, since counting is a full scan done up to n times.

Instead I’ll count once. One pass over the string building a map from character to how many times it appeared. Then a second pass over the string, in order, returning the index of the first character whose count is one.

```
def first_uniq_char(s: str) → int:
    counts = Counter(s)
    for i, ch in enumerate(s):
        if counts[ch] == 1:
            return i
    return -1
```

Two things worth calling out. First, this genuinely has to be two passes. A character that has appeared once so far might appear again at the very last position, so I can’t decide uniqueness from a prefix — the information doesn’t exist until the string ends. I mention that because the instinct is to look for a one-pass version, and there isn’t one for this phrasing.

Second, the second pass walks the *string*, not the dictionary, because the question asks for the first in the original order. Python dictionaries do preserve insertion order so iterating the map would happen to give the same answer, but I’d rather the code say what I mean than rely on that.

Cost: two passes over n characters with $O(1)$ dictionary operations, so $O(n)$ time — and two passes is still linear, not quadratic. Space is one entry per distinct character. For arbitrary input that’s $O(k)$ where k is the alphabet size, bounded by n ; if you’d told me it was lowercase English, k is at most 26 and it’s $O(1)$, and I’d have used a 26-element array indexed by `ord(ch) - ord('a')` for a smaller constant factor.

On `loveleetcode` the counts are l:2, o:2, v:1, e:4, t:1, c:1, d:1, and walking the string gives position 2 for the `v` — not position 7 for the `t`, even though both are unique, because `v` comes first.

Edge cases: empty string returns minus one because the second loop never runs; a single character returns 0; and a string where everything repeats falls through to minus one.”

9 Recall card

- “How many times”, “most common”, “appears once”, “duplicate” → **build a frequency map.**
- `Counter(s)` in practice; `counts.get(ch, 0) + 1` or `defaultdict(int)` when asked to do it by hand. Plain `counts[ch] += 1` is a `KeyError`.
- **First-unique needs two passes.** Uniqueness is a property of the whole string. Two passes is still $O(n)$.
- **Pass two walks the string, not the map**, because “first” means first in the input.
- `ord(ch) - ord("a")` is $O(1)$ **space and silently wrong on anything but lowercase a–z** — negative indices do not raise.

1 What this is, and why they ask it

GraphQL is a way of building an API where the **client** says exactly which fields it wants, and the server returns exactly those and nothing else. There is one endpoint instead of many, and the shape of the response is decided by the caller rather than fixed by the server.

It exists because of two specific complaints about REST, both of which you have already met. From [day 016](#): a REST resource is the unit, so `GET /users/42` returns the whole user even when the screen needs two fields — **over-fetching**. And from [day 015](#)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — **under-fetching**, usually called the $N + 1$ problem when it happens in a list.

Facebook built GraphQL in 2012 for exactly this, on mobile, and open-sourced it in 2015.

Interviewers ask this because it is a **trade-off question with no right answer**, which makes it a good test of judgement. Someone who says “GraphQL is more modern and flexible” has read the marketing. Someone who says “it moves complexity from the client to the server, and here is the caching you give up” has used it. The question is not which is better. It is whether you can name what each one costs.

2 The story

There is a lunch place near Rekha's office where about two hundred people from the surrounding buildings eat every day, and for nine years it worked one way. You paid at the counter, took a token, and at the hatch a man handed you a steel plate. The plate was the same for everyone: rice, two vegetables, a dal, a small sweet, two chapatis, curd, a piece of lemon and a spoon of pickle.

Most days that is roughly what Rekha wanted. Some days it was not. She almost never ate the sweet, and it went in the bin with the lemon and the pickle. On the days she wanted extra curd she had to go back and stand in the queue again, pay separately, and come back with a second little bowl. And her colleague who does not eat rice took the plate anyway and left half of it.

In February they changed it. Now there is a woman at the front with a tablet, and she asks you what you want on the plate. You say it once — rice, one vegetable, extra curd, no sweet, no pickle, three chapatis — and that exact plate comes out. One queue, one wait, and nothing in the bin.

Rekha likes it, and she has also watched what it did to the place.

The old way, the man at the hatch did not think. He handed over a plate, four hundred times, four seconds each. Now somebody has to listen to a different sentence from every person, and the kitchen has to be able to assemble any combination anybody asks for. They put on two more people at the back.

There is a line at the front now that did not exist before, because saying what you want takes twenty seconds and taking a plate took four. And one man asked for eleven chapatis, four curds and every vegetable they had, and the kitchen made it, because nobody had thought to say no. They have a rule about that now.

The one thing they genuinely lost is the shortcut they used to have. At half past twelve they would make forty identical plates in advance and stack them, and the first forty people got served instantly. They cannot do that any more. Every plate is different, so every plate has to be made when it is asked for.

3 The idea in plain English

The fixed steel plate is REST. The tablet at the front is GraphQL. Everything that follows — including the costs — is in that story.

What a REST call gives you

A REST endpoint returns a fixed shape decided by the server:

```
GET /users/42
→ { "id": 42, "name": "Rekha", "email": "...", "avatar_url": "...",
    "bio": "...", "created_at": "...", "settings": {...}, "location": "..." }
```

The screen needed `name` and `avatar_url`. It received nine fields. That is **over-fetching**, and it is the sweet and the pickle in the bin.

And if the screen also needs the user's last three orders and their unread notification count, that is two more calls:

```
GET /users/42
GET /users/42/orders?limit=3
GET /users/42/notifications?unread=true&count_only=true
```

Three round trips, done one after another because the second may depend on the first. That is **under-fetching**: no single endpoint gives you what one screen needs.

What a GraphQL call gives you

One endpoint — `POST /graphql` — and the request body is a **query** naming the fields:

```
query {
  user(id: 42) {
    name
    avatarUrl
    orders(limit: 3) {
      id
      total
      status
    }
    unreadNotificationCount
  }
}
```

And the response mirrors the query exactly:

```
{
  "data": {
    "user": {
      "name": "Rekha",
      "avatarUrl": "https:// ...",
      "orders": [
        { "id": "8812", "total": 340, "status": "delivered" },
        { "id": "8790", "total": 120, "status": "delivered" },
        { "id": "8654", "total": 890, "status": "cancelled" }
      ],
      "unreadNotificationCount": 3
    }
  }
}
```

One round trip. Nothing unwanted. **The response shape is a copy of the request shape**, which is the single most useful thing to say about GraphQL — you can predict the answer by reading the question.

The schema, which is the real product

A GraphQL API is defined by a **schema**: every type, every field, and every field's type, written down and strongly typed.

```

type User {
  id: ID!
  name: String!
  avatarUrl: String
  orders(limit: Int = 10): [Order!]!
  unreadNotificationCount: Int!
}

type Order {
  id: ID!
  total: Float!
  status: OrderStatus!
  items: [OrderItem!]!
}

type Query {
  user(id: ID!): User
  orders(status: OrderStatus): [Order!]!
}

```

`!` means non-null. `[Order!]!` is a non-null list of non-null orders. That schema is machine readable, so the client tooling can autocomplete queries, check them before they are sent, and generate types — which is a genuine day-to-day advantage over reading REST documentation and hoping.

Introspection — asking the API to describe itself — is built in, which is what powers those tools. It is also something you usually turn off in production, so that an attacker cannot enumerate your entire schema.

Resolvers: where the work actually happens

Each field in the schema has a **resolver** — a function that knows how to produce that field. The server walks the query, calls the resolvers it needs, and assembles the response.

This is the part people underestimate. The engine does not know that `orders` and `unreadNotificationCount` could be fetched from one database in one go; it calls each resolver independently. So a query for 50 users, each with their orders, naively runs **1 query for the users and 50 for the orders** — the N+1 problem, moved from the network into the database.

The standard fix is **DataLoader**: a per-request batching layer that collects all the ids requested in one tick of the event loop and issues a single `WHERE id IN (...)` query. Every serious GraphQL deployment uses one. Naming DataLoader in an interview is a strong signal, because it shows you know where the difficulty actually is.

The three operation types, and one thing REST does better

- **Query** — reads. All queries in one request run in parallel.
- **Mutation** — writes. Multiple mutations in one request run **in order**, deliberately.

- **Subscription** — a stream of updates over a websocket, for live data.

Note what is missing: HTTP semantics. **Every GraphQL request is a POST to one URL**, so there is no method, no resource path, and — the important one — no HTTP caching. A CDN cannot cache a POST to `/graphql`, and from [day 016](#)'s numbers, HTTP caching was the single largest lever in a read-heavy design. That is the forty pre-made plates.

There is also no meaningful use of status codes. A GraphQL server returns `200 OK` even when the query failed, with the problems listed in an `errors` array alongside a possibly partial `data`. That is exactly the thing [day 018](#) called the worst sin in API design, and here it is by specification — because a single request can partly succeed. It is defensible, and it does mean every monitoring tool you own needs teaching.

4 The picture

The same screen, both ways:

REST — three round trips, fixed shapes	GraphQL — one round trip, chosen shape
app server --- GET /users/42 -----> ← 9 fields (needed 2) ----- --- GET /users/42/orders ---> ← 20 fields (needed 9) ---- --- GET .../notifications ---> ← 6 fields (needed 1) ----- ~360 ms, ~12 KB cacheable at the CDN	app server --- POST /graphql -----> { user(id:42) { name avatarUrl orders(limit:3){..} unreadCount } } ← exactly those fields --- ~120 ms, ~2 KB not cacheable at the CDN

What to notice: the two numbers at the bottom disagree about which is better, and that is the whole lesson. GraphQL wins on round trips and bytes. REST wins on cacheability — and a cached response never reaches your servers at all, which is worth more than either of the savings above it on a read-heavy public site.

Where the complexity goes:

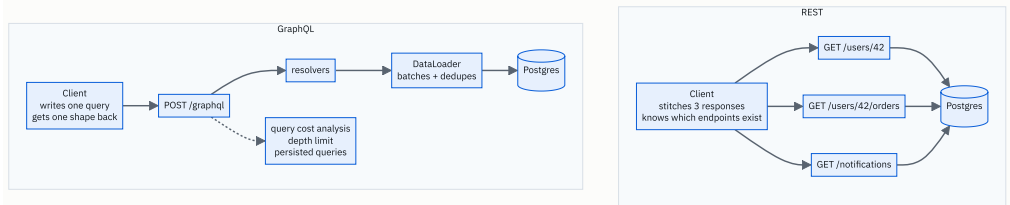


FIGURE 21.1

What to notice: the dotted box. It has no equivalent on the REST side, and it is not optional — without depth limits and cost analysis, a single query can ask for a user's friends' friends' friends and take your database down. That is the man in accounts asking for eleven chapatis.

5 How it actually works

One request, start to finish

1. The client `POST`s a query string (and variables) to `/graphql`.
2. The server **parses** it into a tree and **validates** it against the schema — unknown fields are rejected before anything runs, which is a real advantage of having a schema.
3. Optionally, **cost analysis**: estimate how expensive this query is and reject it if it is over budget.
4. **Execution**: walk the tree, calling each field's resolver. Sibling fields resolve in parallel; a child resolver waits for its parent.
5. Assemble `{"data": ..., "errors": [...]}` and return `200`.

The N+1 problem, concretely

```
query { orders(limit: 50) { id total customer { name } } }
```

Naively: 1 query for the 50 orders, then 50 separate queries for the 50 customers. **51 queries**, and if 20 of those orders share a customer, 20 of them are duplicates.

With DataLoader, the 50 customer requests are collected during one tick and issued as:

```
SELECT * FROM customers WHERE id IN (7, 12, 19, ...);
```

2 queries. DataLoader also caches within the request, so the 20 duplicates collapse to one id in the list. This is the single biggest operational difference between a GraphQL API that works and one that falls over, and it has to be built deliberately — it does not come for free with the framework.

Stopping abusive queries

Because the client writes the query, the client can write a bad one. Four defences, and you need more than one:

- **Depth limiting.** Reject queries nested more than, say, 10 levels. Stops `user { friends { friends { friends { ... } } } }`.

- **Cost analysis.** Assign each field a cost, multiply by list sizes, reject over a budget. GitHub's public GraphQL API does exactly this and publishes the formula.
- **Persisted queries.** The client registers its queries at build time and sends only a hash at run time. The server refuses anything it does not recognise. This is what large deployments do — it removes arbitrary queries entirely, and as a bonus the request becomes tiny and, being a known hash, **cacheable**.
- **Timeouts and pagination limits**, exactly as in REST.

Note that persisted queries give back most of what GraphQL took away — a fixed set of known operations. That is worth noticing out loud: at scale, GraphQL converges towards something REST-shaped, with the difference that the shapes are defined by clients rather than by the server.

Caching, which is the real loss

REST caches at four levels, and GraphQL loses the first three:

LEVEL	REST	GRAPHQL
Browser HTTP cache	works	no — it is a <code>POST</code>
CDN	works	no — unless persisted queries over <code>GET</code>
Reverse proxy	works	no
Application cache (Redis)	works	works, per resolver

The GraphQL answer is to cache **inside** the server, per field or per entity, which is more work and catches less traffic — a Redis hit still costs you a request, a connection and a server, where a CDN hit costs you nothing at all. Client libraries such as Apollo Client and Relay maintain a normalised cache in the browser, which is genuinely good and helps only that one user.

Real deployments worth naming

- **GitHub** — the best-known public GraphQL API, running alongside its REST API, with published rate limits based on query cost rather than request count.
- **Shopify** — GraphQL-first for its storefront and admin APIs.
- **Facebook** — where it was built, for mobile news feed.
- **Netflix** — federated GraphQL as a gateway in front of many backend services.
- **Stripe, Twilio, AWS** — REST, deliberately, and none of them show signs of moving.

That last line matters. Payment and infrastructure APIs stay REST because their operations are few, stable, need HTTP caching and idempotency semantics, and are called by machines rather than by screens.

The middle option nobody mentions

You do not have to choose globally. A REST API with a `fields` parameter — `GET /users/42?fields=name,avatarUrl` — solves over-fetching with none of GraphQL's cost, and **purpose-built endpoints** — `GET /screens/home` returning exactly what the home screen needs — solve under-fetching. This is sometimes called Backend For Frontend. It is less elegant and it is often the right answer, and proposing it is a strong move in an interview because it shows you are solving the problem rather than picking a technology.

6 The numbers

Take a mobile home screen needing user profile, last 3 orders, and an unread count. Mobile round trip: **120 ms**.

Round trips

```
REST, sequential : 3 × 120 ms = 360 ms
REST, parallel   :    120 ms (only if none depends on another)
GraphQL          :    120 ms + a little server time ≈ 150 ms
```

Parallel REST closes most of the gap. **Say that** — it is the honest counter to the round-trip argument, and it applies whenever the calls are independent.

Bytes over the wire

```
REST:      user 2 KB + orders 8 KB + notifications 1 KB = 11 KB
           of which actually rendered                  ≈ 2 KB
GraphQL:    exactly what was asked                      ≈ 2 KB
saving                                = 9 KB per screen load
```

At 1 million screen loads a day:

```
9 KB × 1,000,000 = 9 GB/day saved
```

Real, and worth more on a 2G connection than the numbers suggest: 9 KB at 100 kbps is about 0.7 seconds.

The caching loss, which is bigger

From [day 016](#): at 420 requests/second with 80% cacheable traffic and a 90% CDN hit rate, the CDN absorbed 302 requests/second and origin load fell from 420 to 118.

Lose HTTP caching and **all 420 hit your origin**. That is $3.5\times$ the server load, and the CDN-served responses were also coming back in 20 ms instead of 150.

```
REST + CDN : 118 req/s at origin, ~20 ms for cached reads
GraphQL    : 420 req/s at origin, ~150 ms for everything
```

This single comparison is why a public, read-heavy, largely anonymous site should not move to GraphQL, and it is the most persuasive thing you can say on this topic. It also explains why the calculus flips for a logged-in app where responses are per-user and therefore uncacheable anyway — there, the CDN was never helping.

The N+1 cost

50 orders each with a customer, at 1 ms per query:

```
naive      : 51 queries × 1 ms = 51 ms of database time
DataLoader:  2 queries × 1 ms =  2 ms
```

25× on one query, and at 100 requests per second the naive version needs 5.1 seconds of database time per second — six cores — against 0.2.

The abuse ceiling

Uncapped depth, on a social graph averaging 200 friends:

```
user { friends { friends { friends { name } } } }
= 200 × 200 × 200 = 8,000,000 nodes from one request
```

Eight million rows from a request that fits in a text message. **This is why depth limiting is not optional**, and it is the number that makes the point.

7 The trade-offs

What GraphQL costs you

HTTP caching. The largest loss, quantified above. Persisted queries over `GET` recover some of it and constrain your clients in exchange.

Server complexity. Resolvers, DataLoader batching, depth limits, cost analysis, and a monitoring story that copes with everything being `200 POST /graphql` — you cannot see slow endpoints in an access log any more, because there is only one endpoint.

Performance becomes unpredictable. In REST, `GET /users/42` costs what it costs, every time. In GraphQL, one query can be cheap and the next expensive, and you did not write either of them.

Rate limiting gets harder. Requests-per-minute is meaningless when one request can be a thousand times heavier than another, so you need cost-based limits — which is exactly what GitHub does.

Errors lose their HTTP meaning. Everything is `200` with an `errors` array. Defensible for partial success; a real cost in tooling.

What REST costs you

Over-fetching, under-fetching, endpoint proliferation as clients diverge, and versioning pain — GraphQL evolves by adding fields and deprecating old ones, with introspection telling you who still uses what, which is genuinely nicer than shipping `/v2`.

When to choose which

GraphQL when: many different clients need different slices of the same data; mobile round trips dominate; the data is a graph people traverse in varied ways; front-end teams iterate faster than the back end can ship endpoints; responses are per-user and therefore uncacheable anyway.

REST when: the API is public and read-heavy with cacheable responses; the operations are few and stable; consumers are machines rather than screens; you need HTTP semantics — caching, status codes, idempotency keys; or the team is small and every hour spent on DataLoader is an hour not spent on the product.

The sentence that separates candidates

I would not move to GraphQL for a public, read-heavy, largely anonymous API. The single biggest lever there is HTTP caching at the CDN, and GraphQL gives it up — 420 requests a second reaching my origin instead of 118, and 150 ms instead of 20. I would reach for GraphQL when the clients are logged-in apps whose responses were never cacheable to begin with, and when several front-end teams need genuinely different shapes of the same data. And before either, I would try a `fields` parameter and a couple of screen-shaped endpoints, because that fixes both complaints in an afternoon.

8 In the interview

How it gets asked

- “When would you choose GraphQL over REST?” — the direct version. The answer is a condition, not a preference.

- “What problems does GraphQL solve?” — over-fetching and under-fetching, named, with the numbers.
- “What are the downsides of GraphQL?” — the real filter. Caching and $N + 1$ are the two answers.
- “How do you stop a client from writing an expensive query?” — depth limiting, cost analysis, persisted queries.
- “Design the API for a mobile feed.” — where this becomes a live decision rather than a quiz.

What to say out loud, in the first ninety seconds

1. **Name the two problems it solves, concretely.** “GraphQL exists for over-fetching — a REST resource returns the whole object when the screen wants two fields — and under-fetching, where one screen needs three endpoints and pays three round trips.”
2. **Say what it is in one sentence.** “One endpoint, and the client sends a query naming exactly the fields it wants. The response shape mirrors the query.”
3. **Give the cost immediately, before being asked.** “The price is HTTP caching. Every request is a POST to one URL, so no CDN, no browser cache, no reverse proxy — and on a read-heavy public API that is usually the single biggest lever you have.”
4. **Name the second cost.** “And $N + 1$ in the resolvers. Fetching 50 orders with their customers naively runs 51 database queries, so you need a per-request batching layer — DataLoader — from day one.”
5. **Name the third.** “And because the client writes the query, you need depth limits and cost analysis, or one request can ask for friends-of-friends-of-friends and pull eight million rows.”
6. **Give the condition, not a preference.** “So I’d choose GraphQL when the clients are logged-in apps — where responses were per-user and uncacheable anyway — and several front-end teams need different shapes. I’d stay with REST for a public read-heavy API, or a machine-facing one that needs status codes and idempotency.”
7. **Offer the middle path.** “And before either, I’d try a `fields` query parameter and one or two screen-shaped endpoints. That fixes both complaints in an afternoon with no new infrastructure.”

The follow-ups

“What are the downsides of GraphQL?” Four, in order of how much they hurt. Caching: every request is a `POST` to a single URL, so browser caches, CDNs and reverse proxies are all out, and you are left doing application-level caching in Redis, which still costs you a request and a server where a CDN hit costs nothing. $N + 1$: the engine resolves fields independently, so a list query naively issues one database query per item, and you need DataLoader batching from the start. Unpredictable cost: the client writes the query, so one request may be a thousand times heavier than another, which makes both capacity

planning and rate limiting harder — you need cost-based limits rather than request counts. And observability: your access log says `200 POST /graphql` for everything, so you cannot see slow endpoints without instrumenting resolvers deliberately.

“How do you stop a malicious or careless query?” Layered. Depth limiting first, because it is cheap and stops the obvious recursion — reject anything nested beyond about ten levels. Then cost analysis: assign a cost to each field, multiply by requested list sizes, and reject queries over a budget; GitHub publishes exactly such a formula for its public API. Then pagination limits enforced server-side, so `first: 100000` is capped regardless of what was asked. And the strongest option, for a client you control: persisted queries, where the client registers its queries at build time and sends only a hash, and the server refuses anything unrecognised — which eliminates arbitrary queries entirely. It is worth noticing that persisted queries bring you back to a fixed set of known operations, which is REST-shaped again, with the difference that the shapes were defined by the clients.

“Can you cache GraphQL at all?” Not at the HTTP layer, in the normal case, because it is a `POST` and caches key on method and URL. Three things you can do. Persisted queries sent as a `GET` with the hash in the query string are cacheable by a CDN, which recovers most of it for a controlled client. Application-level caching per resolver or per entity in Redis works well and is what most deployments do — you cache the `User:42` object, not the response, so it is reused across differently-shaped queries. And client-side, Apollo Client and Relay keep a normalised cache in the browser keyed by type and id, which is genuinely good and helps exactly one user. What you cannot get back is the property that made the CDN so valuable: one cached copy serving thousands of different people from a nearby city.

“Facebook built it for mobile. Does that reasoning still hold?” Partly. The round-trip argument is weaker than it was, because HTTP/2 multiplexes several requests over one connection so parallel REST calls no longer queue behind each other, and connections are usually already warm. The payload argument still holds on poor networks — 9 KB saved is about 0.7 seconds at 100 kbps. The argument that has actually strengthened is organisational rather than technical: when several front-end teams ship faster than the backend can add endpoints, letting them choose the shape removes a queue. That is the reason most teams adopt it today, and I would rather say that honestly than repeat the 2012 mobile rationale.

A model answer

“GraphQL exists to fix two specific complaints about REST, and I’d answer the question by looking at whether I actually have those complaints.

The first is over-fetching. In REST the resource is the unit, so `GET /users/42` returns the whole user — say nine fields — when the screen renders two. The second is under-fetching: one screen often needs several resources, so it makes three calls and pays three round trips, which on mobile is roughly 360 milliseconds instead of 120.

GraphQL replaces the many endpoints with one, and the client sends a query naming exactly the fields it wants, nested however it wants. The response mirrors the query, so you can predict the shape by reading the request. One round trip, and about 2 KB instead of 11.

The costs are where the real answer is. The biggest is HTTP caching. Every GraphQL request is a POST to a single URL, so the browser cache, the CDN and any reverse proxy are all useless. On the read-heavy API I’d sized earlier, a CDN at a 90% hit rate took origin load from 420 requests a second down to 118, and served those reads in 20 milliseconds instead of 150. Giving that up is a much larger cost than the 9 KB saved per screen.

Second, $N + 1$. Resolvers run independently, so a query for 50 orders with their customers naively runs 51 database queries. You need a per-request batching layer — DataLoader — which collects the ids and issues one `WHERE id IN (...)`. That’s 51 queries down to 2, and it doesn’t come for free with the framework; you have to build it in from the start.

Third, the client writes the query, so the client can write a terrible one. On a social graph averaging 200 friends, three levels of nesting is eight million nodes from a request that fits in a text message. So depth limits and cost analysis are mandatory, not optional, and rate limiting has to be cost-based rather than request-based.

So my answer is a condition rather than a preference. I’d choose GraphQL when the clients are logged-in applications whose responses are per-user and therefore weren’t cacheable anyway, when several front-end teams need genuinely different shapes of the same data, and when the domain is a graph people traverse in varied ways. I’d stay with REST for a public, read-heavy, largely anonymous API where CDN caching is the biggest lever I have, and for machine-facing APIs like payments, where I want status codes, idempotency keys and predictable cost — which is why Stripe and AWS are still REST and show no sign of changing.

And before committing to either, I'd try the cheap middle: a `fields` query parameter to fix over-fetching, and one or two screen-shaped endpoints to fix under-fetching. That solves both complaints in an afternoon with no new infrastructure, and if it isn't enough, the reasons why will tell me whether GraphQL is actually the answer."

9 Recall card

- **GraphQL: one endpoint, the client names the fields, the response mirrors the query.** Fixes over-fetching and under-fetching.
- **The big cost is HTTP caching** — every request is a `POST` to one URL, so no browser cache, no CDN, no proxy.
- **N+1 is real and needs DataLoader** — 50 orders with customers is 51 queries naively, 2 with batching.
- **The client writes the query, so cap it:** depth limits, cost analysis, persisted queries, server-side pagination caps.
- **Choose by condition:** GraphQL for logged-in multi-client apps; REST for public, cacheable, machine-facing APIs. Consider `?fields=` and screen-shaped endpoints first.

DSA topic: Character counting and frequency maps **System design topic:** GraphQL versus REST

Code these, in this order

Four problems where the answer is *build a count, then look at it*. The skill being trained is recognising that in the first five seconds, so before you write anything, say out loud which phrase in the problem statement told you.

Before each one, ask:

1. Do I need **how many**, or only **whether**? (Counter or set.)
2. What is the alphabet? Can I use 26 slots, or do I need a dictionary?
3. How many passes does this need, and can it be done in one?
4. What is the answer for the empty input?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Ransom Note	LeetCode 383 (Easy)	Counter subtraction, and whether you notice that “can I build A from B” is a counting question and not a searching one.
2	First Unique Character in a String	LeetCode 387 (Easy)	Two passes, and being able to say why one pass is impossible.
3	Sort Characters By Frequency	LeetCode 451 (Medium)	<code>most_common</code> , then building the output with <code>join</code> — not <code>+=</code> . Two lessons meeting.
4	Find All Anagrams in a String	LeetCode 438 (Medium)	A count that moves : add the entering character, remove the leaving one. This is the sliding window of day 033 arriving early.

On problem 1, do it three ways

Solve it with `Counter` subtraction, then with a plain dictionary and an explicit check, then with a 26-element array. Time all three on a large input. Then say which you would write in an interview and why — and note that the answer is the `Counter` one, with the others available if asked.

On problem 2, argue with yourself

Spend five minutes genuinely trying to solve it in one pass before reading anything. Then write down the exact reason it cannot be done. The sentence you want is about *when the information becomes available*, not about cleverness.

Then look up the stream version — “first non-repeating character so far, after each character” — and work out why that one **can** be done incrementally, and what extra structure it needs.

On problem 4, build the moving count by hand

Do not jump to the solution. Take `s = "cbaebabacd"` and `p = "abc"` and, for each window of length 3, say out loud what the count looks like and whether it matches the count of `p`. Then notice you are recomputing the whole count each time, which is $O(n \times k)$.

Then find the improvement: when the window slides one step, only two characters change. Decrement the one leaving, increment the one arriving. Say the cost of that version before you code it.

The three-ways drill

Write the character count for `"hello"` three ways from memory, in under sixty seconds:

```
# 1. plain dict
# 2. defaultdict
# 3. Counter
```

Then run this and explain each result:

```
counts = {}
for ch in "hello":
    counts[ch] += 1
```

```
from collections import defaultdict
counts = defaultdict(int)
counts["a"] += 1
if counts["z"] == 0:
    pass
print(dict(counts))
```

```
from collections import Counter
c = Counter("hello")
print(c["z"], dict(c))
```

The second one has a surprise in it. Name it.

The silent-bug drill

Predict the output before running:

```
def counts26(s):  
    arr = [0] * 26  
    for ch in s:  
        arr[ord(ch) - ord("a")] += 1  
    return arr  
  
print(counts26("Hello"))
```

Most people predict an `IndexError`. Say what actually happens and why, then say which position the capital `H` was counted into and how you would work that out without running it. Then fix the function two different ways.

The pattern-recognition drill

For each phrase, say in under three seconds which structure you reach for — **counter**, **set**, **sorted**, or **moving counter** — and why:

1. “the most common word in this document”
2. “does this array contain any duplicates”
3. “are these two strings anagrams”
4. “the first character that appears exactly once”
5. “how many distinct characters”
6. “the longest substring with at most two distinct characters”
7. “can this note be built from these letters”
8. “which element appears more than $n/2$ times”

Numbers 2 and 5 want a set, not a counter. Number 6 is a window. Number 8 has a famous `O(1)`-space answer worth looking up after you have given the counting one.

The cost drill

State time **and** space for each, counting out loud rather than naming a class:

1. `Counter(s)` on a string of length n .
2. The two-pass first-unique solution.
3. The naive `s.count(ch)` inside a loop.
4. A 26-element array count on a string of length n .
5. `len(set(s))`.

For number 4, say why the space is `O(1)` and not `O(26)`, and be able to defend it.

The GraphQL drill

Answer each in one or two sentences, out loud:

1. What are the two specific problems GraphQL was built to solve? Give an example of each.
2. What does a GraphQL response shape have to do with the request shape?
3. What is the single biggest thing you give up, and roughly what does it cost in server load?
4. What is the $N + 1$ problem in GraphQL, and what fixes it?
5. Why is depth limiting not optional?
6. Why does a GraphQL server return `200` even when the query failed?
7. Name two large companies using GraphQL and two deliberately staying with REST, and say why the second group is right for their case.
8. What is the cheap middle option, and why might you try it first?

The arithmetic drill

From memory, in under two minutes:

- Three REST calls at 120 ms each, sequentially and in parallel, against one GraphQL call.
- 11 KB of REST responses where 2 KB is rendered — what is saved per screen, and per million loads?
- 420 requests a second, 80% cacheable, 90% CDN hit rate — origin load with REST, and with GraphQL.
- 50 orders each needing a customer — queries with and without DataLoader.
- A social graph averaging 200 friends, three levels of nesting — how many nodes?

Then say the sentence those numbers buy you: *“the caching loss is bigger than the payload saving.”*

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. Find the first non-repeating character in a string. Clarify index-or-character and the alphabet. Name the `O(n2)` version. State the two-pass structure, and — before being asked — say why one pass is impossible. Give `O(n)` time and the space answer that depends on the alphabet.

2. *When would you choose GraphQL over REST?* Name the two problems it solves, then immediately name what it costs, with the caching number. Then give a **condition**, not a preference. Finish with the `?fields=` middle path.
3. *Here is a string of a billion characters that will not fit in memory. Find the first character that appears exactly once.* The counter is bounded by the alphabet, not the length, so pass one stream in constant memory. Then the interesting half: how do you avoid a second pass over the data? (Record first positions.)
-

Before you move on

- ☐ I hear “how many times” and reach for a counter without thinking about it.
- ☐ I use a set when the question is “whether”, not “how many”.
- ☐ I can write the count three ways, and I know why plain `counts[ch] += 1` raises.
- ☐ I can say why first-unique needs two passes, and that two passes is still `O(n)`.
- ☐ I state the alphabet assumption out loud before using `ord(ch) - ord("a")`.
- ☐ I can name GraphQL’s two problems solved and its four real costs.
- ☐ I know what DataLoader is for and can say the 51-versus-2 number.
- ☐ I answer “GraphQL or REST” with a condition, never a preference.
- ☐ I can redraw the both-ways round-trip diagram from memory, in whatever tool I like.

Anagrams: the sorting versus counting choice

DATA STRUCTURES AND ALGORITHMS

Anagrams: the sorting versus counting choice

You can give two solutions with different complexities and say which one you would ship.

SYSTEM DESIGN

gRPC and when binary protocols win

You can say why internal services often do not speak JSON over HTTP.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Are these two strings anagrams? Now group a list of words into anagram groups.*
- *Why would a company use gRPC between its own services?*

Anagrams: the sorting versus counting choice

1 What this is, and why they ask it

Two strings are **anagrams** if one is a rearrangement of the other — same letters, same number of each, different order. `listen` and `silent`. `eat` and `tea`.

The question has exactly one idea in it, and it is a big one: **find a form of the string that is identical for all its anagrams, and different for everything else**. That is called a **canonical form**, and once you have one, “are these anagrams?” becomes “are their canonical forms equal?”, and “group these words” becomes “put them in a dictionary keyed by canonical form”. There are two natural canonical forms — the letters in sorted order, and the count of each letter — and choosing between them is the whole exercise.

Interviewers like this because it has **two correct answers with different complexities**, which is rare and useful. Sorting is one line and $O(k \log k)$ per word; counting is five lines and $O(k)$. Producing both, unprompted, and then saying which you would actually ship and why, is a complete demonstration of the thing they are trying to measure. Producing only the one-liner is fine and unremarkable. Producing only the counting version and never mentioning that `sorted(a) == sorted(b)` exists looks like you have memorised a solution.

It is LeetCode 242 and 49, it appears in phone screens everywhere, and the canonical-form idea itself comes back on [day 064](#) as a design skill in its own right.

2 The story

Murugan drives a school van in Coimbatore, the small yellow one, and he has eight children on the afternoon round. He picks them up from the gate at half past three and drops the last one at Ganapathy by about a quarter past four.

Getting them into the van is not the problem. The problem is knowing that the eight who got in are the eight who are meant to be in it, because at half past three there are two hundred children coming out of one gate and four vans parked in a line, and children are not careful.

They never sit in the same places. Whoever gets there first takes the back seat, the two brothers fight about the window, and one girl always sits behind him because she says the back makes her sick. So he cannot check by looking at the seats. The arrangement is

different every single day and it does not mean anything.

The first thing he does takes two seconds. He counts heads. If there are seven, something is wrong, and if there are nine, something is very wrong, and either way he does not need to know anything else yet — he gets out and goes and looks. Most days it is eight and he carries on.

Then he does the actual check, and he has two ways of doing it depending on his mood.

Some days he makes them sit in the order he always uses — the two brothers at the front, then the girl from Race Course, and so on down the van, the same order every day — and then one look tells him. Everyone in their proper place, nobody extra, nobody missing.

Most days he does not bother with that, because getting eight children to move is harder than it sounds. Instead he reads out the names from the list on his phone and each one shouts. Eight names, eight shouts, and if he gets to the end and a name has not been answered, or somebody shouts twice, he knows.

Both ways work. The first takes longer because he has to rearrange everybody. The second is quicker but he has to have the list. And on the day a boy from another van climbed in by mistake, the head count had already told Murugan something was wrong before he started either of them.

3 The idea in plain English

Murugan's two methods are the two solutions. Making everyone sit in the fixed order is **sorting**. Reading the names off the list is **counting**. And counting heads first is the length check, which you should always do.

What the question is really asking

Two strings are anagrams when they contain the same characters with the same multiplicities, ignoring order. So you need a way of describing a string that **throws the order away and keeps nothing else**. Any two anagrams must produce the same description, and any two non-anagrams must produce different ones.

That description is the **canonical form**, and there are two obvious choices.

Canonical form one: sort the letters

```
sorted("listen")    # ['e', 'i', 'l', 'n', 's', 't']
sorted("silent")    # ['e', 'i', 'l', 'n', 's', 't']
```


`sorted` on a string returns a **list of characters**, not a string — surprising the first time, and it does not matter for comparison because two equal lists compare equal. If you need a string, as you will for a dictionary key, use `"".join(sorted(s))`.

Sorting puts the letters into one fixed order regardless of how they started, so any two anagrams land on the same result. That is Murugan making everyone sit in the same seats.

```
def is_anagram(a: str, b: str) → bool:
    return sorted(a) == sorted(b)
```

One line, correct, and worth writing first.

Canonical form two: count the letters

```
Counter("listen")    # {'l':1, 'i':1, 's':1, 'e':1, 'n':1, 't':1}
Counter("silent")    # {'s':1, 'i':1, 'l':1, 'e':1, 'n':1, 't':1}
```

Two `Counter`s compare equal when they hold the same keys with the same values, and dictionaries do not care about insertion order for equality. So:

```
def is_anagram(a: str, b: str) → bool:
    if len(a) != len(b):
        return False
    return Counter(a) == Counter(b)
```

That is Murugan reading out the names. The frequency map from [day 021](#), used as an identity rather than as a count.

The length check, which is free

`if len(a) != len(b): return False` costs nothing — `len` is `O(1)` on both a string and a list from [day 019](#) — and it exits immediately on a whole class of inputs. It is Murugan counting heads.

Strictly, the sorting version does not need it and the counting version does not either, since different lengths produce different counts. Write it anyway and say why: **it is the cheapest possible rejection and it makes the intent obvious**. Interviewers notice.

Grouping: the same idea, one level up

“Group these words into anagram groups” — LeetCode 49 — is the canonical form used as a **dictionary key**:

```
groups = defaultdict(list)
for word in words:
    groups[canonical(word)].append(word)
return list(groups.values())
```

That is the entire algorithm. All the thinking is in `canonical`. With sorting:

```
groups["".join(sorted(word))].append(word)
```

`"".join(sorted(word))` rather than `sorted(word)`, because a **dictionary key must be hashable** and a list is not — from [day 019](#), only immutable things can be keys. §7 shows the exact error.

With counting, the same constraint applies, so the count has to become a tuple:

```
key = [0] * 26
for ch in word:
    key[ord(ch) - ord("a")] += 1
groups[tuple(key)].append(word)           # tuple, because a list cannot be a key
```

A 26-element tuple of counts, identical for all anagrams of a word and different for everything else.

Which to ship

Sorting is $O(k \log k)$ per word; counting is $O(k)$. So counting wins asymptotically, and that is the answer to “*can you do better?*”.

But be honest about the size of the win. Measured in Python on 2,000 words:

word length	sorting	counting
10	0.0028 s	0.0027 s
100	0.0155 s	0.0162 s
1,000	0.1834 s	0.1172 s
5,000	0.9841 s	0.7152 s

At realistic word lengths they are the same, and counting only pulls ahead around a thousand characters — because `sorted` runs as compiled C while the counting loop runs as Python bytecode, and that constant factor swamps a $\log k$ of about 5. **For ordinary words I would ship the sorted version, because it is one line and obviously correct. For long strings, or if the interviewer asks for better than $O(k \log k)$, I would switch to counting.** That sentence is the answer to the question.

4 The picture

The two canonical forms:

SORTING

```
"eat" → a e t
"tea" → a e t   same
"ate" → a e t
"tan" → a n t   different
```

cost per word: $O(k \log k)$
key type: a string

COUNTING

```
"eat" → a:1 e:1 t:1
"tea" → a:1 e:1 t:1   same
"ate" → a:1 e:1 t:1
"tan" → a:1 n:1 t:1   different
```

cost per word: $O(k)$
key type: a tuple of 26 counts

What to notice: both columns collapse the three arrangements of `eat` onto one thing, and keep `tan` apart. That is all a canonical form has to do. Anything with those two properties would work — these two are simply the cheapest to compute.

Grouping, as one pass into a dictionary:

word	canonical key	the dictionary after this word
eat	"aet"	{ "aet": ["eat"] }
tea	"aet"	{ "aet": ["eat", "tea"] }
tan	"ant"	{ "aet": ["eat", "tea"], "ant": ["tan"] }
ate	"aet"	{ "aet": ["eat", "tea", "ate"], "ant": ["tan"] }
nat	"ant"	{ "aet": [...], "ant": ["tan", "nat"] }
bat	"abt"	{ "aet": [...], "ant": [...], "abt": ["bat"] }

answer = list of the values = [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

What to notice: one pass, and each word is looked at exactly once. There is no comparing of words against each other at all — which is what makes it $O(n \cdot k)$ instead of the $O(n^2 \cdot k)$ you would get by testing every pair.

The count key, laid out:

```
"tan"      t a n
           | | |
index:     0  1  2  ... 13  ... 19  ... 25
           +---+---+---+   +---+   +---+   +---+
           | 1 | 0 | 0 |   | 1 |   | 1 |   | 0 |
           +---+---+---+   +---+   +---+   +---+
           a  b  c         n         t         z

key = (1,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,0)
```

What to notice: the key is a fixed 26 numbers whatever the word, so building it is $O(k)$ to fill and $O(26)$ to hash — constant. Twenty-five of the twenty-six entries here are noise, which is the price of the fixed shape.

5 The code, built step by step

The one-liner, and why to write it first

```
def is_anagram_sorting(a: str, b: str) → bool:
    return sorted(a) == sorted(b)
```

Correct for every input including empty strings, handles any characters including Unicode, and needs no explanation. Write it, say $O(k \log k)$, and then offer the better one. Starting from the simple correct thing and improving it is exactly the shape an interviewer wants.

The counting version

```
def is_anagram_counting(a: str, b: str) → bool:
    if len(a) != len(b):
        return False
    return Counter(a) == Counter(b)
```

Two `Counter`s are equal when every key has the same value in both. The length guard is the free early exit.

The counting version without the library

Asked to do it without `Counter`, this is the one to write — and it is better than it looks, because it does a single combined pass rather than two:

```
def is_anagram_manual(a: str, b: str) → bool:
    if len(a) != len(b):
        return False
    counts = [0] * 26
    for ch in a:
        counts[ord(ch) - ord("a")] += 1      # lowercase a-z only
    for ch in b:
        counts[ord(ch) - ord("a")] -= 1
        if counts[ord(ch) - ord("a")] < 0:   # b has a letter a did not
            return False
    return True
```

Two things worth saying out loud while you write it.

The second loop subtracts rather than building a second count. If the two strings are anagrams, every entry returns to zero.

The early exit inside the second loop is what removes the final check. The moment any count goes negative, `b` contains a character more often than `a` does, so they cannot be anagrams. And because the lengths are equal, if no count ever goes negative then none can be left positive either — so there is no need to scan the array at the end. That last sentence is a genuinely good thing to say in an interview.

Grouping, with sorted keys

```
def group_anagrams_sorting(words: list[str]) → list[list[str]]:
    groups: defaultdict[str, list[str]] = defaultdict(list)
    for word in words:
        groups["".join(sorted(word))].append(word)
    return list(groups.values())
```

Four lines. `"".join(sorted(word))` because a list cannot be a dictionary key.

Grouping, with count keys

```
def group_anagrams_counting(words: list[str]) → list[list[str]]:
    groups: defaultdict[tuple[int, ...], list[str]] = defaultdict(list)
    for word in words:
        key = [0] * 26
        for ch in word:
            key[ord(ch) - ord("a")] += 1
        groups[tuple(key)].append(word)      # tuple: hashable, list is not
    return list(groups.values())
```

The only differences are how the key is built and that it must be frozen into a tuple.

The complete solutions

```
from collections import Counter, defaultdict

def is_anagram_sorting(a: str, b: str) → bool:
    """LeetCode 242, the one-liner.  $O(k \log k)$ . Works on any characters."""
    return sorted(a) == sorted(b)

def is_anagram_counting(a: str, b: str) → bool:
    """The same, in  $O(k)$ . The length check is a free early exit."""
    if len(a) != len(b):
        return False
    return Counter(a) == Counter(b)

def is_anagram_manual(a: str, b: str) → bool:
    """Without the library. Lowercase a-z only. One count, incremented then decremented."""
    if len(a) != len(b):
        return False
    counts = [0] * 26
    for ch in a:
        counts[ord(ch) - ord("a")] += 1
    for ch in b:
        index = ord(ch) - ord("a")
        counts[index] -= 1
        if counts[index] < 0:
            # b has more of this letter than a does
            return False
    return True
    # equal lengths + nothing negative ⇒ anagram

def group_anagrams_sorting(words: list[str]) → list[list[str]]:
    """LeetCode 49. Key = the letters in sorted order.  $O(n * k \log k)$ ."""
    groups: defaultdict[str, list[str]] = defaultdict(list)
    for word in words:
        groups["".join(sorted(word))].append(word)
    return list(groups.values())

def group_anagrams_counting(words: list[str]) → list[list[str]]:
    """The same in  $O(n * k)$ . Key = a 26-tuple of letter counts."""
    groups: defaultdict[tuple[int, ...], list[str]] = defaultdict(list)
    for word in words:
        key = [0] * 26
        for ch in word:
            key[ord(ch) - ord("a")] += 1
        groups[tuple(key)].append(word)
    return list(groups.values())

if __name__ == "__main__":
    for f in (is_anagram_sorting, is_anagram_counting, is_anagram_manual):
        print(f.__name__,
              f("anagram", "nagaram"),      # True
              f("rat", "car"),                # False
              f("a", "ab"),                  # False
              f("", ""),                      # True
              f("aacc", "ccac"))             # False — same letters, different counts

    words = ["eat", "tea", "tan", "ate", "nat", "bat"]
    print(group_anagrams_sorting(words))
    # [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
    print(group_anagrams_counting(words))
    # [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
    print(group_anagrams_sorting([]))        # []
    print(group_anagrams_sorting([""]))     # [['']]
```

Note the test `f("aacc", "ccac")`. Both have four characters from `{a, c}`, and they are **not** anagrams — `aacc` has two of each, `ccac` has one `a` and three `c`. Any solution that checks only *which* letters appear, using a set, gets this wrong. It is the input to reach for when you want to break somebody's set-based answer, including your own.

6 What it costs

Let `k` be the length of a word and `n` the number of words.

`is_anagram`

Sorting. `sorted` on `k` characters is $O(k \log k)$, done twice, then a comparison of two lists of length `k` which is $O(k)$. Total $O(k \log k)$ time. Space is two lists of `k` characters, so $O(k)$.

Counting. One pass over each string, `k` steps each, with constant-time dictionary operations — `2k` steps. Then comparing two dictionaries of at most 26 entries. Total $O(k)$ time. Space is one entry per distinct character, at most 26 for lowercase English, so $O(1)$.

So counting is asymptotically better in both time and space, and that is the answer when asked “can you do better?”.

Grouping

Each word is processed once and appended once, so:

- **Sorting:** `n` words $\times$ $O(k \log k)$ each = $O(n \cdot k \log k)$.
- **Counting:** `n` words $\times$ $O(k)$ each = $O(n \cdot k)$.

Space for both is $O(n \cdot k)$ — every word is stored once in the output, plus the keys.

Notice what is **not** here: no comparison between words. The naive approach of testing every pair would be $O(n^2 \cdot k)$; at `n = 10,000` that is 100 million pair tests against 10,000 key computations. **The dictionary turns a pairwise problem into a single pass, and that is the real lesson of the grouping question.**

The honest measurement

2,000 randomly generated lowercase words, grouped:

word length	sorting	counting	ratio
10	0.0028 s	0.0027 s	1.0x
100	0.0155 s	0.0162 s	0.96x (sorting slightly faster)
1,000	0.1834 s	0.1172 s	1.6x
5,000	0.9841 s	0.7152 s	1.4x

At `k = 10` and `k = 100` there is nothing between them, and at `k = 100` sorting is marginally ahead. The reason is that `log2(100)` is about 7, so sorting does roughly seven times the *asymptotic* work — but `sorted` is compiled C and the counting loop is interpreted Python, and that constant factor is worth more than the $7\times$. Only when `k` reaches a thousand does the asymptotic difference start to show.

The honest conclusion: counting is the better answer to give, sorting is usually the better code to ship for word-sized strings, and being able to say both — with the reason — is worth more than either.

The number to have ready

Sorting is $O(k \log k)$ per word, counting is $O(k)$. Grouping `n` words is $O(n \cdot k \log k)$ or $O(n \cdot k)$ — and either way it beats the $O(n^2 \cdot k)$ of comparing every pair, which at ten thousand words is a hundred million comparisons against ten thousand key computations.

7 The traps

The real error: a list as a dictionary key

```
groups = {}
groups[sorted("eat")] = ["eat"]
```

```
TypeError: unhashable type: 'list'
```

A dictionary key must be **hashable**, and only immutable things are — the reason from [day 019](#): a key that could change after being filed could never be found again. So `sorted(word)`, which is a list, has to become `"".join(sorted(word))` or `tuple(sorted(word))`. Same for the count key: `tuple(key)`, not `key`.

This is the single most common error in the grouping problem and the message names the cause exactly.

The near-miss: using a set instead of a count

```
def is_anagram(a, b):
    return set(a) == set(b)    # wrong
```

```
is_anagram("aacc", "ccac")    # True  - should be False
is_anagram("aab", "abb")      # True  - should be False
```


A set records **which** characters appeared and throws away **how many**. Anagram is about multiplicities, so a set is the wrong structure — exactly the distinction drawn on [day 021](#). It passes on `listen / silent` and on most casual test data, which is why it survives long enough to reach a grader.

The near-miss: the missing length check in the manual version

```
def is_anagram(a, b):
    counts = [0] * 26          # no length check
    for ch in a:
        counts[ord(ch) - ord("a")] += 1
    for ch in b:
        counts[ord(ch) - ord("a")] -= 1
        if counts[ord(ch) - ord("a")] < 0:
            return False
    return True

print(is_anagram("aab", "ab"))
```

True

Wrong. `"aab"` and `"ab"` are not anagrams, but nothing ever goes negative — the leftover `a` sits in the array as a positive 1 and is never examined. **The early-exit logic is only valid because the lengths are equal**, and removing the length check silently removes that guarantee. If you want to drop the length check you must scan the array for a non-zero entry at the end, which costs another 26 steps. The length check is cheaper and clearer.

The silent bug: `ord` on anything but lowercase

```
counts[ord(ch) - ord("a")] += 1
print(is_anagram_manual("Listen", "Silent"))
```

`ord("L") - ord("a")` is `-21`, a perfectly legal negative index, so the capital letters are counted into the wrong slots with no error at all — the trap from [day 021](#). It happens to return `True` here, for the wrong reason. Normalise first with `.lower()`, and say the assumption out loud before you rely on it.

The contract question: what counts as an anagram?

Three things the interviewer may or may not want, and you should ask rather than guess:

- **Case.** Is `Listen` an anagram of `Silent`? Usually yes, so `.lower()` both.
- **Spaces and punctuation.** Is `"conversation"` an anagram of `"voices rant on"`? The classic phrase-anagram version strips non-letters first.

- **Unicode.** `é` can be one code point or `e` plus a combining accent, and those compare unequal. If the input may be Unicode, normalise with `unicodedata.normalize("NFC", s)` first.

Asking one of these takes five seconds and shows you read the problem rather than pattern-matched it.

The near-miss: comparing `sorted` results of different types

```
sorted("abc") == "abc"          # False
```

`sorted` returns a **list**, so `['a', 'b', 'c'] == "abc"` is `False`. Harmless when you compare two `sorted` calls with each other, and a real bug the moment you compare one against a string. If you want a string, `"".join(sorted(s))`.

8 In the interview

How it gets asked

- “Are these two strings anagrams?” — LeetCode 242, the warm-up. Then, almost always, “can you do it faster than sorting?”
- “Group these words into anagram groups.” — LeetCode 49, and the real question. It is about using a canonical form as a dictionary key.
- “Find all anagrams of *p* inside *s*.” — LeetCode 438, which is this plus a sliding window from [day 033](#).
- “Given a list of words, find any two that are anagrams of each other.” — the version that tempts people into a nested loop. The dictionary makes it one pass.

What to say out loud, in the first ninety seconds

1. **Ask the contract questions.** “Does case matter? Should I ignore spaces and punctuation? Can I assume lowercase English letters, or arbitrary Unicode?” The last one decides whether the 26-element array is available.
2. **Name the idea before either solution.** “The core idea is a canonical form — some representation that’s identical for all anagrams and different for anything else. Then the whole problem is comparing, or grouping by, that form.”
3. **Give the simple one first.** “The simplest canonical form is the letters in sorted order.
`sorted(a) == sorted(b)`, one line, $O(k \log k)$.”
4. **Then improve it, unprompted.** “But I can do better. Counting the letters is also a canonical form, and counting is $O(k)$ instead of $O(k \log k)$. Two counts, compare them — or one count, incremented for the first string and decremented for the second.”

5. **Mention the free check.** *“And the length check first, which costs nothing and rejects a whole class of inputs immediately.”*
6. **For grouping, say the key insight explicitly.** *“For grouping, the canonical form becomes a dictionary key. One pass, each word appended to its group. That turns what looks like a pairwise comparison problem — $O(n^2k)$ — into a single pass.”*
7. **Say which you would ship, and why.** *“I’d write the sorted version for word-length strings because it’s one obviously-correct line, and switch to counting for long strings or if you want better than $O(k \log k)$.”*

The follow-ups

“Can you do better than sorting?” Yes — count instead. Sorting is $O(k \log k)$ because it establishes a total order on the characters, which is strictly more information than I need; I only need how many of each. Counting is one pass, $O(k)$, and for a fixed alphabet the space is constant. The neat version uses a single array: add one for each character of the first string, subtract one for each character of the second, and return false the moment any entry goes negative. That early exit is only correct because I checked the lengths are equal first — with equal lengths, if nothing ever goes negative then nothing can be left positive either, so I do not need a final scan. Without the length check, “aab” and “ab” would wrongly come back as anagrams.

“How would you group a million words?” The same one pass, and the constant factors start to matter. I would use the 26-tuple count key rather than the sorted string, since it is $O(k)$ per word and the tuple hashes in constant time. Memory is the real question: a million words at 20 characters is roughly 20 MB of text, plus a key per word and the group lists — comfortably a single machine, so I would not distribute it. If it genuinely did not fit, this parallelises perfectly: the canonical key is a pure function of the word, so I can shard by $\text{hash}(\text{key}) \% N$, have each machine group its own shard, and concatenate the results with no merging at all, since every anagram of a word produces the same key and therefore lands on the same machine. That is a map-reduce with a trivial reduce.

“What if the strings are Unicode, or contain spaces and capitals?” The sorted version keeps working unchanged, which is a real argument in its favour. The 26-element array does not, and worse, it fails silently rather than raising, because `ord(ch) - ord('a')` for a capital letter is a negative number and negative indices are legal in Python — so a capital **L** gets counted into slot 5 with no error at all. For Unicode I would use a `Counter`, which handles any character, and normalise first with `unicodedata.normalize`, because the same visible character can have more than one encoding and those would count as different keys. For phrase anagrams I would lower-case and filter to letters before doing anything else, and I would ask which of those the problem wants rather than assuming.

“Now find all the anagrams of p inside s .” That is the sliding window version. I keep a count of p , and a count of the current window of s which has the same length as p . If the two counts match, the window’s start index is an answer. Then I slide: increment the character entering on the right, decrement the one leaving on the left, and compare again. Recomputing the whole window count each time would be $O(n \cdot k)$; updating two entries per step makes it $O(n)$, with $O(1)$ space for a fixed alphabet. The detail worth mentioning is that comparing two 26-entry counts on every step is technically constant but not free, so a common refinement tracks a single “number of letters currently matching” counter and updates it as the two edges move — which makes the per-step work genuinely $O(1)$.

A model answer

“First, a couple of contract questions. Does case matter — is `Listen` an anagram of `Silent`? Should I ignore spaces and punctuation? And can I assume lowercase English letters, or could this be arbitrary Unicode?

...Lowercase English, case-sensitive. Good.

The idea underneath this problem is a canonical form: some representation of a string that is identical for every anagram of it and different for anything else. Once I have one, ‘are these anagrams’ is just comparing two canonical forms, and ‘group these words’ is putting them in a dictionary keyed by that form.

The simplest canonical form is the letters in sorted order, so `sorted(a) == sorted(b)`. That is one line, correct for any input including Unicode, and it is $O(k \log k)$.

I can do better, because sorting produces a total ordering and I only need multiplicities. Counting the letters is also a canonical form and it is $O(k)$:

```
def is_anagram(a: str, b: str) → bool:
    if len(a) != len(b):
        return False
    counts = [0] * 26
    for ch in a:
        counts[ord(ch) - ord("a")] += 1
    for ch in b:
        index = ord(ch) - ord("a")
        counts[index] -= 1
        if counts[index] < 0:
            return False
    return True
```

The length check first is free and rejects a whole class of inputs. The second loop subtracts rather than building a second count, and bails the moment an entry goes negative, which means `b` has a letter more often than `a` does. And I want to point out that the early exit is only valid *because* the lengths are equal — with equal lengths, if nothing goes negative then nothing can be left positive either, so there is no final scan. Drop the length check and `aab` versus `ab` wrongly returns true.

For grouping, the same canonical form becomes a dictionary key:

```
groups = defaultdict(list)
for word in words:
    groups["".join(sorted(word))].append(word)
return list(groups.values())
```

The important thing there is that there is no comparison between words at all. The obvious approach would test every pair, which is $O(n^2 \cdot k)$ — a hundred million comparisons at ten thousand words — and keying a dictionary makes it a single pass. I have to join the sorted characters into a string because a list is not hashable and cannot be a key; with the counting version the key would be a 26-element tuple for the same reason.

On which to ship: I actually measured this, and for word-length strings the sorted version is no slower, because `sorted` is compiled C while the counting loop is interpreted. The asymptotic win only shows up somewhere around a thousand characters. So I would write the sorted one-liner by default, and switch to counting for long strings or if you specifically want better than $O(k \log k)$.”

9 Recall card

- **Anagram** = **same multiset of characters**. Find a **canonical form**: sorted letters, or letter counts.
- `sorted(a) == sorted(b)` is $O(k \log k)$; **counting** is $O(k)$. Give both, then say which you would ship.
- **Length check first** — free, and it is what makes the single-array subtract-and-bail version correct.
- **Grouping** = **canonical form as a dictionary key**, one pass. Keys must be hashable: `"".join(sorted(w))` or `tuple(counts)`.
- **A set is the wrong structure** — it loses multiplicities, so `aacc` and `ccac` compare equal.

1 What this is, and why they ask it

gRPC is a way for one service to call a function in another service as though it were a local function call. You write down the functions and their arguments in a **schema file**, a compiler generates client and server code in whichever languages you use, and the call travels as compact binary over a long-lived HTTP/2 connection.

That is three separate changes from REST, and each one buys something:

- **A schema, checked at compile time** — you cannot send a field that does not exist, or forget a required one, because the code will not build.
- **Binary encoding instead of JSON** — the field names are not on the wire at all, only the values, which is roughly three to ten times smaller.
- **One persistent HTTP/2 connection instead of a request per connection** — no repeated handshakes, many calls in flight at once, and streaming in both directions.

Interviewers ask this because it is where a candidate reveals whether they think about the *inside* of a system. Almost everyone can talk about the public API. gRPC is a question about the traffic between your own services, which at any real company is the overwhelming majority of it — Google runs on the order of tens of billions of internal calls per second on this. Knowing when to use it, and what you lose, is a mid-level-and-above signal.

2 The story

There is a small shop near the auto stand where Sanjay does mobile recharges, bus tickets, bill payments and photocopies, and there are usually three or four people standing in front of it.

Ravi has been going there every month for six years. He walks up, says “*nine eight seven six five four three two one zero, one nine nine, Jio*”, hands over two hundred rupees, and walks away with the change. It takes eleven seconds and neither of them says a full sentence.

That works because they both know the same three things, in the same order, without being told: number, amount, operator. Ravi does not need to say “*the number I want to recharge is*”, because what else would the first thing be. Sanjay does not need to ask “*and*

which operator?”, because it is always third. The form is in both their heads, so only the answers have to be spoken.

Behind Ravi in the queue there is a man doing this for the first time, and it takes him a minute and a half. He explains that he wants to put money on his wife’s phone, and Sanjay asks for the number, and he reads it out, and then Sanjay asks how much, and then which company, and then he is not sure and has to check. Everything he says is perfectly clear to anybody listening. It is just three times longer.

Two things Ravi has noticed over six years.

Once, the shop switched to a different system where the amount came before the number. Ravi said his usual three things in his usual order and Sanjay had already typed the first four digits into the amount box before he stopped. They both laughed, but it was a genuine mistake, and it happened because one of them had changed the form and the other had not been told.

And when Sanjay’s nephew was minding the shop in December, Ravi’s eleven-second version was useless. The boy had not learnt the order. Ravi had to go back to full sentences, like the man behind him, and it took him a minute and a half like everybody else.

3 The idea in plain English

Ravi’s eleven seconds is gRPC. The man behind him is JSON over HTTP. Both convey the same three values; one of them names each field out loud and the other relies on a form both sides already have.

What is actually on the wire

A REST call sends the field names with every message, every time:

```
{"user_id": 42, "name": "Ravi", "active": true, "score": 1200}
```

That is 58 bytes, and 34 of them are the words `user_id`, `name`, `active` and `score`, plus quotes, colons and braces. Send it a billion times and you have sent those four words a billion times.

The gRPC equivalent sends the values and a small tag saying which field each one is:

```
08 2A 12 04 52 61 76 69 18 01 20 A0 09
```

13 bytes. The names are nowhere on the wire, because both sides already have the form.

That form is the `.proto` file — the schema, written in a language called Protocol Buffers:


```

syntax = "proto3";

message User {
    int32  user_id = 1;      // the number 1 is the FIELD TAG, not a value
    string name     = 2;
    bool   active   = 3;
    int32  score    = 4;
}

service UserService {
    rpc GetUser (GetUserRequest) returns (User);
    rpc ListUsers (ListUsersRequest) returns (stream User);
}

```

The numbers after the equals signs are **field tags**, and they are what travels instead of the names. Tag 2 means `name`. That is Ravi’s “the second thing I say is the amount”.

Those tags are a permanent promise. Change `name` from tag 2 to tag 5 and every old client is suddenly reading the wrong field — which is exactly the morning Sanjay’s form changed and Ravi’s phone number went into the amount box. You may rename a field freely, because the name is not on the wire. You may **never** reuse or renumber a tag.

The generated code

You run the compiler — `protoc` — on the `.proto` file and it produces real classes and methods in your language. Then calling a remote service looks like calling a local object:

```

response = user_stub.GetUser(GetUserRequest(user_id=42))
print(response.name)

```

No URL, no JSON parsing, no checking a status code by hand. And because it is generated from a schema, `response.name` is a compile-time or lint-time error rather than a `None` at three in the morning. **That is the biggest day-to-day benefit, and it has nothing to do with speed.**

The style has a name — **RPC**, remote procedure call — and it is older than REST. gRPC is the modern, well-engineered version of a very old idea.

HTTP/2, and why it matters here

REST usually runs on HTTP/1.1, where one connection carries one request at a time. gRPC requires **HTTP/2**, which brings three things:

- **Multiplexing.** Many calls in flight on one connection at once, with no head-of-line blocking at the HTTP layer.

- **A persistent connection.** Set it up once and keep it. No repeated TCP and TLS handshakes — and from [day 006](#), a cold HTTPS connection costs several round trips before a single byte of your request is sent.
- **Header compression.** Repeated headers are sent as small references rather than in full.

Streaming, which REST simply does not do

Because the connection stays open, gRPC offers four call shapes:

SHAPE	WHAT IT LOOKS LIKE	EXAMPLE
Unary	one request, one response	<code>getUser(id)</code> — the ordinary case
Server streaming	one request, many responses	“send me every trade as it happens”
Client streaming	many requests, one response	uploading a file in chunks
Bidirectional	both at once, independently	a chat service, live telemetry

Request-and-response is the only shape REST has. If you need a stream over REST you reach for websockets or server-sent events, which are a separate mechanism with separate tooling. In gRPC it is a keyword in the schema.

What you give up, and it is not small

You cannot use `curl`. The message is binary, so you cannot read a request in a log, cannot paste one into a browser, and cannot eyeball a response. There is `grpcurl`, and it needs the schema. That single fact is why gRPC is used **inside** systems and almost never for public APIs.

Browsers cannot speak it directly. Browser JavaScript has no access to the raw HTTP/2 frames gRPC needs, so a web front end talks to a proxy — **gRPC-Web** with Envoy — that translates. It works, and it is an extra component in your path.

The schema must be shared. Both sides need the same `.proto`, which means a repository for them, a build step, and a discipline about changes. That is real organisational overhead, and it is the morning Sanjay’s nephew did not know the order.

No HTTP caching. Same problem as GraphQL on [day 021](#): a CDN cannot cache what it cannot understand. For internal calls that is usually irrelevant, because the responses were per-caller anyway.

4 The picture

The same call, both ways:

REST + JSON over HTTP/1.1

POST /users HTTP/1.1

Host: users.internal

Content-Type: application/json

Content-Length: 58

Authorization: Bearer ...

```
{ "user_id": 42, "name": "Ravi",  
  "active": true, "score": 1200 }
```

~250 bytes with headers

new connection per client, often

one request at a time per connection

readable by anyone

gRPC + protobuf over HTTP/2

[one connection, opened once, kept open]

stream 7: GetUser

08 2A (2 bytes)

stream 9: GetUser

08 2B (in parallel)

stream 11: ListUsers

... (streaming back)

~13 bytes of payload

headers compressed to a few bytes

many calls at once on one connection

needs the .proto to read at all

What to notice: the header block on the left is bigger than the body, and it is re-sent on every single request. On the right the connection was negotiated once and the per-call cost is almost entirely the data itself.

Where each belongs in a real system:

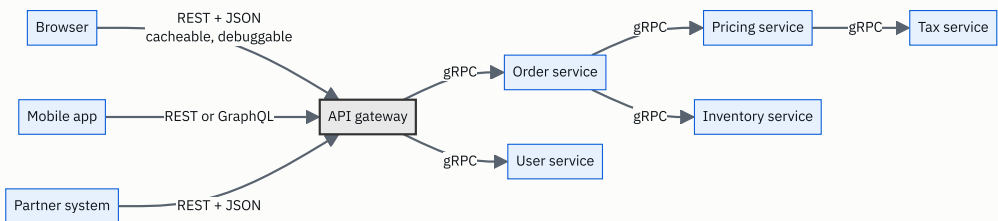


FIGURE 22.1

What to notice: the line through the gateway. Outside it, traffic is human-readable, cacheable and consumed by clients you do not control. Inside it, traffic is dense, typed, and between machines whose code you own on both sides. **That boundary is the answer to today's question**, and drawing it is worth more than any list of features.

5 How it actually works

Protocol Buffers on the wire

Each field is encoded as a **tag byte** followed by the value. The tag byte packs the field number and a wire type — a 3-bit code saying whether what follows is a varint, a fixed 32 or 64 bits, or a length-delimited block.

```
field 1 (user_id), varint, value 42      → 08 2A
field 2 (name), length-delimited, 4 bytes → 12 04 52 61 76 69    ("Ravi")
field 3 (active), varint, value 1        → 18 01
field 4 (score), varint, value 1200      → 20 A0 09
```

Two consequences worth knowing:

Small numbers are cheap. A **varint** uses one byte for values under 128, two under 16,384, and so on. So `42` costs one byte where JSON's `"score":1200` costs eleven.

Absent fields cost nothing. A field that is not set is simply not on the wire. JSON either sends `"middle_name": null` or omits it and leaves the receiver guessing.

Evolving a schema without breaking anyone

This is the part with real rules, and it is what the field tags are for.

Safe: - Add a new field with a **new** tag. Old clients ignore what they do not recognise.
- Rename a field. The name is not on the wire. - Delete a field, provided you mark its tag `reserved` so nobody reuses it later.

Never: - Change a field's tag number. - Change a field's type in an incompatible way (`int32` to `string`). - Reuse a tag from a deleted field.

```
message User {  
    reserved 3;                                // 'active' used to live here – never reuse it  
    reserved "active";  
    int32 user_id = 1;  
    string name = 2;  
    int32 score = 4;  
    string email = 5;                          // new field, new tag, old clients unaffected  
}
```

In proto3 every field is optional and unset fields read as a zero value, which makes adding fields inherently backward compatible. **This is why gRPC services do not carry /v1 in a path the way REST does** — the schema evolves in place, and versioning is a property of field tags rather than of URLs.

Deadlines, which are mandatory rather than optional

Every gRPC call carries a **deadline** — an absolute time by which it must complete — and it **propagates**. If the gateway gives a call 2 seconds and it spends 300 ms in the order service, the call the order service makes onward inherits 1.7 seconds, not a fresh 2.

This matters more than it sounds. Without propagation, a chain of five services each with its own 2-second timeout can take 10 seconds while the original caller gave up after 2, and every one of those five is still doing work nobody will ever read. Deadline propagation is one of the genuinely good ideas in gRPC and it is worth naming.

Alongside it: **cancellation** propagates too. If the caller goes away, everything downstream is told to stop.

Errors

gRPC has its own status codes — 17 of them, rather than HTTP’s several dozen — and they are the same in every language: `OK`, `NOT_FOUND`, `ALREADY_EXISTS`, `PERMISSION_DENIED`, `UNAUTHENTICATED`, `RESOURCE_EXHAUSTED`, `DEADLINE_EXCEEDED`, `UNAVAILABLE`, `INTERNAL`, and so on. `UNAVAILABLE` and `DEADLINE_EXCEEDED` are the two that mean “retry may help”, and gRPC libraries know that, so retry policy can be configured rather than hand-written.

Load balancing is different, and it catches people out

An ordinary layer-4 load balancer balances **connections**. gRPC opens one connection and keeps it, so all of a client’s traffic pins to whichever backend it first reached — and adding a new backend receives nothing at all until connections happen to be re-established.

The fixes: a layer-7 proxy that understands HTTP/2 and balances individual streams (Envoy, nginx with gRPC support, Linkerd), or client-side load balancing where the client resolves all backend addresses and spreads calls itself. **This is the most common operational surprise with gRPC**, and mentioning it unprompted is a strong signal that you have run it rather than read about it.

Who uses it

Google — where it was built, and where essentially all internal traffic runs on it or its predecessor Stubby. **Netflix, Square, Dropbox, Cloudflare, Uber** internally. **etcd** and **Kubernetes** speak gRPC between components. **CockroachDB** and **TiDB** use it between nodes.

And who does not: **Stripe, Twilio, GitHub, AWS’s public APIs** are all REST or JSON-based, for the same reason in every case — their callers are strangers with any language, any tooling, and a strong preference for being able to try something with `curl`.

6 The numbers

Payload size

The `User` message above:

```
JSON      : {"user_id":42,"name":"Ravi","active":true,"score":1200} = 58 bytes
protobuf  : 08 2A 12 04 52 61 76 69 18 01 20 A0 09                = 13 bytes
ratio     : 4.5x smaller
```

Now the headers, which are the bigger half for small messages:

```
HTTP/1.1 request headers (Host, Content-Type, Content-Length, Auth, ...) = 200 bytes, every request
HTTP/2 with HPACK compression, after the first                               = 10-20 bytes
```

Total for one call:

```
REST : 200 + 58 = 258 bytes
gRPC : 15 + 13 = 28 bytes          about 9x smaller
```

At **100,000 internal calls per second**, which is unremarkable for a mid-size company:

```
REST : 258 × 100,000 = 25.8 MB/s = 2.2 TB/day
gRPC : 28 × 100,000 = 2.8 MB/s = 242 GB/day
saving                                     ≈ 2 TB/day
```

Two terabytes a day of cross-service traffic that simply does not exist. Inside one data centre that is mostly a latency and CPU win; across availability zones, where transfer is billed at roughly \$0.01 per GB in each direction, it is real money:

```
2,000 GB/day × $0.02 × 365 ≈ $14,600/year
```

Not huge on its own — and it is the smallest of the three savings.

Serialisation CPU

Parsing JSON means scanning text, matching field names as strings, and allocating. Protobuf means reading a tag and copying bytes.

```
JSON parse of a 1 KB message      ≈ 10 µs
protobuf parse of the equivalent ≈ 1-2 µs
```

At 100,000 calls per second, decoded once on each side:

```
JSON      : 100,000 × 2 × 10 µs = 2.0 seconds of CPU per second = 2 cores
protobuf  : 100,000 × 2 × 2 µs = 0.4 seconds of CPU per second = 0.4 cores
```

Roughly 1.6 cores freed, on every service in the chain. Multiply by twenty services and it is a rack.

Connection setup — usually the largest win

From [day 006](#), a cold HTTPS connection costs a TCP handshake plus a TLS handshake: 2–3 round trips before the request is sent. Inside a data centre a round trip is about 0.5 ms.

```
new connection per call : 3 × 0.5 ms = 1.5 ms of pure setup, before any work
persistent HTTP/2       : 0 ms after the first call
```

On a call whose actual work is 2 ms, that is a **75% latency reduction from connection reuse alone**. And in a chain of five services:

```
REST, cold connections : 5 × (1.5 + 2) = 17.5 ms
gRPC, warm connection  : 5 × (0   + 2) = 10 ms
```

Note honestly that HTTP keep-alive gives REST most of this too, so it is a benefit of persistent connections rather than of gRPC specifically — and saying so is the mark of an honest answer.

When none of this matters

```
1,000 internal calls/second, 500-byte messages:
  REST : 500 × 1,000 = 500 KB/s
  JSON parsing      = 1,000 × 2 × 10 µs = 0.02 cores
```

Twenty milliseconds of CPU per second and half a megabyte a second. **At this scale gRPC saves you nothing you can measure, and costs you a build step, a schema repository, a proxy for the browser, and the ability to debug with `curl`**. Being able to say *that* — that the numbers do not justify it below a certain scale — is worth more in an interview than reciting the advantages.

7 The trade-offs

What gRPC costs

Debuggability. This is the big one and it is not a small inconvenience. You cannot read a request in a log, cannot reproduce one from a browser, and cannot hand a colleague a `curl` command. Every tool in your organisation — proxies, WAFs, log aggregators, the intern's script — has to be taught about it.

Tooling and build. A schema repository, a code-generation step in every language's build, and version skew between generated stubs. Real ongoing overhead for a small team.

Browsers. Not supported natively. gRPC-Web plus an Envoy proxy works and is one more moving part.

Load balancing. Long-lived connections defeat ordinary connection-level balancers, so you need a layer-7 proxy or client-side balancing.

No HTTP caching. Irrelevant inside, disqualifying outside.

Coupling. A shared schema is a shared dependency. Changing it means coordinating a rollout, and proto3's compatibility rules only protect you if everyone follows them.

What you would use instead

- **REST + JSON** for anything public, anything a browser touches, anything a partner integrates with, and any internal service where the call volume does not justify the machinery.
- **GraphQL** when many clients need different shapes of the same data — a different problem, from [day 021](#).
- **A message queue** — Kafka, RabbitMQ — when the caller does not need an answer now. gRPC is request-response; a queue is fire-and-forget with durability, and choosing gRPC where you wanted a queue is a much more expensive mistake than choosing REST where you wanted gRPC.
- **JSON over HTTP/2**, which is a real and underused middle: you get multiplexing and header compression and keep readability, giving up only the encoding size and the generated types.

The sentence that separates candidates

I would not use gRPC for a public API, and I would not use it internally below a few thousand calls a second. Public callers want `curl`, browsers, any language and no build step, and the CDN caching REST gives me is worth more than any encoding saving. Internally, at low volume, the win is a fraction of a core and a few megabytes a second, and the price is a schema repository, a generation step and losing the ability to read my own traffic. Where gRPC earns its keep is a service mesh with high call volume, deep call chains where deadline propagation actually prevents outages, polyglot teams who benefit from generated typed clients, and anything that genuinely needs streaming.

8 In the interview

How it gets asked

- *“Why would a company use gRPC between its own services?”* — the direct version. Three reasons and one boundary.
- *“What’s the difference between REST and RPC?”* — the conceptual version. REST models resources; RPC models function calls.
- *“Why not use gRPC for your public API?”* — the follow-up that checks whether you know the cost.
- *“How would two microservices communicate?”* — asked mid-design, where the right answer names more than one option and picks by condition.

What to say out loud, in the first ninety seconds

1. **Say what it is in one sentence.** *“gRPC lets one service call a function in another as if it were local. You define the functions in a schema file, a compiler generates typed clients and servers, and calls travel as binary over a persistent HTTP/2 connection.”*
2. **Give the three wins, in order of what actually matters.** *“First, the schema — the contract is checked at compile time, so a wrong field name is a build error rather than a null at three in the morning. Second, the connection — HTTP/2 is persistent and multiplexed, so you stop paying a handshake per call. Third, the encoding — field names aren’t on the wire, so messages are three to ten times smaller and parse several times faster.”*
3. **Lead with the schema, not the speed.** Most candidates lead with binary. The generated typed client is the thing engineers actually feel every day.
4. **Give one number.** *“At a hundred thousand internal calls a second, that’s roughly 2 terabytes a day of traffic that doesn’t exist, and a couple of cores per service freed from JSON parsing.”*
5. **Mention streaming.** *“And because the connection is open, gRPC gives you server streaming, client streaming and bidirectional streaming as part of the schema — REST only does request-response.”*
6. **Draw the boundary immediately.** *“But it’s for inside. Public APIs stay REST, because callers want curl, browsers and any language — which is why Stripe and GitHub are REST and Google’s internals are not.”*
7. **Name a cost unprompted.** *“The real price is debuggability. You cannot read a binary request in a log, and every tool in the organisation has to be taught about it.”*

The follow-ups

“Why not use gRPC for a public API?” Four reasons, and any one of them is usually enough. Debuggability: an external developer wants to paste a `curl` command and see JSON, and a binary payload they cannot read without your schema is a serious barrier to adoption. Browsers: JavaScript cannot make raw gRPC calls, so every web client needs a gRPC-Web proxy, which is infrastructure you are asking your callers to care about. Caching: a CDN cannot cache what it cannot parse, and for a public read-heavy API that is the single largest lever you have — from the earlier arithmetic, a 90% CDN hit rate took origin load from 420 requests a second to 118. And coupling: a shared `.proto` is a shared dependency, and you cannot make thousands of external developers regenerate their stubs on your schedule. Internally none of those apply, because you own both sides and deploy them together.

“What’s the actual difference between REST and RPC?” They model different things. REST models **resources** — nouns with addresses, and a fixed small set of verbs applied uniformly to all of them, which is why you can guess `/users/42` once you have

seen `/orders/17`. RPC models **procedures** — you are calling a named function with arguments, so the API is a list of operations rather than a list of things. That makes RPC a much more natural fit when the operations genuinely are actions: `RecalculateInvoice`, `RebalanceShards`, `TranscodeVideo`. Forcing those into resources produces the awkward invented nouns we discussed on [day 017](#). The cost is that RPC gives up everything HTTP knows how to do for you — caching, idempotency semantics, the meaning of a `GET`. So: resources and public callers, REST; actions and internal callers, RPC.

“How do you version a gRPC API?” Mostly you do not, in the URL sense, because Protocol Buffers is designed to evolve in place. Adding a field with a **new** tag number is backward compatible — old clients ignore tags they do not know, and in proto3 an unset field reads as its zero value. Renaming a field is free, because the name is not on the wire. Deleting a field is fine as long as you mark its tag `reserved` so nobody reuses it. What you must never do is change a field’s tag number or its type, because the tag is the identity of the field and old clients would silently read the wrong thing. For a genuinely breaking change you introduce a new package — `myservice.v2` — and run both services for a migration period, exactly as with REST, but that should be rare rather than routine.

“What happens to load balancing?” This is the operational surprise, and it catches most teams once. A normal layer-4 load balancer distributes **connections**, and gRPC opens one long-lived connection and keeps it — so all of a client’s calls pin to whichever backend it first reached, and a newly added backend gets no traffic at all. There are two fixes. A layer-7 proxy that understands HTTP/2 and balances individual streams rather than connections — Envoy, Linkerd, nginx with gRPC support. Or client-side load balancing, where the client resolves all backend addresses and picks per call, which removes a hop and moves the policy into every client. Most service meshes take the first route, and that is one of the main reasons service meshes exist.

A model answer

“gRPC lets one service call a function in another as though it were a local call. You define the service and its messages in a `.proto` schema file, a compiler generates client and server code in whatever languages you use, and the calls go as compact binary over a persistent HTTP/2 connection.

There are three wins and I’d rank them in the opposite order from how they’re usually presented.

The one engineers feel every day is the **schema**. The contract is a file, and the client is generated from it, so a misspelt field is a build failure rather than a `None` at three in the morning. With JSON over REST the contract lives in documentation and in hope.

The one that usually matters most for latency is the **connection**. HTTP/2 is persistent and multiplexed, so you set up TCP and TLS once instead of per call. Inside a data centre a handshake is around 1.5 milliseconds against maybe 2 milliseconds of actual work, so connection reuse alone can nearly halve the latency of a chain of calls. I’d be honest that HTTP keep-alive gives REST most of this too — it’s a benefit of persistent connections rather than of gRPC as such.

The one people lead with is the **encoding**. Field names aren’t on the wire, only small numeric tags, so that `User` message is 13 bytes against 58 as JSON, and with HTTP/2 header compression the whole call is about 28 bytes against 258. At a hundred thousand internal calls a second that’s roughly 2 terabytes a day of traffic that simply doesn’t exist, and parsing is about five times cheaper, which is a core or two freed on every service in the chain.

Two more things I’d mention. Streaming is part of the schema — server streaming, client streaming and bidirectional — where REST only does request-response and you’d reach for websockets. And deadlines propagate: if the gateway gives a call two seconds and 300 milliseconds are spent in the first hop, the next hop inherits 1.7 seconds rather than a fresh two. Without that, a five-service chain can spend ten seconds doing work the original caller abandoned after two.

But all of that is about traffic **inside** the system. I’d keep REST at the edge — for browsers, mobile apps and partners — because those callers want `curl`, any language, no build step, and because CDN caching is worth more to a public read-heavy API than any encoding saving. That’s why Google’s internals are gRPC and Stripe’s public API is REST.

And the cost I'd name unprompted is debuggability. You cannot read a binary request in a log or reproduce one from a browser, and every proxy and logging tool has to be taught about it. Plus the load balancing trap: long-lived connections defeat connection-level balancers, so you need a layer-7 proxy or client-side balancing, and that surprises most teams the first time they add a backend and watch it receive nothing."

9 Recall card

- **gRPC = call a remote function like a local one.** Schema (`.proto`) → generated typed clients → binary over persistent HTTP/2.
- **Three wins, in the order that matters:** compile-time contract, persistent multiplexed connection, small fast encoding. Plus streaming and propagating deadlines.
- **Field tags are the identity**, not names. Add new tags freely; never renumber or reuse one.
- **The boundary is the answer:** gRPC inside, REST at the edge. Public callers want `curl`, browsers, any language, and CDN caching.
- **The costs:** no `curl`, no browsers without a proxy, no HTTP caching, and long-lived connections break layer-4 load balancing.

PRACTICE

Practice — Anagrams: the sorting versus counting choice

DSA topic: Anagrams: the sorting versus counting choice **System design topic:** gRPC and when binary protocols win

Code these, in this order

Four problems built on one idea: **find a canonical form, then compare or group by it**. For every one of them, say out loud what your canonical form is before writing a line.

Before each one, ask:

1. What is my canonical form, and why is it identical for exactly the things that should match?
2. Is my key hashable? (A list is not.)
3. Does the length check help, and is anything else relying on it?
4. Case, spaces, Unicode — what did I assume, and did I say so?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Valid Anagram	LeetCode 242 (Easy)	Two solutions with different complexities, offered unprompted. The one-liner alone is an incomplete answer.
2	Group Anagrams	LeetCode 49 (Medium)	The canonical form as a dictionary key, and knowing that a list cannot be one.
3	Find All Anagrams in a String	LeetCode 438 (Medium)	A count that slides. Recomputing per window is $O(n \cdot k)$; updating two entries per step is $O(n)$.
4	Group Shifted Strings	LeetCode 249 (Medium)	Inventing a canonical form yourself. <code>abc</code> and <code>bcd</code> are the same “shape” — what key captures that? This is the real skill.

On problem 1, produce both

Write `sorted(a) == sorted(b)` first, state $O(k \log k)$, and then say “*but I can do better*” without being asked. Write the counting version. Then write the single-array version that increments for `a` and decrements for `b`, and answer:

1. Why can it return early the moment a count goes negative?
2. Why does that early exit need the length check to be correct?
3. Run it on `("aab", "ab")` with the length check removed. What comes back, and why?

On problem 2, break your own key

Write the grouping solution with `groups[sorted(word)].append(word)` — with `sorted` and no `join` — and run it. Read the error. Say the reason in one sentence, using the word *hashable*.

Then write the count-key version and confirm both produce the same groups on `["eat", "tea", "tan", "ate", "nat", "bat"]`.

Then test both on `[]` and on `[""]`. One of those is easy to get wrong.

On problem 4, design the key before you code

`["abc", "bcd", "xyz", "az", "ba", "a", "z"]` groups into `[["abc", "bcd", "xyz"], ["az", "ba"], ["a", "z"]]`, because each of those can be shifted into the others.

Do not look anything up. Spend ten minutes deciding what a canonical form for “same shape” is. Say your candidate out loud, then find the input that breaks it. The wrap-around at `z` is where most first attempts fail.

This is the most valuable exercise on this page, because inventing a canonical form is the skill the whole day is about, and [day 064](#) is entirely built on it.

The set-versus-counter drill

Predict, then run:

```
print(set("aacc") == set("ccac"))
print(sorted("aacc") == sorted("ccac"))
print(sorted("aab") == sorted("abb"))
```

Then say in one sentence what a set throws away that a count keeps, and why that makes it the wrong structure for this problem but the right one for “how many distinct characters”.

The measurement drill

Run this and read the numbers before you decide what to ship.

```

from collections import defaultdict
import random, string, time

random.seed(1)

def g_sort(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())

def g_count(words):
    groups = defaultdict(list)
    for w in words:
        key = [0] * 26
        for ch in w:
            key[ord(ch) - ord("a")] += 1
        groups[tuple(key)].append(w)
    return list(groups.values())

for k in (10, 100, 1000, 5000):
    words = ["".join(random.choices(string.ascii_lowercase, k=k)) for _ in range(2000)]
    row = [f"k={k:>5}"]
    for f in (g_sort, g_count):
        s = time.perf_counter(); f(words)
        row.append(f"{f.__name__} {time.perf_counter()-s:.4f}s")
    print(" ".join(row))

```

Then answer:

1. At `k = 100`, which is faster? Is that what the complexity said would happen?
2. At what word length does the `O(k)` version start to win?
3. Explain the gap. (One is compiled C, the other is interpreted Python.)
4. Which would you ship for ordinary words, and which would you say in an interview?

Questions 3 and 4 have different answers, and being comfortable with that is the point.

The canonical-form drill

For each, name a canonical form in under ten seconds:

1. Two strings are anagrams.
2. Two words are the same ignoring case.
3. Two phone numbers are the same, written differently (`+91 98765 43210`, `09876543210`).
4. Two points are on the same line through the origin.
5. Two lists contain the same elements regardless of order **and** duplicates matter.
6. Two binary trees have the same shape and values.
7. Two strings are shifts of each other (`abc`, `bcd`).

Numbers 4 and 6 are the ones to think about; both come back much later in the course.

The gRPC drill

Answer each in one or two sentences, out loud:

1. What three things does gRPC change compared with REST + JSON over HTTP/1.1?
2. Which of those three do engineers actually feel every day, and why?
3. What is a field tag in a `.proto` file, and what may you never do to one?
4. Name the four call shapes gRPC supports. Which one does REST have?
5. What is deadline propagation, and what goes wrong without it?
6. Why can a browser not call a gRPC service directly?
7. Why does gRPC break an ordinary layer-4 load balancer, and what are the two fixes?
8. Draw the boundary: which traffic in a system is gRPC, and which is REST?

The arithmetic drill

From memory, in under two minutes:

- A four-field message as JSON versus protobuf — the two sizes and the ratio.
- With headers: a REST call versus a gRPC call, total bytes.
- At 100,000 calls a second — traffic per day, both ways, and the difference.
- JSON parse at 10 μ s versus protobuf at 2 μ s, decoded on both sides — cores at 100,000 calls/second.
- A cold HTTPS handshake at 3 round trips of 0.5 ms, against work of 2 ms — what fraction of the latency is setup?

Then say the sentence those numbers buy you: *“at a thousand calls a second none of this is measurable, and the price is a build step and losing `curl`.”*

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Are these two strings anagrams? Now group a list of words into anagram groups. Ask the contract questions. Name the canonical-form idea before either solution. Give the one-liner, then improve it unprompted. For grouping, say explicitly that there is no comparison between words — a dictionary turns $O(n^2 \cdot k)$ into one pass. Finish with which you would ship.*
2. *Why would a company use gRPC between its own services? Three changes, ranked by what matters. One number. Streaming and deadlines. Then draw the boundary — inside gRPC, edge REST — and name debuggability as the cost, unprompted.*

3. *Given ten thousand words, find any two that are anagrams of each other.* The trap is the nested loop. Say what the naive version costs, then the dictionary version, then what changes if this has to run on a million words across several machines. (The key is a pure function of the word, so sharding by key needs no merge step.)

Before you move on

- [] I say the words “canonical form” when I see this family of problem.
- [] I offer two solutions with different complexities, unprompted.
- [] I know a dictionary key must be hashable, and I reach for `tuple` or `".join` automatically.
- [] I never use a set where multiplicities matter, and I can name the input that proves it.
- [] I write the length check and can say what depends on it.
- [] I can name gRPC’s three changes and rank them by what engineers actually feel.
- [] I can say why field tags may never be renumbered.
- [] I answer “gRPC or REST” by drawing the boundary, not by picking a winner.
- [] I can redraw the inside-versus-edge diagram from memory, in whatever tool I like.

Palindromes and the two-ends habit

DATA STRUCTURES AND ALGORITHMS

Palindromes and the two-ends habit

You can check a palindrome with two pointers and handle the messy input rules.

SYSTEM DESIGN

Rate limiting and API gateways

You can describe the token bucket and say where the limiter sits in the architecture.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Is this string a palindrome, ignoring punctuation and case?*
- *How would you stop one client from hammering your API?*

1 What this is, and why they ask it

A **palindrome** is a sequence that reads the same forwards and backwards. `madam`. `racecar`. `12321`.

Checking one introduces a habit you will use for the next fifty days: **put one index at each end and walk them towards each other, comparing as you go**. Two indices, $O(n)$ time, $O(1)$ extra space, and the loop stops when they meet. That is the two-pointer pattern, arriving four days before [day 027](#) makes it official, and palindromes are the gentlest possible introduction to it.

The reason interviewers keep asking it is not the algorithm — everyone gets the algorithm. It is the **input rules**. The real question is almost always “*ignoring punctuation, spaces and case*”, and that turns a five-line problem into a test of whether you handle messy input carefully: which characters count, what happens when both ends land on characters that do not count, and what an empty string returns. Candidates who reach straight for `s = s[::-1]` get a correct answer and miss the point of the exercise, which is $O(1)$ space.

It is LeetCode 125, 680 and 5, and it is one of the most common opening questions at every level.

2 The story

The level crossing near Nithya’s school shuts for about twelve minutes at a time, and on a bad evening her father catches it twice. They sit in the car with the engine off and the window down, and they have a game they have played since she was seven.

They look at the number plate of whichever car is in front, and they see whether the numbers read the same both ways.

The plate says TN 09 BX 4554. Her father does not read the whole thing out and then reverse it in his head; he tried that once and gave up. What they do instead is that she takes the left end and he takes the right end, and they work towards each other.

She starts at the far left. T. That is a letter, so it does not count — the game is only about the numbers — and she moves along. N, also a letter, move along. Then a zero, and she stops there. He starts at the far right. Four. That counts, and he stops there. Zero against four, no match, and the game is over in about three seconds.

The next car is TN 37 AC 8228. She skips the letters and lands on the three; wait, no — she reads it again properly. The numbers, in order, are 3, 7, 8, 2, 2, 8. She is on the 3 and he is on the last 8. Three against eight. No.

Then a scooter goes past with 2002 on it and they both say it at the same time.

Two things about how they do it. Neither of them ever has to hold the whole plate in their head — she only ever remembers where she is, and he only remembers where he is. And they stop the moment their hands would cross. On 2002 she checks the first 2 against the last 2, then the 0 against the 0, and at that point she has reached where he is, and there is nothing in the middle left to check. On a plate with five numbers the middle one has nobody to be compared with, and they skip it, because a thing is always the same as itself.

The gates go up. He starts the car.

3 The idea in plain English

Nithya at the left and her father at the right are **two indices**, usually called `left` and `right`. Everything today is those two, and the rules for moving them.

The core loop

```
left, right = 0, len(s) - 1
while left < right:
    if s[left] != s[right]:
        return False
    left += 1
    right -= 1
return True
```

Start at the two ends. Compare. If they differ, it is not a palindrome and you are finished. If they match, step both inwards. If you get all the way to the middle without a mismatch, it is a palindrome.

`while left < right, not ≤`. When `left = right` the two indices are on the same character, and comparing a character with itself is always true — that is the middle number on a five-digit plate, with nobody to be compared with. Using `≤` is not wrong, merely one wasted comparison, but saying *why* `<` is enough shows you have thought about the odd-length case.

Notice what is **not** here: any copy of the string. `left` and `right` are two integers. That is the `O(1)` space, and it is the entire reason this version is better than reversing.

The reversing version, and when it is fine

```
def is_palindrome(s: str) → bool:  
    return s == s[::-1]
```

One line, correct, and — from [day 019](#) — `s[::-1]` builds a whole new string, so it costs `O(n)` extra space. Write it, say that, and then offer the two-pointer version. In real code with short strings the one-liner is what you would ship; in an interview asking for `O(1)` space it is the thing being ruled out.

The messy input rules, which are the actual question

The real problem says “*consider only alphanumeric characters and ignore case*”. Two rules, and each needs handling.

Which characters count. `ch.isalnum()` is `True` for letters and digits and `False` for spaces, commas, colons and everything else. That is Nithya skipping the letters on the plate — except in the real problem letters *do* count and punctuation does not.

Case. `A` and `a` must match, so compare `s[left].lower()` against `s[right].lower()`.

There are two ways to apply them.

Clean first, then check. Simple and readable:

```
cleaned = "".join(ch.lower() for ch in s if ch.isalnum())  
return cleaned == cleaned[::-1]
```

Two lines, obviously correct, and `O(n)` extra space for the cleaned copy.

Skip as you go. No copy at all, so `O(1)` space — and this is the version the question is asking for:

```
while left < right:  
    while left < right and not s[left].isalnum():  
        left += 1  
    while left < right and not s[right].isalnum():  
        right -= 1  
    if s[left].lower() != s[right].lower():  
        return False  
    left += 1  
    right -= 1
```

The two inner loops are the whole difficulty, and there are two things to get right about them.

They must be `while`, not `if`. A string can have several unwanted characters in a row — `"a,,, ;a"` — and one `if` skips only the first.

They must **also** test `left < right`. Without it, a string of nothing but punctuation runs `left` straight past `right` and off the end of the string. That inner guard is the difference between correct code and an `IndexError`, and §7 shows it.

Why the pointers can never cross wrongly

At the point of comparison, one of two things is true: either `left < right` and both are on characters that count, or `left == right`, in which case the loops stopped there and the comparison is a character against itself, which passes harmlessly before the outer condition ends the loop. Either way you never read outside the string. Being able to say that sentence is what separates code you trust from code you hope about.

The two shapes of palindrome, which matter later

```
odd length, centre is one character
r a c e c a r
    ^
  one middle

7 characters, 3 comparisons
```

```
even length, centre is a gap
a b b a
    ^
  between two

4 characters, 2 comparisons
```

For the simple check this only decides whether the last comparison happens. For *finding* palindromes — §5's longest-substring problem — it matters a great deal, because every centre has to be tried both ways, and forgetting the even case is the standard bug.

4 The picture

`"racecar"`, with the two indices walking inwards:

```

index  0  1  2  3  4  5  6
+---+---+---+---+---+---+
| r | a | c | e | c | a | r |
+---+---+---+---+---+---+
      ^               ^
    left             right    r = r, step both

          ^           ^
        left         right    a = a, step both

              ^       ^
            left     right    c = c, step both

                  ^
                left,right    left = right: STOP
                                nothing left to compare

```

What to notice: three comparisons for seven characters, not seven. Each step settles two positions at once, which is why the loop runs $n/2$ times.

Now "A man, a plan, a canal: Panama", where the skipping happens:

```

A  m a n ,   a   p l a n ,   a   c a n a l :   P a n a m a
^                               ^
left                           right    A vs a → match

      ^                       ^
    left                       right    m vs m → match

      ^ skip the comma and space      ^ skip the colon and space
      |                               |
+-- inner while advances left        +-- inner while retreats right

```

What to notice: the outer loop compares; the two inner loops do nothing but move past characters that do not count. Keeping those three loops separate in your head is the trick to writing this correctly under pressure.

The two centre shapes, for finding palindromes rather than checking one:

```

expand from a single character          expand from between two
  b a b a d                            c b b d
    ^                                  ^ ^
  i, i                                i, i+1
"aba" grows outward                    "bb" grows outward

every position gives TWO centres to try, so 2n - 1 centres in total

```

What to notice: $2n - 1$ centres, not n . A string of length 5 has 5 single-character centres and 4 gaps between characters. Missing the gaps means never finding an even-length palindrome, and "cbbd" is the four-character input that exposes it.

5 The code, built step by step

The simplest correct thing

```
def is_palindrome_slice(s: str) → bool:
    cleaned = "".join(ch.lower() for ch in s if ch.isalnum())
    return cleaned == cleaned[::-1]
```

Filter, lower-case, join — the pattern from [day 020](#) — then compare against the reverse. Write this first in an interview. It is correct, it takes fifteen seconds, and it gives you something to improve.

Its cost: $O(n)$ time and $O(n)$ extra space, because `cleaned` and `cleaned[::-1]` are two new strings.

The two-pointer skeleton

```
left, right = 0, len(s) - 1
while left < right:
    if s[left] != s[right]:
        return False
    left += 1
    right -= 1
return True
```

Correct when every character counts. `left < right` because the middle character of an odd-length string needs no comparison.

Adding the skip loops

```
while left < right and not s[left].isalnum():
    left += 1
while left < right and not s[right].isalnum():
    right -= 1
```

These go **inside** the outer loop, before the comparison. `while` and not `if`, because runs of punctuation exist. And `left < right` inside each condition, because a string with no alphanumeric characters at all would otherwise walk off the end.

The comparison, case-folded

```
if s[left].lower() != s[right].lower():
    return False
```

`.lower()` on a single character is cheap. Some solutions upper-case instead; either is fine as long as you do the same to both sides.

Putting it together

```
def is_palindrome(s: str) → bool:
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() ≠ s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

Eleven lines, `O(n)` time, `O(1)` extra space, and it handles `" "`, `" "`, `".,"` and `"0P"` correctly. Test all four.

That last one is worth pausing on. `"0P"` is **not** a palindrome — `0` and `P` are both alphanumeric and they differ. A solution that uses `isalpha()` instead of `isalnum()` skips the `0` and wrongly returns `True`. It is the standard trap input for this problem.

Allowing one deletion

LeetCode 680: “can it become a palindrome by deleting at most one character?” Walk inwards as usual, and at the first mismatch there are exactly two possibilities — delete the left character, or delete the right one:

```
def valid_palindrome(s: str) → bool:
    def is_pal(i: int, j: int) → bool:
        while i < j:
            if s[i] ≠ s[j]:
                return False
            i += 1
            j -= 1
        return True

    left, right = 0, len(s) - 1
    while left < right:
        if s[left] ≠ s[right]:
            return is_pal(left + 1, right) or is_pal(left, right - 1)
        left += 1
        right -= 1
    return True
```

Why only two possibilities? Because everything outside `left` and `right` has already matched pairwise, so the deletion has to happen at one of those two positions — deleting anywhere else leaves the mismatch untouched. That argument is the whole answer, and it is what turns an apparently exponential problem into two linear checks.

The cost stays $O(n)$: the main loop is at most $n/2$ steps, and the branch happens **at most once**, after which each helper call is another $n/2$ at worst.

Finding the longest palindromic substring

LeetCode 5. The idea: every palindrome has a centre, so try every centre and grow outwards while the characters match.

```
def expand(s: str, left: int, right: int) → tuple[int, int]:
    while left ≥ 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    return left + 1, right - 1          # step back to the last matching pair
```

The `left + 1, right - 1` at the end is the off-by-one people get wrong. The loop exits **after** stepping one too far — either off the end of the string or onto a mismatch — so the answer is one step back on each side.

```
for i in range(len(s)):
    a, b = expand(s, i, i)           # odd-length centre
    ...
    a, b = expand(s, i, i + 1)       # even-length centre
```

Both centres for every position, which is the $2n - 1$ from §4.

The complete solutions

```
def is_palindrome_slice(s: str) → bool:
    """LeetCode 125, the readable version. O(n) time, O(n) space."""
    cleaned = "".join(ch.lower() for ch in s if ch.isalnum())
    return cleaned == cleaned[::-1]

def is_palindrome(s: str) → bool:
    """LeetCode 125 in O(1) space. Two indices walking inwards, skipping what does not count."""
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum(): # while, not if
            left += 1
        while left < right and not s[right].isalnum(): # and guard left < right
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True

def valid_palindrome(s: str) → bool:
    """LeetCode 680. Palindrome after deleting at most one character."""
    def is_pal(i: int, j: int) → bool:
        while i < j:
            if s[i] != s[j]:
                return False
            i += 1
            j -= 1
        return True

    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            # everything outside left/right already matched, so the deletion
            # must be at one of these two positions
            return is_pal(left + 1, right) or is_pal(left, right - 1)
        left += 1
        right -= 1
    return True

def longest_palindrome(s: str) → str:
    """LeetCode 5. Expand around every centre. O(n^2) time, O(1) space."""
    if not s:
        return ""

    def expand(left: int, right: int) → tuple[int, int]:
        while left ≥ 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return left + 1, right - 1 # the loop overshot by one on each side

    start, end = 0, 0
    for i in range(len(s)):
        a, b = expand(i, i) # odd-length palindrome centred on i
        if b - a > end - start:
            start, end = a, b
        a, b = expand(i, i + 1) # even-length palindrome centred between i and i+1
        if b - a > end - start:
            start, end = a, b
    return s[start:end + 1]

if __name__ == "__main__":
    cases = ["A man, a plan, a canal: Panama", "race a car", "", " ", ".", "0P",
            "aba", "abba", "ab", "a"]
    print([is_palindrome_slice(c) for c in cases])
    # [True, False, True, True, True, False, True, False, True]
    print([is_palindrome(c) for c in cases])
```

```
# identical

print([valid_palindrome(x) for x in ("aba", "abca", "abc", "", "a", "deeee")])
# [True, True, False, True, True, True]

print([longest_palindrome(x) for x in ("babad", "cbbd", "a", "", "ac", "forgeeksskeegfor")])
# ['bab', 'bb', 'a', '', 'a', 'geeksskeeg']
```

6 What it costs

`is_palindrome`, the two-pointer version

Time. Each turn of the outer loop moves `left` forward at least one and `right` back at least one. The inner skip loops also only ever move them in those directions. So across the whole run, `left` moves at most `n` positions in total and `right` moves at most `n`, and they stop when they meet — **every character is looked at at most once by each index**. That is $O(n)$ time, not $O(n^2)$, even though there are loops inside a loop.

That last sentence is worth practising, because nested `while` loops make people say $O(n^2)$ reflexively. The right argument is not “there are two loops” but “count how far the indices travel in total”.

Space. Two integers. $O(1)$ extra space — and this is the whole reason the two-pointer version exists.

`is_palindrome_slice`

Time. One pass to filter and lower-case, one to join, one to reverse, one to compare: $4n$ steps, so $O(n)$ — the same class.

Space. `cleaned` is up to `n` characters, and `cleaned[::-1]` is another `n`. $O(n)$ extra space, which at a hundred megabytes of input is two hundred megabytes of copies for a question that needs two integers.

`valid_palindrome`

The main loop is at most $n/2$ turns. The branch fires **at most once**, and each helper call is at most another $n/2$ turns. Worst case $n/2 + n/2 + n/2$, so $O(n)$ time and $O(1)$ space.

Compare with the naive answer — try deleting each of the `n` characters and check each result, which is `n` deletions $\times$ $O(n)$ check = $O(n^2)$, plus $O(n)$ space per deleted copy. The insight that only two positions can possibly matter is what removes a whole factor of `n`.

longest_palindrome

$2n - 1$ centres. Each `expand` runs until it fails, which is at most $n/2$ steps. So $(2n - 1) \times n/2$, which is about n^2 : **$O(n^2)$ time, $O(1)$ space.**

At $n = 1,000$ that is around a million operations — instant. At $n = 100,000$ it is ten billion, which is not. If an interviewer pushes past $O(n^2)$ there is Manacher's algorithm at $O(n)$; it is genuinely tricky and almost never expected. **The right move is to name it, say it is $O(n)$, and say you would look it up rather than derive it under time pressure.** That is a better answer than a half-remembered attempt.

The number to have ready

Two pointers: $O(n)$ time and $O(1)$ space, and the nested skip loops do not make it quadratic because each index only ever travels forward. Reversing is the same $O(n)$ time but $O(n)$ space. Longest-palindromic-substring by expanding around $2n - 1$ centres is $O(n^2)$.

7 The traps

The real error: forgetting `left < right` in the skip loops

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        while not s[left].isalnum():           # no left < right guard
            left += 1
        while not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True

print(is_palindrome("."))
```

```
Traceback (most recent call last):
  File "t.py", line 12, in <module>
    print(is_palindrome(".", ""))
    ~~~~~^~~~~~
  File "t.py", line 5, in is_palindrome
    while not s[left].isalnum():
    ~~~~~^~~~~~
IndexError: string index out of range
```

".,," has no alphanumeric characters, so the first inner loop walks `left` past `right`, past the end of the string, and off into nothing. **Every skip loop needs the bound test in its own condition.** Test with `".,,"`, `" "` and `"!!!"` — those three inputs find this bug and almost nothing else does.

The near-miss: `isalpha` instead of `isalnum`

```
cleaned = "".join(ch.lower() for ch in s if ch.isalpha())
print(cleaned == cleaned[::-1])      # on "0P"
```

```
True
```

Wrong. `"0P"` is not a palindrome — the digit and the letter are both alphanumeric and they differ. `isalpha()` drops the `0`, leaving `"p"`, which reads the same both ways. This is the standard trap input on LeetCode 125 and it exists precisely to catch this. Read the problem statement: it says **alphanumeric**.

The near-miss: `if` instead of `while` when skipping

```
if not s[left].isalnum():
    left += 1
```

One skip per turn. On `"a,,, ,a"` the left index steps past one comma, then compares a comma against an `a`, and returns `False` for a string that is a palindrome under the rules. Runs of unwanted characters are normal in real text, so this fails on almost any realistic sentence.

The near-miss: expanding without stepping back

```
def expand(s, left, right):
    while left >= 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    return left, right      # forgot the +1 / -1
```

The loop exits **after** the step that failed, so `left` and `right` are each one position too far out. The returned substring is two characters too long and includes the mismatching pair — or `s[left:right+1]` raises, or silently produces nonsense from a negative index. Return `left + 1, right - 1`.

The near-miss: only trying odd centres

```
for i in range(len(s)):
    a, b = expand(i, i)      # and nothing else
```

Every palindrome found will have odd length. On "cbbd" the answer is "bb" and this returns "c". "abba", "cbbd" and "aa" are the inputs that expose it — all of them even-length, all of them easy to leave out of your own test list because the first example you were given was "babad".

The near-miss: greedily choosing which character to delete

```
if s[left] != s[right]:
    if s[left + 1] == s[right]:
        left += 1          # assume deleting the left one is right
    else:
        right -= 1
    ...
```

This decides on one character of look-ahead and can be wrong. On "ecccccbabaeabebcccees" a greedy choice at the first mismatch commits to a branch that fails later while the other branch would have succeeded. **You must actually try both**, which is what `is_pal(left+1, right)` or `is_pal(left, right-1)` does — and because the branch fires at most once, trying both is still $O(n)$.

8 In the interview

How it gets asked

- “Is this string a palindrome, ignoring punctuation and case?” — LeetCode 125. The last five words are the question.
- “Do it without using extra space.” — the follow-up that rules out `s[::-1]` and asks for two pointers.
- “Can it be a palindrome if you delete at most one character?” — LeetCode 680, and the interesting one, because the two-branch insight is a real idea.
- “Find the longest palindromic substring.” — LeetCode 5, expanding around centres.
- “Is this number a palindrome, without converting it to a string?” — the arithmetic variant, using `% 10` and `// 10`.

What to say out loud, in the first ninety seconds

1. **Pin the rules.** “Which characters count — letters and digits only, or everything? Is it case-insensitive? And what should an empty string return?” All three change the code, and the empty case is usually `True`.
2. **Give the simple version and its cost.** “The simplest correct answer is to strip out the non-alphanumeric characters, lower-case them, and compare against the reverse. $O(n)$ time, but $O(n)$ space because reversing builds a new string.”

3. **Offer the better one, unprompted.** *"I can do it in $O(1)$ space with two indices walking inwards from each end."*
4. **Describe the three loops before writing them.** *"The outer loop compares and steps both inwards. Inside it, two small loops move each index past characters that don't count."*
5. **Name the two details that break it.** *"Those inner loops have to be `while`, not `if`, because punctuation comes in runs. And each one has to test `left < right` in its own condition, or a string of pure punctuation walks straight off the end."*
6. **Say why `<` and not `≤`.** *"The outer loop stops when they meet, because on an odd-length string the middle character has nothing to be compared with."*
7. **Give the cost with the argument.** *" $O(n)$ time — the nested loops don't make it quadratic, because each index only ever moves in one direction, so between them they travel at most n positions. $O(1)$ space, just two integers."*
8. **Name your test cases.** *"I'd test the empty string, a single space, a string of only punctuation, and `0P` — that last one catches using `isalpha` instead of `isalnum`."*

The follow-ups

"Can you do it without extra space?" That is the two-pointer version, and it is the reason the pattern is worth learning. One index at each end, walking inwards, comparing as they go, with small inner loops skipping characters that do not count. Two integers of state regardless of input size. The reversing version is the same $O(n)$ time but allocates two full copies of the string, which for a very large input is the difference between constant memory and doubling your footprint. The one thing to be careful about is that the inner skip loops must repeat their own bound check, because a string containing no alphanumeric characters at all would otherwise run an index off the end — `".,"` is the input that proves it.

"Now allow one deletion." Walk inwards as before. At the first mismatch, the character causing it must be either the one on the left or the one on the right — everything outside those two positions has already matched pairwise, so deleting anywhere else leaves the mismatch exactly where it was. So I check whether the remainder is a palindrome with the left character skipped, or with the right one skipped, and return true if either holds. That branch fires at most once, so the whole thing is still $O(n)$ time and $O(1)$ space. The version to avoid is deciding greedily which one to delete based on one character of look-ahead — that is wrong on inputs where the losing branch survives longer than the winning one, and you have to actually try both.

"Find the longest palindromic substring." Every palindrome has a centre, so I try every centre and expand outwards while the characters match, keeping the longest. The detail that matters is that there are $2n - 1$ centres, not n : each character is a centre for odd-length palindromes, and each gap between two characters is a centre for even-length

ones. Missing the even centres means never finding `"bb"` in `"cbbd"`. That gives $O(n^2)$ time and $O(1)$ space, which is what is normally expected. Dynamic programming gives the same $O(n^2)$ time but $O(n^2)$ space, so it is strictly worse here. There is an $O(n)$ algorithm — Manacher's — and I would name it and say I would look it up rather than reconstruct it under time pressure, because getting it subtly wrong is worse than not offering it.

“Is this integer a palindrome, without converting it to a string?” Negative numbers are never palindromes, because of the minus sign, and any number ending in zero is not unless it is zero itself. Otherwise I reverse half the number arithmetically: repeatedly take the last digit with `% 10`, build it onto a reversed value with `reversed = reversed * 10 + digit`, and drop it from the original with `// 10`, stopping when the original is no longer greater than the reversed half. For an odd number of digits the middle digit ends up alone in the reversed half, so I compare `original == reversed` or `original == reversed // 10`. Reversing only half avoids overflow in languages with fixed-width integers, which is the actual reason the problem is posed that way — Python has arbitrary precision so it would not matter here, and saying that shows you know why the constraint exists.

A model answer

“First, the rules, because they are most of this problem. Which characters count — is it letters and digits only, and does case matter? And what should the empty string return?

...Alphanumeric only, case-insensitive, empty is true. Good.

The simplest correct answer is to filter out everything non-alphanumeric, lower-case it, and compare the result against its reverse. That is two lines and $O(n)$ time — but $O(n)$ extra space, because reversing a string in Python builds a whole new one.

I can do it in constant space with two indices. One starts at the left end, one at the right, and they walk towards each other comparing as they go.

```
def is_palindrome(s: str) → bool:
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

Three things I want to call out.

The inner loops have to be `while` and not `if`, because unwanted characters come in runs — in `'a,,, ,a'` a single `if` only skips one comma and then compares a comma against a letter.

Each inner loop repeats the `left < right` test in its own condition. Without that, a string like `'.,'` with no alphanumeric characters at all runs the left index straight past the right one and off the end of the string, and you get an `IndexError`. That is the one input that finds this bug.

And the outer loop is `left < right`, not `≤`, because on an odd-length string the middle character has nothing to compare against — comparing it with itself is always true.

On cost: $O(n)$ time. It is worth being explicit that the nested loops don't make it quadratic — each index only ever moves in one direction, so between them they travel at most n positions in total. And $O(1)$ extra space, which is the whole point of doing it this way.

For tests I'd use the empty string, a single space, a string of pure punctuation, and '0P'. That last one is the interesting one: it is not a palindrome, and a solution using `isalpha` instead of `isalnum` drops the zero and wrongly says it is."

9 Recall card

- **Two indices from the ends, walking inwards.** `while left < right` — the middle character needs no comparison.
- `O(n)` time, `O(1)` space. The nested skip loops stay linear because each index only ever moves one way.
- **Skip loops must be** `while`, **not** `if`, **and must repeat** `left < right` — or `".,"` runs off the end.
- `isalnum`, **not** `isalpha`. `"0P"` is the input that catches it.
- **Finding palindromes:** `2n - 1` centres, odd and even. Expanding overshoots, so return `left + 1, right - 1`.

1 What this is, and why they ask it

Rate limiting is capping how many requests one caller may make in a period of time. Over the cap, you return `429 Too Many Requests` and a `Retry-After` header, and you do it *before* the request reaches anything expensive.

It exists because a shared service has finite capacity and callers do not know about each other. One client with a retry loop and no backoff — the retry storm from [day 018](#) — can consume everything and take the service down for everyone else. Rate limiting is how a system stops one caller's mistake from becoming everybody's outage. It is also how you enforce paid tiers, and how you make credential-stuffing and scraping expensive.

The **API gateway** is where it usually lives: a single component in front of your services that every request passes through, doing the things every request needs — terminating TLS, checking the token, applying the limit, routing, and recording what happened.

Interviewers ask this constantly, and it has a satisfying shape: a small, concrete algorithm to describe (the token bucket), a placement decision to argue (where does the limiter sit?), and a distributed-systems problem hiding inside it (how do ten servers share one counter?). It appears in almost every design round from mid-level upwards, and “design a rate limiter” is a whole interview question at some companies.

2 The story

The building Kamala lives in has eleven flats and one water problem, which is the same water problem every building on that road has.

The corporation supply comes at about five in the morning and it is not much. It fills the sump in the basement, and from there a motor pushes it up to the tanks on the roof — one tank for each flat, five hundred litres each, standing in a row.

It did not always work that way, and the reason it does now is the family in 2B. They had put in a bigger motor, and when the water came they ran it, and by half past six they had filled everything they owned and the flats on the fourth floor had nothing at all. Not less. Nothing. It went on for two years and there were some very bad meetings about it.

What they have now is a timer on each tank. Water goes into Kamala's tank at a steady twenty litres an hour, all day, whether she is using it or not.

She has got used to thinking about it in a particular way. If she has not used much for a day or two, the tank is full, and she can wash everything in the house at once, because five hundred litres are sitting up there waiting. That is the point of having a tank rather than a pipe. But if she does that, the tank is empty, and the only water she has after that is the twenty litres an hour trickling in — so a second big wash that afternoon is not possible however much she wants it.

And there is a limit to saving up. When the tank is full it is full; the extra water coming in just runs out of the overflow pipe on the side and down the wall. She cannot bank three days of water for a wedding. Five hundred is five hundred.

She says the good thing about it is not really the fairness, although that matters. It is that she knows. Before the timers, whether there was water depended on what the people in 2B were doing that morning, and she could not plan anything. Now she can look at the tank and know exactly where she stands.

3 The idea in plain English

Kamala’s tank is a **token bucket**, and it is the algorithm you should describe when asked. Every part of it maps:

THE TANK	THE ALGORITHM
500-litre capacity	burst size — the most you can spend at once
20 litres an hour going in	refill rate — the sustained rate you are allowed
taking water to wash	spending a token ; one request costs one token
the tank being empty	no tokens left → the request is refused, 429
overflow running down the wall	tokens beyond capacity are discarded — you cannot save up forever

So: **each caller has a bucket. Tokens are added at a fixed rate up to a maximum. Each request removes one token. No token, no request.**

The reason this is the standard answer is that it allows a **burst** while capping the **average**. A mobile app that opens and fires eight requests at once is normal and should not be punished; a script sending eight requests a second for an hour is not, and will drain its bucket in the first second and then be held to the refill rate. One mechanism, both behaviours.

The four algorithms, and why token bucket wins

You should be able to name all four and say what is wrong with the first two.

Fixed window. Count requests per calendar minute. Simple, one counter per caller, and it has a real flaw: a caller can send the full limit at 10:00:59 and the full limit again at 10:01:00, which is **twice the limit in one second**. §6 does the arithmetic. Interviewers ask about this boundary problem specifically.

Sliding window log. Store the timestamp of every request; to decide, count the ones inside the last 60 seconds. Exactly correct, and it stores one entry per request — at 1,000 requests a second that is 60,000 timestamps per caller in memory at any moment. Accurate and expensive.

Sliding window counter. Keep the current window's count and the previous window's, and weight the previous one by how far into the current window you are. Approximate, cheap, no boundary spike. This is what Cloudflare uses and it is a good answer.

Token bucket. Two numbers per caller — the token count and the time it was last updated — and you compute the refill lazily when a request arrives, rather than running a timer:

```
elapsed = now - last_refill
tokens = min(capacity, tokens + elapsed * refill_rate)
last_refill = now
if tokens ≥ 1: tokens -= 1; allow
else:         refuse with 429
```

Two numbers per caller, exact, and it handles bursts by design. **This is the one to describe.**

There is a fifth, the **leaky bucket**, which is the token bucket's mirror image: requests go into a queue that drains at a constant rate, and the queue overflowing is the rejection. It smooths output to a perfectly steady rate, which is what you want in front of something fragile downstream, and it removes the ability to burst. Name it as the alternative and say when you would use it.

What you limit on

The **key** is as much of the design as the algorithm. Get it wrong and you either fail to stop the attacker or you block a whole office.

- **By API key or user id** — the right default for an authenticated API. Precise, and it survives the caller changing address.
- **By IP address** — the only option before login. Its weakness is that a whole company or a whole mobile network can sit behind one address, so limiting by IP can block thousands of innocent users at once.

- **By endpoint** — `POST /login` deserves a far tighter limit than `GET /products`. Limits should be per operation, not one global number.
- **By tier** — free 100/hour, paid 10,000/hour. The same mechanism, a different bucket size.

In practice you run several at once: a generous per-user limit, a tighter per-IP limit for unauthenticated endpoints, and a very tight one on login and password reset.

What you send back

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1767203612
```

`429` from [day 018](#), and `Retry-After` is the important one — without it every rejected client guesses, and they all guess the same interval, and you have built yourself a thundering herd. Sending the three `X-RateLimit-*` headers on **every** response, not only rejections, lets a well-behaved client slow down before it is refused, which is strictly better for both sides.

The API gateway

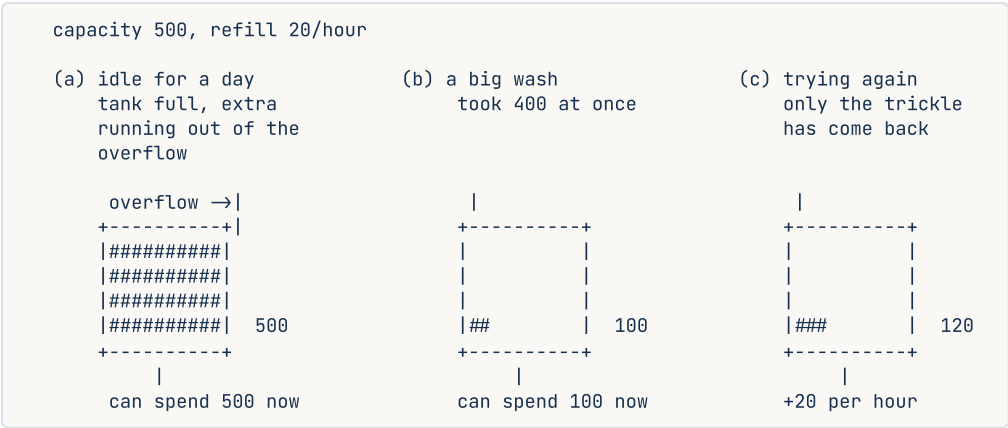
The limiter has to run somewhere every request passes through, and that place is the **API gateway**: one component in front of everything, doing the work common to all requests.

- Terminate TLS.
- Authenticate — verify the token, reject with `401` before anything downstream is touched.
- Rate limit — reject with `429`, cheaply.
- Route to the right service.
- Translate protocols — REST at the edge, gRPC inside, which is [day 022](#)'s boundary.
- Record metrics and logs in one place.

The value is that these are written once rather than in every service, and — for rate limiting specifically — that **the rejection happens before the expensive part**. A `429` from the gateway costs one Redis lookup. The same request reaching a service costs a database query, and it was going to be refused anyway.

4 The picture

The token bucket, in three moments:



What to notice: the overflow pipe in (a). You cannot bank more than the capacity, which is what stops a caller saving a month of quota and spending it in one second. And between (b) and (c) nothing happened except time passing — the refill is computed from the clock, not from a background job.

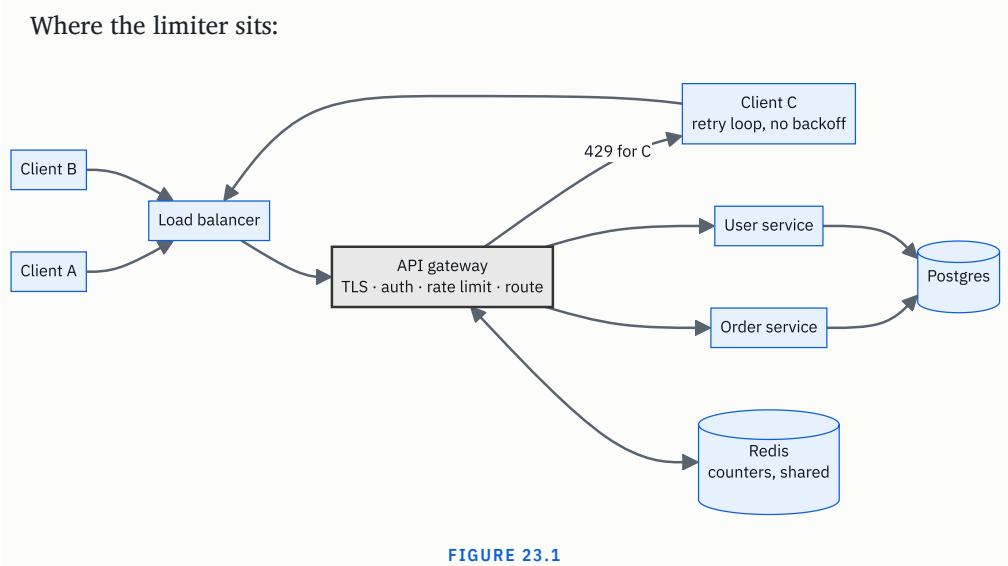


FIGURE 23.1

What to notice: the arrow that turns round at the gateway. Client C's flood never reaches a service, never touches the database, and costs one Redis round trip to refuse. If the limiter lived inside each service instead, every one of those requests would have consumed a connection and a query before being rejected — and the counters would be per-service rather than shared, so ten replicas would each allow the full limit.

5 How it actually works

Where the state lives

The counters have to be shared, because with ten gateway instances and a per-instance counter, a caller limited to 100 a minute can do 1,000. So the state goes in **Redis**: fast, shared, and it has atomic operations and expiry built in.

The naive fixed-window implementation is two commands:

```
INCR    rate:user42:1767203600    → returns the new count
EXPIRE  rate:user42:1767203600 60 → so old windows clean themselves up
```

The key includes the window's start time, so a new window is a new key and old ones disappear on their own.

The problem is atomicity. Two requests arriving simultaneously can both read a count of 99, both decide they are allowed, and both proceed — the same race as the idempotency key claim on [day 018](#). `INCR` is itself atomic, which is why the fixed-window version uses it, but a token bucket needs *read, compute refill, compare, write* as one indivisible step.

The standard answer: **a Lua script**, which Redis executes atomically:

```
-- KEYS[1] = bucket key, ARGV = now, rate, capacity, requested
local bucket = redis.call("HMGET", KEYS[1], "tokens", "ts")
local tokens = tonumber(bucket[1]) or tonumber(ARGV[3])
local ts     = tonumber(bucket[2]) or tonumber(ARGV[1])
local elapsed = math.max(0, tonumber(ARGV[1]) - ts)
tokens = math.min(tonumber(ARGV[3]), tokens + elapsed * tonumber(ARGV[2]))
local allowed = tokens ≥ tonumber(ARGV[4])
if allowed then tokens = tokens - tonumber(ARGV[4]) end
redis.call("HMSET", KEYS[1], "tokens", tokens, "ts", ARGV[1])
redis.call("EXPIRE", KEYS[1], 3600)
return { allowed and 1 or 0, tokens }
```

You would not write that on a whiteboard. **You would say “the check-and-decrement has to be atomic, so I’d do it in a Redis Lua script or with a `MULTI` / `EXEC` transaction”** — and that one sentence is what the question is testing.

Local counters plus periodic sync

At very high volume, a Redis round trip per request is itself a cost — half a millisecond and a shared dependency on the hot path. The usual optimisation: each gateway keeps a **local** bucket holding its share of the limit, and syncs with Redis every second or so.

The trade is accuracy. With 10 nodes and a limit of 100/minute, each gets 10, and a caller whose traffic happens to land unevenly gets refused early on one node while another node has spare. It is approximate, and for rate limiting approximate is usually fine — you are protecting capacity, not counting money. Say which you would pick and why: **exact via Redis for low and medium volume, local-with-sync when the Redis hop itself becomes the bottleneck.**

What happens when Redis is down

A real decision, and interviewers press on it.

- **Fail open** — allow everything. The service stays up for legitimate users and is unprotected for as long as the outage lasts. Right for ordinary product endpoints.
- **Fail closed** — refuse everything. Safe and it turns a Redis outage into a total outage. Right only where the limit is a security control.

The usual answer is **fail open, with a local fallback limit**: if Redis is unreachable, fall back to a conservative per-node in-memory limit. You lose global accuracy and keep a ceiling.

The gateway's other jobs

Rate limiting is one of six or seven things a gateway does, and being able to list them is worth marks:

TLS termination · authentication and token verification · rate limiting and quotas · routing and load balancing · request and response transformation (REST outside, gRPC inside) · caching · centralised logging, metrics and tracing · circuit breaking.

Real products: Kong and Envoy are the common open-source choices; Envoy is also the data plane inside Istio and Linkerd. AWS API Gateway, Google Apigee and Azure API Management are the hosted ones. nginx does a good deal of it with `limit_req`, which implements a leaky bucket. Cloudflare applies limits at the edge before traffic reaches your network at all, which is the only place that helps against a genuine flood.

Real published limits worth quoting: GitHub allows 5,000 requests an hour on an authenticated token and 60 unauthenticated, and its GraphQL API limits by computed query cost rather than request count, for the reasons in [day 021](#). Twitter's API famously used 15-minute windows. Stripe limits to roughly 100 requests a second in live mode.

The limits of rate limiting

Worth saying, because it shows perspective: rate limiting protects you from **many requests from one caller**. It does nothing about **one request from many callers**, which is a distributed denial-of-service attack and has to be handled upstream of you — at the

CDN or by a scrubbing provider — because by the time the traffic reaches your gateway it has already consumed your bandwidth.

6 The numbers

The fixed-window boundary problem

Limit: **100 requests per minute**, fixed calendar windows.

```
10:00:59 → 100 requests (all inside the 10:00 window)
10:01:00 → 100 requests (a fresh window, counter reset)
-----
200 requests in a 1-second span, against a limit of 100 per minute
```

Twice the limit, delivered in one second. That is the whole argument for sliding windows or a token bucket, and quoting the two timestamps is far more convincing than saying “fixed windows have a boundary issue”.

Memory per algorithm

10 million users, of whom 1 million are active in any given minute.

fixed window	: 1 counter + expiry	≈ 50 bytes	→ 1M × 50 B = 50 MB
token bucket	: tokens + timestamp	≈ 60 bytes	→ 1M × 60 B = 60 MB
sliding window log	: one timestamp per request		
	at 10 req/min each	≈ 10 × 16 B = 160 B	→ 1M × 160 B = 160 MB
	at 1,000 req/min	≈ 16 KB	→ 1M × 16 KB = 16 GB

The log is fine for modest rates and falls apart for high ones. **The token bucket is two numbers per caller regardless of traffic**, which is the practical reason it wins.

Redis load

100,000 requests a second, one Redis operation each:

```
100,000 ops/s - one Redis node handles well over 100,000 ops/s, so this fits, with little headroom
latency added : 0.5 ms on every request, including the ones that are allowed
```

Half a millisecond on every request is the price of exact global limiting. With local counters synced each second:

```
Redis ops/s = 10 gateway nodes ÷ 1 s = 10 ops/s
latency added ≈ 0 (an in-memory check)
accuracy      ≈ ±10% at the boundaries
```

Four orders of magnitude less Redis traffic, in exchange for approximate limits. **That trade is the answer to “how would you scale this?”**

What the limit is worth

A service sized for 10,000 requests a second. One client with a broken retry loop:

```
unlimited : one client can send 50,000 req/s and consume 5x your capacity
limited   : that client is capped at, say, 100 req/s – 1% of capacity
```

And the cost of refusing:

```
429 at the gateway : 1 Redis lookup           ≈ 0.5 ms, no database, no service
request served      : auth + routing + query + serialise ≈ 50 ms, 1 database connection
```

A hundred times cheaper to refuse than to serve — which is why the limiter belongs at the front and not inside each service.

Choosing the burst size

A mobile app opens 8 requests when a screen loads, and a user might open 4 screens a minute:

```
sustained needed : 8 × 4 = 32 requests/minute → refill ≈ 0.53 tokens/second
burst needed      : 8 in one instant           → capacity ≥ 8, so pick 20 for headroom
```

So: capacity 20, refill 0.5 per second. That allows the natural burst and caps the average at 30 a minute. **Deriving the two numbers from actual client behaviour is the answer to “what limit would you set?”** — much stronger than picking a round number.

7 The trade-offs

Where the limiter runs

At the gateway — one place, shared state, requests refused before they cost anything. And the gateway is now on every request path, so it must be replicated and it is a component that can fail.

In each service — no extra hop and no shared component, but the logic is duplicated, the counters are per-service so a caller gets N times the limit across N services, and the request has already travelled through your system before being refused.

At the CDN or edge — the only placement that helps against a real flood, because it rejects traffic before it reaches your network and consumes your bandwidth. It cannot see per-user state as richly, so it is coarse.

Most real systems use all three: coarse at the edge, precise at the gateway, and a last-resort local cap inside critical services.

The key you limit on

Per-user is precise and needs authentication, so it cannot protect the login endpoint — which is exactly where you need protection most. Per-IP works for anonymous traffic and punishes shared addresses: one office, one university or one mobile carrier's NAT can be thousands of people behind a single address, and a strict per-IP limit blocks all of them. The usual compromise is a generous per-IP limit combined with a tight per-account limit on login attempts, so an attacker cannot spray one password across many accounts from one address, and cannot brute-force one account from many addresses either.

Exact or approximate

Exact means shared state and a network hop on every request. Approximate means local counters and some slack at the boundaries. **For protecting capacity, approximate is fine.** For enforcing a paid quota that a customer is billed against, it is not — there you want exact, and you want it recorded durably rather than in a cache.

Fail open or fail closed

Covered above, and the point is to have decided. Fail open for product endpoints, fail closed where the limit is genuinely a security control, and a local fallback ceiling in both cases.

Is a gateway worth it at all?

For three services and one team, a gateway is a component to run, deploy and debug, and putting the limit in a shared library may be simpler. For thirty services and six teams, writing auth and rate limiting thirty times is worse in every way. **The gateway earns its place when the number of services times the number of cross-cutting concerns gets large,** and saying that rather than assuming a gateway is a sign of judgement.

The sentence that separates candidates

I would not put the rate limiter only inside the services. By the time the request gets there it has already cost me a connection, a token verification and often a query, and it was going to be refused anyway — refusing at the gateway is about a hundred times cheaper. And per-service counters mean a caller gets the full limit against each service independently, which is not a limit at all. I would keep a conservative local cap inside critical services as a backstop, but the real limit belongs at the front where the state is shared.

8 In the interview

How it gets asked

- “How would you stop one client from hammering your API?” — the direct version.
- “Design a rate limiter.” — a whole question at some companies, where they want the algorithm, the storage, the distributed problem and the response format.
- “What algorithm would you use, and what’s wrong with a fixed window?” — the specific one, fishing for the boundary spike.
- “You have ten servers. How do they share the counter?” — the distributed half, which is the interesting part.
- “What’s an API gateway for?” — the placement question, usually mid-design.

What to say out loud, in the first ninety seconds

1. **Say what you are protecting against.** “Rate limiting protects finite capacity from a caller who doesn’t know about the other callers — usually a retry loop with no backoff rather than an attacker.”
2. **Name the algorithm and describe it concretely.** “I’d use a token bucket. Each caller has a bucket with a capacity and a refill rate. Tokens are added at a steady rate up to the capacity, each request takes one, and no token means a 429.”
3. **Say why that one.** “It allows a burst while capping the average, which matches how clients actually behave — an app opening a screen fires eight requests at once, and that’s fine, but eight a second for an hour isn’t.”
4. **Name the alternative and its flaw, unprompted.** “A fixed window is simpler, but it lets a caller send the full limit at 10:00:59 and the full limit again at 10:01:00 — twice the limit in one second.”
5. **Say where it runs.** “At the API gateway, not inside each service. Refusing there costs one Redis lookup instead of a connection and a query, and per-service counters would give a caller the full limit against every service independently.”
6. **Say where the state lives, and the hard part.** “Counters in Redis so all gateway instances share them. The check-and-decrement has to be atomic — a Lua script or a transaction — otherwise two simultaneous requests both read the same count and both get through.”
7. **Say what you return.** “429 with `Retry-After`, plus rate-limit headers on every response so a well-behaved client can slow down before it’s refused rather than after.”
8. **Name the key.** “Per API key for authenticated traffic, per IP before login, and a much tighter limit specifically on login and password reset.”

The follow-ups

“What’s wrong with a fixed window?” The boundary. With a limit of 100 a minute and calendar windows, a caller can send 100 requests at 10:00:59 and another 100 at 10:01:00 when the counter resets — 200 requests in about a second against a limit of 100 a minute. So the effective peak is twice the intended limit, and it can be delivered in the worst possible instant. The fixes are a sliding window log, which stores a timestamp per request and is exact but expensive at high rates, a sliding window counter that weights the previous window’s count by how far into the current one you are, which is cheap and approximate, or a token bucket, which has no windows at all and computes the refill from elapsed time. I would take the token bucket: two numbers per caller regardless of traffic volume, and bursts come out naturally rather than being an accident of the boundary.

“You have ten gateway servers. How do they share the counter?” They cannot each keep their own, or a caller limited to 100 a minute gets 1,000. So the state goes in a shared store — Redis — keyed by caller. The subtlety is atomicity: the token bucket needs read, compute the refill, compare, and write as one indivisible operation, and if two requests interleave they both see the same token count and both proceed. So the whole thing goes in a Redis Lua script, which Redis executes atomically, or in a `MULTI / EXEC` transaction. That adds about half a millisecond to every request and a shared dependency. At very high volume I would switch to local buckets holding each node’s share of the limit, synced with Redis every second — four orders of magnitude less Redis traffic in exchange for roughly ten per cent slack at the boundaries, which for protecting capacity is an easy trade. And I would decide explicitly what happens when Redis is down: fail open with a conservative local ceiling, because a limiter outage should not become a service outage.

“What limit would you actually set?” I would derive it from client behaviour rather than pick a round number. If a mobile screen fires eight requests when it opens and a user opens about four screens a minute, the sustained need is around 32 a minute and the burst need is 8 at once — so a capacity of 20 and a refill of half a token a second, which allows the natural burst and caps the average around 30. Then different limits per endpoint, because a login attempt and a product listing are not comparable: I might allow thousands an hour on reads and five a minute on login. And different limits per tier, which is the same mechanism with a bigger bucket. I would also ship the limit in headers on every response from day one, so clients can see where they stand, and I would watch how often the limit is actually hit before tightening it — a limit that fires constantly for legitimate users is a bug, not a defence.

“Does this protect you from a DDoS?” No, and it is worth being clear about why. Rate limiting stops **many requests from one caller**. A distributed denial of service is **one request from each of a hundred thousand callers**, so every individual bucket

looks perfectly innocent. Worse, by the time the traffic reaches my gateway it has already consumed my bandwidth and my TLS handshake capacity, so refusing it there costs me real resources. That defence has to live upstream — at a CDN or scrubbing provider like Cloudflare or AWS Shield — where traffic can be dropped before it reaches my network. What rate limiting does protect me from, which is far more common, is one badly written client with a retry loop, one scraper, and credential stuffing against a login endpoint.

A model answer

“First, what I’m actually protecting against. In practice it is almost never an attacker — it’s one client with a retry loop and no backoff, or a scraper, and the effect is that one caller consumes capacity everyone else needed.

The algorithm I’d use is a token bucket. Every caller has a bucket with two properties: a capacity, and a refill rate. Tokens are added at the refill rate up to the capacity, each request consumes one, and if there are none the request gets a 429. The refill is computed lazily from elapsed time when a request arrives, so there’s no background job — the state is just the token count and the timestamp it was last updated.

The reason I’d choose it over the alternatives is that it separates burst from average. A mobile app firing eight requests when a screen opens is normal behaviour and shouldn’t be punished; eight a second sustained for an hour isn’t. The capacity handles the first, the refill rate caps the second, and it’s one mechanism.

The alternative people reach for is a fixed window — count requests per calendar minute — and it has a specific flaw worth naming. With a limit of 100 a minute, a caller sends 100 at 10:00:59 and another 100 at 10:01:00 when the counter resets. That’s 200 requests in one second against a limit of 100 a minute. A sliding window log fixes it exactly but stores a timestamp per request, which at a thousand requests a minute per caller is 16 KB each; the token bucket is two numbers per caller however much traffic there is.

On placement: this runs at the API gateway, not inside each service. Two reasons. Refusing at the gateway costs one Redis lookup, about half a millisecond, whereas letting it reach a service costs a connection and a query — roughly a hundred times more, for a request that was going to be refused anyway. And per-service counters aren’t a limit at all, because a caller would get the full allowance against each service independently.

The state goes in Redis so all gateway instances share it. The part that needs care is atomicity: read the tokens, compute the refill, compare, decrement and write has to be one indivisible operation, or two simultaneous requests both read the same count and both get through. I’d do that in a Redis Lua script. If the Redis hop itself became the bottleneck, I’d move to local buckets holding each node’s share, synced every second — approximate to within about ten per cent, and four orders of magnitude less Redis traffic.

For the response: 429, with `Retry-After` so clients don't all guess the same interval and come back together, and `X-RateLimit-Limit`, `-Remaining` and `-Reset` on every response, not just the rejections, so a well-behaved client can slow down before it hits the wall.

On the key: per API key for authenticated traffic, per IP for anonymous — accepting that an office behind one NAT address is many people, so that limit has to be generous — and a much tighter, per-account limit on login and password reset specifically.

And I'd say what this doesn't do: it protects against many requests from one caller, not one request from a hundred thousand callers. A real DDoS has to be handled at the CDN, before the traffic reaches my network at all."

9 Recall card

- **Token bucket:** capacity = burst, refill rate = sustained rate, one token per request, no token → 429. Two numbers per caller.
- **Fixed windows leak at the boundary** — full limit at 10:00:59 plus full limit at 10:01:00 is twice the limit in a second.
- **It runs at the gateway**, with counters in **Redis**, and the check-and-decrement must be **atomic** (Lua script).
- **Return** 429 + `Retry-After`, and send `X-RateLimit-*` on every response so clients slow down before they are refused.
- **Key by API key, by IP before login, and per endpoint.** It does not stop a DDoS — that belongs at the edge.

DSA topic: Palindromes and the two-ends habit **System design topic:** Rate limiting and API gateways

Code these, in this order

Four problems built on two indices walking towards each other. The algorithm is never the hard part; the input rules and the boundaries are.

Before each one, ask:

1. Which characters count, and does case matter?
2. Is the loop `left < right` or `left ≤ right`, and why?
3. What are the three inputs that break skip loops? (Empty, one space, pure punctuation.)
4. Am I allowed extra space, or is `O(1)` the point of the question?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Valid Palindrome	LeetCode 125 (Easy)	The messy input rules, and whether you can do it in <code>O(1)</code> space rather than reversing. <code>"0P"</code> is the trap input.
2	Valid Palindrome II	LeetCode 680 (Easy)	That only two deletions are possible at the first mismatch, and that you must try both rather than guess.
3	Longest Palindromic Substring	LeetCode 5 (Medium)	<code>2n - 1</code> centres, not <code>n</code> . Forgetting the even-length centres is the standard bug, and <code>"cbbd"</code> finds it.
4	Palindrome Linked List	LeetCode 234 (Easy)	The same idea where you cannot index backwards. Reverse the second half in place, compare, restore. Two pointers again, in a shape you will meet properly on day 082 .

On problem 1, write both versions

Write `cleaned = cleaned[::-1]` first. Say its cost out loud: `O(n)` time, `O(n)` space. Then write the two-pointer version and say why it is better, in one sentence.

Then run **both** on this list and check they agree:

```
["A man, a plan, a canal: Panama", "race a car", "", " ", ".", "0P", "aba", "abba", "ab", "a"]
```

Expected: `[True, False, True, True, True, False, True, True, False, True]`

If your two-pointer version crashes on `".,"`, you have found the missing `left < right` guard. Read the traceback before you fix it.

On problem 1, break it deliberately

Run each of these and say what is wrong:

```
# A
cleaned = "".join(ch.lower() for ch in s if ch.isalpha())
```

```
# B
while left < right:
    if not s[left].isalnum():
        left += 1
    if not s[right].isalnum():
        right -= 1
    ...
```

```
# C
while left < right:
    while not s[left].isalnum():
        left += 1
    ...
```

A fails on `"0P"`. B fails on `"a,,,a"`. C raises on `".,"`. Reproduce all three and name the rule each one broke.

On problem 2, prove the two-branch claim

Before coding, answer this out loud: at the first mismatch between `s[left]` and `s[right]`, why must the character to delete be one of exactly those two? Why can it not be something in the middle?

The answer is one sentence about what has already been checked. If you cannot produce it, the code will feel like a guess.

Then write the greedy version — the one that peeks at `s[left+1]` and commits — and find an input where it is wrong. Try `"eccccbebaeeabebcccees"`.

On problem 3, count the centres

Take `"abcd"` and list every centre you would try. There should be seven: four characters and three gaps. Then take `"cbdd"` and confirm your solution returns `"bb"` and not `"c"`.

Then answer:

1. Why does `expand` return `left + 1, right - 1` rather than `left, right`?

2. What is the cost, counted out loud from the two loops?
3. At what input size does $O(n^2)$ stop being acceptable?
4. What is the $O(n)$ algorithm called, and what would you say about it in an interview?

Question 4 has a specific right answer, and it is not “I’d implement it”.

The boundary drill

Answer without running anything:

1. In `while left < right`, what happens on a string of length 1? Length 0?
2. Why is `<=` not wrong, merely wasteful?
3. In the skip loops, why is the `left < right` test needed **inside each one** and not only in the outer loop?
4. `is_palindrome("")` — what should it return, and what does your code return?
5. Trace `is_palindrome(" ")` step by step. Which loop runs, and how many times?

The cost-argument drill

There are two nested `while` loops inside a `while` loop, and the answer is still $O(n)$. Say the argument out loud, in one sentence, without using the word “amortised”.

Then do the same for these:

1. The two-pointer palindrome check.
2. `valid_palindrome` with one deletion. (Why does the branch not make it $O(n^2)$?)
3. `longest_palindrome` by expanding around centres. (Why is this one $O(n^2)$?)

The rate-limiter drill

Answer each in one or two sentences, out loud:

1. Describe the token bucket in three sentences, naming capacity and refill rate.
2. What is the fixed-window boundary problem? Give the two timestamps.
3. Why does the token bucket use two numbers per caller while a sliding window log uses one entry per request?
4. Where does the limiter run, and what are the two reasons for that placement?
5. Ten gateway servers share one limit. Where does the state live, and what must be atomic?
6. Redis is down. Do you allow or refuse? Defend it.
7. What do you return, and which header stops you creating a thundering herd?
8. What do you key on before the user has logged in, and what is wrong with that key?
9. Does any of this protect you from a DDoS?

Number 5 is the one interviewers push on. Have the word *atomic* ready.

The design drill

Set limits for these five endpoints and justify each in one sentence:

ENDPOINT	YOUR LIMIT	WHY
GET /products		
POST /login		
POST /password-reset		
POST /orders		
GET /search?q=		

Then derive one of them properly: if a screen fires 8 requests when it opens and a user opens 4 screens a minute, what capacity and refill rate would you choose, and why is that better than picking a round number?

The arithmetic drill

From memory, in under two minutes:

- Limit 100/minute, fixed windows — the worst-case burst, and when it happens.
- 1 million active callers — memory for a token bucket, and for a sliding window log at 1,000 requests a minute each.
- 100,000 requests a second with one Redis operation each — Redis load and latency added.
- The same with local counters synced every second across 10 nodes — Redis load, and the accuracy cost.
- Cost of a 429 at the gateway versus serving the request.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Is this string a palindrome, ignoring punctuation and case?* Pin the rules first. Give the one-liner and its space cost, then offer two pointers unprompted. Name the three loops, the while-not-if point, and the inner bound check. Finish with the $O(n)$ -not- $O(n^2)$ argument and your four test inputs.

2. *How would you stop one client from hammering your API?* Token bucket, described concretely. Then the fixed-window flaw with the two timestamps. Then placement and why. Then the shared-state and atomicity point. Then the response and its headers. Ninety seconds.
 3. *Can this string become a palindrome by deleting at most one character?* The point is the argument, not the code: at the first mismatch, everything outside has already matched, so the deletion must be at one of those two positions — and you must try both rather than guess. Then say why it is still $O(n)$.
-

Before you move on

- [] I reach for two indices from the ends without thinking about it.
- [] I write `while left < right` and can say why `<=` is unnecessary.
- [] My skip loops are `while`, not `if`, and each repeats the bound check.
- [] I test with `""`, `" "`, `".,"` and `"0P"` every time.
- [] I can argue $O(n)$ for nested loops by counting how far the indices travel.
- [] I remember there are $2n - 1$ centres, not n .
- [] I can describe the token bucket in three sentences with both of its numbers.
- [] I can state the fixed-window boundary problem with real timestamps.
- [] I say “the check-and-decrement must be atomic” without being prompted.
- [] I can redraw the token-bucket and gateway diagrams from memory, in whatever tool I like.

Substrings versus subsequences: the distinction they test

DATA STRUCTURES AND ALGORITHMS

Substrings versus subsequences: the distinction they test

You never confuse the two again, and you know how many of each a string has.

SYSTEM DESIGN

API revision and interview questions

You can design and defend an API for an unseen feature in fifteen minutes.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *How many substrings does a string of length n have? How many subsequences?*
- *Design the API for a food delivery app's order flow.*

Substrings versus subsequences: the distinction they test

1 What this is, and why they ask it

A **substring** is a run of characters that are **next to each other** in the original string. A **subsequence** is characters taken in order but **allowed to skip**. From "abcde", "bcd" is a substring and also a subsequence; "ace" is a subsequence and **not** a substring, because the characters are not adjacent.

One word — *contiguous* — is the entire difference, and it decides everything downstream. It changes how many there are: a string of length n has $n(n+1)/2$ non-empty substrings and 2^n subsequences. It changes which technique solves the problem: substrings are sliding windows and two pointers, subsequences are dynamic programming or a greedy walk. And it changes what is even possible, because you can enumerate every substring of a 1,000-character string and you cannot enumerate every subsequence of a 100-character one.

Interviewers test this deliberately and often silently. They will say “*longest substring without repeating characters*” and “*longest common subsequence*” in the same interview, and the words are the only signal you get about which of two completely different techniques to reach for. Misreading it costs you the whole question, and it happens constantly — which is exactly why this gets its own day rather than a footnote.

2 The story

The market street behind the bus stand in Madurai is one straight lane with twelve shops down the left-hand side, and Selvi and her mother go there most Saturday mornings.

The street only runs one way for them. They come in at the temple end and they walk down to the junction, and they do not turn round — partly because it is crowded and partly because Selvi’s mother has a knee that does not like it. So whatever they do that morning, they do in the order the shops happen to stand in.

Some mornings they only need one thing, and they walk in at shop four, buy it, and cut out at the lane by shop six. They have covered shops four, five and six. Not four and six — they physically pass five, whether they want to or not, because it is between the other two. Any walk they do is a **stretch** of the street with nothing missing out of the middle of it.

But the shops they actually go into are a different matter. Last Saturday they went into the vegetable shop, then the one that sells buttons and ribbon, and then the sweet shop right at the end, and they walked past everything else without stopping. Vegetables, buttons, sweets — in that order, because that is the order the shops stand in, but with plenty left out.

Selvi’s daughter, who is fifteen and doing something with numbers at school, asked her two questions on the way home and would not let it go.

First: how many different walks could you do? Selvi worked it out on her fingers. You pick where you go in and you pick where you come out, and the second one has to be at or after the first. Seventy- eight, they decided, for twelve shops.

Then: how many different sets of shops could you go into? That one is different, because for each of the twelve shops the only question is in or not in, separately from all the others. Her daughter did it on her phone. Four thousand and ninety-six.

Selvi said that could not possibly be right, and her daughter showed her the working, and it was.

3 The idea in plain English

Selvi’s walk is a **substring**. The shops she went into are a **subsequence**. Both keep the order of the street, and only one of them is allowed to leave gaps.

The definitions, on one example

Take "abcde" .

	SUBSTRING	SUBSEQUENCE
must be adjacent?	yes	no
order preserved?	yes	yes
"bcd"	✓	✓
"ace"	✗ — b and d are skipped	✓
"eca"	✗	✗ — wrong order
" "	✓ (usually excluded)	✓

Every substring is a subsequence. The reverse is not true. That one sentence, said out loud, settles the definition for good.

There is a third word you will meet: a **subarray** is the same thing as a substring, for arrays. Same rule, different container. And a **subset** would allow reordering, which neither of these does — if a problem says “subset” about a string, ask what they mean,

because they probably mean subsequence.

How many substrings

A substring is fixed by two choices: where it starts and where it ends. Pick a start `i` from 0 to `n-1`, and an end at or after it.

- Starting at 0: `n` choices of end.
- Starting at 1: `n-1` choices.
- ... down to starting at `n-1`: 1 choice.

$$n + (n-1) + (n-2) + \dots + 1 = n(n+1) / 2$$

That is Selvi picking where to go in and where to come out. For `n = 3` it is `3 × 4 / 2 = 6`, and here they are for `"abc"`: `a`, `ab`, `abc`, `b`, `bc`, `c`. For `n = 12` it is `78`.

The count is **quadratic**, and that is a friendly number. A 1,000-character string has about 500,000 substrings — you can list them all if you have to.

How many subsequences

A subsequence is fixed by a completely different kind of choice: for **each character independently**, in or out.

$$2 \times 2 \times 2 \times \dots \times 2 \quad (n \text{ times}) = 2^n$$

Including the empty one. For `"abc"` that is 8: `"`, `a`, `b`, `c`, `ab`, `ac`, `bc`, `abc`. For `n = 12` it is 4,096. **The count is exponential**, and that is a hostile number: a 100-character string has about 10^{30} subsequences, which is more than the number of atoms you could count in a lifetime.

This is why the two words lead to different techniques. Enumerating substrings is sometimes reasonable. Enumerating subsequences essentially never is, so any subsequence problem has to be solved without listing them — which is what dynamic programming is for, and why the DP phase from [day 143](#) is full of subsequence problems.

Put the two side by side and the gap is the whole lesson:

N	SUBSTRINGS $N(N+1)/2$	SUBSEQUENCES 2^n
3	6	8
5	15	32
10	55	1,024
20	210	1,048,576
50	1,275	$\sim 10^{15}$
100	5,050	$\sim 10^{30}$

How to tell which one you are being asked about

The word itself, if it is there. When it is not, these are the tells:

THE PROBLEM SAYS	IT MEANS	REACH FOR
“substring”, “subarray”, “contiguous”, “window”	substring	sliding window, two pointers
“subsequence”, “in order”, “delete some characters”	subsequence	DP, or a greedy two-index walk
“longest ... without repeating characters”	substring	sliding window (day 032)
“longest common ...”	almost always subsequence	2-D DP
“is A a subsequence of B”	subsequence	greedy, one pass
“count subarrays summing to k”	substring	prefix sums (day 038)

If you genuinely cannot tell, **ask**. “Does that need to be contiguous?” takes three seconds and decides which of two unrelated solutions you are about to write.

The two techniques, in one line each

Substrings: a sliding window. Because substrings are contiguous, you can hold one with two indices and move them. Extend the right edge to grow, advance the left edge to shrink, and never look at the same character more than a constant number of times. That gives $O(n)$ for problems whose brute force is $O(n^2)$ or worse. It is the whole of days [031](#) to [035](#).

Subsequences: a greedy walk or a table. Is A a subsequence of B? is a single greedy pass with an index into each — take the next character of A whenever B offers it. *Longest common subsequence* needs a 2-D table, because at each pair of positions you either match and move both, or skip one of them, and you cannot tell which is better without looking ahead. That is $O(m \times n)$ and it is the canonical DP problem.

4 The picture

"abc", both lists in full:

SUBSTRINGS – pick a start, pick an end
 $n(n+1)/2 = 6$

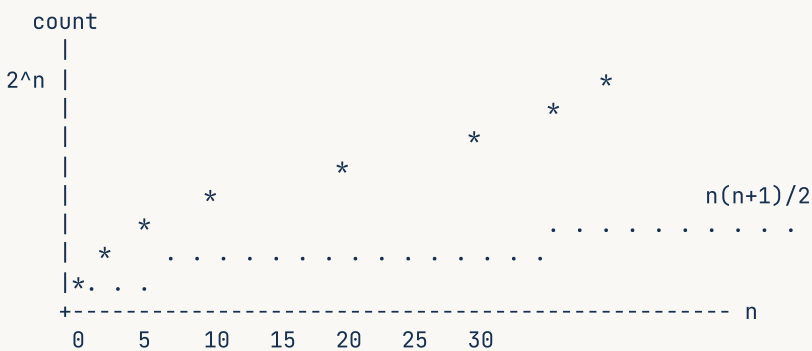
a . . → "a"
a b . → "ab"
a b c → "abc"
. b . → "b"
. b c → "bc"
. . c → "c"

SUBSEQUENCES – each character: in or out
 $2^n = 8$

. . . → ""
a . . → "a"
. b . → "b"
. . c → "c"
a b . → "ab"
a . c → "ac" ← NOT a substring
. b c → "bc"
a b c → "abc"

What to notice: the only extra one on the right, apart from the empty string, is "ac" — and it is the one with a hole in the middle. That hole is the entire distinction, and at $n = 3$ it costs you two extra items. At $n = 20$ it costs you a million.

The two growth curves, from day 004:



at $n=20$: substrings 210

subsequences 1,048,576

What to notice: the dotted line is flat enough to enumerate. The starred one is not, at any size worth calling a string. That is why “enumerate them all” is a plan for one and never for the other.

A sliding window over substrings, which is what contiguity buys you:

```

"a b c a b c b b"
  ^       ^
  L       R       window "abc", all distinct, length 3

"a b c a b c b b"
  ^       ^
  L       R       'a' repeats → move L past the old 'a'

"a b c a b c b b"
  ^       ^
  L       R       window "bca", still length 3

```

What to notice: neither index ever moves backwards, so each character is visited at most twice across the whole run. That is what makes it $O(n)$ — and it only works because a substring is contiguous, so a window can represent it. There is no equivalent trick for subsequences.

5 The code, built step by step

Listing every substring

```

for i in range(len(s)):
    for j in range(i + 1, len(s) + 1):
        print(s[i:j])

```

Two loops: `i` picks the start, `j` picks one past the end. `j` starts at `i + 1` so the empty string is excluded, and goes to `len(s) + 1` so the last character is included — that upper bound is the off-by-one people get wrong.

Count the iterations and you get $n(n+1)/2$, which is the formula from §3, derived rather than remembered.

Listing every subsequence

```

out = []
for ch in s:
    out += [prev + ch for prev in out]

```

Beautifully small, and worth understanding because it is the 2^n . Start with one thing, the empty subsequence. For each character, every subsequence you already have gives rise to two: itself, and itself plus this character. So the list doubles on every character.

```

start      ['']
after 'a'  ['', 'a']
after 'b'  ['', 'a', 'b', 'ab']
after 'c'  ['', 'a', 'b', 'c', 'ab', 'ac', 'bc', 'abc']

```

Watch it double: 1, 2, 4, 8. **Never run this on a long string.** At `n = 30` it needs a billion strings, and at `n = 40` your machine dies. It is here to make the number real, not to be used.

Is A a subsequence of B?

The greedy walk, LeetCode 392, and it is much simpler than people expect:

```

def is_subsequence(a: str, b: str) → bool:
    i = 0
    for ch in b:
        if i < len(a) and a[i] == ch:
            i += 1
    return i == len(a)

```

Walk through B once. Keep an index into A. Whenever B offers the character A is waiting for, take it and move on. A is a subsequence of B exactly when you got all the way through A.

Why is greedy correct here? Because taking the *earliest* possible match is never worse. If some solution matches `a[i]` at a later position in B, replacing it with the earlier one leaves at least as much of B available for the rest of A. There is nothing to be gained by waiting, so there is no choice to agonise over — which is exactly why this is $O(n)$ while longest-common-subsequence is not.

That argument is the interesting part of the question, and saying it is worth more than the code.

Longest substring without repeating characters

LeetCode 3, and it is a substring problem, so: sliding window.

```

last: dict[str, int] = {}
start = 0
best = 0

```

`last[ch]` records the most recent position of each character; `start` is the left edge of the window.

```

for i, ch in enumerate(s):
    if ch in last and last[ch] ≥ start:
        start = last[ch] + 1
    last[ch] = i
    best = max(best, i - start + 1)

```

If this character was seen before, and that sighting is inside the current window, move the left edge just past it. Then record where we saw it, and update the best length.

`last[ch] ≥ start` is the whole problem. Without it, a character seen long ago — before the window began — drags `start` backwards, and the window grows instead of shrinking. The input that proves it is `"dvdvf"`: the answer is 3 (`"vdf"`), and a version without the guard returns 2. Test with `"abba"` too, which fails the same way.

Longest common subsequence

LeetCode 1143, and the shape of every subsequence DP problem:

```

dp = [[0] * (n + 1) for _ in range(m + 1)]
for i in range(1, m + 1):
    for j in range(1, n + 1):
        if a[i - 1] == b[j - 1]:
            dp[i][j] = dp[i - 1][j - 1] + 1
        else:
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
return dp[m][n]

```

`dp[i][j]` is the answer for the first `i` characters of `a` and the first `j` of `b`. If the current characters match, they can both be used, so it is one more than the answer without either. If they do not, you must drop one of them, and you take whichever gives more.

Notice why greedy fails here and worked above. In `is_subsequence` there was only one string to match, so taking the earliest match was always safe. Here there are two, and skipping a character in `a` might unlock a longer match later in `b` — you cannot know without trying, so you try both and keep the better. That is the difference between an $O(n)$ problem and an $O(m \times n)$ one, and it is a good thing to be able to explain.

`[[0] * (n + 1) for _ in range(m + 1)]`, never `[[0] * (n+1)] * (m+1)` — the aliasing trap from [day 016](#), and it bites in every DP problem you will ever write.

The complete solutions

```
def all_substrings(s: str) → list[str]:
    """Every non-empty substring. There are  $n(n+1)/2$  of them."""
    out: list[str] = []
    for i in range(len(s)):
        for j in range(i + 1, len(s) + 1): # j is one PAST the end
            out.append(s[i:j])
    return out

def all_subsequences(s: str) → list[str]:
    """Every subsequence, including the empty one. There are  $2^n$  of them.

    Demonstration only – never run this on a long string.
    """
    out = [""]
    for ch in s:
        out += [prev + ch for prev in out] # the list doubles per character
    return out

def is_subsequence(a: str, b: str) → bool:
    """LeetCode 392. Greedy: take the earliest match, which is never worse."""
    i = 0
    for ch in b:
        if i < len(a) and a[i] == ch:
            i += 1
    return i == len(a)

def length_of_longest_substring(s: str) → int:
    """LeetCode 3. A SUBSTRING problem, so: sliding window.  $O(n)$ ."""
    last: dict[str, int] = {}
    start = 0
    best = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] ≥ start: # ≥ start: only if it is INSIDE the window
            start = last[ch] + 1
        last[ch] = i
        best = max(best, i - start + 1)
    return best

def longest_common_subsequence(a: str, b: str) → int:
    """LeetCode 1143. A SUBSEQUENCE problem, so: a table.  $O(m*n)$ ."""
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)] # never  $[[0]*(n+1)] * (m+1)$ 
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]

if __name__ == "__main__":
    print(all_substrings("abc"))
    # ['a', 'ab', 'abc', 'b', 'bc', 'c'] 6 = 3*4/2
    print(sorted(all_subsequences("abc"), key=lambda x: (len(x), x)))
    # ['', 'a', 'b', 'c', 'ab', 'ac', 'bc', 'abc'] 8 = 2**3

    for n in (1, 2, 3, 5, 10, 20):
        print(f"n={n}>3} substrings {n*(n+1)//2:>5} subsequences {2**n:>10}")

    print([is_subsequence(x, y) for x, y in
           (("abc", "ahbgdc"), ("axc", "ahbgdc"), ("", "abc"), ("abc", ""), ("abc", "abc"))])
    # [True, False, True, False, True]
```

```

print([length_of_longest_substring(x) for x in
      ("abcabcbb", "bbbbbb", "pwwkew", "", "au", "dvdf")])
# [3, 1, 3, 0, 2, 3]

print(longest_common_subsequence("abcde", "ace")) # 3
print(longest_common_subsequence("abc", "def"))   # 0

```

6 What it costs

Listing them

Substrings. The outer loop runs n times; for start i the inner loop runs $n - i$ times. Adding those up gives $n(n+1)/2$ iterations — but each iteration also **builds** a slice, and a slice of length k costs $O(k)$ to copy, from [day 019](#). The total length of all substrings is about $n^3/6$, so listing them all is $O(n^3)$ time and $O(n^3)$ space.

That surprises people. *Counting* them is $O(n^2)$ — or $O(1)$ with the formula — but *materialising* them is cubic. At $n = 1,000$ that is around 170 million characters, which is a few hundred megabytes. **If a problem asks you to examine every substring, work with indices, not copies.**

Subsequences. 2^n of them, so listing them is $O(2^n)$ at best. At $n = 30$ that is a billion strings; at $n = 40$, a trillion. There is no machine and no patience that makes this work.

The techniques

PROBLEM	KIND	APPROACH	TIME	SPACE
longest substring without repeats	substring	sliding window	$O(n)$	$O(k)$, alphabet size
all substrings, examined	substring	two loops over indices	$O(n^2)$	$O(1)$
is A a subsequence of B	sub-sequence	greedy, one pass	$O(n)$	$O(1)$
longest common subsequence	sub-sequence	2-D table	$O(m \cdot n)$	$O(m \cdot n)$, or $O(n)$ rolled
longest palindromic sub-sequence	sub-sequence	2-D table	$O(n^2)$	$O(n^2)$

The pattern to notice: substring problems are usually $O(n)$ once you spot the window, because contiguity lets two indices represent the whole thing. Subsequence problems are usually $O(n^2)$ or $O(m \cdot n)$, because there is no window that can represent a set with holes in it, so you pay for a table.

Because a substring must be contiguous, a mismatch ends the run and the count resets to zero — and the answer is the maximum cell anywhere in the table, not the bottom-right corner. **Read the word. If it is not there, ask.**

The near-miss: the sliding-window guard

```
def length_of_longest_substring(s):
    last = {}
    start = 0
    best = 0
    for i, ch in enumerate(s):
        if ch in last:                # missing: and last[ch] ≥ start
            start = last[ch] + 1
            last[ch] = i
        best = max(best, i - start + 1)
    return best

print(length_of_longest_substring("dvdf"))
```

2

The answer is 3 — "vdf". Here is what went wrong. At the second `d`, position 2, the code sets `start = 1`, correctly. Then at `f`, position 3, nothing repeats and all is well. But run it on "abba": at the second `a`, position 3, `last["a"]` is 0 — a sighting from **before** the window started — and `start` is dragged backwards from 2 to 1, so the window now contains a repeated `b`.

start must never move backwards. Either guard with `last[ch] ≥ start`, or write `start = max(start, last[ch] + 1)`. Both are correct; the second says the intent more plainly.

The near-miss: the substring loop bound

```
for i in range(len(s)):
    for j in range(i + 1, len(s)):    # should be len(s) + 1
        out.append(s[i:j])
```

Every substring ending at the last character is missing, including the whole string itself. On "abc" you get `['a', 'ab', 'b']` — three instead of six. `s[i:j]` excludes position `j`, so `j` must be allowed to reach `len(s)`.

The near-miss: `is_subsequence` without the bounds check

```
def is_subsequence(a, b):
    i = 0
    for ch in b:
        if a[i] == ch:                # no i < len(a) guard
            i += 1
    return i == len(a)

print(is_subsequence("ab", "abc"))
```

```
Traceback (most recent call last):
...
  if a[i] == ch:
      ~^^^
IndexError: string index out of range
```

Once all of `a` has been matched, `i` equals `len(a)` and the next character of `b` reads off the end. The guard has to come first: `if i < len(a) and a[i] == ch`. Note that Python's `and` short-circuits, so the second half is never evaluated when the first is false — that ordering is doing real work.

The contract corner: empty strings

Both of these are true, and both catch people:

- `is_subsequence("", "abc")` is **True** — the empty string is a subsequence of everything.
- `is_subsequence("abc", "")` is **False**.

The first is the one that feels wrong and is right. It also falls out of the greedy code for free, which is a small sign the code is correct.

The vocabulary trap: “subarray”

For arrays, **subarray** means contiguous — it is the same idea as substring. **Subsequence** means the same as it does for strings. But people say “subarray” loosely in conversation, so when an interviewer says it, confirming “*contiguous, yes?*” costs three seconds and can save you fifteen minutes of the wrong solution.

8 In the interview

How it gets asked

- “How many substrings does a string of length n have? How many subsequences?” — the direct version, usually as a warm-up. $n(n+1)/2$ and 2^n , derived rather than recited.

- “Longest substring without repeating characters.” — LeetCode 3. Substring, so window.
- “Is A a subsequence of B?” — LeetCode 392. Subsequence, so a greedy pass.
- “Longest common subsequence.” — LeetCode 1143. Table.
- “Count the substrings that ...” — where the answer is often to count as you go rather than to enumerate.

What to say out loud, in the first ninety seconds

1. **Say the distinction before anything else.** “Substring means contiguous; subsequence keeps the order but may skip. Every substring is a subsequence, not the other way round.”
2. **If the word is missing, ask.** “Does that have to be contiguous?” Three seconds, and it decides which technique you are about to spend twenty minutes on.
3. **Derive the counts, do not recite them.** “A substring is fixed by a start and an end, so it’s $n + (n-1) + \dots + 1 = n(n+1)/2$. A subsequence is an independent in-or-out choice per character, so 2^n .”
4. **Say what the counts imply.** “So enumerating substrings is sometimes reasonable — half a million for a thousand characters — and enumerating subsequences never is. That’s why one gets a window and the other gets a table.”
5. **Name the technique that follows.** “Because a substring is contiguous, two indices can represent it, so a sliding window gives $O(n)$. A subsequence has holes, so nothing can represent it in two indices, which is why it costs a DP table.”
6. **Then solve the actual question**, having already told them you know which one it is.

The follow-ups

“Why is `is_subsequence` greedy but longest-common-subsequence needs DP?” Because of how many choices there are. In `is_subsequence` I am matching one fixed string into another, and taking the earliest possible match is never worse — if some solution matches this character later in B, swapping it for the earlier occurrence leaves at least as much of B for everything that follows, so there is no decision to regret and one greedy pass suffices. In longest common subsequence both strings are being chosen from, so when the current characters differ I have a genuine choice: skip a character of A, or skip one of B. Skipping in A might unlock a longer match later in B, and I cannot tell without exploring, so I evaluate both and keep the better. That is exactly the situation dynamic programming exists for, and it takes the cost from $O(n)$ to $O(m \cdot n)$.

“How would you count substrings with some property, without enumerating them?” Enumerating is $O(n^2)$ at best and $O(n^3)$ if you materialise them, so for anything large I count as I go. The general move is: for each right endpoint, count how many left endpoints make a valid substring, and add that to a running total. With a sliding window, if the window from `left` to `right` is valid then every window starting at or after

`left` and ending at `right` is also valid — so this endpoint contributes `right - left + 1` to the answer in `O(1)`, and the whole thing is `O(n)`. That trick — counting `right - left + 1` per position instead of enumerating — turns a large family of “count the substrays where...” problems from quadratic into linear, and it is the heart of [day 034](#).

“What’s the difference between longest common substring and longest common subsequence?” One line of the recurrence, and it follows directly from contiguity. Both build a table where `dp[i][j]` is the answer for the first `i` characters of one string and the first `j` of the other, and both do `dp[i-1][j-1] + 1` when the characters match. The difference is the mismatch case. For a subsequence I carry forward the best I had — `max(dp[i-1][j], dp[i][j-1])` — because skipping a character is allowed and does not destroy what I have built. For a substring a mismatch **breaks the run**, so `dp[i][j] = 0` and I start again. And the answer is in a different place: for the subsequence it is the bottom-right cell, because it is about the whole of both strings; for the substring it is the maximum cell anywhere in the table, because the best run may have ended in the middle.

“A string of a million characters — how do you handle the substring problems then?” `O(n2)` is a trillion operations, so anything that touches every substring is out, and I need a technique that is linear or close to it. For most of these problems the sliding window already is linear, so it holds up: longest substring without repeats on a million characters is a million dictionary operations, which is well under a second. What I would watch is memory rather than time — specifically, never slicing. `s[i:j]` copies, so building substrings inside a loop turns a linear algorithm into a quadratic one silently. I would work with the two indices and only materialise the single answer at the end.

A model answer

“The distinction is one word: contiguous. A substring is a run of characters that are next to each other in the original. A subsequence keeps their order but is allowed to skip. So in `abcde`, `bcd` is both, and `ace` is a subsequence but not a substring, because `b` and `d` are missing from the middle. Every substring is a subsequence; the reverse is not true.

For the counts: a substring is completely determined by where it starts and where it ends, and the end has to be at or after the start. So it's n choices of end for a start at position 0, $n-1$ for position 1, and so on — which sums to $n(n+1)/2$.

A subsequence is a different kind of choice: for each character, independently, it's in or out. That is 2 multiplied by itself n times, so 2^n , including the empty subsequence.

The reason I'd bother deriving them rather than just stating them is that the *shape* of those two numbers is what decides the technique. At $n = 20$ it's 210 against a million. Substrings are quadratic, so enumerating them is sometimes a legitimate plan — half a million for a thousand-character string. Subsequences are exponential, so enumerating them is never a plan at any useful size.

That falls straight out of contiguity. Because a substring is a contiguous run, two indices can represent it completely, which is why substring problems collapse into sliding windows and come out at $O(n)$ — longest substring without repeating characters is one pass with a dictionary of last positions. A subsequence has holes in it, so no pair of indices can describe it, which is why subsequence problems need a table over pairs of positions and land at $O(m \cdot n)$ — longest common subsequence is the standard example.

The one place this genuinely costs people marks is that the two words look alike in a problem statement. Longest common substring and longest common subsequence differ by a single line in the recurrence: on a mismatch, the subsequence version carries forward the best it has, and the substring version resets to zero because the run is broken. So the first thing I do with any of these is check which word was used, and if it isn't there, ask whether it has to be contiguous.”

9 Recall card

- **Substring = contiguous. Subsequence = order kept, gaps allowed.** Every substring is a subsequence.

- **Counts:** $n(n+1)/2$ substrings, 2^n subsequences. At $n = 20$, 210 against a million.
- **Contiguous** → sliding window, $O(n)$. Gaps allowed → DP table, $O(m \cdot n)$.
- **If the word is missing**, ask “does it have to be contiguous?” — it decides the whole solution.
- **Substring vs subsequence DP differ in the mismatch line:** reset to 0, or carry the max forward.

1 What this is, and why they ask it

Days 15 to 23 built the API phase: what an API is, REST and its constraints, designing endpoints, status codes and idempotency, authentication and authorisation, sessions and tokens, GraphQL, gRPC, and rate limiting. Today converts that from nine documents you have read into one performance you can give.

This is not a summary. It is a **drill**, for the same reason [day 018](#) was a drill: the gap between knowing something and producing it out loud, in order, under mild pressure, is much wider than anyone expects, and interviews only ever test the second thing.

API design questions appear in three places. As a ten-minute warm-up before a larger system design round — *“before we start, design me the endpoints for X”*. As a component of every full design, since the moment you draw two boxes you have to say what one asks the other for. And as the whole question in back-end interviews at product companies, where “design the API for a food delivery order flow” is a complete forty-five minutes.

The thing being measured is **judgement with nothing to lean on**. Nobody expects you to recall RFC numbers. They expect you to name the resources before writing paths, paginate without being told, know that an empty list is not a [404](#), and mention idempotency before they ask about the payment.

2 The story

Farhat has been teaching driving for eleven years at a school off the Bypass road, and she says the same thing to every student on the first day, and almost none of them believe her.

The bit that stops people passing is not the driving. It is that they only ever practise one route.

She had a student last year, a young man who was genuinely good — smooth with the clutch, careful, patient at junctions. He had done nineteen lessons and he knew the route around the test centre the way you know the way to your own kitchen. He knew which junction had the bad camber and where the school bus parks at eleven o'clock. On the test they took him a different way, out past the new buildings where he had never been, and he came apart in about four minutes. Not because it was harder. Because everything he had was tied to that one road.

So the last four lessons with Farhat are different. She does not choose the route. She has a list on her phone with about thirty places on it, and when the student gets in she reads one out — the hospital, the wholesale market, the temple with the awkward left turn — and that is where they are going, and she does not say anything else for the rest of the hour.

Students hate it. They say it is unfair because they have not practised that one.

That is the point, she tells them. On the day, the man beside you will pick, and you will have not practised it. What you are practising now is not a road. It is the thing you do when you are handed a road you do not know: check the mirrors, work out which lane, ask if you are not sure, and go slowly enough to think.

Her pass rate is very good, and she says it has nothing to do with the driving.

3 The idea in plain English

Farhat’s thirty places are today’s exercise. You are not revising nine documents. You are practising **the thing you do when handed a feature you have not seen** — which is a fixed procedure, and it is the same procedure every time.

The seven-step procedure

Whatever the feature is, this is the order. It never changes, and knowing it means you are never stuck at the start wondering what to say first.

STEP	ROUGHLY	WHAT YOU DO
1. Scope	1 min	Ask three or four questions. Who calls this? What is in and out? Any scale figure?
2. Nouns	2 min	List the resources. Out loud, before any path.
3. Paths and methods	4 min	Collection and item for each noun. Nest for list and create only.
4. One response body	2 min	Sketch the JSON for the main resource. This is where the real questions live.
5. Errors	2 min	The five or six failure cases, each with a status code.
6. Cross-cutting	2 min	Auth, pagination, idempotency, rate limits — in that order.
7. Trade-offs	2 min	One thing you would do differently at ten times the scale.

Fifteen minutes. Steps 4, 5 and 6 are what separate candidates, and they are exactly the steps people run out of time for — which is why steps 1 to 3 must be fast.

The nine things you must produce without being asked

An interviewer is ticking these off silently. Say each one before they have to prompt you:

1. **Resources are nouns, plural, consistent.** `/orders`, never `/getOrder`.
2. **Nest for list and create; top-level for read, update and delete.** `POST /restaurants/9/reviews` but `PATCH /reviews/41`.
3. **Filters, sorting and paging go in the query string.** `?status=delivered&limit=20`.
4. **Every collection is paginated from day one** — default limit, hard server cap, cursor not offset.
5. **An empty collection is `200` with `[]`, not `404`.** `404` is the parent missing.
6. **`POST` is not idempotent**, so anything that creates or moves money takes an idempotency key.
7. `401` is “I don’t know you”; `403` is “I know you and no”. The check is server-side, in the query where possible.
8. **Versioning is in the path**, `/v1/`, and you can name the header alternative.
9. **Rate limits exist**, keyed per user and per endpoint, returning `429` with `Retry-After`.

If you produce nine of nine in fifteen minutes, you have given a strong answer whatever the feature was.

The nine days, in one line each

DAY	THE ONE SENTENCE THAT SURVIVES
015 API	A published promise: the operations, how to ask, what comes back. The caller never gets the machinery.
016 REST	Six constraints; uniform interface and statelessness are the two that matter; nobody implements HATEOAS.
017 Endpoints	Nouns first. Nest for list and create. Paginate everything. Empty is not missing.
018 Status & idempotency	Three outcomes, not two. A timeout tells you nothing. Client-generated key, claimed atomically.
019 Authn/authz	Who are you, versus what may you do. Once, versus every request. Argon2id at ~250 ms.
020 Sessions/JWT/OAuth	The whole trade is revocation. Short tokens plus revocable refresh is a session store again.
021 GraphQL	Fixes over- and under-fetching; costs you HTTP caching, and needs DataLoader and depth limits.
022 gRPC	Schema, persistent HTTP/2, binary. Inside the system; REST at the edge.
023 Rate limiting	Token bucket: capacity is burst, refill is average. At the gateway, in Redis, atomically.

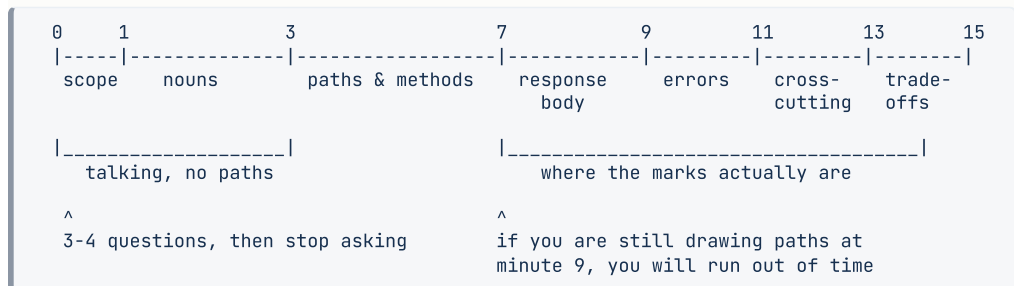
The choosing questions

Half of what gets asked in this phase is *which one*, and each has a condition rather than a preference:

- **REST or GraphQL?** GraphQL when clients are logged-in apps whose responses were never cacheable anyway and several front-end teams need different shapes. REST when it is public, read-heavy and cacheable.
- **REST or gRPC?** gRPC inside, REST at the edge. The boundary is whether the caller is a stranger.
- **Session or JWT?** Session for a first-party web app — instant revocation, 400 MB for ten million users. JWT when many services must verify without a central lookup.
- **PUT or PATCH?** `PUT` replaces the whole resource; `PATCH` changes part. Most resources want `PATCH`.
- **POST /x/action or PATCH /x?** State change → `PATCH`. Genuine operation with its own permissions and audit trail → action sub-resource. Keep the number of those small.
- **Offset or cursor pagination?** Cursor, for anything where items arrive while a user scrolls, and for anything deep.

4 The picture

The fifteen minutes, as a shape:



What to notice: paths take four minutes and everything after them takes six. Candidates spend twelve minutes perfecting paths and never reach idempotency or pagination, which are the parts being scored.

The whole phase, as one system:

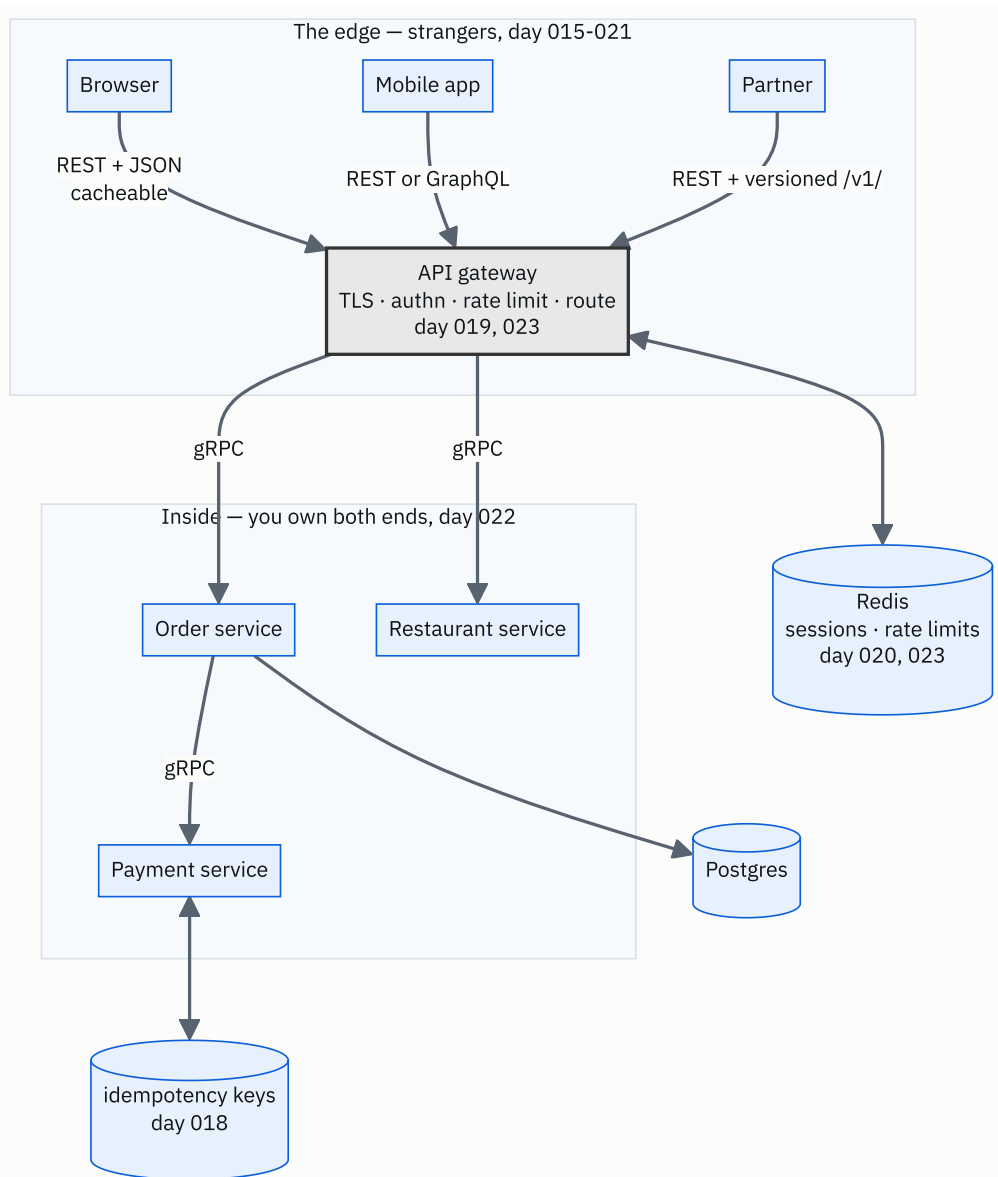


FIGURE 24.1

What to notice: every day of the phase has a place on this one diagram. If you can draw this from memory and say which day each label came from, you have revised the phase.

5 How it actually works

Two worked designs. Read them as transcripts, not as reference material — the value is in the order things are said.

Worked design one: a food delivery order flow

Step 1 — scope, four questions.

“Before I start: is this the customer-facing API, the restaurant’s, or the delivery rider’s — or all three? Does an order need to be modifiable after it is placed? Is payment inside this flow or a separate service? And roughly what scale — orders per day?”

Say: customer-facing, no modification after placement, payment is a separate service called synchronously, one million orders a day.

Step 2 — nouns, out loud.

“Restaurant, menu item, cart, order, payment, delivery. The cart is interesting — I’ll come back to whether it needs to be server-side at all.”

Step 3 — paths.

GET	/restaurants?lat=&lng=&radius=&cuisine=&limit=20&after=	200	
GET	/restaurants/{id}	200	
GET	/restaurants/{id}/menu-items?limit=50&after=	200	
GET	/carts/me	200	
PUT	/carts/me/items/{menu_item_id} {"quantity": 2}	200	idempotent
DELETE	/carts/me/items/{menu_item_id}	204	
POST	/orders	Idempotency-Key: <uuid>	201 + Location
GET	/orders/{id}	200	
GET	/orders?status=active&limit=20&after=	200	
POST	/orders/{id}/cancel	200	409
GET	/orders/{id}/delivery	200	

Three decisions to say out loud while writing them:

- **PUT on a cart item, not POST**. `PUT /carts/me/items/{id}` with a quantity means *the quantity of this item is now 2*. Tapping “add” twice on a flaky connection leaves quantity 2, not two separate lines. `POST /carts/me/items` would create duplicates.
- **/carts/me rather than /carts/{id}**. The caller only ever has one cart and it is theirs; putting an id in the path invites the insecure-direct-object-reference bug from [day 019](#).

- `cancel` is an action sub-resource. It has its own permission rules and a time limit, and burying it in a `PATCH` would hide a significant state change inside a generic update. This is the one exception, and I would keep it the only one.

Step 4 — one response body.

```
{
  "id": "ord_8f2a",
  "status": "preparing",
  "placed_at": "2026-08-27T13:40:11Z",
  "restaurant": { "id": "res_19", "name": "Anand Bhavan", "image_url": "https:// ..." },
  "items": [
    { "menu_item_id": "mi_77", "name": "Masala dosa", "quantity": 2, "unit_price": 90 }
  ],
  "totals": { "items": 180, "delivery": 30, "taxes": 11, "grand_total": 221 },
  "delivery": { "eta_minutes": 28, "rider": null },
  "cancellable_until": "2026-08-27T13:42:11Z"
}
```

Four decisions visible here: the restaurant is embedded as a summary so the client does not make a second call; totals are broken down because a single number generates support tickets; `rider` is `null` until assigned rather than absent, so clients need not test for the key; and `cancellable_until` is computed server-side so every client agrees about the deadline.

Step 5 — errors.

CASE	CODE
restaurant does not exist	404
restaurant exists but is closed	409
item no longer available	409 with the offending item id in the body
cart empty	422
not logged in	401
cancelling somebody else's order	403
cancelling after the window	409
too many orders in a minute	429 + <code>Retry-After</code>

Step 6 — cross-cutting, and this is where the marks are.

“POST /orders takes an idempotency key generated by the client when the checkout screen opens — not when the button is pressed — so a double tap and a network retry both carry the same key. The server claims it atomically, and a retry gets the stored response rather than a second order. Every collection is cursor-paginated with a default of 20 and a hard cap. Auth is a session or token at the gateway; ownership is checked in the query, not after it. Rate limits are per user and per endpoint — generous on restaurant browsing, tight on order creation.”

Step 7 — one trade-off.

“The server-side cart is the thing I would question. It costs a write on every tap and it only exists so the cart survives a device change. If that is not a product requirement, I would keep the cart on the client and send it whole at checkout, which removes an entire resource. At ten times the scale I would also make order placement asynchronous — 202 Accepted with a status endpoint — because holding a connection open through a synchronous payment call is what falls over first.”

Worked design two: a notifications feature, compressed

Same procedure, three minutes:

```
GET    /notifications?unread=true&limit=20&after=      200
PATCH /notifications/{id} {"read": true}          200
POST   /notifications/read-all                       200   the one action endpoint
DELETE /notifications/{id}                           204
GET     /notification-preferences/me                  200
PATCH /notification-preferences/me                  200
```

Say out loud: unread is a **filter**, not a path. **PATCH** for one, because setting `read: true` twice is idempotent. `read-all` is an action because it is not a state change on any single resource. Zero notifications is `200` with `[]`.

6 The numbers

The arithmetic you should be able to produce without notes. These are the ones that come up.

Traffic from users

```
10,000,000 daily users × 8 requests each = 80,000,000 requests/day
80,000,000 ÷ 86,400                      = 926 requests/second average
peak ≈ 3-4x average                     = 3,700 requests/second
```

What caching buys

```
80% of traffic cacheable, 90% CDN hit rate:
  absorbed by the CDN = 3,700 × 0.8 × 0.9 = 2,664 req/s
  reaching your origin = 3,700 - 2,664 = 1,036 req/s
```

Roughly 3.5x fewer servers, and cached reads come back in ~20 ms instead of ~150.
This is the number that decides REST versus GraphQL for a public API.

Read-to-write ratio

```
300,000 writes/day ≈ 3.5 writes/second
80,000,000 reads/day ≈ 926 reads/second
ratio ≈ 265 : 1
```

A ratio like that says: cache hard, denormalise counts, stop worrying about write throughput. **One ratio drives more decisions than any other number.**

Storage

```
1,000,000 orders/day × 2 KB = 2 GB/day
2 GB × 365 = 730 GB/year
with indexes, roughly double ≈ 1.5 TB/year
```

Which fits on one machine for years — so the answer to “would you shard?” is no, with arithmetic behind it.

Login cost

```
bcrypt cost 12 ≈ 250 ms → 4 logins/second/core
500,000 logins/day, 25% in the peak hour = 125,000 ÷ 3,600 ≈ 35/second
35 ÷ 4 ≈ 9 cores on hashing alone
```

Rate limiting

```
fixed window, 100/min : 100 at 10:00:59 + 100 at 10:01:00 = 200 in one second
token bucket          : 2 numbers per caller – 1M active callers ≈ 60 MB in Redis
429 at the gateway    : ~0.5 ms vs serving the request ~50 ms → 100x cheaper to refuse
```

Pagination

```
50,000 comments × 250 bytes = 12.5 MB unpaginated ≈ 50 seconds on a 2 Mbps connection
20 × 250 bytes              = 5 KB paginated
offset at page 100,000      : scans 5,000,000 rows ≈ 2,000 ms
cursor at any page          : index seek ≈ 1 ms
```

Payload comparison

```
REST      : 11 KB per screen, ~2 KB actually rendered
GraphQL   : ~2 KB
gRPC+HTTP2: 28 bytes vs 258 for a small message – about 9x
```

7 The trade-offs

The five arguments this phase turns on. Each has a condition, not a winner.

Nest or flat? Nest when the parent type is stable and the child is meaningless without it. Go flat when the feature is heading for several parent types, since `/photos/9/comments` and `/videos/4/comments` will otherwise multiply.

Embed or reference? Embed a small summary — id, name, image — to save a round trip per rendered item. Reference when the data changes often or matters legally, because embedded copies go stale in every cache.

Session or token? Revocation against a lookup. Sessions for first-party web; tokens where the verifier cannot reach a central store.

REST, GraphQL or gRPC? Cacheability, client diversity, and who owns the caller. Public and read-heavy: REST. Many logged-in clients wanting different shapes: GraphQL. Between your own services at volume: gRPC.

Exact or approximate? Rate limits can be approximate — you are protecting capacity. Billed quotas and payments cannot.

The sentence that separates candidates

I would not add a component this design does not need. At a million orders a day the writes are about twelve a second and the storage is a terabyte and a half a year — one Postgres instance handles that without noticing, so I would not shard, and I would not put a queue in front of order creation until synchronous payment latency actually became the problem. What I *would* add on day one is pagination, idempotency on order creation, and rate limits, because retrofitting those into a live API is far harder than adding a database later.

8 In the interview

How it gets asked

- “*Design the API for a food delivery app’s order flow.*” — the full version, fifteen to forty-five minutes.
- “*Design the endpoints for X.*” — the warm-up, where X is comments, notifications, a URL shortener, a parking lot, a chat.
- “*Here’s an endpoint. What would you change?*” — the critique version.
- “*Walk me through what happens when a user taps ‘Pay’.*” — the same knowledge, told as a story, and the one where idempotency has to appear unprompted.

What to say out loud, in the first ninety seconds

1. **Ask three or four scoping questions, then stop.** Who calls it, what is in scope, what happens after creation, roughly what scale. More than four and you are stalling.
2. **List the nouns out loud, before any path.** “*Restaurant, menu item, cart, order, payment, delivery.*”
3. **State your nesting rule as a rule,** not as a series of decisions.
4. **Say “and every collection is paginated” before anyone asks.**
5. **Say “order creation takes an idempotency key” before anyone asks about payments.**
6. **Then start writing paths,** and narrate the two or three that involved a real decision — `PUT` on a cart item, `/carts/me` rather than an id, `cancel` as the single action endpoint.
7. **Leave six minutes** for the response body, the errors and the cross-cutting concerns. Those are the marks.

The follow-ups

“**Walk me through what happens when the user taps Pay.**” The client already holds an idempotency key generated when the checkout screen opened, not when the button was pressed — so a double tap and a network retry carry the same key. It sends `POST /orders` with that key in a header. The gateway terminates TLS, verifies the session token, checks the rate limit for this user on this endpoint, and routes on. The order service claims the idempotency key atomically; if it already exists and is complete it returns the stored response and does nothing else, and if it exists and is in flight it returns `409` so the client waits rather than racing. Otherwise it validates the cart, checks the restaurant is open and the items are available — `409` with the offending item if not — writes the order in a pending state, and calls the payment service synchronously with its own

idempotency key. On success it returns `201` with a `Location` header. If the payment call times out I do not blindly retry; I query the payment service for the status of that key, because a timeout means it may well have succeeded and only the response was lost.

“Someone changes the id in `GET /orders/1055` and sees another user’s order. What went wrong?” That is an insecure direct object reference: the endpoint authenticated the caller but never authorised the object. It verified *who* you are and not *whether this order is yours*. The fix is to put the ownership condition into the query itself — `SELECT ... WHERE id = $1 AND user_id = $2` — so a wrong id simply returns nothing and there is no separate check for anyone to forget. Fetching by id and then checking in code works but depends on every developer remembering every time. I would return `404` rather than `403` here, because a `403` confirms that order 1055 exists. And I would use non-sequential ids like UUIDs, which does not fix the flaw but stops an attacker walking through every order by counting.

“How would you version this API?” Version in the path — `/v1/orders` — because it is unambiguous, trivial to route, and visible in every log and every support ticket. The header alternative, like GitHub’s `Accept: application/vnd.github.v3+json`, is cleaner in theory and much harder to test by hand. Whichever I choose, the more important discipline is not needing a new version often: adding an optional field to a response is backward compatible and needs no version bump, and so is adding an optional request parameter. A version is for changes that break existing callers — removing a field, changing a type, changing what an operation means. I would also date-pin per customer the way Stripe does if this were a paid API with long-lived integrations, since that gives callers a stable world without freezing my own development.

“The order service is getting 10x the traffic. What breaks first, and what do you change?” The synchronous payment call, almost certainly — it holds a connection open for the length of a third-party round trip, so at ten times the load I run out of connections and threads long before I run out of CPU or storage. The change is to make order placement asynchronous: accept the order, return `202 Accepted` with a status URL, put the payment on a queue, and let the client poll or receive a push. That converts a latency problem into a throughput problem, which is much easier to scale. Second thing to break would be the reads on restaurant listings, and those are highly cacheable and mostly identical between users, so a CDN and a Redis layer take most of it. I would explicitly not shard the database yet — at ten times a million orders a day the storage is still only about 15 TB a year, and one instance with read replicas handles it.

A model answer

Compressed, for the fifteen-minute version.

“Four questions first. Is this the customer app, the restaurant’s, or the rider’s? Can an order be changed after it is placed? Is payment inside this flow or a separate service? And roughly how many orders a day?

...Customer-facing, no changes after placing, payment separate and synchronous, a million a day.

The resources are: restaurant, menu item, cart, order, payment, delivery. I’ll nest for listing and creating, since a menu item has no meaning without its restaurant, and keep everything else top-level because ids are already unique.

```
GET    /restaurants?lat=&lng=&cuisine=&limit=20&after=
GET    /restaurants/{id}/menu-items?limit=50&after=
GET    /carts/me
PUT    /carts/me/items/{menu_item_id} {"quantity": 2}
POST   /orders          Idempotency-Key: <uuid>          201 + Location
GET    /orders/{id}
GET    /orders?status=active&limit=20&after=
POST   /orders/{id}/cancel          200 | 409
```

Three deliberate choices. `PUT` on a cart item rather than `POST`, because `PUT` means ‘the quantity is now 2’ — so a double tap on a flaky connection leaves one line at quantity 2, where `POST` would create duplicates. `/carts/me` rather than a cart id, because the caller only ever has one and putting an id there invites someone to read another user’s cart. And `cancel` is an action sub-resource, because it has its own permission rules and a time window; that’s my one exception to nouns-only and I’d keep it the only one.

Every collection is cursor-paginated with a default limit of 20 and a hard server cap — cursors rather than offsets because new restaurants and orders arrive while a user scrolls, and offsets duplicate rows when that happens.

Order creation takes an idempotency key that the client generates when the check-out screen opens, not when the button is pressed, so both a double tap and a network retry carry the same key. The server claims it atomically and replays the stored response for any repeat. That matters because `POST` is not idempotent and a timeout tells you nothing about whether the order was placed.

Errors: `404` if the restaurant doesn’t exist, `409` if it exists but is closed or an item ran out, `422` for an empty cart, `401` unauthenticated, `403` for cancelling someone else’s order, `409` for cancelling after the window, `429` with `Retry-After` on rate limits. And an empty order list is `200` with an empty array, not a `404`.

One trade-off: the server-side cart costs a write per tap and only buys cross-device persistence. If that isn't a requirement I'd hold the cart on the client and send it whole at checkout. And at ten times this scale I'd make order placement asynchronous — 202 plus a status endpoint — because holding a connection open through a synchronous payment call is what falls over first. What I would not do is shard: a million orders a day is about 1.5 TB a year, which one Postgres instance handles comfortably.”

9 Recall card

- **The procedure: scope → nouns → paths → one response body → errors → cross-cutting → trade-off.** Fifteen minutes; the marks are in the last three.
- **Say without being asked:** pagination on every collection, idempotency key on anything creating or paying, an empty list is 200 with [] .
- **Nest for list and create; top-level for read, update, delete.** Filters go in the query string.
- **Every “which one” question has a condition:** cacheability and who owns the caller decide REST vs GraphQL vs gRPC; revocation decides session vs JWT.
- **Do not add components the arithmetic does not demand.** Do add pagination, idempotency and rate limits on day one.

Practice — Substrings versus subsequences: the distinction they test

DSA topic: Substrings versus subsequences: the distinction they test **System design topic:** API revision and interview questions

Code these, in this order

Four problems chosen so that two are substring problems and two are subsequence problems, and the words in the titles are the only clue. **Before writing a line of any of them, say out loud which kind it is and which technique follows.**

Before each one, ask:

1. Contiguous or not? Which word in the statement told me?
2. How many candidates are there — quadratic or exponential?
3. Sliding window, greedy walk, or a table?
4. What does the empty input return?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Is Subsequence	LeetCode 392 (Easy)	The greedy walk, and whether you can say <i>why</i> taking the earliest match is never worse.
2	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	A window, and the <code>last[ch] ≥ start</code> guard. <code>"dvdvf"</code> and <code>"abba"</code> are the inputs that expose it.
3	Longest Common Subsequence	LeetCode 1143 (Medium)	The 2-D table, and why greedy fails here when it worked in problem 1.
4	Maximum Length of Repeated Subarray	LeetCode 718 (Medium)	The <i>substring</i> version of problem 3. One line of the recurrence differs, and the answer is in a different cell. Do these two back to back.

Do problems 3 and 4 together, deliberately

They look almost identical and they are not. Solve 3, then solve 4, then put the two recurrences side by side and answer:

1. Which line differs, and why does contiguity force that?
2. Where is the answer in each — the bottom-right cell, or the maximum anywhere?
3. If someone handed you the problem statement with the word removed, what would you ask?

This pair is the single most valuable half-hour on this page.

On problem 1, argue before you code

Say out loud why greedy is correct: *if some solution matches this character later in B, replacing it with the earlier occurrence leaves at least as much of B for everything that follows*. Then write it. Then test the two empty cases and predict both:

```
is_subsequence("", "abc")    # ?
is_subsequence("abc", "")    # ?
```

Then delete the `i < len(a)` guard and run `is_subsequence("ab", "abc")`. Read the traceback and say why Python's short-circuiting `and` was doing real work.

On problem 2, break the window

Run this version and predict the output on all four inputs first:

```
def length_of_longest_substring(s):
    last = {}
    start = 0
    best = 0
    for i, ch in enumerate(s):
        if ch in last:                # the guard is missing
            start = last[ch] + 1
            last[ch] = i
            best = max(best, i - start + 1)
    return best

for x in ("abcabcbb", "pwwkew", "abba", "dvdf"):
    print(x, length_of_longest_substring(x))
```

Two of those four are wrong. Say which, trace the exact step where `start` moved backwards, and then state the fix two ways — the `≥ start` guard, and `start = max(start, last[ch] + 1)`.

The counting drill

Without running anything:

1. How many substrings does a string of length 7 have? Length 20?
2. How many subsequences?
3. At what length does the subsequence count pass a million? A billion?
4. "aaa" — how many substrings, and how many **distinct** substrings? Why do those differ?
5. Listing every substring of a 1,000-character string — how much memory, roughly, and why is it cubic rather than quadratic?

Question 4 is a real interview follow-up. Question 5 is the reason you work with indices.

The classification drill

For each phrase, say **substring** or **subsequence** in under three seconds, then name the technique:

1. “longest substring with at most two distinct characters”
2. “longest increasing subsequence”
3. “does string A appear inside string B”
4. “minimum window containing all characters of T”
5. “delete some characters from A to make it equal B”
6. “count subarrays whose sum equals k”
7. “longest palindromic substring”
8. “longest palindromic subsequence”
9. “maximum sum of any k consecutive elements”
10. “can you interleave A and B to make C”

Numbers 7 and 8 are the same words apart from one, and they need completely different code. Say what each needs.

The enumeration drill

Run this and watch the second column stop being reasonable:

```
def all_substrings(s):
    return [s[i:j] for i in range(len(s)) for j in range(i + 1, len(s) + 1)]

def all_subsequences(s):
    out = [""]
    for ch in s:
        out += [prev + ch for prev in out]
    return out

for n in (3, 5, 10, 15, 20):
    t = "abcdefghijklmnopqrst"[:n]
    print(n, len(all_substrings(t)), len(all_subsequences(t)))
```

Stop at 20. Then work out on your own what `n = 40` would need, and say why that is not a matter of waiting longer.

The API design drill

Design the API for a **ride-hailing app's trip flow**, in fifteen minutes, using the seven steps. Cover: finding nearby drivers, requesting a ride, cancelling, tracking the driver's position, ending the trip, paying, and rating.

Time yourself against the shape:

```
0-1    scope: three or four questions, then stop
1-3    nouns, out loud, before any path
3-7    paths and methods
7-9    one response body
9-11   errors with codes
11-13  auth, pagination, idempotency, rate limits
13-15  one trade-off
```

Then score yourself out of nine on the checklist:

- ☐ nouns, plural, consistent
- ☐ nested for list and create only
- ☐ filters in the query string
- ☐ every collection paginated, with a cap
- ☐ empty collection is `200` with `[]`
- ☐ idempotency key on ride request and on payment
- ☐ `401` versus `403` used correctly, ownership checked in the query
- ☐ versioning mentioned
- ☐ rate limits mentioned with `429` and `Retry-After`

Anything under seven means do it again tomorrow with a different feature.

The unseen-feature drill

This is Farhat's list. Pick one at random and design it in ten minutes, with no preparation:

a group chat · a hotel booking flow · a movie ticket seat selection · a bank statement export · a job application tracker · a fitness app's workout log · a school attendance system · a parcel tracking service · a subscription billing page · a photo album with sharing

The point is not the design. It is that you never rehearsed that one.

The choosing drill

Give the **condition**, not a preference, for each. One sentence each, out loud:

1. REST or GraphQL?
2. REST or gRPC?
3. Session or JWT?
4. `PUT` or `PATCH`?
5. `PATCH /x` or `POST /x/action`?
6. Offset or cursor pagination?

7. Fail open or fail closed when the rate-limit store is down?
8. Shard the database or not?

For number 8, the answer should contain arithmetic.

The arithmetic drill

From memory, in under three minutes:

- 10M daily users $\times$ 8 requests — requests per second, average and peak.
 - 80% cacheable at a 90% hit rate — origin load, and the latency difference.
 - 300,000 writes against 80M reads — the ratio, and the one conclusion it forces.
 - 1M orders a day at 2 KB — storage per year with indexes, and whether you shard.
 - bcrypt at 250 ms, 125,000 logins in the peak hour — cores.
 - Fixed window at 100/min — the worst-case burst and when.
 - 50,000 comments unpaginated — megabytes, and seconds on a 2 Mbps connection.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How many substrings does a string of length n have? How many subsequences?* Derive both rather than reciting. Then say what the two shapes imply about technique — quadratic means you may enumerate, exponential means you never can, which is why one gets a window and the other gets a table.
 2. *Design the API for a food delivery app's order flow.* Four scoping questions, then the nouns out loud, then paths. Say “every collection is paginated” and “order creation takes an idempotency key” before anyone asks. Leave time for errors and one trade-off. Fifteen minutes.
 3. *What is the difference between longest common substring and longest common subsequence?* One line of the recurrence, and where the answer lives. Then the deeper point: contiguity means a mismatch breaks the run, which is why one resets to zero and the other carries the maximum forward.
-

Before you move on

- [] I say “contiguous?” out loud whenever the word substring or subsequence is missing.
- [] I can derive $n(n+1)/2$ and 2^n rather than reciting them.

- [] I know contiguous means sliding window and gaps mean a table, and I can say why.
- [] I can write the window guard `last[ch] ≥ start` and name the input that proves it.
- [] I can state the one line that differs between the two “longest common” problems.
- [] I can run the seven-step API procedure on an unseen feature without preparation.
- [] I produce pagination and idempotency before anyone asks for them.
- [] I answer every “which one” question with a condition, not a preference.
- [] I can redraw the whole-phase diagram from memory, in whatever tool I like.

Pattern matching, the simple way

DATA STRUCTURES AND ALGORITHMS

Pattern matching, the simple way

You can write naive substring search and state its cost honestly.

SYSTEM DESIGN

What a database gives you that a file does not

You can list the four things a database does that your own file-handling code would get wrong.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Implement `strStr`: find the first occurrence of a needle in a haystack.*
- *Why not just store the data in a JSON file?*

1 What this is, and why they ask it

Given a long string — the **haystack** — and a short one — the **needle** — find the position where the needle first appears, or report that it does not. `strStr("sadbutsad", "sad")` is `0`. `strStr("leetcode", "leeto")` is `-1`.

The straightforward answer is: try every starting position, and at each one compare the needle against the haystack character by character until it either runs out or fails. Two loops, about ten lines, and it is called **naive** or **brute-force** substring search.

The interesting part is the cost, and it is genuinely subtle. The worst case is $O(n \times m)$ — you can build an input where nearly every starting position matches almost the whole needle before failing. The *typical* case on real text is close to $O(n)$, because ordinary English mismatches on the first or second character almost every time. Both statements are true, and being able to say both, with the input that produces each, is exactly what the question is testing. §6 has the measurements: 43 comparisons on English text, 10,901 on an adversarial one of the same size.

Interviewers ask it because it is a five-minute problem with a real conversation attached. Anyone can write the loops. The follow-up — “*can you do better than $O(n \cdot m)$?*” — separates candidates who know that KMP, Rabin-Karp and Boyer-Moore exist, and what each buys, from those who do not. You are not expected to implement KMP under time pressure. You are expected to name it and say what it does.

2 The story

The curtains in Ayesha’s front room have been up since before her daughter was born, and last month the one on the left tore right down the seam where the sun has been hitting it for years. The rest of the cloth is fine. She wants one more panel of the same print, and the shop she bought it from has closed.

So she cuts a piece about the size of her hand from the torn end and takes it to the big cloth market, and this is where it becomes a proper job of work.

The shop she ends up in has rolls standing on end all along one wall, and the man unrolls one across the counter — twenty feet of printed cotton, small cream flowers on a dark ground, close enough that she cannot tell from across the room.

The way he checks is the only way there is. He lays her piece down at the very start of the roll, lines up the top corner, and looks along it. Flower matches flower. Second one matches. Third one is slightly the wrong shape, and that is that — no match here.

He does not throw the whole roll out. He slides her piece along by one repeat and starts again from the top corner of her piece. First flower, second flower, third, fourth, and then the little gap between the rows is wrong. Slide along again, start from the top of her piece again.

The frustrating rolls are the ones that are nearly right. There is one with a print so close to hers that he gets seven or eight flowers in every time before something is off, and checking that roll takes him ten minutes on its own. The obviously wrong ones take two seconds each, because the very first flower is a different colour and he is done.

On the fourth roll it goes all the way. Every flower, every gap, right to the end of her little piece, and he says that is it and cuts her three metres.

She asks how he knew where to start each time. He says you always start at the top of the piece again. You cannot carry over what matched before, because you have moved.

3 The idea in plain English

The roll of cloth is the haystack. Ayesha's hand-sized piece is the needle. The shopkeeper sliding along one repeat at a time and starting from the top of her piece each time is the naive algorithm, exactly.

The algorithm

```
for start in range(n - m + 1):    # every position the needle could begin at
    k = 0
    while k < m and haystack[start + k] == needle[k]:
        k += 1
    if k == m:                    # the inner loop ran out of needle: full match
        return start
return -1
```

Two loops. The outer one picks a starting position. The inner one compares the needle against the haystack from that position, stopping at the first mismatch or at the end of the needle. If it stopped because it reached the end of the needle — `k == m` — every character matched, so this is the answer.

The two details that are easy to get wrong

`range(n - m + 1)`, not `range(n)`. A needle of length `m` cannot start later than position `n - m`, because there would not be enough haystack left. `range` excludes its upper bound, so the `+ 1` is what lets the last valid position be tried. On `haystack = "hello"`

(5) and `needle = "lo"` (2), the valid starts are 0, 1, 2, 3 — that is `range(5 - 2 + 1) = range(4)`. Off by one here and you either miss a match at the end or read off the string.

k resets to 0 at every start. That is the shopkeeper's own answer: *you always start at the top of the piece again*. Whatever matched at the previous starting position tells you nothing about this one, because the alignment moved. That resetting is precisely the wasted work KMP removes.

The empty-needle convention

`strStr(anything, "")` returns 0, by convention — an empty needle is found immediately, at the start. It matches `"abc".find("")`, which is 0, and it also falls out of the code for free: with `m = 0` the inner loop runs zero times, `k = m` is `0 = 0`, and it returns the first `start`. Ask about it anyway; some versions of the problem want -1.

The cost, honestly

Worst case. The outer loop runs about `n` times, and the inner loop can run up to `m` times each, so $O(n \times m)$. To make that actually happen you need a needle that almost matches everywhere:

```
haystack = "aaaaaaaaa...aaab"    (1,000 a's then a b)
needle   = "aaaaaaaaaab"        (10 a's then a b)
```

At every starting position the inner loop matches all ten `a`s and then fails on the `b`. §6 measures it: **10,901 comparisons** for a haystack of 1,001 characters.

Typical case. On ordinary text, the inner loop almost always fails on the very first character, because most letters are not the one you are looking for. Measured on English prose, finding an eight-character needle took **43 comparisons** — barely more than the 35 positions it had to walk past. In practice the naive algorithm behaves like $O(n)$ on natural language.

Both are true. The honest sentence is: “It is $O(n \cdot m)$ in the worst case and close to $O(n)$ on real text, and the worst case needs a highly repetitive haystack and needle to trigger.”

What Python actually does

```
haystack.find(needle)    # position, or -1
haystack.index(needle)   # position, or raises ValueError
needle in haystack       # True or False
```

All three use the same underlying search, which is a hybrid of Boyer-Moore and Horspool with a worst-case fallback — far faster than the naive loop and written in C. §6 measures it at about **23,000 times faster** on the adversarial input. In real code, use `find`. In an interview, write the loop and then say `find` exists.

The better algorithms, by name

You will be asked “*can you do better?*”. Have one sentence each ready. Do **not** attempt to implement KMP from memory unless you have practised it.

ALGORITHM	IDEA	TIME
KMP (Knuth-Morris-Pratt)	Precompute, for each prefix of the needle, the longest prefix that is also a suffix. On a mismatch, jump the needle forward by that much instead of restarting — so the haystack index never moves backwards.	$O(n + m)$
Rabin-Karp	Compare a rolling hash of each window instead of the characters, and only compare characters when the hashes agree.	$O(n + m)$ average, $O(n \cdot m)$ worst
Boyer-Moore	Compare from the right end of the needle, and on a mismatch skip ahead by however far that character is from the needle's end. Often skips most of the haystack.	$O(n/m)$ best, sublinear in practice

KMP is the expected answer, because it is the one with a guaranteed linear bound. The sentence to say: “*KMP precomputes a table of how far the needle can safely jump on a mismatch, so the haystack index never goes backwards — that makes it $O(n + m)$.*” Rabin-Karp is the right answer when you want to search for **many** needles at once, because one pass of hashing serves them all.

4 The picture

Searching for "sad" in "sadbutsad" :

```
start = 0
s a d b u t s a d
| | |
s a d                → all three match, k reaches 3 = m, return 0
```

Now "issip" in "mississippi", where the sliding is visible:

```
start = 0      m i s s i s s i p p i
                x                      'm' vs 'i' → fail after 1 comparison
                i

start = 1      m i s s i s s i p p i
                | | x                    'i','s' match, 's' vs 's'... 's' vs 'i' fails at k=2
                i s s i p

start = 4      m i s s i s s i p p i
                | | | | |                all five match → return 4
                i s s i p
```

What to notice: at `start = 1` the comparison got two characters in before failing, and every one of those comparisons is thrown away — the next attempt begins again at `k = 0`. That discarded work is what KMP recovers.

The worst case, drawn:

```
haystack: a a a a a a a a a a a a ... a a a b
needle:   a a a a a a a a a a b

start = 0  a a a a a a a a a a b      10 matches, then 'a' vs 'b' fails → 11 comparisons
start = 1   a a a a a a a a a a b      10 matches, then fails           → 11 comparisons
start = 2   a a a a a a a a a a b      10 matches, then fails           → 11 comparisons
...
~n starting positions, ~m comparisons each → n x m
```

What to notice: the needle nearly matches everywhere and only fails at the last character. That is the shopkeeper's roll of nearly-identical cloth, and it is the only shape of input that makes the naive algorithm slow.

5 The code, built step by step

The bound on the outer loop

```
n, m = len(haystack), len(needle)
for start in range(n - m + 1):
    ...
```

Work it out on a small case rather than remembering it: haystack length 5, needle length 2, so the needle can start at 0, 1, 2 or 3 — four positions, which is $5 - 2 + 1$.

If `m > n` then `n - m + 1` is zero or negative, `range` produces nothing, and the function correctly falls through to `return -1`. No special case needed, which is worth noticing out loud.

The inner comparison

```
k = 0
while k < m and haystack[start + k] == needle[k]:
    k += 1
```

Walk forward while the characters agree. `k < m` comes first, and it must, because Python's `and` short-circuits — without that ordering, `needle[k]` would read off the end of the needle the moment everything matched.

After this loop there are exactly two possibilities: `k == m`, meaning the needle ran out and every character matched, or `k < m`, meaning a character differed.

The test

```
if k == m:
    return start
```

`k == m`, not `k == m - 1`. The loop increments `k` after each successful comparison, so a full match leaves `k` one past the last character. This is the same off-by-one as the `range` bound, in a different costume.

The guard, and the whole thing

```
def str_str(haystack: str, needle: str) → int:
    if not needle:
        return 0
    n, m = len(haystack), len(needle)
    for start in range(n - m + 1):
        k = 0
        while k < m and haystack[start + k] == needle[k]:
            k += 1
        if k == m:
            return start
    return -1
```

The explicit empty guard is not strictly needed — the loop handles it — but it states the contract plainly, and stating the contract is worth more than saving a line.

Counting the comparisons, to make the cost real

```
def count_comparisons(haystack: str, needle: str) → tuple[int, int]:
    """Returns (position, number of character comparisons made)."""
    comparisons = 0
    n, m = len(haystack), len(needle)
    for start in range(n - m + 1):
        k = 0
        while k < m and haystack[start + k] == needle[k]:
            k += 1
            comparisons += 1
        if k < m:
            comparisons += 1          # the one that failed
        if k == m:
            return start, comparisons
    return -1, comparisons
```

Run this on two inputs of similar size and the difference between the worst case and the typical case stops being an abstraction. §6 has the numbers.

The complete solutions

```
import time

def str_str(haystack: str, needle: str) → int:
    """LeetCode 28. First index of needle in haystack, or -1.

    Naive: try every start, compare forward from k = 0 each time.
    O(n*m) worst case, close to O(n) on ordinary text.
    """
    if not needle:
        return 0 # convention: empty needle is found at 0
    n, m = len(haystack), len(needle)
    for start in range(n - m + 1): # last valid start is n - m
        k = 0
        while k < m and haystack[start + k] == needle[k]: # k < m FIRST: short-circuits
            k += 1
        if k == m: # ran out of needle, so all matched
            return start
    return -1

def count_comparisons(haystack: str, needle: str) → tuple[int, int]:
    """Same search, but reports how many character comparisons it took."""
    comparisons = 0
    n, m = len(haystack), len(needle)
    for start in range(n - m + 1):
        k = 0
        while k < m and haystack[start + k] == needle[k]:
            k += 1
            comparisons += 1
        if k < m:
            comparisons += 1
        if k == m:
            return start, comparisons
    return -1, comparisons

def find_all(haystack: str, needle: str) → list[int]:
    """Every occurrence, including overlapping ones. 'aaa' in 'aaaa' → [0, 1]."""
    if not needle:
        return []
    out: list[int] = []
    n, m = len(haystack), len(needle)
    for start in range(n - m + 1):
        if haystack[start:start + m] == needle: # slice-compare: clear, and O(m)
            out.append(start)
    return out

if __name__ == "__main__":
    tests = [("sadbutsad", "sad"), ("leetcode", "leeto"), ("hello", "ll"),
             ("", ""), ("", "a"), ("a", ""), ("aaaaa", "bba"), ("mississippi", "issip")]
    print([str_str(h, n) for h, n in tests])
    # [0, -1, 2, 0, -1, 0, -1, 4]
    print([h.find(n) for h, n in tests])
    # identical – the naive version agrees with the library on all of them

    print(count_comparisons("a" * 1000 + "b", "a" * 10 + "b"))
    # (990, 10901) ← the adversarial case: ~n*m
    print(count_comparisons("the quick brown fox jumps over the lazy dog" * 10, "lazy dog"))
    # (35, 43) ← ordinary English: barely more than n

    print(find_all("aaaa", "aa")) # [0, 1, 2] overlapping

    haystack = "a" * 200_000 + "b"
    needle = "a" * 1_000 + "b"
    start = time.perf_counter()
```

```

str_str(haystack, needle)
naive = time.perf_counter() - start
start = time.perf_counter()
haystack.find(needle)
builtin = time.perf_counter() - start
print(f"naive {naive:.4f}s    builtin {builtin:.6f}s    ratio {naive / builtin:.0f}x")
# naive 13.9039s    builtin 0.000594s    ratio 23403x

```

6 What it costs

Counted from the loops

The outer loop runs $n - m + 1$ times, which is about n when the needle is short. For each of those, the inner loop runs at most m times. So the body executes at most $(n - m + 1) \times m$ times, and each execution is one character comparison — constant work.

$O(n \times m)$ time. Space is k , `start`, n and m : four integers, so **$O(1)$ extra space.**

The worst case, measured

```

haystack = 1,000 'a's followed by 'b'    (length 1,001)
needle   = 10 'a's followed by 'b'      (length 11)

comparisons made: 10,901
n x m             = 1,001 x 11 = 11,011

```

10,901 against a theoretical ceiling of 11,011 — so this input drives the algorithm to about 99% of its worst case. Every start matches ten characters and fails on the eleventh.

The typical case, measured

```

haystack = 430 characters of ordinary English
needle   = "lazy dog" (8 characters)
found at position 35

comparisons made: 43

```

Forty-three comparisons to walk past 35 positions. The inner loop essentially never ran more than once, because in English the chance that a random position starts with `l` is small, and if it does not, the attempt costs one comparison. **On natural language the naive algorithm is effectively linear**, and that is why it survives in real code far more often than its worst case suggests.

Against the library

```
haystack = 200,000 'a's + 'b'
needle   = 1,000 'a's + 'b'

naive loop : 13.9039 s
str.find   : 0.000594 s
ratio      : about 23,000x
```

Two things are happening. The library uses a better algorithm — a Boyer-Moore-Horspool hybrid that skips ahead rather than sliding by one — and it runs as compiled C rather than interpreted Python. Neither alone explains 23,000×; together they do.

Say this in an interview: “In production I would call `find`, which is a smarter algorithm in C. I am writing the loop because you asked me to implement it.”

The better algorithms

	TIME	EXTRA SPACE	WHEN IT WINS
Naive	$O(n \cdot m)$ worst, $\sim O(n)$ typical	$O(1)$	short needles, ordinary text
KMP	$O(n + m)$ guaranteed	$O(m)$ for the table	repetitive input, worst-case guarantees
Rabin-Karp	$O(n + m)$ average	$O(1)$	searching many needles at once
Boyer-Moore	sublinear in practice, $O(n/m)$ best	$O(\text{alphabet})$	long needles, large alphabets

For `n = 200,000` and `m = 1,000`: naive is up to 200 million comparisons, KMP is at most 201,000. That is the thousand-fold difference the table is describing, and it is the number to quote.

7 The traps

The near-miss: the outer loop bound

```
for start in range(n):                                # should be n - m + 1
    k = 0
    while k < m and haystack[start + k] == needle[k]:
        k += 1
    if k == m:
        return start
```

```
print(str_str("hello", "lo"))
```


This one happens to work, because the inner loop's `k < m` test stops it before it reads too far — `haystack[start + k]` with `start = 4` and `k = 0` is `'o'`, which mismatches `'l'`, so nothing runs off the end. But it does `m` extra useless iterations of the outer loop, and if you write the inner comparison as a slice instead — `haystack[start:start+m] == needle` — the slice silently comes back short and can never match, so a needle at the very end is missed. **Compute `n - m + 1` and mean it.**

The near-miss: not resetting `k`

```
k = 0
for start in range(n - m + 1):          # k declared outside the loop
    while k < m and haystack[start + k] == needle[k]:
        k += 1
    if k == m:
        return start
```

After the first failed attempt, `k` keeps whatever value it reached, so the next attempt starts comparing from the middle of the needle against the wrong part of the haystack. It produces confident wrong answers. This is the shopkeeper's rule violated: **you always start at the top of the piece again.**

The real error: comparing before checking the bound

```
while haystack[start + k] == needle[k] and k < m:      # order swapped
    k += 1
```

```
Traceback (most recent call last):
...
    while haystack[start + k] == needle[k] and k < m:
                                ~~~~~^^^
IndexError: string index out of range
```

The moment every character matches, `k` reaches `m` and `needle[k]` reads off the end. Python's `and` evaluates left to right and stops as soon as the answer is known, so `k < m` **must come first**. This ordering is not stylistic; it is the guard.

The contract corner: the empty needle

```
str_str("abc", "")      # 0, by convention
"abc".find("")          # 0
```

Every string contains the empty string at position 0. It falls out of the loop naturally, so the explicit guard is documentation rather than logic — but **ask**, because some problem statements specify `-1` and you would be marked wrong for the convention.

The near-miss: `find` versus `index`

```
"abc".find("z")      # -1
"abc".index("z")     # ValueError: substring not found
```

`find` reports failure with a sentinel; `index` raises. Choose deliberately — `find` when absence is normal, `index` when absence is a bug you want to hear about. Writing `if s.index(x):` is a bug twice over: it raises when absent, and it is falsy when the answer is 0.

The trap in the follow-up: attempting KMP from memory

The failure-function construction in KMP is short and extremely easy to get subtly wrong, and a half-correct KMP is worse than a correct naive solution plus an accurate description. **The right move is: write the naive version, state both costs, name KMP with one sentence about what it does, and offer to work through the table if they want to spend the time on it.** That reads as judgement. Fumbling a half-remembered table for ten minutes does not.

8 In the interview

How it gets asked

- “Implement `strStr` / `indexOf`.” — LeetCode 28. Ten minutes, and the follow-up is the real question.
- “What’s the time complexity?” — where the good answer has two parts and an example input for each.
- “Can you do better than $O(n \cdot m)$?” — name KMP, say what it does, say why it is linear.
- “Find all occurrences, including overlapping ones.” — the small variant that catches people who jump `start` by `m` instead of by 1.
- “How would you search for a thousand different needles in the same text?” — where Rabin-Karp or Aho-Corasick is the right name to produce.

What to say out loud, in the first ninety seconds

1. **Ask the contract questions.** “What should I return for an empty needle — 0 or -1? And do you want the first occurrence or all of them?”
2. **Describe the approach before writing it.** “Try every starting position where the needle could fit, and at each one compare forward until a mismatch or until the needle runs out.”
3. **Say the bound out loud as you write it.** “The last position the needle can start at is n minus m , so the loop is `range(n - m + 1)`.”

4. **Flag the short-circuit.** *“The bound test goes first in the while condition, or the moment everything matches I read past the end of the needle.”*
5. **Give both costs, with the input for each.** *“ $O(n \cdot m)$ worst case — you need something like a thousand a’s with a needle of ten a’s and a b, where every position matches almost the whole needle before failing. On ordinary text it’s close to $O(n)$, because most positions fail on the first character.”*
6. **Say what the library does.** *“In production I’d call `find`, which uses a Boyer-Moore variant in C.”*
7. **Offer the improvement before being asked.** *“If you want a guaranteed linear bound, that’s KMP.”*

The follow-ups

“What’s the actual worst case, and can you construct it?” $O(n \cdot m)$, and yes. I need the needle to almost match at every position and fail only at the end — so a haystack of a thousand a s followed by a b, and a needle of ten a s followed by a b. At every one of the roughly thousand starting positions, the inner loop matches ten characters and then fails on the eleventh, so it does about $n \times m$ comparisons. I measured that exact case: 10,901 comparisons against a theoretical maximum of 11,011, so it drives the algorithm to about 99% of its worst case. The reason this rarely bites in practice is that natural language has almost no repetition of that kind — on English prose the same code took 43 comparisons to find an eight-character needle 35 characters in, because the inner loop almost always fails on the very first character. So the honest answer is that it is $O(n \cdot m)$ in theory and behaves linearly on text, and the inputs that break it are DNA, binary data, and anything an adversary chose.

“Can you do better?” KMP gives a guaranteed $O(n + m)$. The idea is that when the naive algorithm fails partway through the needle, it throws away everything it just learned and restarts at the top of the needle one position along. But it *does* know something: it knows exactly which characters of the needle it just matched. KMP precomputes, for every prefix of the needle, the length of the longest proper prefix that is also a suffix of it — and on a mismatch it slides the needle forward by that amount instead of by one, so the haystack index never moves backwards. That is what makes it linear. Building the table is $O(m)$ and it uses $O(m)$ extra space. I would not try to write it from memory unless you want to spend the time, because the table construction is easy to get subtly wrong; I would rather give you a correct naive solution and an accurate description of KMP.

“Find all occurrences, including overlapping ones.” Almost the same loop, with two changes. Instead of returning at the first match, append the position and carry on. And critically, advance `start` by one, not by `m` — because “aa” occurs in “aaaa” at positions 0, 1 and 2, and jumping by the needle length would find only 0 and 2. If over-

lapping matches are explicitly not wanted, jumping by `m` is the correct behaviour, so this is a contract question rather than a bug. Worth asking, because the phrase “all occurrences” is genuinely ambiguous and the two answers differ.

“How would you search for a thousand needles in the same text?” Not by running this a thousand times, which is `O(1000 · n · m)`. Two answers depending on the shape. Rabin-Karp handles it well: it hashes the current window of the haystack and compares against a set containing the hashes of all the needles, so one pass serves every needle, provided they are all the same length. For needles of different lengths, the right answer is Aho-Corasick, which builds a single automaton from all the patterns — essentially a trie with failure links, which is the KMP idea generalised to many patterns — and then makes one pass over the haystack finding every occurrence of every needle simultaneously, in `O(n + total pattern length + matches)`. That is what tools like `grep -f` and intrusion-detection systems actually use. Tries arrive on [day 120](#), and Aho-Corasick is the natural sequel.

A model answer

“Two questions first: what should I return for an empty needle, 0 or -1? And do you want just the first occurrence?

...0, and just the first. Good.

The straightforward approach is to try every position in the haystack where the needle could possibly start, and at each of those compare the needle forward, character by character, until either something differs or the needle runs out. If it runs out, everything matched, so that position is the answer.

```
def str_str(haystack: str, needle: str) → int:
    if not needle:
        return 0
    n, m = len(haystack), len(needle)
    for start in range(n - m + 1):
        k = 0
        while k < m and haystack[start + k] == needle[k]:
            k += 1
        if k == m:
            return start
    return -1
```

Three details worth calling out. The outer loop goes to $n - m + 1$, because the last position the needle can start at is $n - m$ — on a five-character haystack with a two-character needle the valid starts are 0 through 3. The $k < m$ test has to come first in the while condition, because Python’s `and` short-circuits and without that ordering the moment everything matches I’d read past the end of the needle. And `k` resets to zero at every start: whatever matched at the last position tells me nothing here, because the alignment moved.

On cost: it’s $O(n \cdot m)$ in the worst case and $O(1)$ space. To actually hit the worst case you need a highly repetitive input — a thousand `a` s with a needle of ten `a` s and a `b` — where every position matches almost the whole needle and fails on the last character. I measured that: about 10,900 comparisons for a thousand-character haystack. But on ordinary text it behaves linearly, because almost every position fails on the very first character — the same code found an eight-character phrase in 430 characters of English in 43 comparisons.

In production I’d just call `find`, which uses a Boyer-Moore variant written in C — on that adversarial input it was about 23,000 times faster than my loop.

And if you want a guaranteed linear bound, that’s KMP. The insight is that when the naive version fails partway through the needle it discards everything it learned, but it does know which characters it just matched — so KMP precomputes, for each pre-

fix of the needle, the longest proper prefix that is also a suffix, and slides the needle forward by that much on a mismatch rather than by one. The haystack index then never moves backwards, which gives $O(n + m)$ with $O(m)$ extra space for the table. I'd rather describe it than write it from memory — the table construction is famously easy to get subtly wrong — but I'm happy to work through it if you'd like."

9 Recall card

- **Naive search:** try every start, compare forward, reset to the top of the needle each time.
- $\text{range}(n - m + 1)$ — the last valid start is $n - m$. And $k < m$ goes **first** in the while condition.
- $O(n \cdot m)$ **worst case**, $O(1)$ **space** — but close to $O(n)$ on real text, because most positions fail on the first character.
- **The worst case needs repetition:** "aaaa...b" with needle "aaab". Measured: 10,901 comparisons for $n = 1,001$.
- **Better: KMP** $O(n + m)$ (never move the haystack index back), **Rabin-Karp** (rolling hash, many needles), **Boyer-Moore** (skip from the right). Name them; do not improve them.

What a database gives you that a file does not

1 What this is, and why they ask it

A **database** is a program whose entire job is storing data on behalf of other programs, safely, concurrently, and in a way that lets you find things without reading everything.

That sounds obvious until you try to build it. Every one of those properties is a hard problem, and the reason you use Postgres rather than writing to `data.json` is that Postgres has solved all of them and your file-handling code has solved none of them. There are four things your own file handling would get wrong:

- **Durability** — once it says the write succeeded, the data survives the process dying mid-write.
- **Concurrency** — many readers and writers at once, without one silently overwriting another.
- **Fast lookup** — find one row among a hundred million without reading a hundred million.
- **Integrity** — data that violates your rules is refused rather than stored.

And one mechanism that ties the first two together — the **transaction**, a group of changes that either all happen or none do.

Interviewers ask this at the start of the database phase for a good reason: it establishes whether you know *why* the component exists, rather than just that it does. It also has a real answer in the other direction. Sometimes a file **is** the right choice, and a candidate who says “always use a database” is repeating a rule rather than thinking. Configuration, static content and single-writer append logs are all perfectly good as files.

This is day one of eighteen on databases, running to [day 042](#). Everything after this — tables, keys, SQL, joins, indexes, transactions, replication — is a detail of one of the four things above.

2 The story

Imran and Faisal run a small transport business out of a room behind their father’s shop, two lorries and a driver each, mostly moving cement and steel to sites around the district.

For four years the entire business lived in the contacts on Imran's phone. Not just names and numbers — he used the notes field. *Rajan Constructions, site at Perundurai, ₹18,000 pending, pays on the 10th, don't call before 11.* Four hundred and some entries like that, and he could find anything in it in seconds, because he had put it all there himself.

Three things went wrong, and they went wrong in a particular order.

The first was Faisal. When he started handling half the customers they copied the whole lot onto his phone so they would both have it. Within a month it had come apart. Imran changed a number on his phone on the Tuesday; Faisal added two new customers on his; and when they sent the list across at the end of the week, whichever one arrived last simply replaced the other, and a week of somebody's work was gone. Twice they lost the wrong week and did not notice for a month.

The second was the question their accountant asked in April. Which customers in Erode district owe more than ten thousand, and how much is it altogether? Imran can search his contacts for a name. He cannot search them for that. He spent an evening scrolling through four hundred entries with a calculator, got a number, and was fairly sure it was wrong.

The third was in June, when Imran's phone went into a bucket of water at a site. He had a backup, from March.

The thing that actually annoys him most, though, is smaller than any of those. There is nothing in a phone that stops you saving a customer with no number at all, or the same customer twice under two spellings, or a payment date of the thirty-first of February. He has all three in there. Nobody typed them wrong on purpose. There was simply nothing there to say no.

3 The idea in plain English

Imran's contacts are a file. Everything that went wrong is one of the four things a database does, and they map one to one.

Durability: it survives the crash

Imran's backup was from March. The general version of that problem is worse and more subtle: **a file write is not one action**. If your program writes 5 MB of JSON and the process is killed at 3 MB, the file on disk is half old and half new, and it is no longer valid JSON at all — you have not lost the last change, you have lost **everything**.

A database avoids this with a **write-ahead log**. Before changing the actual data, it appends a record of what it is about to do to a log file, and forces that to disk. If the power fails halfway, the log is replayed on restart and the change is either fully applied

or fully discarded. When Postgres says `COMMIT` succeeded, the data is on disk, and that promise is the **D** in **ACID** — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [day 033](#).

The naive file version of this is `open("data.json", "w")`, which **truncates the file to zero before writing a single byte**. A crash at that moment leaves you with nothing at all.

Concurrency: two writers do not destroy each other

This is Imran and Faisal, and it is the failure most people underestimate.

```
10:00 Imran's process reads data.json (400 customers)
10:00 Faisal's process reads data.json (400 customers)
10:01 Imran adds a customer, writes 401
10:02 Faisal adds a customer, writes 401 ← Imran's addition is gone
```

Nothing errored. No warning. One of the two changes simply does not exist, and this is called a **lost update**. Do it with two threads inside one process and you get the same thing plus corrupted files, because two writers interleaving in the same file produce bytes that are neither version.

A database handles this with **locking** and **transactions**. Two transactions touching the same row are serialised; two touching different rows run in parallel. You get the answer you would have got if they had run one after another, which is the **I** in ACID — isolation.

You can build file locking yourself. People do, and then they discover that it does not work across machines, that a crashed process leaves the lock held forever, and that a lock around the whole file means one writer at a time for the entire dataset.

Fast lookup: finding without reading everything

This is the accountant's question. Contacts can search by name because that is the one thing the phone maintains a lookup for. Anything else means reading all four hundred.

For a file, finding one record means reading and parsing **the whole file**, which is $O(n)$. That is fine for four hundred records and ruinous for ten million:

```
10,000,000 records × 200 bytes = 2 GB
reading 2 GB from an SSD at ~500 MB/s ≈ 4 seconds, per query
```

A database keeps an **index** — a separate structure, usually a B-tree, that maps a value to the location of the matching rows. Looking up one row among ten million becomes about three or four disk reads rather than a scan, so **milliseconds instead of seconds**. Indexes

are [day 030](#); for today the point is that this is a thing you get, and that you would have to build it yourself otherwise.

Integrity: bad data is refused

This is the customer with no number and the thirty-first of February. A file will store anything you put in it, because a file has no opinions. A database refuses:

```
CREATE TABLE customers (  
    id          BIGSERIAL PRIMARY KEY,  
    phone       TEXT NOT NULL UNIQUE,  
    district    TEXT NOT NULL,  
    balance_due NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (balance_due ≥ 0),  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

`NOT NULL` refuses a missing phone. `UNIQUE` refuses the duplicate. `CHECK` refuses a negative balance. `TIMESTAMPTZ` refuses the thirty-first of February outright, because it knows what a date is and `"31/02"` in a JSON file is just a string.

The value is that the rule lives **in one place**. Three applications and a script all write to that table, and none of them can break the rule, even by accident, even the one written in a hurry last Tuesday.

Transactions: the mechanism behind the first two

Durability and concurrency are both promises about *groups* of changes, and the transaction is what delivers them. Move ₹5,000 from one account to another and there are two changes: subtract from one, add to the other. If the process dies between them, money has vanished.

```
BEGIN;  
    UPDATE accounts SET balance = balance - 5000 WHERE id = 1;  
    UPDATE accounts SET balance = balance + 5000 WHERE id = 2;  
COMMIT;
```

Either both take effect or neither does. That is **atomicity** — the **A** in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

When a file is genuinely the right answer

Say this unprompted, because it is what distinguishes thinking from reciting.

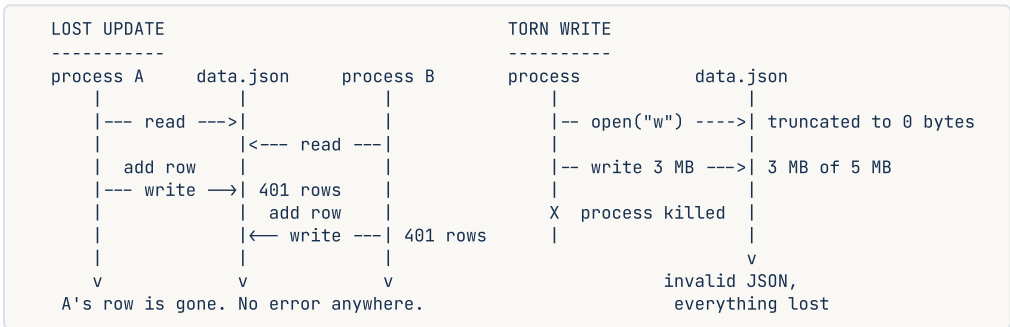
- **Configuration** that is read at start-up and written by a human. A file is better: it goes in version control, it is readable, and it needs no running service.
- **Static content** — images, CSS, documents. Files on disk or in object storage like S3, with the database holding only the metadata and the path.

- **Append-only logs** with one writer. Appending is the one file operation that is close to safe, and it is why log files work.
- **Data small enough and single-user enough that none of those problems exist.** A CLI tool’s local state is fine in a JSON file.

The tell is the **number of writers**. One writer, no queries, no rules to enforce: a file. Anything else: a database. And note there is a middle option — **SQLite** — which is a real transactional database that is a single file, with no server to run. It handles durability, transactions, integrity and indexes, and it is limited on concurrent writers. For a lot of “would a file do?” situations, SQLite is the correct answer and almost nobody mentions it.

4 The picture

The two failures a file cannot survive:



What to notice: neither failure produces an error message. The first silently loses one change; the second silently loses all of them. Code that “works” in testing has both of these in it.

What a database puts between your code and the disk:

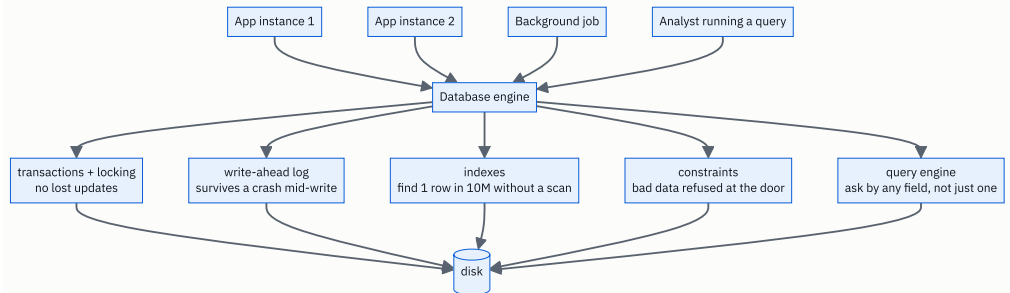


FIGURE 25.1

What to notice: four different writers, one engine. Every one of those five boxes is something you would otherwise write yourself, once per application, and get subtly wrong in a different way each time.

5 How it actually works

The write-ahead log, concretely

When you run an `UPDATE` inside a transaction, Postgres does roughly this:

1. Load the affected page from disk into memory (the **buffer pool**) if it is not already there.
2. Change it **in memory**.
3. Append a description of the change to the **write-ahead log** — the WAL.
4. On `COMMIT`, force the WAL to disk with `fsync` and only then report success.
5. Write the changed page itself to disk **later**, in the background.

Step 4 is the durability promise, and step 5 is the performance trick: the WAL is written sequentially, which on any storage is far faster than the random writes the data pages would need. If the machine loses power between 4 and 5, the WAL is replayed on restart and the change is recovered.

That single mechanism also gives you replication — a replica is a machine replaying the same WAL — and point-in-time recovery. It is one of the highest-value ideas in the whole subject.

Concurrency, concretely

Two mechanisms, and you should know both names.

Locking. A transaction writing a row takes a lock on it; another transaction wanting the same row waits. Different rows do not block each other.

MVCC — multi-version concurrency control, used by Postgres, MySQL's InnoDB, and Oracle. Instead of readers waiting for writers, an `UPDATE` creates a **new version** of the row and leaves the old one in place. Readers that started earlier keep seeing the old version. So **readers never block writers and writers never block readers**, which is the single biggest reason these databases perform well under mixed load. The price is that old versions accumulate and have to be cleaned up — Postgres calls this `VACUUM`, and a neglected `VACUUM` is one of the classic production problems.

Indexes, concretely

Without an index, `SELECT * FROM customers WHERE phone = '98765...'` is a **sequential scan**: read every row. With an index on `phone`, it is a **B-tree** lookup — a shallow, wide tree kept balanced on disk, where each node is one disk page.

```
10,000,000 rows, B-tree fanout ~500 per page:

depth 1 :      500 entries
depth 2 :    250,000
depth 3 : 125,000,000    → 3 levels is enough for 10M rows
```

So about **3 or 4 page reads** rather than reading 2 GB. Trees arrive on [day 098](#) and B-trees specifically in the database phase; the number to hold on to is that an index turns a scan into three reads.

The cost: an index is extra storage, and **every write has to update it**. Ten indexes on a table means ten extra structures maintained on every insert. That trade — reads faster, writes slower — is the whole of index design.

The products, and what each is for

PRODUCT	KIND	USE IT FOR
PostgreSQL	relational	The sensible default. Strong on correctness, JSON support, extensions.
MySQL	relational	The other default. Enormously deployed, simpler, very fast for read-heavy loads.
SQLite	relational, embed- ded	One file, no server. Mobile apps, desktop apps, CLI tools, tests.
Redis	in-memory key- value	Caches, sessions, rate-limit counters, queues. Durability is optional and weaker.
MongoDB	document	Schema-flexible JSON documents. Genuinely useful when the shape varies.
Cassandra / DynamoDB	wide-column key- value	Enormous write volume, horizontal scale, weaker query flexibility.
Elasticsearch	search index	Full-text search and analytics — not a system of record.
S3	object storage	Files. Images, videos, backups, data lake. Not queryable by field.

The honest default is Postgres, and saying so plainly is a better answer than an enthusiastic recommendation of something exotic. The vast majority of systems you will be asked to design fit comfortably on one Postgres instance with replicas.

What you are actually buying

A database is a large amount of extremely well-tested code solving problems that are easy to describe and hard to get right. Postgres is over thirty years old and millions of lines. The point is not that you *could* not write a WAL and a B-tree and a lock manager. It is that yours would be new code on the most dangerous path in your system, and theirs has been run by millions of people for decades.

6 The numbers

Scanning a file versus an index

10 million customer records, about 200 bytes each:

```
total size                = 10,000,000 × 200 B = 2 GB
read 2 GB from SSD @ 500 MB/s      ≈ 4 seconds
parse 2 GB of JSON @ ~100 MB/s     ≈ 20 seconds
-----
one lookup by phone number, from a file ≈ 24 seconds

same lookup via a B-tree index      ≈ 3-4 page reads
                                   ≈ 0.4 ms on SSD
```

24 seconds against 0.4 milliseconds — about 60,000 times. And the file version costs that on *every single query*, while using 2 GB of memory to hold the parsed result.

Memory

```
2 GB of JSON parsed into Python objects ≈ 6-10 GB of RAM
                                         (object overhead, roughly 3-5x)
```

Which is why “just load it into a dictionary at start-up” stops working long before the data looks big.

Concurrency, quantified

At **100 writes per second** with a read-modify-write cycle taking 50 ms, two writes overlap whenever they arrive within 50 ms of each other:

```
expected overlapping pairs ≈ 100 writes/s × 0.05 s ≈ 5 collisions per second
```

Five lost updates a second. Over an eight-hour day that is **144,000 silently lost changes**. This is the number that makes the concurrency argument land, because people imagine lost updates are rare.

The write-ahead log's real benefit

```
random 8 KB writes to SSD  ≈ 10,000-20,000 IOPS
sequential writes to SSD   ≈ 500 MB/s = ~60,000 8 KB writes/s
```

Roughly $4\times$ more throughput just from writing sequentially, plus the ability to batch many transactions into one `fsync`. That is why the log-then-apply design is faster **and** safer, which is a rare combination and worth pointing out.

When a file is fine

```
1,000 records × 500 bytes = 500 KB
read + parse           ≈ 5 ms
one writer, no queries → no concurrency problem, no index needed
```

Five milliseconds and no moving parts. **Adding Postgres here makes the system worse**, and saying so is the part of the answer most candidates miss.

Index cost on writes

```
table with 1 index : 1 insert into the table + 1 B-tree update
table with 8 indexes: 1 insert into the table + 8 B-tree updates
                    → roughly 3-5x slower writes in practice
```

Which is the trade to name when someone says “why not index everything?”.

7 The trade-offs

What a database costs

Operations. A process to run, monitor, back up, upgrade and secure. Connection limits to manage — Postgres handles a few hundred connections comfortably and needs PgBouncer beyond that, which surprises people the first time.

Latency. A network round trip, typically 0.5–2 ms inside a data centre, on every query. A local file read is microseconds. It almost never matters, and it is honest to mention.

Impedance. Your objects are not rows. You either write SQL by hand or adopt an ORM, and ORMs generate the $N+1$ query problem exactly as GraphQL resolvers do on [day 021](#).

Schema discipline. Changing a column on a live table with a hundred million rows is a planned operation, not an edit. That rigidity is the same property that prevents bad data — you cannot have one without the other.

What a file costs

Everything in §3, and the specific danger is that **the costs are invisible until they are catastrophic**. Lost updates produce no error. A torn write looks fine until the restart. A full scan is fast until the data grows. Every one of these works perfectly in development.

Which database

- **Relational (Postgres, MySQL)** when the data has structure and relationships, when you need transactions across several rows, and when you will query it in ways you have not thought of yet. This is most systems.
- **Document (MongoDB)** when documents are genuinely self-contained and the shape varies per record. Be aware that Postgres has a `JSONB` column type that covers a great deal of this without giving up transactions and joins.
- **Key-value (Redis, DynamoDB)** when access is always by a single key and you need very high throughput. Redis is for caches and ephemeral state; DynamoDB is a system of record with a strict access-pattern-first design.
- **Wide-column (Cassandra)** for enormous write volume across many machines, accepting weaker querying and eventual consistency.
- **Search (Elasticsearch)** alongside a real database, never instead of one. It is an index, and indexes can be rebuilt; systems of record cannot.

The sentence that separates candidates

I would use a file if there is exactly one writer, no queries beyond “read it all”, and no rules to enforce — configuration, static assets, an append-only log. The moment there are two writers I need locking; the moment there is a second way to look the data up I need an index; and the moment the process can die mid-write I need a log. Those three are what a database is, so at that point the choice is between using one and writing a bad one. And if the objection is operational overhead, SQLite is a real transactional database in a single file with no server, which covers a surprising amount of the middle ground.

8 In the interview

How it gets asked

- “Why not just store the data in a JSON file?” — the direct version, often as a warm-up before a design round.
- “What does a database actually give you?” — the same question, phrased positively.
- “When would you not use a database?” — the version that catches people who have only learned one side.
- “What’s the difference between SQL and NoSQL?” — where this vocabulary gets used, and where the honest answer starts with “what are the access patterns?”.
- “Two users update the same record at the same time. What happens?” — the concurrency half, asked as a scenario.

What to say out loud, in the first ninety seconds

1. **Name the four things, in order.** “Durability, concurrency, fast lookup and integrity — plus transactions, which are how the first two are actually delivered. A file gives you none of them.”
2. **Lead with concurrency, because it is the one people underestimate.** “With a file, two processes that read, change and write back will silently lose one of the changes. No error, nothing in the log. At a hundred writes a second that’s several lost updates a second.”
3. **Then durability, with the mechanism.** “Opening a file for writing truncates it first, so a crash halfway through loses everything, not just the last change. A database appends to a write-ahead log and forces that to disk before reporting success, so a crash replays cleanly.”
4. **Then the lookup, with the number.** “Finding one record in a file means reading and parsing all of it — 2 GB and about 24 seconds for ten million records. A B-tree index makes it three or four page reads, under a millisecond.”
5. **Then integrity, briefly.** “And the rules live in one place, so no application can violate them — `NOT NULL`, `UNIQUE`, `CHECK`, and real date types instead of strings.”
6. **Give the other side, unprompted.** “Files are genuinely right for configuration, static assets and single-writer append logs. And SQLite is a real transactional database that is a single file with no server, which covers a lot of the middle.”
7. **Say the default.** “For most systems I’d start with Postgres and not be clever about it.”

The follow-ups

“When would you actually use a file?” Whenever none of those problems exists. Configuration read at start-up and edited by a human is better as a file: it lives in version control, it is diffable and reviewable, and it needs no running service. Static assets — images, video, documents — belong on disk or in object storage like S3, with the database holding only the metadata and the key, because storing large binaries in a database

bloats backups and buffer pools for no benefit. Append-only logs work as files because appending is the one file operation that is nearly safe, which is exactly why every log file in the world is a file. And a single-user tool's local state is fine as JSON. The test I apply is the number of writers: one writer, no queries, no invariants to protect, and a file is not just acceptable but better. The moment any of those three changes, I need locking, indexing or a log — and those three things *are* a database, so I would be choosing between using one and writing a worse one.

“Two users update the same record at the same time. What happens?” With a file, one of the updates disappears silently. Both processes read the current contents, both modify their in-memory copy, and both write the whole thing back — whichever writes second wins completely, and the first one's change never existed. There is no error anywhere, which is what makes it dangerous. With a database, the two updates are serialised: the second transaction either waits for a lock or, under MVCC, works on a snapshot and gets a serialisation error it can retry. Either way the result is the same as if they had run one after the other. Worth adding that the database only protects you at the row level automatically — application-level lost updates are still possible if you read a value, compute in your code, and write it back. The fix there is either to do the arithmetic in the database (`SET balance = balance - 5000` rather than reading and subtracting) or to use optimistic locking with a version column.

“What's the difference between SQL and NoSQL, and how do you choose?” The label is less useful than it sounds, because “NoSQL” covers four unrelated families. What actually differs is the data model and what you give up. Relational databases give you a fixed schema, joins, and transactions across many rows, at the cost of being harder to scale horizontally. Document stores give you flexible per-record shapes and easy horizontal scaling, at the cost of joins and, historically, transactional guarantees. Key-value stores are extremely fast when every access is by one key and useless otherwise. Wide-column stores handle enormous write volume by making you design the table around your query in advance. So the question I would actually ask is: what are the access patterns, does anything need to be atomic across records, and how much data is there? For most systems the answer is a relational database, and I would say that plainly — Postgres also has a JSONB type that covers a lot of the document use case without giving up transactions.

“Isn't a database just a file underneath?” Yes, and that is the interesting part. Postgres writes to ordinary files on an ordinary filesystem. The difference is entirely in what it does between your `INSERT` and those bytes landing: a write-ahead log forced to disk before the commit is acknowledged, a buffer pool so hot pages are not re-read, a lock manager and MVCC so concurrent transactions do not corrupt each other, B-tree indexes maintained alongside the data, a query planner deciding how to satisfy your request, and constraint checking. So “just use a file” is really “reimplement those six things”, and the

honest framing is that you are not choosing between a database and no database — you are choosing between a mature one and one you are about to write badly, on the most dangerous path in your system.

“You can, and for some things you should — but a database gives you four things a file does not, and the failures you get without them are all silent, which is what makes them dangerous.

The one people underestimate is **concurrency**. With a file, two processes that read it, change it and write it back will silently lose one of the changes: both read the same 400 records, both write back 401, and whichever writes second wins completely. There’s no error and nothing in the log. At a hundred writes a second with a 50-millisecond read-modify-write cycle, that’s about five lost updates a second — over a working day, more than a hundred thousand silently discarded changes.

Second, **durability**. Opening a file for writing truncates it to zero first, so a crash partway through doesn’t lose the last change — it loses the entire file, and what’s left isn’t even valid JSON. A database writes a description of the change to a write-ahead log and forces *that* to disk before reporting success, so a crash replays cleanly on restart. That same log is also what gives you replication and point-in-time recovery.

Third, **lookup**. Finding one record in a file means reading and parsing all of it. Ten million records at 200 bytes is 2 GB — about four seconds to read and twenty to parse, on every query, and six to ten gigabytes of RAM once it’s parsed into objects. A B-tree index turns that into three or four page reads, well under a millisecond. That’s roughly sixty thousand times.

Fourth, **integrity**. A file stores whatever you give it. A database refuses a missing phone number, a duplicate, a negative balance or the thirty-first of February, and the rule lives in one place so no application can break it — including the script somebody wrote in a hurry.

And underneath two of those, **transactions**. Moving money is two changes, and either both happen or neither should. You cannot get that right with a file without writing the log-and-replay machinery yourself, at which point you have written a database.

But I’d say the other side too, because sometimes a file is right. Configuration read at start-up belongs in a file — it goes in version control and needs no running service. Static assets belong in object storage with only the metadata in the database. Append-only logs work as files because appending is the one nearly-safe file operation. The test is the number of writers: one writer, no queries, no invariants, and a

file is genuinely better. And there's a middle option people forget — SQLite is a real transactional database in a single file with no server to operate, which covers a lot of the ground between the two.”

9 Recall card

- **Four things a database gives you:** durability, concurrency, fast lookup, integrity — delivered by **transactions**.
- **The failures a file gives you are silent.** Lost updates on concurrent writes; a truncated file on a crash mid-write; a full scan that is fine until it is not.
- **Write-ahead log** = append the intent, `fsync`, then apply. That is durability, and it is also replication.
- **An index turns a 2 GB scan into 3-4 page reads** — roughly 24 seconds down to 0.4 ms.
- **Files are right for configuration, static assets and single-writer logs.** And SQLite is a real database in one file — the answer nobody mentions.

DSA topic: Pattern matching, the simple way **System design topic:** What a database gives you that a file does not

Code these, in this order

Four problems where a short pattern is checked against a long one. The code is never more than a dozen lines; the marks are in the boundaries and in what you can say about the cost.

Before each one, ask:

1. What are `n` and `m`, and what is the last valid starting position?
2. Does the bound test come before the character comparison in my `while`?
3. What does the empty pattern do?
4. What input would make this hit its worst case?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Find the Index of the First Occurrence in a String	LeetCode 28 (Easy)	<code>range(n - m + 1)</code> , resetting <code>k</code> , and whether you can state both costs with an example input for each.
2	Repeated Substring Pattern	LeetCode 459 (Easy)	Whether you spot the <code>(s + s)[1:-1].find(s) != -1</code> trick — and can explain <i>why</i> it works, which is the actual question.
3	Longest Common Prefix	LeetCode 14 (Easy)	Comparing many strings position by position, and stopping at the first disagreement. The empty-list case.
4	Implement strStr with KMP	LeetCode 28 again	Only after the naive version passes. Build the failure table by hand on <code>"aabaaac"</code> before writing any code.

On problem 1, measure it rather than guessing

Add a counter to your solution and run it on these two inputs:

```
count_comparisons("a" * 1000 + "b", "a" * 10 + "b")
count_comparisons("the quick brown fox jumps over the lazy dog" * 10, "lazy dog")
```

You should get roughly `(990, 10901)` and `(35, 43)`. Then answer:

1. What is `n × m` for the first one, and how close did you get to it?
2. Why is the second one barely more than `n`?

3. Which of those two inputs would you show an interviewer, and why both?

On problem 1, break it three ways

Run each and say what happens and why:

```
# A – bound test after the comparison
while haystack[start + k] == needle[k] and k < m:
    k += 1
```

```
# B – k declared outside the outer loop
k = 0
for start in range(n - m + 1):
    while k < m and haystack[start + k] == needle[k]:
        k += 1
```

```
# C – wrong outer bound, with a slice comparison
for start in range(n):
    if haystack[start:start + m] == needle:
        return start
```

A raises. B gives confident wrong answers. C silently misses a match at the very end — construct the input that proves it.

On problem 2, explain before you accept

`(s + s)[1:-1].find(s) != -1` decides whether `s` is a whole number of repeats of some shorter string. Do not use it until you can say why it works, in two sentences, out loud.

Hint to work from: if `s` is `k` copies of a block, then `s + s` is `2k` copies, and `s` must appear starting somewhere in the middle. The `[1:-1]` exists to stop it matching at position 0.

Then write the honest version — try every divisor length of `n`, check whether repeating that prefix rebuilds `s` — and say which you would present first in an interview and why.

The timing drill

```
import time
haystack = "a" * 200_000 + "b"
needle = "a" * 1_000 + "b"

start = time.perf_counter(); str_str(haystack, needle); naive = time.perf_counter() - start
start = time.perf_counter(); haystack.find(needle); lib = time.perf_counter() - start
print(naive, lib, naive / lib)
```

Expect roughly 14 seconds against 0.0006 seconds. Then answer:

1. How much of that gap is a better algorithm and how much is C versus Python? Argue it.
2. What would KMP do on this input, in comparisons?
3. What is the right sentence to say in an interview about `find`?

The KMP table drill

Before coding anything, build the failure table for `"aabaaac"` by hand. For each prefix, the value is the length of the longest proper prefix of that prefix which is also a suffix of it.

prefix:	a	aa	aab	aaba	aabaa	aabaaa	aabaaac
value:	?	?	?	?	?	?	?

Then say, in one sentence, how that table is used: when a mismatch happens after matching `j` characters, what does the algorithm do instead of restarting?

If you can do the table and say that sentence, you can answer the follow-up in an interview without writing KMP at all — which is the point.

The naming drill

For each situation, name the algorithm and say why in one sentence:

1. One needle, one haystack, ordinary English text.
2. One needle, one haystack, both highly repetitive (DNA).
3. A thousand different needles, one haystack, all the same length.
4. A thousand needles of different lengths, one haystack, streaming.
5. A very long needle in a very long haystack, large alphabet.

Numbers 3 and 4 have different answers, and the difference is worth knowing.

The database drill

Answer each in one or two sentences, out loud:

1. Name the four things a database gives you that a file does not.
2. Two processes read a JSON file, each add a record, each write it back. What happens, and what is it called?
3. Why does a crash halfway through writing a file lose *everything*, not just the last change?
4. What is a write-ahead log, and what two other features does it also enable?
5. Ten million records at 200 bytes — how long to find one by scanning, and how long with an index?

6. Give three constraints a database enforces that a file cannot.
7. Name three situations where a file is genuinely the better choice.
8. What is SQLite, and why is it the answer nobody gives?
9. What does an index cost you?

Number 8 is the one that makes you sound like you have built things.

The arithmetic drill

From memory, in under two minutes:

- 10M records at 200 bytes — total size, seconds to read from SSD, seconds to parse as JSON.
- The same lookup through a B-tree with a fanout of 500 — how many levels, how many page reads?
- 2 GB of JSON parsed into objects — how much RAM, roughly, and why the multiplier?
- 100 writes a second with a 50 ms read-modify-write cycle — how many lost updates per second, and per eight-hour day?
- Random 8 KB writes versus sequential writes on SSD — the two throughput figures, and why the WAL exploits the gap.

The “why not a file” drill

For each, say **file** or **database**, and give the deciding reason in one sentence:

1. Application configuration read at start-up.
2. A user’s uploaded profile photograph.
3. Which users are currently logged in.
4. A list of 400 customers, edited by two people.
5. Server access logs.
6. Product catalogue for a shop with 50,000 items and a search box.
7. Local state for a command-line tool.
8. A record of every money transfer.

At least three of those are files. If you said “database” to all eight, re-read §7 of the lesson.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Implement strStr: find the first occurrence of a needle in a haystack.* Contract questions first. Then the two loops, saying the `n - m + 1` bound and the short-circuit ordering as you write them. Then both costs with an input for each. Then `find` exists, and then KMP in one sentence.
 2. *Why not just store the data in a JSON file?* Four things, concurrency first because it is the one people underestimate. Give the lost-update sequence, the truncation problem, the 24-seconds-versus-0.4-milliseconds number, and one integrity example. Then give the other side unprompted — configuration, static assets, append-only logs, and SQLite.
 3. *Can you do better than $O(n \cdot m)$?* KMP, and the reason: the naive version discards what it just learned, but it knows which prefix of the needle matched, so a precomputed table lets it slide forward without moving the haystack index backwards. $O(n + m)$ time, $O(m)$ space. Then say you would describe it rather than improvise it.
-

Before you move on

- [] I write `range(n - m + 1)` and can derive it on a small example.
- [] I put the bound test first in the `while` and can say why.
- [] I reset the inner index at every starting position, and know that is what KMP avoids.
- [] I can give both costs with a concrete input for each, not just the worst case.
- [] I can name KMP, Rabin-Karp and Boyer-Moore, with one sentence each.
- [] I do not attempt KMP from memory unless I have practised the table.
- [] I can name the four things a database gives me, leading with concurrency.
- [] I can describe a lost update as a sequence of four steps with no error message.
- [] I can name three cases where a file is right, and I remember SQLite exists.

Strings revision and mock round

DATA STRUCTURES AND ALGORITHMS

Strings revision and mock round

You can solve two unseen string problems under time pressure.

SYSTEM DESIGN

Tables, rows, and keys

You can design a small schema with primary and foreign keys and justify every column.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two string problems, no hints, talk as you go.*
- *Design the tables for a blog with users, posts and comments.*

1 What this is, and why they ask it

Days 19 to 25 built the string toolkit: what a string is and why it is immutable, building one without the quadratic trap, frequency maps, anagrams and canonical forms, palindromes and the two-ends habit, substrings versus subsequences, and naive pattern matching. Today is the second mock round, and it is about a different skill from [day 018](#).

Day 18 was about **process**: the six beats, narrating, testing with your own edge cases. That still applies and you should still do it. Today adds the thing that actually decides string questions: **recognition**. String problems are almost never new. They are one of about six patterns wearing different words, and the whole game is deciding which one in the first ninety seconds. Get that right and the code is fifteen lines you already know. Get it wrong and you spend twenty minutes writing something correct that answers a different question.

There are six patterns, and §3 lists them with their tells. Every string problem in the last seven days, and most of the ones you will be asked, is one of them.

2 The story

The workshop where Ramesh mends two-wheelers is under a tin roof at the end of a lane, and on a Monday there will be nine or ten machines waiting by eight in the morning.

What he has learnt in nineteen years is not really about engines. It is that people cannot tell him what is wrong, and they are not supposed to be able to, and it does not matter.

A man came in on Thursday and said the scooter was making a noise. Ramesh asked when, and the man said when he goes over the humps near the school. On Friday a woman said hers felt loose at the front. Last month a boy said his handlebar shook when he braked hard on the main road. Three completely different sentences, three people who have never met, and the same worn part underneath all three.

He does not have a list he goes through. What he has is that after nineteen years there are perhaps six things that actually go wrong on a scooter of that age, and every complaint he has ever heard is one of the six in different words. So he listens for about

twenty seconds — not to the words, to what is behind them — and by the time the man has finished talking Ramesh has usually decided, and the rest is checking whether he is right.

His nephew has been with him for two years now and is good with a spanner. He is faster than Ramesh at the actual work. But when a customer describes something he has not heard described in that way before, he stands there, and then he starts taking things off the machine to find out. Sometimes that works and it takes him an hour and a half.

Ramesh keeps telling him the same thing. It is not a new problem. You have fixed this exact thing eleven times. What you have not heard before is the sentence.

3 The idea in plain English

Ramesh’s six faults are today’s six patterns. The customer’s sentence changes every time; what is underneath does not.

The six string patterns, with their tells

#	PATTERN	THE TELL IN THE PROBLEM STATEMENT	COST	DAY
1	Count and compare	“how many times”, “most common”, “appears once”, “duplicate”	$O(n)$	021
2	Canonical form as a key	“are these the same apart from order”, “group these”	$O(n \cdot k)$	022
3	Two ends inwards	“palindrome”, “reverse”, “from both sides”	$O(n)$, $O(1)$	023
4	Sliding window	“substring”, “contiguous”, “longest ... without”, “at most k”	$O(n)$	032
5	Build with a list and join	the answer is a string you construct in a loop	$O(n)$	020
6	Two indices, one per string	comparing two strings position by position; “is A a subsequence of B”	$O(n + m)$	024

That is the whole list. Print it into your head, because **the first ninety seconds of every string question is deciding which row you are in.**

The three questions that pick the row

Ask these, in this order, and the pattern usually falls out:

1. **What shape is the answer?** A number → count or window. A boolean → compare or two ends. A string → build with a list. A list of groups → canonical form as a key.

2. **Contiguous or not?** From [day 024](#): contiguous means a window; gaps allowed means two indices or a table.
3. **One string or two?** One → window or two ends. Two → one index per string, or a canonical form for each.

The five facts that must be automatic

These come up in almost every string question and you should never have to think about them:

- **Strings are immutable.** `s[0] = "x"` is a `TypeError`. Building with `+=` in a loop is $O(n^2)$; collect into a list and `"".join`.
- **`len` is $O(1)$; slicing is $O(k)$ and copies.** So slicing inside a loop turns linear into quadratic, silently.
- **`sorted(s)` returns a list, not a string.** You need `"".join(sorted(s))` for a dictionary key, because a list is not hashable.
- **Every string method returns a new string.** `s.replace(...)` on its own does nothing.
- **`ord(ch) - ord("a")` is $O(1)$ space and silently wrong** on anything but lowercase a–z, because negative indices do not raise.

The four contract questions for any string problem

Forty seconds, and they catch most of the traps in the last seven days:

1. **Which characters count?** Letters only, alphanumeric, everything? (`isalnum` versus `isalpha` — `"0P"` from [day 023](#).)
2. **Does case matter?**
3. **What is the alphabet?** Lowercase English lets you use 26 slots; Unicode does not.
4. **What do I return for the empty string?** Almost always a defined value, almost never an error.

The two beats that matter most today

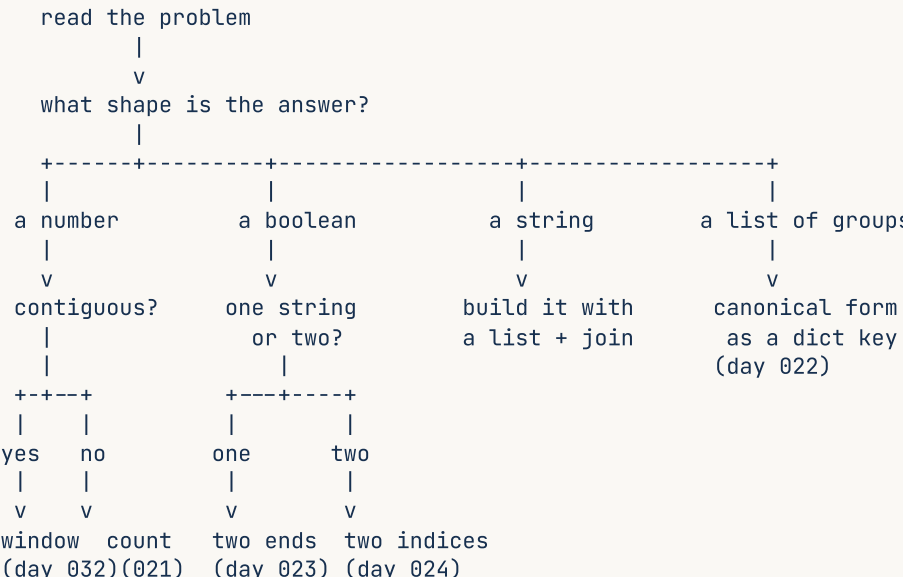
Day 18's six beats still hold — clarify, example, brute force, optimise, code, test. Two of them carry extra weight for string problems:

Beat 3, the brute force, is where you say the pattern out loud. Not “*I’d check every substring*”, but “*this is asking for a contiguous run, so it’s a sliding window, and the brute force would be to check every substring at $O(n^2)$* ”. Naming the family is the thing being scored.

Beat 6, testing, has a fixed list for strings. Empty string. One character. All the same character. Everything different. And, for anything with rules, one input that is only punctuation. Those five find almost every string bug you can write.

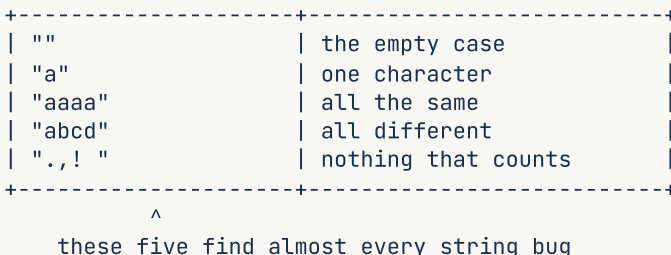
4

The decision, as a shape you can run in ninety seconds:



What to notice: four questions, six leaves, and every string problem from the last seven days lands on one of them. This is the picture to be able to redraw from memory.

The five test inputs, as a checklist you run every time:



What to notice: none of them is the example the interviewer gave you. That example is guaranteed to pass. Testing it proves nothing, and testing these proves quite a lot.

5 The code, built step by step

Two problems, worked as transcripts. Read them for the order things are said, not for the solutions.

Problem one — Isomorphic Strings

“Two strings are isomorphic if you can replace the characters of the first to get the second, with each character mapping to exactly one other character and no two characters mapping to the same one. `egg` and `add` are isomorphic. `foo` and `bar` are not.”

Beat 1, clarify. *“So it’s a one-to-one mapping in both directions? That is, can two different characters of `s` both map to the same character of `t` ?”* No — that is the whole trap. *“And are the strings guaranteed the same length?”*

Beat 2, example. *“`egg` → `add` : `e` goes to `a`, `g` goes to `d`, and the second `g` also goes to `d`, which is consistent. `foo` → `bar` : `o` would have to go to both `a` and `r`, so no.”*

Then produce the input that shows you understood the follow-up question you asked:

“And `badc` → `baba` is the interesting one. Every character of the first maps consistently — `b` → `b`, `a` → `a`, `d` → `b`, `c` → `a`. But `b` and `d` both map to `b`, so it fails the one-to-one requirement in the other direction.”

Producing that example unprompted is the whole question. It is the difference between a solution with one dictionary and a correct one.

Beat 3, brute force. *“There isn’t much of one here. The naive thing is to build the mapping and check consistency, which is already linear.”*

Beat 4, name the pattern. *“Shape of the answer is a boolean, and I’m comparing two strings position by position — so it’s the two-indices family, walking both together. I’ll need two dictionaries, one for each direction.”*

Beat 5, code.

```
if len(s) != len(t):
    return False
forward, backward = {}, {}
```

“Length check first, free. Two maps because the relationship has to hold both ways.”


```

for a, b in zip(s, t):
    if a in forward and forward[a] != b:
        return False
    if b in backward and backward[b] != a:
        return False
    forward[a] = b
    backward[b] = a
return True

```

“`zip` walks both strings together, which is cleaner than indexing. For each pair: if I have seen this character of `s` before and it mapped somewhere else, fail. If this character of `t` has already been claimed by a different character, fail. Otherwise record both directions.”

Beat 6, test. `("egg", "add")` True. `("foo", "bar")` False. `("paper", "title")` True. `("badc", "baba")` False — the one that needs the second map. `("", "")` True. `("ab", "aa")` False.

Problem two — String Compression

“Compress a string by replacing runs with the character and the count: `aabcccccaaa` becomes `a2b1c5a3`. If the compressed version is not shorter, return the original.”

Beat 1, clarify. “Runs of one become `a1`, or just `a`? And if the result is the same length as the original, do I return the original or the compressed one?” Say `a1`, and return the original on a tie.

Beat 2, example. “`aabcccccaaa` → `a2b1c5a3`, which is 8 against 11, so return the compressed one. `abcdef` → `a1b1c1d1e1f1`, which is 12 against 6, so return the original.”

Beat 3 and 4, name the pattern. “The answer is a string I build in a loop, so this is the build-with-a-list-and-join family. And it’s a grouping loop over consecutive characters, which means there’s a last group that ends by running out rather than by changing — I’ll need to handle that.”

Naming the flush before writing it is exactly what [day 020](#) drilled, and interviewers watch for it.

Beat 5, code. Using the two-index form, which avoids the flush entirely:

```

parts: list[str] = []
i = 0
while i < len(s):
    j = i
    while j < len(s) and s[j] == s[i]:
        j += 1

```

“`i` marks the start of a run and `j` walks to the end of it. When the inner loop stops, the run is `s[i:j]` and its length is `j - i`.”

```
parts.append(s[i])
parts.append(str(j - i))
i = j
```

“Emit the character and the count, then jump `i` to the start of the next run.”

This shape has no last-group problem, because the run is consumed and emitted inside one turn of the outer loop rather than being detected by a change. Say that out loud — it is a real reason to prefer this form.

```
out = "".join(parts)
return out if len(out) < len(s) else s
```

“Join once, and honour the contract about not being shorter.”

Beat 6, test. `"aabcccccaaa"` → `"a2b1c5a3"` . `"abcdef"` → `"abcdef"` . `"aabb"` → `"aabb"` (the compressed form is the same length, so the original wins). `""` → `""` . `"a"` → `"a"` .

The complete solutions

```
def is_isomorphic(s: str, t: str) → bool:
    """LeetCode 205. One-to-one mapping in BOTH directions - 'badc'/'baba' proves it."""
    if len(s) ≠ len(t):
        return False
    forward: dict[str, str] = {}
    backward: dict[str, str] = {}
    for a, b in zip(s, t):
        if a in forward and forward[a] ≠ b:
            return False
        if b in backward and backward[b] ≠ a:      # without this, badc/baba passes
            return False
        forward[a] = b
        backward[b] = a
    return True

def compress(s: str) → str:
    """'aabcccccaaa' → 'a2b1c5a3'. Returns the original if compressing does not shorten it.

    Two indices: i starts a run, j walks to its end. No last-group flush needed.
    """
    parts: list[str] = []
    i = 0
    while i < len(s):
        j = i
        while j < len(s) and s[j] == s[i]:
            j += 1
        parts.append(s[i])
        parts.append(str(j - i))
        i = j
    out = "".join(parts)
    return out if len(out) < len(s) else s

if __name__ == "__main__":
    print([is_isomorphic(a, b) for a, b in
          (("egg", "add"), ("foo", "bar"), ("paper", "title"),
           ("badc", "baba"), ("", ""), ("ab", "aa"))])
    # [True, False, True, False, True, False]

    print([compress(x) for x in ("aabcccccaaa", "abcdef", "aabb", "", "a")])
    # ['a2b1c5a3', 'abcdef', 'aabb', '', 'a']
```

6 What it costs

`is_isomorphic`

`zip(s, t)` produces `n` pairs, where `n` is the common length. Each turn does at most two dictionary lookups and two assignments, all $O(1)$ on average. So **$O(n)$ time**.

Space: two dictionaries, each holding at most one entry per distinct character. For lowercase English that is at most 26 each, so **$O(1)$ space** for a bounded alphabet, $O(k)$ in general.

Count the comparisons rather than naming a class: `n` turns $\times$ 4 dictionary operations = `4n` operations, which is linear.

compress

The outer `while` and the inner `while` between them advance through the string exactly once — every character is consumed by exactly one run, and `i` jumps straight to `j`. So **$O(n)$ time**, despite the nested loop. That is the same argument as [day 023](#): count how far the indices travel in total, not how many loops there are.

Space: `parts` holds at most $2n$ pieces in the worst case (every character distinct, so a character and a `"1"` each), and the joined result is at most $2n$ characters. **$O(n)$ space**.

The in-place version from [day 020](#), which mutates a `list[str]` and returns a length, is **$O(1)$ extra space** — and it only works because the problem hands you a list rather than a string.

The pattern costs, as a table to have memorised

PATTERN	TIME	SPACE
count and compare	$O(n)$	$O(k)$, $O(1)$ for a bounded alphabet
canonical form: sorting	$O(n \cdot k \log k)$	$O(n \cdot k)$
canonical form: counting	$O(n \cdot k)$	$O(n \cdot k)$
two ends inwards	$O(n)$	$O(1)$
sliding window	$O(n)$	$O(k)$
build with <code>+</code> <code>join</code>	$O(n)$	$O(n)$
build with <code>+=</code> in a loop	$O(n^2)$	$O(n)$
naive pattern matching	$O(n \cdot m)$ worst, $\sim O(n)$ typical	$O(1)$
longest common sub-sequence	$O(m \cdot n)$	$O(m \cdot n)$, $O(n)$ rolled

The only $O(n^2)$ in that table that you can accidentally write is the `+=` one, and it is the one that most often goes unnoticed, because the code is correct.

The numbers to have ready

Building a 100,000-character string with `+=` is about 5 billion character copies; with a list and `join` it is about 300,000 operations. A string of length `n` has $n(n+1)/2$ substrings and 2^n subsequences — 210 against a million at `n = 20`. Naive substring search took 43 comparisons on English text and 10,901 on an adversarial input of the same size.

7 The traps

The near-miss: one map instead of two

```
def is_isomorphic(s, t):
    mapping = {}
    for a, b in zip(s, t):
        if a in mapping and mapping[a] != b:
            return False
        mapping[a] = b
    return True

print(is_isomorphic("badc", "baba"))
```

True

Wrong. Every character of `s` maps consistently, but `b` and `d` both map to `b`, so the mapping is not one-to-one. **A single dictionary only checks that the mapping is a function; it does not check that it is injective.** The second dictionary is what enforces the other direction, and `"badc" / "baba"` is the input that finds it. This one passes `"egg" / "add"`, `"foo" / "bar"` and `"paper" / "title"` — every example the problem gives you.

The real error: mutating a string

```
s = "hello"
s[0] = "H"
```

TypeError: 'str' object does not support item assignment

Seven days on and it is still the most common string error there is. `list(s)`, mutate, `"".join`.

The near-miss: the accidental quadratic

```
def compress(s):
    out = ""
    i = 0
    while i < len(s):
        j = i
        while j < len(s) and s[j] == s[i]:
            j += 1
        out += s[i] + str(j - i)      # here
        i = j
    return out
```

Correct output, wrong complexity. The algorithm is $O(n)$ and the string building makes it $O(n^2)$. There is no error and no wrong answer — it simply gets slow, and only on inputs bigger than the ones you tested. Whenever you see `+=` on a string inside any loop, that is the bug.

The near-miss: slicing inside a loop

```
for i in range(len(s)):
    if s[i:] .startswith(needle):    # s[i:] copies the rest of the string, every turn
        return i
```

`s[i:]` is $O(n - i)$, so the loop is $O(n^2)$ even though it looks linear. Use `s.startswith(needle, i)`, which takes a start position and copies nothing, or compare with indices. **Slicing is the silent quadratic of string code**, and it is much harder to spot than `+=`.

The near-miss: testing only the given example

The example in the problem statement is guaranteed to pass, because you built the solution around it. It is the least informative test you have. Run the five from §4 instead: `""`, `"a"`, `"aaaa"`, `"abcd"`, and one input containing nothing that counts.

The process trap: pattern-matching too fast

Recognition is today's skill and it has a failure mode. A problem that *sounds* like a window and is actually a subsequence question costs you the whole round. **Recognition tells you which family to consider; the contract questions confirm it.** Say the family out loud and then verify it against the statement — *"this says contiguous, so a window is right"* — rather than jumping straight into the code you remember.

8 In the interview

How it gets asked

- *"We'll do two problems. Talk me through your thinking."* — the standard opening.
- *"Here's a string problem"* — where the first ninety seconds of pattern recognition decide the next thirty minutes.
- *"Can you do it in $O(1)$ space?"* — the follow-up that means "stop building a copy", and usually points at two pointers or a write pointer.
- *"How would you test this?"* — a gift, if you have the five inputs ready.

What to say out loud, in the first ninety seconds

The same script for every string problem:

1. **Restate it in your own words.** If your restatement is wrong you find out now.
2. **Ask the four contract questions.** Which characters count, does case matter, what alphabet, what about the empty string.
3. **Name the family, explicitly.** *“The answer is a boolean and I’m comparing two strings position by position, so this is the two-indices family.”* This is the sentence today exists to install.
4. **Confirm it against the statement.** *“It says contiguous, so a window rather than a sub-sequence table.”*
5. **Give the brute force with its cost,** then the better approach with its cost, then pause for agreement.
6. **Name the trap you are about to avoid,** before you write it — the second map, the flush, the `join` instead of `+=`.
7. **Test with your five,** not with their example.

The follow-ups

“How did you know which technique to use?” A genuine question, and worth answering properly rather than saying “experience”. I ask three things. What shape is the answer — a number points at counting or a window, a boolean at comparison, a string at building with a list, a list of groups at a canonical form used as a key. Is it contiguous — that word decides between a sliding window and a table, and if it is not in the statement I ask. And is it one string or two — one suggests a window or two ends, two suggests one index per string. Those three questions land on one of about six patterns, and every string problem I have seen is one of them wearing different words.

“Do it in $O(1)$ space.” For string problems that almost always means one of two things. Either stop building a copy — replace `cleaned = "".join(...)` and a reverse with two indices walking inwards, which is the palindrome answer. Or, if I have been handed a `list[str]` rather than a `str`, use a write pointer and mutate in place, returning a length. The signature tells me which: `s: str` means build and return, `chars: list[str]` means mutate and count. The one thing I cannot do is mutate a Python string, because it is immutable — so if the input is a string and you want constant space, the answer has to be an algorithm that never materialises anything, not a clever way to edit it.

“Your solution passes all the examples. Convince me it’s correct.” I would not argue from the examples, because I built it around them. I would state the property the code maintains and then attack it. For isomorphic strings, the property is that after processing `k` pairs, `forward` and `backward` describe a bijection consistent with the first `k` characters — and the two checks before the assignments are exactly what preserve that. Then I would deliberately look for the input that breaks a *nearly* correct version, which

here is `badc / baba` : it passes every example in the problem statement and fails on one dictionary. Being able to produce the input that breaks the plausible-but-wrong solution is a much stronger correctness argument than any number of passing tests.

“What would you test?” Five inputs, and none of them is the example you gave me. The empty string, because it is the one that raises. A single character, because it exercises the loop-runs-once path. All-the-same, because it maximises runs and windows. All-different, because it minimises them. And one input containing nothing that counts — pure punctuation — because that is what walks an index off the end in any solution with skip loops. For a problem with two strings I would add unequal lengths, and one where the answer is at the very last position, since off-by-one errors in the loop bound hide there.

A model answer

Written out for problem one.

“Let me restate it: I need to decide whether there is a one-to-one character mapping that turns `s` into `t`. Two questions — is it one-to-one in both directions, meaning two different characters of `s` cannot both map to the same character of `t`? And are the strings the same length?

...Both ways, and lengths may differ. Good, that second point matters and I'll guard for it.

On `egg` and `add`: `e` maps to `a`, `g` maps to `d`, and the second `g` maps to `d` again, which is consistent. On `foo` and `bar`, `o` would have to map to both `a` and `r`, so no.

The interesting case is `badc` and `baba`. Every character of the first maps consistently — `b` → `b`, `a` → `a`, `d` → `b`, `c` → `a` — but `b` and `d` both end up at `b`, so it fails in the other direction. I mention it because a solution with one dictionary passes all three of the standard examples and fails on that one.

Pattern-wise: the answer is a boolean and I am walking two strings position by position, so this is the two-indices family, and because the relationship has to hold both ways I need two maps.

```
def is_isomorphic(s: str, t: str) → bool:
    if len(s) ≠ len(t):
        return False
    forward, backward = {}, {}
    for a, b in zip(s, t):
        if a in forward and forward[a] ≠ b:
            return False
        if b in backward and backward[b] ≠ a:
            return False
        forward[a] = b
        backward[b] = a
    return True
```

`zip` walks both together, which reads better than indexing and stops at the shorter one — though the length check already ruled that out. For each pair: if I have seen this character of `s` mapping somewhere else, fail; if this character of `t` has already been claimed by a different source, fail; otherwise record both directions.

That is $O(n)$ time — n pairs, four constant-time dictionary operations each — and $O(k)$ space where k is the alphabet size, so $O(1)$ for lowercase English.

For tests: the three examples, then `badc` / `baba` for the injectivity check, then the empty string, a single character, and unequal lengths.”

9 Recall card

- **Six patterns:** count · canonical key · two ends · sliding window · list-and-join · two indices. Decide which in the first ninety seconds.
- **Three questions that pick it:** what shape is the answer, is it contiguous, one string or two?
- **Four contract questions:** which characters count, does case matter, what alphabet, what about empty?
- **Five test inputs, never the given example:** `""`, `"a"`, `"aaaa"`, `"abcd"`, and one with nothing that counts.
- **The two silent quadratics:** `+=` on a string in a loop, and slicing inside a loop. Both are correct and both are slow.

1 What this is, and why they ask it

A **relational database** stores data in **tables**. A table is a fixed set of named **columns**, each with a type, and any number of **rows**, each holding one value per column. That is the whole model, and it has survived fifty years because two small ideas make it enough for almost anything:

- Every row has a **primary key** — a value that identifies it uniquely, forever.
- A row in one table refers to a row in another by storing that key. That stored reference is a **foreign key**, and the database refuses to let it point at nothing.

From those two, everything else follows: joins, integrity, and the rule that each fact is stored in exactly one place.

Interviewers ask you to design a small schema constantly, because it is fast and it is unfaked. In ten minutes it shows whether you can identify the entities, choose keys, model a relationship, pick types that mean something, and say why each column exists. Candidates who have only used an ORM often produce something that works and cannot explain why `user_id` is on the `posts` table rather than a list of post ids on the `users` table — and that one question separates people cleanly.

Yesterday was *why* a database. Today is *how the data is shaped inside it*, and the next fifteen days are all built on this vocabulary.

2 The story

Muthu has run the office of a school in Salem for twenty-two years, and the thing he is quietly proudest of is the numbering.

Every child who joins gets a number, and that number is theirs until they leave. Not the class roll number, which changes every June when the class is reordered — a school number, issued once, in sequence, and never given to anyone else even after the child has left. 4417 was a boy who finished in 2019 and 4417 is still nobody else.

The number means nothing. It is not the year, it is not the class, it is not their initials. Muthu is firm about this, and the reason is a mistake somebody made before him. They used to use initials and a year, which was fine until two boys called Senthil Kumar joined the same June, and then it was not fine, and it took most of a term to untangle.

The child’s details — full name, date of birth, father’s name, address, the phone number that actually gets answered — are written in one place and one place only. Everything else in that office refers to them by number. The attendance record has numbers. The fee register has numbers. The marks for the half-yearly have numbers.

Which means that when a family moves house, Muthu changes the address in exactly one place, and every other record in the building is instantly correct, because none of them ever held the address in the first place.

There is one more rule and he enforces it absolutely. You cannot put marks against a number that is not in the main register. New teachers try — a child turns up mid-term and somebody wants to record a test — and Muthu makes them enter the child properly first. It takes four minutes and they are always slightly annoyed. He says the alternative is a mark that belongs to nobody, and once you have three of those in a register you can never trust the register again.

3 The idea in plain English

Muthu’s school number is a **primary key**. The numbers written on the fee register are **foreign keys**. His refusal to record marks for an unknown number is **referential integrity**, and the address living in exactly one place is **normalisation**.

Tables, rows, columns

A **table** holds one kind of thing. A **row** is one of those things. A **column** is one attribute of it, with a **type** that constrains what can go there.

users

id	email	display_name	created_at
BIGINT	TEXT	TEXT	TIMESTAMPZ
4417	muthu@school.in	Muthu	2026-03-04 09:12:00
4418	asha@school.in	Asha	2026-03-05 11:40:00

^

primary key – unique, never null, never reused

Rows have no order. A table is a set, and if you want them in an order you say so with `ORDER BY`. Relying on “the order they came back” is a bug that works until the day the database changes its plan.

The primary key

One column (or a few) that identifies a row uniquely. Three properties, and they are worth stating as requirements rather than conventions:

- **Unique.** No two rows share it.
- **Never null.** Every row has one.
- **Never changes.** Other tables point at it, so changing it would orphan them.

There are two kinds, and choosing is a real decision:

A **natural key** is something already meaningful — an email address, a vehicle registration, an ISBN. Tempting, and usually a mistake, because meaningful things change. People change email addresses. Two students really are called Senthil Kumar. A country really does renumber its vehicle plates.

A **surrogate key** is a meaningless number or identifier the database invents. 4417 is a surrogate key: it stands for the child and says nothing about them. Because it means nothing, nothing can happen in the world that makes it wrong.

Use a surrogate key. That is the default, and the reason to say out loud is exactly Muthu's: *a key that means something can stop being true, and then every table that pointed at it is wrong.*

In Postgres:

```
id BIGSERIAL PRIMARY KEY      -- an auto-incrementing 64-bit integer
id UUID PRIMARY KEY DEFAULT gen_random_uuid() -- a random 128-bit identifier
```

§7 has the trade-off between those two. It comes up in interviews.

The foreign key

A column holding the primary key of a row in another table.

```
posts
+-----+-----+-----+-----+
| id    | author_id | title                | published_at          |
+-----+-----+-----+-----+
| 91    | 4417      | Notes on the fee register | 2026-05-02 08:00:00 |
| 92    | 4418      | The new timetable       | NULL                 |
+-----+-----+-----+-----+
          ^
      foreign key → users.id
```

Declaring it as a foreign key does something a plain column does not: **the database refuses to store a value that does not exist in the other table**, and refuses to delete a user who still has posts (unless you tell it otherwise). That is referential integrity, and it

is Muthu refusing to record marks against an unknown number.

Without it you get **orphan rows** — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

Which side holds the key

This is the question that separates candidates, and it has a simple rule.

The foreign key goes on the “many” side. A user has many posts and a post has one author, so `author_id` lives on `posts`. It cannot live on `users`, because a column holds one value and a user has many posts — you would need a list in a column, which relational databases deliberately do not have.

The three relationship shapes:

SHAPE	EXAMPLE	WHERE THE KEY GOES
one-to-many	a user has many posts	on the many side: <code>posts.author_id</code>
many-to-many	a post has many tags, a tag has many posts	a third table : <code>post_tags(post_id, tag_id)</code>
one-to-one	a user has one profile	either side; usually the optional one

The many-to-many case is the one people forget. You cannot express it with a column on either side, so you create a **join table** whose rows are the pairs. Its primary key is usually the two columns together — a **composite key** — which also enforces that the same pair cannot appear twice.

One fact, one place

Muthu’s address rule. If the address is on the child’s record and *also* copied onto the fee register, then the day it changes there are two versions and one of them is wrong. Storing each fact once is called **normalisation**, and it is [day 029](#); the one-sentence version to have now is:

Do not store a fact in two places. Store it once, and refer to it by key everywhere else.

The exception, which is real and deliberate: counts and totals that are expensive to compute are often copied — a `comment_count` on a post rather than counting rows every time. That is **denormalisation**, it is a considered trade for read-heavy workloads, and it needs a job to keep it honest. Doing it by accident is a bug; doing it on purpose with a reconciliation job is engineering.

`NULL` , which is not zero

`NULL` means *no value here*, and it is neither `0` nor `" "`. It has one property that surprises everyone: `NULL = NULL` is **not true**. It is unknown. So you write `WHERE published_at IS NULL` , never `= NULL` , and a `COUNT(column)` skips nulls while `COUNT(*)` does not.

Because nulls complicate everything, the discipline is: **mark every column `NOT NULL` unless you can say what its absence means.** `published_at` being null meaning “not published yet” is a good use. A nullable `email` on a users table usually means nobody thought about it.

Types are constraints

Choosing `TEXT` for everything works and throws away most of what the database is for.

USE	NOT
<code>TIMESTAMPZ</code> for a moment in time	<code>TEXT</code> — then <code>"31/02/2026"</code> is storable
<code>NUMERIC(12,2)</code> for money	<code>FLOAT</code> — <code>0.1 + 0.2</code> is not <code>0.3</code>
<code>BOOLEAN</code>	<code>TEXT</code> holding <code>"true"</code> , <code>"TRUE"</code> , <code>"1"</code> , <code>"yes"</code>
an <code>ENUM</code> or a lookup table for a status	free text, so <code>"shipped"</code> and <code>"Shipped"</code> both exist
<code>TEXT</code> for strings in Postgres	<code>VARCHAR(255)</code> , which is a MySQL habit with no benefit in Postgres

The money one is worth being emphatic about. **Never store money in a floating-point column.** Binary floating point cannot represent `0.1` exactly, so totals drift by fractions of a rupee and eventually somebody notices. Use `NUMERIC` / `DECIMAL` , or store integer paise.

4 The picture

The blog schema, as tables with the keys marked:

users			posts		
+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+
id email	<---+		id author_id title		
+-----+-----+			+-----+-----+	+-----+-----+	+-----+-----+
1 asha@example.com			91 1		Notes on fees
2 ravi@example.com			92 1		The new timetable
+-----+-----+	+-----+	+-----+	93 2		Sports day
^			^		
PRIMARY KEY			PRIMARY	FOREIGN KEY → users.id (on the MANY side)	

comments				post_tags (many-to-many)		
+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+
id post_id author_id body				post_id tag_id		
+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+
7 91	2	Useful.		91	3	
8 91	1	Thanks.		91	5	
+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	92	3	
				+-----+-----+		
v		v		PRIMARY KEY (post_id, tag_id)		
posts.id		users.id		- composite, and it also stops the same pair twice		

What to notice: every arrow points at an `id`, and no arrow points at a name or an email. That is the surrogate-key rule made visible. And `post_tags` has no `id` of its own — the pair is the identity.

The same thing as an entity diagram, which is what you would draw in an interview:

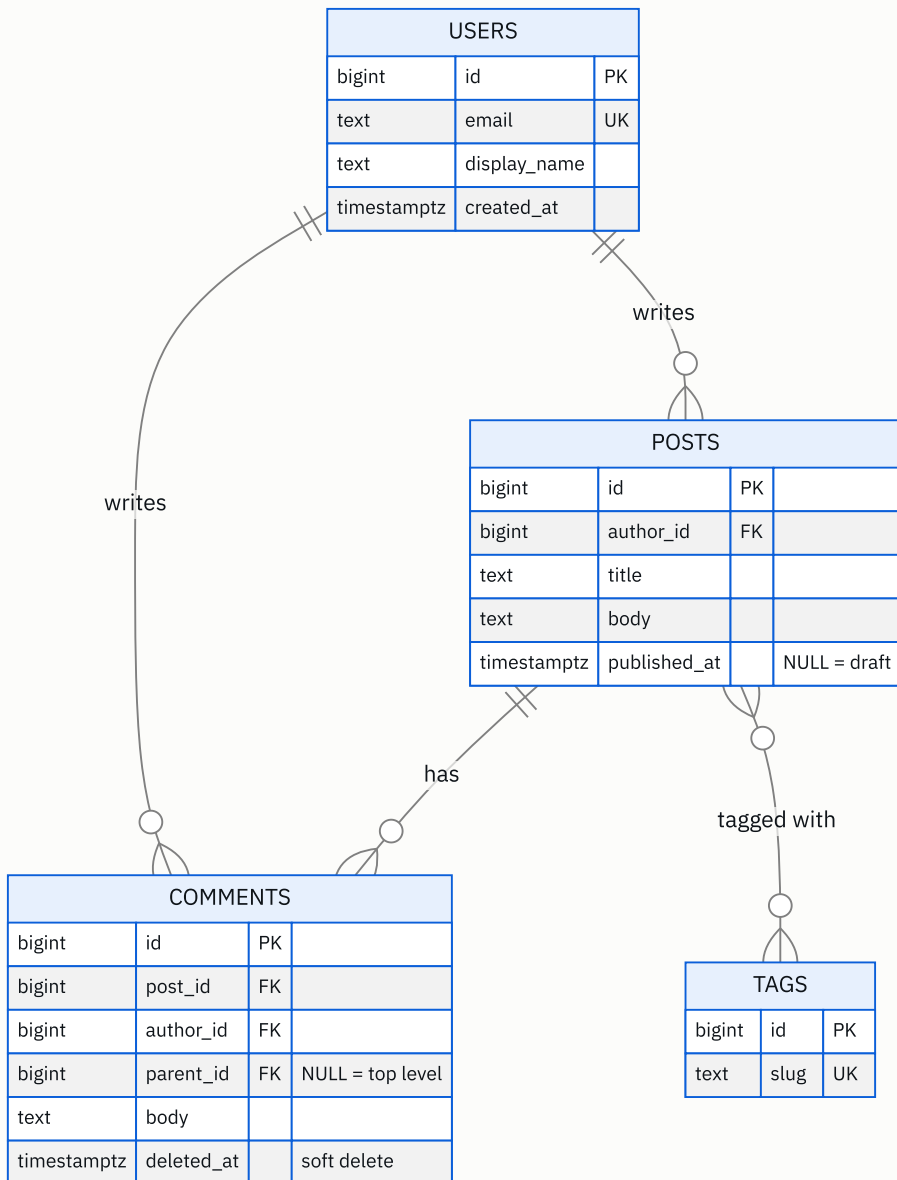


FIGURE 26.1

What to notice: the crow's-foot ends. `||-o{` means one-to-many, and the many end is always where the foreign key lives. `}o--o{` between posts and tags is many-to-many, which is why a join table has to exist even though the diagram does not show it. And `comments.parent_id` pointing back at `comments` is a **self-referencing** foreign key — a row referring to another row in the same table — which is how you model replies.

5 How it actually works

The schema, written out

```
CREATE TABLE users (  
  id          BIGSERIAL      PRIMARY KEY,  
  email       TEXT          NOT NULL UNIQUE,  
  display_name TEXT          NOT NULL,  
  created_at  TIMESTAMPTZ    NOT NULL DEFAULT now()  
);  
  
CREATE TABLE posts (  
  id          BIGSERIAL      PRIMARY KEY,  
  author_id   BIGINT         NOT NULL REFERENCES users(id) ON DELETE RESTRICT,  
  title       TEXT          NOT NULL,  
  body        TEXT          NOT NULL,  
  published_at TIMESTAMPTZ,      -- NULL means "still a draft"  
  created_at  TIMESTAMPTZ    NOT NULL DEFAULT now()  
);  
  
CREATE TABLE comments (  
  id          BIGSERIAL      PRIMARY KEY,  
  post_id     BIGINT         NOT NULL REFERENCES posts(id) ON DELETE CASCADE,  
  author_id   BIGINT         NOT NULL REFERENCES users(id) ON DELETE RESTRICT,  
  parent_id   BIGINT         REFERENCES comments(id) ON DELETE CASCADE,  
  body        TEXT          NOT NULL CHECK (length(body) > 0),  
  created_at  TIMESTAMPTZ    NOT NULL DEFAULT now(),  
  deleted_at  TIMESTAMPTZ  
);  
  
CREATE TABLE tags (  
  id          BIGSERIAL      PRIMARY KEY,  
  slug        TEXT          NOT NULL UNIQUE  
);  
  
CREATE TABLE post_tags (  
  post_id     BIGINT         NOT NULL REFERENCES posts(id) ON DELETE CASCADE,  
  tag_id      BIGINT         NOT NULL REFERENCES tags(id) ON DELETE CASCADE,  
  PRIMARY KEY (post_id, tag_id)      -- composite: the pair IS the identity  
);  
  
CREATE INDEX ON posts (author_id);      -- foreign keys are NOT auto-indexed  
CREATE INDEX ON comments (post_id, created_at);  -- the query this table exists to serve
```

Be ready to justify every line of that. The two lines at the bottom especially — see below.

ON DELETE , which is a design decision

When the referenced row is deleted, what happens to the rows pointing at it? Four options, and you must choose:

OPTION	EFFECT	USE FOR
<code>RESTRICT / NO ACTION</code>	refuse the delete	the default you want. Deleting a user who has posts should fail loudly.
<code>CASCADE</code>	delete the children too	genuinely owned children — a post’s comments die with the post.
<code>SET NULL</code>	blank the reference	optional relationships, e.g. <code>assigned_to</code> on a ticket.
<code>SET DEFAULT</code>	point at a fallback row	rare; an “unknown user” placeholder.

Be deliberate. `CASCADE` on the wrong relationship deletes half your data from one statement, and in practice most user deletions should be a soft delete or an anonymisation rather than a real `DELETE` at all.

Foreign keys are not automatically indexed

This surprises almost everyone and it is worth knowing: **Postgres indexes the primary key automatically and does not index foreign keys.** So `SELECT * FROM posts WHERE author_id = 5` does a full scan until you add the index yourself. And deleting a user has to check every post for a reference, which without an index is also a full scan.

The rule of thumb: **index every foreign key**, unless you have measured that you never query in that direction.

Choosing the identifier type

	<code>BIGSERIAL</code> (AUTO-INCREMENT)	<code>UUID</code> (RANDOM)
size	8 bytes	16 bytes
generated by	the database, on insert	the application, before insert
ordering	sequential — index inserts land at the end	random — index inserts scatter
leaks information	yes: <code>/orders/1055</code> tells a competitor your volume	no
across shards / off-line	needs coordination	works anywhere, no coordination

The nuance worth knowing: random UUIDs are bad for B-tree indexes because every insert lands in a different page, which fragments the index and hurts cache locality. **ULID** and **UUIDv7** fix this by putting a timestamp in the high bits, so they are unique *and* roughly sequential. If asked “UUID or auto-increment”, the strong answer is: “`BIGSERIAL`

by default; UUIDv7 or ULID if I need ids generated outside the database or must not leak volume; plain random UUIDv4 only if I genuinely need unguessability, and I would accept the index cost.”

Soft delete

`deleted_at TIMESTAMPTZ` instead of removing the row. The comment stays so its replies still make sense, and the API returns a tombstone — [day 017](#)’s comments feature. The cost is that **every query must now remember** `WHERE deleted_at IS NULL`, and the day one query forgets, deleted content reappears. The usual mitigations are a database view or a partial index that filters them out, so the filter lives in one place rather than in every query.

What this looks like in real products

Every relational database works this way: **PostgreSQL, MySQL, SQLite, SQL Server, Oracle**, and the managed versions — **Amazon RDS, Aurora, Cloud SQL, PlanetScale**. The syntax varies slightly (`BIGSERIAL` in Postgres, `BIGINT AUTO_INCREMENT` in MySQL) and the model does not.

One warning worth carrying: **PlanetScale and Vitess do not support foreign key constraints**, because enforcing them across shards is expensive — so the integrity has to be enforced by the application. That is a real trade some companies make, and knowing it exists is a good signal.

6 The numbers

Row size and storage

A `posts` row: `id` 8 bytes, `author_id` 8, `title` about 60, `body` about 2,000, two timestamps 16, plus around 24 bytes of per-row overhead in Postgres.

```
per row ≈ 2,120 bytes
100,000 posts × 2,120 B ≈ 212 MB
```

A `comments` row is smaller — say 250 bytes with overhead:

```
10,000,000 comments × 250 B = 2.5 GB
plus indexes, roughly double ≈ 5 GB
```

Five gigabytes for ten million comments. That number is worth carrying, because it is the answer to “would you shard?” — no, and here is the arithmetic.

What a key costs

```
BIGSERIAL : 8 bytes per row + 8 bytes per index entry
UUID      : 16 bytes       + 16 bytes per index entry

10,000,000 comments, primary key + 2 foreign key indexes:
  bigint : 10M × 8 × 3 = 240 MB
  uuid   : 10M × 16 × 3 = 480 MB
```

240 MB of difference — not decisive on its own. The decisive part is the insert pattern: sequential ids append to the rightmost index page, which stays in cache, while random UUIDs touch a different page every time.

```
sequential inserts: ~1 page dirtied per insert
random UUID inserts: ~1 page dirtied per insert, but a DIFFERENT page each time
                    → the working set becomes the whole index rather than its last page
```

On a large table that is the difference between an index that fits in memory where it matters and one that does not, and it can be several times the write throughput.

The unindexed foreign key

```
posts table, 100,000 rows:
  SELECT * FROM posts WHERE author_id = 5
    no index : sequential scan of 212 MB ≈ 400 ms
    indexed  : B-tree lookup ≈ 0.5 ms
```

About 800 times, on a table that is not even large. And the same applies to `DELETE FROM users WHERE id = 5`, which must check `posts` for references.

Normalised versus denormalised counts

Counting comments on a post at read time:

```
SELECT COUNT(*) FROM comments WHERE post_id = 91
  with an index on post_id, 200 comments : ≈ 1 ms
  at 3,700 reads/second                  : 3.7 seconds of database time per second → ~4 cores
```

Storing `comment_count` on the post instead:

```
read cost : 0 (it is already in the row you fetched)
write cost: one extra UPDATE per comment, at ~3.5 writes/second → negligible
```

With the 265:1 read-to-write ratio from [day 017](#), that is an obvious trade — **and it is only obvious because of the ratio**, which is why you compute it first.

Identifier space

```
BIGINT signed max  $\approx 9.2 \times 10^{18}$   
at 1,000,000 inserts/second, that lasts  $\approx 292,000$  years  
INT signed max  $\approx 2.1 \times 10^9$   
at 1,000 inserts/second, that lasts  $\approx 24$  days
```

Use **BIGINT**, not **INT**, for anything that grows. Running out of 32-bit ids in production is a famous and entirely avoidable outage, and the fix on a live table with a billion rows is a migration measured in hours.

7 The trade-offs

Natural or surrogate key

A natural key needs no extra column and reads well — an ISBN really does identify a book. It also ties your primary key to a fact about the world, and facts change: email addresses change, ISBNs get reissued, and two people genuinely do share a name and a date of birth. Since every other table stores that key, a change means updating every reference. **Surrogate keys by default**, with a **UNIQUE** constraint on the natural key so you still get the uniqueness guarantee without the fragility.

CASCADE or **RESTRICT**

CASCADE is convenient and dangerous: one **DELETE** can remove far more than you intended, and there is no undo. **RESTRICT** forces you to think, and produces errors in code paths that assumed a delete would work. **Default to RESTRICT, and use CASCADE only where the child genuinely cannot exist without the parent** — a post's comments, a join-table row. For users, the honest answer is usually neither: anonymise rather than delete, because deleting a user with five years of content is almost never what the product wants.

Normalise or denormalise

Normalised data has one source of truth and cannot go inconsistent. Denormalised data is faster to read and can drift. **Normalise first, denormalise where a measured read-to-write ratio justifies it**, and always with a reconciliation job. The failure mode of premature denormalisation is not slowness; it is two numbers that disagree and no way to know which is right.

Soft delete or hard delete

Soft delete keeps history, makes deletion reversible, and keeps threads readable. It also means every query needs a filter, deleted data is still in your database when a legal erasure request arrives, and your tables grow forever. **Soft delete for user-visible content; a real purge path alongside it for compliance; and hard delete for things nobody will ever ask about again.**

Foreign keys on or off

Constraints cost a check on every insert and a lock on the referenced row, and some very high-throughput systems turn them off — Vitess and PlanetScale do not support them at all. What you give up is the guarantee, and the guarantee is what stops orphan rows accumulating silently. **Keep them on unless you have measured that they are the bottleneck**, because data you cannot trust is worth much less than data you get slightly slower.

The sentence that separates candidates

I would not use a natural key as the primary key, even when one exists and looks stable. Every other table stores that value, so the day the world changes — a customer changes email, a supplier gets reissued a code — I am updating references across the whole schema instead of changing one column. I would put a `UNIQUE` constraint on the natural key so I still get the guarantee, and keep a meaningless surrogate as the thing everything else points at. A key that means something can stop being true; a key that means nothing cannot.

8 In the interview

How it gets asked

- “*Design the tables for a blog with users, posts and comments.*” — the standard version, ten minutes.
- “*How would you model tags on posts?*” — the many-to-many question, checking whether you reach for a join table.
- “*UUID or auto-increment?*” — a specific trade-off with a real answer.
- “*Where does the foreign key go?*” — the one-line question that reveals whether you understand cardinality.
- “*What happens when a user is deleted?*” — the `ON DELETE` question, and the one where “it depends” is the correct opening.

What to say out loud, in the first ninety seconds

1. **List the entities before drawing anything.** *“Users, posts, comments, tags. And a join table for post-to-tag, because that relationship is many-to-many.”*
2. **State the relationships with their cardinality.** *“A user has many posts; a post has one author. A post has many comments. A comment may have a parent comment. Posts and tags are many-to-many.”*
3. **Say the key rule as a rule.** *“Surrogate primary keys everywhere — `BIGSERIAL` — because a key that means something can stop being true, and every other table points at it.”*
4. **Say where the foreign key goes and why.** *“The foreign key always goes on the many side, so `author_id` is on `posts`. It cannot go on `users`, because a column holds one value and a user has many posts.”*
5. **Mention `NOT NULL` deliberately.** *“Everything is `NOT NULL` unless I can say what its absence means — `published_at` being null means it is still a draft, and that is a real use.”*
6. **Say the types matter.** *“`TIMESTAMPZ` for times, `NUMERIC` for money, never `FLOAT`.”*
7. **Add the index line unprompted.** *“And I’d index every foreign key, because Postgres indexes primary keys automatically and foreign keys not at all.”*

The follow-ups

“Where does the foreign key go, and why not the other side?” On the many side, always. A post has exactly one author, so `posts.author_id` holds one value and that fits in a column. A user has many posts, so putting `post_ids` on `users` would need a list in a column — which relational databases deliberately do not offer, because you could not index it, could not constrain it, and could not join on it efficiently. The general rule is that the side which has *one* of the other thing holds the reference. For many-to-many, neither side works, so the relationship gets its own table whose rows are the pairs — and I would make the pair itself the primary key, which enforces that the same pair cannot be inserted twice.

“UUID or auto-increment?” Auto-increment — `BIGSERIAL` — by default. It is 8 bytes rather than 16, and more importantly it is sequential, so index inserts land on the right-most page which stays hot in cache, whereas random UUIDs touch a different page every insert and effectively make the whole index the working set. I would move to UUIDs when I need ids generated outside the database — by clients, or across shards without coordination — or when a sequential id would leak information, since `/orders/1055` tells a competitor roughly how many orders you have had. If I need that, I would use UUIDv7 or ULID rather than UUIDv4, because they put a timestamp in the high bits so they are unique *and* roughly ordered, which recovers most of the index locality. And whichever I choose, `BIGINT` not `INT` — a 32-bit id runs out at about 2.1 billion, which at a thousand inserts a second is 24 days.

“What happens when a user is deleted?” It depends, and that is the honest opening. Technically I choose per foreign key. `ON DELETE RESTRICT` refuses the delete while the user still has posts, which is my default because a silent cascade can remove enormous amounts of data from one statement. `ON DELETE CASCADE` is right where the child truly cannot exist without the parent — a post’s comments, a join-table row. `ON DELETE SET NULL` suits optional relationships. But for users specifically, the product answer is usually none of those: a user with five years of posts and comments should be anonymised rather than deleted, so the content survives with the identity removed. And if there is a legal erasure obligation, that is a separate, deliberate purge path rather than a foreign key setting.

“How do you model comment replies?” A `parent_id` column on `comments` that references `comments.id` — a self-referencing foreign key, nullable, where null means a top-level comment. That models arbitrary depth with one column. The difficulty is reading it: fetching a thread twelve deep means twelve queries unless you do something better, so in practice I would either cap the product at one level of replies, which is what Instagram and YouTube do and which makes every response bounded, or store a materialised path — the chain of ancestors as a string like `91/104/118` — so the whole subtree under a comment is one indexed prefix scan. Postgres has an `ltree` type for exactly that. I would start with one level, because it covers most of the product need and makes pagination possible at every point.

A model answer

“Entities first: users, posts, comments, tags. Posts to tags is many-to-many, so that relationship needs its own table.

Relationships and cardinality: a user writes many posts and a post has one author. A post has many comments; a comment has one post and one author. A comment may have a parent comment, for replies. Posts and tags are many-to-many.

```
CREATE TABLE users (  
  id          BIGSERIAL  PRIMARY KEY,  
  email       TEXT      NOT NULL UNIQUE,  
  display_name TEXT      NOT NULL,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE posts (  
  id          BIGSERIAL  PRIMARY KEY,  
  author_id   BIGINT     NOT NULL REFERENCES users(id) ON DELETE RESTRICT,  
  title       TEXT      NOT NULL,  
  body        TEXT      NOT NULL,  
  published_at TIMESTAMPTZ,          -- NULL means draft  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE comments (  
  id          BIGSERIAL  PRIMARY KEY,  
  post_id     BIGINT     NOT NULL REFERENCES posts(id) ON DELETE CASCADE,  
  author_id   BIGINT     NOT NULL REFERENCES users(id) ON DELETE RESTRICT,  
  parent_id   BIGINT     REFERENCES comments(id) ON DELETE CASCADE,  
  body        TEXT      NOT NULL CHECK (length(body) > 0),  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  deleted_at  TIMESTAMPTZ  
);  
  
CREATE TABLE post_tags (  
  post_id BIGINT NOT NULL REFERENCES posts(id) ON DELETE CASCADE,  
  tag_id  BIGINT NOT NULL REFERENCES tags(id)  ON DELETE CASCADE,  
  PRIMARY KEY (post_id, tag_id)  
);  
  
CREATE INDEX ON posts (author_id);  
CREATE INDEX ON comments (post_id, created_at);
```

The decisions I'd want to justify:

Surrogate keys everywhere. Email is unique and I've constrained it, but it is not the primary key, because every other table stores the primary key and email addresses change. A key that means something can stop being true.

`author_id` is on `posts`, not a list of post ids on `users`, because the foreign key goes on the many side — a column holds one value, and a user has many posts.

`post_tags` has no id of its own; the pair is the primary key. That is a composite key, and it also prevents the same tag being attached twice.

`ON DELETE CASCADE` on comments-to-post, because a comment genuinely cannot exist without its post. `RESTRICT` on the author references, because I do not want deleting a user to silently remove years of content — in practice I'd anonymise rather than delete.

Everything is `NOT NULL` except `published_at`, where null means draft, `parent_id`, where null means top-level, and `deleted_at`, where null means not deleted. Each of those absences means something specific.

And the two index lines at the bottom: Postgres indexes primary keys automatically and foreign keys not at all, so without them, listing a user's posts is a full table scan — about 400 milliseconds on a hundred thousand rows against half a millisecond indexed. The comments index is on `(post_id, created_at)` because that is the query the table exists to serve: the comments on one post, in order.

On scale: ten million comments at about 250 bytes is 2.5 gigabytes, roughly five with indexes. That is comfortably one Postgres instance, so I would not shard, and I would say so with the arithmetic rather than assuming."

9 Recall card

- **Table = one kind of thing; row = one of them; column = one attribute with a type.** Rows have no order.
- **Primary key: unique, never null, never changes.** Use a surrogate (`BIGSERIAL`), not a natural key — meaningful things change.
- **The foreign key goes on the “many” side.** Many-to-many needs a third table with a composite key.
- **`NOT NULL` unless absence means something.** `TIMESTAMPZ` for time, `NUMERIC` for money, never `FLOAT`.
- **Index every foreign key** — primary keys are indexed automatically, foreign keys are not. And choose `ON DELETE` deliberately.

DSA topic: Strings revision and mock round **System design topic:** Tables, rows, and keys

Code these, in this order

Four problems from four different families. **Before writing anything, say which of the six patterns it is and why.** If you cannot name the family in fifteen seconds, that is the thing to practise, not the code.

Run each as a timed round: twenty minutes, six beats, narrate out loud.

#	PROBLEM	SOURCE	FAMILY, AND WHAT IT IS REALLY TESTING
1	Isomorphic Strings	LeetCode 205 (Easy)	Two indices. Whether you find the second map — <code>"badc" / "baba"</code> passes every example in the statement with one dictionary.
2	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	Sliding window. The <code>last[ch] ≥ start</code> guard, which <code>"abba"</code> and <code>"dvdvf"</code> expose.
3	Group Anagrams	LeetCode 49 (Medium)	Canonical form as a key, and that a list is not hashable.
4	Longest Palindrome	LeetCode 409 (Easy)	Counting, with a genuine think at the end: how many odd-count characters can you keep, and why exactly one?

Before each problem, run the recognition script

Out loud, in under fifteen seconds:

1. What shape is the answer — a number, a boolean, a string, a list of groups?
2. Contiguous, or gaps allowed?
3. One string or two?
4. Therefore: which of the six families?

Then confirm it against the statement before writing a line. Getting this wrong costs the whole round; getting it right makes the code fifteen lines you already know.

On problem 1, produce the counterexample first

Before coding, find the input that breaks the one-dictionary version. Do not look it up — reason about what “one-to-one” means and construct it. When you have it, check that it passes with one map and fails with two.

Producing that input unprompted, in the interview, is worth more than the solution.

On problem 4, do the thinking part

"abccccdd" gives 7 — "dccaccd". Work out on your own, before coding:

1. For a character appearing `k` times, how many of them can go into a palindrome?
2. Why can exactly one character contribute an odd count, and no more?
3. What is the answer for "a", for "ab", and for ""?

Then write it. The code is four lines and the reasoning is the question.

The pattern-recognition drill

For each, name the family and the cost, in under five seconds each:

1. “Find the first character that appears exactly once.”
2. “Do these two strings contain the same letters?”
3. “Reverse the words in this sentence.”
4. “Longest substring with at most two distinct characters.”
5. “Is this a palindrome, ignoring punctuation?”
6. “Compress runs of repeated characters.”
7. “Is A a subsequence of B?”
8. “Find the first occurrence of one string inside another.”
9. “Group these words by which letters they contain.”
10. “Longest substring that appears in both strings.”

Number 10 is the trap: it says *substring*, so a mismatch resets to zero and the answer is the maximum cell — not the bottom-right one. Say what would go wrong if you wrote the subsequence version.

The five-inputs drill

Take any solution you wrote today and run it on all five without changing anything:

```
" "      # empty
"a"      # one character
"aaaa"   # all the same
"abcd"   # all different
".,! "   # nothing that counts
```

For each solution, say which of the five was most likely to break it, and why. Then check whether you were right.

The silent-quadratic drill

Both of these are correct and both are $O(n^2)$. Find the line, and rewrite each:

```
# A
def compress(s):
    out = ""
    i = 0
    while i < len(s):
        j = i
        while j < len(s) and s[j] == s[i]:
            j += 1
        out += s[i] + str(j - i)
        i = j
    return out
```

```
# B
def find_first(s, needle):
    for i in range(len(s)):
        if s[i:].startswith(needle):
            return i
    return -1
```

Then say the general rule for each: one is about immutability, the other about slicing. Both produce right answers and only get slow, which is why neither shows up in testing.

The mock-round drill

Have somebody pick two of these without telling you which, or pick at random. Twenty minutes each, timed, narrating.

Valid Anagram · Reverse Words in a String · First Unique Character · Valid Palindrome II · Longest Common Prefix · String Compression · Ransom Note · Implement strStr · Isomorphic Strings · Longest Palindromic Substring

Score yourself on process, not on finishing:

- ☐ I restated the problem in my own words.
- ☐ I asked the four contract questions.
- ☐ I named the family out loud before writing code.
- ☐ I gave a brute force and its cost.
- ☐ I paused for agreement before typing.
- ☐ I named the trap before I wrote the line that avoids it.
- ☐ I tested with my five inputs, not the given example.

Fewer than six ticks means run it again tomorrow with different problems.

The schema drill

Design the tables for **a library**: members, books, copies of books, loans, and reservations. Ten minutes.

The interesting part is that a *book* and a *copy of a book* are different things — the library owns four copies of the same title, and a loan is of a copy, not of a title. Get that right and the rest follows.

Then check your own schema:

1. Is every primary key a surrogate?
2. Is every foreign key on the many side?
3. Did you need a join table anywhere? Why or why not?
4. Which columns are nullable, and can you say what each null *means*?
5. What is your `ON DELETE` on every foreign key, and can you justify each?
6. Which foreign keys did you index?
7. What type did you use for the due date, and for a fine amount?

Number 7 has two specific right answers and several wrong ones.

The keys drill

Answer each in one or two sentences, out loud:

1. What three properties must a primary key have?
2. Why is an email address a bad primary key even though it is unique?
3. Where does a foreign key go in a one-to-many relationship, and why not the other side?
4. How do you model many-to-many, and what is the primary key of that table?
5. What is referential integrity, and what is an orphan row?
6. What does `NULL = NULL` evaluate to, and what do you write instead?
7. Are foreign keys indexed automatically? What is the consequence?
8. `BIGSERIAL` or `UUID` — pick one and defend it, then say when you would switch.
9. Name the four `ON DELETE` options and one use for each.
10. Why must money never be stored in a `FLOAT` column?

The arithmetic drill

From memory, in under two minutes:

- 10 million comments at 250 bytes — storage, and with indexes. Do you shard?

- `BIGINT` versus `UUID` for a primary key and two foreign key indexes on 10 million rows — the two index sizes.
 - A 100,000-row table, `WHERE author_id = 5`, with and without an index — the two times.
 - 32-bit `INT` ids at 1,000 inserts a second — how long until you run out?
 - `COUNT(*)` per read at 3,700 reads a second and 1 ms each — cores, and what you would do instead.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two string problems, no hints, talk as you go.* The scoring is process, not completion. Restate, four contract questions, name the family, brute force with cost, pause, code with narration, test with your own five.
 2. *Design the tables for a blog with users, posts and comments.* Entities first, then cardinality, then the schema. Say the surrogate-key rule as a rule. Say where the foreign key goes and why not the other side. Add the index line unprompted. Justify every nullable column by saying what its null means.
 3. *How do you decide which technique a string problem needs?* The three questions — shape of the answer, contiguous or not, one string or two — landing on the six families. Then the honest caveat: recognition narrows it, the contract questions confirm it, and jumping straight to remembered code is how you solve the wrong problem confidently.
-

Before you move on

- [] I can name the six string patterns and one tell for each.
- [] I run the three recognition questions before writing any string code.
- [] I ask the four contract questions every time.
- [] I test with `""`, `"a"`, `"aaaa"`, `"abcd"` and a punctuation-only input — never the given example.
- [] I never write `+=` on a string in a loop, and I never slice inside one.
- [] I can name the three properties of a primary key and say why email is a bad one.
- [] I know the foreign key goes on the many side, and can say why the other side is impossible.
- [] I index every foreign key and know that the database does not do it for me.

- [] I can redraw the pattern-decision tree and the blog schema from memory, in whatever tool I like.

The nine sections, and what each is for

Every lesson in this book, on both tracks, carries the same nine sections in the same order. This is what each one is for, and how to read it.

1. What this is, and why they ask it

The topic in plain words, and the reason an interviewer spends time on it. This section exists so you never learn something without knowing what it buys you.

2. The story

A scene with a person in it, told in ordinary language, with no technical words at all. Two hundred to four hundred words about matching socks, or a canteen queue, or laying tables.

This is the most important section for a beginner and the easiest to skip. The story gives you a place to put the idea before you have the vocabulary for it. Later, when you are asked the question under pressure, the story is what you will reach for first.

3. The idea in plain English

The story, translated. Every term is defined the first time it appears, and defined terms are set in **bold** so you can find them again. Appendix D collects them all.

4. The picture

A diagram. Arrays, memory and pointers are drawn as ASCII boxes with indices above and values below, because adjacency matters and a box drawing keeps it. Trees, graphs, architectures and request flows are drawn as proper figures.

Every diagram is followed by a line telling you what to notice in it.

5. The code, built step by step

Fragments of ten lines or fewer, each one followed by prose explaining it. Then the complete solution in one block, ready to run.

The full answer is always here. This book does not withhold solutions.

6. What it costs

The arithmetic, done where you can see it.

On the DSA track that means counting the loop iterations out loud: not “this is quadratic” but “the inner loop runs sixty times for each of sixty items, so about three thousand comparisons”. On the system design track it means doing the multiplication: not “that is a lot of storage” but $50\text{M} \times 20 \times 2\text{KB} = 2\text{TB}$.

7. The traps

The mistakes people actually make, with the real error text printed exactly as it appears on screen. When your own program breaks this way, you will recognise it.

8. In the interview

The point of the whole document.

Real interviewer phrasings. A script for the first ninety seconds, because that is when candidates either take control of the question or lose it. The three follow-ups that usually come next. And a model answer, written out in full, so you have something to measure yours against.

9. Recall card

The two or three sentences that are the entire lesson. Say them aloud, from memory, in full sentences.

If you can do that, you have finished the lesson. If you cannot, you have read it, which is a different thing.

Recall cards

Section 9 of every lesson in this volume, collected. Cover the card, say it aloud, then check. Cards are grouped by the phase they belong to, on both tracks.

APIs: how services talk

Day 015 · system-design — What an API is

- **An API is a promise, not a mechanism:** the list of operations, how to ask, what comes back.
- **The caller depends on the contract and nothing else** — so everything behind it is free to change. That is the entire point.
- **The price is that you cannot change what it means.** Hence `v1` in the path.
- **Three distances, same idea:** library call (nanoseconds), system call (microseconds), network call (milliseconds, and it can fail).
- **Have one concrete example ready:** `GET https://api.github.com/users/torvalds` → `200 OK` and a JSON body.

Day 016 · system-design — REST, properly

- **REST is a style, not a technology** — six constraints, Fielding 2000. HTTP + JSON alone is not REST.
- **Uniform interface:** nouns as addresses (`/users/42`), a fixed small set of methods, the path never changes when the operation does.
- **Stateless:** every request self-contained, so any server can answer it — that is what makes scaling out easy.
- **Safe:** `GET`. **Idempotent:** `GET`, `PUT`, `DELETE`. **Not idempotent:** `POST` — which is why payments need an idempotency key.
- **HATEOAS is the constraint nobody implements.** Say so. Most real APIs are Richardson level 2, and that is fine.

Day 017 · system-design — Designing a good REST endpoint

- **Nouns first, plural, consistent.** List the resources before writing a single path.
- **Nest for list and create; go top-level for read, update and delete.** Never nest three deep.

- **Filters, sorting and paging go in the query string** — they are views of a collection, not new collections.
- **Paginate every collection from day one:** default limit, hard server cap, opaque cursor rather than offset.
- **An empty collection is 200 with [], not 404.** 404 is the parent missing; 409 is the state forbidding it; 403 is you may not.

Day 018 · system-design — Status codes, errors, and idempotency

- **Three outcomes, not two:** success, failure with an answer, and **no answer** — where you cannot tell whether it happened.
- **4xx do not retry** (except 429 / 408). **5xx and timeouts** may be retried — only if the operation is idempotent.
- **Idempotent = doing it twice equals doing it once.** GET, PUT, DELETE yes. POST no. PATCH depends on the body.
- **Idempotency key:** client generates it once, reuses it on every retry; server claims it atomically and replays the stored response.
- **Never 200 OK with an error inside.** And always back off with jitter, or you build your own outage.

Day 019 · system-design — Authentication and authorisation

- **Authn = who are you. Authz = what may you do.** Authn once, authz on every request.
- **401 = I don't know you** (unauthenticated). **403 = I know you, and no.**
- **Passwords: salted Argon2id or bcrypt at ~250 ms.** Never plaintext, never encrypted, never MD5/SHA-256.
- **Authorisation models:** ownership → ACL → RBAC → ABAC. Start with ownership plus roles.
- **The check is server-side, every request, in the query where possible.** Hiding the button is not a control.

Day 020 · system-design — JWT, sessions, and OAuth

- **Session = an opaque id the server looks up.** All truth server-side; revocation is one delete.
- **JWT = signed, self-describing, verified with no lookup.** Base64 is **not** encryption — nothing sensitive in the payload.

- **The whole trade is revocation.** A JWT cannot be un-issued; the fix is short expiry plus a revocable refresh token — which is a session store again.
- **OAuth is authorisation, OIDC is authentication.** “Log in with Google” is OIDC over the authorisation-code flow with PKCE.
- **Cookies:** `HttpOnly`, `Secure`, `SameSite`. JWTs: `RS256`, check `exp / iss / aud`, never trust `alg` from the token.

Day 021 · system-design — GraphQL versus REST

- **GraphQL: one endpoint, the client names the fields, the response mirrors the query.** Fixes over-fetching and under-fetching.
- **The big cost is HTTP caching** — every request is a `POST` to one URL, so no browser cache, no CDN, no proxy.
- **N + 1 is real and needs DataLoader** — 50 orders with customers is 51 queries naively, 2 with batching.
- **The client writes the query, so cap it:** depth limits, cost analysis, persisted queries, server-side pagination caps.
- **Choose by condition:** GraphQL for logged-in multi-client apps; REST for public, cacheable, machine-facing APIs. Consider `?fields=` and screen-shaped endpoints first.

Day 022 · system-design — gRPC and when binary protocols win

- **gRPC = call a remote function like a local one.** Schema (`.proto`) → generated typed clients → binary over persistent HTTP/2.
- **Three wins, in the order that matters:** compile-time contract, persistent multiplexed connection, small fast encoding. Plus streaming and propagating deadlines.
- **Field tags are the identity**, not names. Add new tags freely; never renumber or reuse one.
- **The boundary is the answer:** gRPC inside, REST at the edge. Public callers want `curl`, browsers, any language, and CDN caching.
- **The costs:** no `curl`, no browsers without a proxy, no HTTP caching, and long-lived connections break layer-4 load balancing.

Day 023 · system-design — Rate limiting and API gateways

- **Token bucket:** capacity = burst, refill rate = sustained rate, one token per request, no token → `429`. Two numbers per caller.
- **Fixed windows leak at the boundary** — full limit at 10:00:59 plus full limit at 10:01:00 is twice the limit in a second.

- It runs at the gateway, with counters in Redis, and the check-and-decrement must be atomic (Lua script).
- Return 429 + Retry-After, and send X-RateLimit-* on every response so clients slow down before they are refused.
- Key by API key, by IP before login, and per endpoint. It does not stop a DDoS — that belongs at the edge.

Day 024 · system-design — API revision and interview questions

- The procedure: scope → nouns → paths → one response body → errors → cross-cutting → trade-off. Fifteen minutes; the marks are in the last three.
- Say without being asked: pagination on every collection, idempotency key on anything creating or paying, an empty list is 200 with [].
- Nest for list and create; top-level for read, update, delete. Filters go in the query string.
- Every “which one” question has a condition: cacheability and who owns the caller decide REST vs GraphQL vs gRPC; revocation decides session vs JWT.
- Do not add components the arithmetic does not demand. Do add pagination, idempotency and rate limits on day one.

Arrays

Day 009 · dsa — What an array really is in memory

1. address = base + i × element_size. One multiply, one add, for any i. That is why indexing is O(1), and it is the whole answer.
2. Three requirements: contiguous, equal-sized slots, known base address. Break any one and the arithmetic fails — which is why an array holds one type.
3. Counting starts at zero because element 0's offset is zero, so the formula needs no adjustment.
4. A Python list stores references, not values. 8 bytes per slot, objects elsewhere. Still O(1) indexing, nine times the memory, worse locality. Use array or NumPy when that matters.
5. Contiguity is a trade. It buys O(1) access and charges O(n) for insertion in the middle. And a cache line is 64 bytes, so reading in order can be five times faster than reading the same elements out of order.

Day 010 · dsa — Traversal: the loop patterns you will reuse forever

1. **Derive the bound, do not recall it.** Ask what the largest index the body touches is, and stop one past it. `items[i]` → `range(n)`. `items[i+1]` → `range(n-1)`. `items[i+k-1]` → `range(n-k+1)`.
2. **n items have $n - 1$ gaps.** Seven elements, six adjacent pairs. Pegs and gaps.
3. **Backwards is `range(n-1, -1, -1)`** — stop-before is `-1` so index 0 is included. Or `reversed(items)`, which copies nothing.
4. **Iterate by value; use `enumerate` when you need the index.** `range(len(items))` with `items[i]` is nearly $2\times$ slower and has one more thing to get wrong.
5. **Never delete while iterating forwards** — it skips elements silently. Go backwards, or use a write pointer. And a sliding window should keep a running total, not re-slice.

Day 011 · dsa — Insert, delete, and the cost of the middle

1. **Delete at i costs $n - i - 1$ moves.** Insert at i costs $n - i$. The cost is everything standing behind the position.
2. **The end is free, the front is $O(n)$.** `append` and `pop()` move nothing; `insert(0, x)` and `pop(0)` move everything.
3. **The holes cannot stay** — indexing is `base + i × size`, which needs unbroken slots. That is the same property that makes indexing $O(1)$.
4. **If order does not matter, deletion is $O(1)$** : copy the last element over the hole and pop the end. Always ask whether order matters.
5. **Removing many is one pass, not many deletes.** Write pointer: $O(n)$ and $O(1)$ space against $O(n^2)$. And when shifting in place, **start at the end you are moving towards**.

Day 012 · dsa — Searching an array: linear search, done properly

1. **Walk, compare, return on the first match, and put the “not found” return AFTER the loop.** Inside the loop it bails out after one element.
2. **`-1` by convention, `None` in Python** — because `-1` is a valid index in Python and will silently read the last element. Ask which the interviewer wants.
3. **Absence is always the worst case: n comparisons.** You cannot be sure something is missing without looking everywhere. Best 1, average $n/2$, worst n . $O(n)$ time, $O(1)$ space.
4. **Ask about duplicates.** First, last, or all — three different correct answers, and only “all” loses the early exit.

5. **Never test the result for truthiness.** `if result:` is false for index 0. Use `if result is not None:` or compare against the sentinel.

Day 013 · dsa — Reversing, rotating, and swapping in place

1. **Rotate right by k = three reversals.** Reverse the whole thing, reverse the first `k`, reverse the last `n - k`. `O(n)` time, `O(1)` extra space.
2. **`k %= n` is the first line, after guarding the empty array.** Rotating by the length is a no-op; without the modulo a large `k` gives `IndexError: list index out of range`.
3. **Why it works:** the full reversal moves the two groups to the right sides but flips them internally; reversing each group undoes exactly that. Every element is reversed twice.
4. **Reverse in place is `left < right`, swap, step both inwards** — `n // 2` swaps, because each swap fixes two positions. Looping to `n` reverses it back to the original.
5. **In place means mutate.** `items[:] = ...` writes through to the caller; `items = ...` only rebinds a local name and silently changes nothing.

Day 014 · dsa — Max, min, second largest: the single-pass habit

1. **A fixed number of extreme values needs one pass and that many trackers.** Sorting arranges all `n` when you wanted `k`. `O(n)` time, `O(1)` space.
2. **Ask what “second largest” means on `[5, 5, 3]`** — 5 by position, 3 by distinct value. Two contracts, one condition apart, and `[4, 4, 4, 4]` has no distinct answer at all.
3. **When a new best arrives, the old best slides down.** Write it as `first, second = x, first` so the ordering bug cannot happen.
4. **The second branch is not optional.** Without `elif ... x > second`, `[10, 1, 2]` returns the sentinel. Always test with the largest element first.
5. **Never initialise a tracker to 0.** Use `items[0]` or `float('-inf')`, and handle empty separately — `max([])` raises `ValueError: max() iterable argument is empty`.

Day 015 · dsa — Moving elements: zeros, duplicates, and the write pointer

- **Two indices, one array.** `read` visits every position; `write` moves only when you keep one.
- **The rule:** `write ≤ read` always, so overwriting `items[write]` can never lose data.
- **At the end** `write` is the number kept; `items[write:]` is leftovers — zero it, or return `write` as the new length.
- **Only the `if` changes** across the family: `≠ 0`, `≠ target`, `≠ items[write - 1]`, `≠ items[write - 2]`.

- **O(n) time, O(1) extra space.** Never delete inside a loop — $O(n^2)$, and it skips elements.

Day 016 · dsa — 2D arrays and matrix traversal

- `matrix[row][column]`. **Row first, always.** `rows = len(matrix)`, `cols = len(matrix[0])`.
- **Same expression, swapped loops:** outer `r` is row-major; outer `c` is column-major.
- **Down-right diagonal: $r - c$ is constant. Down-left (anti-)diagonal: $r + c$ is constant.** There are `rows + cols - 1` of each.
- **Never** `[[0] * cols] * rows` — it aliases one row. Use `[[0] * cols for _ in range(rows)]`.
- **O(rows × cols) time** for any full walk. Check `0 ≤ r < rows` and `0 ≤ c < cols` before reading — negative indices wrap instead of raising.

Day 017 · dsa — Matrix tricks: rotate, spiral, transpose

- **Rotate 90° clockwise = transpose, then reverse each row.** Anticlockwise = transpose, then `matrix.reverse()`. 180° = both flips, no transpose.
- **The transpose loop is** `for c in range(r + 1, n)`. Full range swaps every pair twice and gives you a mirror, with no error.
- **In-place rotation is square-only.** Rectangular changes shape, so it must allocate.
- **Spiral is four boundaries and four passes**, each shrinking as it finishes — with a guard before the bottom pass and before the left pass.
- **Test spiral on a single row and a single column.** The square case passes even when the code is wrong.

Day 018 · dsa — Arrays revision and mock round

- **Six beats:** clarify, example, brute force, optimise, code, test. Half the time is before you type.
- **Never code without agreement.** State the approach and its cost, then pause for the nod.
- **Narrate the decision and the reason, once per decision.** Silence is indistinguishable from luck.
- **Four questions for every array problem:** can I modify it, is it sorted, duplicates/negatives/ empty, does order matter?
- **Test with the edge cases you named yourself** — not with the example you were handed.

Databases from zero

Day 025 · system-design — What a database gives you that a file does not

- **Four things a database gives you:** durability, concurrency, fast lookup, integrity — delivered by **transactions**.
- **The failures a file gives you are silent.** Lost updates on concurrent writes; a truncated file on a crash mid-write; a full scan that is fine until it is not.
- **Write-ahead log** = append the intent, `fsync`, then apply. That is durability, and it is also replication.
- **An index turns a 2 GB scan into 3-4 page reads** — roughly 24 seconds down to 0.4 ms.
- **Files are right for configuration, static assets and single-writer logs.** And SQLite is a real database in one file — the answer nobody mentions.

Day 026 · system-design — Tables, rows, and keys

- **Table = one kind of thing; row = one of them; column = one attribute with a type.** Rows have no order.
- **Primary key: unique, never null, never changes.** Use a **surrogate** (`BIGSERIAL`), not a natural key — meaningful things change.
- **The foreign key goes on the “many” side.** Many-to-many needs a third table with a composite key.
- **NOT NULL unless absence means something.** `TIMESTAMPZ` for time, `NUMERIC` for money, never `FLOAT`.
- **Index every foreign key** — primary keys are indexed automatically, foreign keys are not. And choose `ON DELETE` deliberately.

Foundations: how code costs

Day 001 · dsa — How your code actually runs, and where the time goes

1. Time is **how many instructions run**, not how many lines you wrote.
2. The **hot line** is the deepest line inside the most loops. Find it first, always.
3. One loop over n items means the hot line runs n times. Double the input, double the work.
4. Two nested loops over n items means roughly $n \times n$ times. Double the input, and you get **four times** the work.

5. A line that touches every item is a hidden loop: `x in list`, `sorted`, `max`, `sum`, `join`, `list[:]`.

Day 002 · dsa — Counting steps: your first cost model

1. Read the header, count the values. `range(a, b)` produces $b - a$ iterations.
2. **Nested loops multiply. Loops side by side add.** A line inside both loops runs $n \times m$ times; a line inside the outer one only runs n times.
3. If the inner header mentions `i`, the answer is a **sum**, not a product, and it collapses to $n \times (n - 1) / 2$.
4. Halving each pass means about $\log_2 n$ steps. A thousand is ten, a million is twenty.
5. When the count depends on the values and not just on `n`, quote **best case and worst case**. The worst case is what “the cost” means unless somebody says otherwise.

Day 003 · dsa — Big-O in plain English

1. Big-O is the **shape of the growth**. Get the exact count first, then **drop the constants and keep the biggest term**. $n^2/2 - n/2$ is $O(n^2)$.
2. The test that never fails: **double the input**. Same $\rightarrow O(1)$. Plus one step $\rightarrow O(\log n)$. Doubles $\rightarrow O(n)$. Quadruples $\rightarrow O(n^2)$.
3. **Nested loops multiply, sequential loops add**. Two loops in a row is $O(n)$, not $O(n^2)$.
4. **Check every line inside the loop**. `x in a_list`, `.index()`, slicing and `sorted()` are loops in disguise. `x in a_set` is not.
5. Roughly **10^8 operations per second**. So $n \leq 10^5$ rules out $O(n^2)$, and the constraint printed in the problem tells you the shape before you start writing.

Day 004 · dsa — The growth curves you will meet again and again

1. **The order, best to worst:** $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, $O(n!)$.
2. **The budget is 10^8 operations per second**. Divide the constraint into it and the shape names itself.
3. **The ceilings:** $O(n!)$ dies at 11, $O(2^n)$ at 20, $O(n^3)$ at 500, $O(n^2)$ at 5,000, $O(n \log n)$ at 10^6 , $O(n)$ at 10^7 .
4. **The constraint is a hint**. $n \leq 20$ means try every subset. $n \leq 10^5$ means sort or use a hash map. $n \leq 10^9$ means binary search or a formula.
5. **Python is 10–100× slower**, so treat “just fits” as “fails”. And read *every* variable in the constraints, not only the one called `n`.

Day 005 · dsa — Python for DSA I: lists, tuples, and slicing

1. A Python list is a **dynamic array**: one contiguous block. `items[i]` and `len()` are $O(1)$.
2. **The end is cheap, the front is expensive.** `append` and `pop()` are $O(1)$. `insert(0, x)` and `pop(0)` are $O(n)$, because everything shifts.
3. **append is $O(1)$ amortised** — the list over-allocates by about 12.5%, so resizes get rarer and average out to a constant.
4. **Take from the front → use `collections.deque`.** `popleft()` is $O(1)$. Every BFS depends on this.
5. **Slicing copies.** `items[1:]` is $O(n)$ in time and memory. And `[[0]*3]*3` makes three references to one row — use `[[0]*3 for _ in range(3)]`.

Day 006 · dsa — Python for DSA II: strings, dictionaries, and sets

1. **“Have I seen this before?” means a set.** `x in some_set` is $O(1)$; `x in some_list` is $O(n)$. They are spelled the same. This is the reflex the whole day exists to build.
2. **Dict and set are $O(1)$ average, $O(n)$ worst case**, and insertion is amortised because the table occasionally reshapes.
3. **Keys must be immutable.** Tuples yes, lists no — `TypeError: unhashable type: 'list'`. Grid coordinates go in as `(row, col)`.
4. **Strings are immutable, so building one with `+=` in a loop is $O(n^2)$** — CPython sometimes rescues it with an in-place resize, and that rescue vanishes the moment a second reference exists. Collect pieces in a list and `"".join(pieces)` once.
5. **Counter when you care how many, set when you care whether.** `set("aab") == set("abb")` is `True`, and that is a bug waiting to happen.

Day 007 · dsa — Space complexity, and what in-place really means

1. **Space complexity means extra space.** The input does not count; the output usually does not either. Say which convention you are using.
2. **$O(1)$ extra space = a fixed number of variables**, whatever `n` is. That is what **in-place** means, and it is built out of swaps.
3. **Slicing, `sorted()`, sets and dicts all allocate $O(n)$.** `items.sort()` is in place and still $O(n)$ auxiliary; heapsort is the $O(1)$ -space sort.
4. **Recursion costs $O(\text{depth})$ in call frames** even when the body allocates nothing. Python stops at about 1,000.
5. **`items = items[::-1]` inside a function changes nothing for the caller.** Rebinding a name is not mutation. Use `items[i] = ...` or `items[:] = ...`.

Day 008 · dsa — Reading a problem like the interviewer wrote it

1. **Four things, every time: input, output, constraints, edge cases.** Pull all four out before writing anything.
2. **Restate the problem in your own words, then work one example by hand.** Both take seconds and both catch misunderstandings while they are still free.
3. **Ask three to five questions.** Empty? Duplicates? Negatives? Sorted? What if there is no answer? Indices or values? Can I modify the input?
4. **Read the constraints and name the target complexity out loud** before choosing an approach. $n \leq 10^5$ means $O(n \log n)$ or better.
5. **State the approach in one sentence and get agreement before coding.** The interviewer can correct you for free at that moment, and not at any moment after it.

How computers and the internet work

Day 001 · system-design — What happens when you type google.com and press Enter

1. Six beats: **name** → **number**, **connect**, **verify**, **ask**, **answer**, **assemble**. DNS, TCP, TLS, HTTP request, server work, render.
2. Three round trips of setup happen **before** the first byte of the page is requested.
3. RTT comes from distance: about 200,000 km/s in fibre. Mumbai to Virginia is roughly **195 ms** round trip.
4. Moving the answer close to the user (a **CDN**) turned 685 ms of setup into 45 ms. That is the biggest single lever there is.
5. The second visit is fast for **four** separate reasons: DNS cached, connection reused, TLS resumed, static files cached.

Day 002 · system-design — Client and server, explained properly

1. **The client asks. The server waits and answers.** The client always starts the conversation.
2. A server is a **program listening on a port**, not special hardware. Your own laptop can be one.
3. Frontend code **crosses the boundary** and runs on the user's device. Backend code and database credentials never leave the server.
4. Therefore the client is **untrusted**. Client-side checks are for experience; server-side checks are for correctness. The client sends intent, the server decides facts.
5. Servers are usually **stateless**, so any machine can answer any request. That is what makes adding machines work.

Day 003 · system-design — IP addresses, ports, and DNS

1. **IP address picks the machine. Port picks the program.** A destination is the pair: `142.250.183.206:443`. HTTP is 80, HTTPS is 443, PostgreSQL is 5432, Redis is 6379.
2. **DNS turns a name into an address.** The chain is browser cache → OS cache and hosts file → resolver → root → TLD → authoritative.
3. **Four caches sit on that path**, and almost every real lookup stops at one of the first two. That is why the root servers survive.
4. **TTL is the only control you have.** Long TTL: cheap, fast, slow to change. Short TTL: expensive, nimble, and DNS becomes a hard dependency. Nobody can recall an answer already handed out.
5. **IPv4 is 2³² addresses and ran out**, which is why NAT and private ranges (`10.x`, `172.16-31.x`, `192.168.x`) exist. `127.0.0.1` is this machine.

Day 004 · system-design — TCP and UDP

1. **TCP: handshake, sequence numbers, acknowledgements, retransmission, congestion control.** Everything arrives, in order. **UDP: send and forget.** Four header fields, no promises.
2. **The deciding question: what does a lost piece cost?** Everything → TCP. Nothing, because the next one supersedes it → UDP.
3. **Head-of-line blocking** is why TCP is wrong for media: one loss stalls everything behind it, and the retransmission arrives after the moment it belonged to.
4. **TCP costs 1 round trip to set up, plus 1 for TLS.** UDP costs zero. Over a 250 ms link that is half a second before the first byte of your request.
5. **HTTP/3 runs QUIC over UDP** — not to abandon reliability, but to rebuild it per stream and dodge head-of-line blocking. TCP is still the default for everything else.

Day 005 · system-design — HTTP: the request and the response

1. **A request is four parts:** request line (method + path + version), headers, a blank line, body. The blank line is part of the format.
2. **Headers are facts about the message; the body is the message.** `Host`, `Content-Type`, `Content-Length`, `Authorization`, `Accept`. A `GET` normally has no body.
3. **Status codes by first digit:** 2xx worked, 3xx moved, 4xx you got it wrong, 5xx we got it wrong. 401 = I do not know you; 403 = I know you and no.
4. **Safe = changes nothing** (GET, HEAD). **Idempotent = repeating it is harmless** (GET, PUT, DELETE). **POST is neither** — use an idempotency key to make retries safe.

5. **HTTP is stateless.** Identity is re-sent on every request in a cookie or an `Authorization` header. That is what lets any machine answer any request.

Day 006 · system-design — HTTPS and TLS, without the maths

1. **TLS gives you three things: identity, confidentiality, integrity.** The padlock means those three succeeded and nothing more.
2. **Identity comes from a chain of trust** — leaf signed by intermediate, signed by a root already in your device's store. The CA vouches for *who*, never for *honest*.
3. **Encrypted:** path, headers, cookies, body, response. **Visible:** IP address, domain name via SNI, DNS lookup, timing, byte counts.
4. **A valid certificate does not mean a safe site.** Free automated certificates mean phishing sites have padlocks too.
5. **Asymmetric for the handshake, symmetric for the data.** Handshakes cost ~ 1 ms of CPU each and dominate; bulk encryption is nearly free. That is why resumption and keep-alive matter.

Day 007 · system-design — What a web server actually does

1. **The loop: listen, accept, handle, respond, repeat.** `socket → bind → listen → accept → read → write → close`.
2. **One worker, one request.** Concurrency = number of workers. Overflow waits in the **backlog**; when that fills, connections are refused.
3. **Three models:** processes (30–80 MB, isolated), threads (~ 1 MB, shared), event loop (~ 2 KB, single-threaded).
4. **Most of a request is waiting**, not computing — which is the entire case for async, and the reason it does nothing for CPU-bound work.
5. **Little's Law: concurrency = arrival rate $\times$ duration.** $500 \text{ rps} \times 200 \text{ ms} = 100$ in flight. That is how a latency target becomes a worker count.

Day 008 · system-design — Processes, threads, and concurrency

1. **Process = private memory. Threads = shared memory.** Every other difference follows from that one.
2. **Costs:** process 10–100 MB and $\sim$ ms to create; thread ~ 1 MB and $\sim \mu$ s; goroutine or coroutine ~ 2 KB. That is why pools exist.
3. **Race condition = check-then-act with a gap in the middle.** Two threads read a balance of 100, both spend it. Fix with a **lock**, which makes the pair atomic.
4. **Amdahl's Law:** a 5% serial section caps you at $20 \times$ speedup on any number of cores. Shrink the critical section, or remove the shared state.

5. **Python's GIL:** one thread runs bytecode at a time. `threading` for I/O, `multiprocessing` for CPU, `asyncio` for very high I/O concurrency.

Day 009 · system-design — CPU, RAM, and disk: the speed hierarchy

1. **The ladder:** register → L1 → L2 → L3 → RAM → SSD → HDD → network. Each step is roughly $100\text{--}1,000\times$ slower and cheaper per byte.
2. **The three numbers:** RAM **80 ns**, SSD **100 μ s**, disk seek **10 ms**. So SSD is $\sim 1,000\times$ slower than RAM, and a spinning disk $\sim 100,000\times$ slower.
3. **If one cycle were one second:** L1 = 3 s, RAM = 4 minutes, SSD = 4 days, disk seek = 11 months, cross-world network = 16 years.
4. **Nothing reads one byte.** 64-byte cache lines, 4 KB disk pages. Sequential access can be $100\times$ faster than random on the same hardware.
5. **Misses dominate the average.** 90% hit rate on RAM-vs-SSD still averages 10 μ s; 99% averages 1 μ s. And everything above RAM is volatile.

Day 010 · system-design — Latency numbers every engineer should know

1. **The seven to memorise:** RAM **100 ns**, SSD read **100 μ s**, disk seek **10 ms**, same-DC round trip **0.5 ms**, cross-country **10–50 ms**, cross-world **150–250 ms**, 1 Gbps = **125 MB/s**.
2. **Estimate by legs.** Break the request into steps, put a number on each, add them up, and **optimise only the dominant term**.
3. **150 ms across the world is physics.** 20,000 km at 200,000 km/s. The only lever is distance — which is what CDNs and multi-region deployments buy.
4. **Users feel the tail, not the median.** 20 calls each 99% fast means only 82% of pages are fast. Plan around p99.
5. **A million requests a day is twelve per second.** Peak is $2\text{--}5\times$ average. That one line stops most over-designing before it starts.

Day 011 · system-design — The operating system's job

1. **Your program cannot touch hardware.** It makes a **system call**, the CPU switches to kernel mode, the kernel checks permission and does the work. $\sim 1\ \mu$ s per crossing, about $1,000\times$ a function call.
2. **The four jobs: memory, scheduling, files and devices, networking** — plus permission checks on all of them.
3. **The page cache decides your I/O speed.** A cached read is $\sim 1\ \mu$ s; an SSD read is $\sim 100\ \mu$ s; a disk read is ~ 10 ms. Most reads hit the cache.

4. **Buffering is about crossing less.** 1 MB one byte at a time is a million syscalls and a second; in 4 KB blocks it is 256 syscalls and a millisecond.
5. `write()` is not durable — `fsync()` is. That is why a commit costs milliseconds, and why one fsync per transaction caps you near 1,000 per second on SSD.

Day 012 · system-design — How your code becomes a running service

1. **The chain:** commit → CI → merge → build an **immutable artefact** tagged with the commit hash → registry → deploy → **readiness probe** → load balancer sends traffic.
2. **Config is not code.** The same image runs everywhere; the database URL and secrets are injected at run time. Never bake a secret into an image.
3. **Readiness and liveness are different questions.** Ready = send me traffic. Live = I am not deadlocked. Wiring a database blip to liveness kills the whole fleet.
4. **Roll back first, diagnose after.** The old image still exists, so recovery is ~90 seconds against ~20 minutes to fix forward.
5. **Migrations do not roll back, and both versions are live during a rolling deploy.** Add a column in one release, use it in the next. Never rename in one step.

Day 013 · system-design — Containers and why everyone uses Docker

1. **A container is a process, not a computer.** Namespaces limit what it can see; cgroups limit what it can use; the image gives it its own filesystem. No guest kernel.
2. **The shared kernel explains everything** — 100 ms starts against 45 s, a few MB of overhead against a gigabyte, hundreds per host against tens, and why a Linux container needs a Linux kernel.
3. **Image = template, container = running instance.** Layers are read-only, shared and content-addressed; the writable layer on top dies with the container, so state goes in a volume or an external store.
4. **Docker solved packaging and density.** The image kills environment drift; the process model makes autoscaling possible.
5. **The shared kernel is also the weakness.** Untrusted code gets a real boundary — Firecracker, gVisor, or a VM. Run as non-root, drop capabilities, never mount the Docker socket.

Day 014 · system-design — Fundamentals revision and interview questions

1. **Recognising is not producing.** The only thing that builds recall is attempting the answer out loud before checking it. Say it to an empty room; being wrong on the way is the point.

2. **Every answer has three parts:** one-sentence answer, then the mechanism one layer down, then the limit or trade-off. Ninety seconds, then stop and offer to go deeper.
3. **Days 1–13 are one journey:** name → address → connection → request → process → operating system → hardware, and back. Draw it; every fundamentals question is a point on it.
4. **Three round trips before your first byte** on a cold connection — DNS, TCP, TLS. At 150 ms each that is most of half a second, and it is why CDNs and keep-alive exist.
5. **Each step down the memory hierarchy is about $100\times$ slower.** L1 1 ns, RAM 100 ns, SSD 100 μ s, disk 10 ms, cross-continent 150 ms. Scale them to human time and the answer to “should I cache this?” becomes obvious.

Strings

Day 019 · dsa — What a string is, and why it is immutable

- A string is an immutable array of characters. `s[i]` is `O(1)`; `s[i] = c` is a `TypeError`.
- Every “change” builds a new string. `+=` in a loop is `1 + 2 + ... + n` copies — $O(n^2)$.
- Build with a list and `"".join(parts)` — `O(n)`. To edit characters: `list(s)`, mutate, `"".join`.
- Every string method returns a new string. `s.replace(...)` alone does nothing; you need `s = s.replace(...)`.
- Immutable so it can be hashed — that is why a string can be a dictionary key and a list cannot. Compare with `=`, never `is`.

Day 020 · dsa — Building strings without the quadratic trap

- Accumulate in a list, `"".join(parts)` once. `O(n)` instead of `O(n^2)`. Say it before the loop.
- `join` is a method on the separator and handles the last element for you — never build separators by hand.
- `join` will not convert types: `"".join(map(str, values))`.
- After any grouping loop, flush the last group — it ends by running out, not by changing.
- `chars: list[str]` in the signature means mutate, so use the write pointer and `O(1)` space, not `join`.

Day 021 · dsa — Character counting and frequency maps

- “How many times”, “most common”, “appears once”, “duplicate” → build a frequency map.
- `Counter(s)` in practice; `counts.get(ch, 0) + 1` or `defaultdict(int)` when asked to do it by hand. Plain `counts[ch] += 1` is a `KeyError`.
- **First-unique needs two passes.** Uniqueness is a property of the whole string. Two passes is still $O(n)$.
- **Pass two walks the string, not the map**, because “first” means first in the input.
- `ord(ch) - ord("a")` is $O(1)$ space and silently wrong on anything but lower-case a-z — negative indices do not raise.

Day 022 · dsa — Anagrams: the sorting versus counting choice

- **Anagram = same multiset of characters.** Find a **canonical form**: sorted letters, or letter counts.
- `sorted(a) == sorted(b)` is $O(k \log k)$; **counting** is $O(k)$. Give both, then say which you would ship.
- **Length check first** — free, and it is what makes the single-array subtract-and-bail version correct.
- **Grouping = canonical form as a dictionary key**, one pass. Keys must be hashable: `"".join(sorted(w))` or `tuple(counts)`.
- **A set is the wrong structure** — it loses multiplicities, so `aacc` and `ccac` compare equal.

Day 023 · dsa — Palindromes and the two-ends habit

- **Two indices from the ends, walking inwards.** `while left < right` — the middle character needs no comparison.
- $O(n)$ time, $O(1)$ space. The nested skip loops stay linear because each index only ever moves one way.
- **Skip loops must be `while`, not `if`, and must repeat `left < right`** — or `","` runs off the end.
- `isalnum`, not `isalpha`. `"0P"` is the input that catches it.
- **Finding palindromes:** $2n - 1$ centres, odd and even. Expanding overshoots, so `return left + 1, right - 1`.

Day 024 · dsa — Substrings versus subsequences: the distinction they test

- **Substring** = contiguous. **Subsequence** = order kept, gaps allowed. Every substring is a subsequence.
- **Counts:** $n(n+1)/2$ substrings, 2^n subsequences. At $n = 20$, 210 against a million.
- **Contiguous** → sliding window, $O(n)$. **Gaps allowed** → DP table, $O(m \cdot n)$.
- If the word is missing, ask “does it have to be contiguous?” — it decides the whole solution.
- **Substring vs subsequence DP differ in the mismatch line:** reset to 0, or carry the max forward.

Day 025 · dsa — Pattern matching, the simple way

- **Naive search:** try every start, compare forward, reset to the top of the needle each time.
- $\text{range}(n - m + 1)$ — the last valid start is $n - m$. And $k < m$ goes **first** in the while condition.
- $O(n \cdot m)$ **worst case**, $O(1)$ **space** — but close to $O(n)$ on real text, because most positions fail on the first character.
- **The worst case needs repetition:** “aaaa...b” with needle “aaab”. Measured: 10,901 comparisons for $n = 1,001$.
- **Better: KMP** $O(n + m)$ (never move the haystack index back), **Rabin-Karp** (rolling hash, many needles), **Boyer-Moore** (skip from the right). Name them; do not improve them.

Day 026 · dsa — Strings revision and mock round

- **Six patterns:** count · canonical key · two ends · sliding window · list-and-join · two indices. Decide which in the first ninety seconds.
- **Three questions that pick it:** what shape is the answer, is it contiguous, one string or two?
- **Four contract questions:** which characters count, does case matter, what alphabet, what about empty?
- **Five test inputs, never the given example:** “”, “a”, “aaaa”, “abcd”, and one with nothing that counts.
- **The two silent quadratics:** `+=` on a string in a loop, and slicing inside a loop. Both are correct and both are slow.

APPENDIX C

The complete 180-day map

All one hundred and eighty days, both tracks, in order. Days in **bold** are in this volume.

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
001	How your code actually runs, and where the time goes	What happens when you type google.com and press Enter
002	Counting steps: your first cost model	Client and server, explained properly
003	Big-O in plain English	IP addresses, ports, and DNS
004	The growth curves you will meet again and again	TCP and UDP
005	Python for DSA I: lists, tuples, and slicing	HTTP: the request and the response
006	Python for DSA II: strings, dictionaries, and sets	HTTPS and TLS, without the maths
007	Space complexity, and what in-place really means	What a web server actually does
008	Reading a problem like the interviewer wrote it	Processes, threads, and concurrency
009	What an array really is in memory	CPU, RAM, and disk: the speed hierarchy
010	Traversal: the loop patterns you will reuse forever	Latency numbers every engineer should know
011	Insert, delete, and the cost of the middle	The operating system's job
012	Searching an array: linear search, done properly	How your code becomes a running service
013	Reversing, rotating, and swapping in place	Containers and why everyone uses Docker
014	Max, min, second largest: the single-pass habit	Fundamentals revision and interview questions
015	Moving elements: zeros, duplicates, and the write pointer	What an API is
016	2D arrays and matrix traversal	REST, properly
017	Matrix tricks: rotate, spiral, transpose	Designing a good REST endpoint
018	Arrays revision and mock round	Status codes, errors, and idempotency
019	What a string is, and why it is immutable	Authentication and authorisation
020	Building strings without the quadratic trap	JWT, sessions, and OAuth
021	Character counting and frequency maps	GraphQL versus REST
022	Anagrams: the sorting versus counting choice	gRPC and when binary protocols win
023	Palindromes and the two-ends habit	Rate limiting and API gateways
024	Substrings versus subsequences: the distinction they test	API revision and interview questions

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
025	Pattern matching, the simple way	What a database gives you that a file does not
026	Strings revision and mock round	Tables, rows, and keys

Glossary

Every term this book defines, with the day it first appears. Definitions are the sentence that introduced the term.

4

4 kb [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

A

a replicated log [Day 119](#) — Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around a replicated log: an ordered, numbered list of entries, identical on every machine.

abstract class [Day 052](#) — Definition. An abstract class is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in.

acid [Day 025](#) — When Postgres says COMMIT succeeded, the data is on disk, and that promise is the D in ACID — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [\[day 033\]\(../day-033-window-with-a-map/README.md\)](#).

adaptee [Day 068](#) — The adaptee — the washing machine, which has a different shape and cannot be changed.

adapter [Day 059](#) — An adapter is an implementation that translates to a specific technology (PostgresOrderRepository).

add a tie-breaker [Day 027](#) — And add a tie-breaker — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

algorithm [Day 076](#) — A strategy is an algorithm plugged into a caller; a command is a request with its receiver and arguments already bound, so it can be stored, queued and undone.

always comparable [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

amortised [Day 005](#) — The word for that is amortised: `list.append` is $O(1)$ amortised, meaning any single append might be expensive but a long run of them averages out to constant time.

anaemic model [Day 044](#) — A design where classes hold only data and all the rules live in some `IdolService` is called an anaemic model — objects in name only — and it is the most common way candidates lose this round.

and a direction [Day 079](#) — It carries a floor and a direction — "I am on 4 and I want to go up".

anti-diagonals [Day 016](#) — To collect the down-left diagonals — usually called the anti-diagonals — group cells by the value of $r + c$.

anti-sniping [Day 095](#) — Starting the count again when somebody speaks at the last moment is anti-sniping: extending the deadline whenever a bid arrives near the end.

approximate [Day 102](#) — Also: Redis's LRU is approximate — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access.

array [Day 015](#) — Padma's shelf is an array — the fixed row of numbered boxes you met on [day 009](../day-009-what-an-array-is/README.md).

artefact [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

assignment rule [Day 108](#) — "Walk forwards until you reach a point" is the assignment rule: a key belongs to the first node clockwise.

at-least-once [Day 129](#) — Two: you gain duplicates. Queues deliver at-least-once: if the consumer does not acknowledge in time, the message comes back.

atomic [Day 008](#) — The fix is to make check-and-act atomic — indivisible, so that no other thread can act in the middle of it.

atomicity [Day 025](#) — That is atomicity — the A in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

availability [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *every* non-failing node answers.

aws ecs [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

B

backend [Day 002](#) — The backend is the code that never leaves the kitchen.

bearer token [Day 019](#) — The token is a bearer token: whoever bears it is treated as the user.

best case [Day 002](#) — The best case is 1, the worst case is n.

binary tree [Day 098](#) — A binary tree is a tree where every node has at most two children, called left and right.

blast radius [Day 111](#) — The eleven days revealing water, diesel and medicine is blast radius: what actually stops.

block [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

blocked [Day 007](#) — When the barber stood there holding a head while the boy fetched hair colour, he was blocked — occupying a chair while doing no work.

both [Day 030](#) — Once both are inside the loop, look at the gap between them, measured the way round the loop from fast to slow.

boundary [Day 118](#) — The unifying idea worth stating: *two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.*

bounded context [Day 051](#) — The name for the boundary between those worlds is a bounded context: each context gets its own model of "customer", and they are linked by an id rather than merged.

bounded optimism [Day 116](#) — Spending up to two thousand is bounded optimism — act early where the downside is small.

breadth-first [Day 100](#) — The other family is breadth-first — one whole level at a time — which needs a queue instead of a stack and is [tomorrow](../day-101-bfs-level-order/README.md).

break point [Day 045](#) — The result has one break point — one place where a value is followed by a smaller value.

broken links [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

bucket by time [Day 157](#) — The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so bucket by time: the partition key becomes (channel_id, month).

bucket counts [Day 171](#) — The fix is histograms. Each machine exports bucket counts — "how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms" — and you sum the buckets across machines and compute the percentile from the sum. That is exactly what Prometheus's `histogram_quantile` does, and it is why counters and histograms are the metric types that aggregate and...

bulkhead [Day 111](#) — A bulkhead is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

burst size [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

C

cache [Day 003](#) — Saving it in his phone is caching. A cache is a local copy of an answer, kept so that you do not have to ask again.

cache it [Day 107](#) — Fix: you cannot split a single key across shards, so cache it — the photocopy — or replicate it to every shard, or give it its own shard.

cache stampede [Day 090](#) — And the first cold morning is a cache stampede.

candidates [Day 093](#) — The trays are the candidates — the values you are allowed to pick from.

capacity [Day 005](#) — A list keeps a capacity — how many slots it has room for — which is bigger than its length, the number of slots actually used.

cdn [Day 070](#) — CDN — a geographically distributed caching proxy.

certificate [Day 006](#) — When your browser connects to `google.com`, the machine sends back a certificate — a small document containing the name it claims (`google.com`), a public key, an expiry date, and a signature from a Certificate Authority.

chaining [Day 060](#) — The standard fix is chaining: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket.

channels [Day 092](#) — The group message, the direct message and the phone call are channels — the different ways a message can physically reach a person.

checks permission [Day 011](#) — The kernel checks the arguments and checks permission — this is Balan saying no.

chunks [Day 160](#) — A file is not stored as a file. It is split into chunks — fixed or variable-size blocks, typically around 4 MB — and each chunk is identified by the hash of its contents. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

circular buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position $7 + 1$ is position 0.

class [Day 043](#) — A class is the description; an object is one thing made from that description.

client [Day 068](#) — The client — the wall socket, which will only accept one shape.

collapse key [Day 141](#) — Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a collapse key: a new notification with the same key replaces any undelivered one.

collapsing writes [Day 102](#) — Its real advantage is collapsing writes — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

collection [Day 016](#) — A collection is the set of things: /bags.

column [Day 026](#) — A column is one attribute of it, with a type that constrains what can go there.

compact [Day 011](#) — The price is that you now need a separate way to know which slots are real, and you must eventually compact: one pass that closes all the holes at once.

comparator [Day 058](#) — And the two uncles are the one case that needs a comparator — a rule about a pair, not a fact about an item.

comparison [Day 052](#) — A comparison is one question: **is this one bigger than that one?** A swap is exchanging two values so each sits where the other was.

complete [Day 114](#) — Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree complete — any other position leaves a gap and destroys the array layout.

component [Day 069](#) — The tumblers are the component — the real object that does the actual job.

composite [Day 029](#) — This only ever applies when the primary key is composite — two or more columns together.

composite key [Day 026](#) — Its primary key is usually the two columns together — a composite key — which also enforces that the same pair cannot appear twice.

composition [Day 071](#) — Strategy uses composition — the varying part is an object passed in.

composition root [Day 053](#) — The composition root is the one place in the program where the real concrete classes are named and wired together — usually `main()` or a startup module.

connection [Day 041](#) — Under all of this, every query needs a connection — and opening one is genuinely expensive: TCP, then TLS ([[day 006](#)](../day-006-python-strings-dicts-sets/README.md)'s handshake), authentication, and in Postgres a whole operating-system process per connection ([[day 008](#)](../day-008-reading-a-problem/README.md)'s processes, ~5–10 MB each).

connection-oriented [Day 004](#) — This is why TCP is called connection-oriented — there is a real, agreed-upon connection with state on both sides.

consequence [Day 109](#) — The lorries are the consequence — the number that changes a decision, which is the only number that actually mattered.

consistency [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that **every** non-failing node answers.

constraints [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

constructor [Day 044](#) — `__init__` is the constructor: the code that runs when a new object is made, whose job is to leave the object in a valid state.

container [Day 013](#) — A container is an ordinary process — the same kind of process you met on [[day 008](#)](../day-008-reading-a-problem/README.md) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume.

container image [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

containerd [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

context [Day 073](#) — The object — the context — holds one of those classes and forwards every request to it.

coordinator [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

correlation id [Day 091](#) — So every record carries a correlation id — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

cost [Day 032](#) — It enumerates the plausible combinations, assigns each a cost — an abstract number, not milliseconds — and picks the smallest.

count map [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

counter [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

cri-o [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

critical path [Day 134](#) — The number of levels is the critical path — the minimum time to finish everything if you had unlimited workers.

cycle [Day 125](#) — Cyclic or acyclic is the third. A cycle is a path that starts and ends at the same vertex without reusing an edge.

D

dangerous [Day 104](#) — Compact, and dangerous: any statement that is not deterministic gives a different result on the follower.

dashed [Day 050](#) — Four: implements an interface — the same triangle, but a dashed line:

dau [Day 097](#) — DAU — daily active users.

deep-copied [Day 067](#) — The cooker and the dabbas were deep-copied — genuinely new things that happen to look the same.

deleted [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

delta of delta [Day 137](#) — So you store the difference of the differences — the delta of delta — which is almost always zero, and zero costs one bit.

dependency [Day 053](#) — A dependency is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.

depth [Day 046](#) — First, depth: with Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

depth-first [Day 100](#) — Going depth-first means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

deque [Day 031](#) — The fix for max is to keep a deque — a double-ended queue, from [day 074](../day-074-deques-and-window-max/README.md) — holding indices, arranged so that their values are decreasing from front to back.

diminishing returns [Day 135](#) — But with diminishing returns: the eighth mention adds much less than the second.

direct addressing [Day 066](#) — The wall of numbered holes is direct addressing: because the keys are the numbers 1 to 120, you do not need to hash anything.

domain name [Day 003](#) — "Anjali Sharma" is the domain name. A domain name is a human-readable label for a machine or a group of machines: google.com, leetcode.com, api.github.com.

draw two diagrams [Day 050](#) — If your design genuinely has fifteen classes, draw two diagrams: the core five or six, and then a second showing one subsystem in detail.

duck typing [Day 047](#) — This is duck typing: if it has a send method that takes a string, it is a notifier.

dummy node [Day 080](#) — Sister Mary is a dummy node — sometimes called a sentinel head.

E

earlier [Day 118](#) — The safe pattern: the leader treats its lease as expiring earlier than the grantor does, so there is a gap in which nobody believes they are leader.

edge cases [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

edge compute [Day 103](#) — There is a real modern exception, and mentioning it is a good signal: edge compute — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin.

effect [Day 113](#) — Say it that way: "exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key." Systems that advertise exactly-once are doing precisely this.

elastic scaling [Day 100](#) — The locum starting with no handover is elastic scaling: add a machine and it is immediately useful.

election timeout [Day 118](#) — "By half past five" is the election timeout — the guess that the leader is gone.

empty [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

etl [Day 139](#) — ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce.

event object [Day 072](#) — In practice you push a small, immutable event object — a plain record describing what happened, like `OrderPlaced(order_id, user_id, total, items)`.

eventual consistency [Day 115](#) — At the bottom is eventual consistency: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile.

exactly [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

exactly correct [Day 089](#) — This is exactly correct — no boundary effect, no approximation — and it is the version you already wrote on [\[day 077\]\(../day-077-stacks-queues-revision/README.md\)](#) as a deque of timestamps.

exactly one [Day 171](#) — XOR is exactly one — which is the same as "different", and that is the property the whole of the next XOR day rests on.

exclusive lock [Day 035](#) — When a transaction updates a row, the database gives it an exclusive lock on that row: nobody else may change the row until the transaction commits or rolls back.

expiry [Day 090](#) — The dates are expiry — a completely separate mechanism.

F

facts [Day 082](#) — So: a `FinePolicy`. The loan holds the facts — who, what, when it went out, when it was due, when it came back.

false negative [Day 124](#) — A false negative is failing to notice a machine that really has died.

false positive [Day 124](#) — A false positive is declaring a healthy machine dead.

fanout [Day 030](#) — The critical property is the fanout: because a page is 8 KB and an entry is small, one node holds hundreds of entries.

fee [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day."

fifo [Day 090](#) — FIFO — evict the oldest inserted.

file [Day 093](#) — A file is a thing that is not a box: a plate, a lamp.

filled diamond [Day 050](#) — A filled diamond means *composition* — the parts die with the whole, so `Order` ♦ — `OrderLine` because a line item has no meaning without its order.

finite state machine [Day 073](#) — This is a finite state machine — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which.

float [Day 081](#) — Devi's second tin is a float: coins reserved so tomorrow can start.

four-direction move [Day 096](#) — "Each one immediately beside the one before" is the four-direction move: up, down, left, right.

fragile [Day 021](#) — It is also fragile: it silently breaks on capital letters, spaces, digits and anything non-English, and §7 shows exactly how silently.

frontend [Day 002](#) — The frontend is the code that crosses the counter.

G

galloping mode [Day 057](#) — When merging, if one run keeps winning, Timsort switches to galloping mode — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time.

gap [Day 047](#) — The gap is the answer she is searching for — not something in the array, a number she is choosing.

generator expression [Day 020](#) — That is a generator expression — it produces each item as join asks for it.

greedy simulation [Day 046](#) — It is a greedy simulation: load as much as fits, then start a new trip.

guest operating system [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

H

hash map [Day 066](#) — Kiran's phone is a hash map: a general system that can look up anything by any key, at the price of storing the keys and computing where they go.

hash ring [Day 108](#) — The eleven-kilometre loop is the hash ring — the space of all hash values, joined end to end.

hateoas [Day 016](#) — And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: HATEOAS — Hypermedia As The Engine Of Application State.

header interface [Day 058](#) — A header interface is named after the implementation and lists everything it can do: Repository, Machine, UserService.

heap [Day 117](#) — The six letters on the table are the heap: one candidate per list.

heapsort [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

heartbeat [Day 124](#) — The whistle is a heartbeat. A heartbeat is a small, regular message that says "I am alive".

hidden spof [Day 111](#) — The shared approach road is a hidden SPOF — two components that are not independent.

hiding mechanism [Day 052](#) — Abstraction is about hiding mechanism: the caller says "charge this card" and does not learn which payment provider ran it.

highest node [Day 104](#) — Every path has exactly one highest node — the point where it turns around.

historical snapshot [Day 029](#) — | Copy | Instead of | Why | |---|---|---| | posts.comment_count | COUNT(*) on comments | counting on every read is unaffordable at a high read ratio | | orders.customer_name | joining to customers | a historical snapshot: the name at the time of the order | | order_items.unit_price | joining to products | the price *then*, which must...

hollow triangle [Day 050](#) — Three: inheritance — a line with a hollow triangle pointing at the parent:

hook [Day 075](#) — award is a hook: the base class implements it as `*doing nothing*`, so a subclass may override it and most do not.

horizontal scaling [Day 098](#) — Horizontal scaling means putting more machines beside it and dividing the work.

hypervisor [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

I

image [Day 013](#) — An image is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code.

implicit graph [Day 130](#) — This is an implicit graph — the edges exist by rule rather than by record — and you met the idea on [\[day 125\]\(../day-125-what-a-graph-is/README.md\)](#).

in parallel [Day 111](#) — Components in parallel — any one will do — multiply their `*failure*` probabilities, which is a completely different shape.

in series [Day 111](#) — Components in series — every one is needed — multiply their availabilities, so the total is always worse than the worst one.

in-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

index [Day 025](#) — A database keeps an index — a separate structure, usually a B-tree, that maps a value to the location of the matching rows.

indexed heap [Day 114](#) — Hence the indexed heap: a hash map from value to index, updated on every swap.

inheritance [Day 071](#) — Template Method uses inheritance — an abstract base with the skeleton and subclasses filling in the holes.

inorder [Day 110](#) — The standing order is inorder: left subtree, then root, then right subtree.

input [Day 008](#) — In a problem: the input — its type, its size, and what is in it.

integrity [Day 006](#) — TLS also guarantees integrity: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection.

intent plus multiplicity [Day 070](#) — The honest distinction is intent plus multiplicity: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

interface [Day 052](#) — An interface is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

invariant [Day 028](#) — That is called an invariant — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

is possible [Day 046](#) — `> *Find the smallest X such that some task is possible.*` `> > *Find the largest X such that some condition still holds.*`

item [Day 016](#) — An item is one of them: `/bags/213`.

iterator [Day 075](#) — Iterator is the pattern where "go through this collection one item at a time" is separated from "this collection".

J

json [Day 015](#) — JSON — JavaScript Object Notation — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [\[day 007\]\(../day-007-space-complexity/README.md\)](#); it looks like `{"id": 42, "name": "Zoya"}`.

jump [Day 094](#) — A snake and a ladder are both a jump: a pair (from, to).

jwt [Day 020](#) — a session id — an opaque reference, meaningless by itself, that the server looks up; - a JWT — a self-contained signed token that carries the facts inside it, so no lookup is needed; - OAuth 2.0 and OpenID Connect — a protocol for getting one of the above from *somebody else*, so a user can log in with Google without your service ever...

K

k closest points [Day 116](#) — k closest points — the same shape with a computed key.

k most frequent [Day 116](#) — k most frequent — count first, then top-k on the counts.

k-th largest specifically [Day 116](#) — k-th largest specifically — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

key [Day 023](#) — The key is as much of the design as the algorithm.

key property [Day 108](#) — Adding the temple point disturbing only sixty houses is the key property: adding a node moves only the keys between it and its predecessor.

L

lag [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

largest [Day 055](#) — the 1st smallest is at position 0, so the k-th smallest is at position k - 1; - the 1st largest is at position n - 1, so the k-th largest is at position n - k.

latency [Day 089](#) — The cost is latency — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

lazy [Day 117](#) — Merging sorted streams — `heapq.merge(*iterables)` does exactly this and is lazy, so it works on inputs larger than memory.

lazy deletion [Day 114](#) — Python has neither. The idiomatic workaround is lazy deletion: push a new entry and mark the old one stale, skipping stale entries as they surface.

lazy loading [Day 041](#) — That is lazy loading: the ingredient bought at the moment it is needed.

lazy refill [Day 089](#) — The lazy refill is the implementation trick: no timer, no background job.

leader [Day 105](#) — Counter two is the leader: writes go there, and reads from it are always current.

leaf [Day 098](#) — He is a leaf — an item with no children.

lease [Day 118](#) — A lease is leadership with an expiry: you are the leader until time T unless you renew.

ledger entries [Day 085](#) — An expense produces a set of ledger entries: the payer is owed, each participant owes their share.

left [Day 040](#) — Remove the strip to the left: `prefix[r2 + 1][c1]`.

lfu [Day 090](#) — LFU — evict the least frequently used.

linear scan [Day 062](#) — Scrolling the whole list of names is a linear scan: to answer one question you touch everything.

linearizability [Day 114](#) — C — Consistency. Specifically linearizability: every read sees the most recent write, and the system behaves as if there were one copy of the data.

list [Day 001](#) — heap is a list: a row of values kept in order, like the socks piled on the bed.

list of lists [Day 016](#) — A matrix is a list of lists: the outer list holds the rows, and each row is itself a list holding that row's values.

load balancer [Day 098](#) — Sending students to the right counter is a load balancer.

load factor [Day 060](#) — The number that controls this is the load factor: the number of items divided by the number of buckets.

load shedding [Day 007](#) — This idea has a name — load shedding — and it is one of the things that separates a system that degrades from one that collapses.

log-structured merge tree [Day 031](#) — A log-structured merge tree — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade.

logical cohesion [Day 061](#) — Anjali's first kitchen is logical cohesion — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind.

logical time [Day 113](#) — The replacement is logical time — Lamport clocks and vector clocks — which order events by *causality* rather than by wall time.

loop [Day 001](#) — A loop is a statement that says, "do the following again and again".

loop body [Day 002](#) — A loop body is the indented block underneath the loop header — the part that runs again and again.

low cohesion [Day 061](#) — The kitchen arranged by size is low cohesion — things used together kept apart, so every job takes trips.

lru [Day 090](#) — LRU — evict the least recently used.

M

manifest [Day 158](#) — Adaptive bitrate streaming. The client downloads a manifest — an HLS .m3u8 or DASH .mpd — listing every rendition and every segment.

many times [Day 149](#) — Virtual nodes fix that. Each physical machine is placed on the circle many times — 100 to 256 positions each, by hashing machine-1#0, machine-1#1, and so on.

many-to-many [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

matrix [Day 016](#) — A matrix is a rectangle of values addressed by two numbers instead of one: which row, then which column.

mau [Day 097](#) — MAU — monthly active users.

max-heap [Day 116](#) — The mirror rule: to find the k smallest, keep a max-heap of size k — in Python, a min-heap of negated values.

method resolution order [Day 052](#) — The Python answer. The method resolution order — D._mro_ — is computed by a rule called C3 linearisation, and super() follows that order rather than jumping straight to a parent.

milliseconds [Day 009](#) — About 10 milliseconds — a hundred times slower again than SSD, and a hundred thousand times slower than RAM.

min-heap [Day 118](#) — Each half is a heap, and they face each other: a max-heap holding the lower half so its largest is at the top, and a min-heap holding the upper half so its smallest is at the top — the two elements at the boundary are exactly the ones you need. And the only real work is keeping the two halves the same size, which is two lines and is...

minimum spanning forest [Day 139](#) — Both algorithms then produce a minimum spanning forest — one tree per component — and you detect it by counting: fewer than $V - 1$ edges taken means the graph was disconnected.

mixin [Day 049](#) — A mixin is a small class carrying one piece of behaviour, meant to be inherited alongside others:

monotone [Day 043](#) — A yes-or-no question with that shape is called monotone: once the answer flips from no to yes, it stays yes.

monotonic deque [Day 074](#) — The good solution keeps a small monotonic deque of "elements that could still be the maximum" and is $O(n)$ overall.

N

name mangling [Day 045](#) — Two underscores triggers name mangling: inside the class the attribute is rewritten as `_Account_key`, so a subclass defining its own `_key` does not collide.

namespace [Day 013](#) — A namespace is a kernel feature that gives a process its own private version of one part of the system.

negative [Day 035](#) — A sum over values that can be negative is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [\[day 037\]\(../day-037-prefix-sums/README.md\)](#).

negative infinity [Day 049](#) — LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as negative infinity — imaginary values smaller than anything real, just off each end.

negatives [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

nested loop [Day 001](#) — A nested loop is a loop inside another loop.

network partition [Day 114](#) — The exchange fault is a network partition — the nodes are alive but cannot talk.

never equal [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

never forgets [Day 090](#) — Needs a third structure and a `min_frequency` counter to stay $O(1)$, and it never forgets: an entry that was hot last month holds a huge count and will not leave.

never persisted [Day 157](#) — Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and never persisted — a typing event that arrives late is worse than one that is dropped.

no children [Day 121](#) — It has no children — nothing continues past it.

node [Day 125](#) — The word node means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code.

not a map [Day 067](#) — | not a map — heap or sorted array |

O

$o(1)$ [Day 079](#) — tail — a reference to the last node, which makes append $O(1)$ instead of $O(n)$.

object [Day 043](#) — An object is data and the behaviour that belongs to that data, kept together in one place.

observers [Day 072](#) — The objects on the list are the observers.

occupied [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

offset [Day 130](#) — Each viewer's position is an offset. An offset is just a number — the position in the log that a particular reader has got to.

on first touch [Day 041](#) — Each `post.author` is a `*different table*`, not yet loaded — so the ORM, helpfully and silently, fetches it on first touch.

one-shot [Day 075](#) — A list can be walked many times; a generator is one-shot — walk it twice and the second walk is empty, which is a bug people hit constantly.

opens [Day 126](#) — When failures exceed a threshold, it opens: from then on, calls to that dependency fail instantly without being attempted.

orchestrator [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

origin [Day 101](#) — The storeroom is the origin — the source of truth.

origin pull [Day 103](#) — Sending the finished pages down the wire is origin pull: the edge fetches from the origin on demand.

orphan rows [Day 026](#) — Without it you get orphan rows — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

orphans [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

out-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

overloading [Day 047](#) — Compile-time polymorphism, or overloading: several methods with the same name and different argument types, and the compiler picks.

overriding [Day 047](#) — Runtime polymorphism, or overriding: a subclass replaces a parent's method, and which one runs is decided by the object at run time.

P

page [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

parallel edges [Day 132](#) — And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — parallel edges — then A—B twice genuinely **is** a cycle, and skipping "the parent vertex" wrongly ignores it.

parameters [Day 015](#) — In an API those are the parameters — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

parent [Day 132](#) — The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its parent — and the check becomes "seen, and not my parent".

partial failure [Day 113](#) — The one that matters most is partial failure: a call over a network has a third outcome that does not exist locally, *"I do not know"*, and almost every difficult problem in the rest of this phase is a consequence of it.

participants [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

partition key [Day 039](#) — A wide-column store — Cassandra is the name to know — organises data by two keys: a partition key that decides which machine (and which bucket on it) a row belongs to, and clustering columns that keep rows **sorted inside** that bucket.

partitions [Day 130](#) — It is split into partitions — independent logs that together make up the topic.

path [Day 005](#) — A request has four parts: a method saying what you want done, a path saying what you want it done to, headers carrying facts about the request itself, and an optional body carrying the content.

port [Day 003](#) — A port is a second number that says which program on that machine you want, the way a flat number says which door inside the building.

prefix sums [Day 037](#) — His noted meter readings are the prefix sums — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

preorder [Day 110](#) — The calling order is preorder: root, then left subtree, then right subtree.

priority queue [Day 139](#) — Finding "the cheapest edge leaving the tree" efficiently means a priority queue — which makes Prim's structurally almost identical to [Dijkstra's](../day-136-dijkstra/README.md).

procedure [Day 062](#) — It is a procedure: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first.

process [Day 008](#) — A process is a running program with its own private memory.

program [Day 001](#) — A program is a sequence of statements.

protecting state [Day 052](#) — Definition. Encapsulation is about protecting state: the balance cannot go negative because the only way to change it runs a check.

Q

qps [Day 097](#) — QPS stands for *queries per second* — the number of requests arriving at your system each second.

queue [Day 127](#) — The queue holds the frontier in order. A queue is a line where things come out in the order they went in — first in, first out — which you met on [day 73](../day-073-queues/README.md).

quorum [Day 118](#) — "Three of the four must agree" is a quorum — the majority from [day 117](../day-117-merge-k-sorted/README.md).

R

race condition [Day 008](#) — A race condition is when the result depends on the exact timing of two things running at once.

ram [Day 009](#) — The suitcase on the almirah — main memory, RAM. RAM — random access memory — is where your running program's data lives.

rank [Day 138](#) — Union by rank is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

rebalancing [Day 107](#) — Moving twenty bundles is rebalancing — moving whole logical shards, not rehashing keys.

rebuilding the tree [Day 109](#) — Repacking the whole cart is rebuilding the tree — $O(n)$.

recursion [Day 053](#) — That is recursion: a function whose body calls the same function on a smaller piece of the problem.

references [Day 009](#) — A Python list stores references — memory addresses of objects that live elsewhere:

refill rate [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

register [Day 009](#) — The chair beside the bed — registers. A register is a tiny store inside the processor itself, holding one value.

relative epsilon [Day 048](#) — The fix is a relative epsilon — stop when the gap is small *compared to the value*:

removal [Day 118](#) — The same two heaps, plus removal, which a heap cannot do.

replica [Day 136](#) — A replica is a copy of a shard on another node, serving reads and taking over if the primary fails.

replica divergence [Day 115](#) — Different people knowing different things is replica divergence — and none of them is wrong.

request-aware routing [Day 099](#) — The joint-family card going to counter one is request-aware routing — some requests are much more expensive than others.

resolver [Day 021](#) — Each field in the schema has a resolver — a function that knows how to produce that field.

resources [Day 017](#) — Before writing a single path, list the resources — the things the feature is about.

returned [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day.*

ring buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position 7 + 1 is position 0.

role interface [Day 058](#) — A role interface is named after what a client needs: UserReader, Printer, PriceQuoter.

root [Day 093](#) — The root is the root itself, written /.

rotation [Day 109](#) — The fix is a rotation: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order.

routine [Day 109](#) — Counting paces and multiplying is the routine — the same sequence every time, said out loud.

row [Day 026](#) — A row is one of those things.

run [Day 044](#) — So the copies of a value form one run: a contiguous stretch, described completely by its two ends.

S

same remainder [Day 038](#) — A stretch is divisible by k when the prefixes at its two ends leave the same remainder — so carry running % k and count matching remainders.

schema [Day 021](#) — A GraphQL API is defined by a schema: every type, every field, and every field's type, written down and strongly typed.

scoping [Day 096](#) — The two or three narrowing questions are scoping — deciding out loud what you are *not* building.

script [Day 077](#) — Today is the script — the fixed order of moves you run on every one of these prompts for the next twenty days.

seam [Day 053](#) — A seam is a place where you can change behaviour without editing the class.

search space [Day 042](#) — The stretch `nums[low..high]` is called the search space — the part of the array you have not ruled out yet.

selectivity [Day 032](#) — From those numbers the planner estimates selectivity: what fraction of rows a condition keeps.

sentinel [Day 037](#) — That extra entry is called a sentinel: a value placed at the boundary so that the boundary needs no special treatment.

separator [Day 019](#) — The string before the `.join` is the separator — `"".join` glues with nothing between, `",".join` puts a comma and a space between each pair.

sequence diagram [Day 050](#) — That is a sequence diagram: participants as columns, time running down, and each message an arrow from one column to the next.

serialisable [Day 033](#) — The strongest form is serialisable: the result is as if the transactions had run one after another, in some order.

service credits [Day 173](#) — An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always service credits — money back off the next bill — rather than damages.

session [Day 020](#) — A session is a record the server keeps about a logged-in user.

session guarantees [Day 115](#) — And the interesting rungs are in the middle, particularly causal consistency and the four session guarantees, because they are what users actually notice and they are much cheaper than the top.

set [Day 006](#) — A set stores values and answers "is this in here?" in a single step, no matter how many it holds.

shallow-copied [Day 067](#) — The milk and the newspaper were shallow-copied — the new kitchen got a *reference* to the same arrangement, and changing it from either end changed it for both.

shape [Day 003](#) — It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the shape of the growth.

shard [Day 106](#) — Each room is a shard — a machine holding a subset of the rows.

shared lock [Day 035](#) — A shared lock is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other.

shared store [Day 100](#) — The shared cabinet at the front desk is a shared store — a database, or Redis.

singleton [Day 064](#) — Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it.

singly linked list [Day 076](#) — A singly linked list — where each node knows only the *next* one — cannot do that.

slice [Day 019](#) — `s[1:4]` is a slice — a new string made of part of the old one.

smallest [Day 046](#) — `>` *Find the smallest X such that some task is possible.* `>` `>` *Find the largest X such that some condition still holds.*

snapshot [Day 034](#) — The transaction takes a snapshot at its first read and every statement in it sees that snapshot, however long it runs.

snipe [Day 095](#) — A snipe is a bid placed in the last second, leaving nobody time to respond.

sorted region [Day 052](#) — And the sorted region is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in *which end* the region grows from and *how* the next value gets there.

sorting everything [Day 116](#) — The nephew laying out the whole load is sorting everything — correct, and far more work than asked for.

splits [Day 031](#) — If the leaf is full, it splits: half the keys move to a new page, and a key is pushed up to the parent.

spof [Day 111](#) — The bridge is an SPOF — one component, and everything depends on it.

squares [Day 003](#) — | Example | |---|---|---| | $O(1)$ | constant | does not change | reading items[5] | | $O(\log n)$ | logarithmic | goes up by one step | halving until you reach 1 | | $O(n)$ | linear | doubles | one loop over the list | | $O(n \log n)$ | linearithmic | slightly more than doubles | sorting | | $O(n^2)$ | quadratic | goes up four times | every...

stable [Day 051](#) — Python's sort is Timsort: $O(n \log n)$ worst case, stable, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to $O(n)$.

stack [Day 075](#) — The fridge is a stack: last in, first out.

stack frame [Day 068](#) — When a function calls another function, the machine pushes a stack frame — the local variables and the place to return to — onto a stack.

stack overflow [Day 068](#) — And the bad Sunday is a stack overflow: too many unfinished things, and the one at the bottom is forgotten.

state [Day 143](#) — And the state is the thing to get right. The state is what identifies a subproblem — the arguments to the recursive function.

statements [Day 001](#) — A program is a sequence of statements.

status [Day 086](#) — It has a status — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

still one machine [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

stonith [Day 118](#) — STONITH — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.

strategy [Day 071](#) — Strategy turns an algorithm into a value.

strictly shorter [Day 072](#) — `left[i]` — the index of the nearest bar to the left that is strictly shorter.

structural [Day 058](#) — In Python, `typing.Protocol` is structural — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing ([[day 052](#)](../day-052-quadratic-sorts/README.md)).

subarray [Day 024](#) — There is a third word you will meet: a subarray is the same thing as a substring, for arrays.

subject [Day 072](#) — Observer is the pattern where one object — the subject — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call.

subscription list [Day 072](#) — The group itself is the subscription list, and the one message is the notification.

substring [Day 155](#) — First the definitions. A substring is contiguous — "aba" is a substring of "xabay", but "aa" is not.

subtree [Day 122](#) — If you get to the end, the node you are standing on is the root of a subtree — the whole set of words that begin with what the user typed.

surge multiplier especially [Day 088](#) — The surge multiplier especially is frozen.

swap [Day 052](#) — A comparison is one question: `*is this one bigger than that one?*` A swap is exchanging two values so each sits where the other was.

system call [Day 011](#) — The mechanism for asking is a system call, and the boundary it crosses is the most important line in a computer.

T

table [Day 026](#) — A table holds one kind of thing.

tail call [Day 088](#) — A tail call is a recursive call whose result is returned directly, with nothing left to do afterwards:

tail-recursive [Day 090](#) — This version is also tail-recursive — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free.

tcp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

template [Day 092](#) — The saved wording with the date and the hours swapped in is a template — a fixed piece of text with holes in it, filled in per message.

test double [Day 053](#) — A test double is a stand-in you pass in during a test.

the return address [Day 088](#) — the arguments it was called with, - the local variables it creates, - the return address — the exact point in the caller to resume at, - and a link to the frame below it.

the running median [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — [[day 116](#)](../day-116-top-k/README.md) - merging k sorted lists — [[day 117](#)](../day-117-merge-k-sorted/README.md) - the running median — [[day 118](#)](../day-118-two-heaps/README.md), with two heaps -...

third table [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

thread [Day 008](#) — A thread is one line of execution inside a process, and all the threads in a process share that memory.

three distinct orderings [Day 092](#) — = 6 decision paths — but only three distinct orderings: [1,1,2], [1,2,1], [2,1,1].

three-way partition [Day 054](#) — The fix is a three-way partition: split into *less than*, *equal to*, and *greater than*, and recurse only into the outer two.

tight coupling [Day 061](#) — The balcony light on the reading lamp's switch is tight coupling — two things wired together that have no reason to be, so you cannot change one without changing the other.

time-to-live [Day 037](#) — Bhaskar's clear-out is time-to-live: attach an expiry to a key and the store deletes it, automatically.

tiny constraint [Day 097](#) — A tiny constraint: $n \leq 8$ means factorial, $n \leq 20$ means 2^n , target ≤ 500 with small candidates means combination sum.

toast [Day 134](#) — And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — TOAST — in a side table, compressed, and only reads them when the column is selected.

token [Day 019](#) — The server verifies them and issues a token — a session id or a signed token.

tombstone [Day 011](#) — There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a tombstone — and databases and hash tables use it constantly.

top-k problems [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

total order [Day 123](#) — It gives you a total order — sort by the number, break ties by machine name — that never contradicts causality.

transitions itself [Day 071](#) — In State, the object transitions itself: a Draft order moves itself to Submitted, and the states know about each other.

transitive dependency [Day 029](#) — This is a transitive dependency: id → department_id → department_name.

tree [Day 040](#) — A domain of independent records — a tree — tolerates anything.

ttl [Day 003](#) — The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a TTL — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again.

tuple [Day 005](#) — A tuple is written with round brackets — (3, 7, 2) — and is a list you cannot change.

two [Day 087](#) — In fib, each call makes two — so the work explodes.

two implementations [Day 079](#) — And, as always, two implementations: NearestSuitable as above, and ZonedDispatcher where each car owns a band of floors.

U

udp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

under-fetching [Day 021](#) — And from [day 015](../day-015-the-write-pointer/README.md)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — under-fetching, usually called the N + 1 problem when it happens in a list.

union filesystem [Day 013](#) — The stack of layers is a union filesystem — several directories presented as one merged tree.

upper bound [Day 044](#) — The upper bound is the first index where `nums[i] > 4`, which is index 4 — one past the run.

uri [Day 016](#) — In REST the thing is called a resource, and its address is a URI — the path in the URL.

url [Day 001](#) — A URL is an address for something on the internet.

V

value object [Day 051](#) — A value object is defined entirely by its values.

version [Day 117](#) — And the second half, which people forget: you must be able to tell which value is newest. The overlapping replica has the new value, but the others may return an old one, so every record carries a version — a counter, a timestamp, or a vector clock — and the reader takes the highest.

vertex [Day 125](#) — The places are the vertices. A vertex is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid.

vertical scaling [Day 098](#) — Vertical scaling means replacing the machine with a bigger one — more cores, more memory, faster disk.

victim [Day 035](#) — It watches who waits for whom, and when it finds a circle, it picks a victim — one transaction in the cycle — and kills it with an error.

virtual address space [Day 011](#) — Memory. Each process gets its own virtual address space — its own set of addresses that the OS maps onto real physical memory.

virtual machine [Day 013](#) — A virtual machine is an emulated computer.

volatile [Day 009](#) — It is also volatile: switch the power off and it is gone.

W

weight [Day 122](#) — To do that in code, every word needs a weight — a number saying how often it is chosen.

which class [Day 076](#) — Factory vs Builder — a factory decides which class; a builder assembles one complicated object step by step.

which is hours [Day 166](#) — The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, which is hours — and there is a separate fast path for trending terms, which is the third idea below.

within [Day 131](#) — Consumer group — Kafka's answer, and it is genuinely both: within a group, consumers compete like a queue; between groups, each gets everything like a topic.

worst case [Day 002](#) — The best case is 1, the worst case is n.

write skew [Day 034](#) — It appears in backend interviews at every product company, usually right after ACID, and the follow-up about write skew is where senior candidates are separated from the table-reciters.

write-around with invalidation [Day 102](#) — Wiping a line when the price moves is write-around with invalidation: update the truth, delete the cache entry, and let the next reader repopulate it.

write-through [Day 102](#) — Changing the board at four fifteen is write-through: update the cache when you update the truth.

Y

yes **Day 093** — | Problem | Recurse on | Duplicates in input | Extra rule | |---|---|---| | Combinations (LC 77) | $i + 1$ | no | stop at size k | | Combination Sum (LC 39) | i — reuse allowed | no | stop when remaining $= 0$ | | Combination Sum II (LC 40) | $i + 1$ — each used once | yes | sort, and skip when $i > \text{start}$ and $c[i] == c[i-1]$ | |...

References and further reading

Further reading, arranged by the part of the book it belongs to. Everything listed is a primary source, an official specification, or reference material that is free to read. None of this book is copied from any of them. They are here for the reader who wants the long version, the formal proof, or the original paper.

FOR THE WHOLE BOOK

Python Software Foundation. *The Python Language Reference and Standard Library.* <https://docs.python.org/3/> [PSF Licence, free to read]

The authority for every language behaviour this book relies on.

Python Wiki. *TimeComplexity - cost of the built-in container operations.* <https://wiki.python.org/moin/TimeComplexity> [Free to read]

The table behind almost every complexity claim about list, dict and set.

David Galles, University of San Francisco. *Data Structure Visualizations.* <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html> [Free to use]

Step-through animations for most structures in Parts II to XV.

Steven Halim and others. *VisuAlgo - visualising data structures and algorithms.* <https://visualgo.net/en> [Free for self-study]

Useful when a diagram in this book is not enough and you want to press play.

MIT OpenCourseWare. *6.006 Introduction to Algorithms.* <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Full lecture videos and notes. The formal treatment this book deliberately avoids.

LeetCode. *Problem set.* <https://leetcode.com/problemset/> [Free tier]

Where every problem named in the practice sheets is solved.

DATA STRUCTURES AND ALGORITHMS, BY PART

PART I · FOUNDATIONS: HOW CODE COSTS (DAYS 1–8)

Python Wiki. *TimeComplexity.* <https://wiki.python.org/moin/TimeComplexity> [Free to read]

Read this once now and again at the end of Part VIII.

Eric Rowell and contributors. *Know Thy Complexities - the Big-O cheat sheet.* <https://www.bigocheatsheet.com/> [Free to read]

One page. Pin it up while you work through Days 1 to 8.

Wikipedia. *Big O notation.* https://en.wikipedia.org/wiki/Big_O_notation [CC BY-SA]

The formal definition, for the day you want the mathematics.

Python Software Foundation. *timeit - measure execution time.* <https://docs.python.org/3/library/timeit.html> [PSF Licence]

The right way to check a claim about cost instead of guessing.

PART II · ARRAYS (DAYS 9–18)

CPython. *Objects/listobject.c - the list implementation.* <https://github.com/python/cpython/blob/main/Objects/listobject.c> [PSF Licence]

Read the comments at the top on over-allocation and growth.

Python Software Foundation. *Data Structures tutorial.* <https://docs.python.org/3/tutorial/datastructures.html> [PSF Licence]

The official tour of list, deque, dict, set and comprehensions.

Wikipedia. *Dynamic array.* https://en.wikipedia.org/wiki/Dynamic_array [CC BY-SA]

Where amortised O(1) append comes from.

PART III · STRINGS (DAYS 19–26)

Python Software Foundation. *Text Sequence Type - str.* <https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str> [PSF Licence]

Every string method used in Part III, with its exact behaviour.

Python Software Foundation. *Unicode HOWTO.* <https://docs.python.org/3/howto/unicode.html> [PSF Licence]

Why len() of a string is not always the number of characters you see.

Wikipedia. *Knuth-Morris-Pratt algorithm.* https://en.wikipedia.org/wiki/Knuth%E2%80%93Pratt_algorithm [CC BY-SA]

The linear-time pattern match, for when the interviewer pushes past naive search.

SYSTEM DESIGN, BY PHASE

HOW COMPUTERS AND THE INTERNET WORK (DAYS 1–14)

Mozilla. *MDN Web Docs - HTTP.* <https://developer.mozilla.org/en-US/docs/Web/HTTP> [CC BY-SA 2.5]

The most readable accurate reference on HTTP there is.

IETF. *RFC 9110 - HTTP Semantics.* <https://www.rfc-editor.org/rfc/rfc9110.html> [IETF Trust, free to read]

The specification itself. Skim the method and status-code sections.

IETF. *RFC 9293 - Transmission Control Protocol.* <https://www.rfc-editor.org/rfc/rfc9293.html> [IETF Trust, free to read]

The current consolidated TCP specification, including the handshake.

IETF. *RFC 1035 - Domain Names, Implementation and Specification.* <https://www.rfc-editor.org/rfc/rfc1035.html> [IETF Trust, free to read]

DNS record types and resolution, from the source.

Cloudflare. *Learning Centre.* <https://www.cloudflare.com/learning/> [Free to read]

Short, accurate explainers on DNS, TLS, CDNs and DDoS.

APIS: HOW SERVICES TALK (DAYS 15–24)

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures, chapter 5.* https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm [Free to read]

The dissertation chapter that defined REST. Shorter than its reputation.

OpenAPI Initiative. *OpenAPI Specification.* <https://spec.openapis.org/oas/latest.html> [Apache 2.0]

How an API contract is written down in practice.

gRPC Authors. *gRPC documentation.* <https://grpc.io/docs/> [Apache 2.0]

The alternative to REST that interviewers ask you to compare against.

GraphQL Foundation. *GraphQL specification and learning guide.* <https://graphql.org/learn/> [OWF / CC BY]

Enough to answer the over-fetching question honestly.

DATABASES FROM ZERO (DAYS 25-42)

PostgreSQL Global Development Group. *PostgreSQL documentation.* <https://www.postgresql.org/docs/current/> [PostgreSQL Licence]

The chapters on indexes, transactions and explain are the ones to read.

PostgreSQL Global Development Group. *Transaction Isolation.* <https://www.postgresql.org/docs/current/transaction-iso.html> [PostgreSQL Licence]

Read-committed against repeatable-read against serialisable, with examples.

Wikipedia. *Database normalization.* https://en.wikipedia.org/wiki/Database_normalization [CC BY-SA]

First through third normal form, which is as far as interviews go.

Wikipedia. *B-tree.* <https://en.wikipedia.org/wiki/B-tree> [CC BY-SA]

Why a database index is not a hash map.

MongoDB. *MongoDB Manual - data modelling.* <https://www.mongodb.com/docs/manual/data-modeling/> [Free to read]

The document-model side of the SQL against NoSQL question.

Colophon: how this book was made

This book was typeset from the course sources with free software only. Every typeface is published under the SIL Open Font Licence, every library under a permissive licence, and the page engine is the open-source Chromium print pipeline. Licence labels below describe the linked project, not this book.

TYPE, TOOLS AND LIBRARIES

SIL International, after Matthew Carter. *Charis SIL*. <https://software.sil.org/charis/> [SIL Open Font Licence 1.1]

The text face. An extension of Bitstream Charter, which Carter drew in 1987 to stay readable on coarse output. It is the standard free choice for technical books.

Bold Monday for IBM. *IBM Plex Sans and Plex Sans Condensed*. <https://github.com/IBM/plex> [SIL Open Font Licence 1.1]

Headings, labels, tables, folios and the covers.

JetBrains. *JetBrains Mono*. <https://github.com/JetBrains/JetBrainsMono> [SIL Open Font Licence 1.1]

All code, output and ASCII diagrams.

Google. *Noto Serif Devanagari*. <https://github.com/notofonts/devanagari> [SIL Open Font Licence 1.1]

The Devanagari on the covers and title page.

Knut Sveidqvist and contributors. *Mermaid*. <https://github.com/mermaid-js/mermaid> [MIT Licence]

Renders every flow chart, sequence diagram and tree.

Georg Brandl and contributors. *Pygments*. <https://pygments.org/> [BSD 2-Clause]

Colours the code. The palette used here was written for this book.

Waylan Limberg and contributors. *Python-Markdown*. <https://python-markdown.github.io/> [BSD 3-Clause]

Turns the lesson sources into the pages you are reading.

Martin Thoma and contributors. *pypdf*. <https://github.com/py-pdf/pypdf> [BSD 3-Clause]

Joins the covers, front matter and body, and writes the bookmarks.

The Chromium Authors. *Chromium print-to-PDF, driven by Puppeteer*. <https://pptr.dev/> [Apache 2.0 / BSD 3-Clause]

The typesetting engine. No proprietary software was used to make this book.

HOW A PAGE WAS MADE

The lesson sources are Markdown. Fenced blocks are lifted out before the Markdown is parsed, so diagrams keep every space they have and code reaches the colouring pass untouched. Each monospace block is measured, and its type size chosen so the widest line fits the column rather than falling off the page.

The result is rendered in a browser engine at 6.75 by 9.5 inches and printed to PDF. The body is printed first so that the contents and the index can be built from the page numbers the printing actually produced, then the covers and front matter are printed and the three are joined.

Problem index

Every problem named in a practice sheet, in the order the book reaches it, with its source and the chapter that sets it. Problem statements are not reproduced anywhere in this book. Solve them where they live.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
001	Concatenation of Array	LeetCode 1929 (Easy)	Can you write a single loop at all, and do you know how many times its body runs?
001	Running Sum of 1d Array	LeetCode 1480 (Easy)	One loop that carries a value forward. It is the shape of every prefix problem you will meet on day 037.
001	Richest Customer Wealth	LeetCode 1672 (Easy)	Your first genuinely nested loop. The hot line sits inside both loops. Say the count out loud before you run it.
001	Contains Duplicate	LeetCode 217 (Easy)	The exact function from today's lesson. Write the nested version deliberately, submit it, and watch what happens on the big test case.
002	Final Value of Variable After Performing Operations	LeetCode 2011 (Easy)	The simplest possible count. One loop, no nesting, body runs exactly n times. Say " n " out loud before you start.
002	Number of Steps to Reduce a Number to Zero	LeetCode 1342 (Easy)	The halving loop. For $n = 14$ the answer is 6, not 14. Count the steps for 1,000 and see that it is 10-ish, not 1,000.
002	Plus One	LeetCode 66 (Easy)	The count depends on the input, not just its size. [1,2,3] costs one step; [9,9,9] costs three. Best case and worst case in one small function.
002	Number of Good Pairs	LeetCode 1512 (Easy)	The staircase. Write the nested version first and check the count against $n \times (n - 1) / 2$ before you improve it.
003	Contains Duplicate	LeetCode 217 (Easy)	The exact trap from §7. The nested-loop version is $O(n^2)$, the set version is $O(n)$, and the code looks almost identical.
003	Two Sum	LeetCode 1 (Easy)	Trading space for time. $O(n^2)$ with two loops, $O(n)$ with a dictionary and $O(n)$ extra space. Say the trade out loud.
003	Majority Element	LeetCode 169 (Easy)	Three different shapes for one problem — counting each element is $O(n^2)$, sorting is $O(n \log n)$, and Boyer-Moore is $O(n)$ with $O(1)$ space.
003	Maximum Subarray	LeetCode 53 (Medium)	The brute force is $O(n^3)$ if you are not careful, $O(n^2)$ if you are, and $O(n)$ with Kadane. Notice how easy the cubic version is to write by accident.
004	Running Sum of 1d Array	LeetCode 1480 (Easy)	$n \leq 1000$, so almost anything passes. Notice that the constraint gives you no pressure at all, and that this is rare.
004	Subsets	LeetCode 78 (Medium)	$n \leq 10$. That number is the answer: 2^{10} is 1,024, so generating every subset is intended. Read the constraint before the statement.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
004	Two Sum II — Input Array Is Sorted	LeetCode 167 (Medium)	$n \leq 30,000$ and the array is sorted. Quadratic is 9×10^8 and dies; the sortedness is the hint that two pointers are the target.
004	Kth Largest Element in an Array	LeetCode 215 (Medium)	$n \leq 10^5$. Sorting is $O(n \log n)$ and comfortably fits, so it is a valid answer — but a heap gives $O(n \log k)$. Say what each one costs.
005	Remove Element	LeetCode 27 (Easy)	The tempting solution is <code>remove()</code> or <code>pop(i)</code> in a loop, which is $O(n^2)$. The write-pointer version is $O(n)$ and touches nothing but the end.
005	Move Zeroes	LeetCode 283 (Easy)	Same trap, sharper. <code>pop(i)</code> then <code>append(0)</code> is quadratic; two indices walking forward is linear. This is the shape of [day 015](../day-015-the-write-pointer/README.md).
005	Implement Queue using Stacks	LeetCode 232 (Easy)	Forces you to think about which end of a list is cheap. <code>pop()</code> is free, <code>pop(0)</code> is not, and the whole problem exists because of that asymmetry.
005	Rotate Array	LeetCode 189 (Medium)	The one-line slice solution works and allocates a full copy. The reversal trick does it in $O(1)$ extra space. Write both and say what each costs in memory.
006	Contains Duplicate	LeetCode 217 (Easy)	The reflex, in its purest form. If this does not produce a set within five seconds, drill it until it does.
006	Valid Anagram	LeetCode 242 (Easy)	Set versus Counter. <code>set(a) == set(b)</code> passes some tests and is wrong — find the input that breaks it before you look it up.
006	Two Sum	LeetCode 1 (Easy)	A dictionary storing value $\rightarrow$ index. The insight is that you look for what you <i>*need*</i> , not what you <i>*have*</i> .
006	Group Anagrams	LeetCode 49 (Medium)	<code>defaultdict(list)</code> plus a canonical key. The whole problem is choosing what the key should be.
007	Reverse String	LeetCode 344 (Easy)	The problem statement explicitly demands $O(1)$ extra space, which is why the input is a character list rather than a string. Two pointers and a swap.
007	Remove Duplicates from Sorted Array	LeetCode 26 (Easy)	The write pointer, in its purest form. The signature returns a length rather than a list, and that is the whole hint.
007	Move Zeroes	LeetCode 283 (Easy)	Same pattern, one step harder. Building a new list is trivial; doing it in place with one pass and one extra index is the point.
007	Majority Element	LeetCode 169 (Easy)	Three solutions with three different space costs: a Counter is $O(n)$, sorting is $O(1)$ extra with an in-place sort, and Boyer-Moore is $O(1)$ with one candidate and one count.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
008	Search Insert Position	LeetCode 35 (Easy)	The output is not what people assume. "Where it *would* go" is a different question from "where it is", and the target being absent is the main case, not the edge case.
008	Best Time to Buy and Sell Stock	LeetCode 121 (Easy)	The two questions that decide correctness are unstated: must the sell come after the buy, and what do you return when every trade loses money?
008	Valid Palindrome	LeetCode 125 (Easy)	Almost pure specification. What counts as a character, is case significant, is an empty string valid? Nearly every failed submission here is a reading failure, not a coding one.
008	Find First and Last Position of Element in Sorted Array	LeetCode 34 (Medium)	Duplicates are the whole problem, the array being sorted is load-bearing, and "not present" must return [-1, -1] rather than anything you would naturally invent.
009	Build Array from Permutation	LeetCode 1920 (Easy)	The purest possible use of $O(1)$ indexing: <code>nums[nums[i]]</code> is two direct reaches, not a search. If indexing were $O(n)$ this problem would not exist.
009	Concatenation of Array	LeetCode 1929 (Easy)	Position arithmetic. <code>ans[i]</code> and <code>ans[i + n]</code> are both computed, both direct, both the same cost.
009	Richest Customer Wealth	LeetCode 1672 (Easy)	A 2D list is a list of references to lists. Row-by-row is the natural order and also the cache-friendly one — notice that they agree.
009	Find Pivot Index	LeetCode 724 (Easy)	Two passes over the same array, both sequential. The naive version recomputes a sum inside the loop and is $O(n^2)$; spotting that is the exercise.
010	Monotonic Array	LeetCode 896 (Easy)	The adjacent-pairs bound, exactly. <code>range(n - 1)</code> because the body reaches <code>i + 1</code> , and it must return <code>True</code> for a one-element array without a guard.
010	Maximum Average Subarray I	LeetCode 643 (Easy)	The fixed window. The slicing version is $O(n \times k)$ and passes; the running-total version is $O(n)$. Write the slow one first, then fix it, and say what changed.
010	Reverse String	LeetCode 344 (Easy)	Two pointers from the ends. <code>while left < right</code> , and the loop naturally handles both odd and even lengths without a special case.
010	Remove Element	LeetCode 27 (Easy)	The trap from §7 — deleting while iterating forwards. Write the <code>remove()-in-a-loop</code> version, watch it skip elements, then write the write-pointer version.
011	Remove Element	LeetCode 27 (Easy)	Order does not matter here, which the statement says explicitly. So both the write-pointer solution and the swap-with-last solution are valid — write both and compare.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
011	Remove Duplicates from Sorted Array	LeetCode 26 (Easy)	Order does matter, so swap-with-last is out. Pure write pointer, and the return value is a length rather than a list.
011	Remove Duplicates from Sorted Array II	LeetCode 80 (Medium)	The same pattern with a count. The whole difficulty is deciding what the write pointer compares against — and the answer is <code>items[write - 2]</code> , not the previous element.
011	Merge Sorted Array	LeetCode 88 (Easy)	The direction trap from §7. Merging forwards overwrites data you still need; merging from the end backwards does not. This is the same rule as shifting right.
012	Search Insert Position	LeetCode 35 (Easy)	The "not found" answer is not -1 here — it is where the value <i>would</i> go, which is a different contract entirely. Read the statement twice.
012	Find Numbers with Even Number of Digits	LeetCode 1295 (Easy)	Search by a condition, not by equality. This is the case binary search cannot handle, and the reason linear search survives.
012	Find All Numbers Disappeared in an Array	LeetCode 448 (Easy)	"Find all", not "find first" — so no early exit is possible. The naive version searches for each candidate and is $O(n^2)$; a set makes it $O(n)$.
012	First Bad Version	LeetCode 278 (Easy)	Deliberately the counter-example. The linear scan is correct and too slow, and the problem exists to make you notice the property that allows something better. Solve it linearly first, then see why.
013	Reverse String	LeetCode 344 (Easy)	The two-pointer loop, and the bound. <code>while left < right, n // 2</code> swaps. Write it once and never get it wrong again.
013	Rotate String	LeetCode 796 (Easy)	Whether you understand what a rotation <i>*is*</i> . There is a one-line answer using <code>s + s</code> , and finding it is the point.
013	Rotate Array	LeetCode 189 (Medium)	Today's question, exactly as an interviewer asks it. Do it three ways — extra array, one-at-a-time, three reversals — and time all three.
013	Reverse Words in a String	LeetCode 151 (Medium)	The three-reversal idea again: reverse everything, then reverse each word. Also a lesson in messy input — leading, trailing and repeated spaces.
014	Maximum Product of Two Elements in an Array	LeetCode 1464 (Easy)	The two largest, in one pass. The sorting answer is one line; write the two-tracker version and see that it is barely longer.
014	Third Maximum Number	LeetCode 414 (Easy)	Three trackers, and the distinct-value contract — the statement says third <i>*distinct*</i> maximum, and if there isn't one you return the maximum instead. Read it twice.
014	Maximum Product of Three Numbers	LeetCode 628 (Medium)	Five trackers: the three largest and the two smallest. Two large negatives multiply to a large positive, which is the whole trap.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
014	Kth Largest Element in an Array	LeetCode 215 (Medium)	Where trackers stop working. Solve it three ways — sort, min-heap of size k, quickselect — and be able to say when each is the right choice.
015	Remove Element	LeetCode 27 (Easy)	The bare pattern, and whether you understand that nothing is deleted — you return a count and the grader ignores everything past it.
015	Move Zeroes	LeetCode 283 (Easy)	The same loop plus the second phase. The trap is stopping after the first loop and leaving the stale tail behind.
015	Remove Duplicates from Sorted Array	LeetCode 26 (Easy)	Comparing against items[write - 1] — what you *kept* — rather than items[read - 1]. Also the empty-input guard, since write starts at 1.
015	Remove Duplicates from Sorted Array II	LeetCode 80 (Medium)	items[write - 2]. If you built the habit in problem 3 this is a one-character change; if you compared against read - 1 it is a rewrite.
016	Richest Customer Wealth	LeetCode 1672 (Easy)	The bare row walk, and whether you reach for sum(row) instead of indexing by hand. Two minutes.
016	Transpose Matrix	LeetCode 867 (Easy)	That the output is $n \times m$ when the input is $m \times n$. Write it with explicit loops first, then with zip(*matrix), and be able to explain the second.
016	Diagonal Traverse	LeetCode 498 (Medium)	$r + c$ is constant on an anti-diagonal. The zigzag is a separate, easier concern — solve them in that order or you will tangle them.
016	Matrix Diagonal Sum	LeetCode 1572 (Easy)	Both diagonals at once, and the odd-size trap: on an odd $n \times n$ the centre cell is on both diagonals and must not be counted twice.
017	Transpose Matrix	LeetCode 867 (Easy)	That the answer is $n \times m$ for an $m \times n$ input, so it cannot be done in place unless the matrix is square. Write both versions.
017	Rotate Image	LeetCode 48 (Medium)	Transpose then reverse each row, and the range($r + 1$, n) that keeps each pair swapped once. The whole question is the words *in place*.
017	Spiral Matrix	LeetCode 54 (Medium)	Four boundaries and the two guards. It is rectangular on purpose.
017	Spiral Matrix II	LeetCode 59 (Medium)	The same four passes, writing instead of reading. If you built problem 3 cleanly this is a fifteen-line rewrite.
018	Best Time to Buy and Sell Stock	LeetCode 121 (Easy)	The single-pass habit from [day 014](../day-014-single-pass-habit/README.md), and whether you ask what happens when prices only fall.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
018	Merge Sorted Array	LeetCode 88 (Easy)	The write pointer from [day 015](../day-015-the-write-pointer/README.md), reversed. Filling from the back is the entire insight.
018	Find All Numbers Disappeared in an Array	LeetCode 448 (Easy)	Whether you can find the $O(1)$ -space trick — using the array's own values as markers — after giving the $O(n)$ -space answer first.
018	Set Matrix Zeroes	LeetCode 73 (Medium)	[Day 016](../day-016-2d-arrays/README.md) plus a genuine trap: marking as you go destroys the information you still need. The $O(1)$ version uses the first row and column as the marker store.
019	Reverse String	LeetCode 344 (Easy)	That the input is given as a list of characters, not a string — precisely because a string cannot be reversed in place. Ask yourself why the problem is phrased that way.
019	Reverse Words in a String III	LeetCode 557 (Easy)	split, transform, join. Anyone who builds the result with <code>+=</code> inside two nested loops has not learned today's lesson.
019	Longest Common Prefix	LeetCode 14 (Easy)	Building the answer once rather than growing it character by character, and the two edge cases: an empty list, and one string being a prefix of another.
019	Reverse Words in a String	LeetCode 151 (Medium)	split() with no argument versus split(" "). The whole difficulty is multiple spaces, leading spaces and trailing spaces.
020	Reverse Words in a String III	LeetCode 557 (Easy)	split, transform each word, join. Two minutes if you use the pattern, and a nested-loop <code>+=</code> disaster if you do not.
020	Add Strings	LeetCode 415 (Easy)	Digit-by-digit addition with a carry, built backwards into a list and reversed before joining. The carry after the loop is the same "flush the last thing" reflex as the run-length encoder.
020	String Compression	LeetCode 443 (Medium)	The signature is <code>list[str]</code> , so this is the write pointer in $O(1)$ space, not a join problem. Solve it wrong on purpose first, then right.
020	Encode and Decode Strings	LeetCode 271 (Medium)	Designing your own format. Length-prefixing beats delimiters, and working out why is the whole exercise.
021	Ransom Note	LeetCode 383 (Easy)	Counter subtraction, and whether you notice that "can I build A from B" is a counting question and not a searching one.
021	First Unique Character in a String	LeetCode 387 (Easy)	Two passes, and being able to say why one pass is impossible.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
021	Sort Characters By Frequency	LeetCode 451 (Medium)	most_common, then building the output with join — not + = . Two lessons meeting.
021	Find All Anagrams in a String	LeetCode 438 (Medium)	A count that moves: add the entering character, remove the leaving one. This is the sliding window of [day 033] (../day-033-window-with-a-map/README.md) arriving early.
022	Valid Anagram	LeetCode 242 (Easy)	Two solutions with different complexities, offered unprompted. The one-liner alone is an incomplete answer.
022	Group Anagrams	LeetCode 49 (Medium)	The canonical form as a dictionary key, and knowing that a list cannot be one.
022	Find All Anagrams in a String	LeetCode 438 (Medium)	A count that slides. Recomputing per window is $O(n \cdot k)$; updating two entries per step is $O(n)$.
022	Group Shifted Strings	LeetCode 249 (Medium)	Inventing a canonical form yourself. abc and bcd are the same "shape" — what key captures that? This is the real skill.
023	Valid Palindrome	LeetCode 125 (Easy)	The messy input rules, and whether you can do it in $O(1)$ space rather than reversing. "0P" is the trap input.
023	Valid Palindrome II	LeetCode 680 (Easy)	That only two deletions are possible at the first mismatch, and that you must try both rather than guess.
023	Longest Palindromic Substring	LeetCode 5 (Medium)	$2n - 1$ centres, not n . Forgetting the even-length centres is the standard bug, and "cbbd" finds it.
023	Palindrome Linked List	LeetCode 234 (Easy)	The same idea where you cannot index backwards. Reverse the second half in place, compare, restore. Two pointers again, in a shape you will meet properly on [day 082] (../day-082-runner-technique/README.md).
024	Is Subsequence	LeetCode 392 (Easy)	The greedy walk, and whether you can say *why* taking the earliest match is never worse.
024	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	A window, and the last[ch] >= start guard. "dvdf" and "abba" are the inputs that expose it.
024	Longest Common Subsequence	LeetCode 1143 (Medium)	The 2-D table, and why greedy fails here when it worked in problem 1.
024	Maximum Length of Repeated Subarray	LeetCode 718 (Medium)	The *substring* version of problem 3. One line of the recurrence differs, and the answer is in a different cell. Do these two back to back.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
025	Find the Index of the First Occurrence in a String	LeetCode 28 (Easy)	range(n - m + 1), resetting k, and whether you can state both costs with an example input for each.
025	Repeated Substring Pattern	LeetCode 459 (Easy)	Whether you spot the (s + s)[1:-1].find(s) != -1 trick — and can explain *why* it works, which is the actual question.
025	Longest Common Prefix	LeetCode 14 (Easy)	Comparing many strings position by position, and stopping at the first disagreement. The empty-list case.
025	Implement strStr with KMP	LeetCode 28 again	Only after the naive version passes. Build the failure table by hand on "aabaaac" before writing any code.
026	Isomorphic Strings	LeetCode 205 (Easy)	Two indices. Whether you find the second map — "badc"/"baba" passes every example in the statement with one dictionary.
026	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	Sliding window. The last[ch] >= start guard, which "abba" and "dvdf" expose.
026	Group Anagrams	LeetCode 49 (Medium)	Canonical form as a key, and that a list is not hashable.
026	Longest Palindrome	LeetCode 409 (Easy)	Counting, with a genuine think at the end: how many odd-count characters can you keep, and why exactly one?

H

Index

Page numbers in **bold** are where the term is defined. Terms are those the book defines; the glossary in Appendix D gives each one a sentence.

#

4 kb, 142, 143, 175, 179, **277**, 278, 280, 281, 284, ...

A

a replicated log, 949
abstract class, 949
acid, 869, 870, 949, 950, 966
adaptee, 949
adapter, 949
add a tie-breaker, 949
algorithm, 112, 190, 375, 487, 554, 605, 650, 687, 688, ...
always comparable, 949, 959
amortised, **124**, 126, 269, 633, 666, 669, 949, 969
anaemic model, 949
and a direction, 949
anti-diagonals, **526**, 949
anti-sniping, 949
approximate, 801, 949
array, 25, 27, 37, 38, 53, 67, 68, 97, **484**, ...
artefact, **950**, 952
assignment rule, 950
at-least-once, 617, 622, 950
atomic, **244**, 247, 251, 252, 253, 254, 614, 617, 619, ...
atomicity, **949**, 950
availability, 950, 952, 957, 964
aws ecs, **384**, 389, 431, 950, 960

B

backend, 21, 23, **43**, 49, 50, 51, 144, 247, 322, ...
bearer token, **647**, 950
best case, **31**, 40, 52, 57, 366, 374, 378, 380, 454, ...
binary tree, 950
blast radius, 393, 474, 950
block, 19, 29, 78, 95, 96, 104, 105, 108, **263**, ...
blocked, **207**, 348, 349, 353, 357, 359, 386, 470, 950
both, 5, 7, 9, 12, 13, 15, 16, 18, 20, ...
boundary, 348, 799, 809, 950, 954, 958, 962, 964
bounded context, 950
bounded optimism, 950
breadth-first, 124, 131, 133, 950
break point, 950

broken links, 950, 960
bucket by time, 950
bucket counts, 951
bulkhead, 951
burst size, **793**, 800, 951, 961

C

cache, 18, 19, 20, 21, 22, 23, 24, **70**, 72, ...
cache it, 284, 580, 951
cache stampede, 951
candidates, 508, 544, 581, 618, 655, 691, 728, 764, 801, ...
capacity, **121**, 269, 669, 794, 796, 797, 800, 951, 961
cdn, 17, 18, 20, 21, 22, 23, 24, 48, 75, ...
certificate, 17, **171**, 174, 176, 179, 465, 951
chaining, 951
channels, 951
checks permission, **951**
chunks, 758, 951
circular buffer, 951, 962
class, 21, 55, 161, 162, 237, 270, 376, 392, 393, ...
client, 27, 41, 44, 45, 46, 105, 140, 174, 176, ...
collapse key, 951
collapsing writes, 951
collection, 124, 126, 131, 133, 134, 156, 160, 165, **524**, ...
column, 7, 8, 9, 31, 36, 57, 63, 191, **887**, ...
compact, 195, **484**, 951, 953
comparator, 952
comparison, 6, 7, 59, 157, 369, 370, 372, 373, 374, ...
complete, 282, 491, 526, 562, 600, 637, 673, 710, 746, ...
component, 952, 955, 956, 959, 963
composite, 908, 910, 952
composite key, 952
composition, 952, 954
composition root, 952
connection, 45, 48, 73, 105, 109, 110, 140, 141, 174, ...
connection-oriented, **952**
consequence, 952, 960, 962
consistency, 949, 950, 952, 954, 957, 963
constraints, 223, 228, 229, 517, 538, 551, 871, 907, 952, ...

constructor, 949, 952
container, 353, 401, 424, 425, 428, 429, 430, 431, 432, ...
container image, 473, 950, 952
containerd, 428, 430, 952, 953
context, 110, 354, 950, 952
coordinator, 952, 960
correlation id, 952
cost, 1, 5, 9, 12, 16, 20, 21, 22, 27, ...
count map, 202, 221, 952, 953
counter, 7, 8, 12, 25, 41, 42, 43, 44, 47, ...
cri-o, 427, 430, 952, 953
critical path, 78, 81, 953
cycle, 252, 269, 278, 280, 284, 285, 287, 288, 406, ...

D

dangerous, 591, 612, 615, 953
dashed, 953
dau, 86, 294, 296, 462, 666, 814, 852, 953
deep-copied, 953
deleted, 137, 146, 330, 332, 342, 343, 487, 513, 537, ...
delta of delta, 953
dependency, 388, 953, 960, 965
depth, 14, 98, 197, 201, 202, 203, 222, 260, 466, ...
depth-first, 953
deque, 124, 126, 127, 128, 130, 131, 133, 134, 156, ...
diminishing returns, 953
direct addressing, 953
domain name, 15, 70, 71, 172, 175, 180, 182, 183, 940, ...
draw two diagrams, 953
duck typing, 953
dummy node, 953

E

earlier, 20, 174, 225, 232, 239, 388, 597, 602, 609, ...
edge cases, 223, 224, 226, 228, 232, 233, 235, 239, 255, ...
edge compute, 954
effect, 77, 80, 179, 206, 277, 285, 315, 430, 432, ...
elastic scaling, 954
election timeout, 954
empty, 28, 42, 108, 121, 131, 135, 154, 155, 156, ...
etl, 71, 88, 121, 131, 171, 172, 182, 416, 431, ...
event object, 954
eventual consistency, 954
exactly, 8, 9, 10, 11, 14, 15, 25, 26, 27, ...
exactly correct, 954
exactly one, 10, 14, 39, 122, 140, 158, 162, 190, 228, ...
exclusive lock, 954
expiry, 171, 176, 179, 546, 614, 617, 651, 684, 693, ...

F

facts, 42, 50, 51, 134, 135, 138, 145, 147, 153, ...
false negative, 954
false positive, 954
fanout, 873, 954
fee, 48, 52, 56, 58, 72, 77, 97, 287, 317, ...
fifo, 954
file, 11, 16, 17, 18, 20, 22, 23, 24, 38, ...
filled diamond, 954
finite state machine, 955
float, 8, 38, 90, 91, 92, 123, 126, 160, 193, ...
four-direction move, 955
fragile, 955
frontend, 955

G

galloping mode, 955
gap, 36, 65, 77, 89, 103, 106, 107, 108, 113, ...
generator expression, 670, 955
greedy simulation, 955
guest operating system, 955, 956

H

hash map, 88, 98, 101, 115, 191, 199, 200, 226, 232, ...
hash ring, 955
hateoas, 538, 545, 547, 548, 552, 833, 928, 955
header interface, 955
heap, 3, 4, 5, 6, 10, 20, 31, 48, 58, ...
heapsort, 191, 198, 202, 203, 937, 955
heartbeat, 955
hidden spof, 955
hiding mechanism, 955
highest node, 955
historical snapshot, 955
hollow triangle, 956
hook, 276, 630, 888, 956
horizontal scaling, 956
hypervisor, 428, 955, 956

I

image, 16, 17, 22, 23, 143, 330, 353, 358, 424, ...
implicit graph, 956
in parallel, 759, 956
in series, 956
in-degree, 956, 960
index, 11, 66, 68, 69, 121, 122, 127, 129, 854, ...
indexed heap, 956
inheritance, 956
inorder, 956, 962
input, 9, 10, 11, 12, 13, 25, 26, 27, 223, ...
integrity, 870, 956

intent plus multiplicity, 956
interface, 953, 955, 956, 962
invariant, 956
is possible, 372, 956, 963, 976
item, 5, 6, 7, 8, 9, 10, 11, 13, **523**, ...
iterator, 339, 342, 559, 566, 956

J

json, 21, 137, 138, 139, 140, 141, 142, 143, **501**, ...
jump, 26, 266, 269, 272, 274, 289, 347, 357, 359, ...
jwt, 142, 144, 546, 645, 651, **664**, 682, 683, 684, ...

K

k closest points, 957
k most frequent, 957
k-th largest specifically, 957
key, 2, 14, 17, 45, 66, 68, 87, 108, **794**, ...
key property, 957

L

lag, 13, 81, 107, 142, 147, 203, 287, 308, 380, ...
largest, 83, 86, 88, 92, 94, 95, 98, 115, 116, ...
latency, 213, 214, 216, 217, 219, 292, 310, 313, 314, ...
lazy, 75, 480, 566, 858, 859, 882, 957
lazy deletion, 957
lazy loading, 957
lazy refill, 957
leader, 284, 954, 957, 964
leaf, 175, 179, 183, 940, 957, 963
lease, 211, 212, 218, 244, 246, 248, 249, 252, 253, ...
ledger entries, 957
left, 3, 30, 31, 32, 35, 45, 49, 57, 58, ...
lfu, 957
linear scan, 957, 976
linearizability, 957
list, 1, 3, 4, 5, 6, 7, 8, 10, 11, ...
list of lists, **519**, 532, 958
load balancer, 17, 312, 387, 465, 539, 796, 958
load factor, 958
load shedding, 958
log-structured merge tree, 958
logical cohesion, 958
logical time, 958
loop, 4, 5, 6, 7, 8, 9, 10, 11, 12, ...
loop body, 27, 29, 52, 298, 306, 367, 380, 453, 958
low cohesion, 958
lru, 339, 342, 949, 958

M

manifest, 958
many times, 8, 25, 27, 28, 38, 40, 52, 53, 166, ...

many-to-many, 906, 908, 909, 915, 916, 918, 919, 923, 935, ...
matrix, 258, **517**, 518, 519, 520, 521, 522, 523, 524, ...
mau, 958
max-heap, 958
method resolution order, 958
milliseconds, 8, **351**, 952, 958
min-heap, 459, 460, 479, 480, 958, 977
minimum spanning forest, 959
mixin, 959
monotone, 959
monotonic deque, 959

N

name mangling, 959
namespace, 353, 356, 358, 389, **425**, 426, 428, 429, 430, ...
negative, 228, 233, 234, 373, 374, 452, 453, 480, 530, ...
negative infinity, 959
negatives, 228, 233, 234, 452, 453, 480, 954, 959, 976
nested loop, 8, 161, 162, 959, 973, 978
network partition, 959
never equal, 949, 959
never forgets, 959
never persisted, 959
no children, 957, 959
node, 45, 46, 206, 207, 211, 213, 218, 247, 272, ...
not a map, 959

O

o(1), 57, 58, 59, 62, 63, 64, 65, 67, 68, ...
object, 38, 96, 130, 131, 145, 156, 163, 164, 166, ...
observers, 959
occupied, 953, 954, 959
offset, 543, 839, 959
on first touch, 959
one-shot, 960
opens, 15, 20, 41, 70, 71, 81, 136, 176, 204, ...
orchestrator, 950, 960
origin, 123, 125, 127, 128, 316, 318, 506, 543, 556, ...
origin pull, 960
orphan rows, 906, 915, 960
orphans, 950, 960
out-degree, 956, 960
overloading, 960
overriding, 960

P

page, 5, 10, 14, 15, 16, 17, 19, 20, **278**, ...
parallel edges, 960
parameters, 141, 175, **960**
parent, 577, 580, 909, 910, 917, 918, 919, 956, 958, ...

partial failure, 960
participants, 952, 960, 962
partition key, 950, 960
partitions, 960
path, 13, 23, 73, 78, 81, **135**, 137, 138, 140, ...
port, 5, 6, 7, 8, 15, 42, 45, 46, **54**, ...
prefix sums, 959, 961
preorder, 961
priority queue, 961
procedure, 831, 961
process, 130, 131, **223**, 241, 244, 245, 247, 248, 249, ...
program, 44, 45, 73, 327, 347, 351, 354, 952, 960, ...
protecting state, 961

Q

qps, 961
queue, 46, 47, 102, 111, 112, 113, 124, 130, 131, ...
quorum, 961

R

race condition, 243, 961
ram, 3, 4, 5, 6, 7, 9, 15, 16, **259**, ...
rank, 85, 767, 772, 773, 961
rebalancing, 961
rebuilding the tree, 961
recursion, 98, 196, 197, 199, 202, 961
references, 164, **265**, 267, 268, 269, 454, 838, 910, 918, ...
refill rate, **793**, 802, 805, 806, 809, 810, 930, 951, 961
register, 181, 247, **259**, 269, 277, 278, 280, 285, 287, ...
relative epsilon, 961
removal, 961
replica, 318, 391, 432, 473, 949, 951, 961, 962, 966
replica divergence, 962
request-aware routing, 962
resolver, 17, 18, 19, 20, 23, 73, 74, 75, **722**, ...
resources, 426, 536, 540, 542, 544, 545, 546, 547, **573**, ...
returned, 8, 133, 355, 374, 376, 455, 456, 530, 612, ...
ring buffer, 951, 962
role interface, 962
root, 17, 18, 19, 23, 73, 74, 77, 78, 79, ...
rotation, 407, 439, 556, 559, 563, 962, 976
routine, 90, 211, 212, 247, 249, 250, 252, 254, 390, ...
row, 1, 2, 4, 5, 10, 12, 13, 15, **887**, ...
run, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...

S

same remainder, 962
schema, 212, 394, 508, **721**, 722, 724, 755, 756, 757, ...
scoping, 962
script, 17, 19, 23, 45, 237, 304, 347, 348, 349, ...

seam, 852, 962
search space, 962
selectivity, 962
sentinel, 953, 962
separator, 147, **633**, 636, 641, 667, 677, 681, 698, 943, ...
sequence diagram, 962, 971
serialisable, 962, 970
service credits, 962
session, 137, 139, 283, 312, 539, 651, 653, 654, **664**, ...
session guarantees, 963
set, 1, 11, 16, 17, 18, 20, 21, 22, **152**, ...
shallow-copied, 963
shape, 4, 5, 6, 7, 9, 10, 12, 13, **55**, ...
shard, 321, 579, 580, 589, 751, 766, 773, 839, 840, ...
shared lock, 963
shared store, 538, 546, 617, 654, 683, 686, 691, 692, 803, ...
singleton, 963
singly linked list, 420, 963
slice, 96, 117, 118, 120, 122, 123, 124, 125, **631**, ...
smallest, 444, 445, 449, 451, 452, 453, 952, 956, 957, ...
snapshot, 955, 963
snipe, 963
sorted region, 963
sorting everything, 963
splits, 111, 641, 963
spof, 955, 963
squares, 57, 63, 342, 963
stable, 202, 334, 394, 429, 487, 541, 573, 578, 580, ...
stack, 98, 118, 120, 121, 149, 193, 198, 199, 202, ...
stack frame, 963
stack overflow, 963
state, 4, 5, 16, 46, 48, 49, 50, 51, 58, ...
statements, **961**, 964, 972
status, 19, 135, 137, 139, 142, 145, 146, 147, 148, ...
still one machine, 317, 957, 964
stonith, 964
strategy, 251, 949, 952, 964
strictly shorter, 964
structural, 961, 964
subarray, **825**, 964, 973, 975, 979
subject, 964
subscription list, 964
substring, 628, 782, 812, 813, 815, 817, 818, 819, 821, ...
subtree, 956, 961, 964
surge multiplier especially, 964
swap, 136, 188, 189, 190, 192, 194, 196, 200, 201, ...
system call, 210, 346, 351, 354, 964

T

table, 15, 16, 28, 29, 30, 31, 32, 34, **887**, ...
tail call, 964
tail-recursive, 964
tcp, 16, 17, 18, 20, 22, 23, 24, 46, **85**, ...
template, 283, 956, 964
test double, 964
the return address, 964
the running median, 955, 964, 965
third table, **906**, 919, 935, 958, 965
thread, 46, 204, 206, 210, 211, 212, 214, 216, **223**, ...
three distinct orderings, 965
three-way partition, 965
tight coupling, 965
time-to-live, 965
tiny constraint, 965
toast, 965
token, 46, 50, 141, 142, 144, 153, 154, 156, **645**, ...
tombstone, 965
top-k problems, 955, 964, 965
total order, 751, 753, 965
transitions itself, 965
transitive dependency, 965
tree, 16, 19, 70, 74, 81, 205, 281, 331, 425, ...
ttl, 4, 18, 20, 22, 47, **58**, 70, 72, 73, ...
tuple, 1, 8, **117**, 118, 120, 121, 133, 149, 154, ...
two, 3, 4, 5, 7, 9, 10, 11, 12, 13, ...
two implementations, 965

U

udp, 77, 80, **85**, 102, 104, 105, 107, 108, 109, ...
under-fetching, 965
union filesystem, 966
upper bound, 818, 853, 961, 966
uri, 99, 135, 175, 177, 353, 355, 356, 388, **536**, ...
url, **15**, 45, 48, 50, 137, 138, 141, 144, 150, ...

V

value object, 966
version, 138, 139, 161, 162, 174, 177, 197, 336, 388, ...
vertex, 953, 959, 960, 966
vertical scaling, 966
victim, 966
virtual address space, **347**, 359, 966
virtual machine, 345, 353, 356, 384, 392, 393, **401**, 424, 425, ...
volatile, **279**, 280, 966

W

weight, 356, 358, 382, 383, 424, 435, 538, 794, 803, ...
which class, 966
which is hours, 966
within, 77, 99, 103, 112, 181, 184, 241, 268, 274, ...
worst case, 31, 40, 52, 57, 58, 68, 77, 155, 157, ...
write skew, 966
write-around with invalidation, 966
write-through, 966

Y

yes, 15, 25, 28, 30, 41, 55, 60, 80, 86, ...

Two subjects, every day, for one hundred and eighty days.

Most preparation teaches algorithms for three months, then starts again from nothing on system design. Interviews do not work that way. A single loop will ask you to reverse a linked list in one room and design a rate limiter in the next.

So this book teaches both, every day, for one hundred and eighty days. Written for someone starting from zero - not a graduate, not somebody brushing up - and finishing able to answer either kind of question out loud, to a stranger, without notes.

- ◆ One hundred and eighty chapters. Two lessons and a practice sheet in every one.
- ◆ The same nine sections every time, so you learn one reading rhythm and keep it.
- ◆ Every lesson opens with a story about a person, told without a single technical word.
- ◆ Every solution shown in full, built in fragments and then given complete.
- ◆ The arithmetic done out loud, and error messages printed exactly as they appear.
- ◆ Real interviewer phrasings, an opening script, the follow-ups, and a model answer.